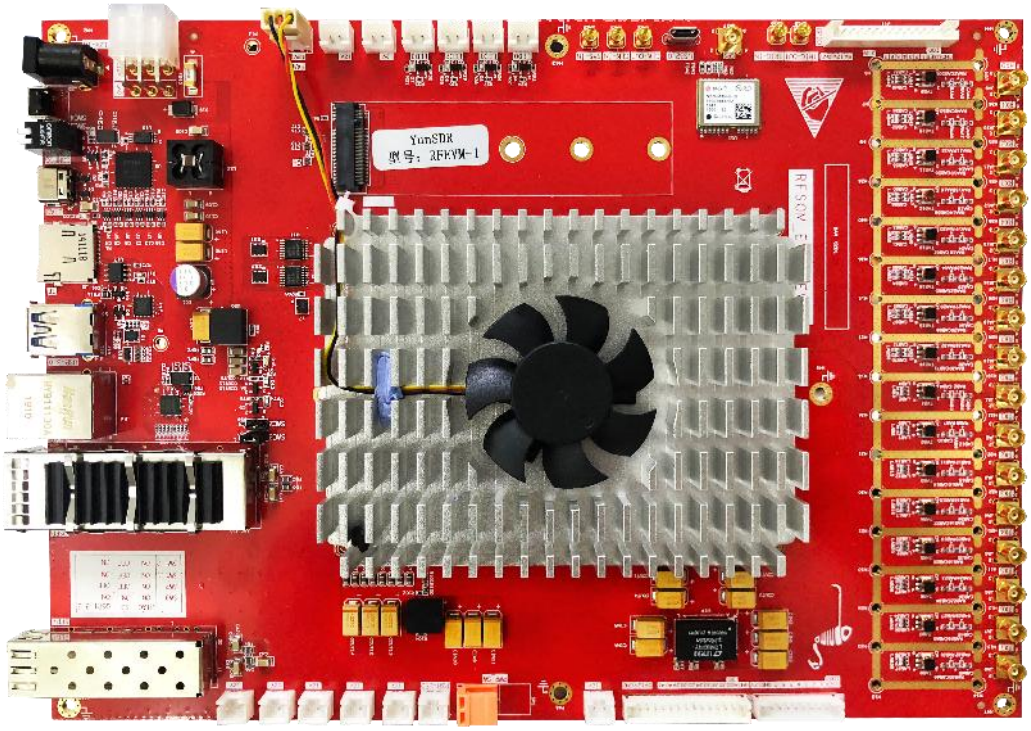
RFEVM 开发平台

FPFA 开发基础教程

Rev. 1.0



修订记录

版本	修订日期	修订内容
1.0	2023 年 07 月 7 日	V1.0 初始版本
1.1	2024 年 07 月 12 日	添加 PS/PL 互联例程

目 录

目 录 1

1 开发板简介 3

2 实验环境搭建 6

软件环境 6

硬件环境 6

Vitis 工程说明 6

3 PL 端外设 7

3.1 LED 呼吸灯实验 7

3.1.1 硬件设计 7

3.1.2 硬件工程搭建 8

3.1.3 下载验证 23

3.2 PL 端 DDR4 读写实验 25

3.2.1 硬件设计 27

3.2.2 硬件工程搭建 28

3.2.3 下载验证 45

3.2.3 参考文档 48

3.3 GTY 收发器误码率测试 IBERT 实验 49

3.3.1 硬件介绍 49

3.3.2 硬件工程搭建 51

3.3.3 下载验证 60

3.3.4 参考文档 66

4 PS 端外设 67

4.1 IPI 设计建立 67

4.1.1 硬件工程搭建 67

4.1.2 软件工程搭建 88

4.2 体验 ARM，裸机输出”Hello World” 93
4.2.1 硬件介绍 93
4.2.2 vitis 程序开发 94
4.3 PS 端 RTC 中断实验 102
4.3.1 软件设计 104
4.3.2 下载验证 108
4.3 PS 端 IIC 的使用 110
4.3.1 硬件设计 110
4.3.2 软件设计 111
4.3.3 下载验证 114
4.4 PS 端以太网使用之 lwip 115
4.4.1 硬件设计 115
4.4.2 软件设计 117
4.4.3 下载验证 128
4.4.4 参考文档 132
4.5 SD 卡读写 TXT 文本实验 133
4.5.1 硬件设计 133
4.5.2 软件设计 134
4.5.3 下载验证 143
4.5.4 参考文档 145
4.6 基于 PS 的 USB slave 功能测试 145
4.6.1 硬件设计 145
4.6.2 软件设计 145
4.6.3 下载验证 148
4.6.4 参考文档 150

1 开发板简介

RFEVM 开发板由 RFSOM 核心板和载板构成，核心板和载板之间使用高速板间连接器连接。

RFSOM 核心板采用 Xilinx 公司的 Zynq™ UltraScale+™ RFSoc Gen3 系列 ZU47DR FPGA 主芯片，它支持 8 路 14 位 RF-ADC，最大采样率可达 5GSPS，8 路 14 位 RF-DAC，最高采样率可达 9.84GSPS 。可以降低 RF 信号处理链的复杂性，最大化输入/输出通道密度，而不会牺牲宽带并利用异构处理能力，并且拥有更低的功耗（取消了 ADC/DAC 组件，消除了 FPGA 至模拟的接口功耗）。Zynq UltraScale+器件中提供 ARM Cortex-A53 处理子系统、UltraScale+可编程逻辑和最高信号处理带宽，能够提供综合 RF 信号链，满足无线、有线电视接入、测试测量、早期预警/雷达以及其它高性能 RF 应用需求。

载板为核心板扩展了丰富的外围接口，其中包含 1 路 M.2 SSD 接口、一路 USB3.0 接口、1 路千兆以太网接口、一路 JTAG/UART 接口、一路 TF 卡接口、一路 sfp 接口和一路 qsfp 接口。

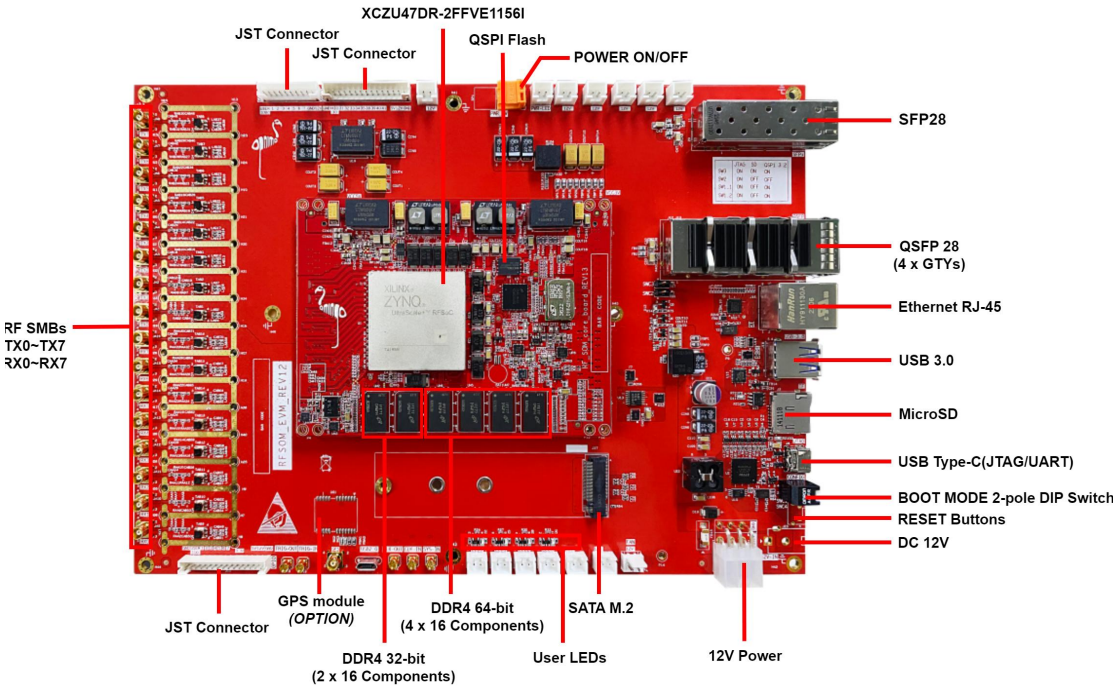


图 1 RFEVM 开发板实物图

下图为整个开发系统的结构示意图：

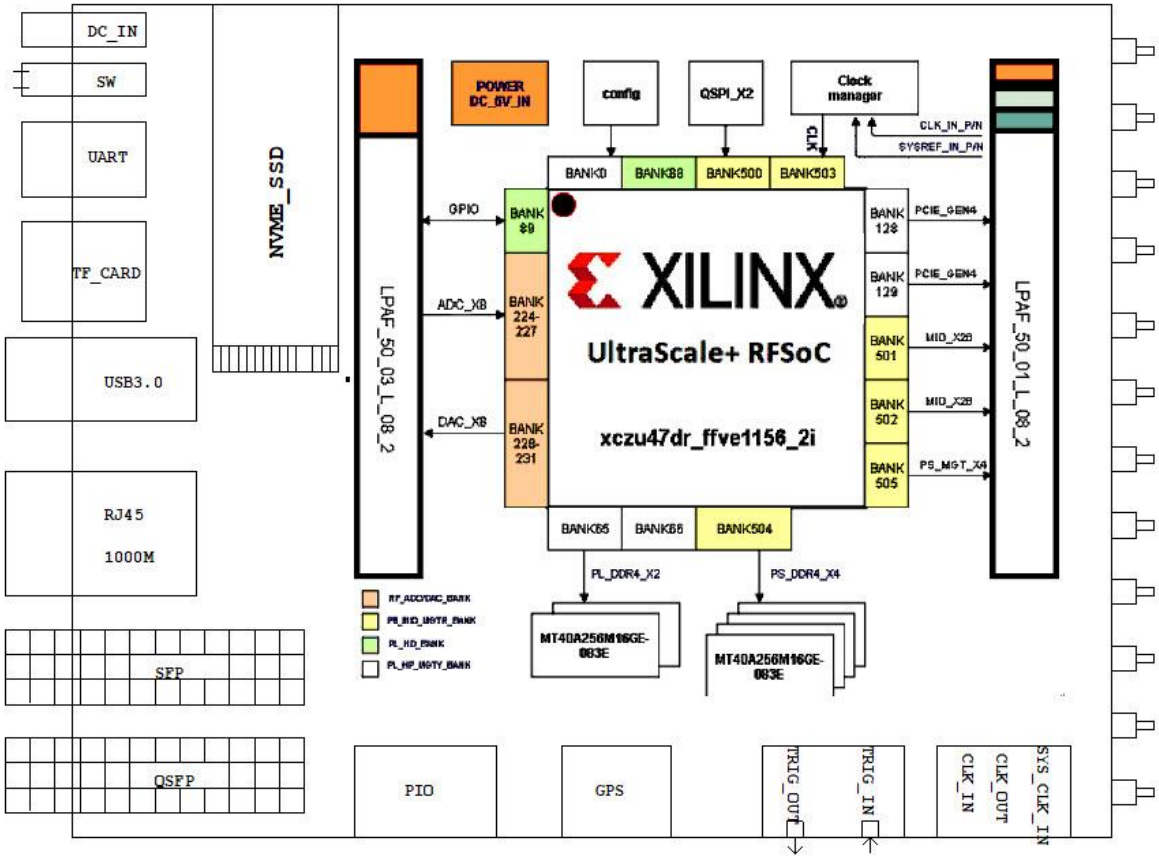


图 2 RFEVM 开发板

通过这个示意图，我们可以看到 RFEVM 开发平台所能包含的接口和功能。

- RFSOM 核心板

由 ZU47DR+4GB DDR4（PS）+2GB DDR4（PL）+1Gb QSPI Flash 组成，另外核心模块提供双晶振时钟源，一个单端 33.3333MHz 有源晶振提供给 PS 系统，一个晶体 32.76MHz 驱动 RFSOC 内部 RTC 电路。

- M.2 接口

一路 PCIe x2 标准的 M.2 接口，用于连接 M.2 的 SSD 固态硬盘。

- USB3.0 接口

一路 USB3.0 接口，支持 HOST、SLAVE 三种模式。

- 千兆以太网接口

一路 10M/100M/1000M 以太网 RJ45 接口，用于和电脑或者其它网络设备行以太网数据交换。

- JTAG&UART 接口

JTAG&UART 调试接口类型为 Type-C 接口，JTAG 和 UART 共用此接口，用于 FPGA 程序的下载和调试。

- Micro SD 卡座

一路 Micro SD 卡座，用于存储操作系统镜像和文件系统。

- QSFP28 光纤接口

一路 QSFP28 光纤接口，支持 40G/100G 的通信速率。

- SFP 光纤接口

一路 SFP 接口，支持 10G 的通信速率。

- 扩展 IO

三组扩展 IO，两组 PL 端扩展 IO，一组 PS 端扩展 IO，可供用户自定义使用。

- LED 灯

4 个可扩展的 LED 直插针座，供用户自定义使用。

- 按键

一个复位按键。

2 实验环境搭建

软件环境

软件开发环境基于 Vivado 2021.2

官方下载链接：

https://www.xilinx.com/support/download/index.html/content/xilinx/en/downloadNav/vivado-design-tools/archive.html

硬件环境

开发板型号	芯片型号
RFEVM	XCZU47DR-2FFVE1156I

Vitis 工程说明

PS 部分的实验，我们这里只有一个硬件工程，在硬件工程里面使能了 PS 所有的外设，所以后面只需要在 vitis 软件下面去添加不同的应用工程即可。

3 PL 端外设

3.1 LED 呼吸灯实验

呼吸灯是在微处理器的控制下，由暗渐渐、亮、然后再由亮渐暗，模仿人呼吸方式方式的 LED 灯。

FPGA 的引脚电平只有“0”和“1”两个等级，因此通过改变引脚单位时间内高电平的输出时间来实现呼吸灯，也就是让 FPGA 引脚输出一系列 PWM 波信号并不断改变 PWM 波的占空比来控制 LED 灯亮度的变化。

PWM 占空比调节示意图如下图所示：

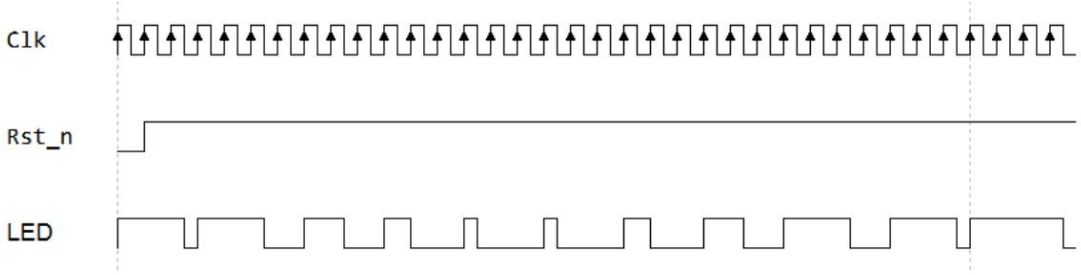


图 3 呼吸灯 PWM 占空比示意图

3.1.1 硬件设计

RFEVM 开发板上有 4 个用户可可控制的 LED 灯。其原理图如下图所示：

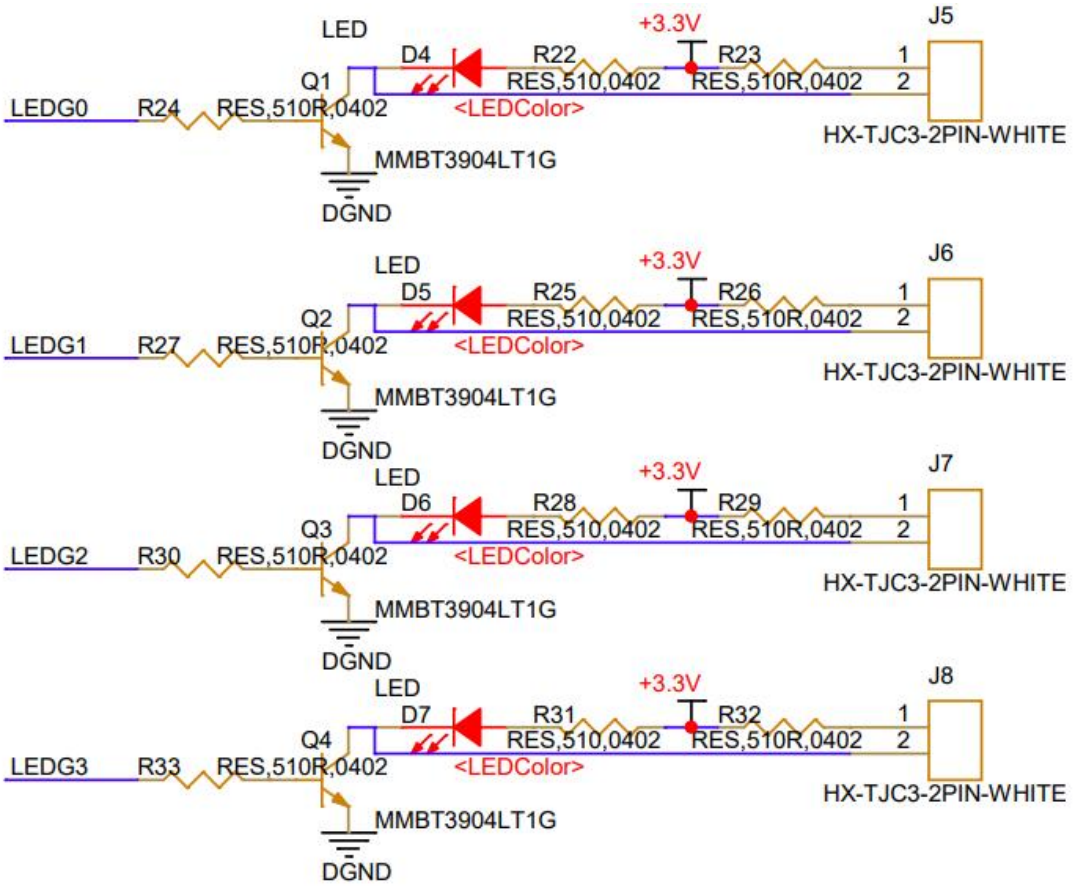


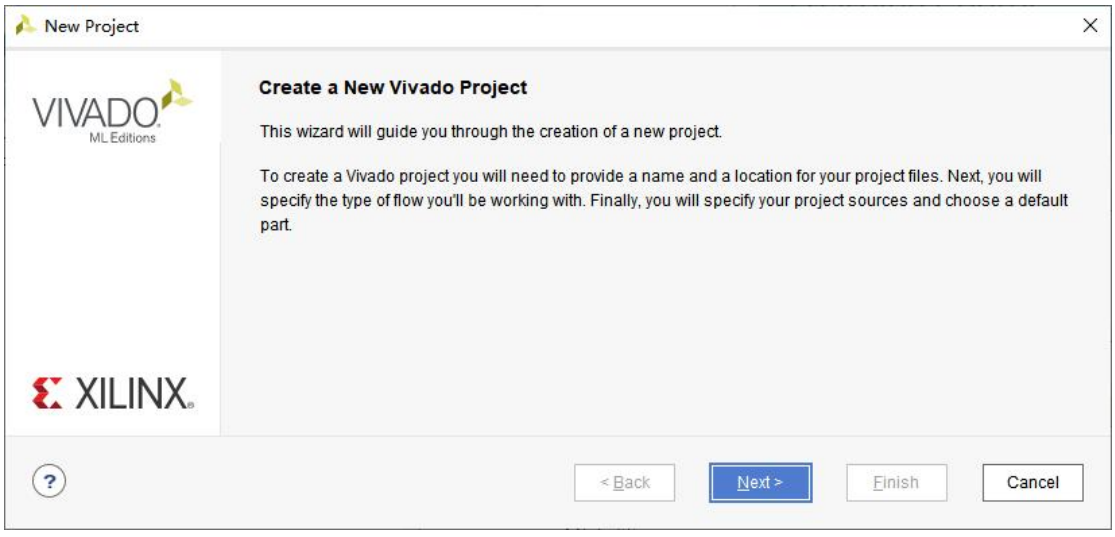
图 4 LED 灯硬件原理图

3.1.2 硬件工程搭建

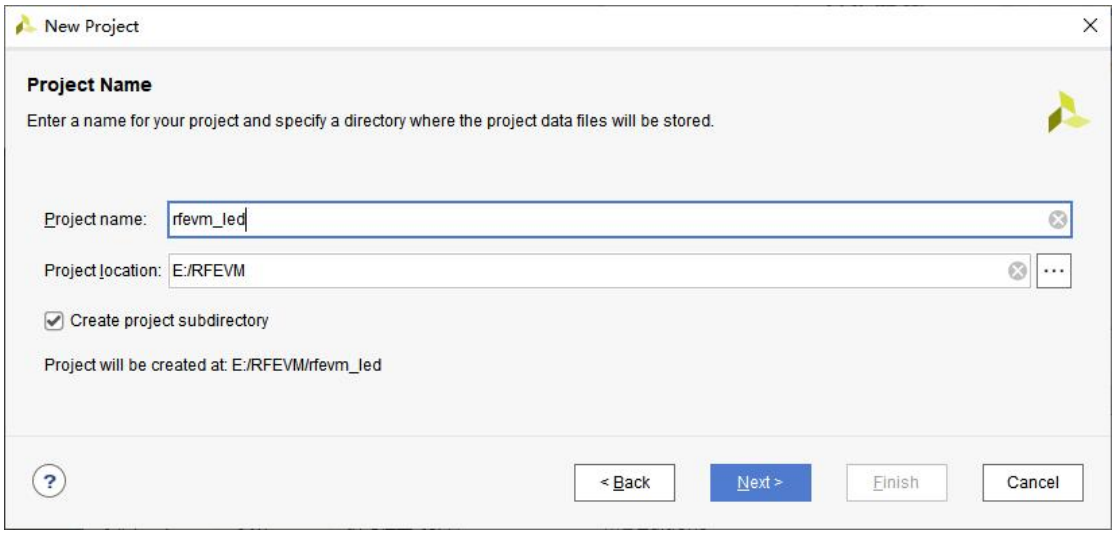
1) 在 PC 端，打开 Vivado

2) 点击 Create Project

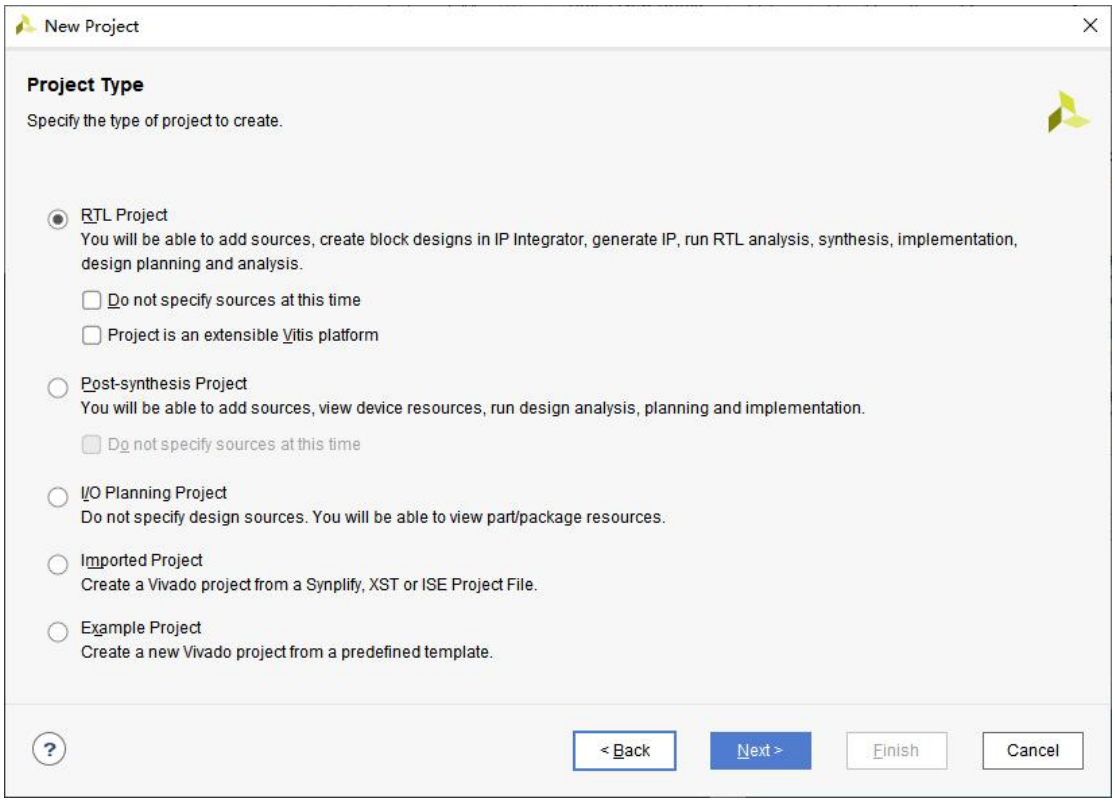
3) 点击 Next



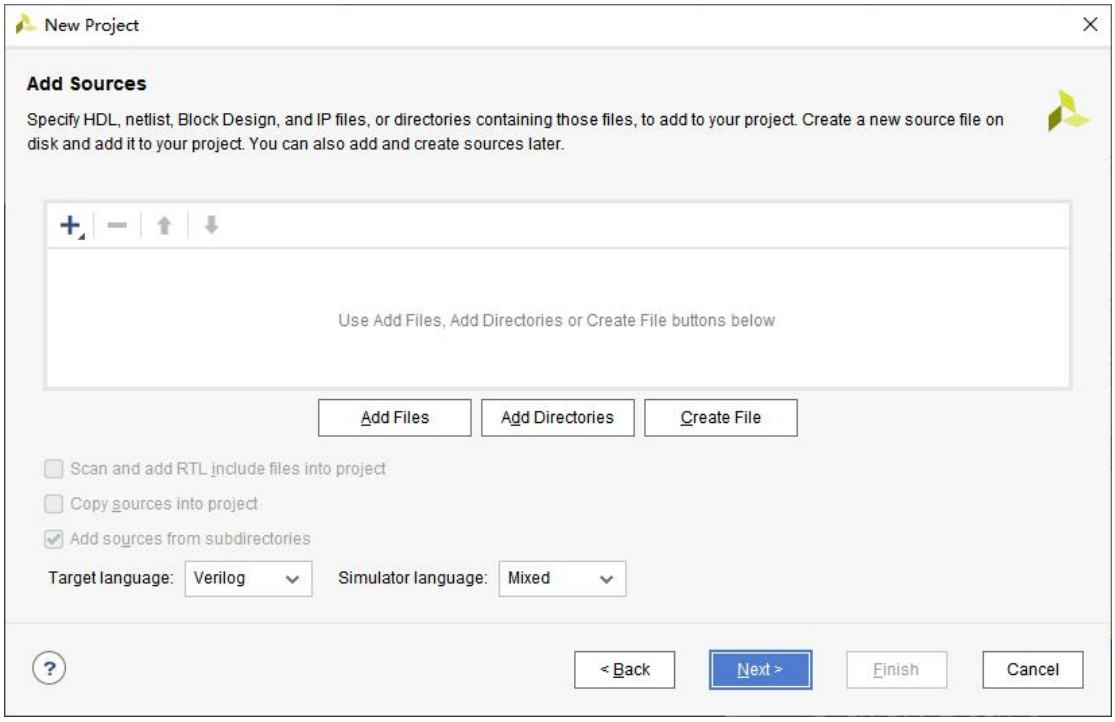
4) 设定工程名称和位置，点击 Next



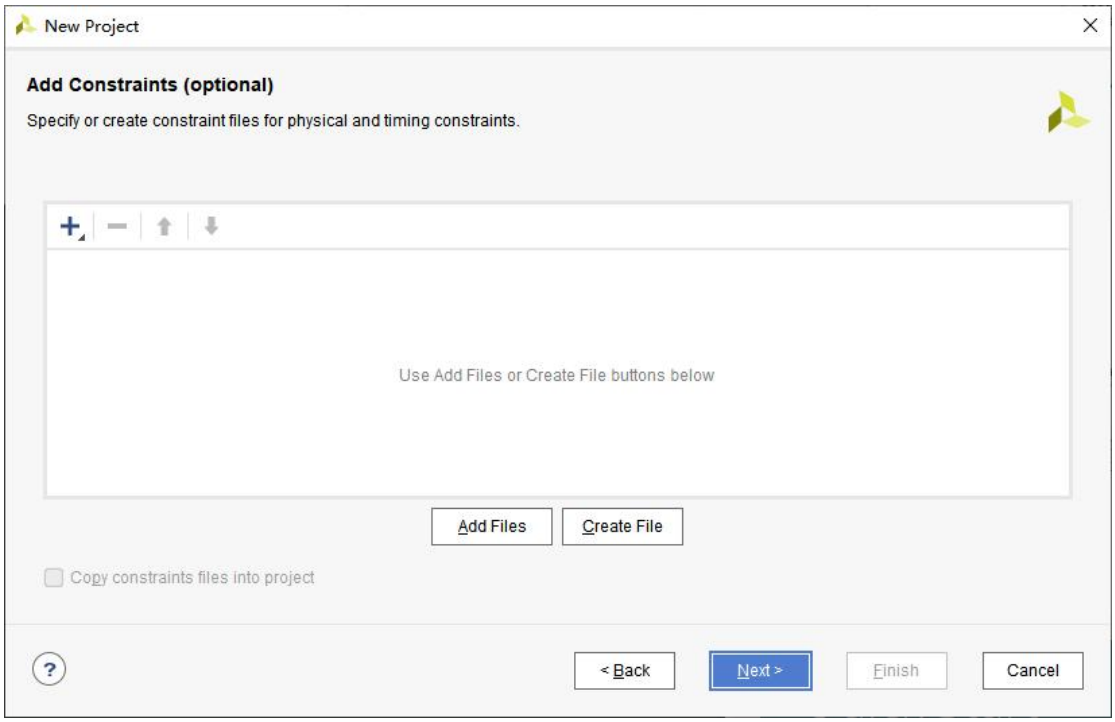
5) 点击 Next



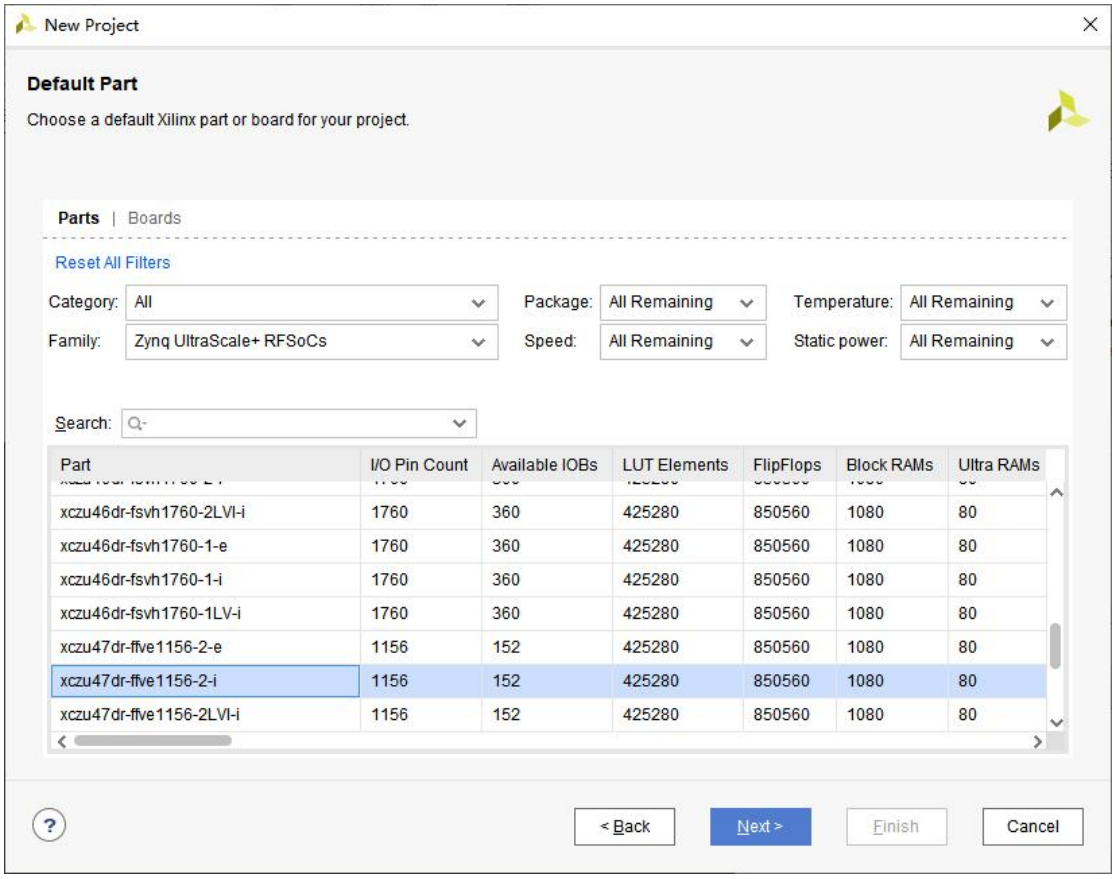
6) 暂时不添加文件，点击 Next



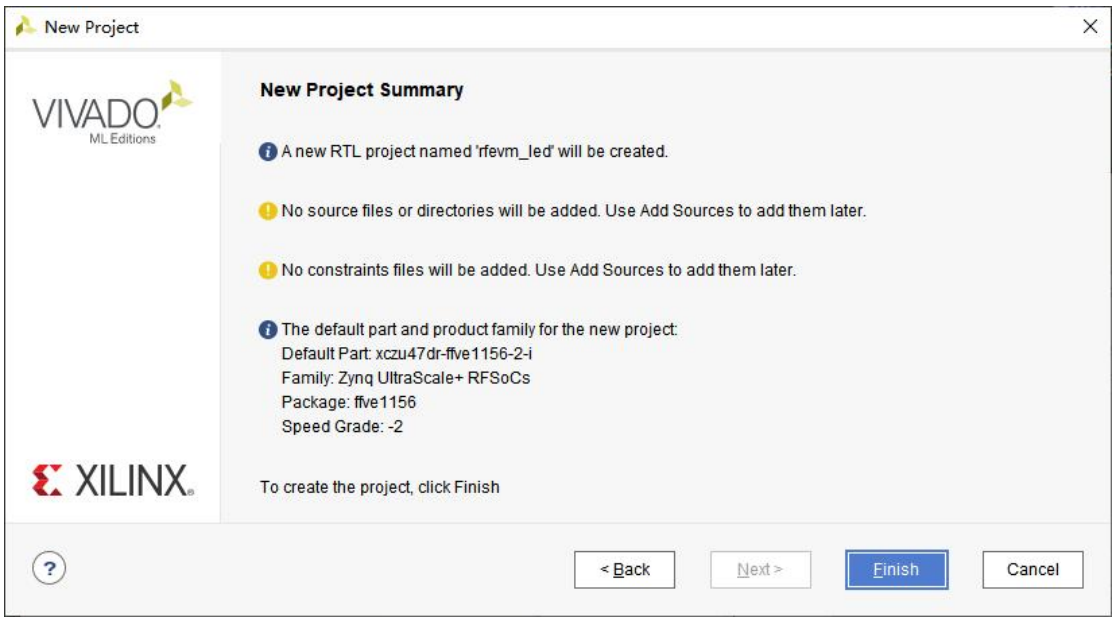
7) 点击 Next



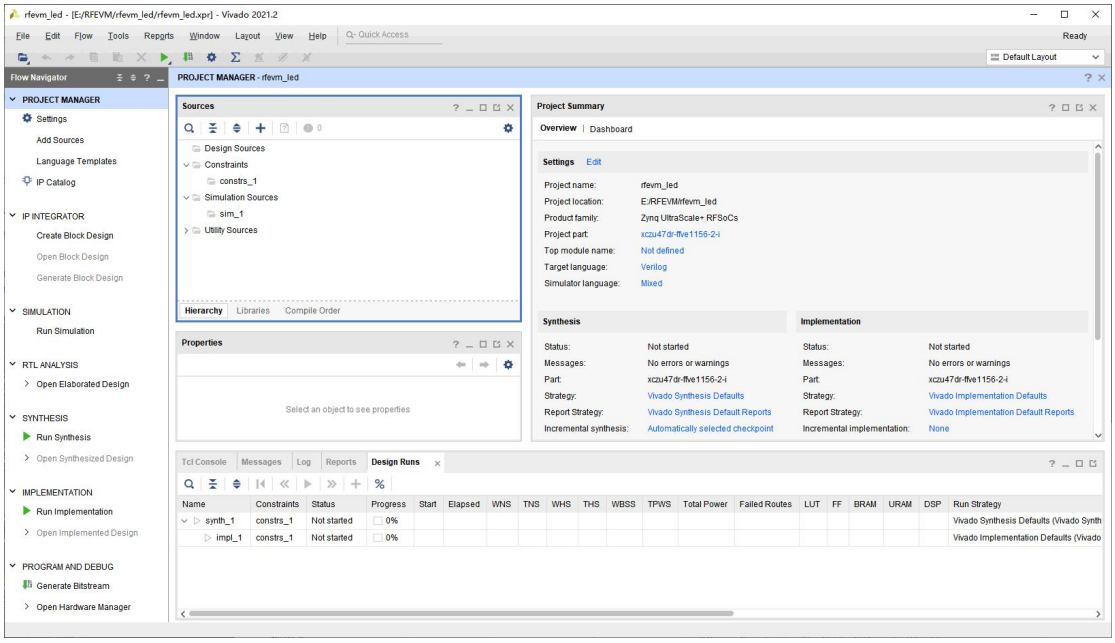
8) 选择 FPGA 型号 xczu47dr-ffve1156-2-i，点击 Next



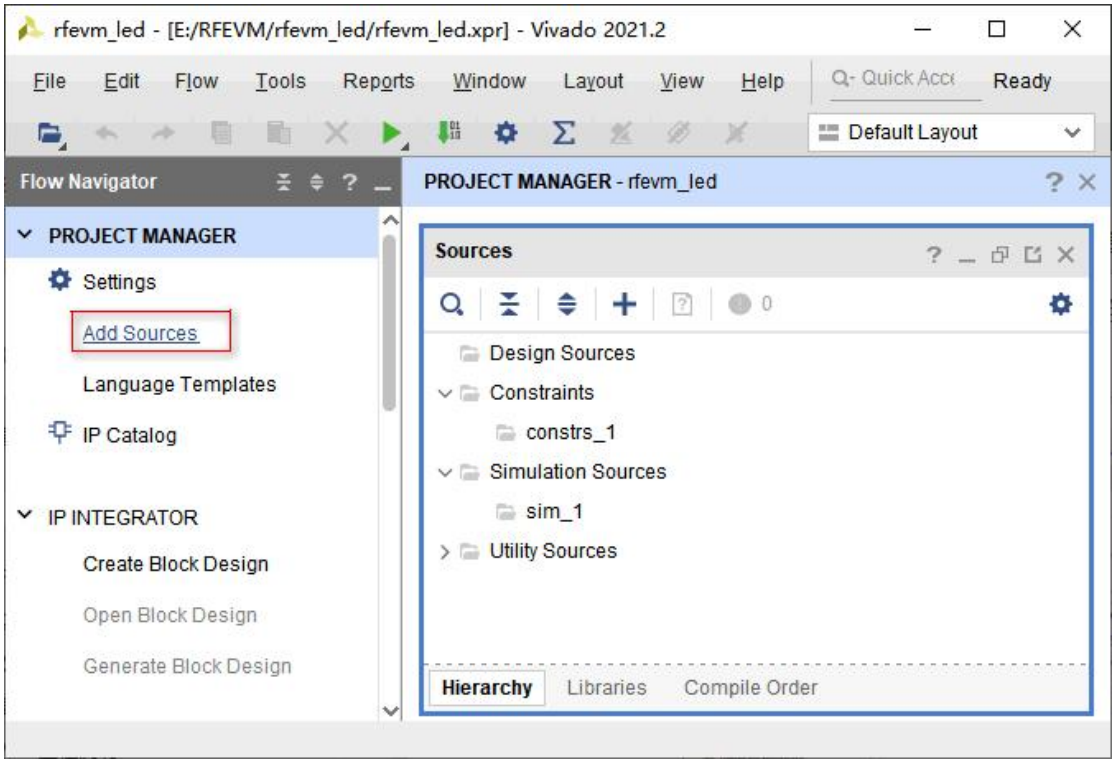
9) 点击 Finish



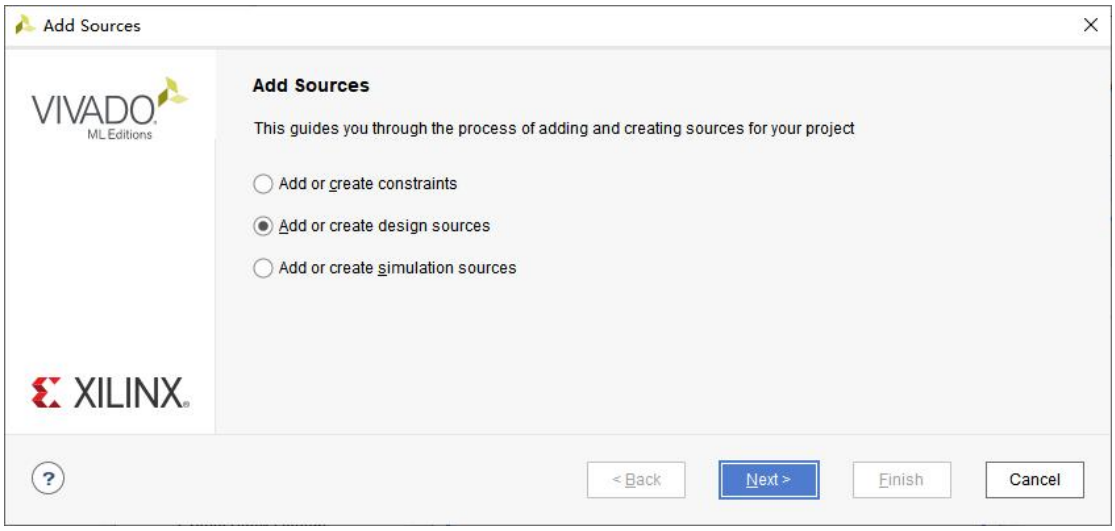
这里我们能看到建立的工程



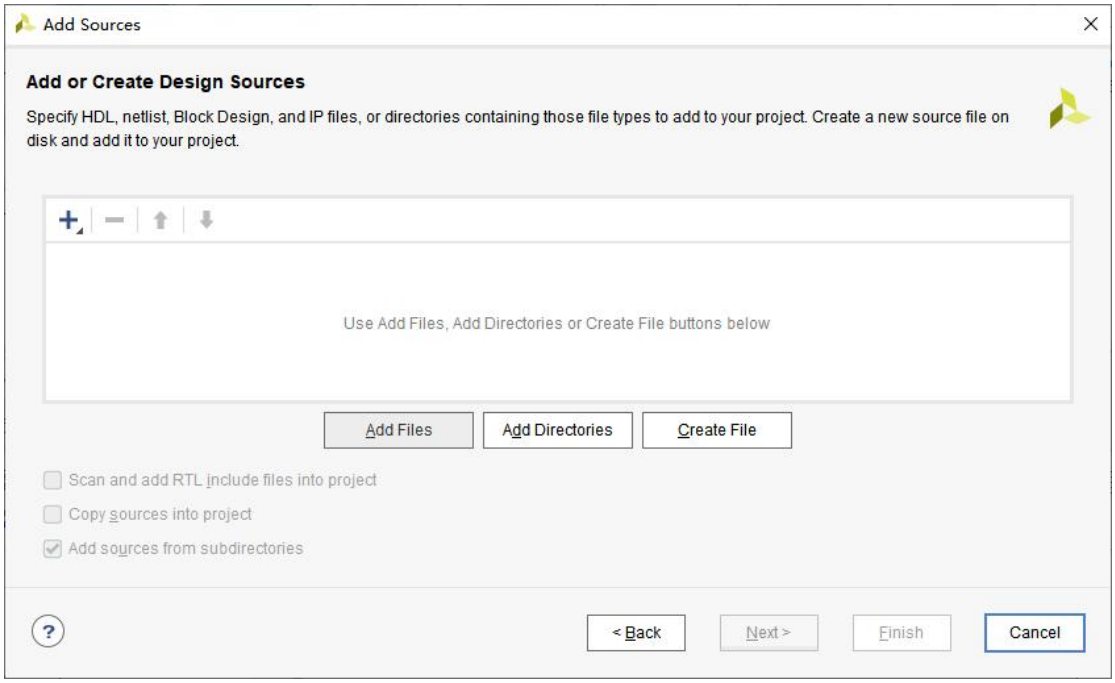
10) 在 Flow Navigator 窗口，点击 Add Source



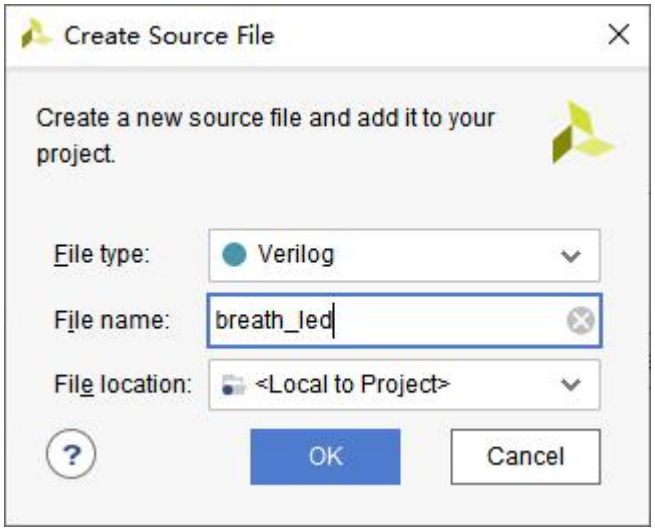
11) 选中添加设计文件，点击 Next



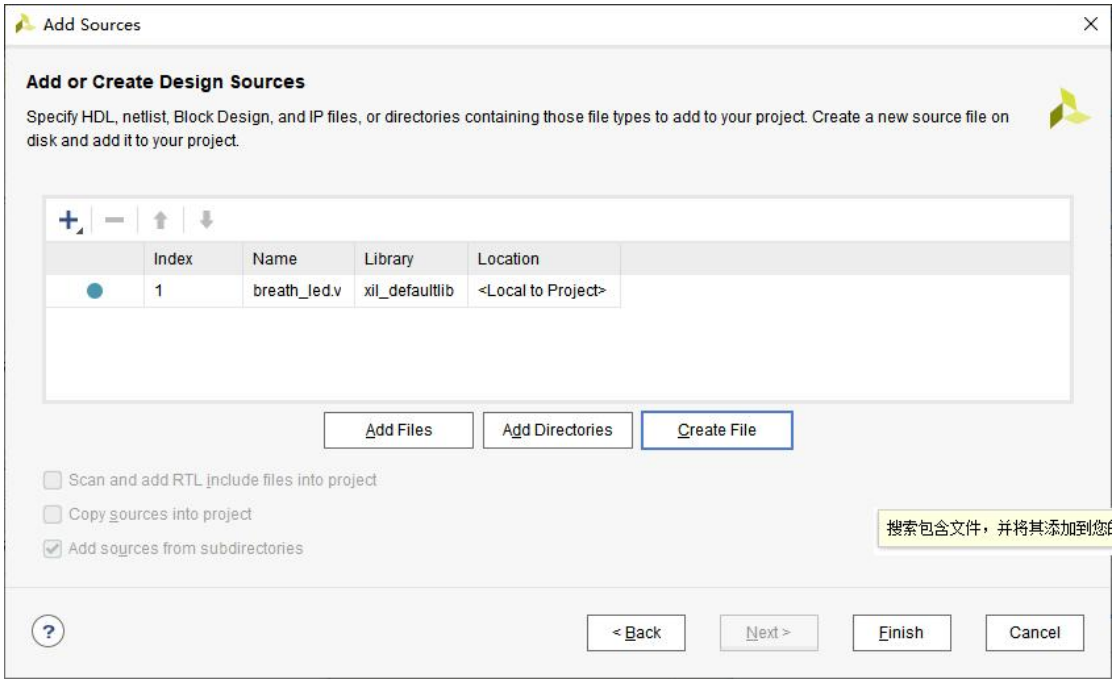
12) 点击 Create Files



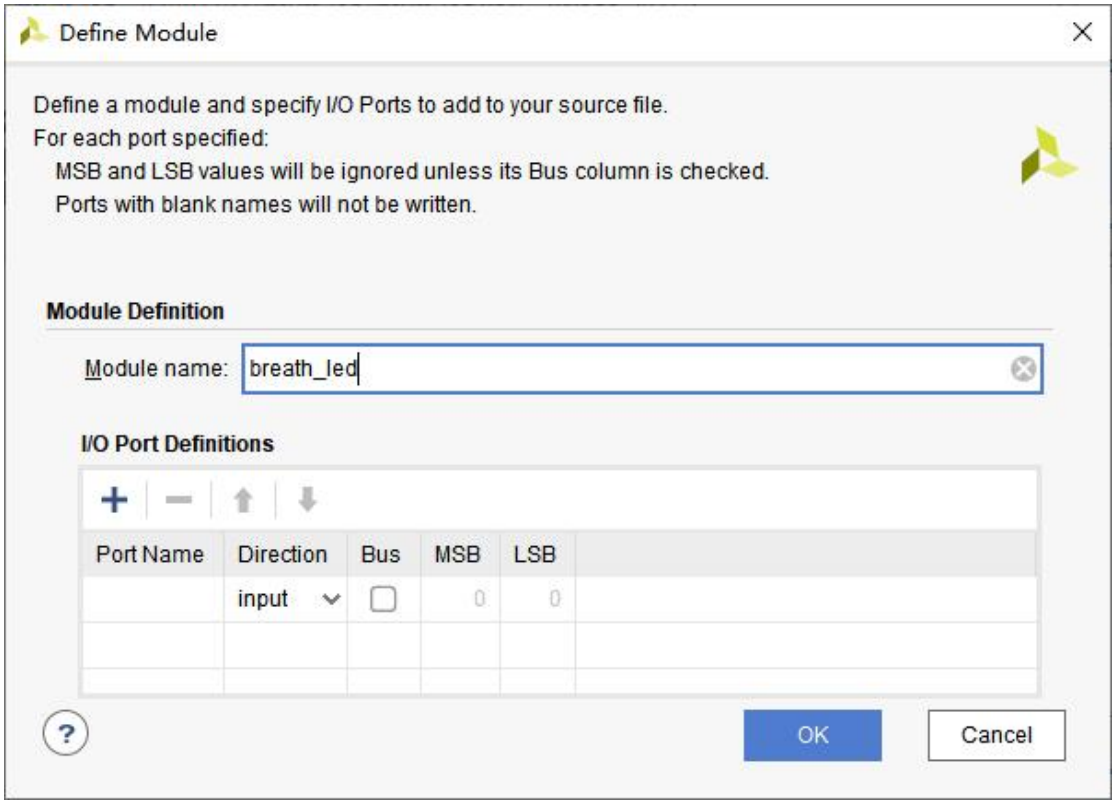
13) 设置名称，点击 OK



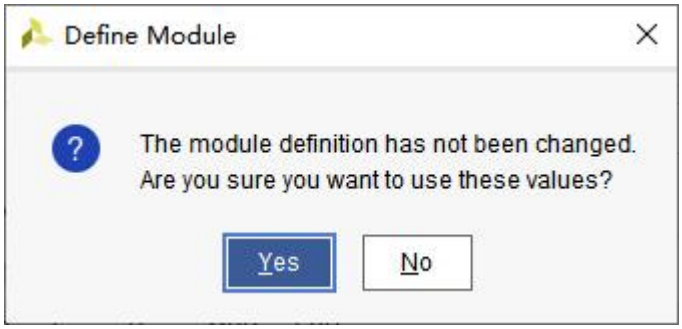
14) 点击 Finish



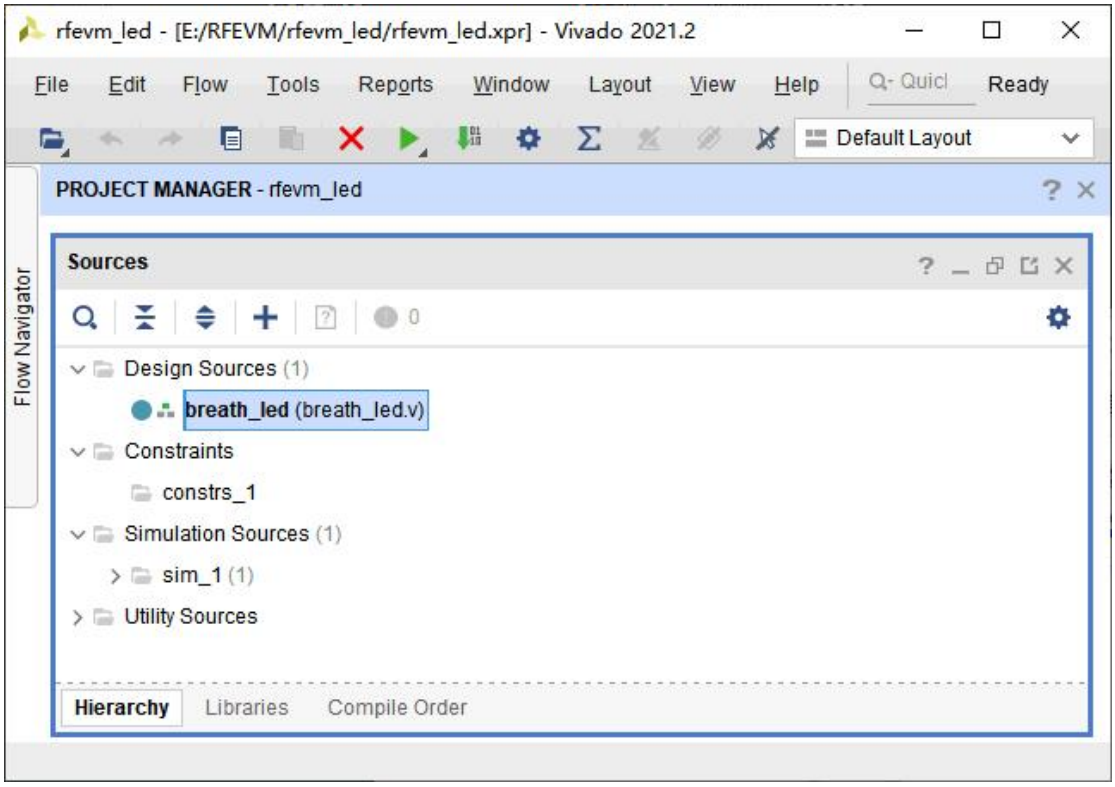
15) 暂时不添加端口，点击 OK



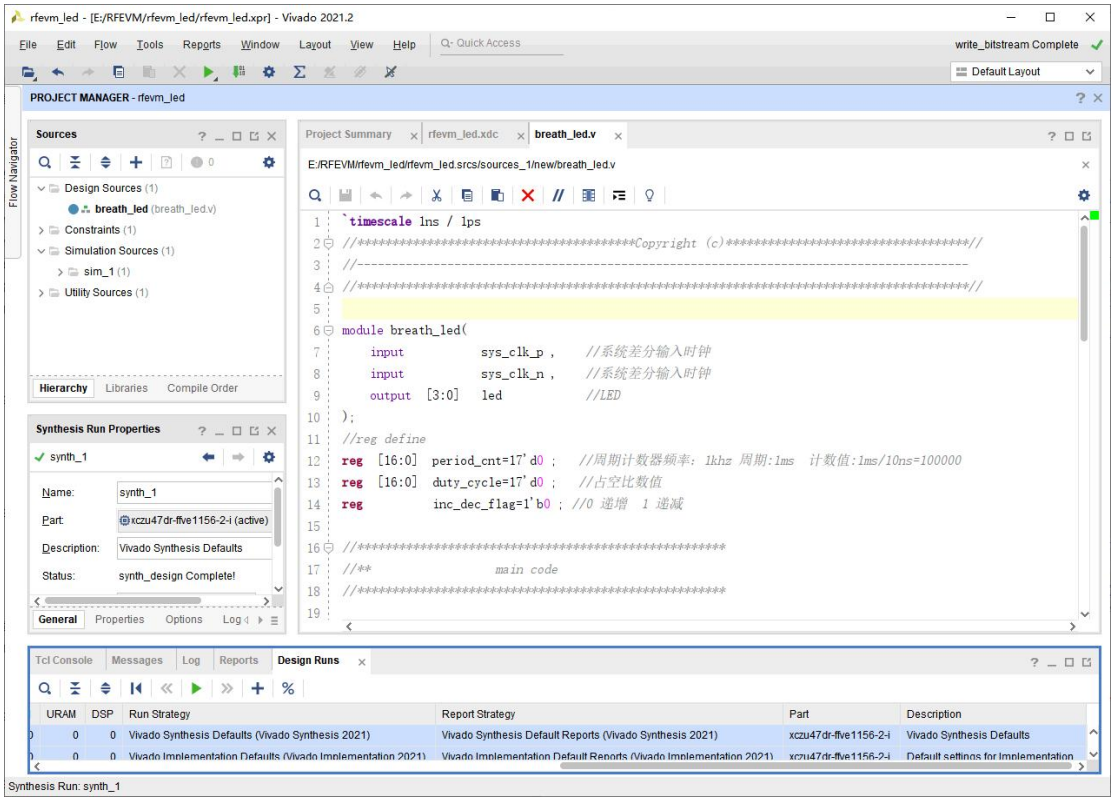
点击 Yes



16) 完成后，可以看到我们新创建的.v 文件



17) 双击 breath_led，然后输入以下代码

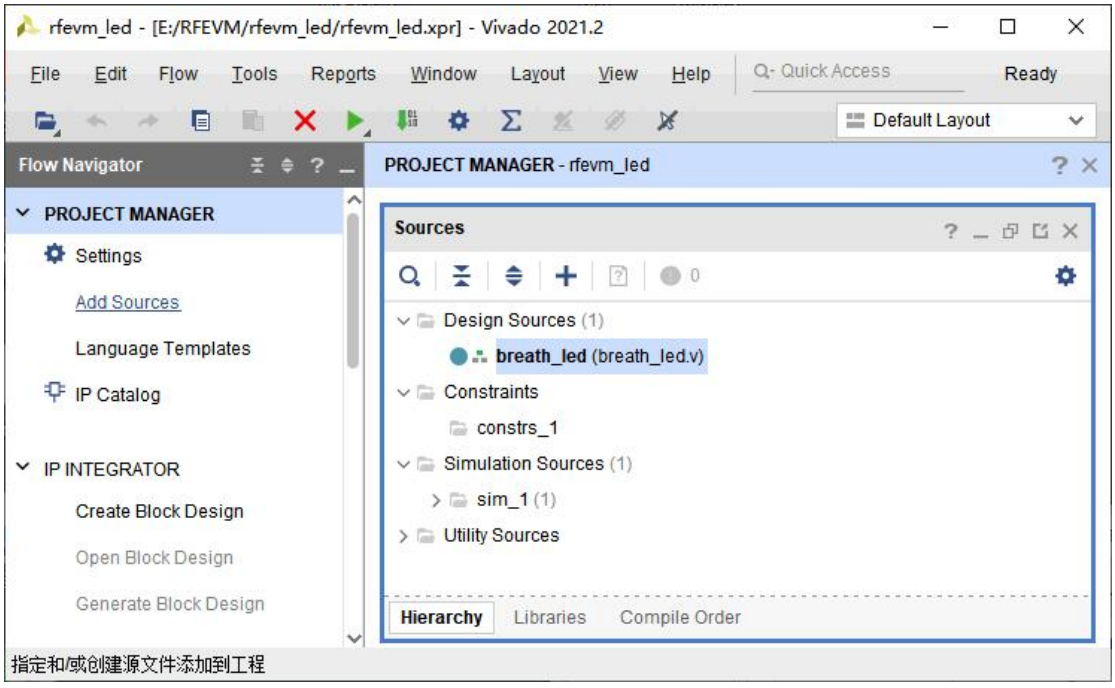



```
`timescale 1ns / 1ps
//****************************************Copyright
```

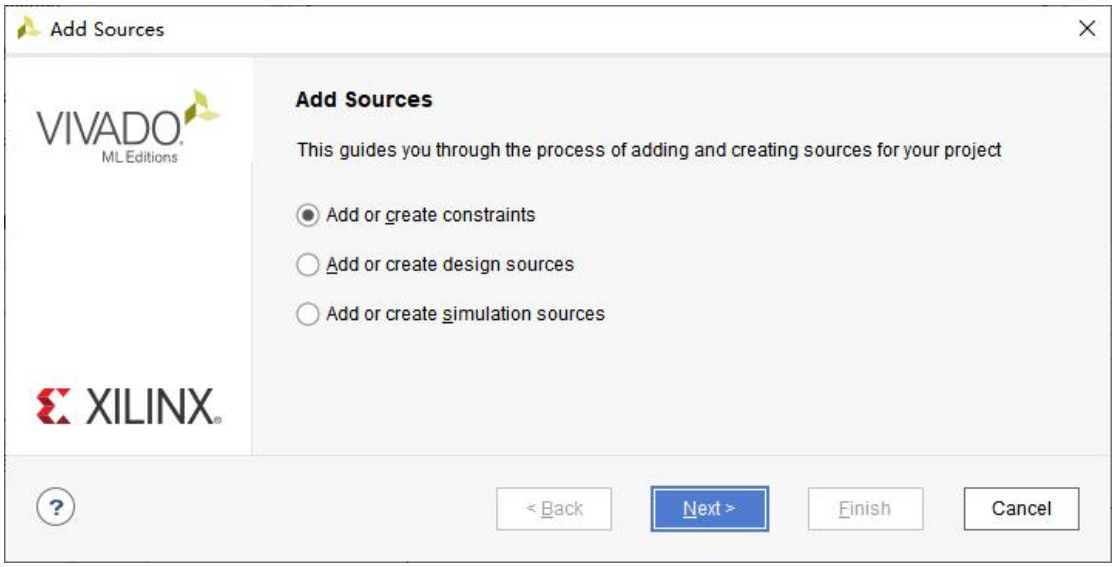
```
//----------------------------------------------------------------------------

module breath_led(
    input           sys_clk_p ,     //系统差分输入时钟
    input           sys_clk_n ,     //系统差分输入时钟
    output  [3:0]   led             //LED
);
//reg define
reg  [16:0]  period_cnt=17'd0 ;   //周期计数器频率：1khz 周期:1ms  计数
值:1ms/10ns=100000
reg  [16:0]  duty_cycle=17'd0 ;   //占空比数值
reg          inc_dec_flag=1'b0 ; //0 递增  1 递减

//*****************************************************
//**                    main code
//*****************************************************

//转换差分信号
IBUFDS diff_clock
(
    .I (sys_clk_p),     //系统差分输入时钟
    .IB(sys_clk_n),     //系统差分输入时钟
    .O (sys_clk)        //输出系统时钟
);

//根据占空比和计数值之间的大小关系来输出 LED
assign   led = (period_cnt >= duty_cycle) ?  4'b1010  :  4'b0101;

//周期计数器
always @(posedge sys_clk) begin
    if(period_cnt == 17'd100000)
        period_cnt <= 17'd0;
    else
        period_cnt <= period_cnt + 1'b1;
end

//在周期计数器的节拍下递增或递减占空比
always @(posedge sys_clk) begin
    if(period_cnt == 17'd100000) begin    //计满 1ms
        if(inc_dec_flag == 1'b0) begin     //占空比递增状态
            if(duty_cycle == 17'd100000)   //如果占空比已递增至最大
                inc_dec_flag <= 1'b1;       //则占空比开始递减
            else                            //否则占空比以 50 为单位递增
                duty_cycle <= duty_cycle + 17'd50;
```

```
            end
            else begin                          //占空比递减状态
                if(duty_cycle == 17'd0)          //如果占空比已递减至 0
                    inc_dec_flag <= 1'b0;        //则占空比开始递增
                else                             //否则占空比以 50 为单位递减
                    duty_cycle <= duty_cycle - 17'd50;
            end
        end
    end

    endmodule
```

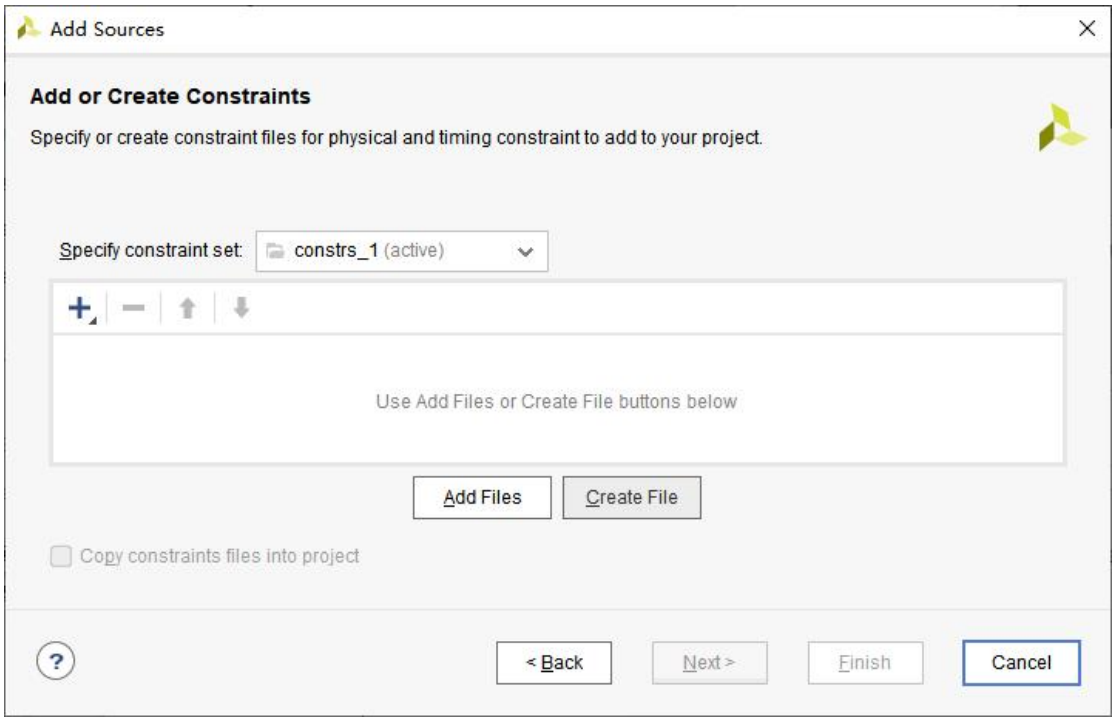
18) 点击 Add Source



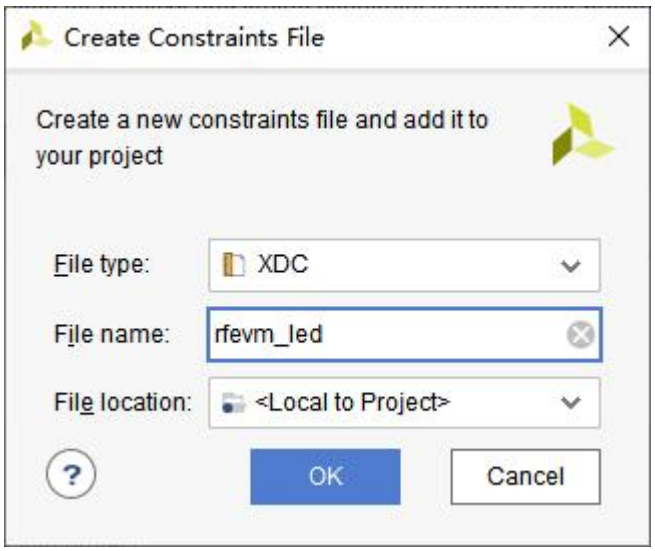
19) 选择添加或创建约束文件，点击 Next



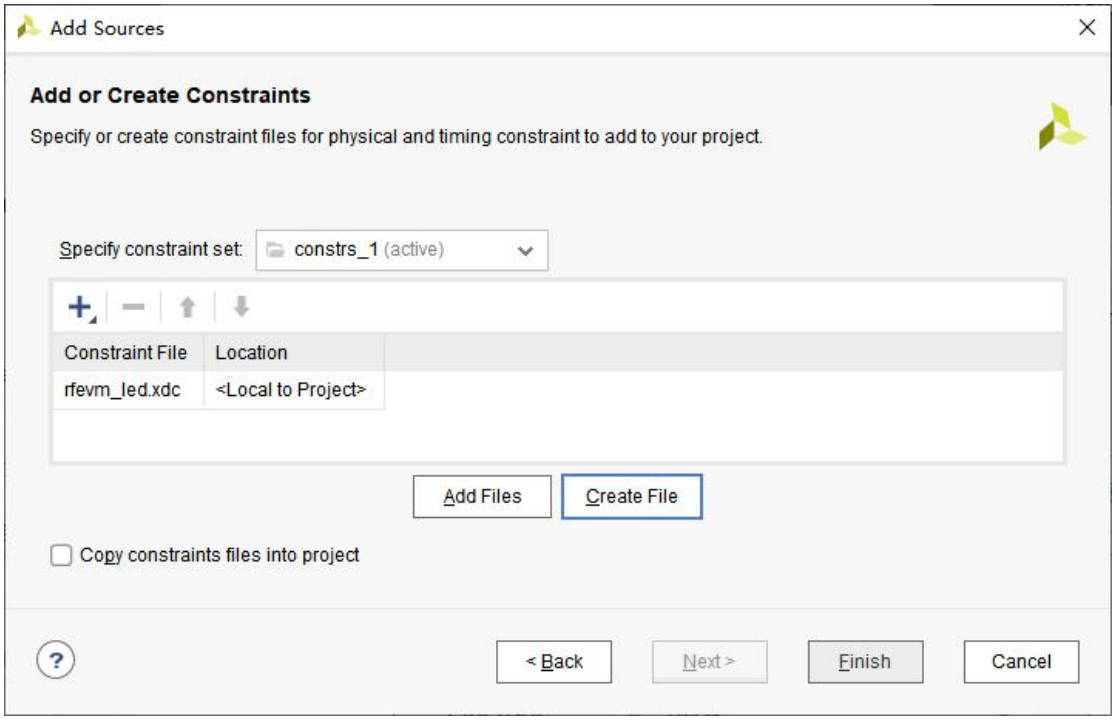
20) 点击 Create File



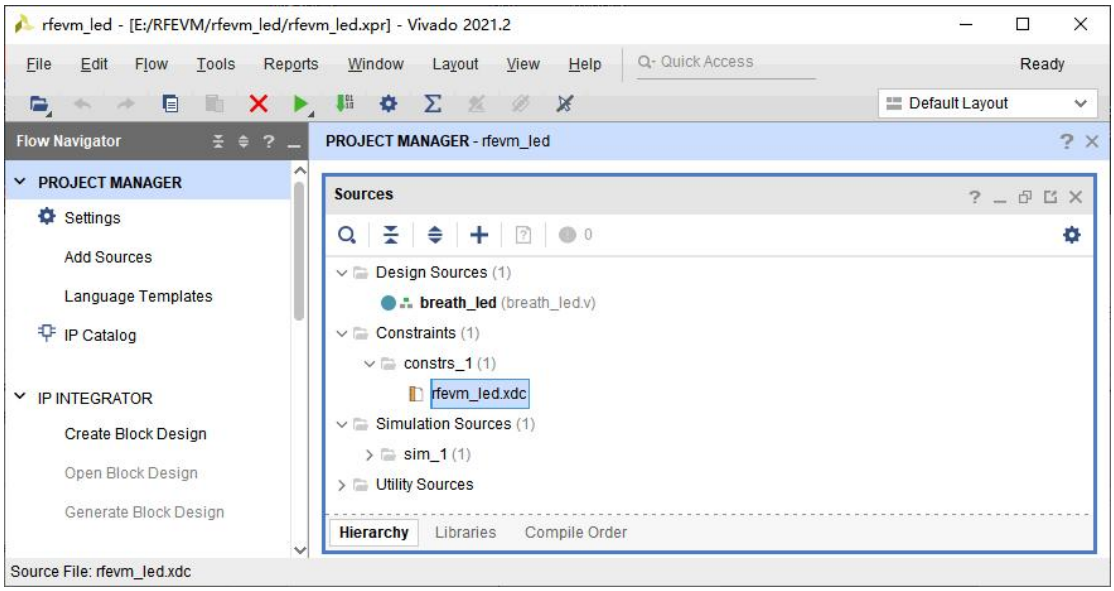
21) 设置名称，点击 OK



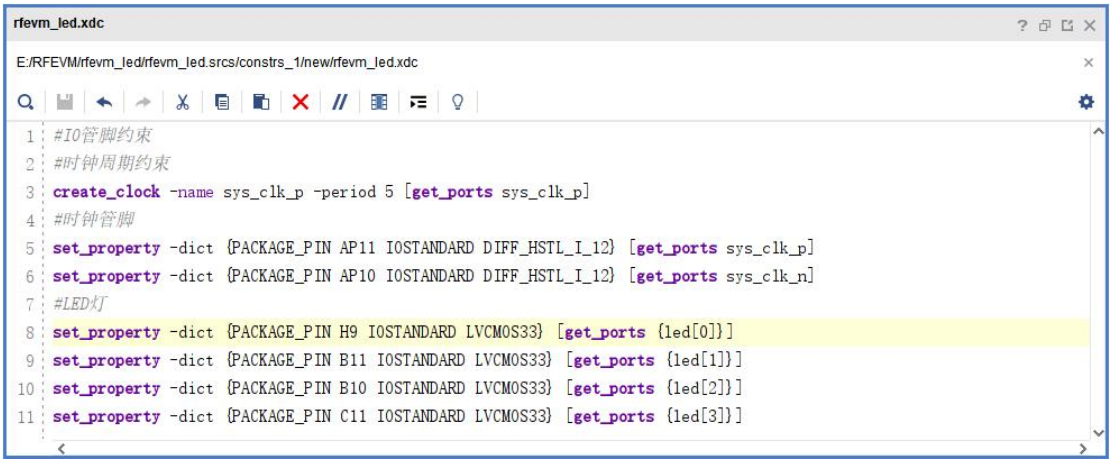
22) 点击 Finish



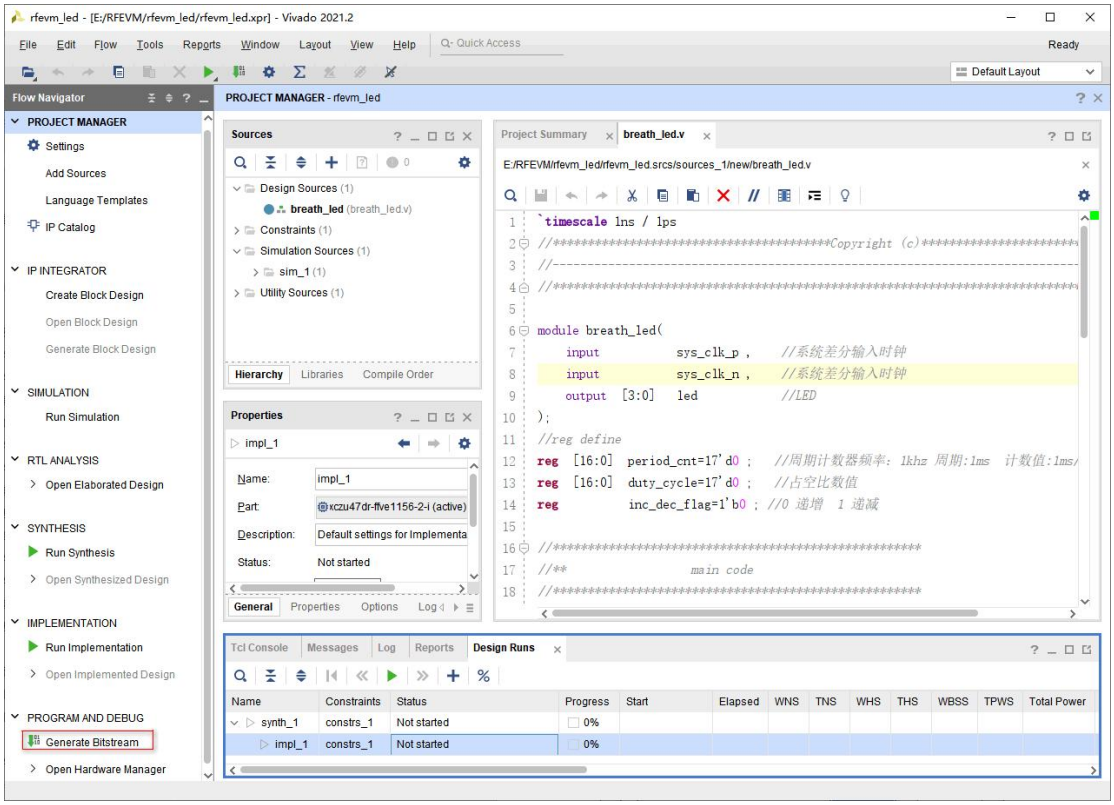
23) 这里可以看到新创建的约束文件



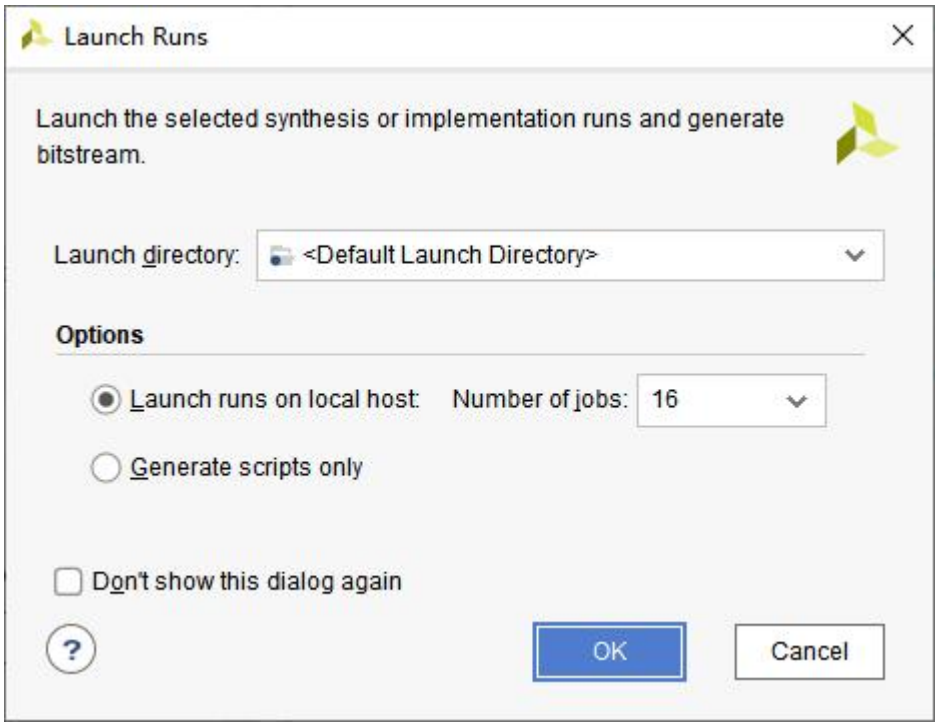
双击 rfevm_led.xdc，输入以下约束：



24) 在 Flow Navigator，点击 Generate Bitstream



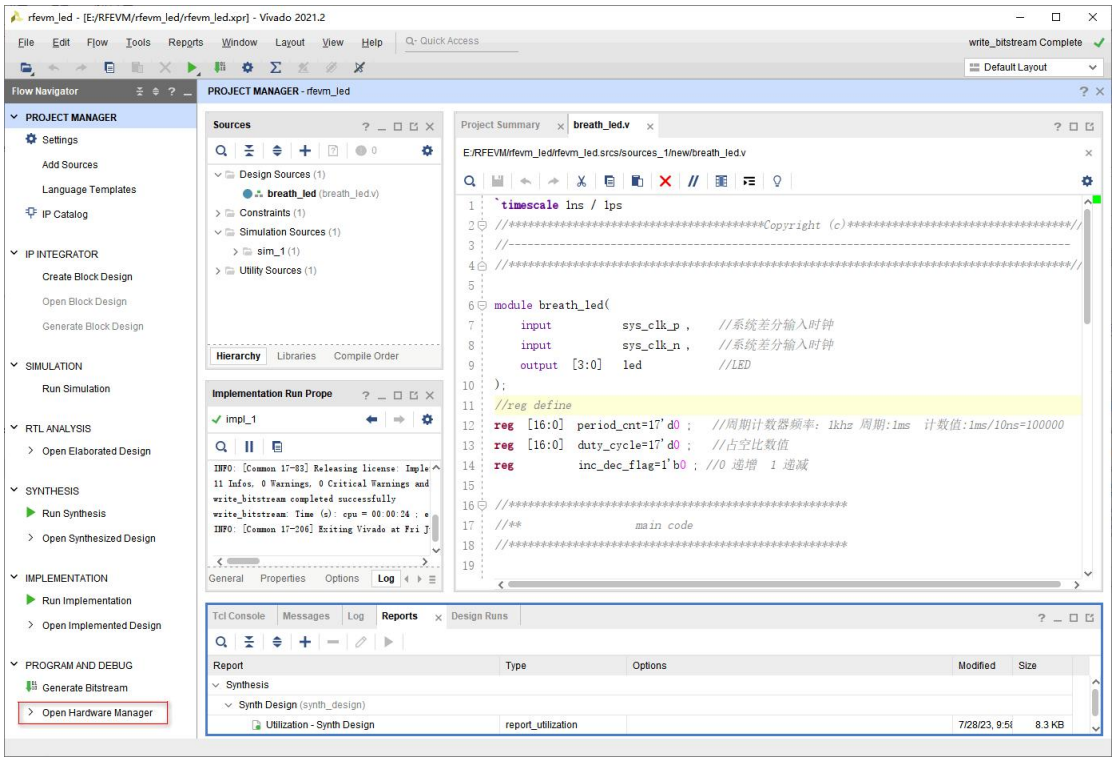
25) 点击 OK，等待生成 bit



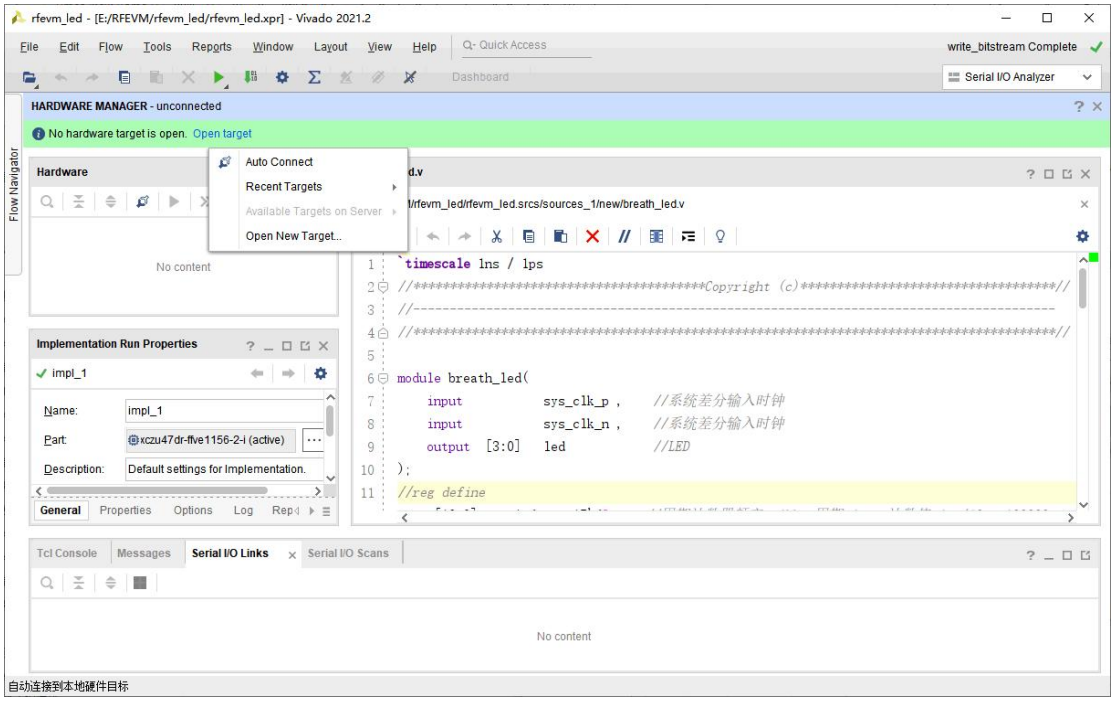
3.1.3 下载验证

用 USB 转 Type-C 线缆，一端连接板卡，一端连到 PC 上，然后给板卡上电。

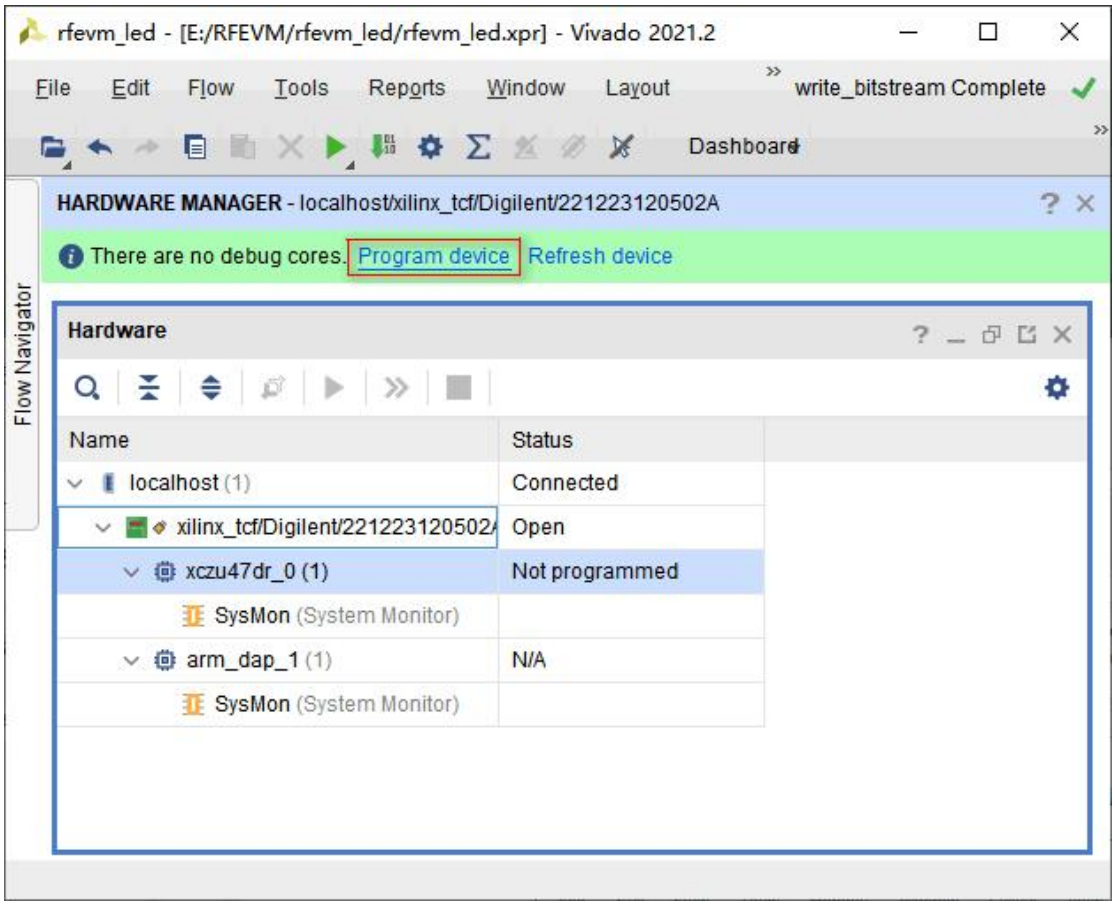
在 Vivado 界面，点击 Open Hardware Manager



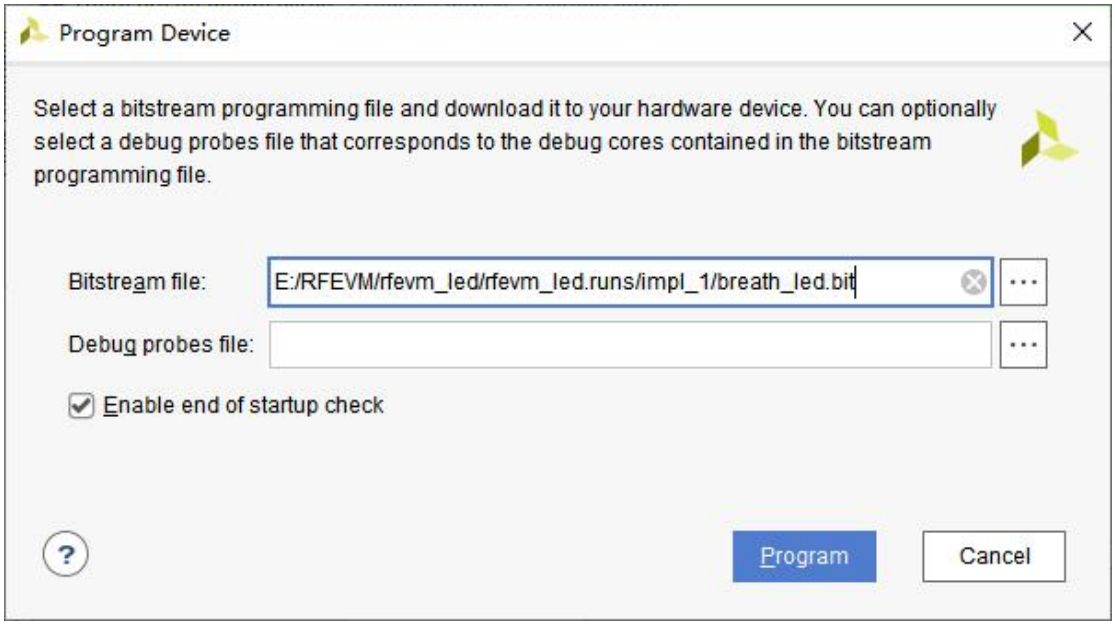
点击 Open target，选择 Auto Connect



点击 Program device



点击 Program



下载完成后，可以看到底板上的 LEDs 灯由暗慢慢变量，再由亮慢慢变暗，即出现“呼吸”的效果。

3.2 PL 端 DDR4 读写实验

DDR4 SDRAM（Double-Data-Rate Fourth Generation Synchronous Dynamic Random Access Memory）是一种高速动态随机存取存储器，它属于 SDRAM 家族的存储器产品，提供了相较于 DDR3 SDRAM 更高的运行性能与更低的电压。

由于 DR4 的时许非常复杂，如果直接编写 DDR4 的控制器代码，那么工作量是非常大的，且性能难以得到保证。值得一提的是，Vivado 软件自带了 DDR4 控制器 IP 核，用户可以直接借助 IP 核来实现对 DDR4 的读写操作，从而大大降低了 DDR4 的开发难度。本次实验将使用 Xilinx 公司的 MIG IP 核来实现 DDR4 的读写操作。

MIG（存储器接口）是一款用于为 AMD FPGA 生成存储器控制器和接口的免费软件工具。下图是 MIG IP 核结构框图。MIG IP 核对外分出了两组接口，左侧是用户接口，用户只有充分掌握这些接口才能操作 MIG；右侧为 DDR 物理芯片接口，负责产生具体的操作时序并直接操作芯片管脚。

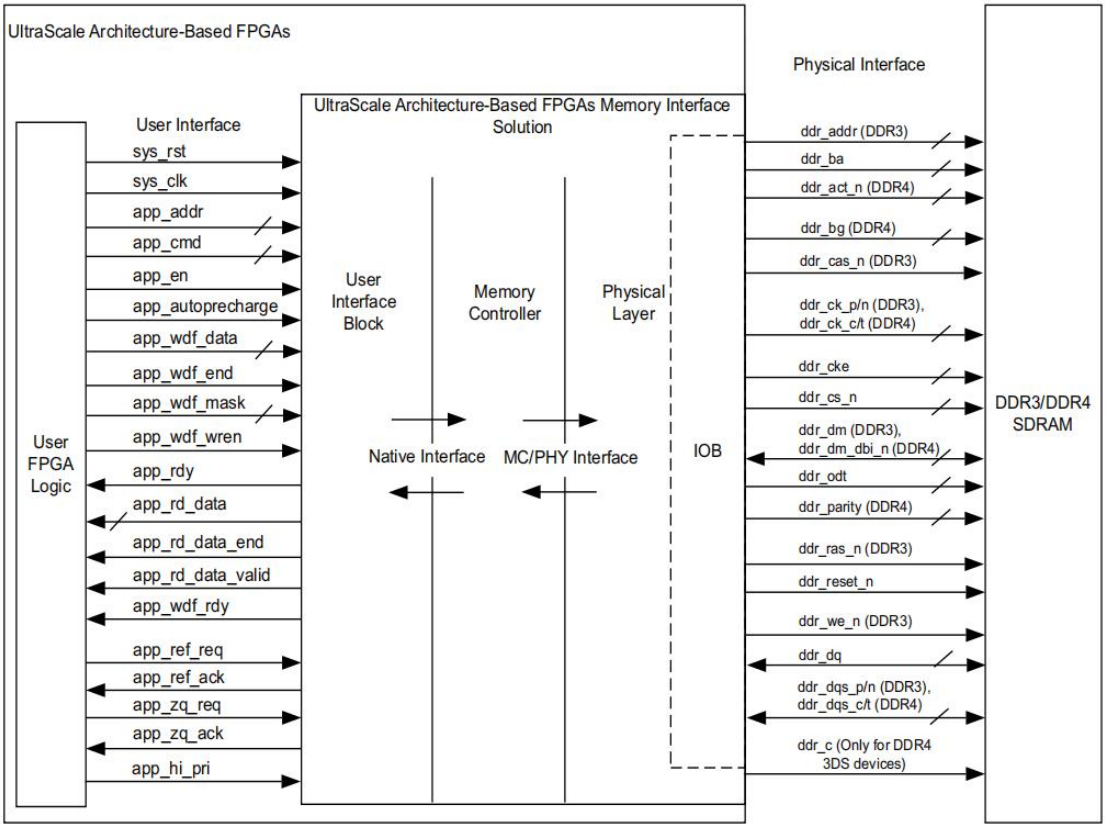


图 5 MIG IP 核结构框图

使用这个 IP 核，用户将可以进行 DDR4 的读写操作，而不必熟悉 DDR4 具体的读写控制时序。

3.2.1 硬件设计

RFEVM 开发板的 PL 端有两颗 16bit DDR4 颗粒，组成 32bit 的 DDR4 通道。

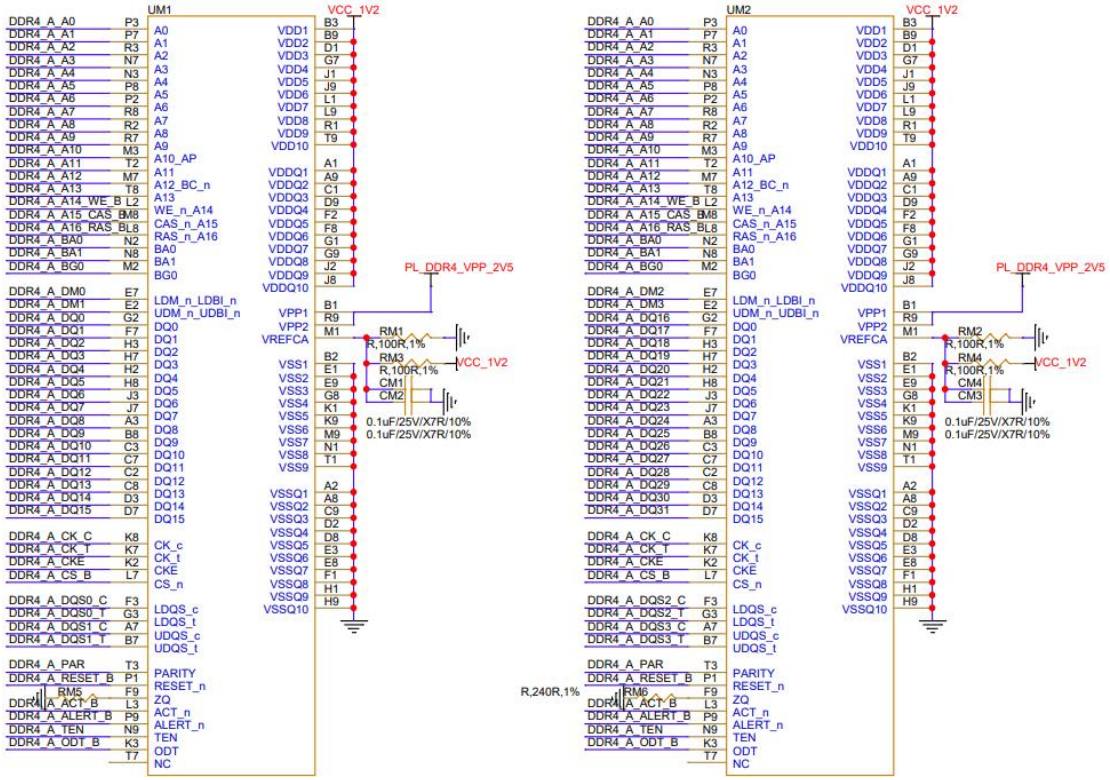


图 6 DDR4 硬件原理图

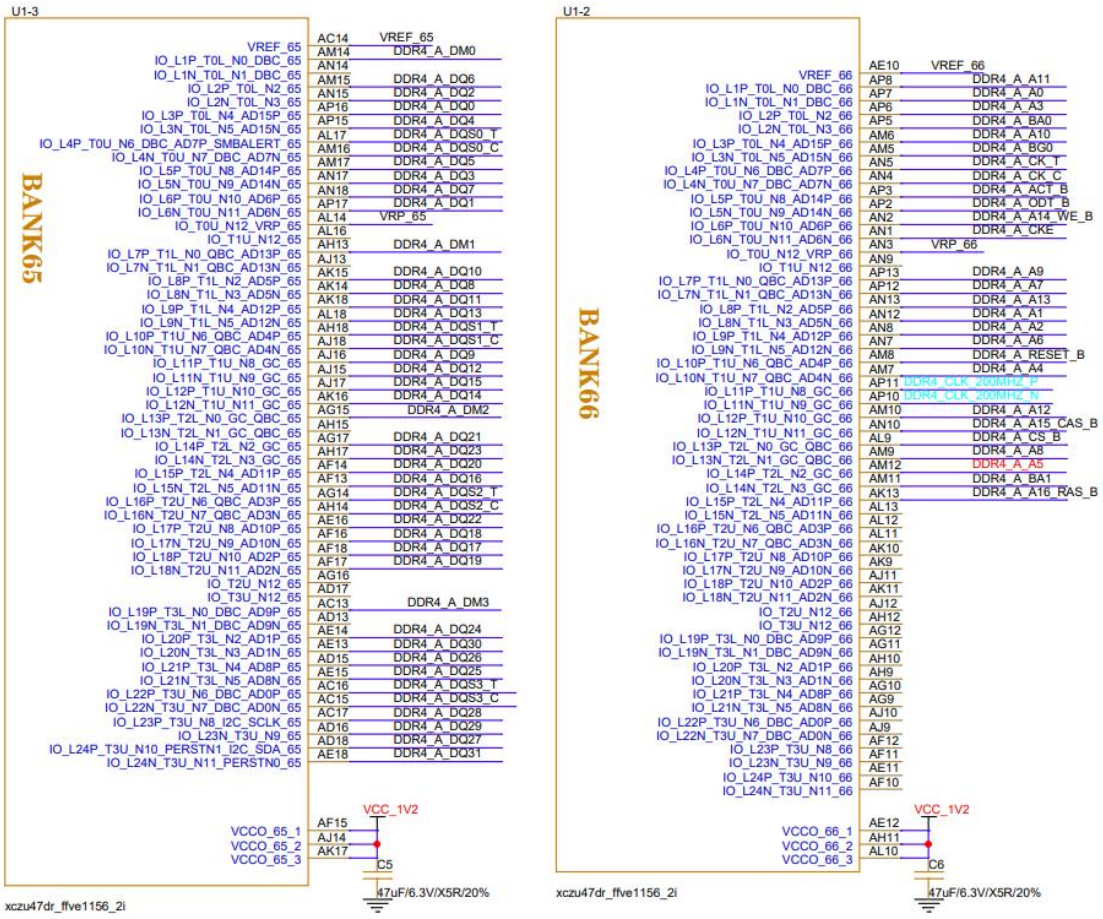
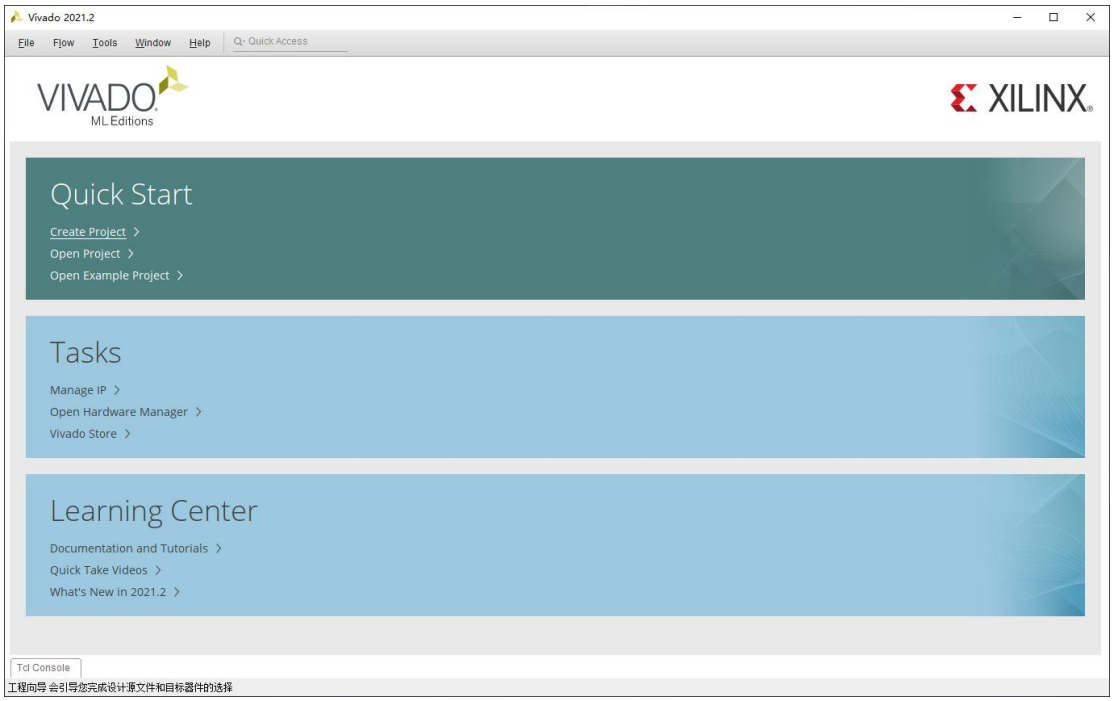


图 7 DDR4 管脚分配硬件原理图

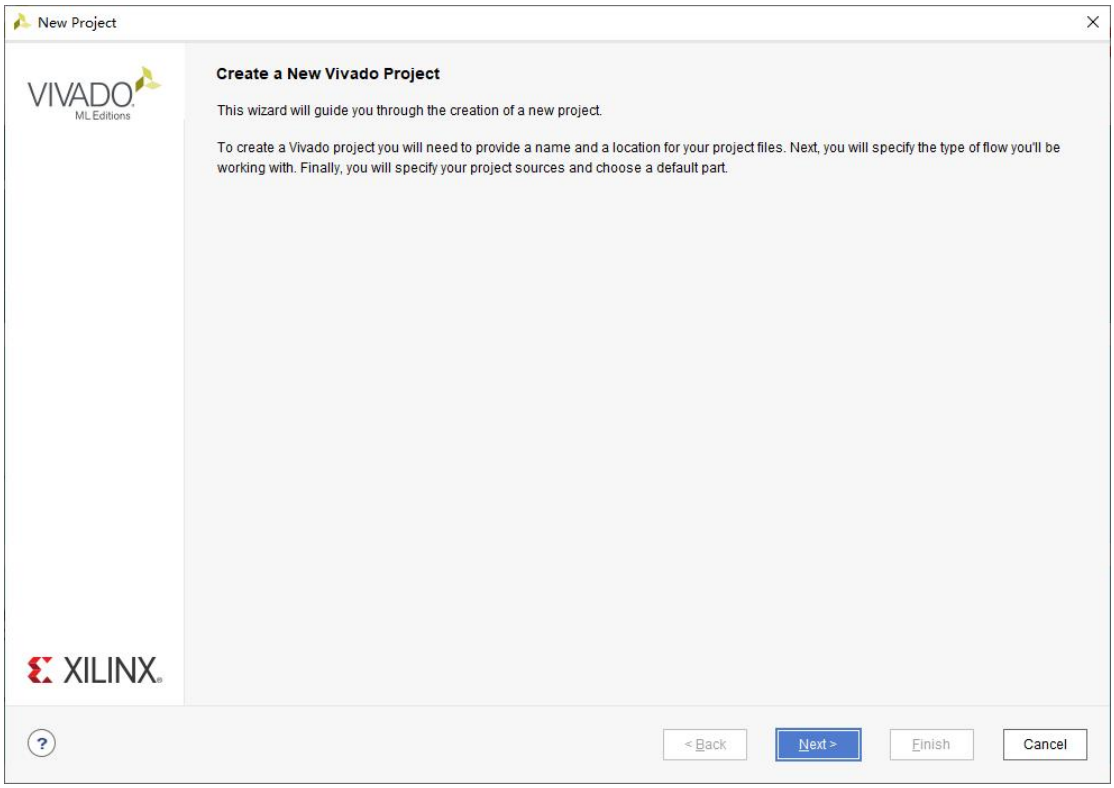
3.2.2 硬件工程搭建

1) 打开 Vivado 软件

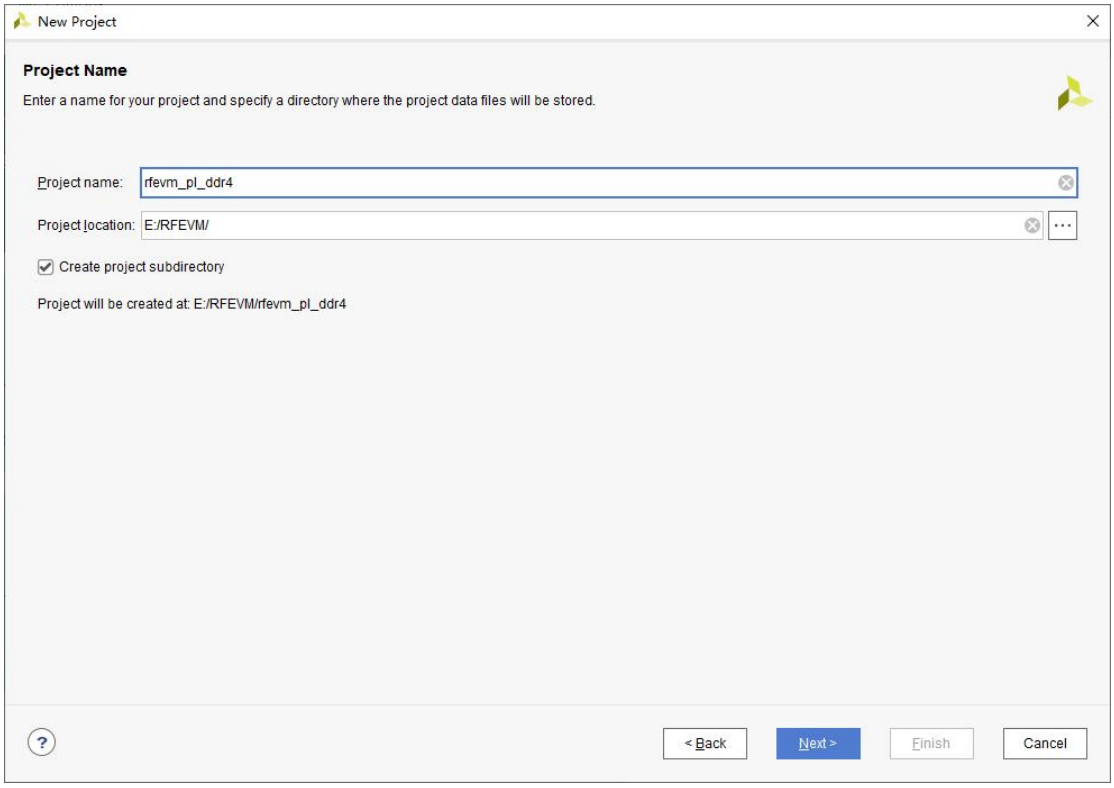
2) 选择 Create Project



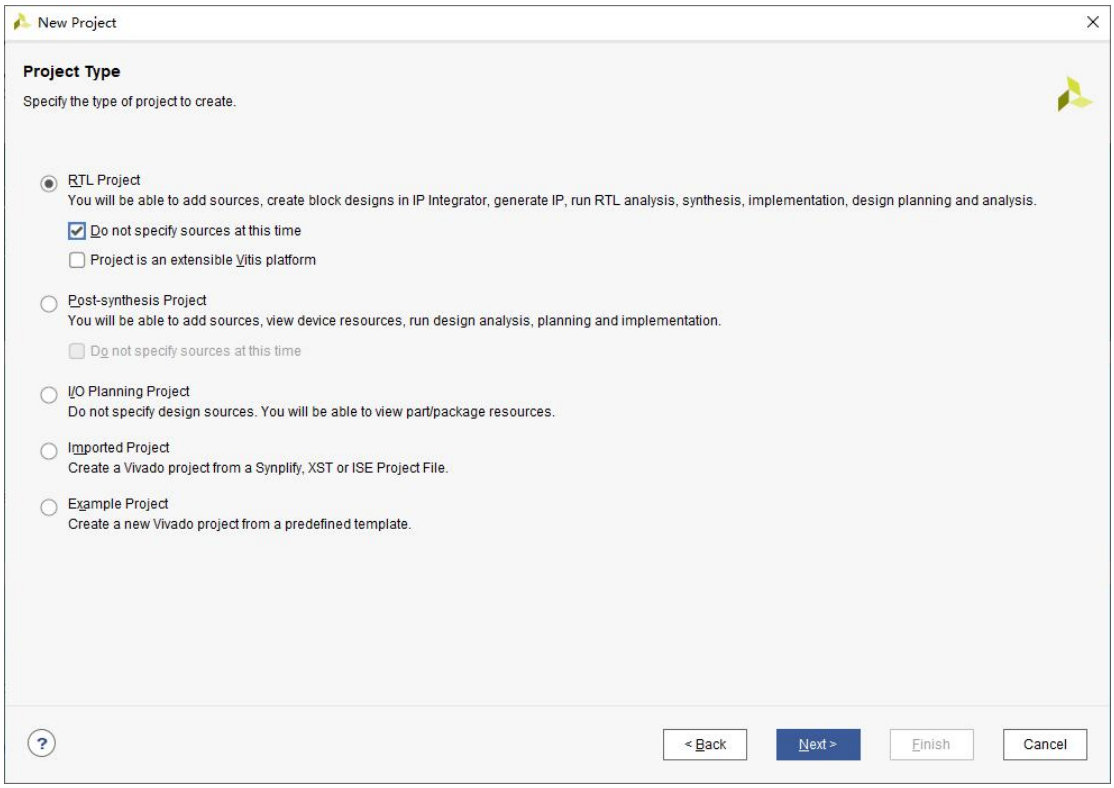
3) 点击 Next



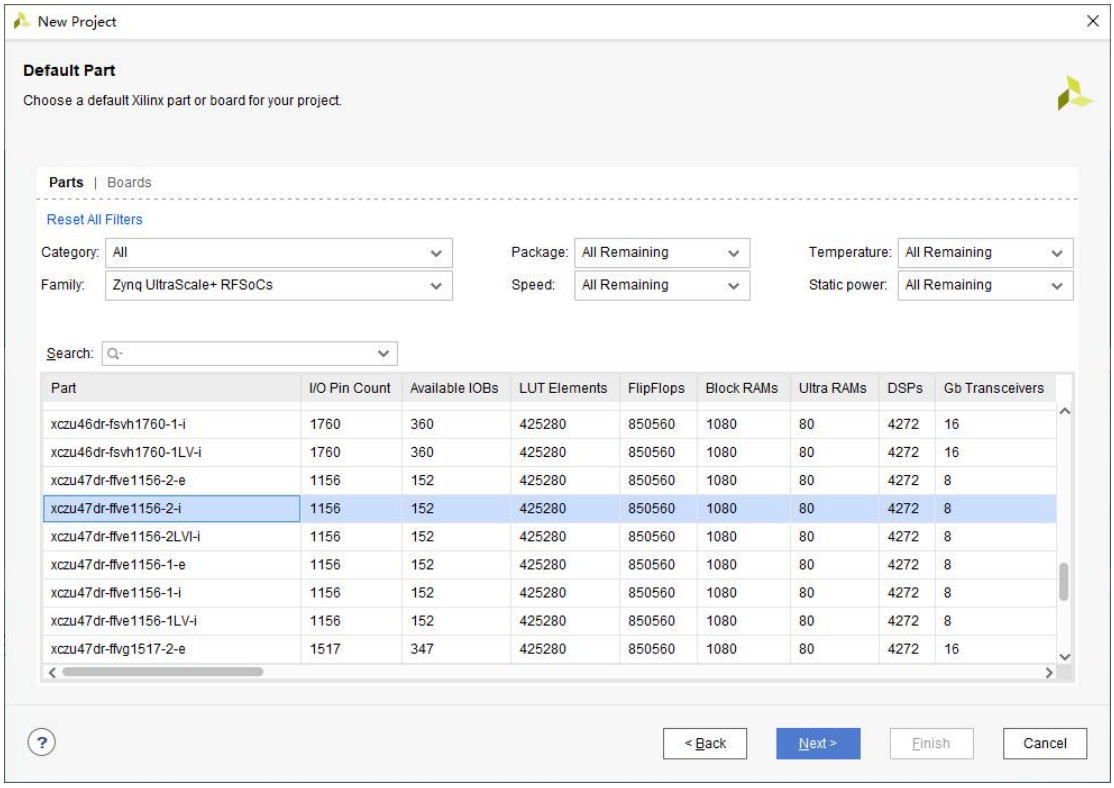
4) 设置工程名称（rfevm_pl_ddr4）和位置（D:/RFEVM/），点击 Next



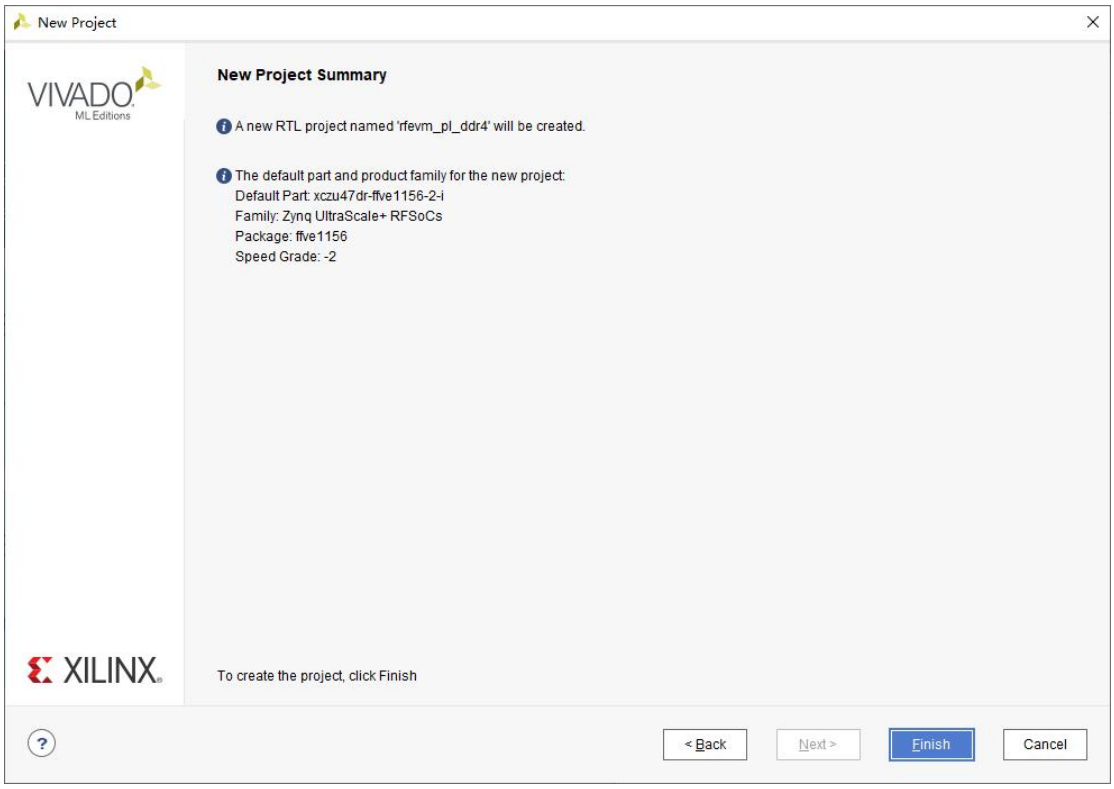
5) 选择 RTL 工程，点击 Next



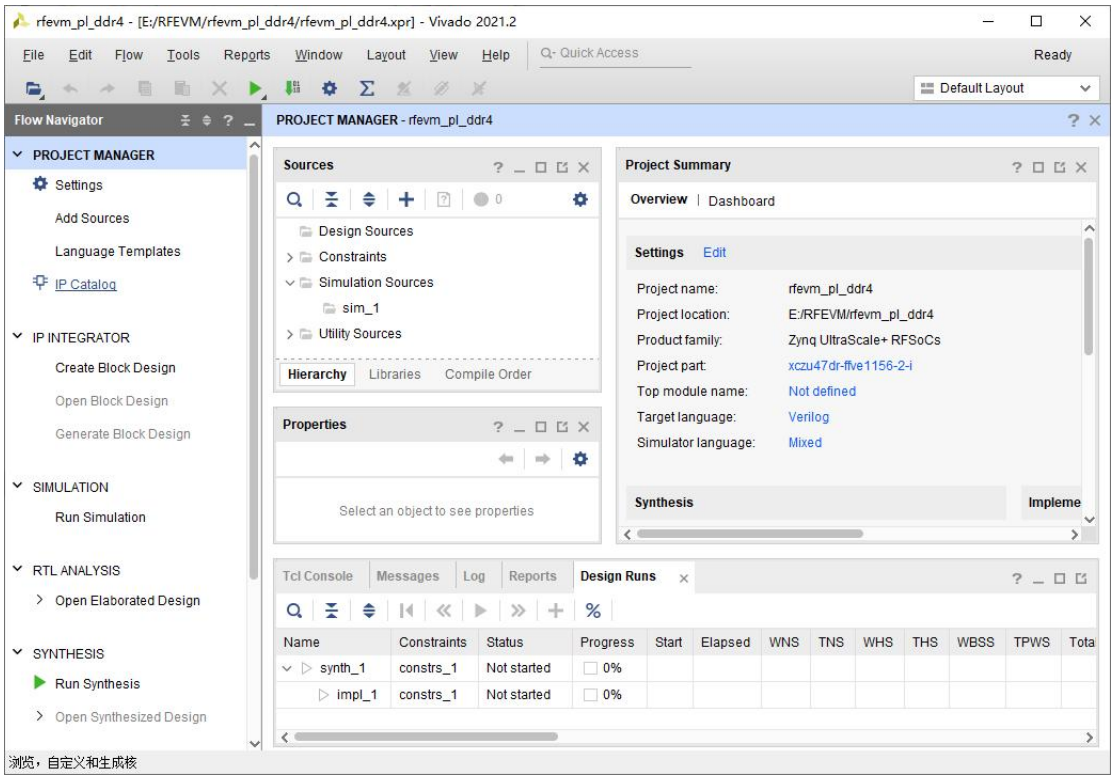
6) 选择芯片型号 xczu47dr-ffve1156-2-i，点击 Next



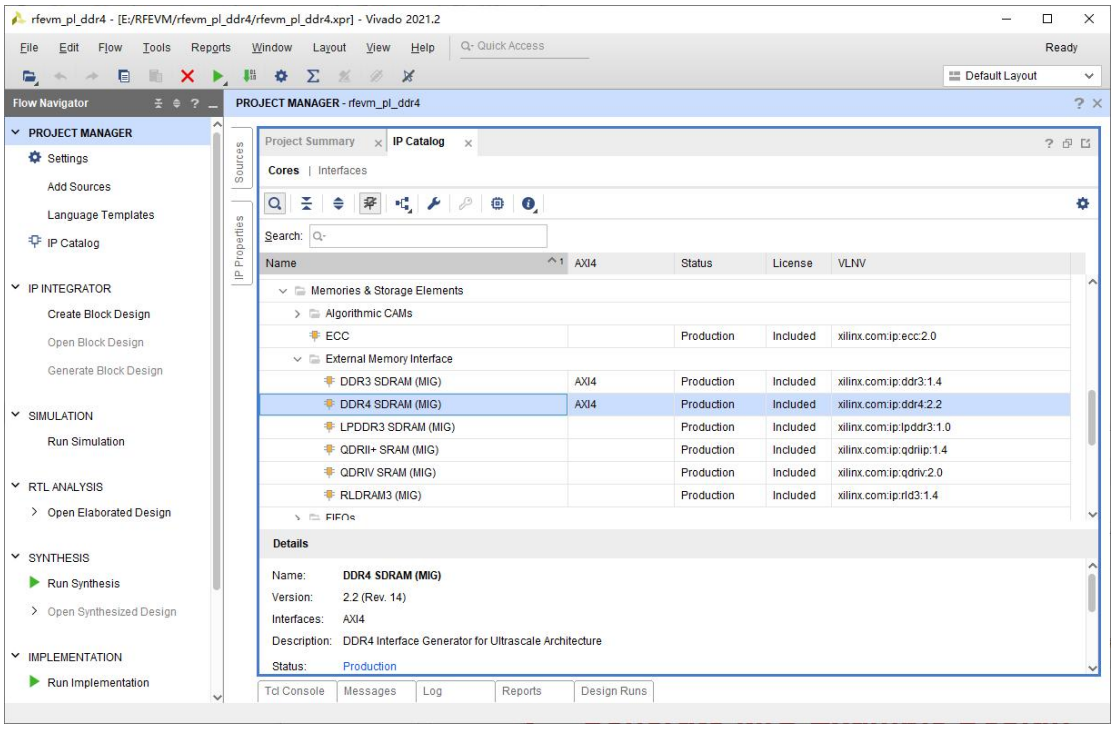
7) 点击 Finish



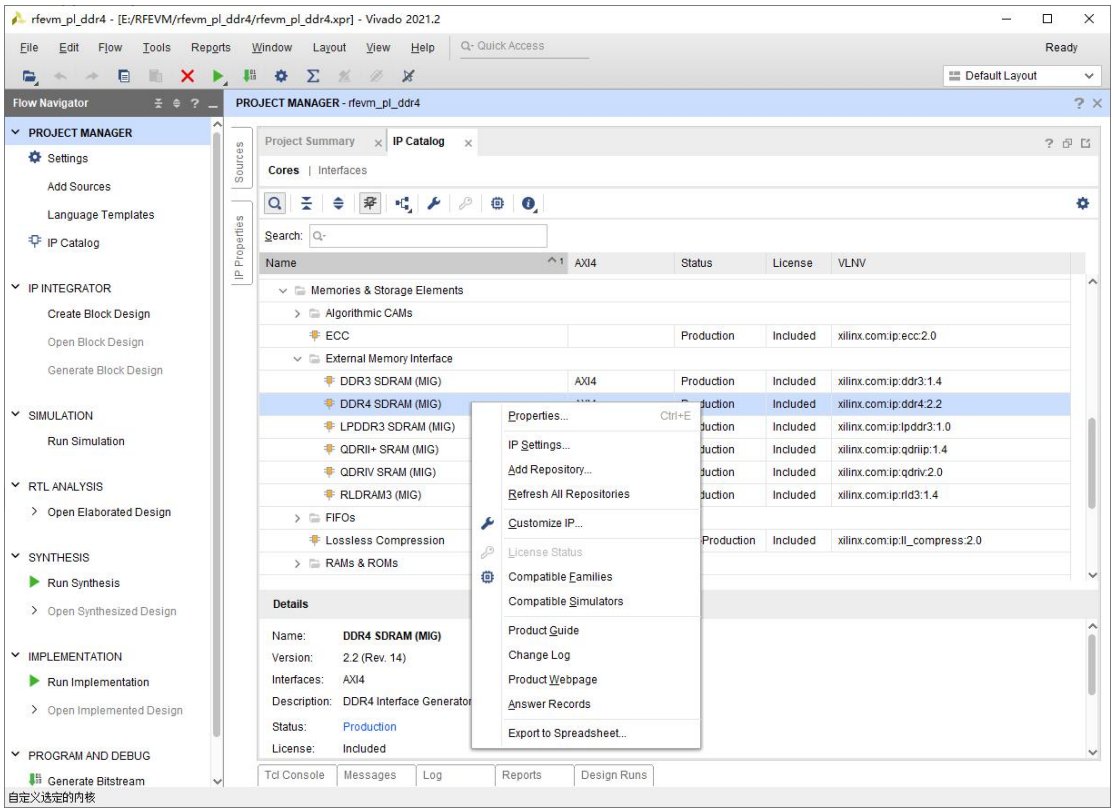
8) 在 Flow Navigator 窗口，点击 IP Catalog



9) 选择 DDR4 SDRAM(MIG),v2.2



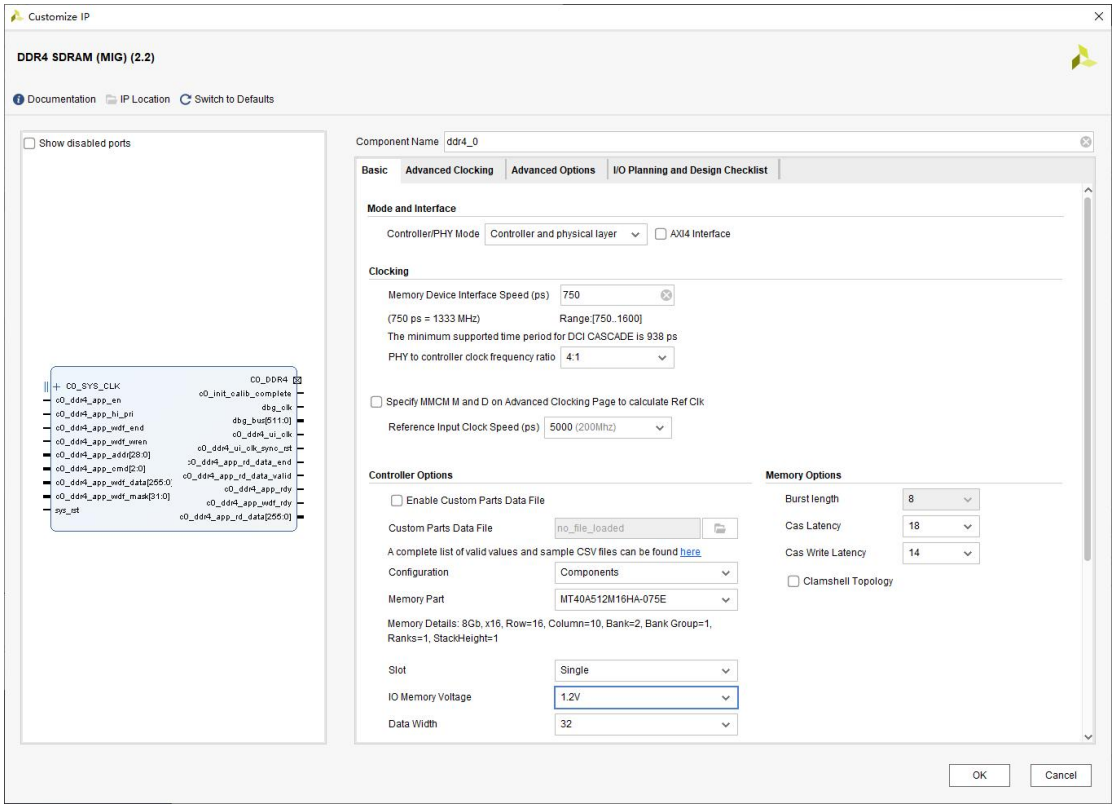
10) 右键 DDR4 SDRAM(MIG),选择 Customize IP



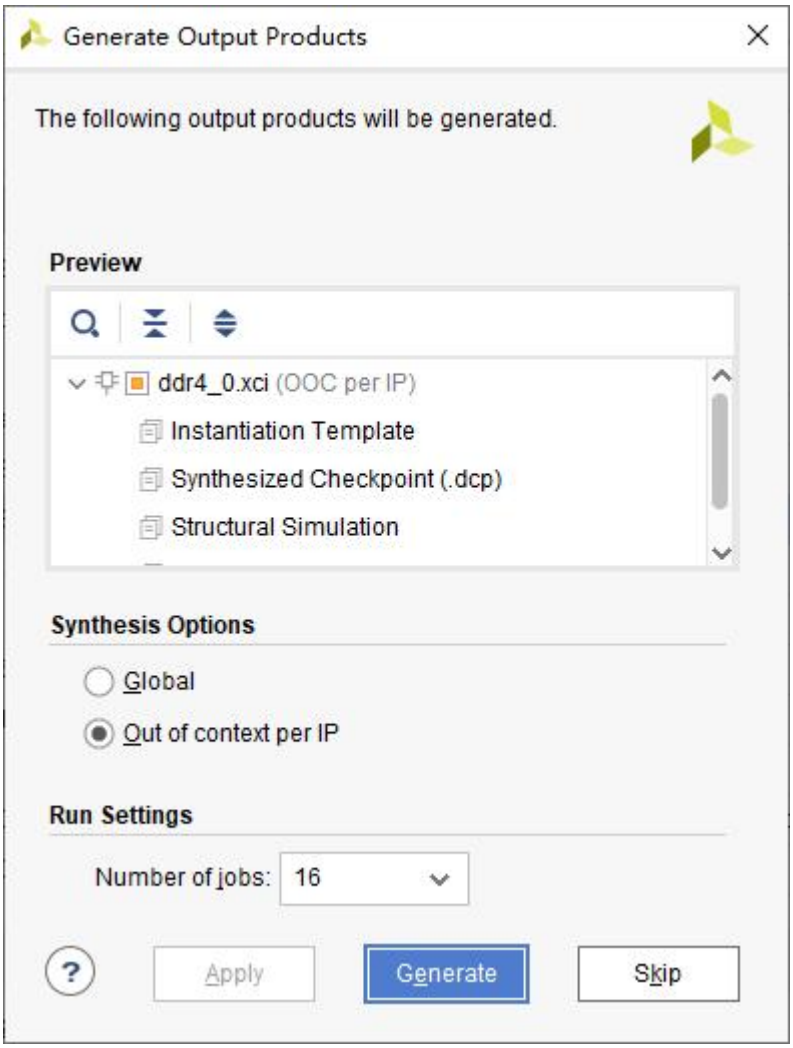
11) 在 Basic 选项卡下，进行如下设置

- Memory Device Interface Speed(ps):750
- Reference Input Clock Speed(ps):5000(200MHz)
- Memory Part:MT40A512M16HA-075E
- Data Width:32

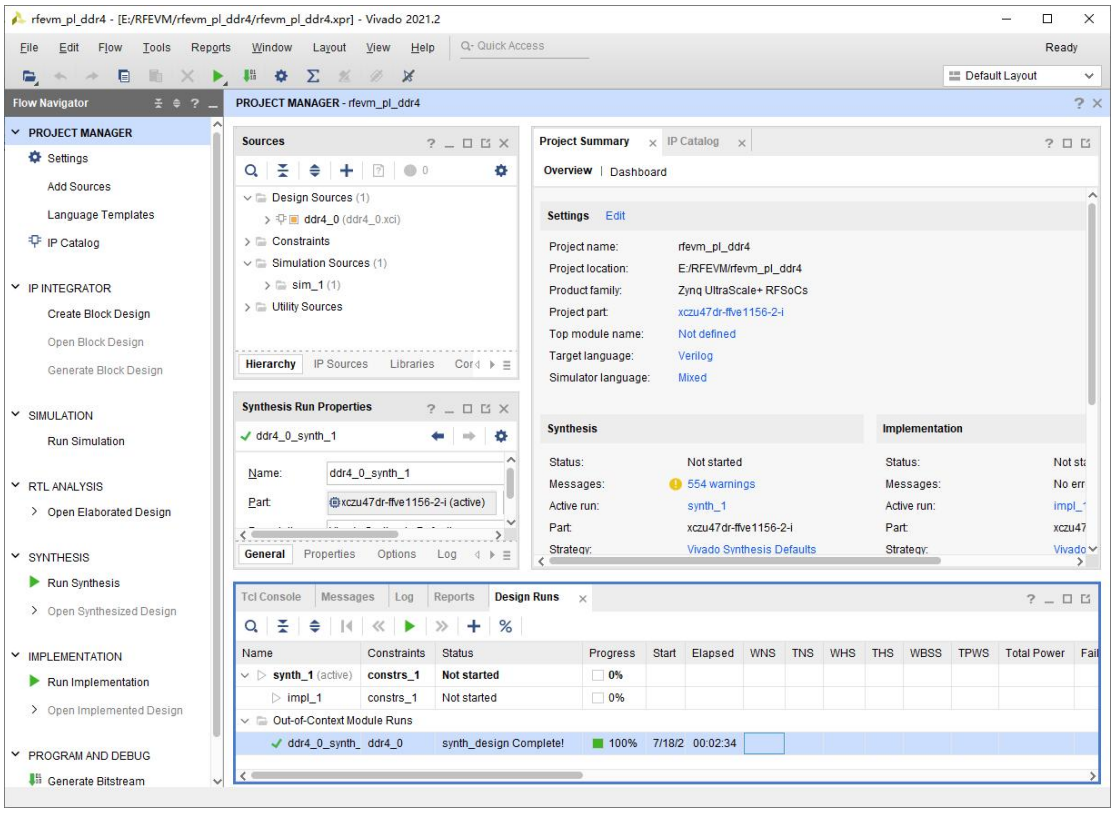
其它设置保持默认，点击 OK



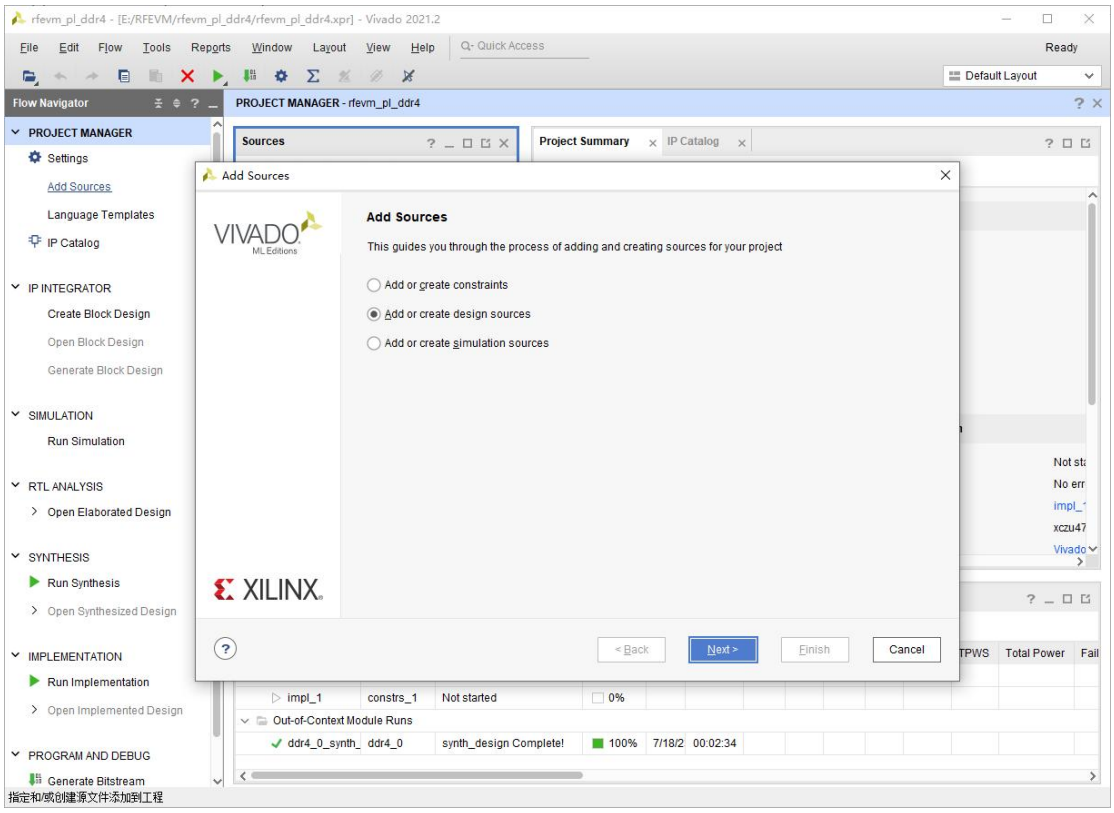
12) 点击 Generate



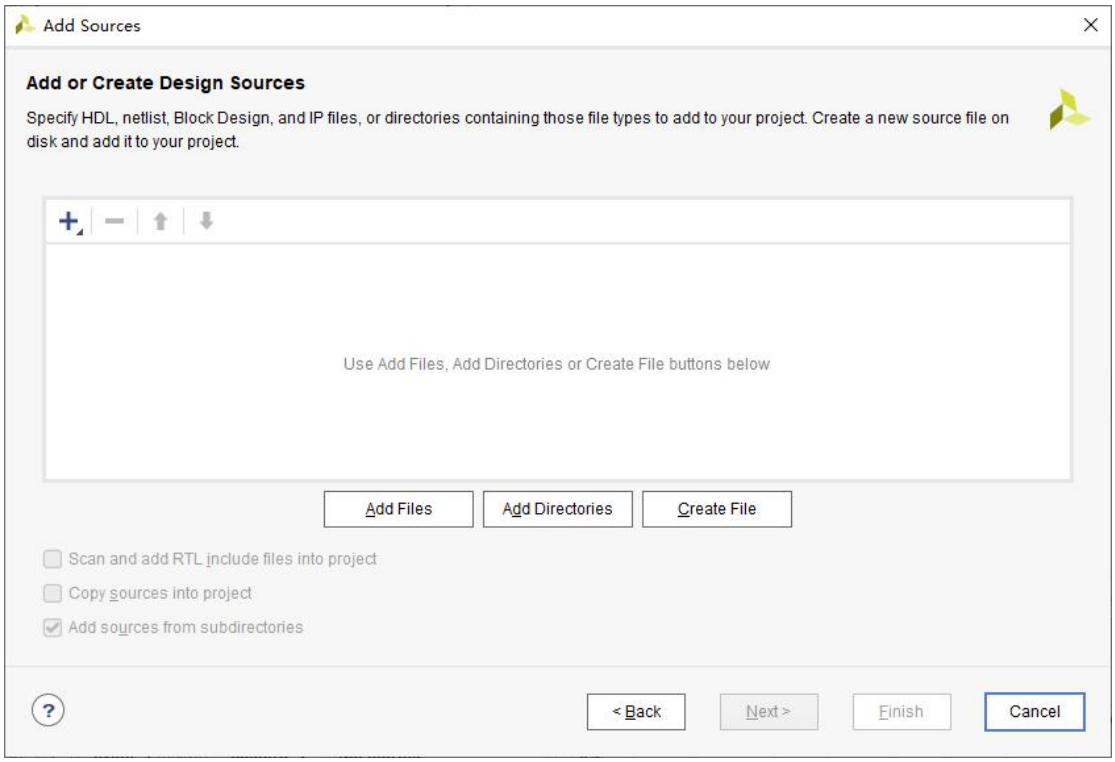
13) 等待直到 ddr4_0_synth_1 出现复选标记



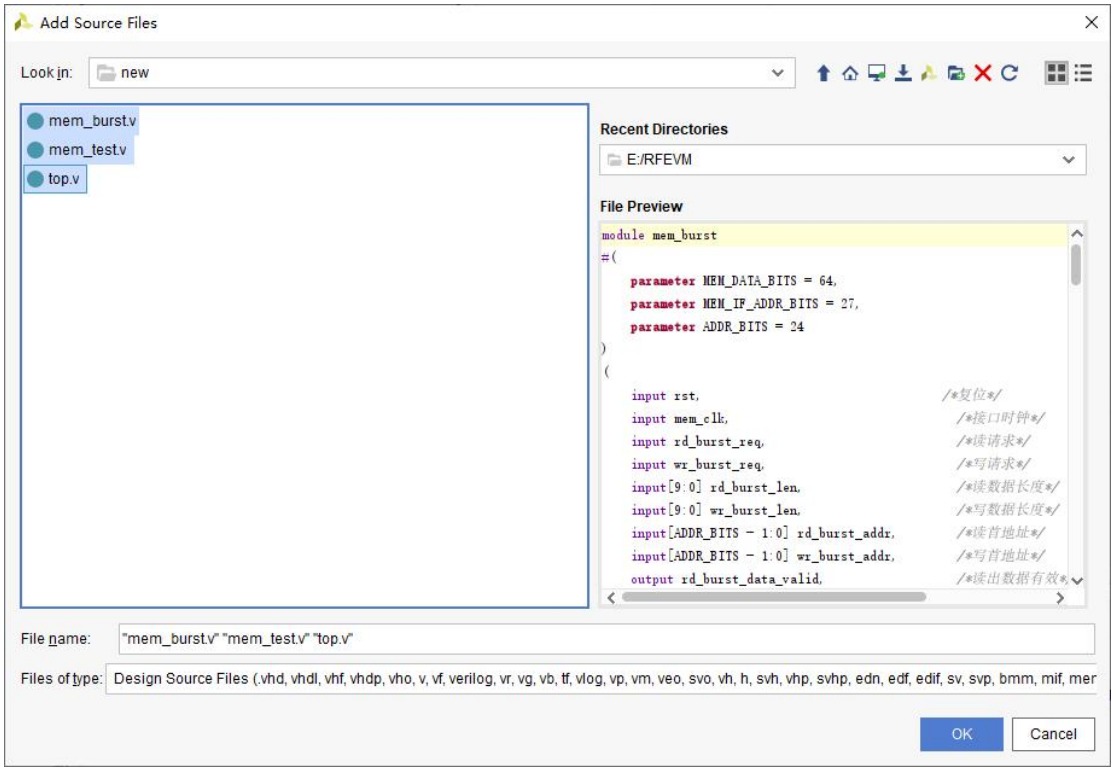
14) 添加源代码 Add Sources，选择 Add or create design sources，点击 Next



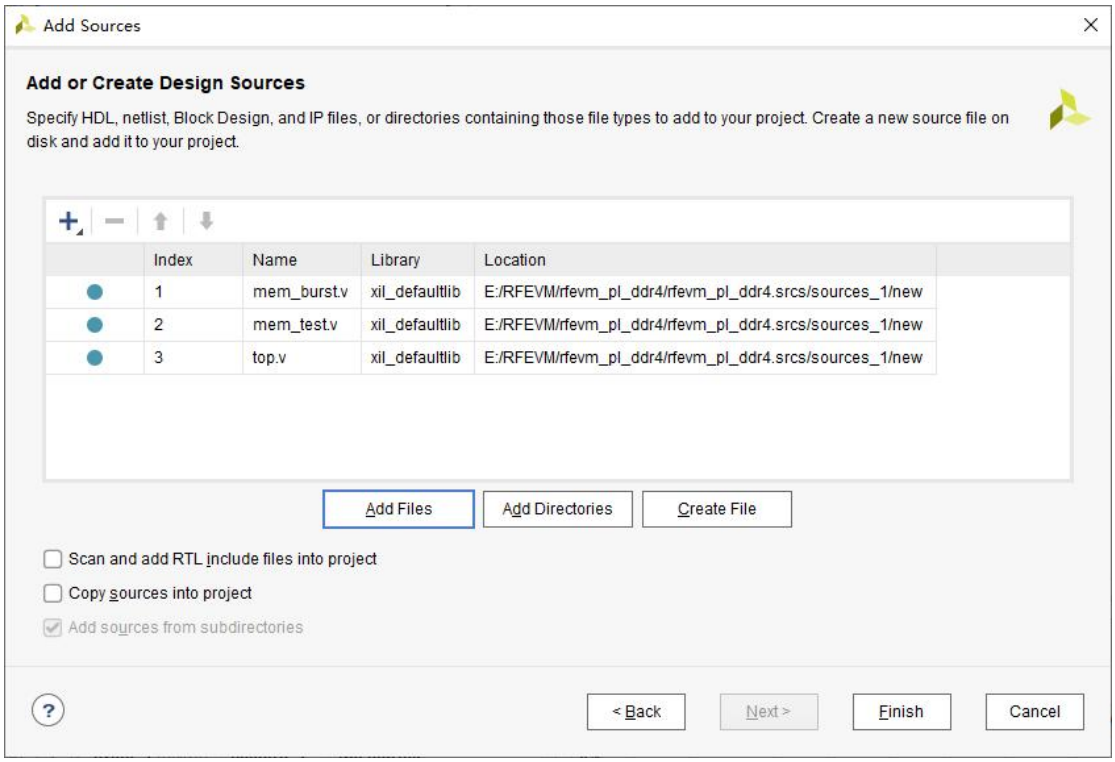
15) 点击 Add Files

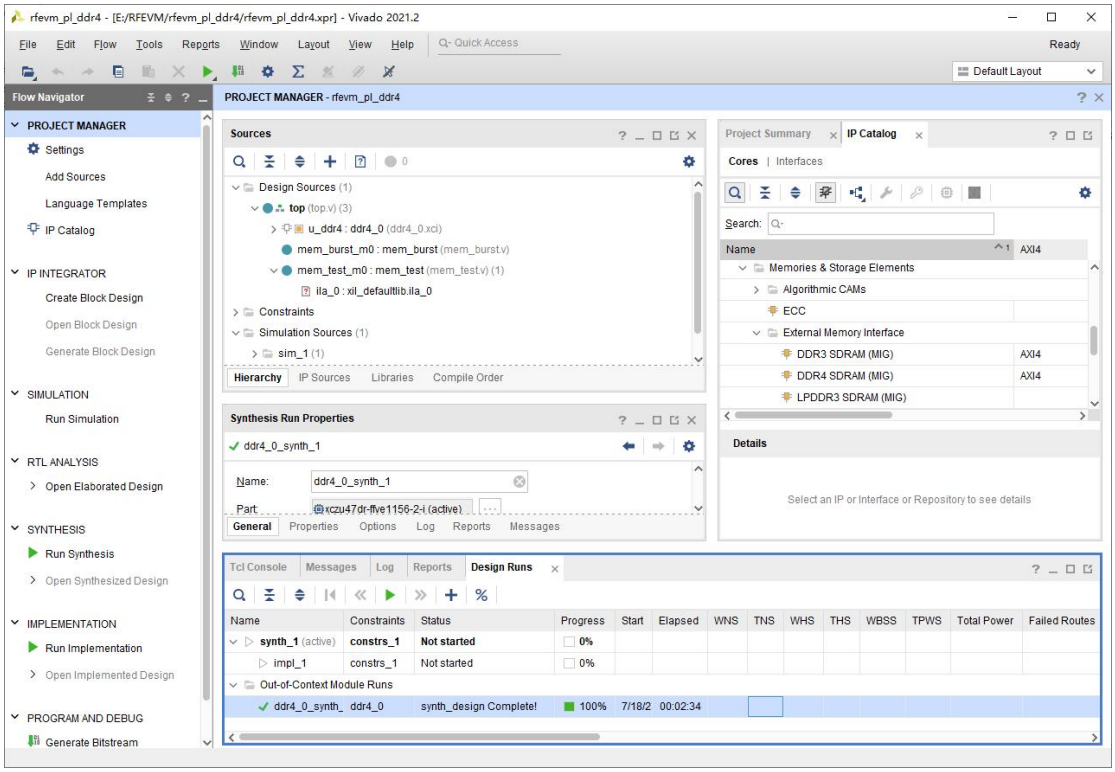


16) 选择测试文件，点击 OK

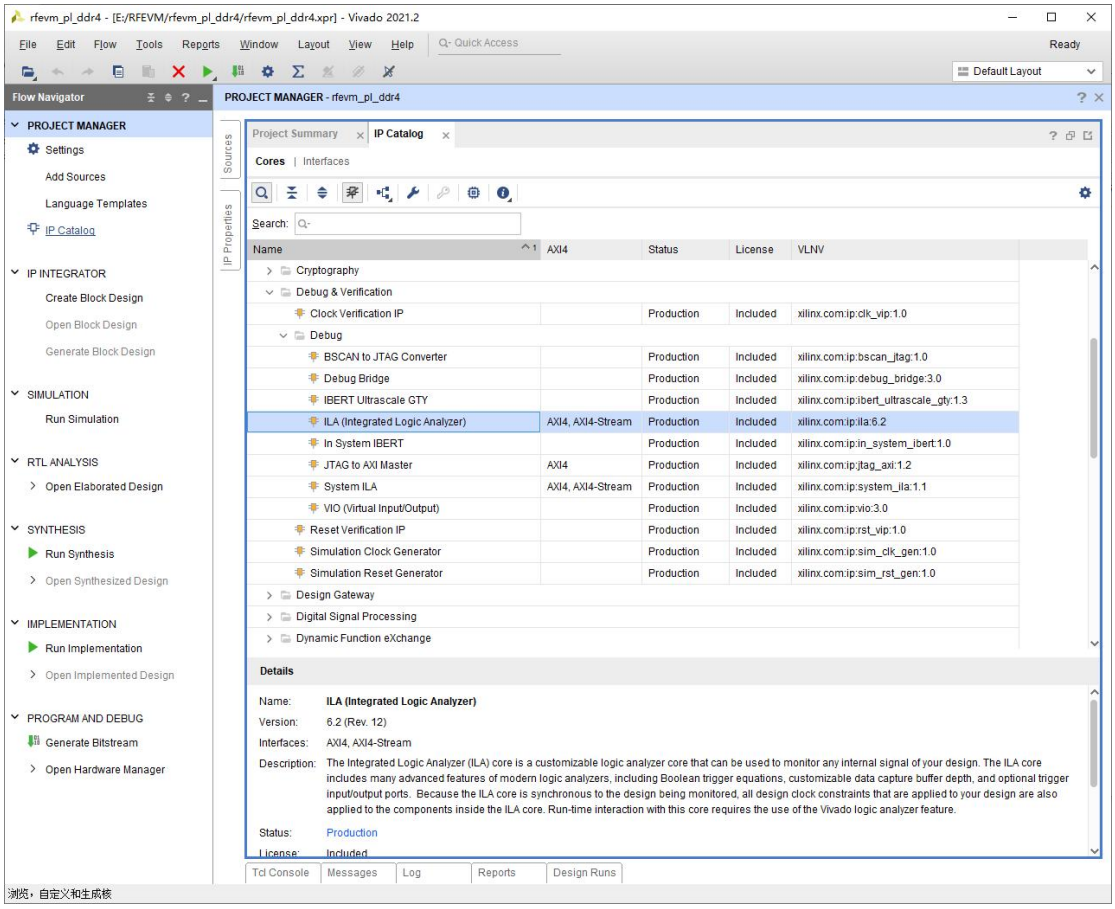


17) 点击 Finish

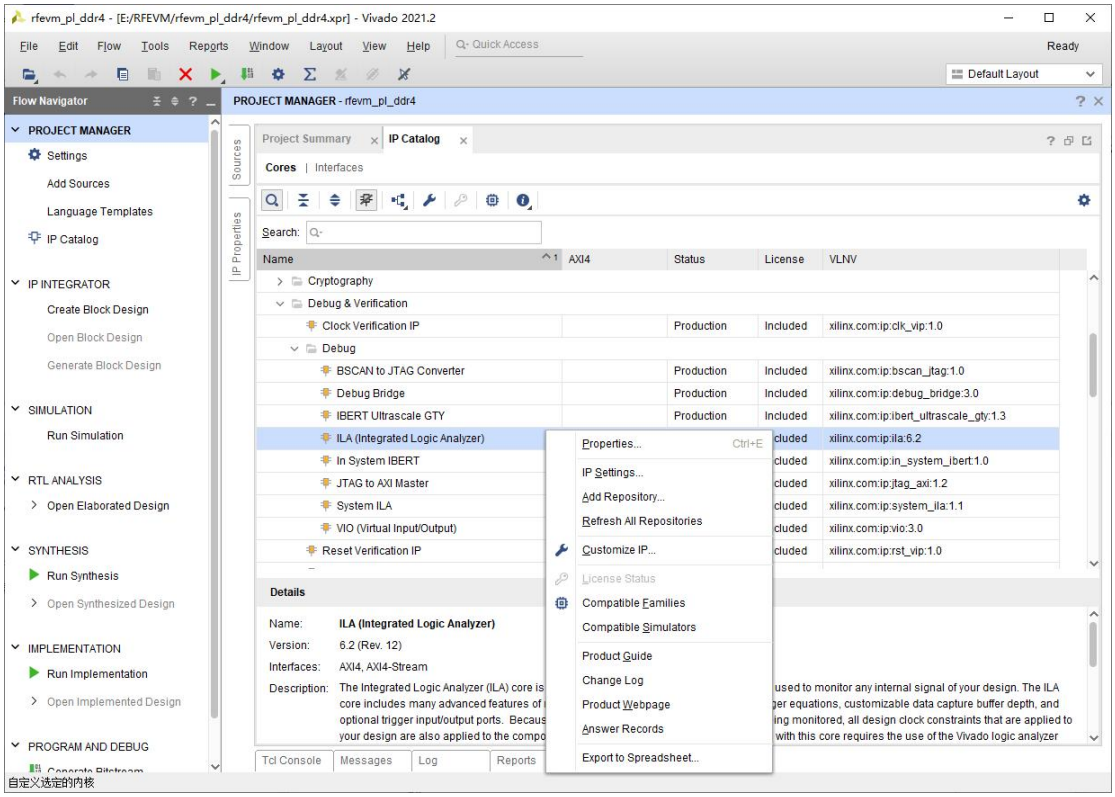




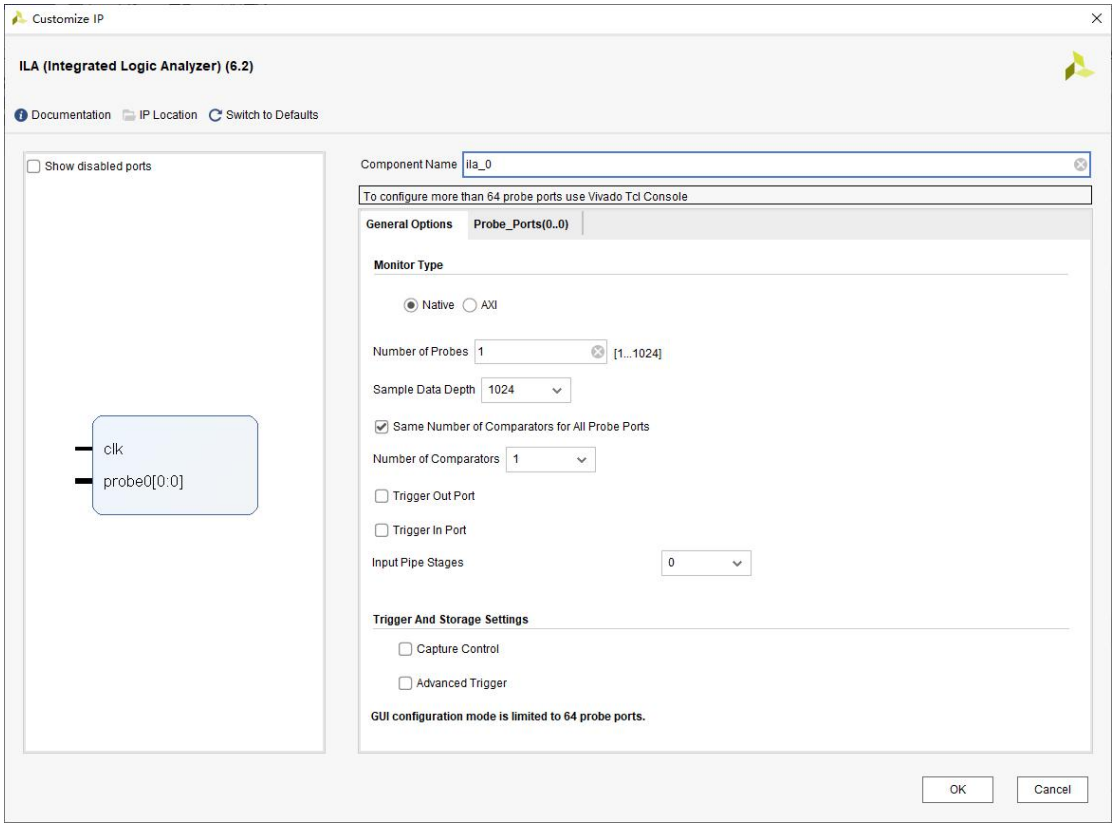
18) 点击 IP Catalog

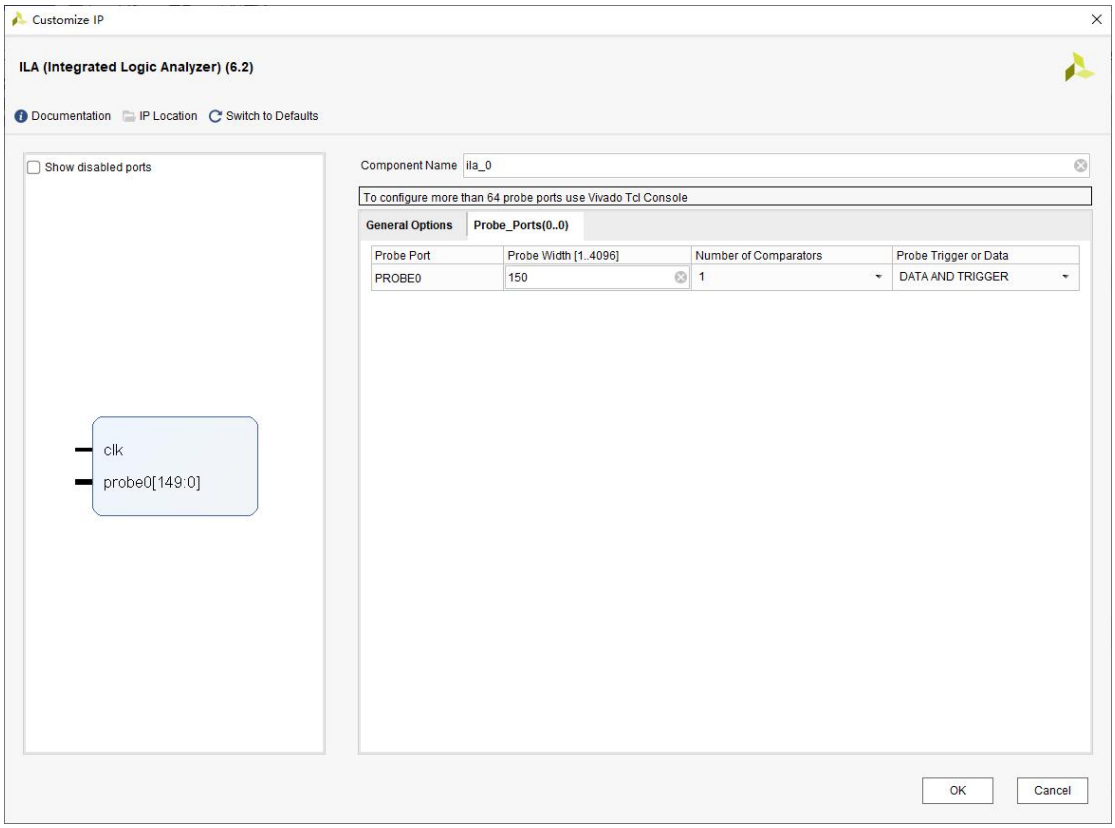


19) 在 ILA(Integrated Logic Analyzer)上右键选择 Customize IP

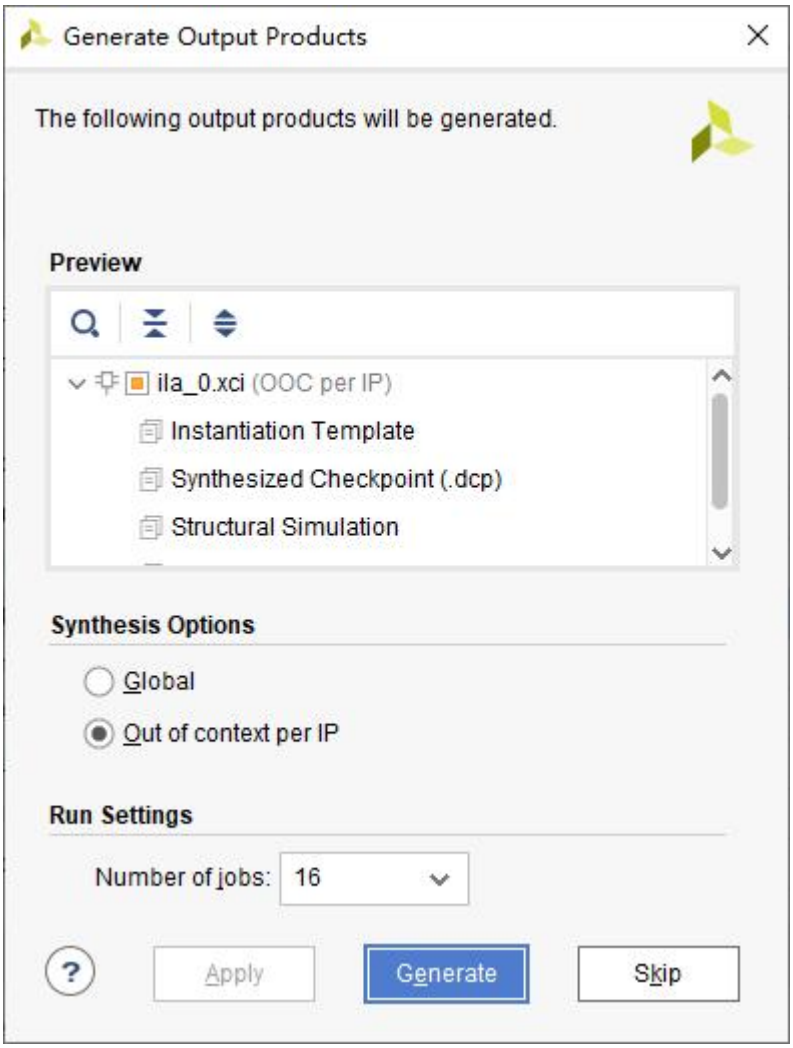


20) 按照如下配置设置 IP 参数后，点击 OK

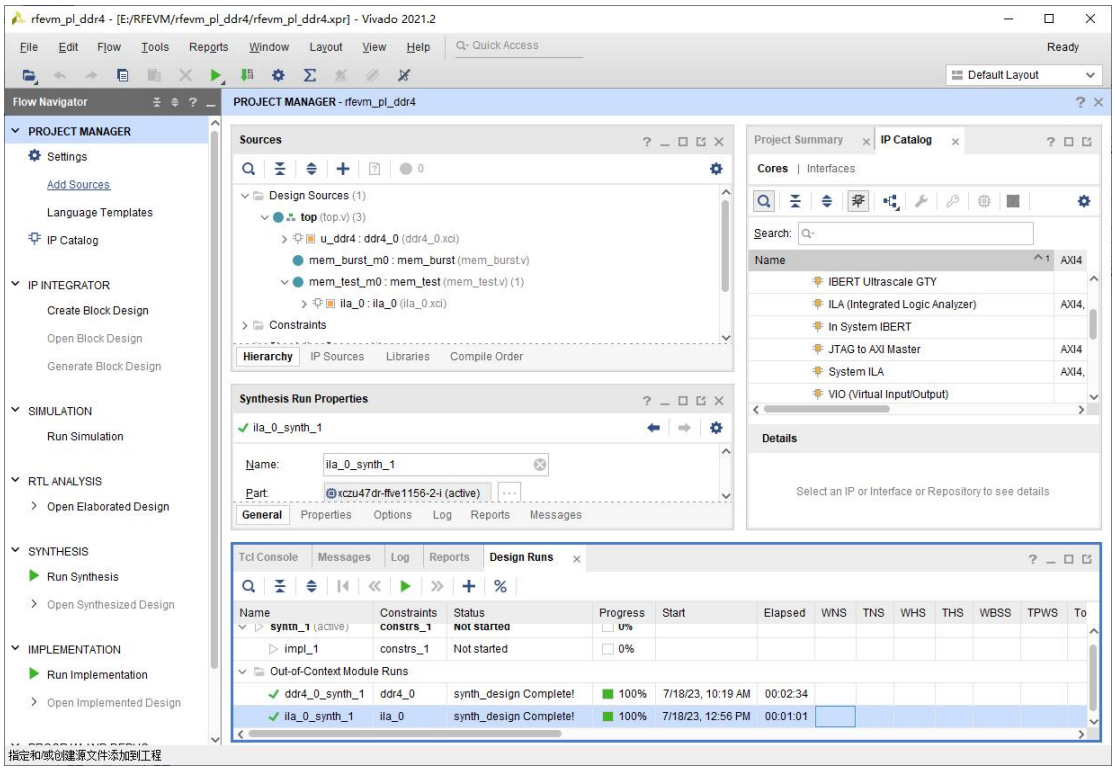




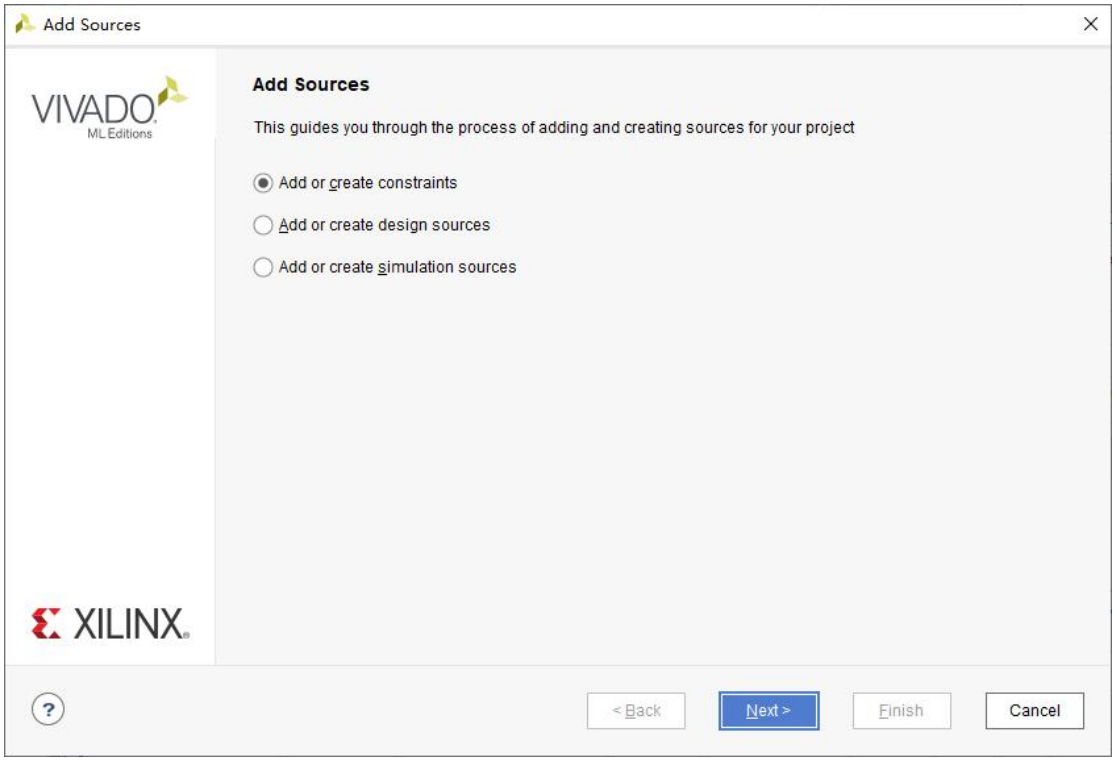
21) 点击 Generate



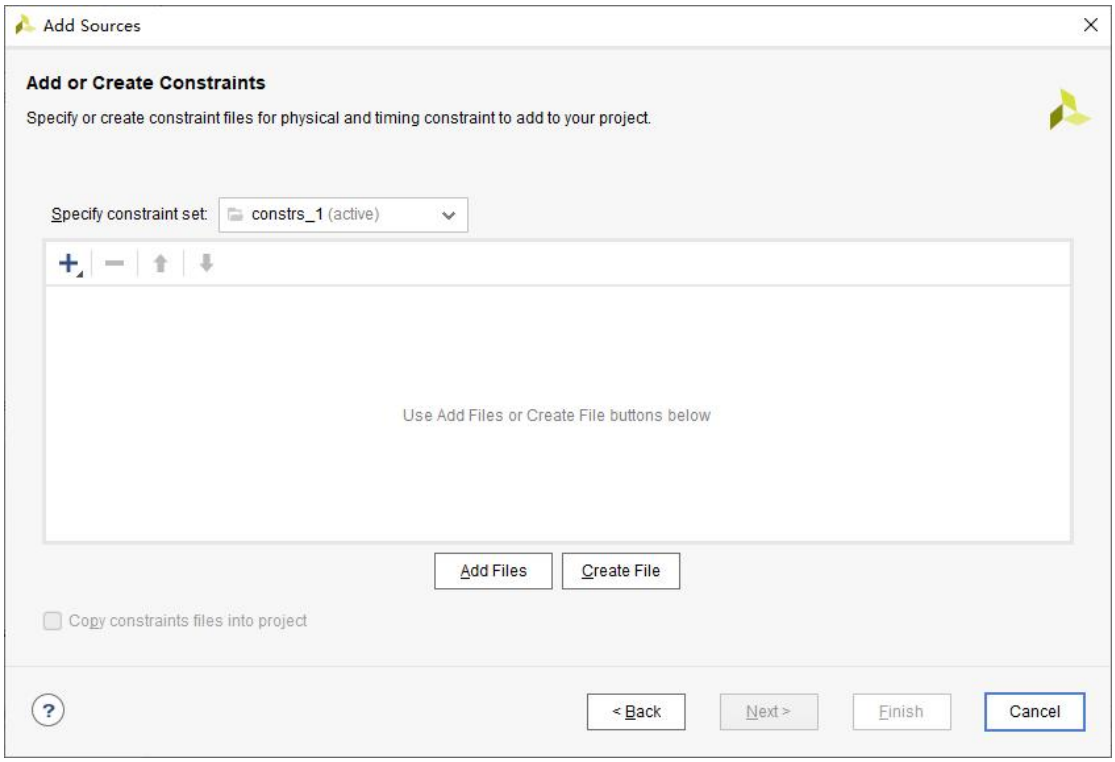
22) 点击 Add Sources



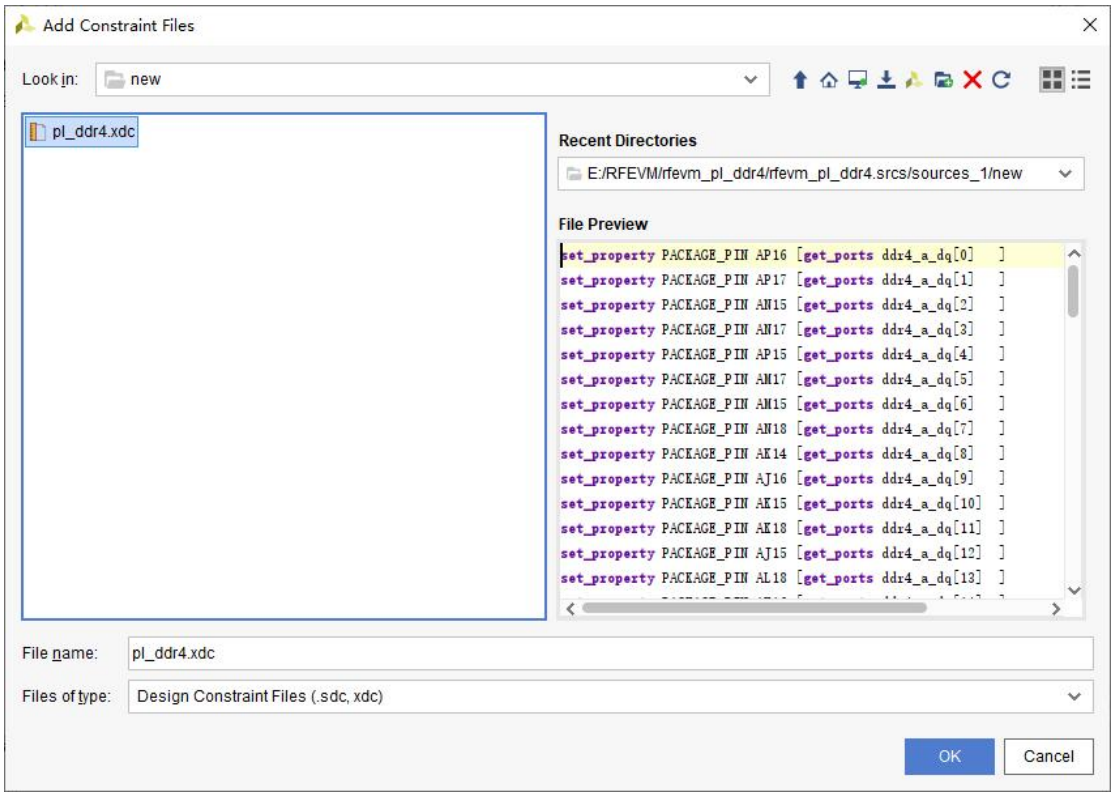
23) 选择 Add or create constraints 添加约束文件，点击 Next



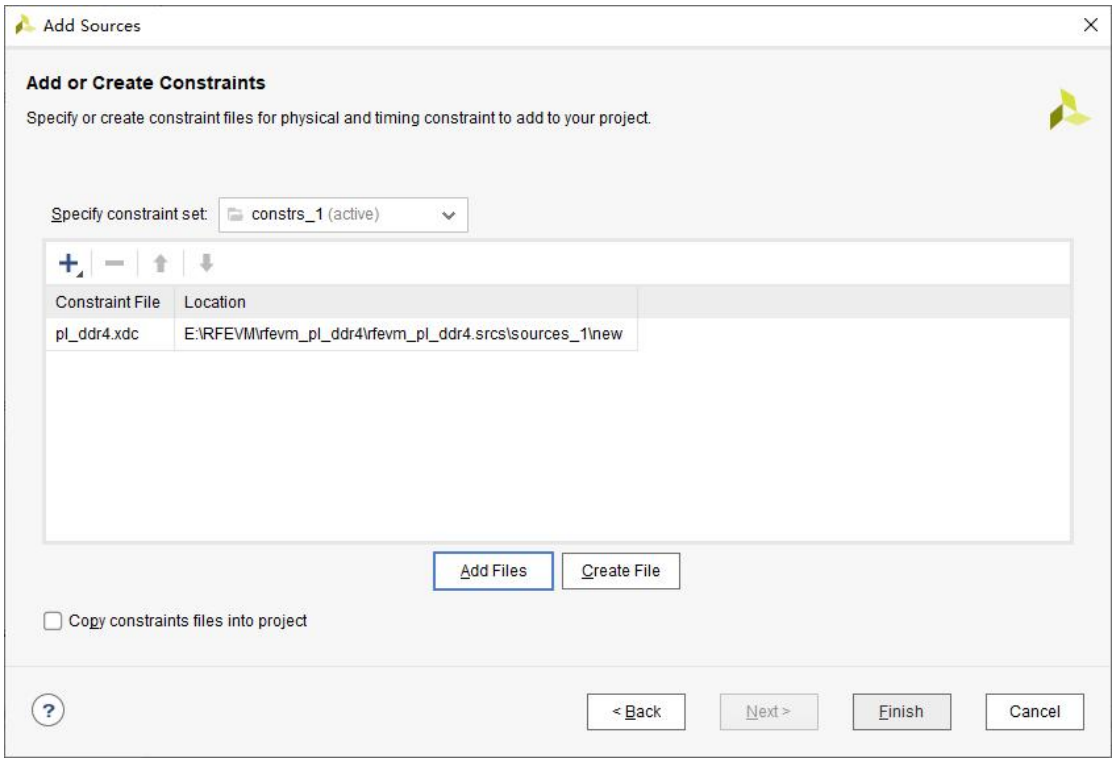
24) 点击 Add Files

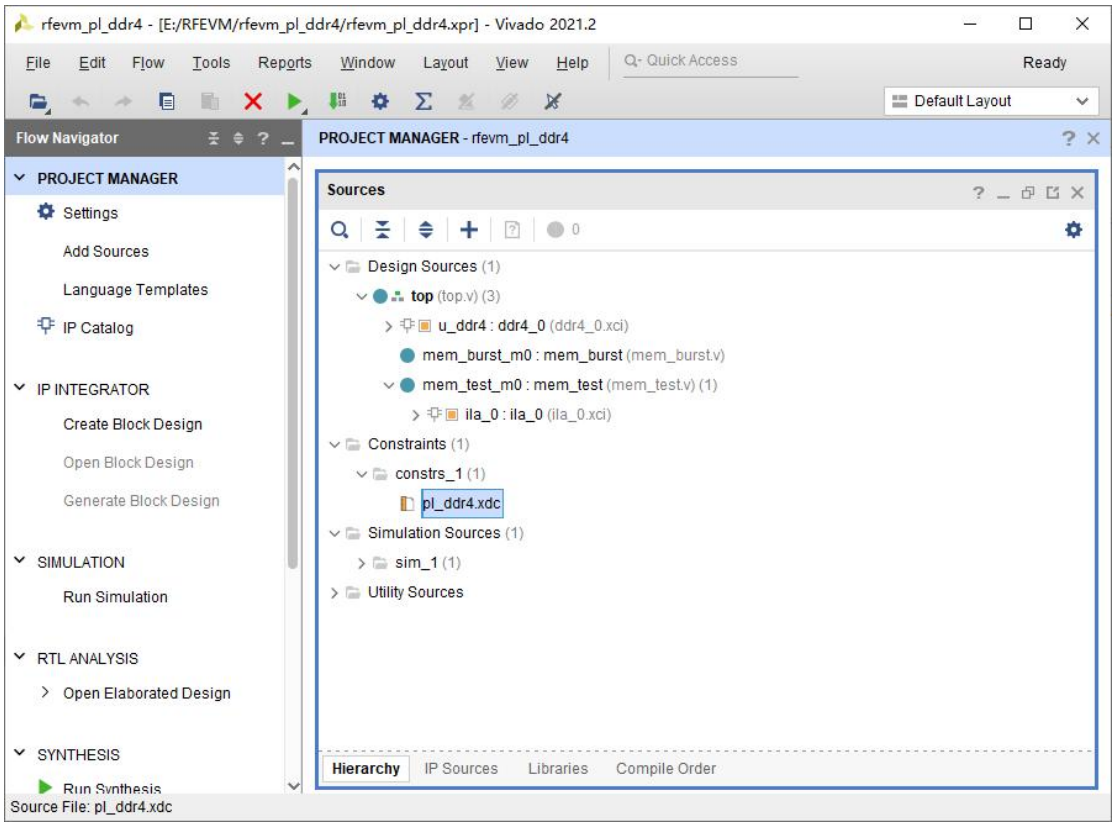


25) 选择 DDR4 的约束文件，点击 OK

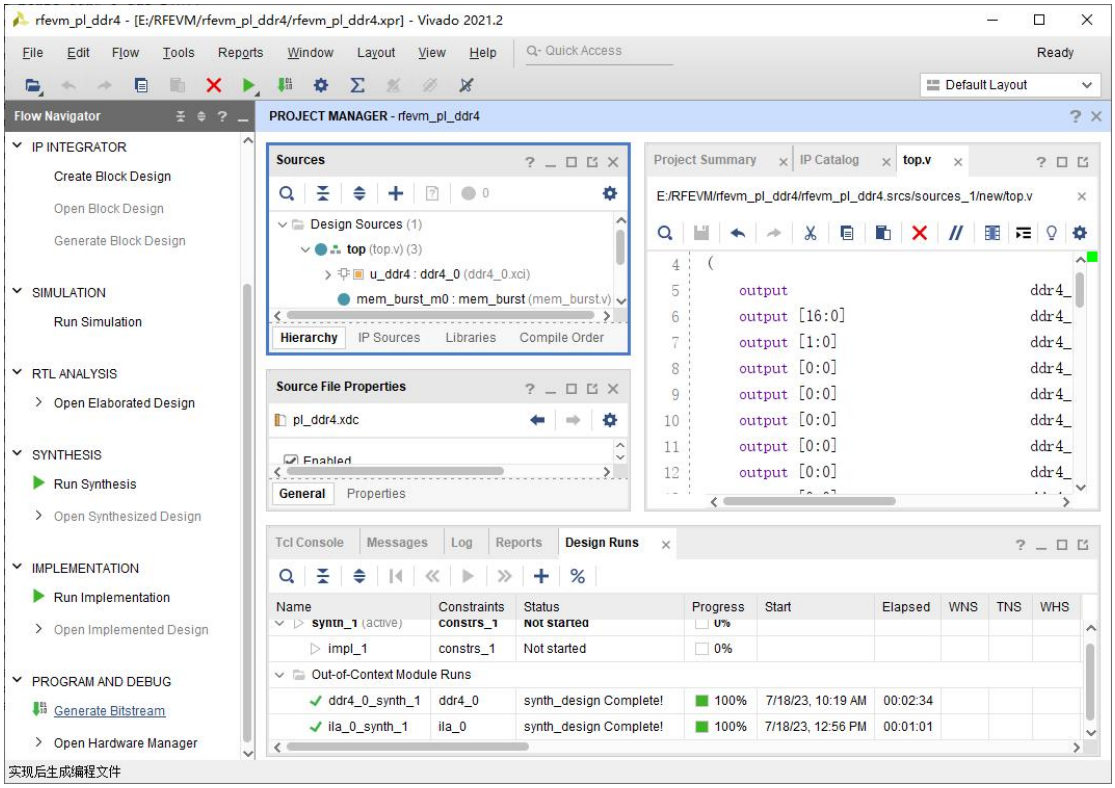


26) 点击 Finish

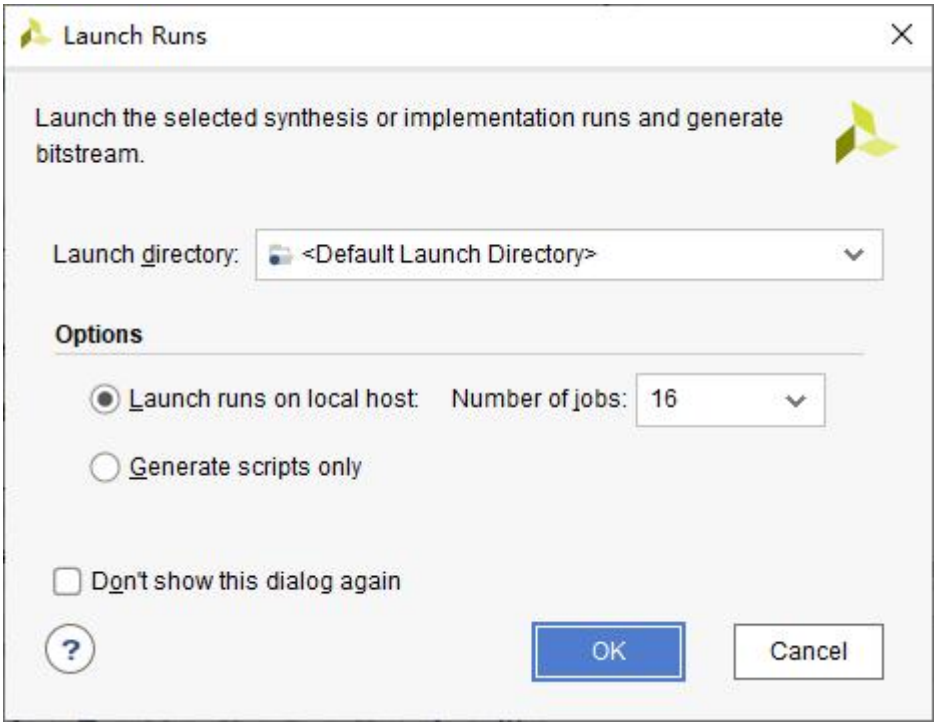




27) 点击 Generate Bitstream，生成 bit

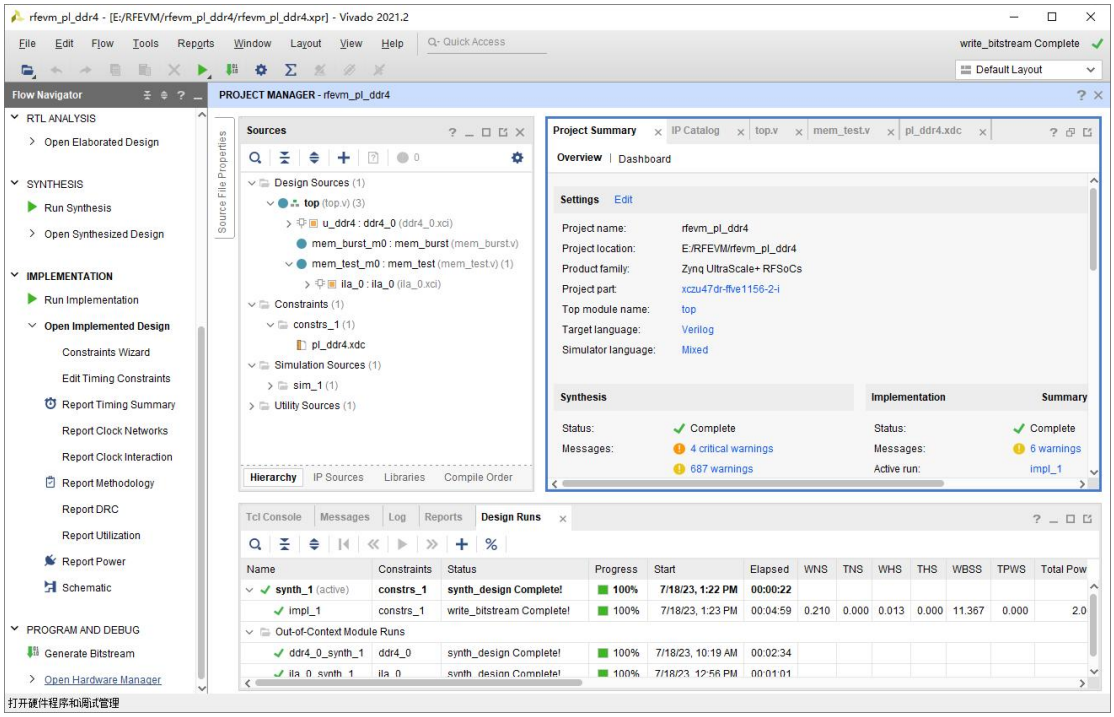


28) 点击 OK

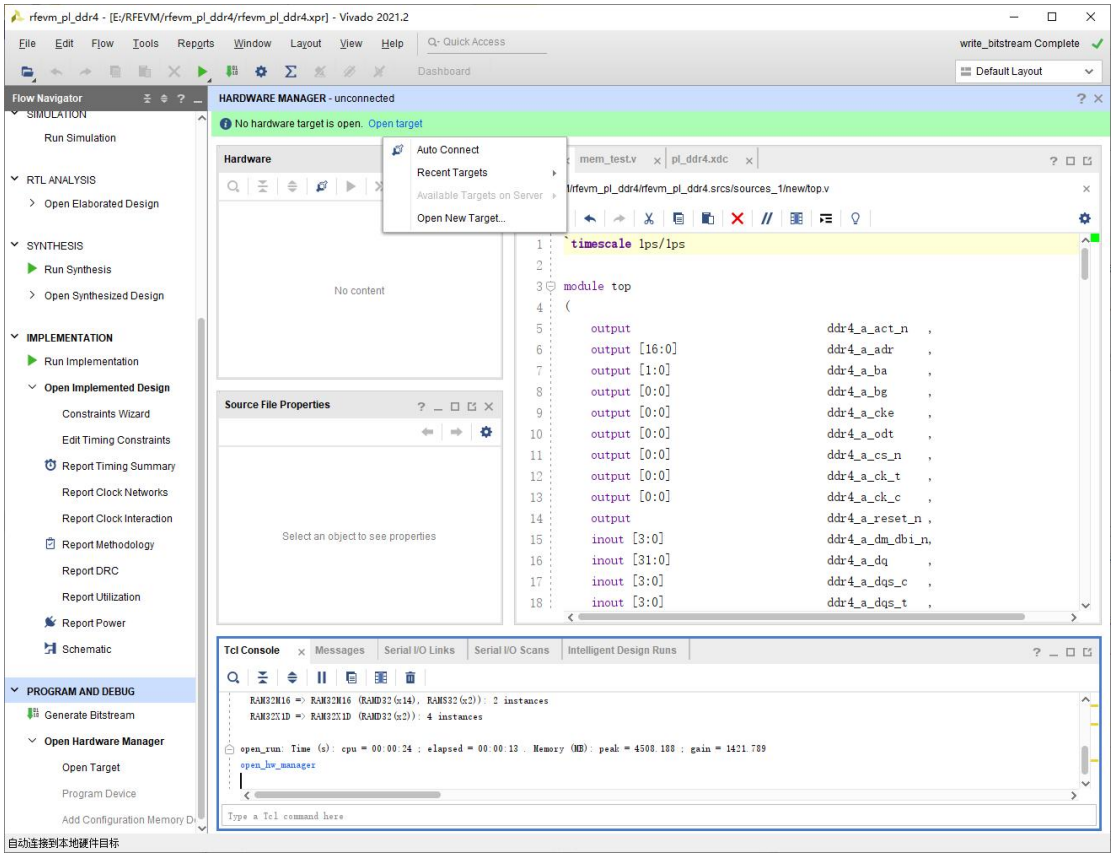


3.2.3 下载验证

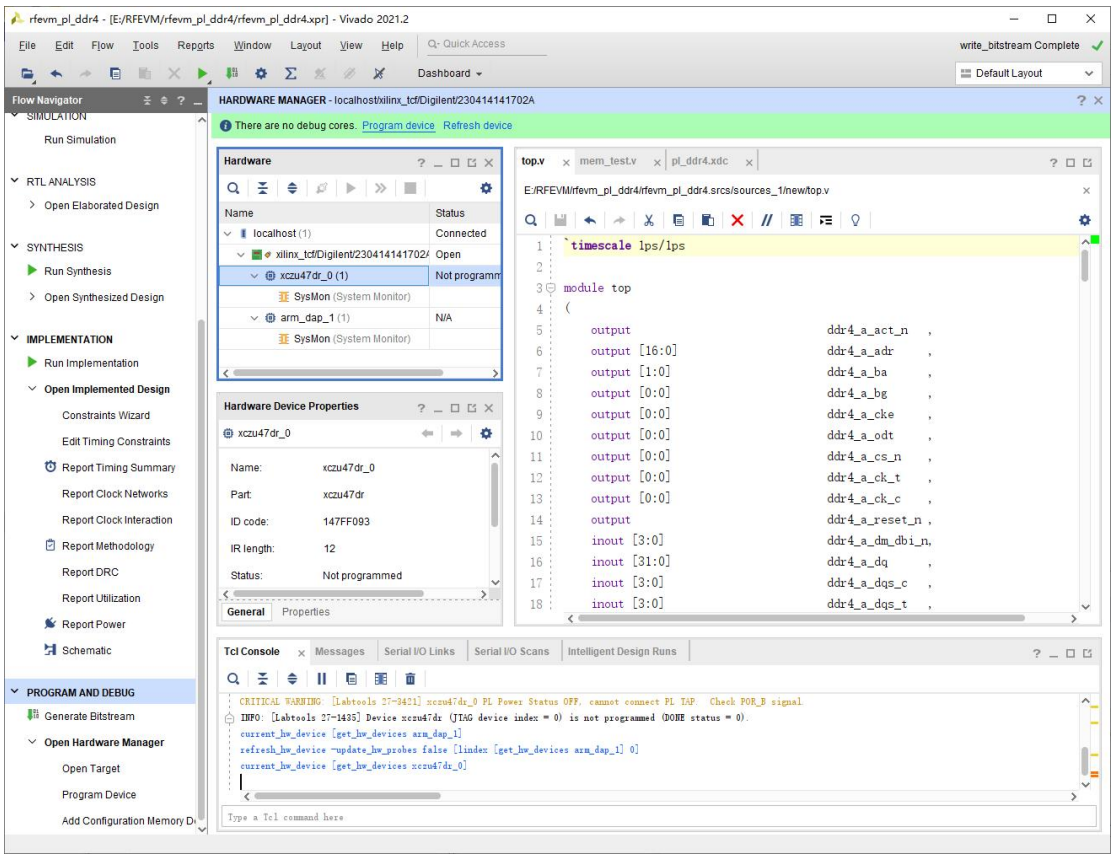
1) 点击 Open Hardware Manager



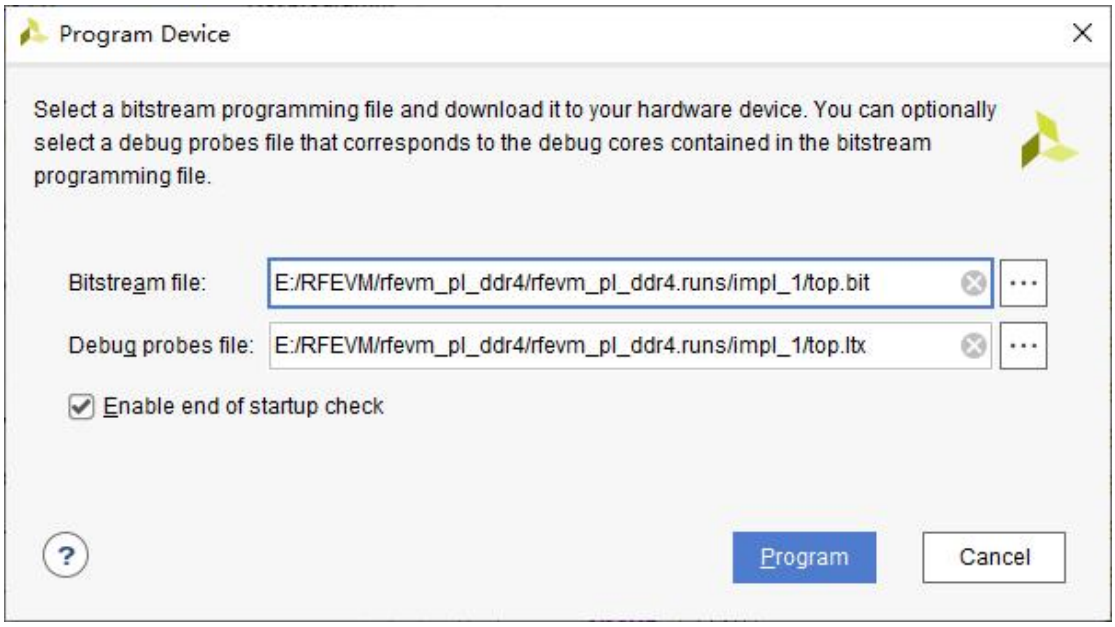
2) 点击 Open target，选择 Auto Connect



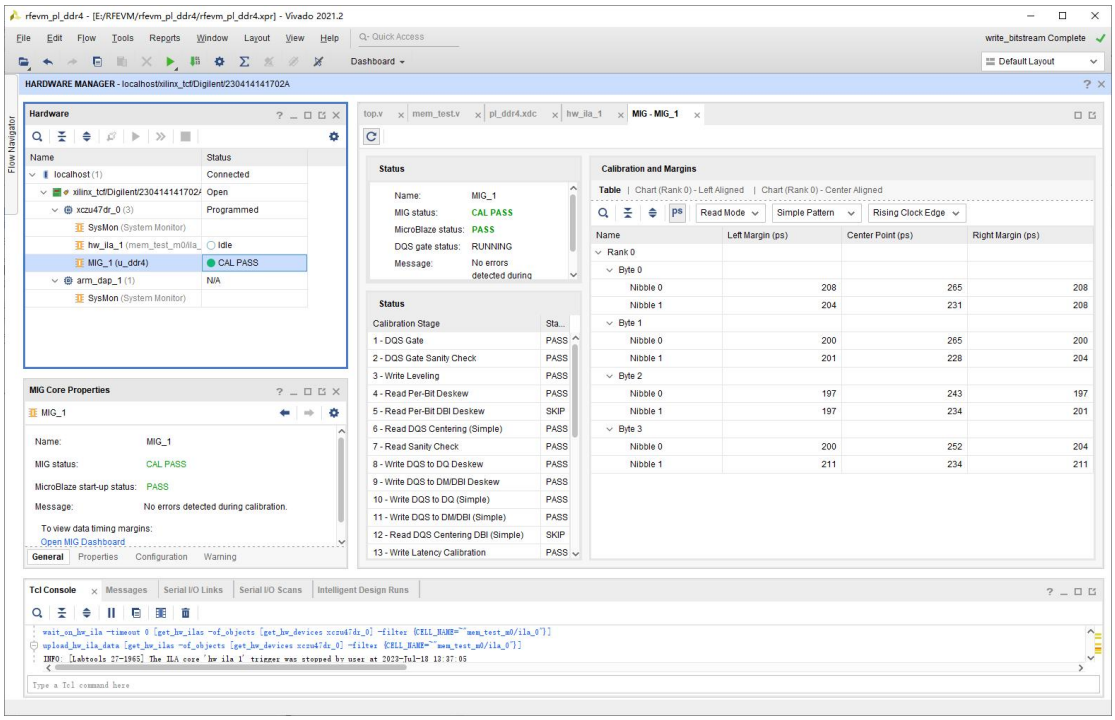
3) 点击 Program device



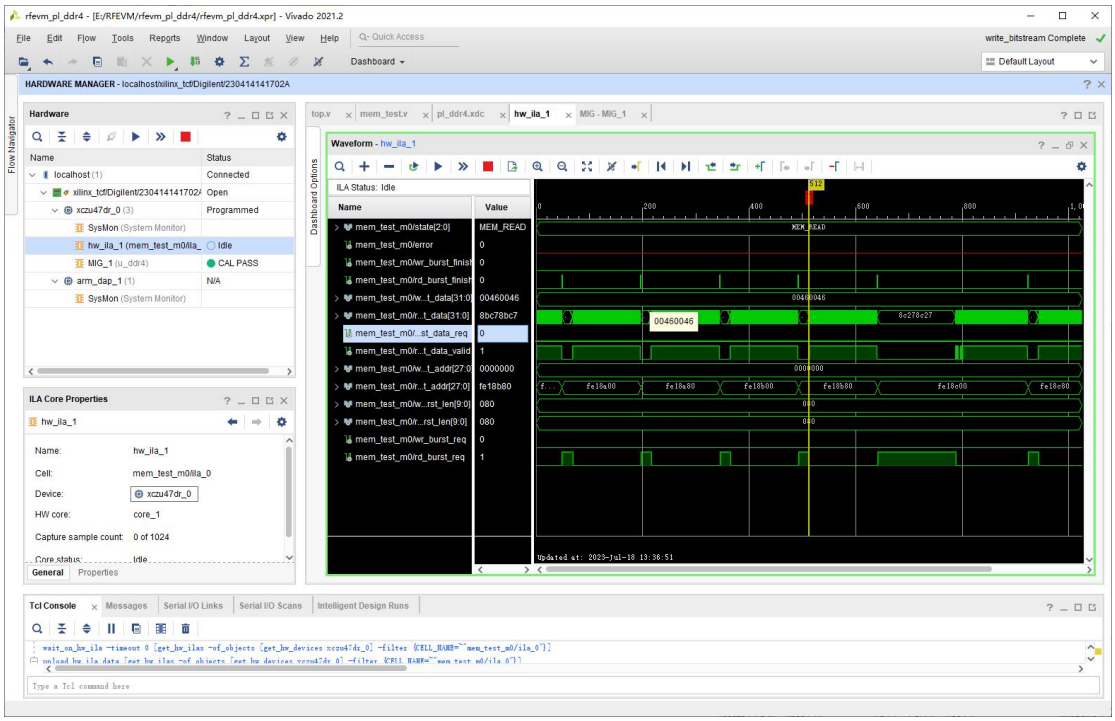
4) 默认即可，点击 Program



5) 下载完成后，会在 MIG_1 窗口会显示 DDR4 校准的信息和校准状态（CAL PASS）



在 hw_ila_0 窗口中可以查看调试信号



3.2.3 参考文档

- Zynq UltraScale Memory

  UltraScale FPGA Memory Interface Solutions Product Guide - PG150

  https://docs.xilinx.com/v/u/en-US/pg150-ultrascale-memory-ip

- Vivado Programming and Debugging

  Vivado Design Suit Programming and Debugging User Guide - UG908

  https://docs.xilinx.com/r/2021.2-English/ug908-vivado-programming-debugging/Introduction

- ILA

  Integrated Logic Analyzer Product Guide - PG172

  https://docs.xilinx.com/v/u/en-US/pg172-ila

- DDR4-MT40A512M16

  https://pdf1.alldatasheet.com/datasheet-pdf/view/928163/MICRON/MT40A512M16.html

3.3 GTY 收发器误码率测试 IBERT 实验

IBERT（集成式比特误码率测试仪）是 Xilinx 面向 UltraScale™/UltraScale+™ 架构 GTY 收发器专门设计的，用于评估和监控 GTY 收发器。IBERT 包括采用 FPGA 逻辑实现的模式生成器和检查器，并能够接入 GTY 收发器的端口和动态重配置端口属性。还包括了通信逻辑，可通过 JTAG 在运行时进行访问。

IBERT 工具用于对 Xilinx FPGA 芯片的高速串行收发器进行板级硬件测试。通过 IBERT 可以获取误码率，观察眼图，调节串行收发器的参数，从而有助于判断可能存在的问题，便于验证硬件的稳定性和信号完整性。

3.3.1 硬件介绍

使用 IBERT 测试误码率和眼图必须有个收发环通的硬件，开发板上有 1 个 qsfp28 光纤接口和 1 个 sfp 光纤接口。本实验可以将 qsfp28 光纤接口通过一分四的电缆/光纤连到 sfp 光纤接口上，形成 2 个收发环通链路。

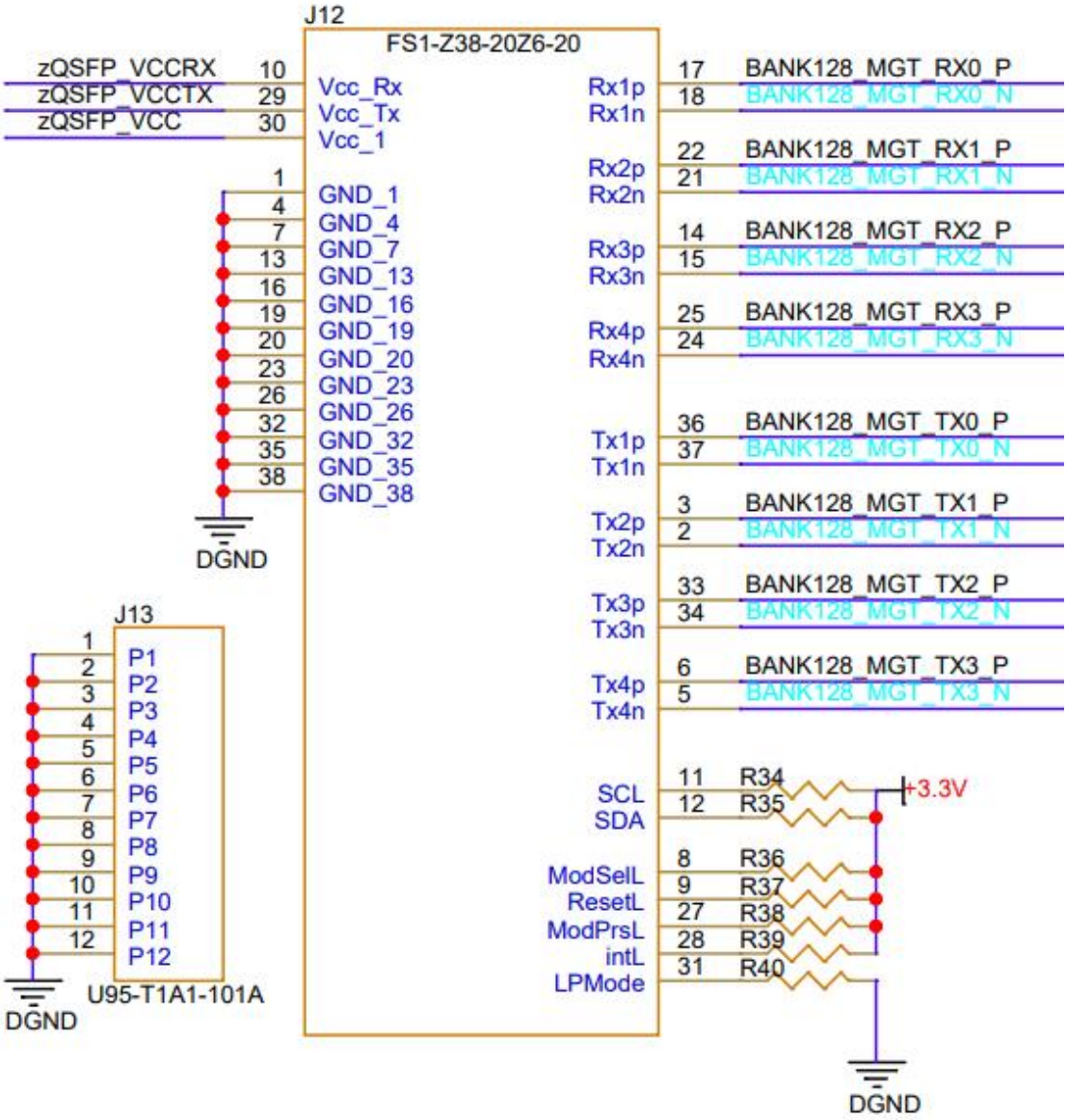


图 8 QSFP 硬件原理图

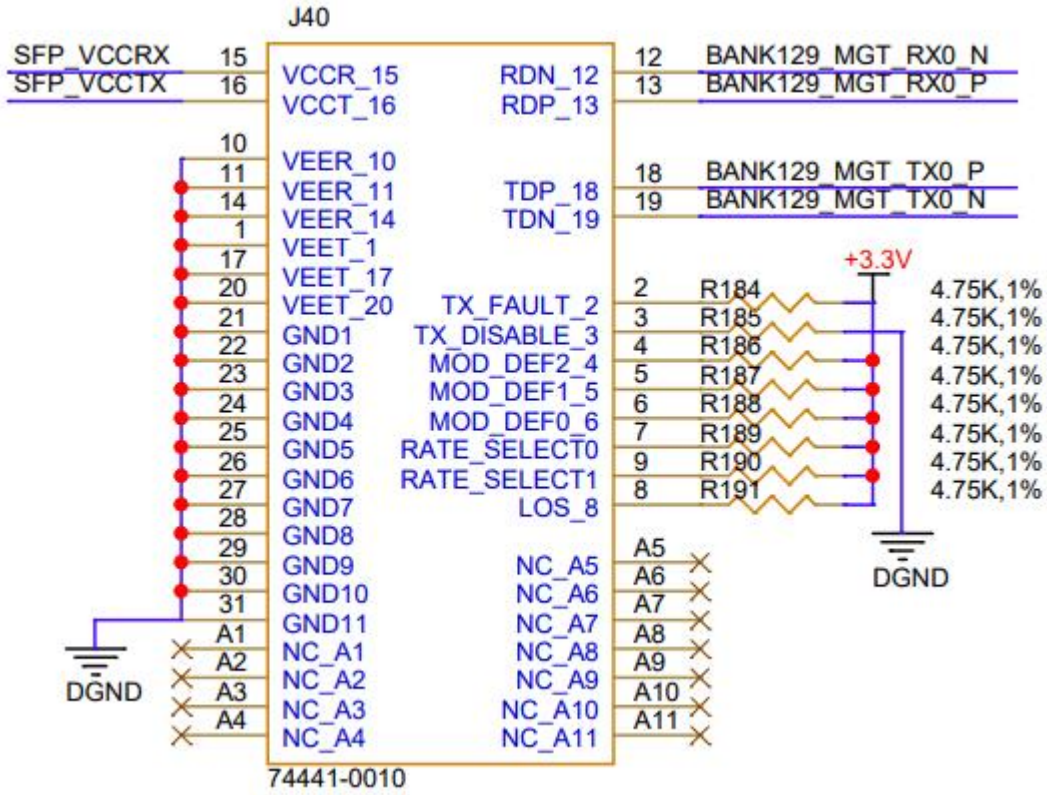
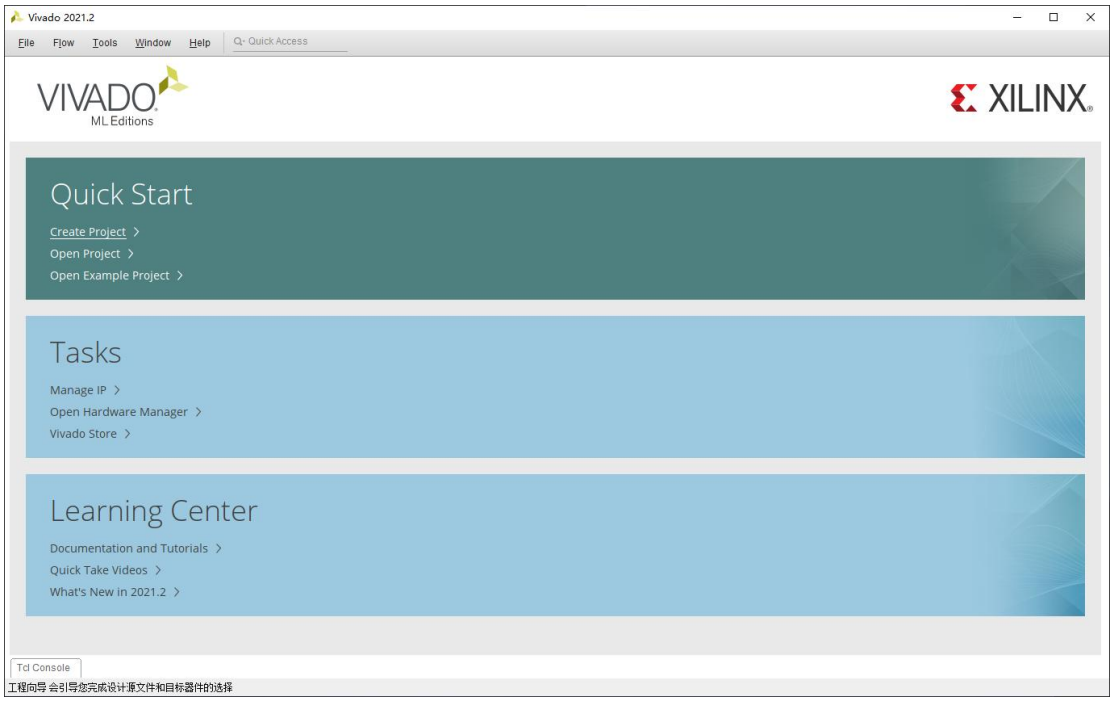


图 9 SFP 硬件原理图

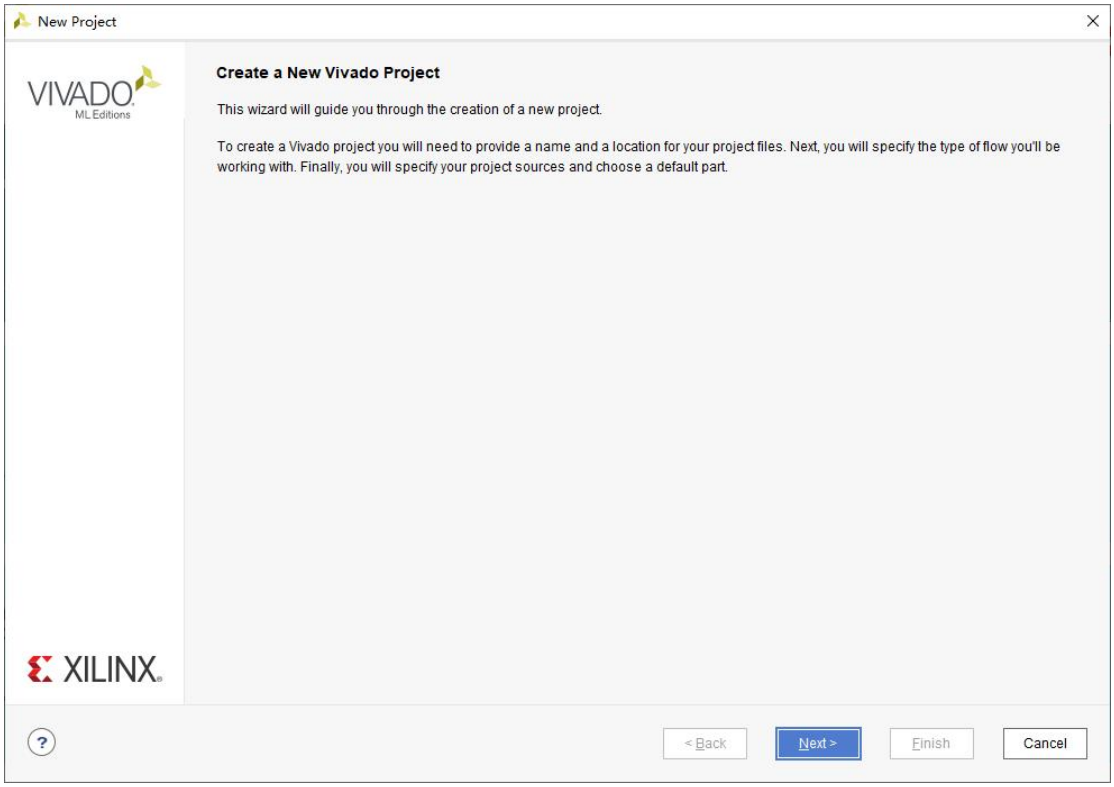
3.3.2 硬件工程搭建

1) 打开 vivado

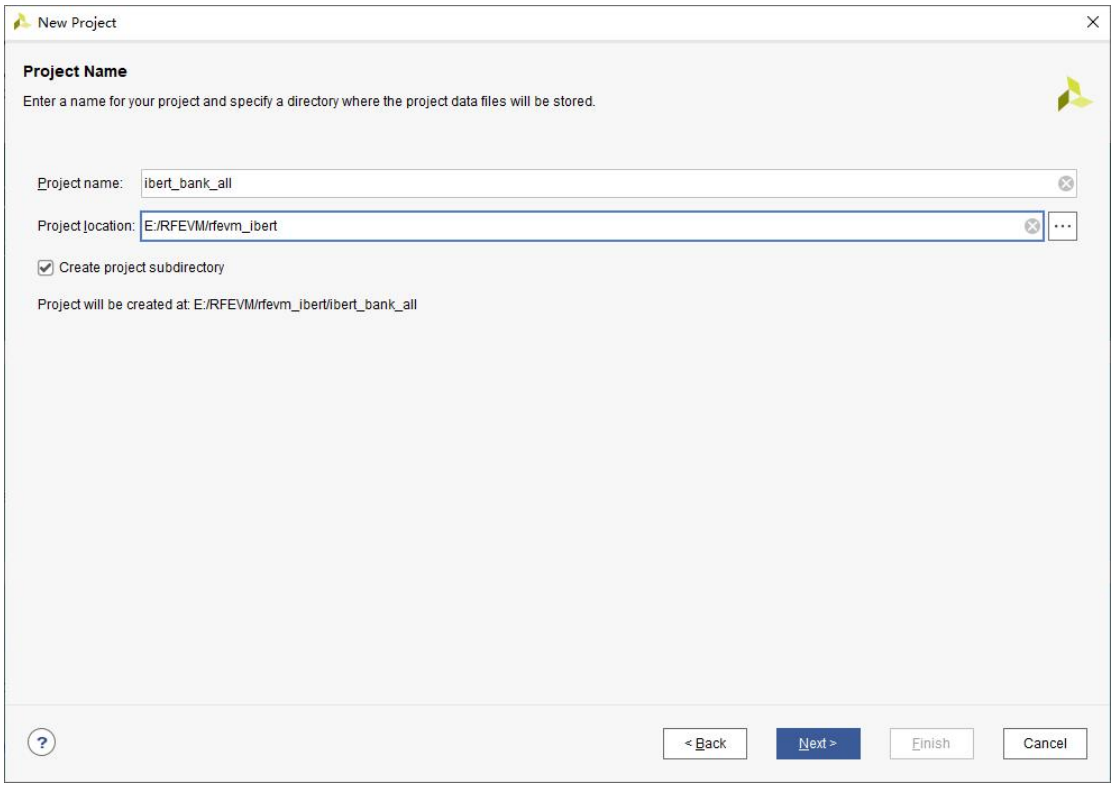
2) 选择 Create Project



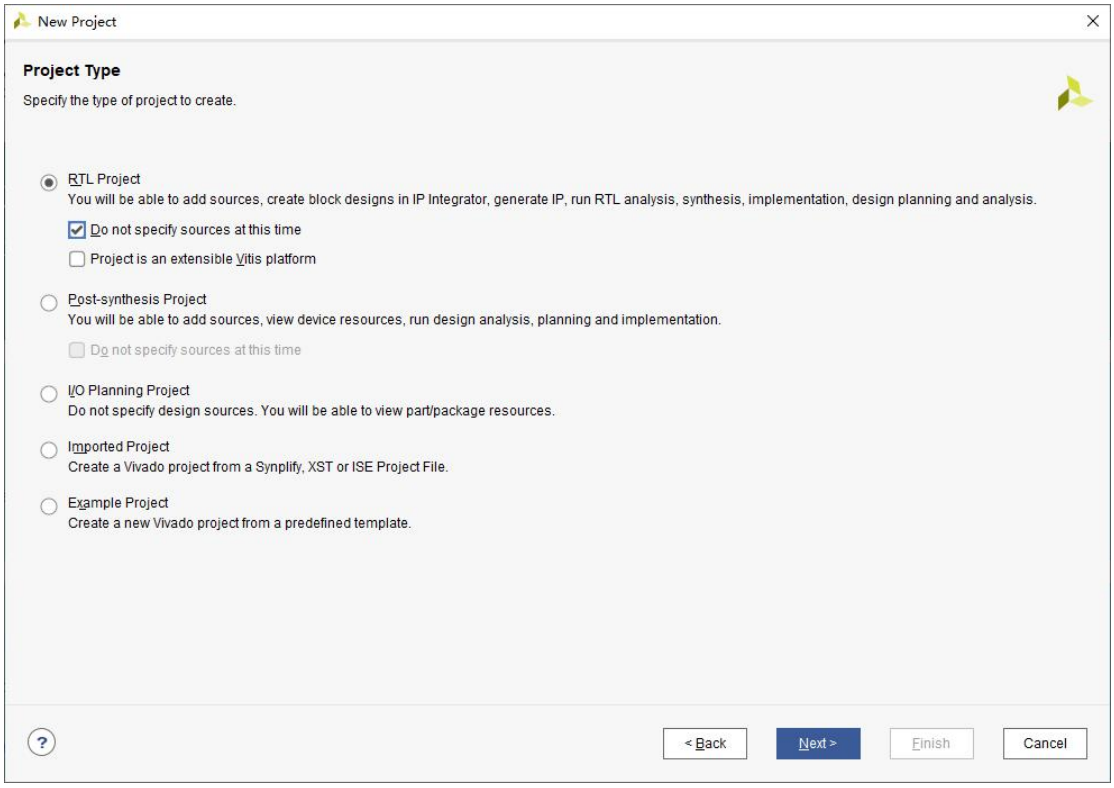
3) 点击 Next



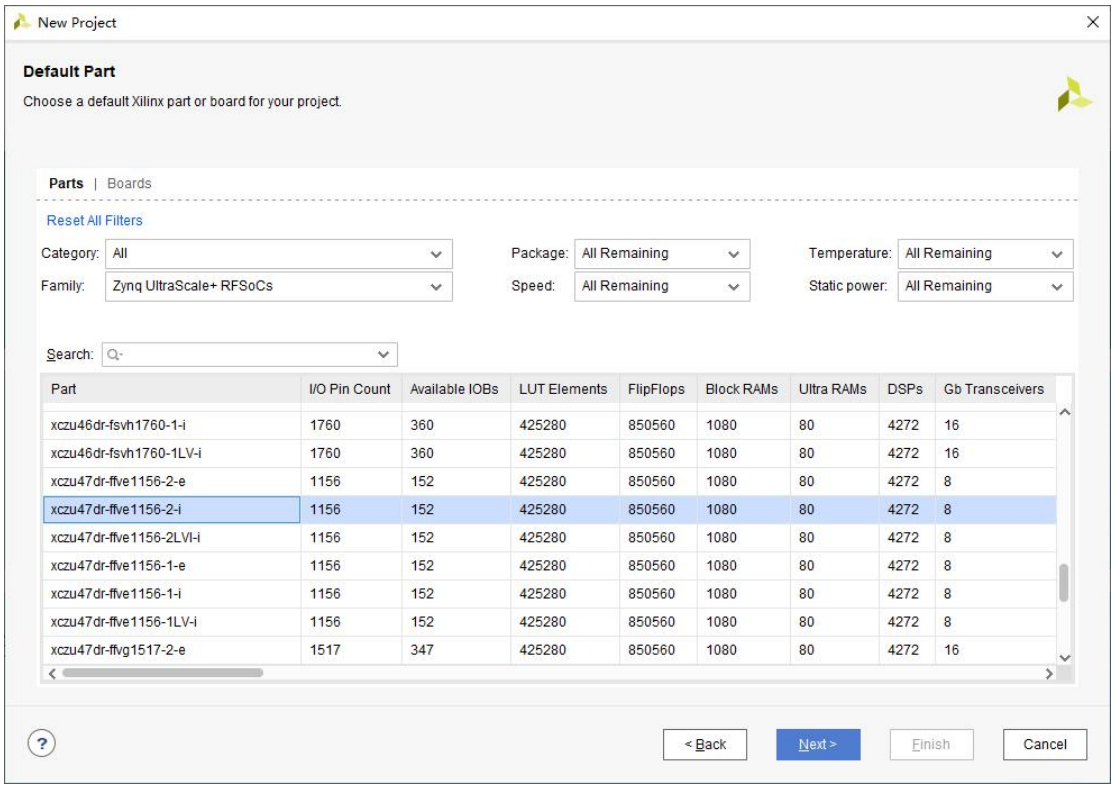
4) 设置工程名称（ibert_bank_all）和位置（D:/RFEVM/rfevm_ibert），点击 Next



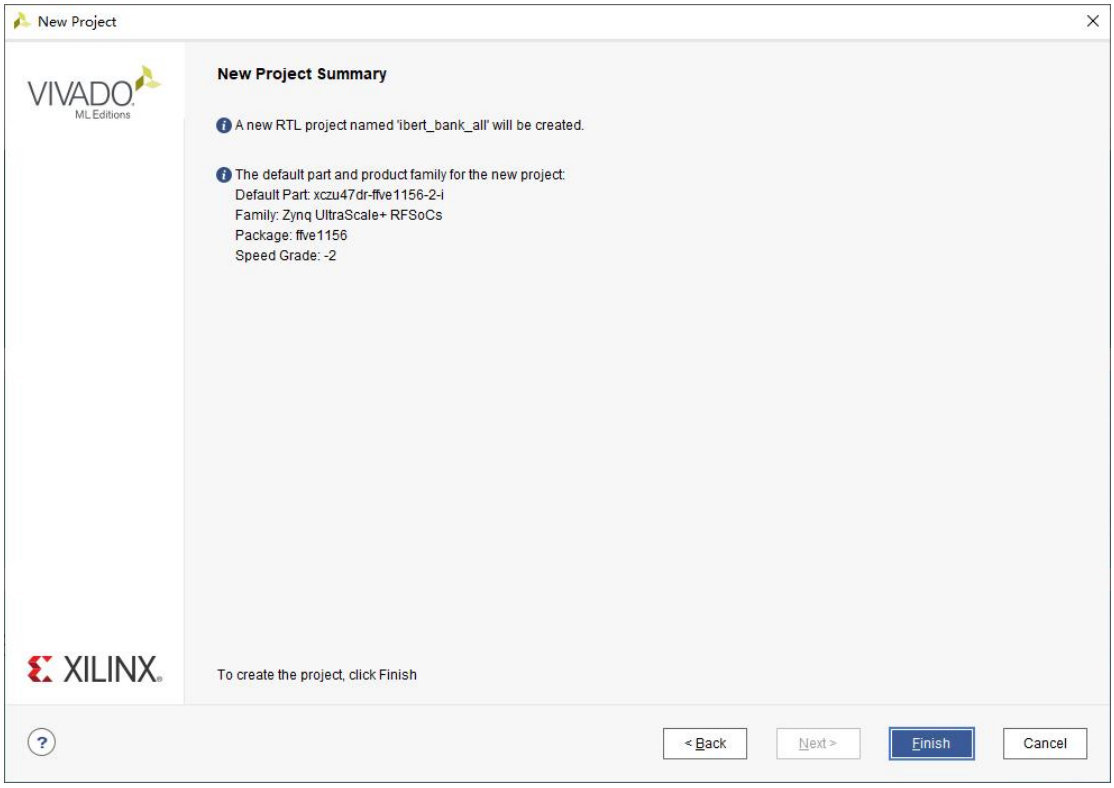
5) 选择 RTL 工程，点击 Next



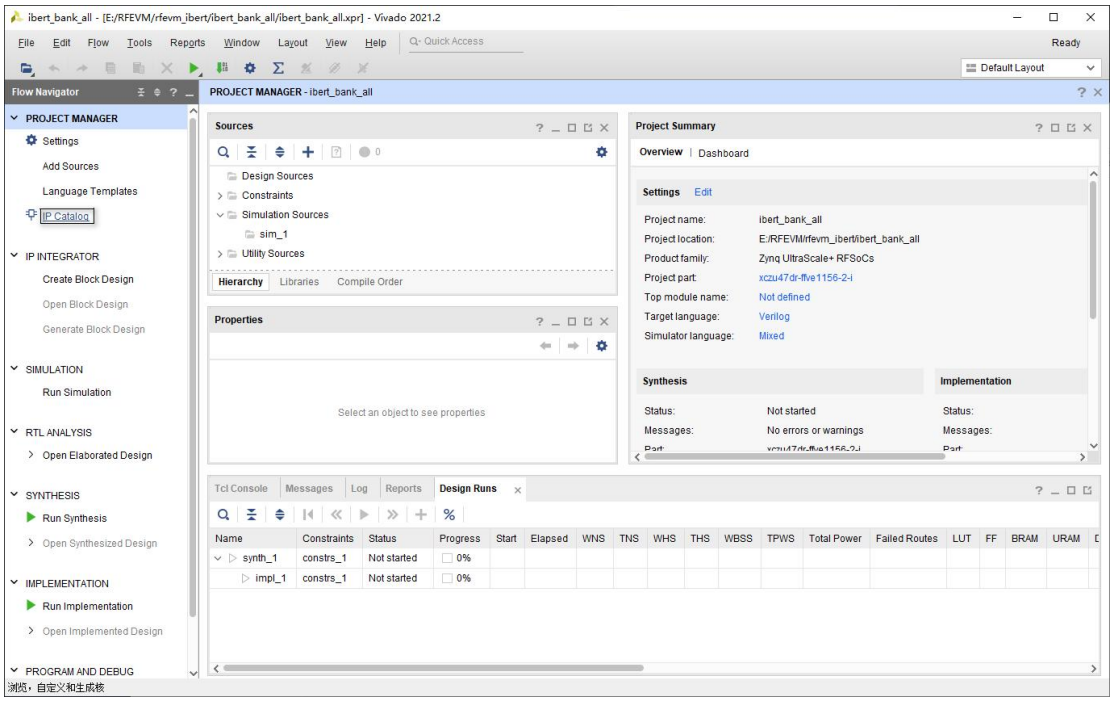
6) 选择芯片型号 xczu47dr-ffve1156-2-i，点击 Next



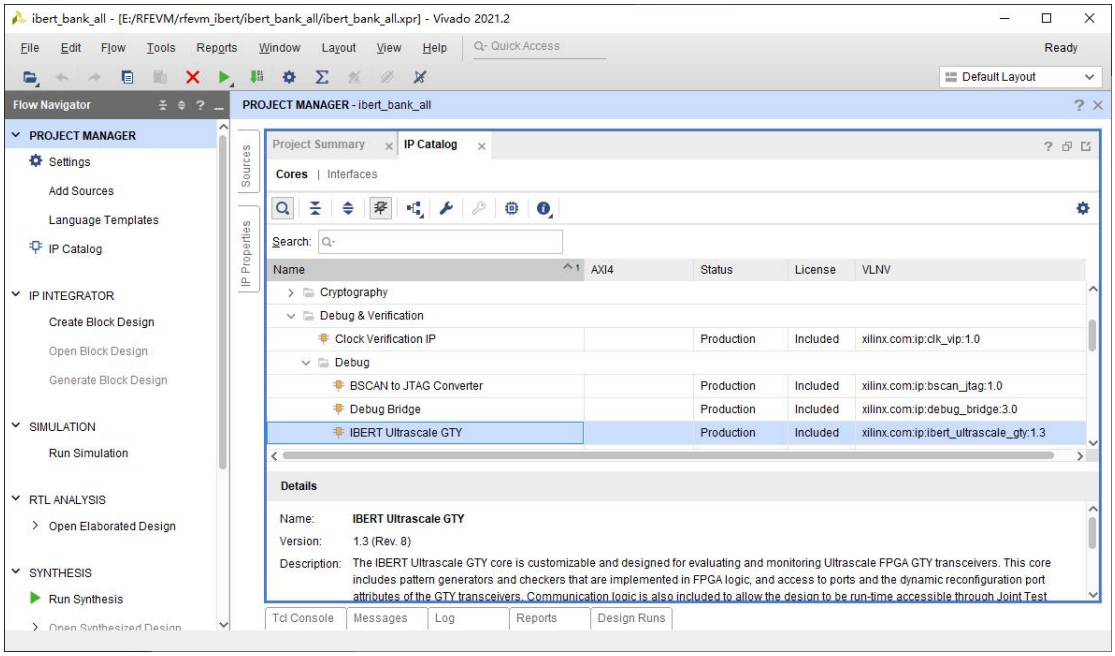
7) 点击 Finish



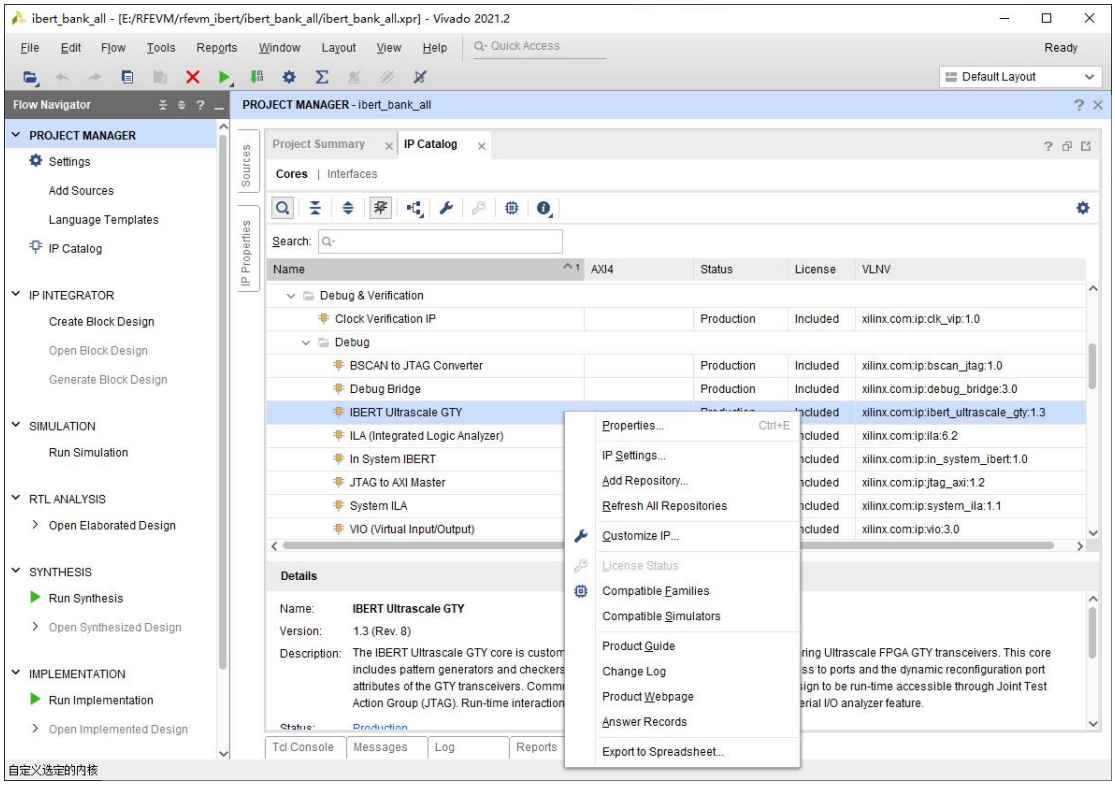
8) 点击 IP Catalog



9) 选择 IBERT UltraScale GTY



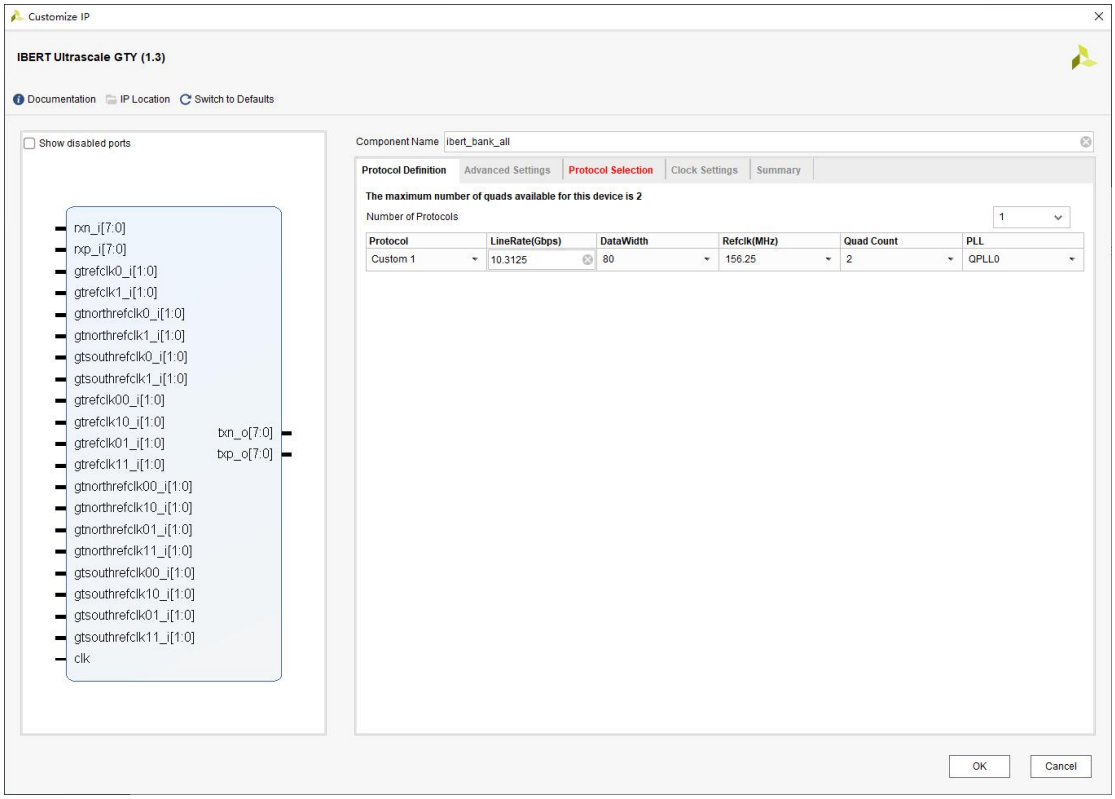
10) 右键 IBERT UltraScale GTY，选择 Customize IP...



11) 设置模块名称：ibert_bank_all

- 在“Protocol Definition”选项卡下

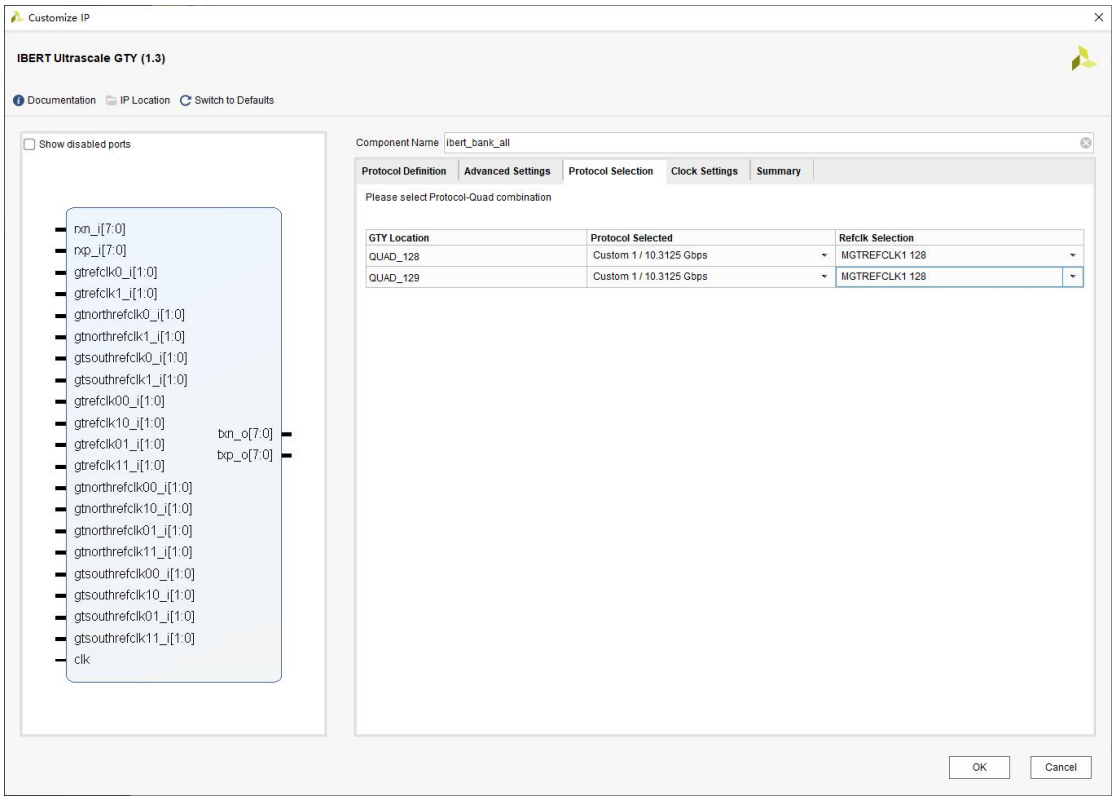
  Protocol Custom 1: LineRate 10.3125, Refclk:156.25, Quad Count:4



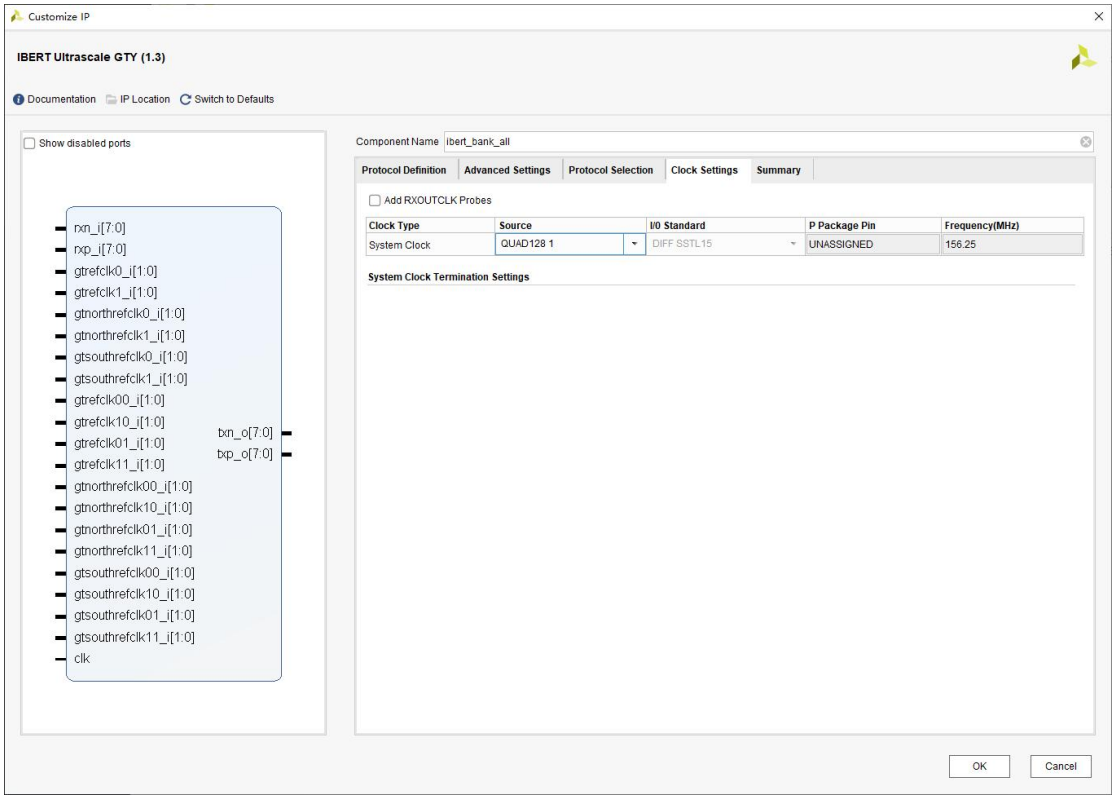
■ 在“Protocol Selection”选项卡下

QUAD_128: Custom 1/10.3125 Gbps, MGTREFCLK1 128

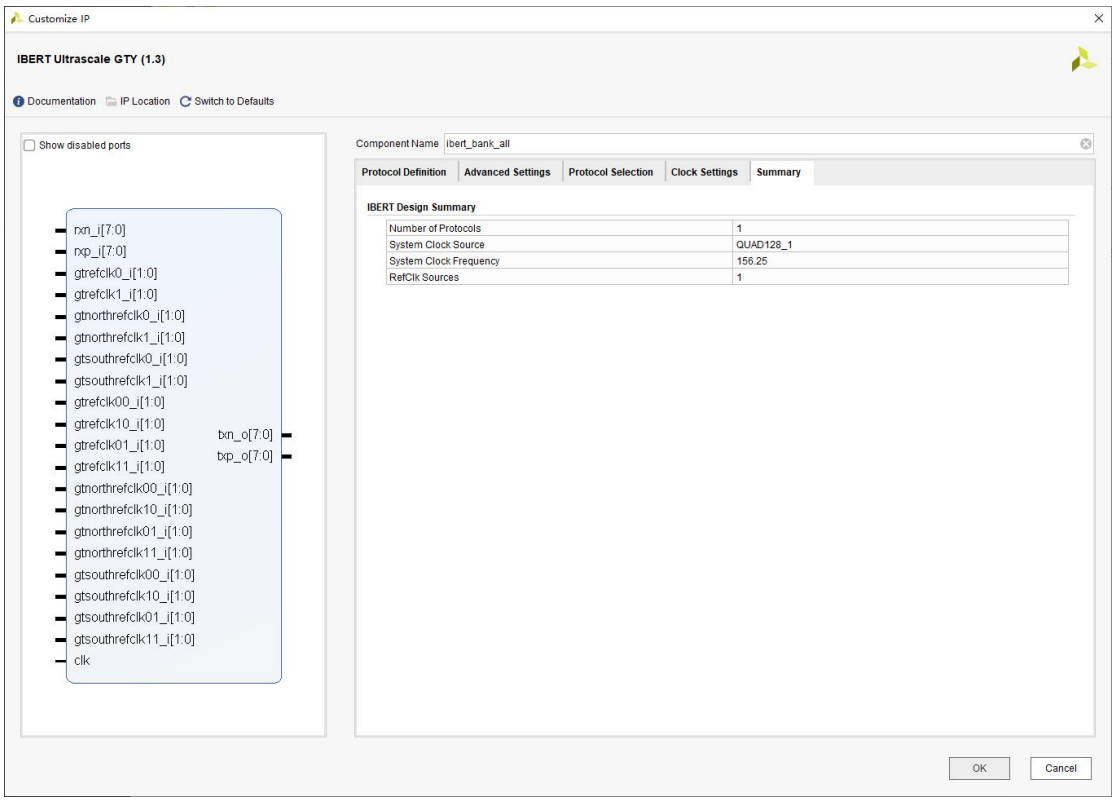
QUAD_129: Custom 1/10.3125 Gbps, MGTREFCLK1 128



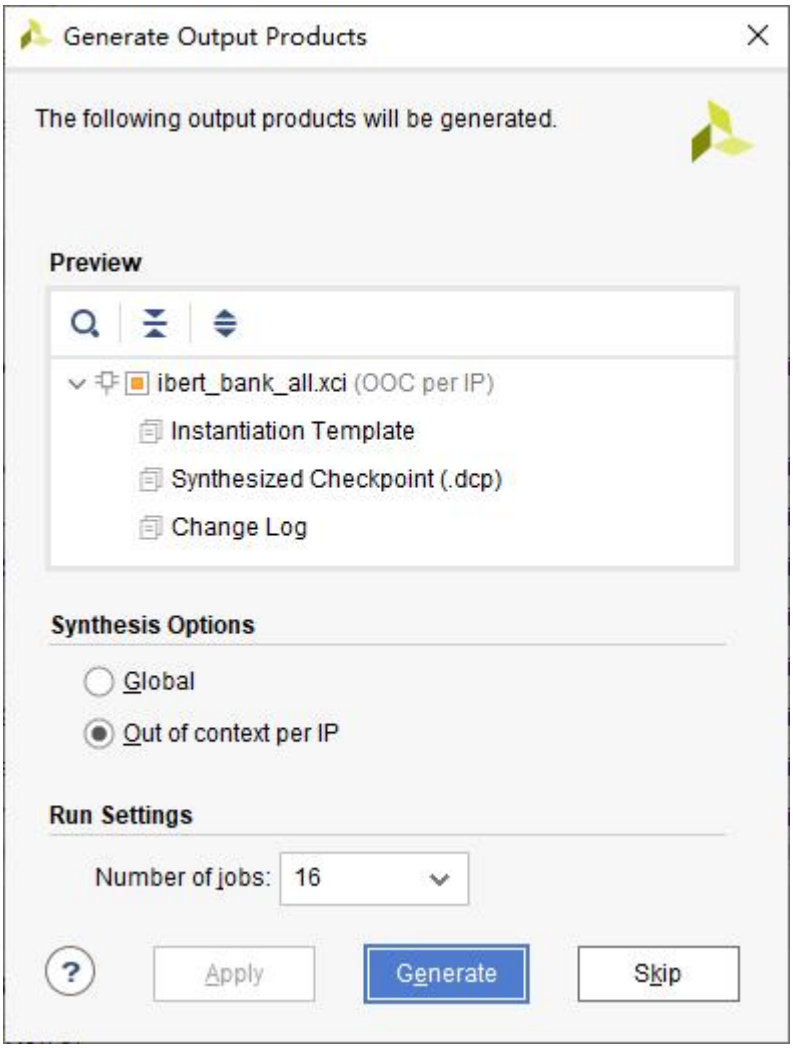
■ 在“Clock Settings”选项卡下，设置时钟



■ 查看摘要，并点击“OK”



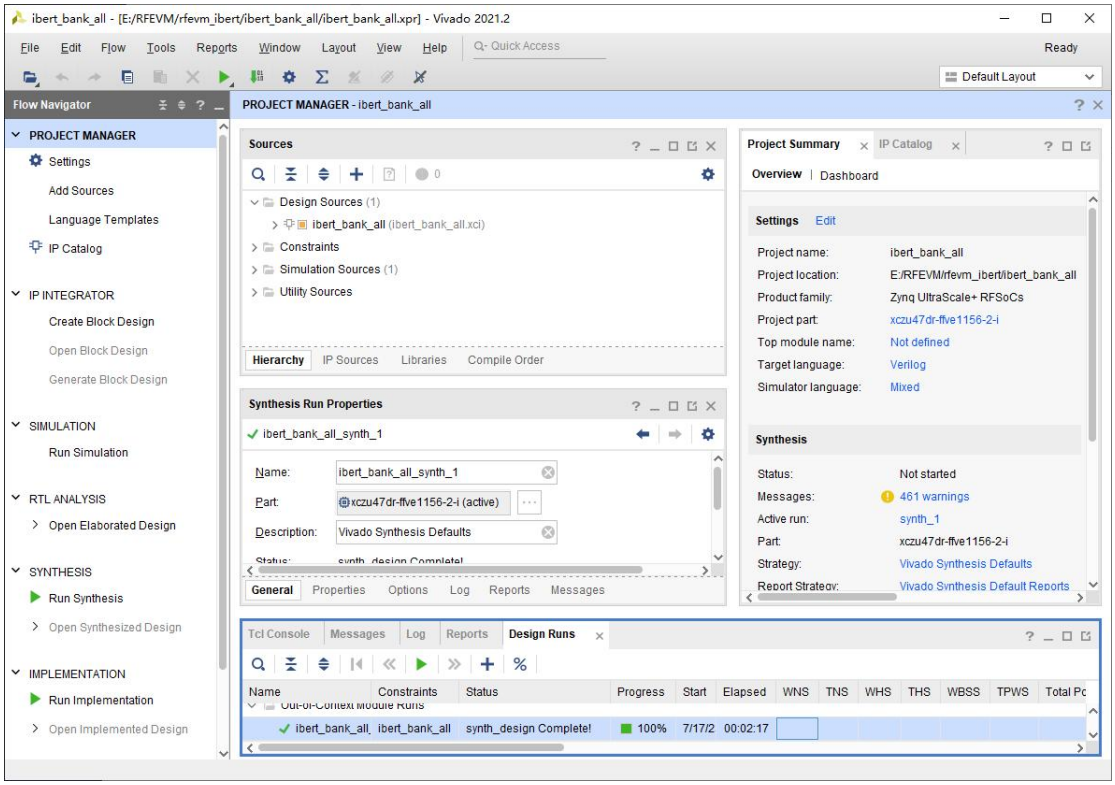
12) 点击“Generate”



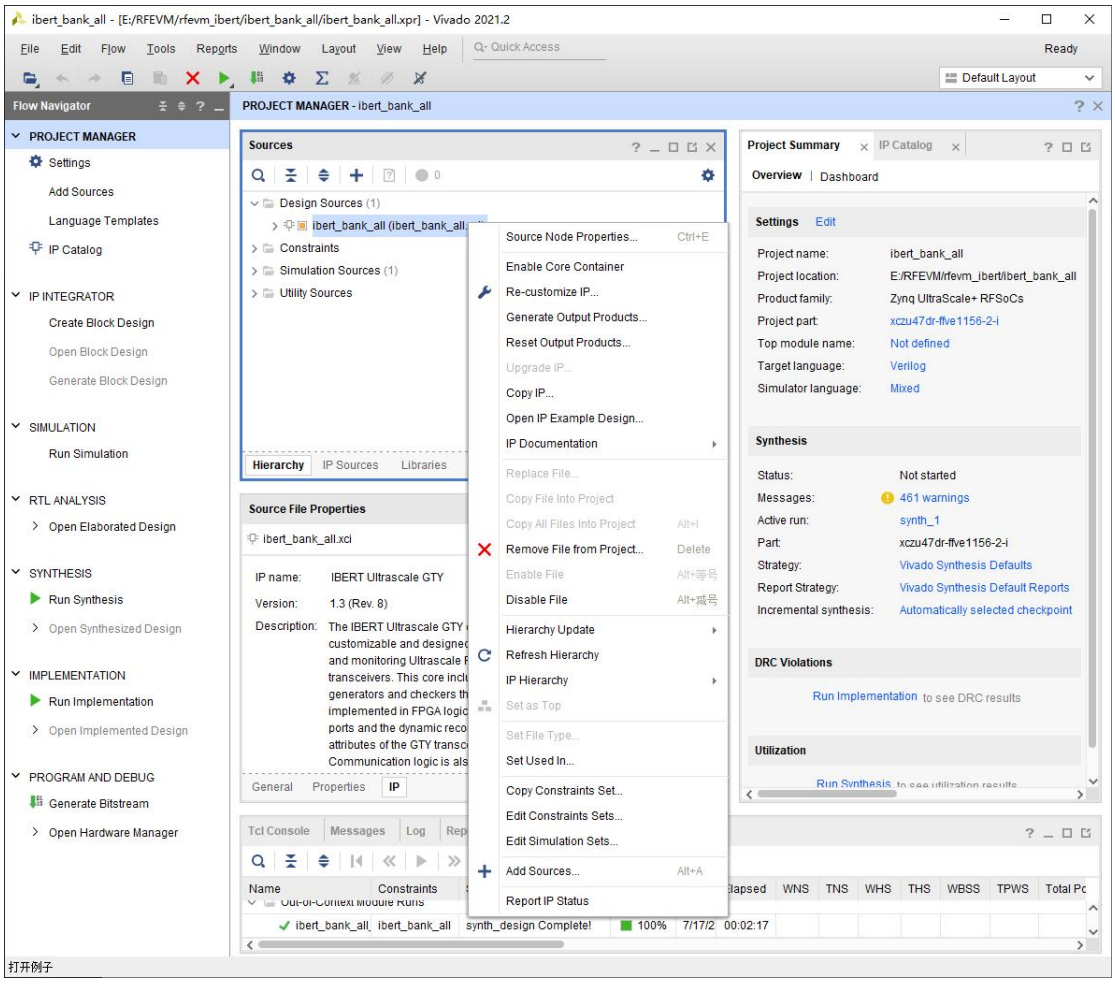
注意：这一步将花费 3 分钟左右，具体视电脑配置和线程而定

- 生成的 IBERT IP 出现在 Design Sources

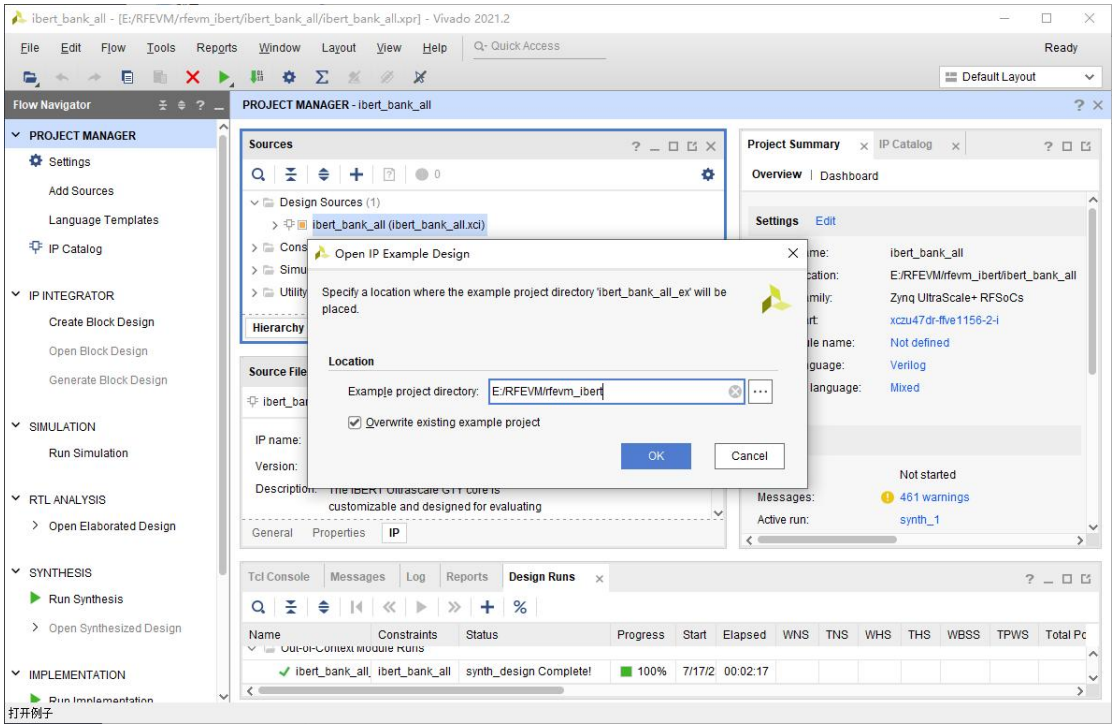
  等待 synth_design_Complete 出现在 ibert_bank_all_synth_1



13) 右键 ibert_bank_all，选择 Open IP Example Design ...

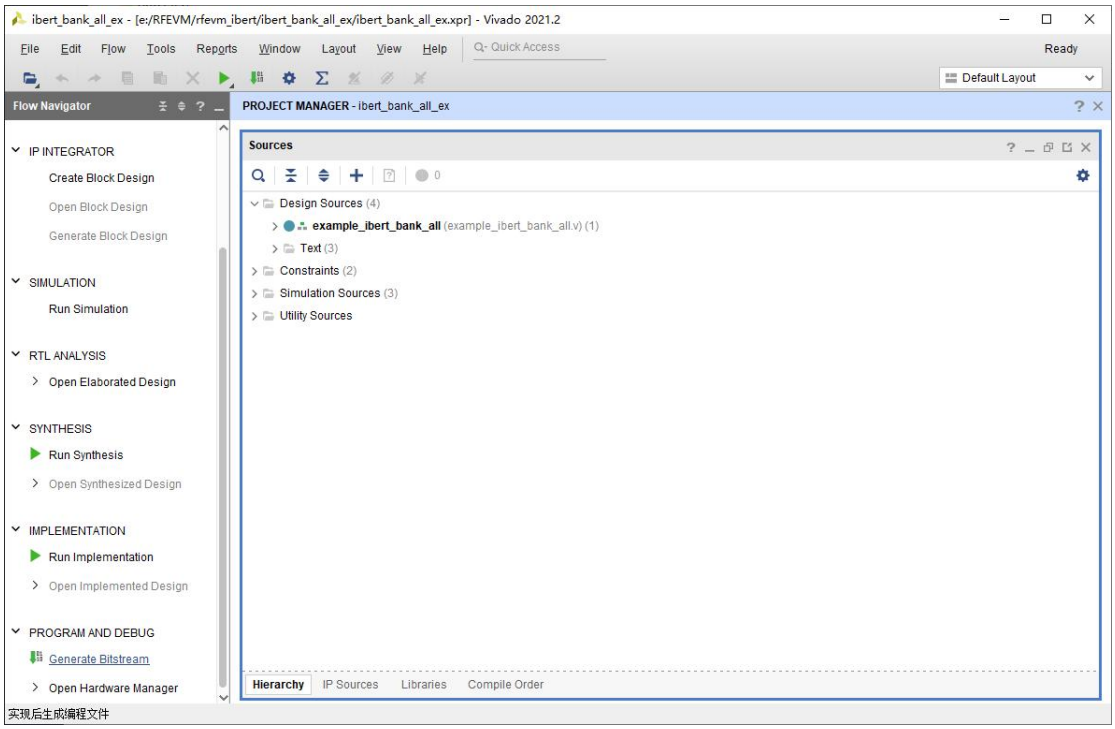


14) 选择位置并点击 OK



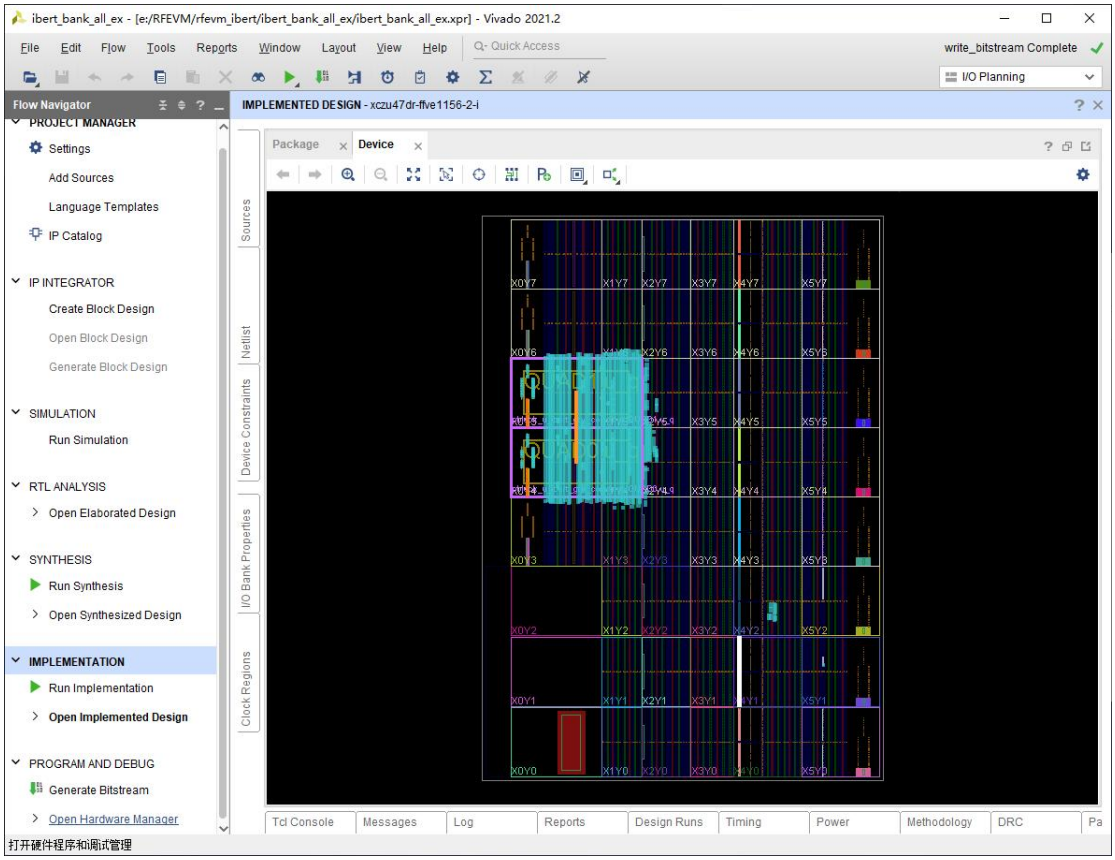
15) 创建一个新的工程

16) 点击 Generate Bitstream

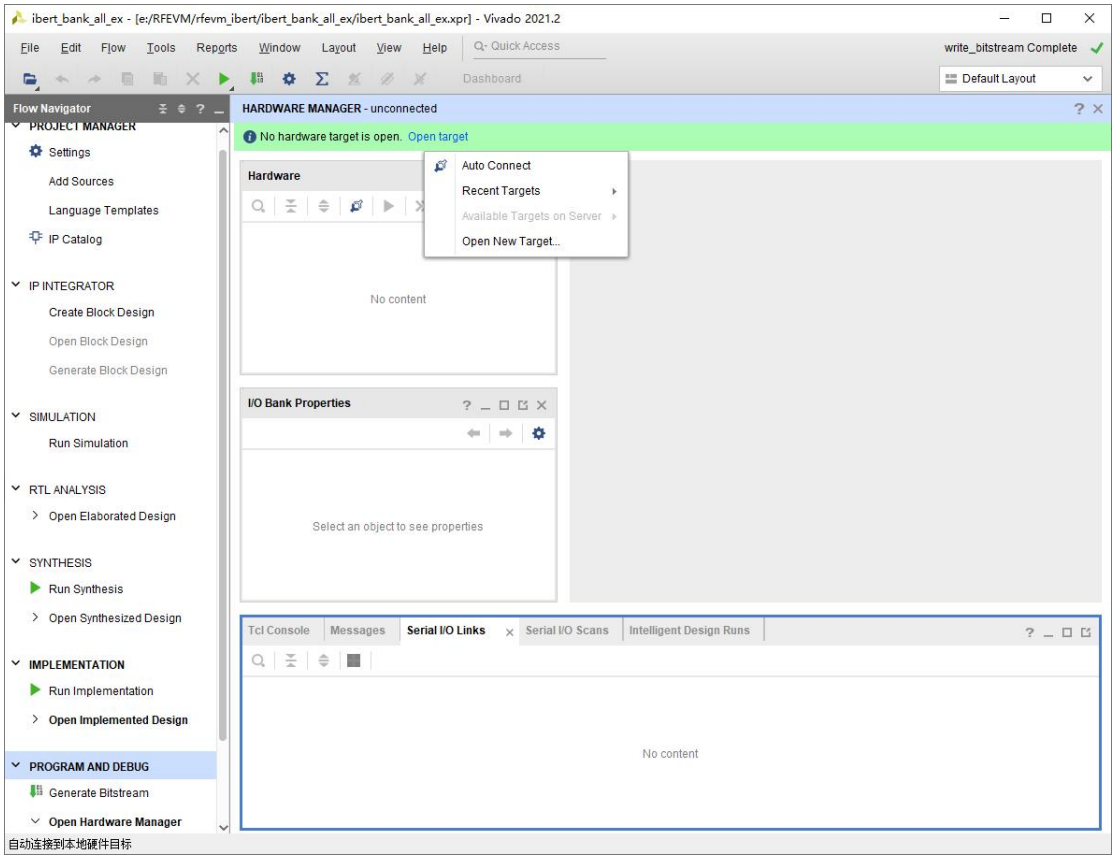


3.3.3 下载验证

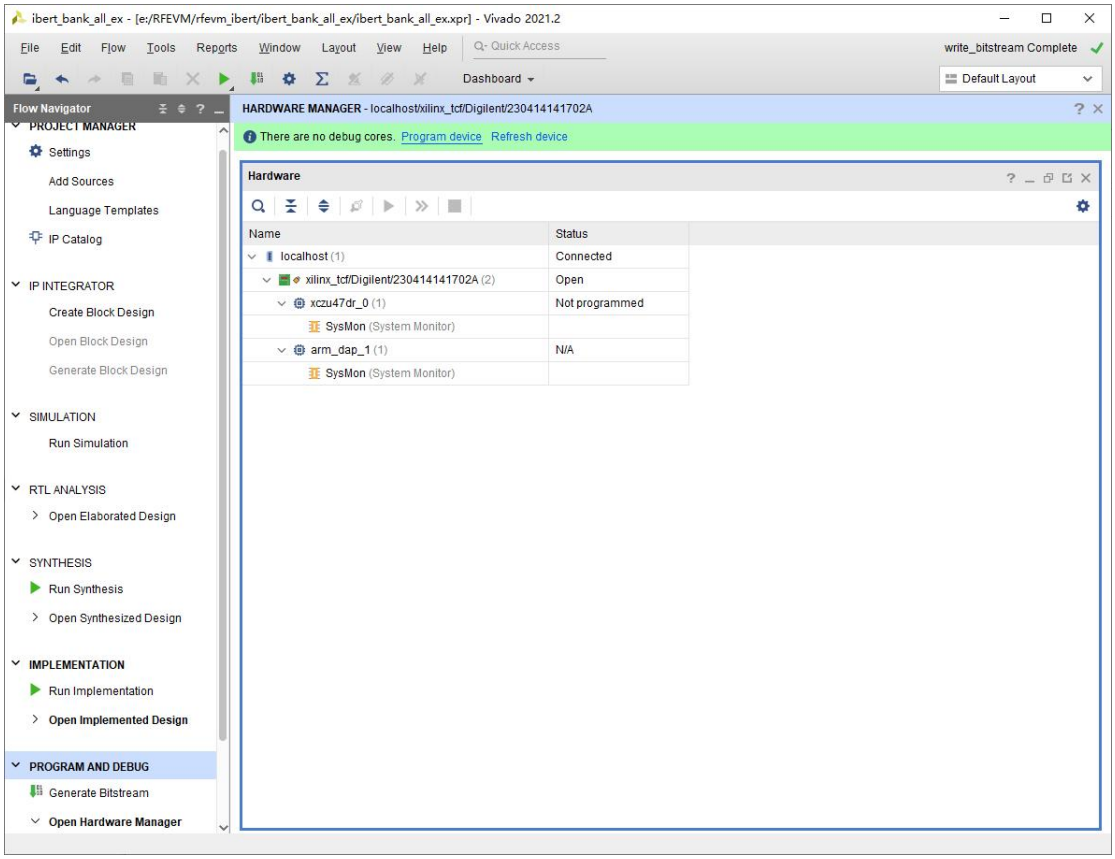
1) 点击 Open Hardware Manager



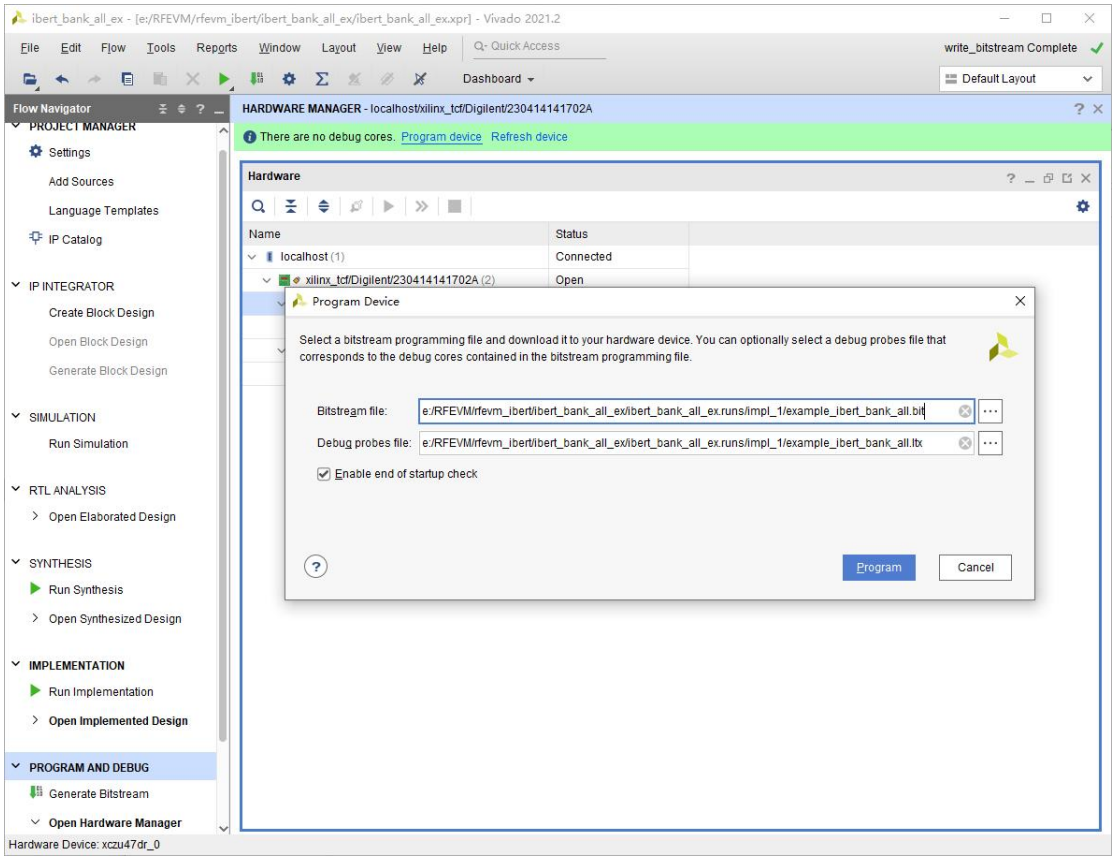
2) 点击 Open target，选择 Auto Connect



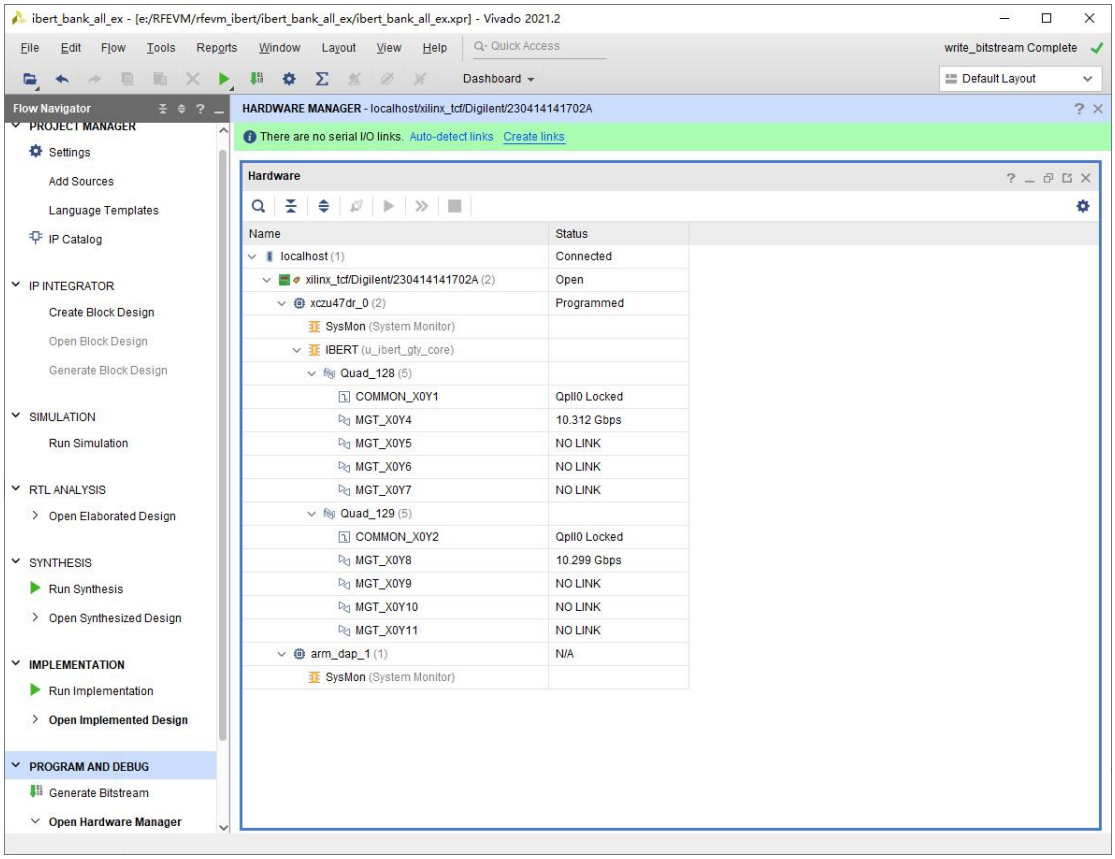
3) 选择 Program device



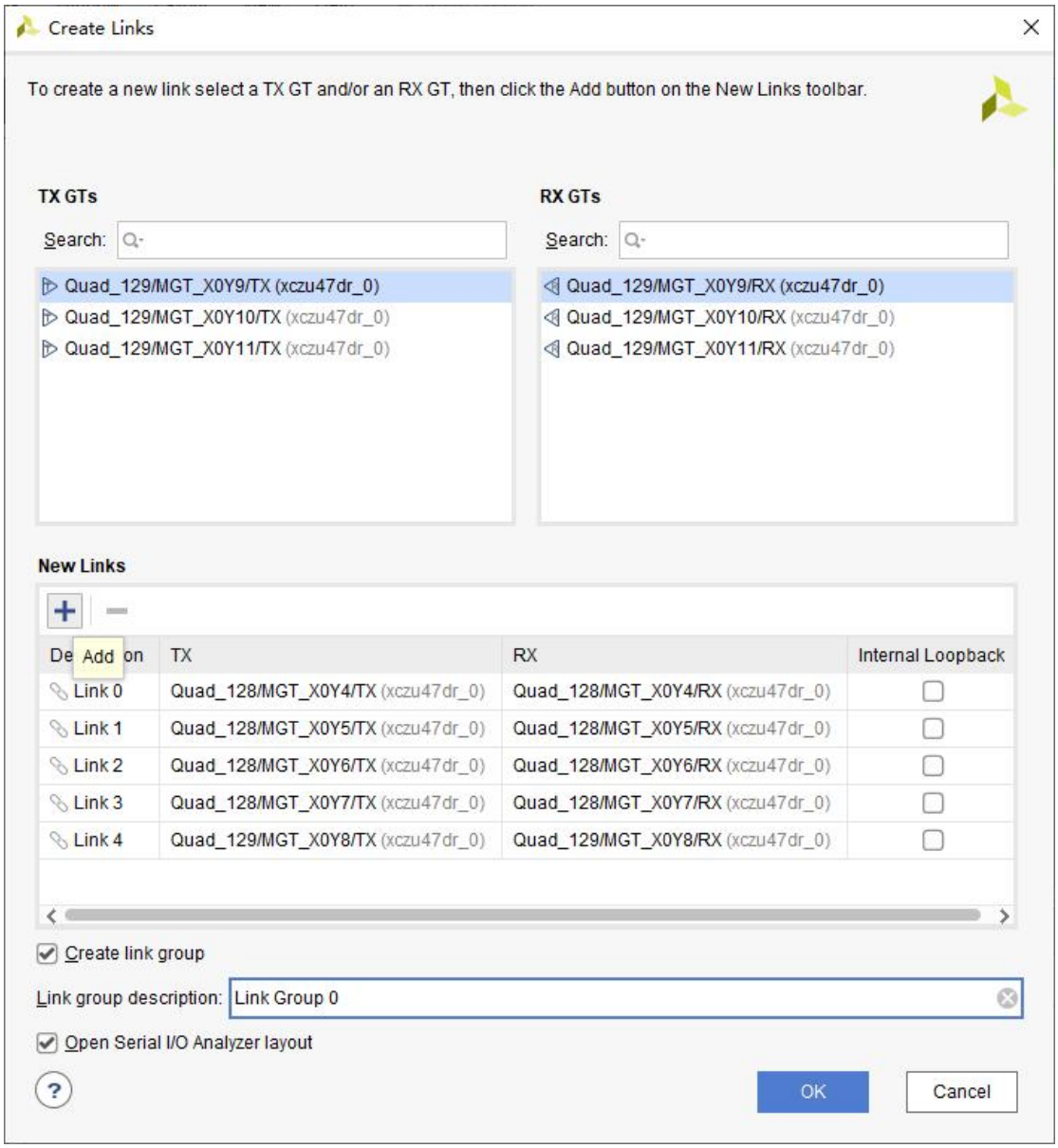
4) Bitstream file 和 LTX 文件默认，点击 Program



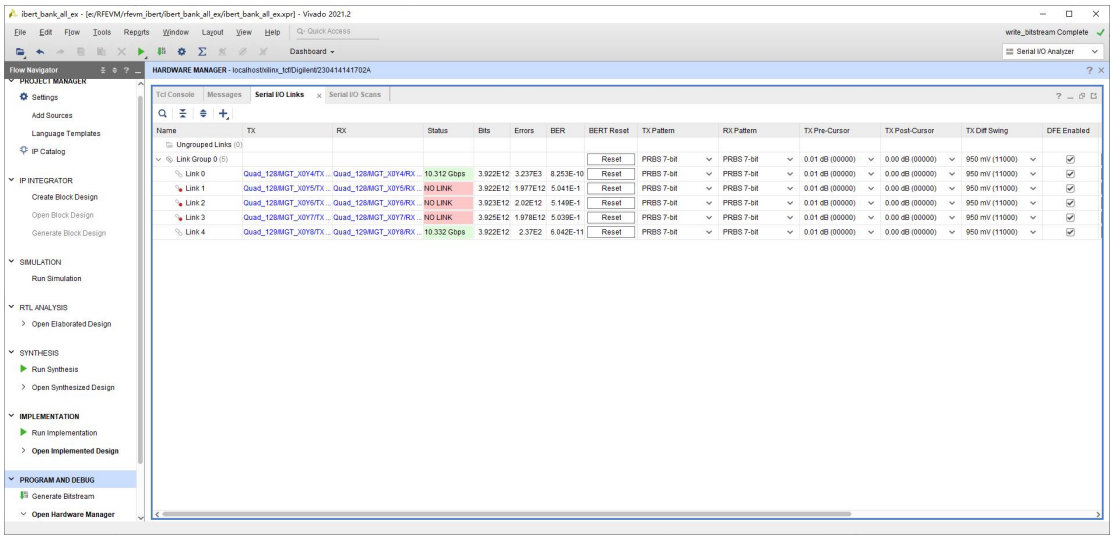
5) 点击 Create links



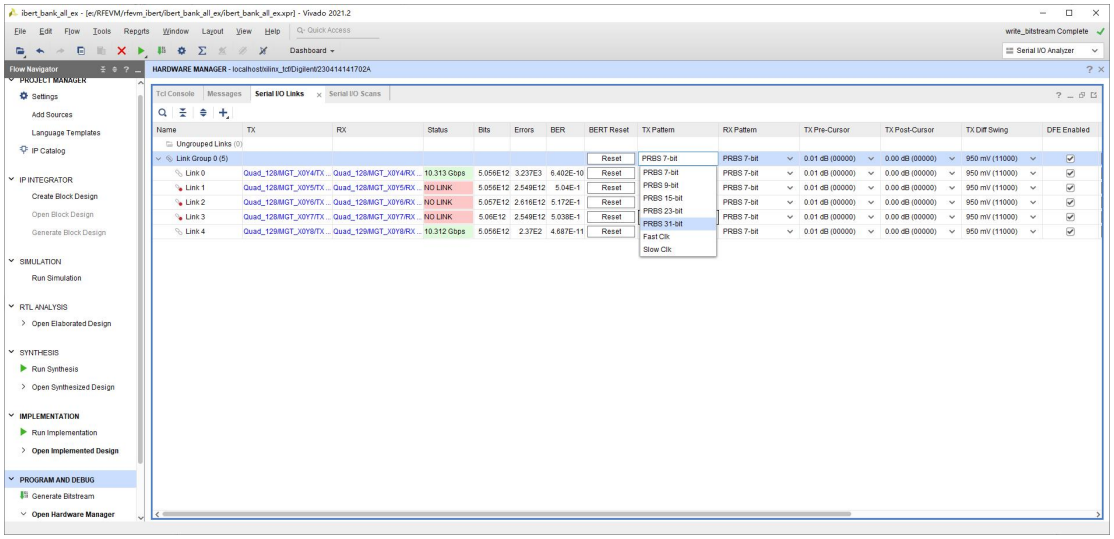
6) 依次选择 MGTX0Y4~MGTX0Y8，点击添加，然后点击 OK

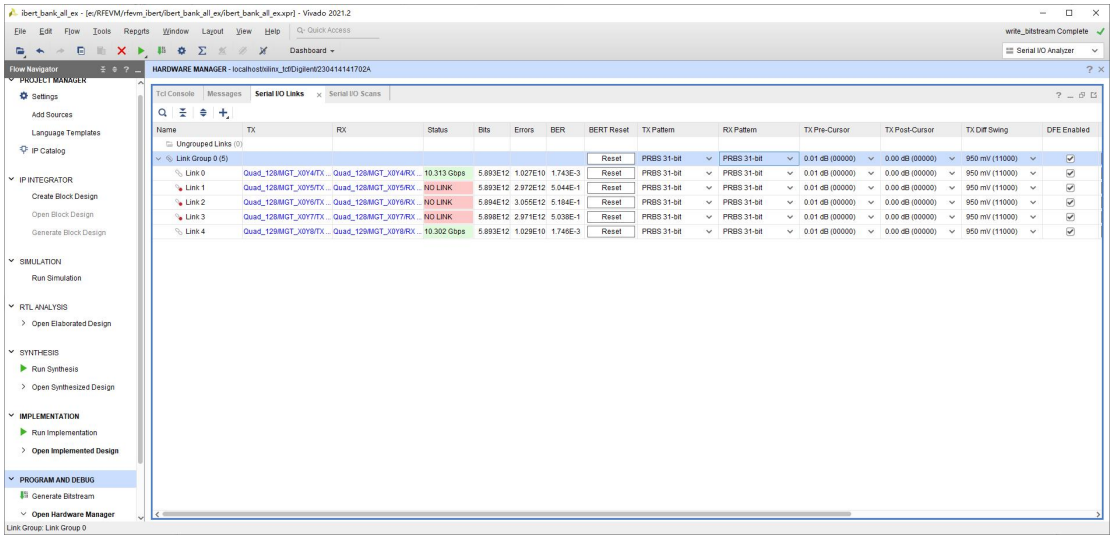


7) Links 会出现在 Serial I/O Links 选项卡下

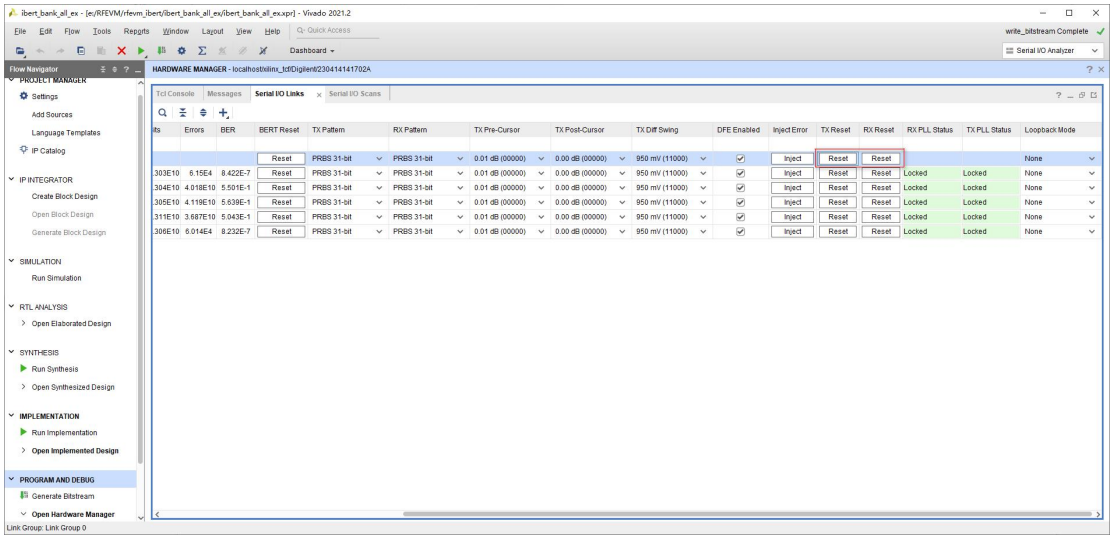


8) 设置 TX Patterns 和 RX Patterns 为 PRBS 31-bit

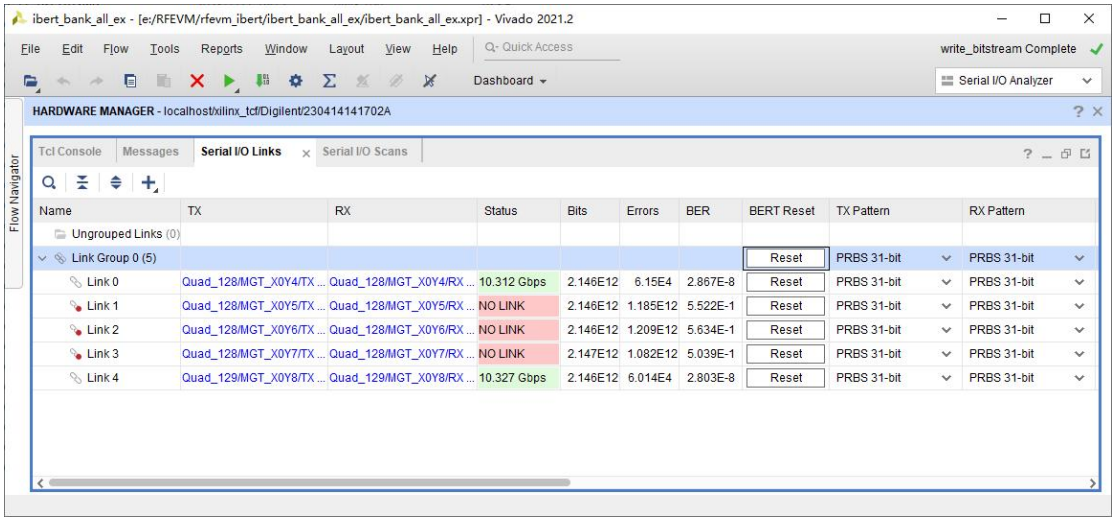




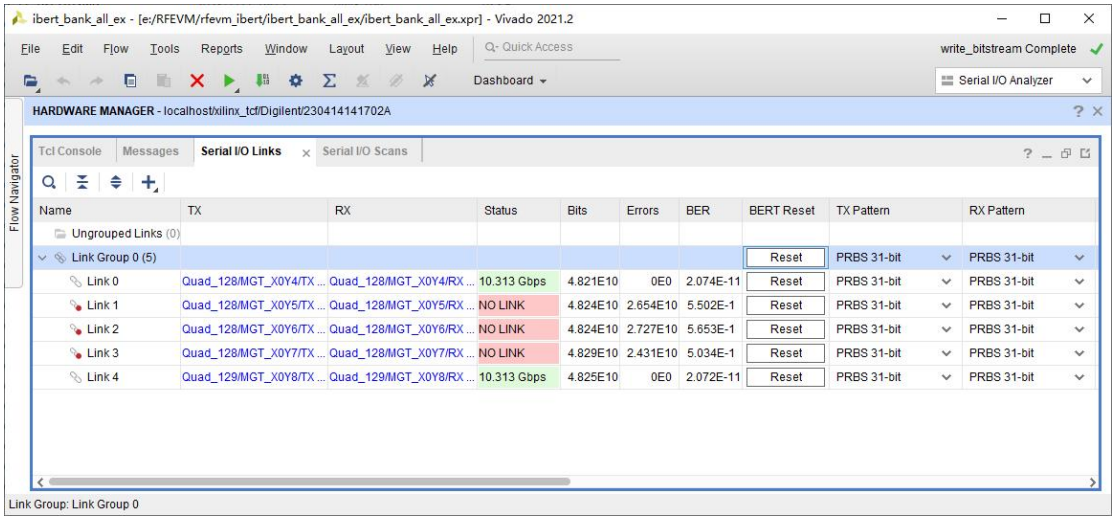
9) 依次点击 RX Reset 和 TX Reset



10) 点击 BERT Reset，复位 link error 计数器



11) links 显示 no errors



3.3.4 参考文档

- IBERT IP

  IBERT for UltraScale GTY Transceivers - PG196

  https://docs.xilinx.com/v/u/en-US/pg196-ibert-ultrascale-gty

- Vivado Programming and Debugging

  Vivado Design Suit Programming and Debugging User Guide - UG908

  https://docs.xilinx.com/r/2021.2-English/ug908-vivado-programming-debugging/Introduction

4 PS 端外设

本章主要介绍 ZYNQ 核的配置，PS 端外设的应用，如 MIO、UART、以太网和 RTC 等的基础实验。

前面的实验都是在 PL 端进行的，可以看到和普通 FPGA 开发流程没有任何区别，ZYNQ 的主要优势就是 FPGA 和 ARM 的合理结合，这对开发人员提出了更高的要求。从本章开始，我们开始使用 ARM，也就是我们说的 PS。

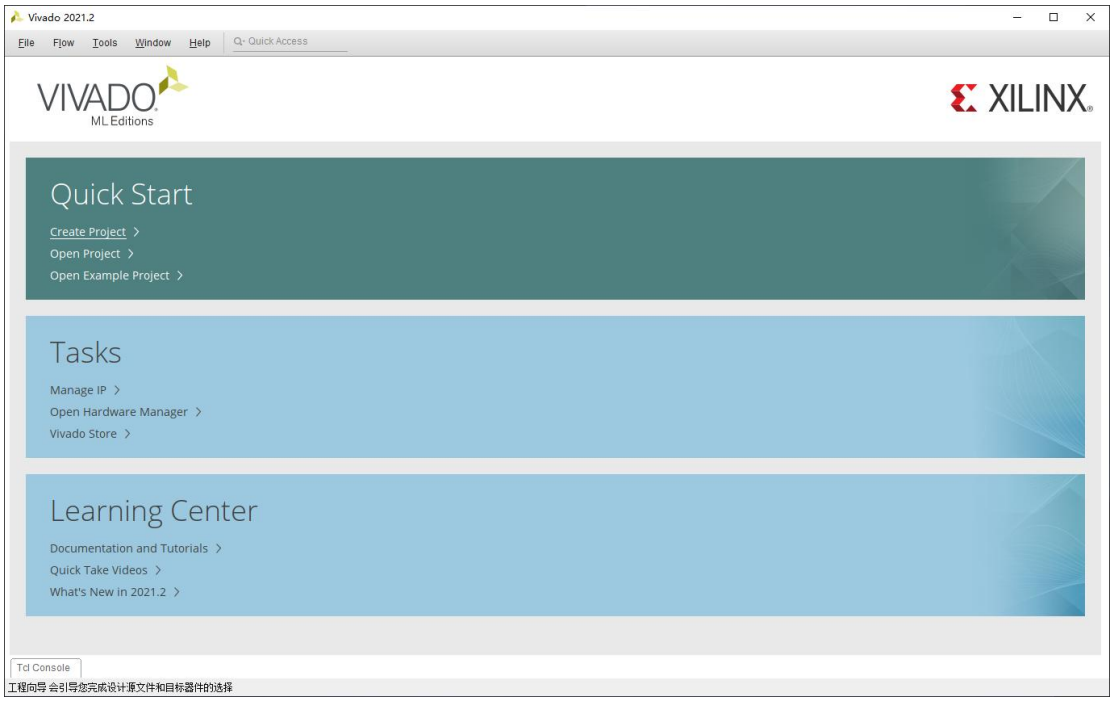
4.1 IPI 设计建立

Vivado IP 集成器（IPI）是 vivado 集成开发环境提供的 IP 集成设计界面。IP 集成器是一个图形化，支持脚本配置与连接的设计开发环境。在 IP 集成器不仅可使用 IP 进行基于处理器的设计还可以进行非处理器的纯逻辑设计开发。

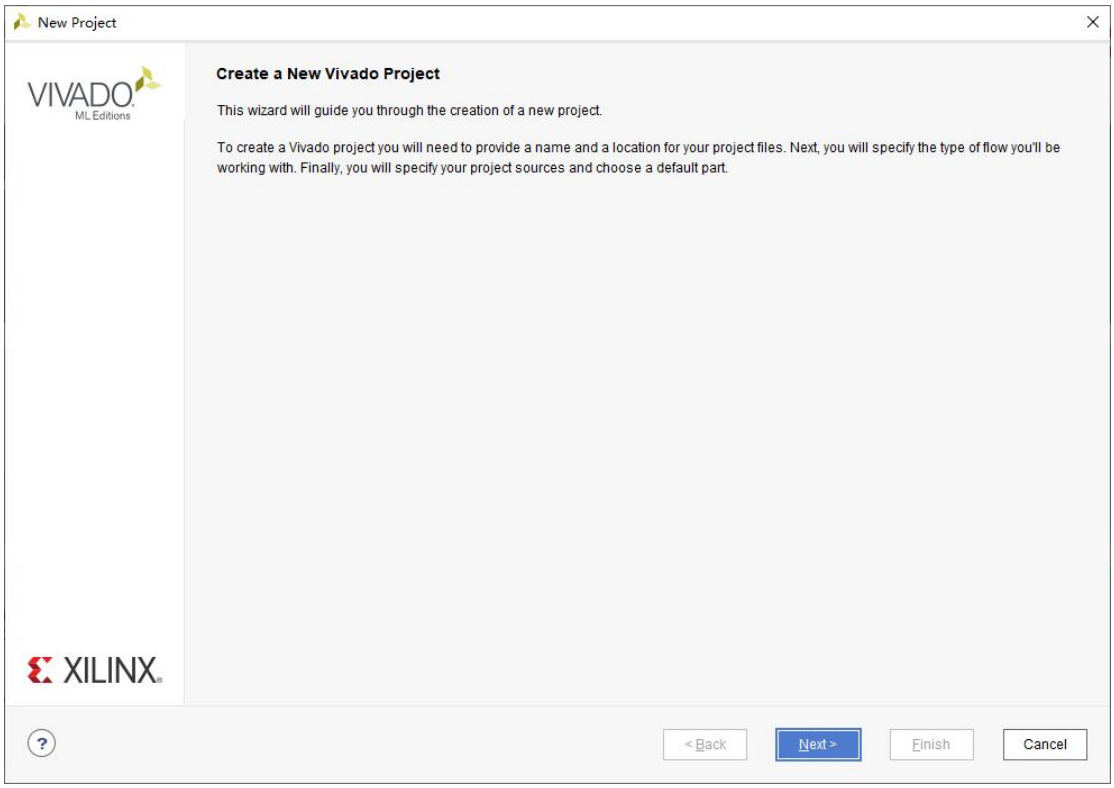
这里我们在 IPI 里面使能所有 PS 的外设，后续的 PS 外设实验就只需在 vitis 下面操作即可。

4.1.1 硬件工程搭建

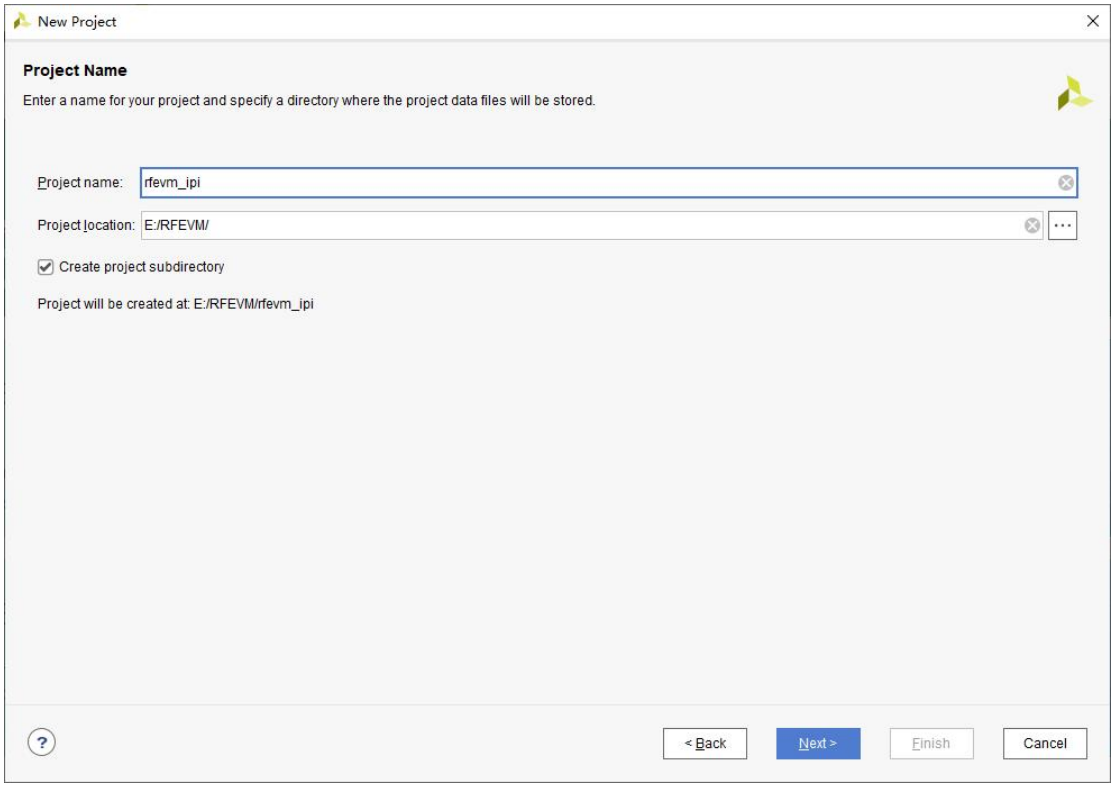
1) 打开 vivado
2) 选择 Create Project



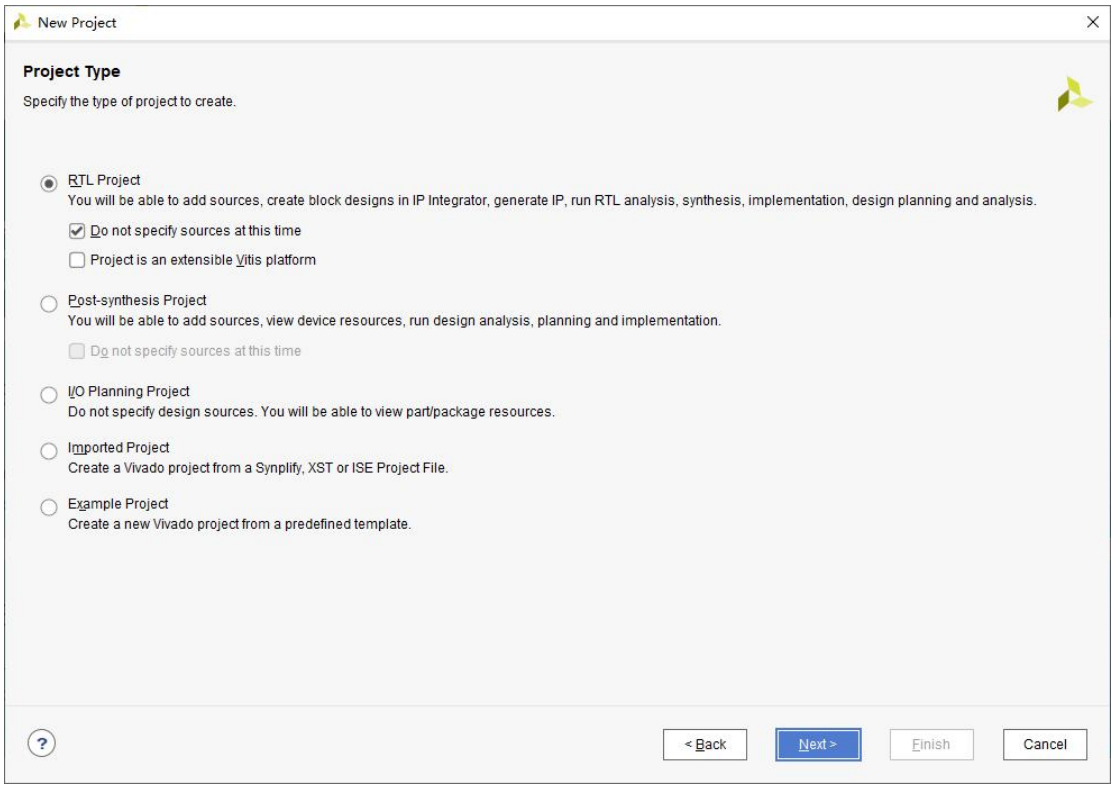
3) 点击 Next



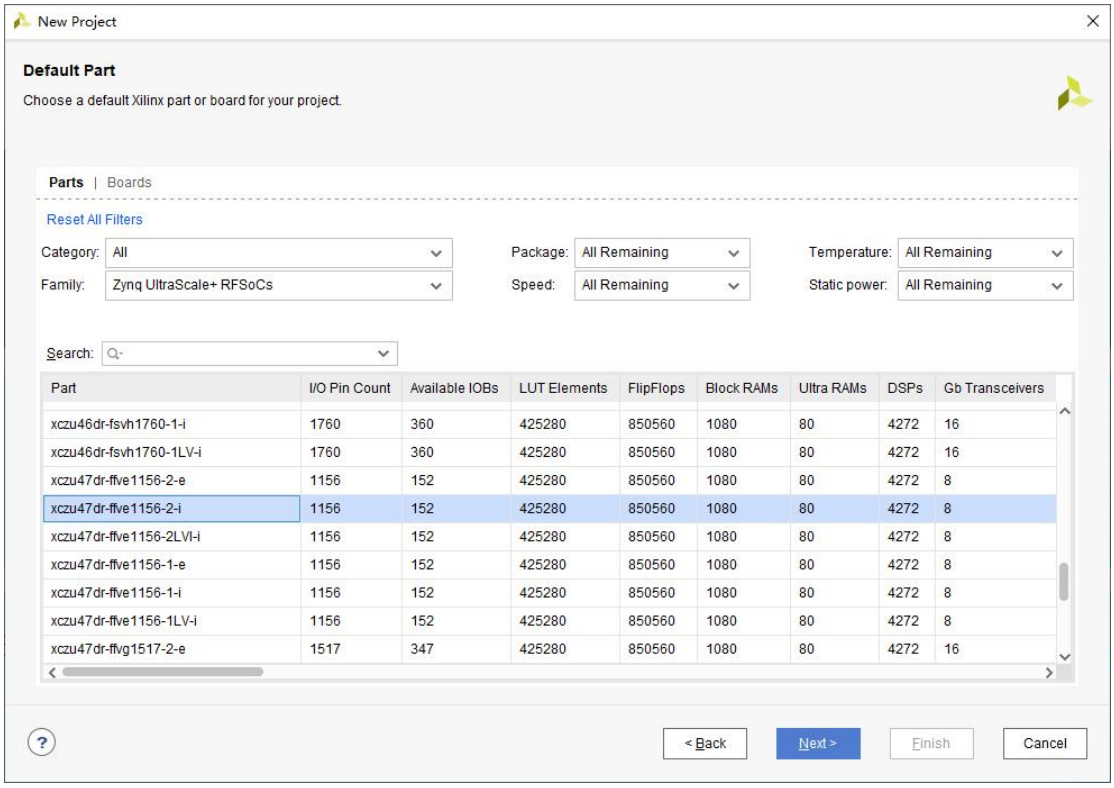
4) 设置工程名称（rfevm_ipi）和位置(E:/RFEVM/),点击 Next；



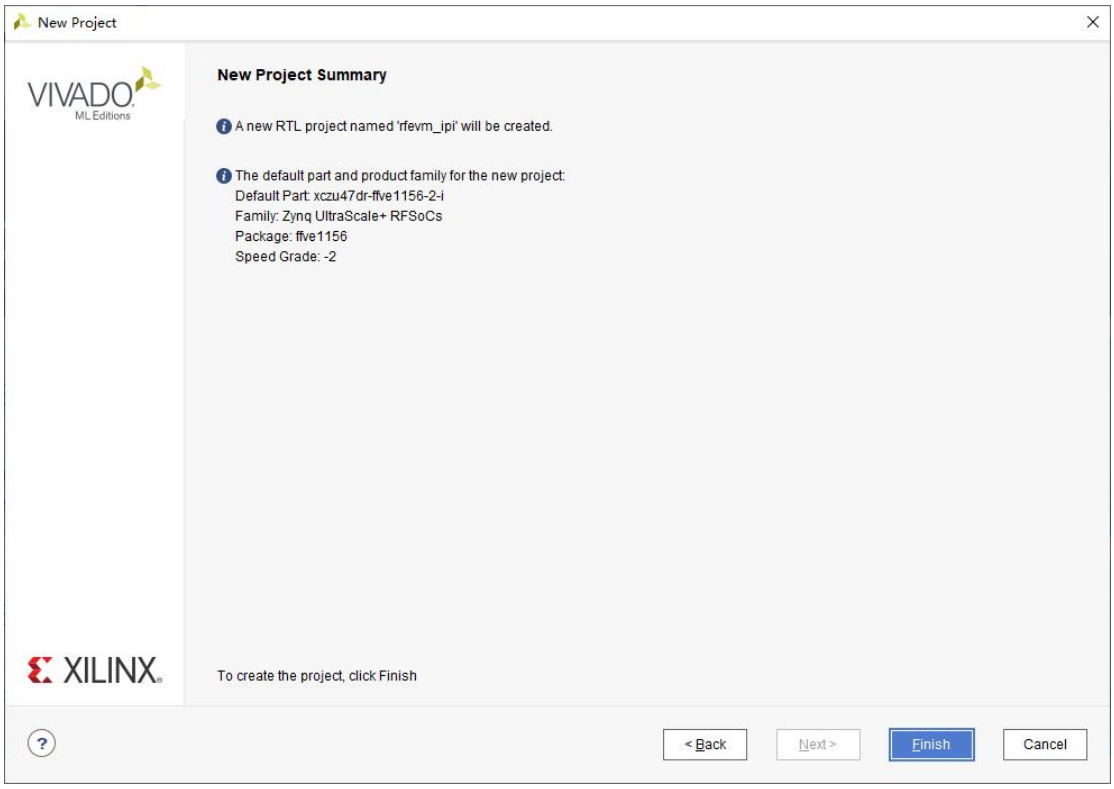
5) 选择 RTL Project，点击 Next



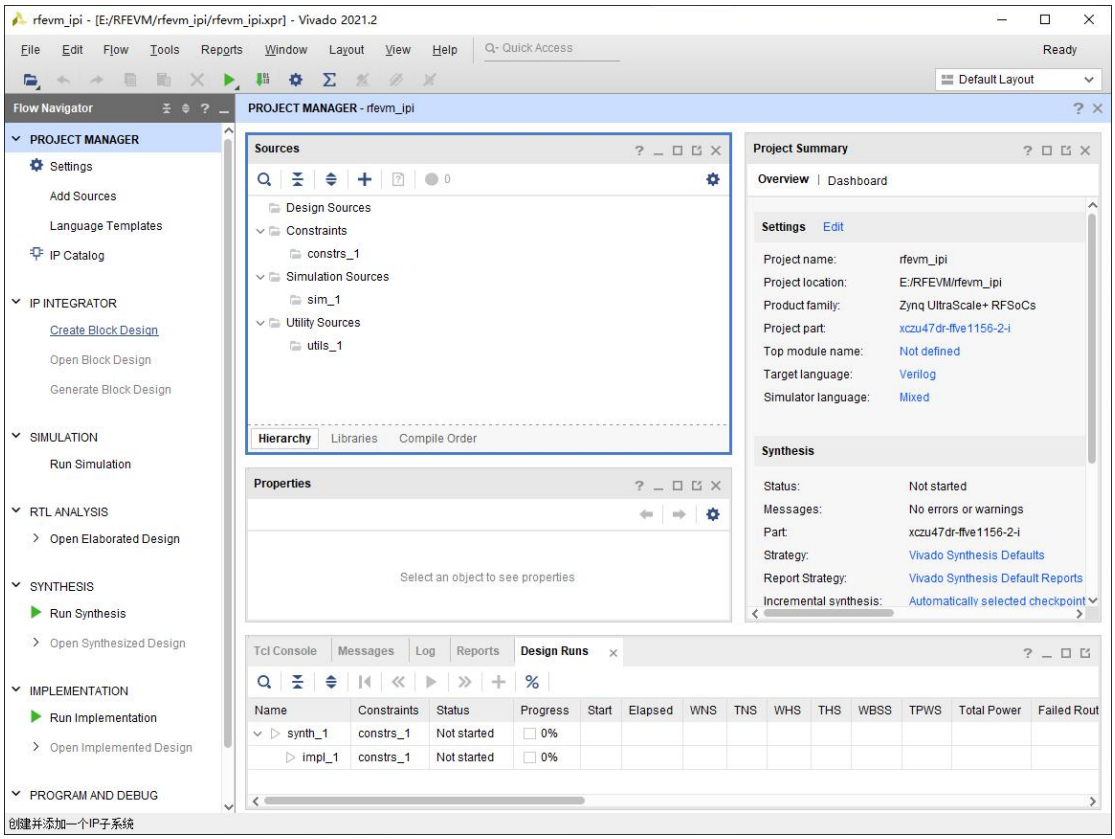
6) 选择芯片型号 xczu47dr-ffve1156-2-i，点击 Next



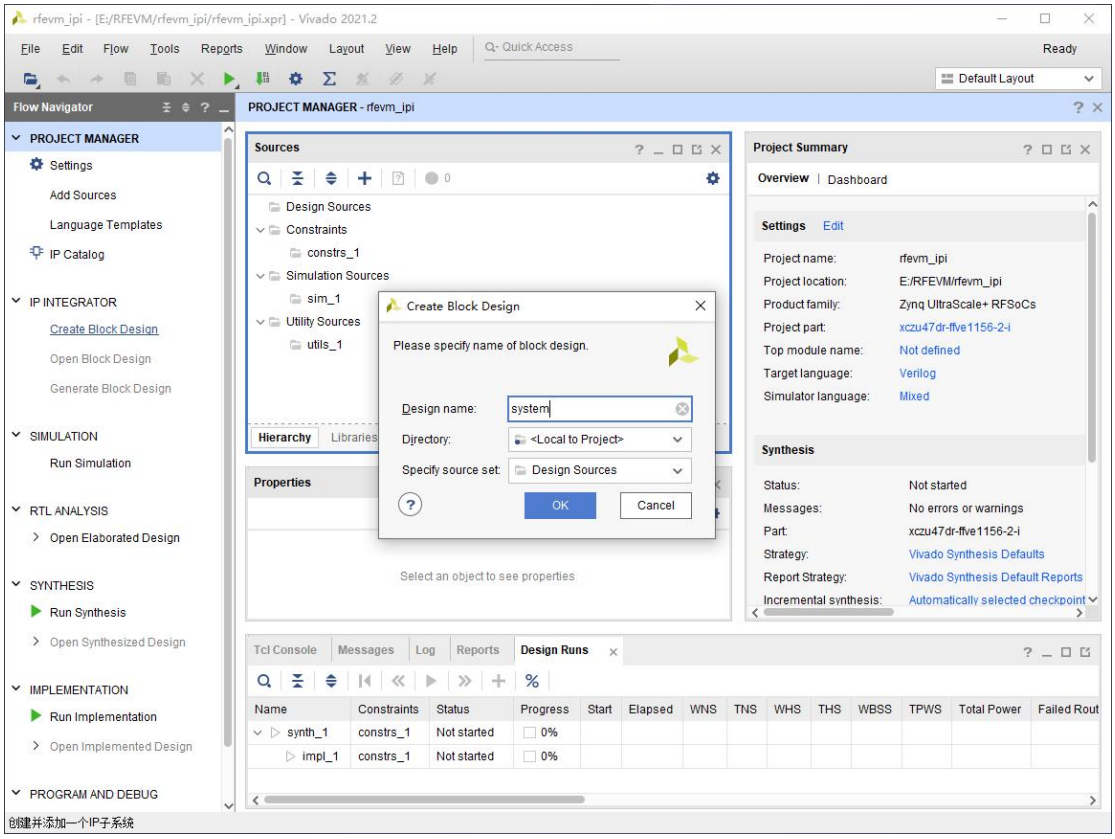
7) 点击 Finish



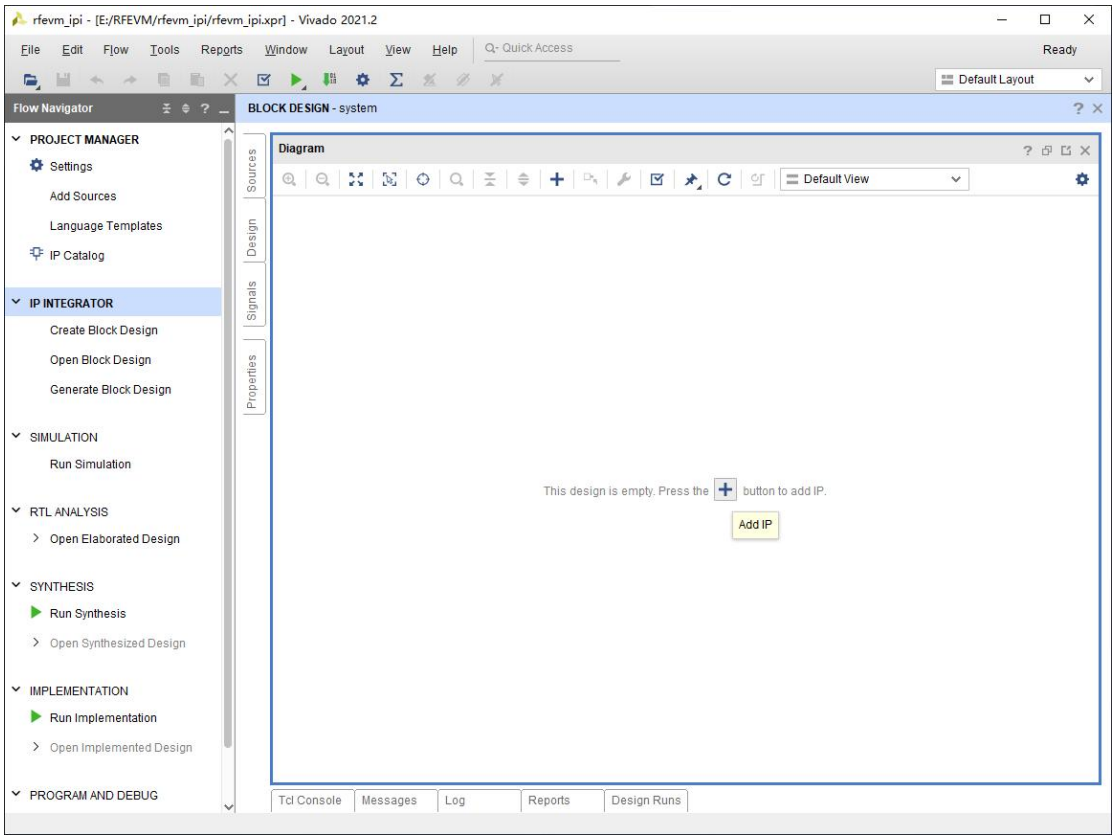
8) 在 IPI（IP INTEGRATOR）下选择 Create Block Design



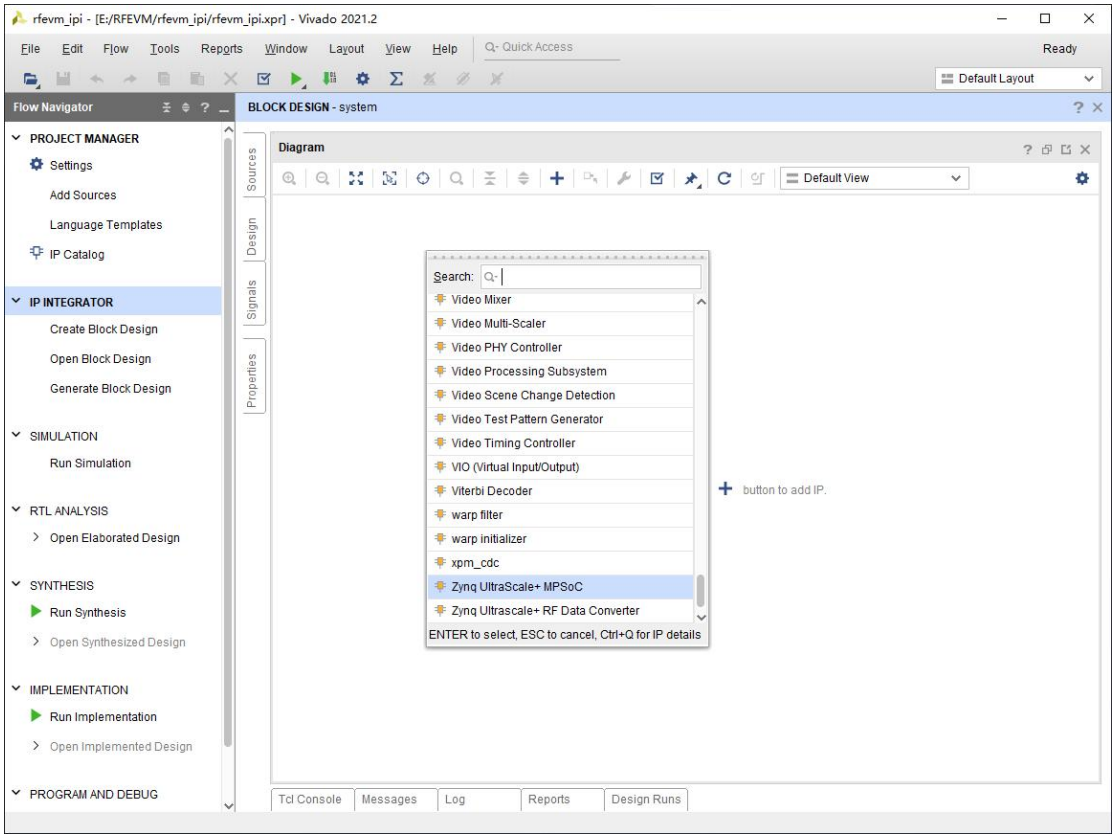
9) 设置设计名称：system，点击 OK



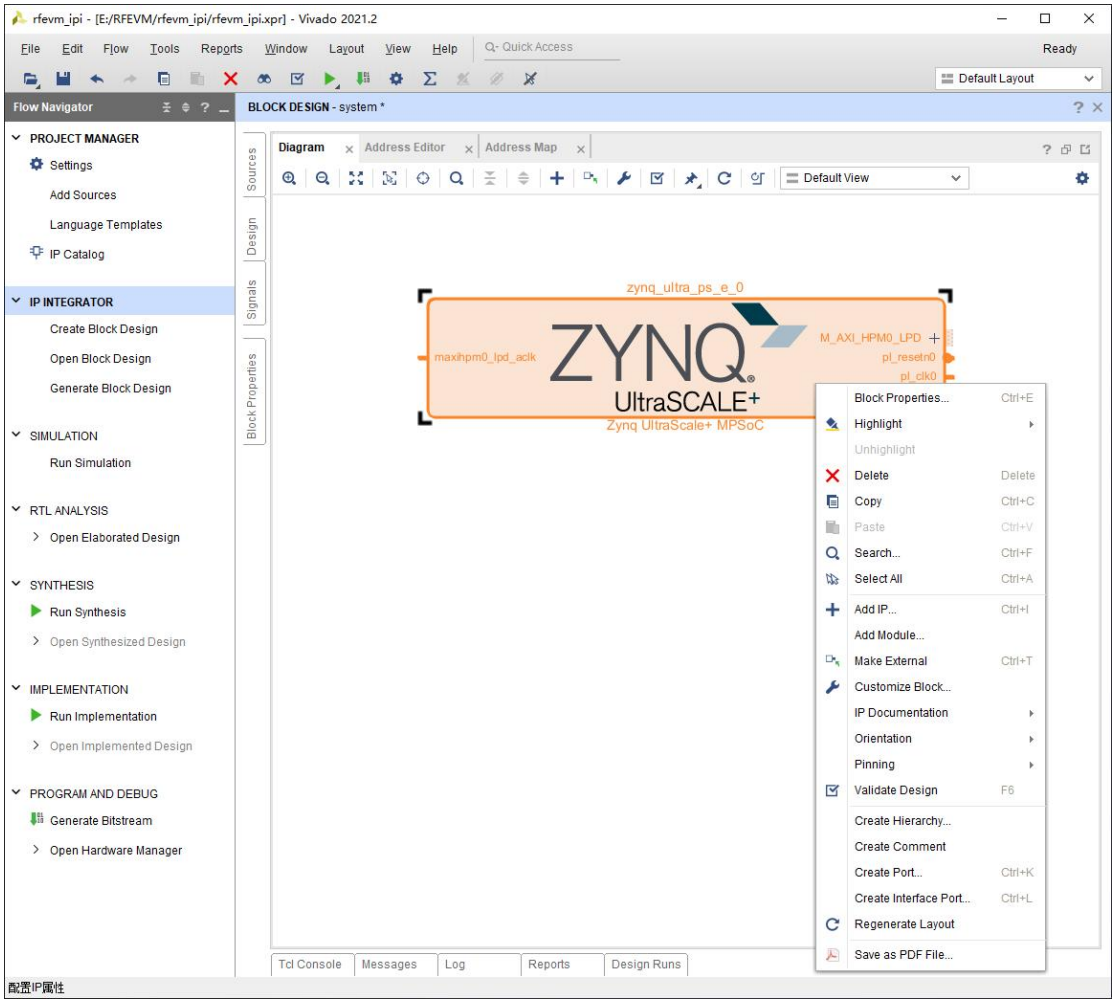
10) 点击+（Add IP）



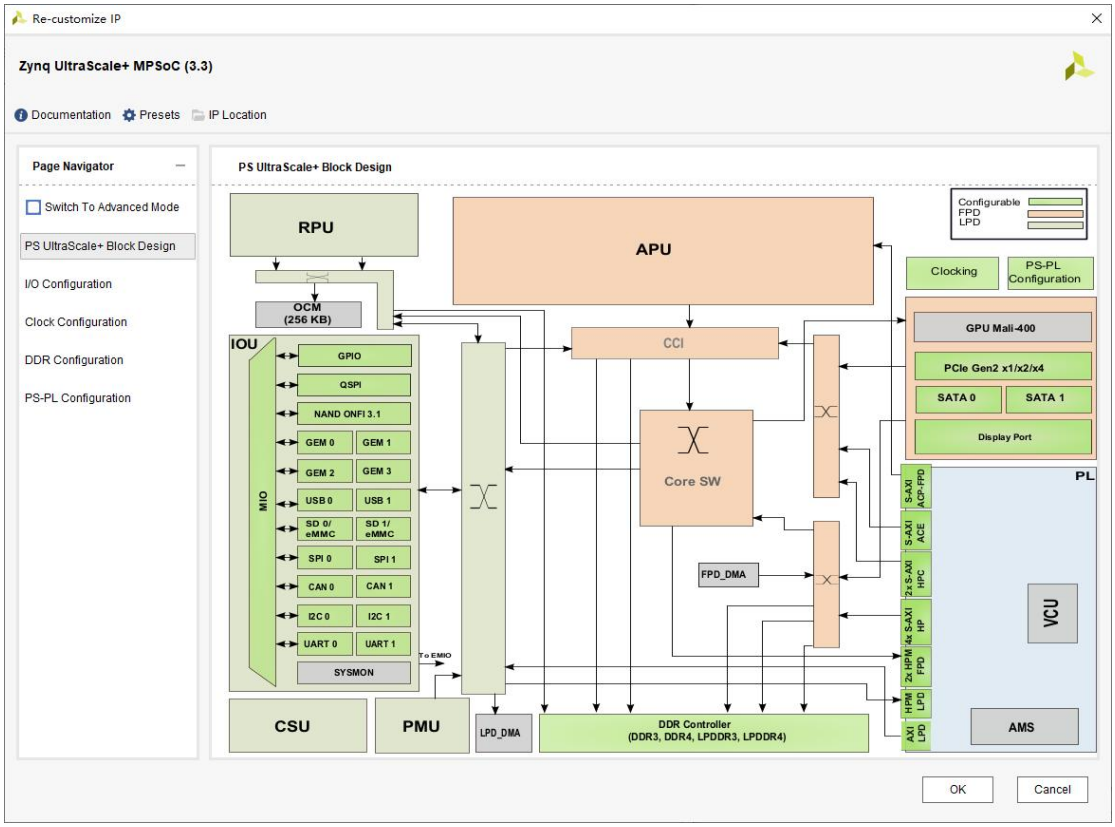
11) 选择 Zynq UltraScale+ MPSoC



12) 双击 Zynq UltraScale+ MPSoC 或者右键选择 Customize Block...

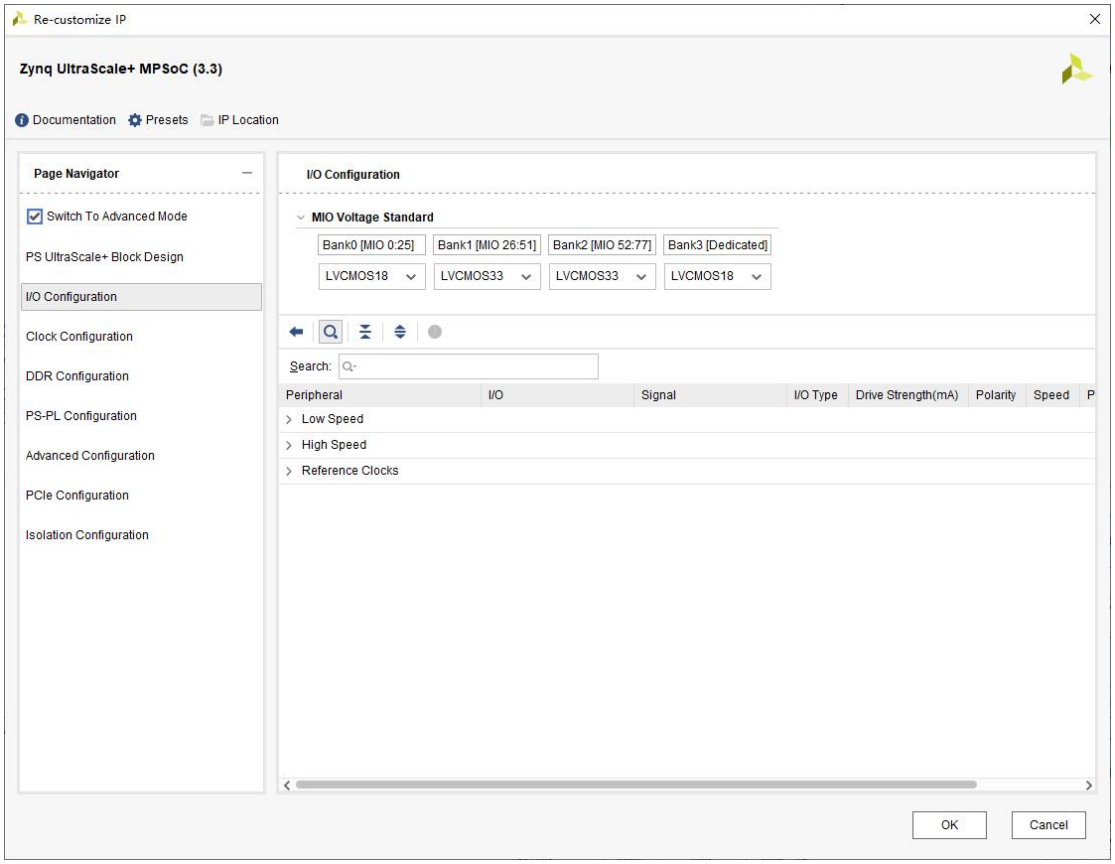


13) 下面会显示 MPSoC 硬核的架构图



◆ 在 IO Configuration 窗口

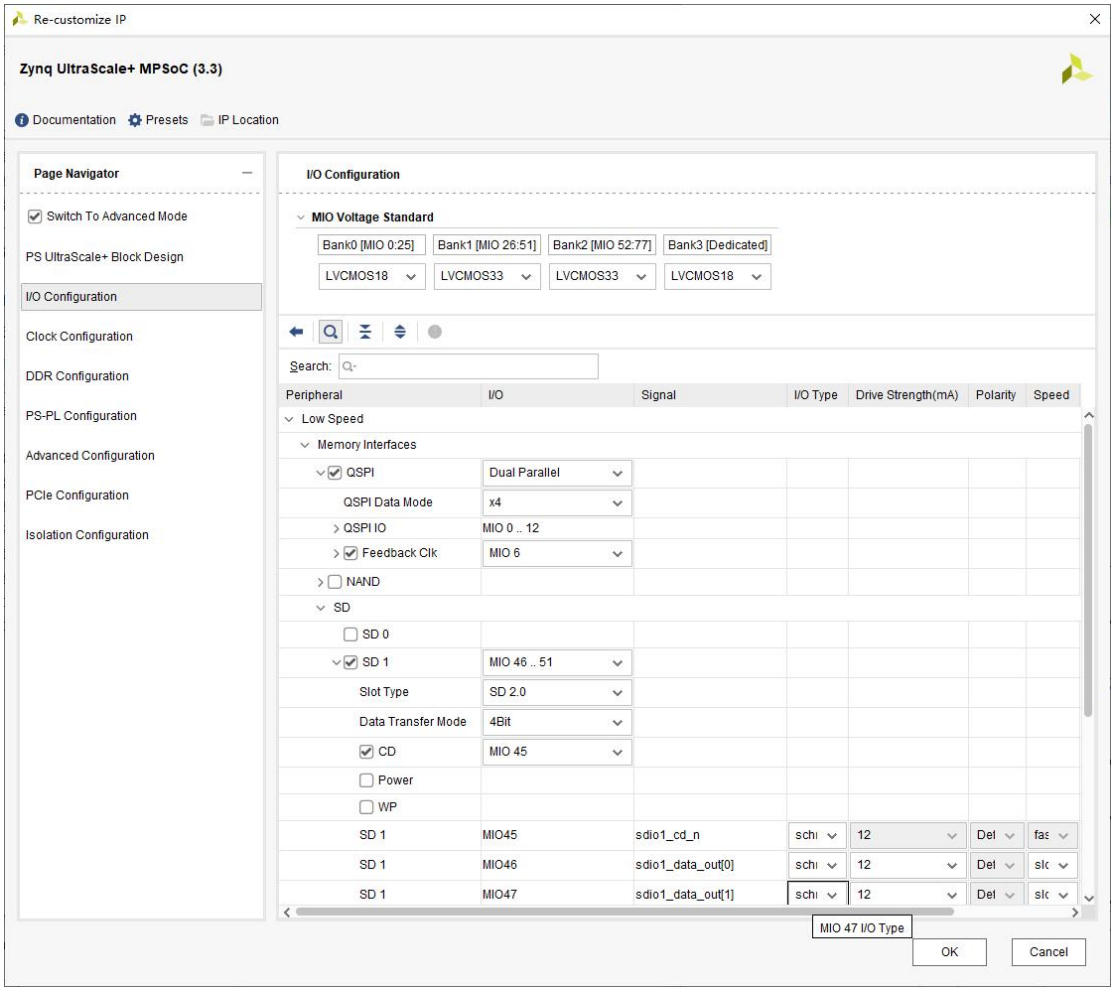
配置 Bank0、Bank3 电压为 LVCMOS18，Bank1、Bank2 电压为 LVCMOS33



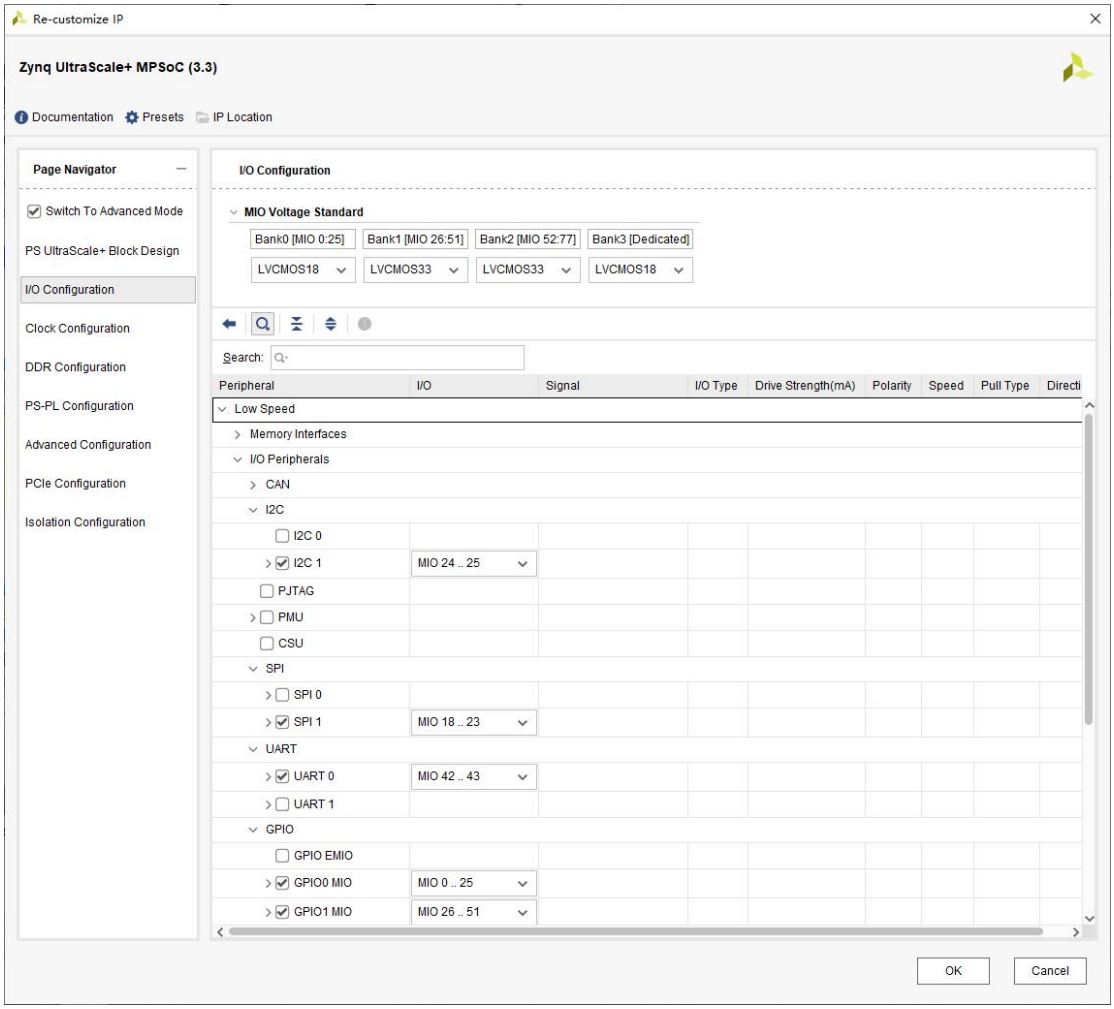
- 在 Low Speed->Memory Interface 界面，勾选 QSPI 和 SD 1；

  QSPI：Dual Parallel、x4

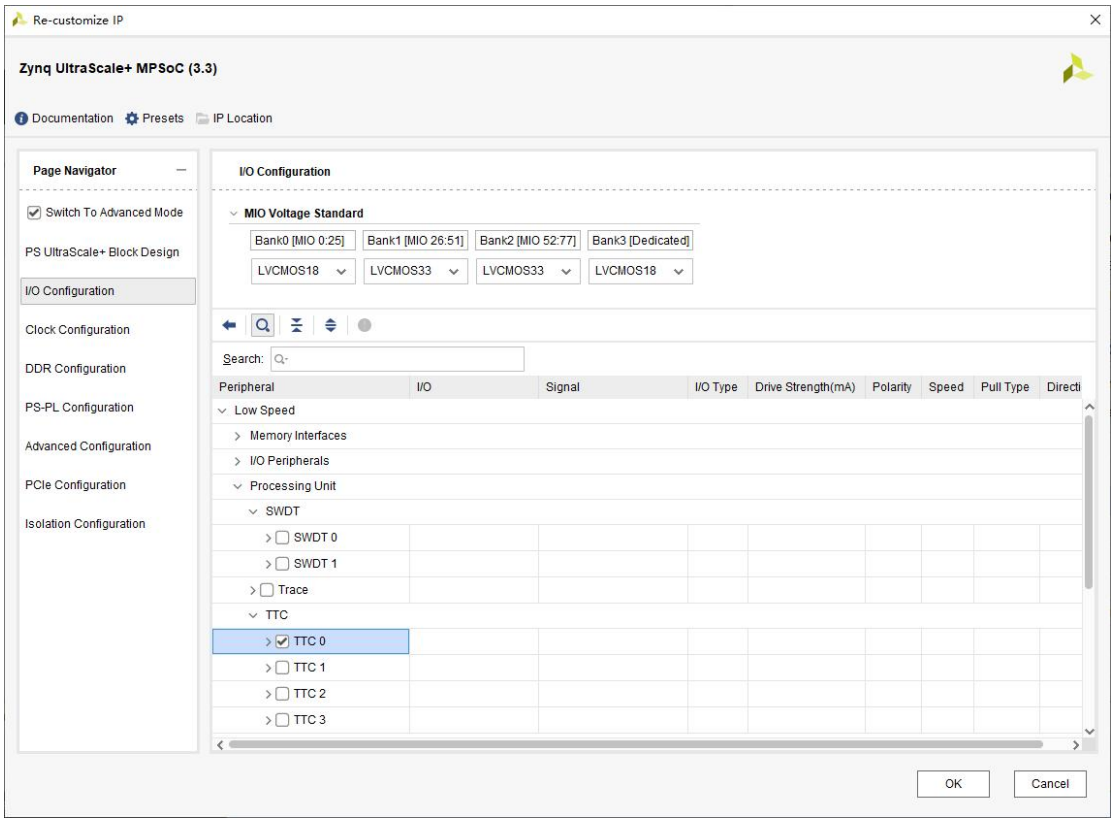
  SD 1：MIO46..51、SD2.0、使能 CD



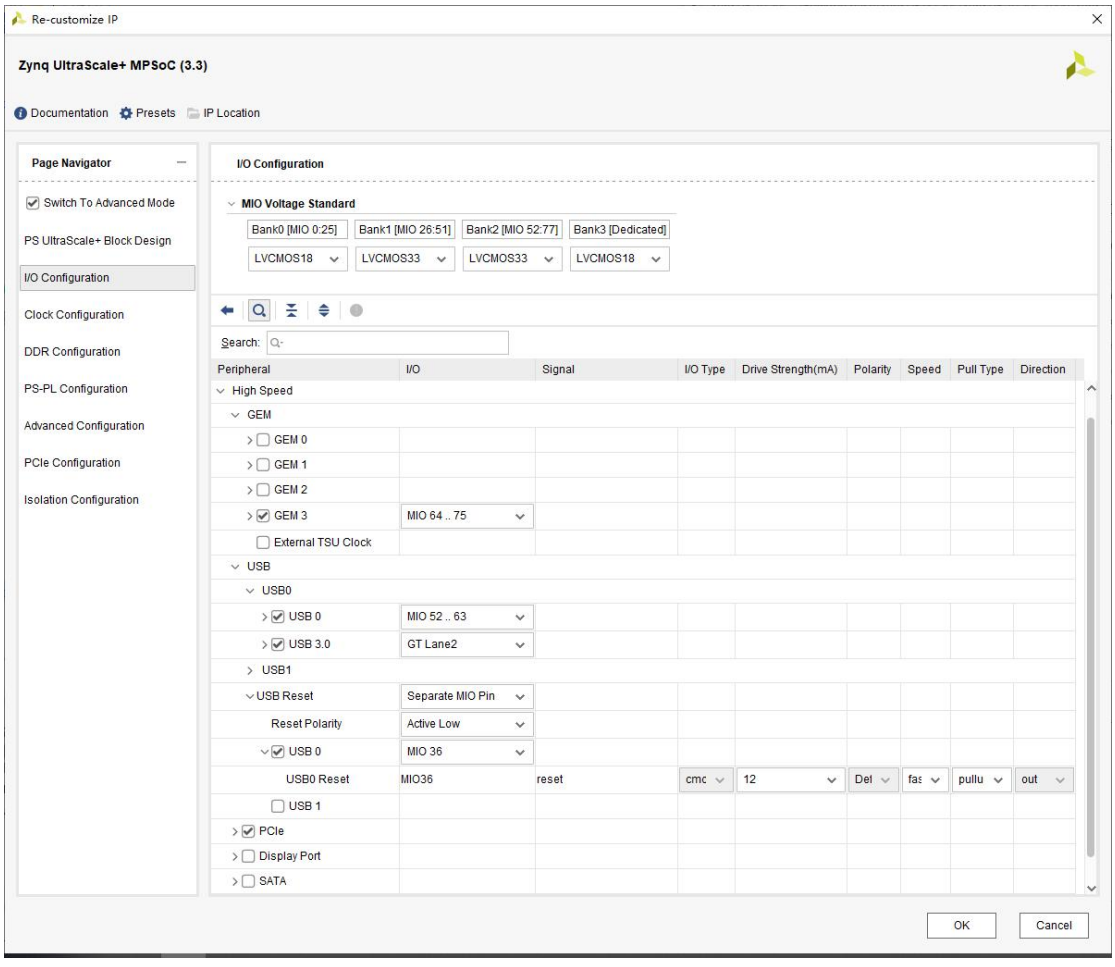
- 在 Low Speed->I/O Peripherals 界面，勾选 I2C 1、SPI 1、UART 0、GPIO0 MIO、GPIO1 MIO；

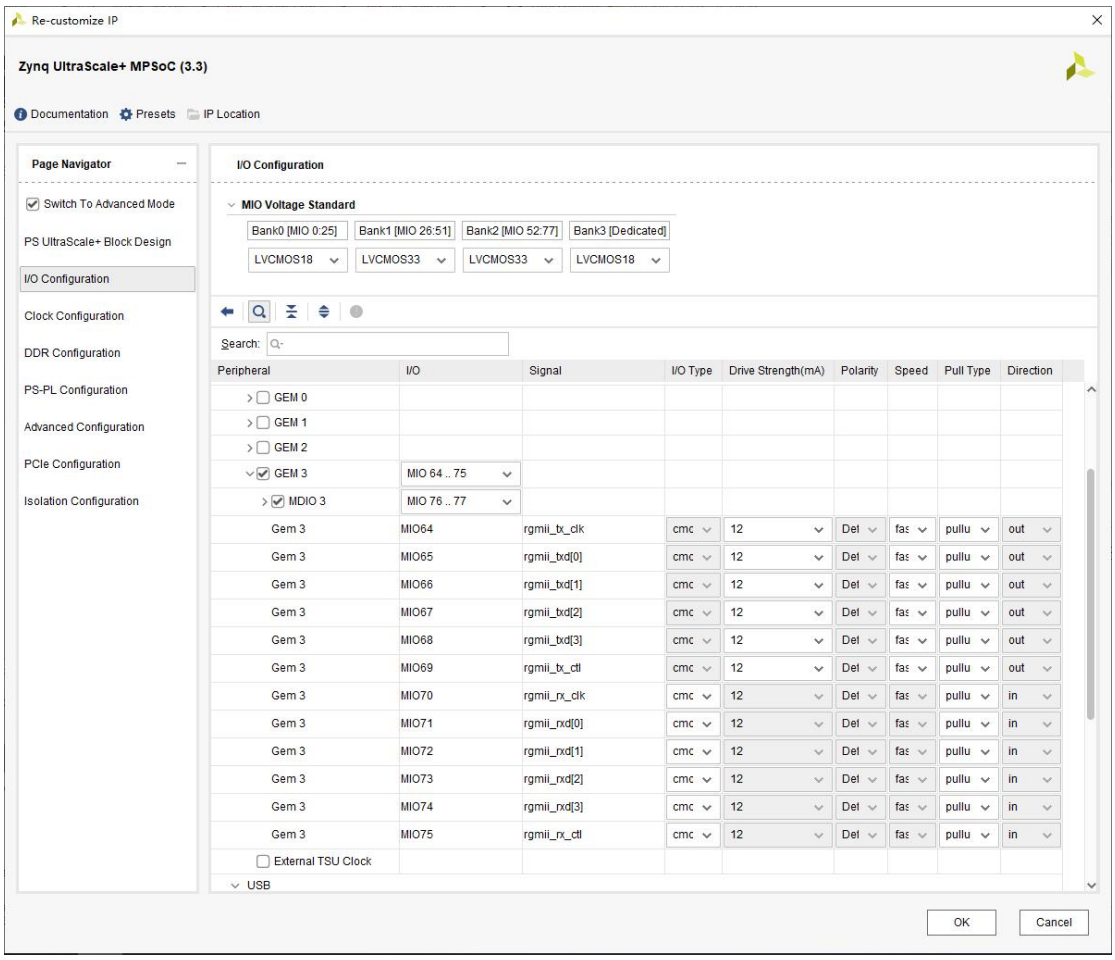


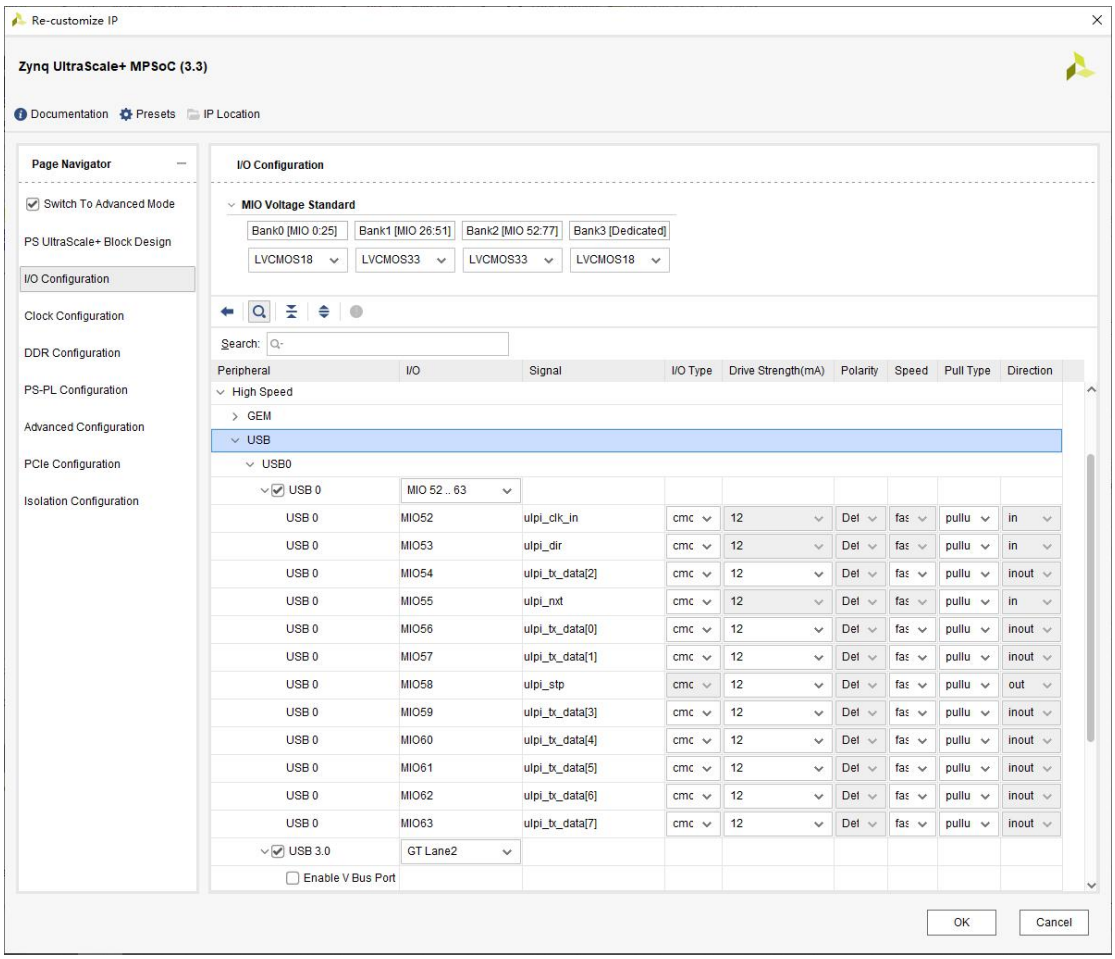
在 Low Speed->Processing Uint 界面，勾选 TTC 0；

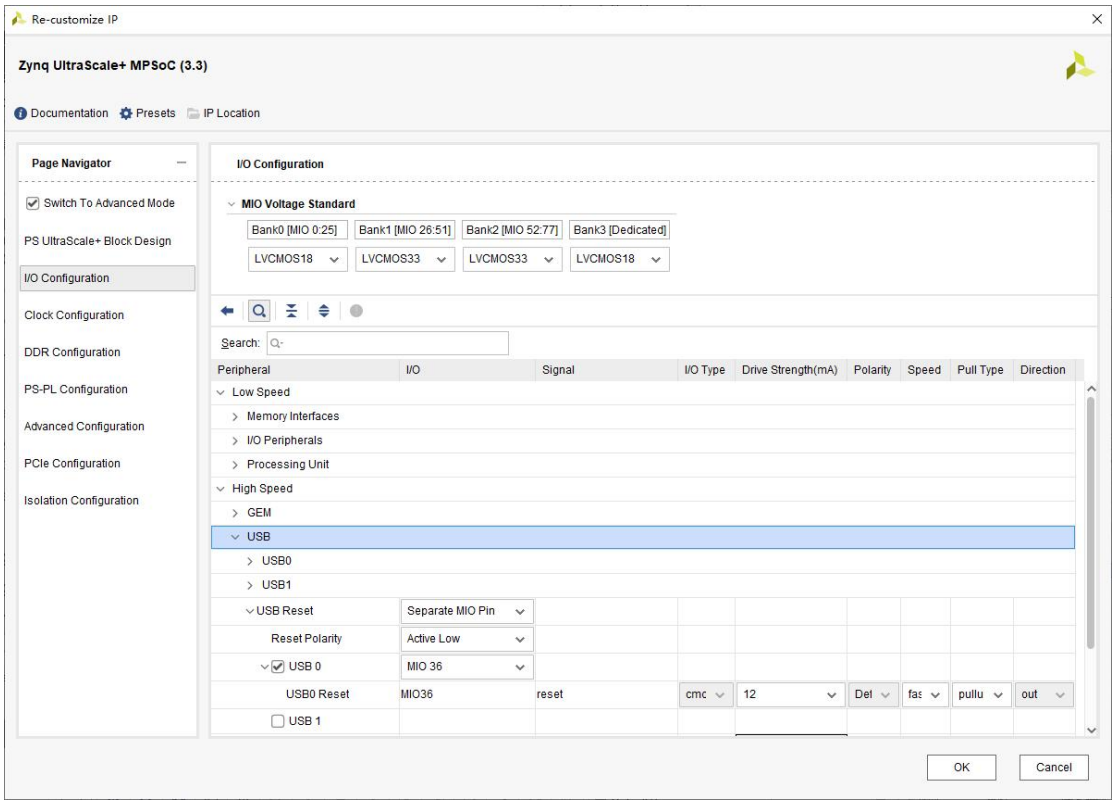


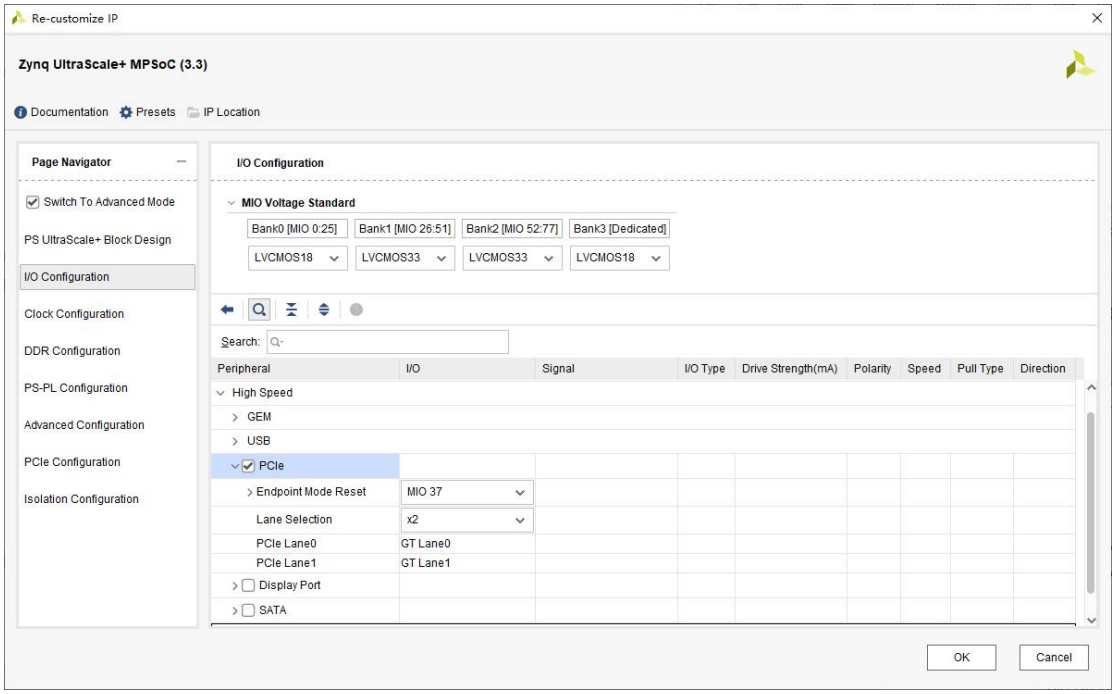
在 High Speed 界面，勾选 GEM 3、USB 0 、USB3.0、PCIe；




Re-customize IP
Zynq UltraScale+ MPSoC (3.3)
Documentation
Presets
IP Location
Page Navigator
Switch To Advanced Mode
PS UltraScale+ Block Design
I/O Configuration
Clock Configuration
DDR Configuration
PS-PL Configuration
Advanced Configuration
PCIe Configuration
Isolation Configuration
I/O Configuration
MIO Voltage Standard
Bank0 [MIO 0:25]
Bank1 [MIO 26:51]
Bank2 [MIO 52:77]
Bank3 [Dedicated]
LVCMOS18
LVCMOS33
LVCMOS33
LVCMOS18
Search:
Peripheral
I/O
Signal
I/O Type
Drive Strength(mA)
Polarity
Speed
Pull Type
Direction
GEM 0
GEM 1
GEM 2
GEM 3
MIO 64 .. 75
MDIO 3
MIO 76 .. 77
Gem 3 MIO64 rgmii_tx_clk
Gem 3 MIO65 rgmii_txd[0]
Gem 3 MIO66 rgmii_txd[1]
Gem 3 MIO67 rgmii_txd[2]
Gem 3 MIO68 rgmii_txd[3]
Gem 3 MIO69 rgmii_tx_ctl
Gem 3 MIO70 rgmii_rx_clk
Gem 3 MIO71 rgmii_rxd[0]
Gem 3 MIO72 rgmii_rxd[1]
Gem 3 MIO73 rgmii_rxd[2]
Gem 3 MIO74 rgmii_rxd[3]
Gem 3 MIO75 rgmii_rx_ctl
External TSU Clock
USB
OK
Cancel

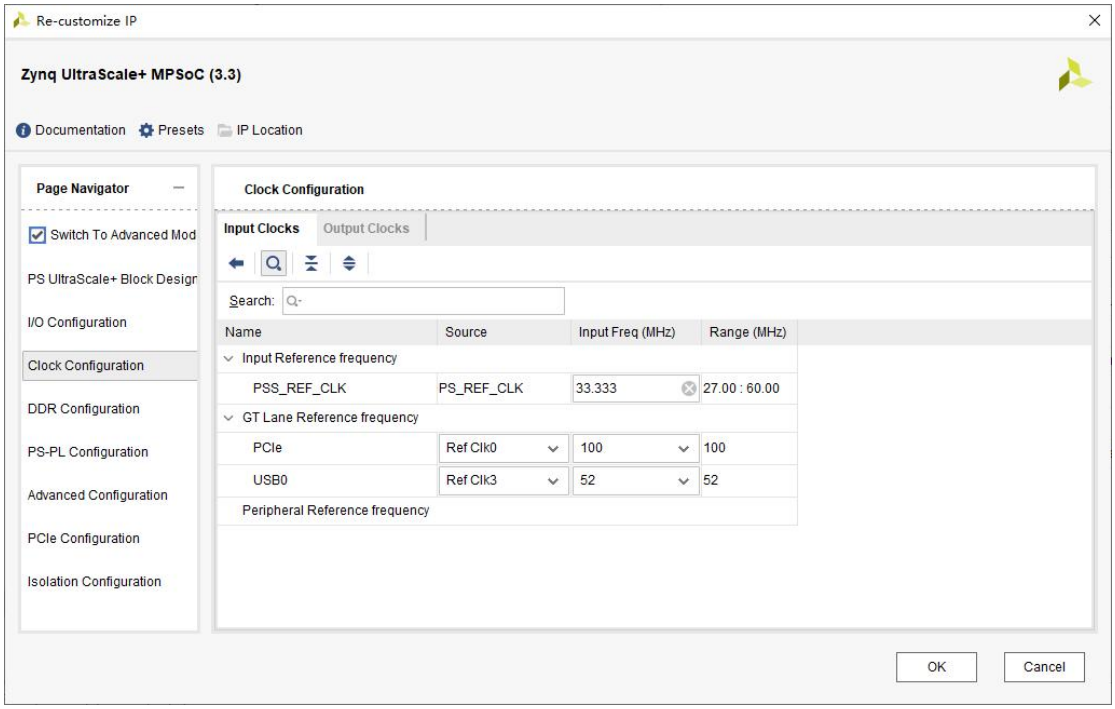

Re-customize IP
Zynq UltraScale+ MPSoC (3.3)
Documentation
Presets
IP Location
Page Navigator
Switch To Advanced Mode
PS UltraScale+ Block Design
I/O Configuration
Clock Configuration
DDR Configuration
PS-PL Configuration
Advanced Configuration
PCIe Configuration
Isolation Configuration
I/O Configuration
MIO Voltage Standard
Bank0 [MIO 0:25]
Bank1 [MIO 26:51]
Bank2 [MIO 52:77]
Bank3 [Dedicated]
LVCMOS18
LVCMOS33
LVCMOS33
LVCMOS18
Search:
Peripheral
I/O
Signal
I/O Type
Drive Strength(mA)
Polarity
Speed
Pull Type
Direction
High Speed
GEM
USB
USB0
USB 0
MIO 52 .. 63
USB 0
MIO52
ulpi_clk_in
USB 0
MIO53
ulpi_dir
USB 0
MIO54
ulpi_tx_data[2]
USB 0
MIO55
ulpi_nxt
USB 0
MIO56
ulpi_tx_data[0]
USB 0
MIO57
ulpi_tx_data[1]
USB 0
MIO58
ulpi_stp
USB 0
MIO59
ulpi_tx_data[3]
USB 0
MIO60
ulpi_tx_data[4]
USB 0
MIO61
ulpi_tx_data[5]
USB 0
MIO62
ulpi_tx_data[6]
USB 0
MIO63
ulpi_tx_data[7]
USB 3.0
GT Lane2
Enable V Bus Port
OK
Cancel


Re-customize IP
Zynq UltraScale+ MPSoC (3.3)
Documentation
Presets
IP Location
Page Navigator
Switch To Advanced Mode
PS UltraScale+ Block Design
I/O Configuration
Clock Configuration
DDR Configuration
PS-PL Configuration
Advanced Configuration
PCIe Configuration
Isolation Configuration
I/O Configuration
MIO Voltage Standard
Bank0 [MIO 0:25]
Bank1 [MIO 26:51]
Bank2 [MIO 52:77]
Bank3 [Dedicated]
LVCMOS18
LVCMOS33
LVCMOS33
LVCMOS18
Search:
Peripheral
I/O
Signal
I/O Type
Drive Strength(mA)
Polarity
Speed
Pull Type
Direction
Low Speed
Memory Interfaces
I/O Peripherals
Processing Unit
High Speed
GEM
USB
USB0
USB1
USB Reset
Separate MIO Pin
Reset Polarity
Active Low
USB 0
MIO 36
USB0 Reset
MIO36
reset
USB 1
OK
Cancel

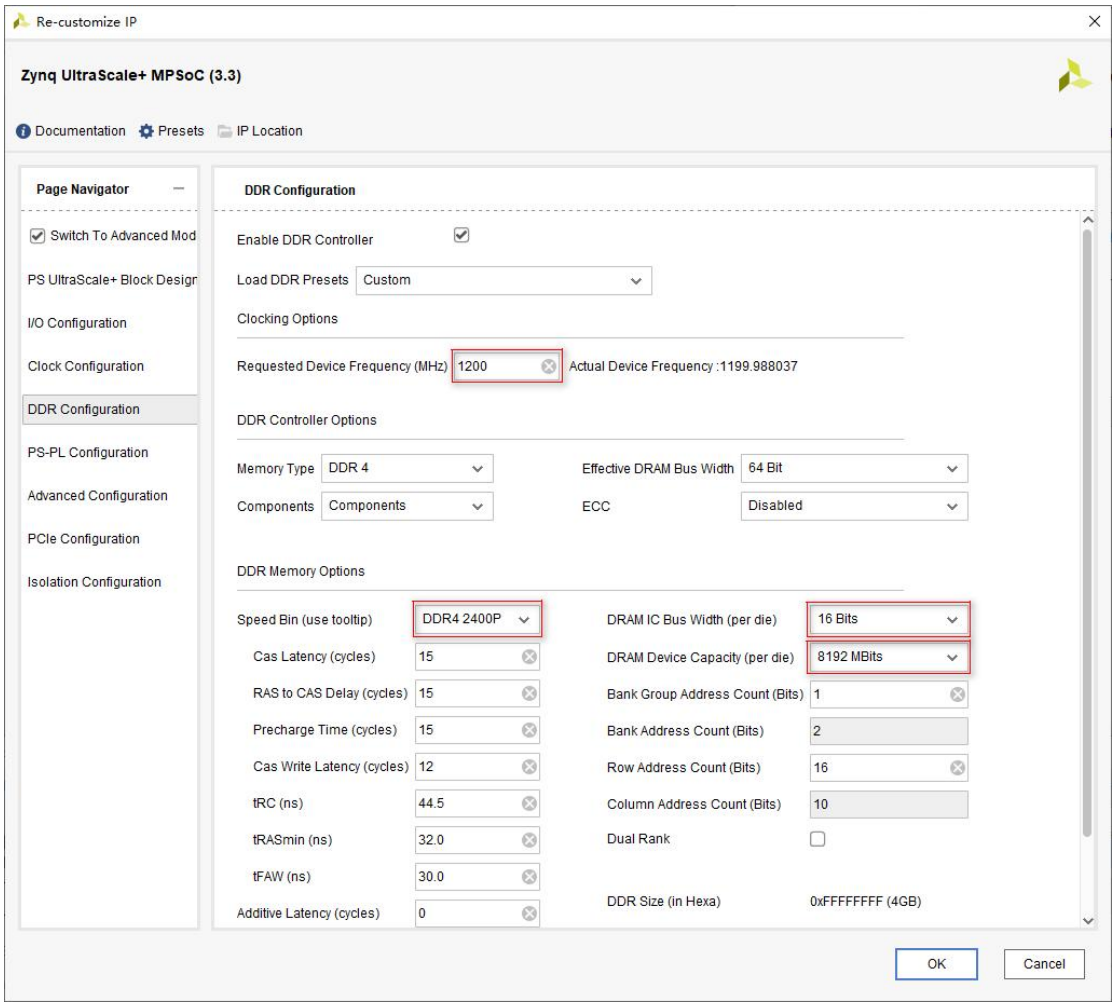


- 在 Clock Configuration 窗口

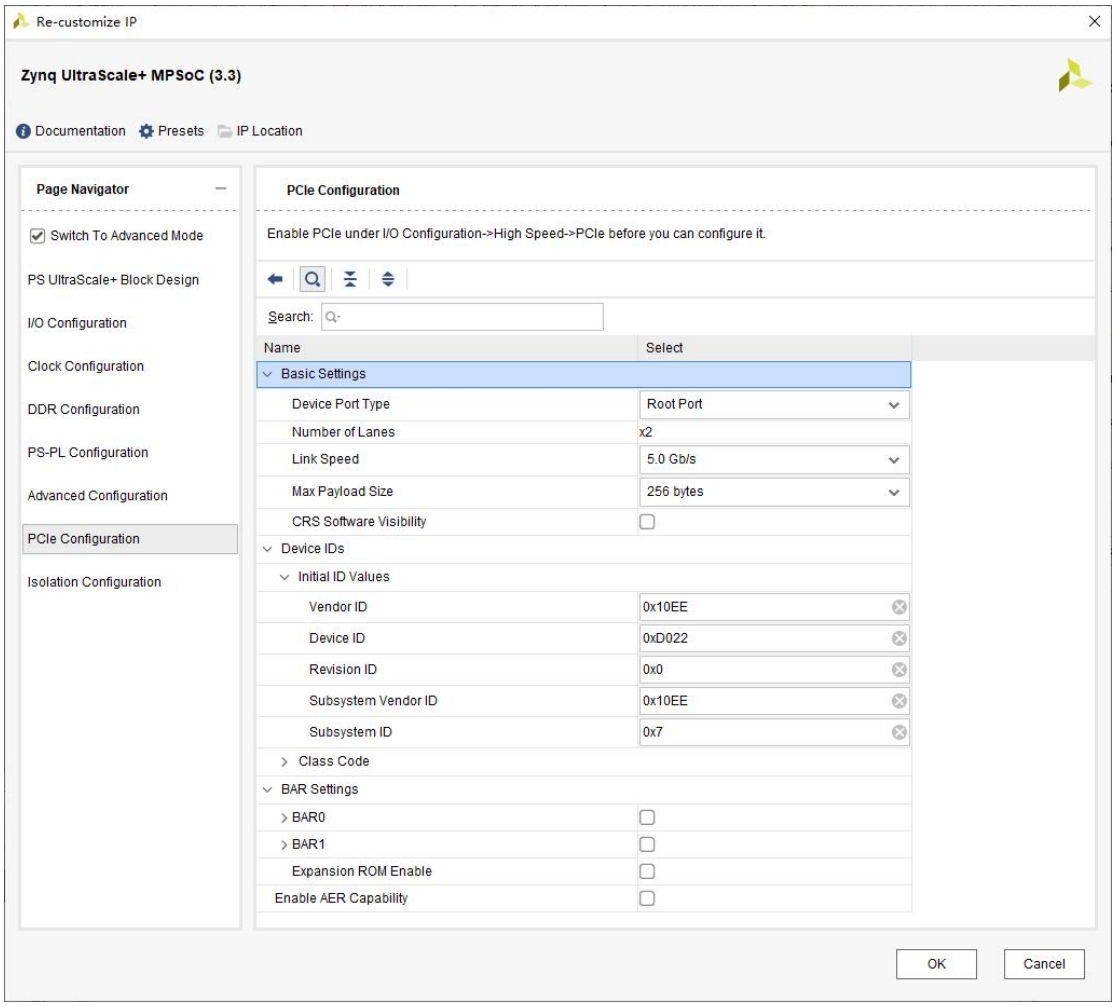
  按照下图配置输入时钟，其他默认即可；



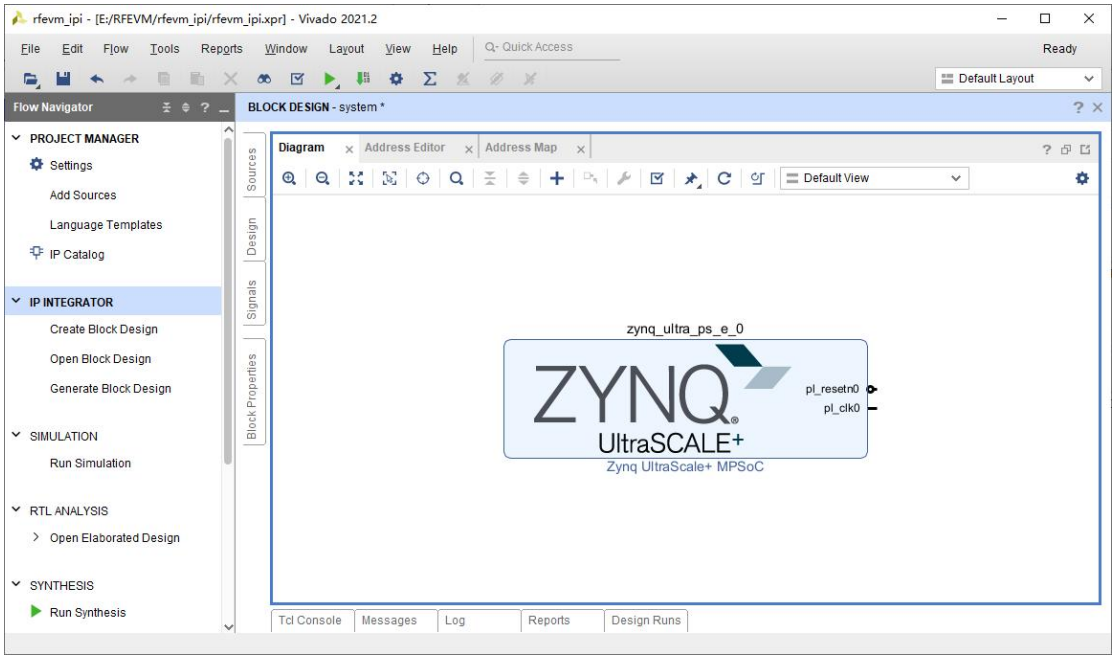
- 在 DDR Configuration 窗口，配置 DDR 的参数，如下图



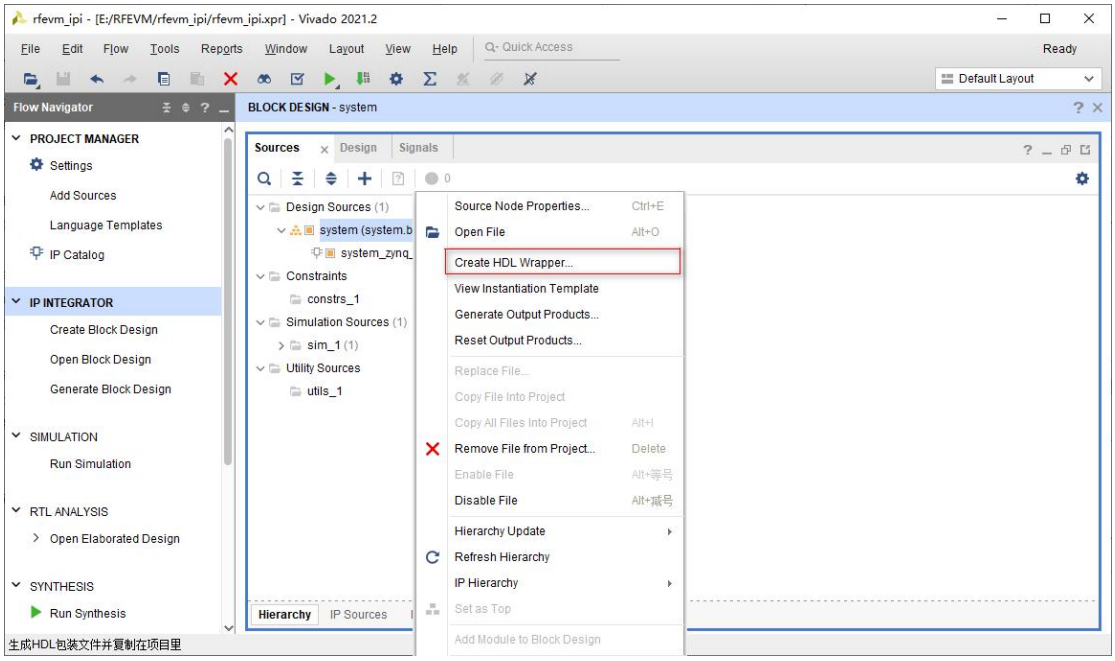
- 在 PCIe Configuration 窗口，主要配置设备端口类型和设备 ID，其他默认即可；



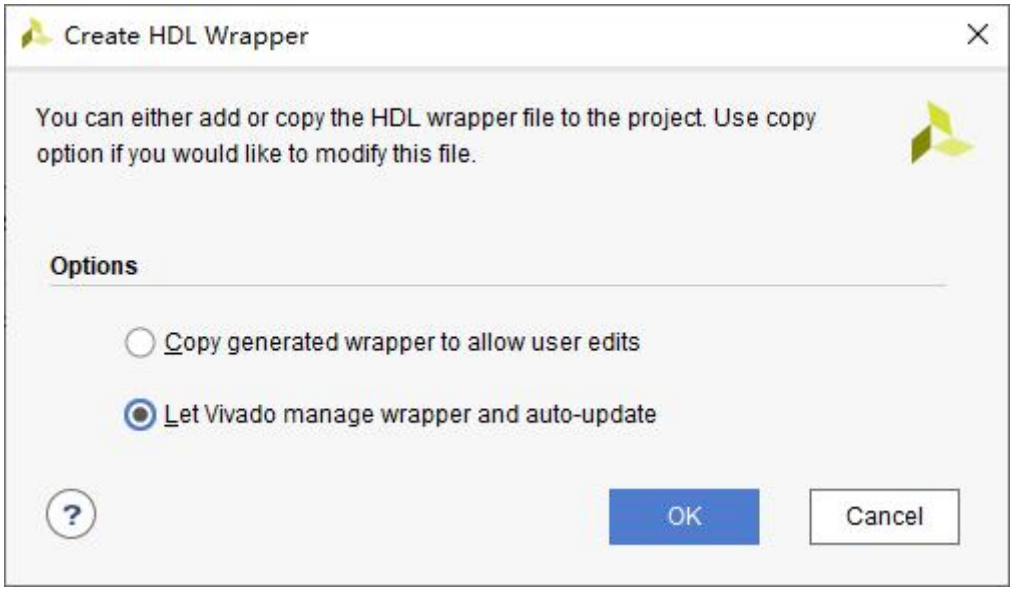
14) 点击 OK



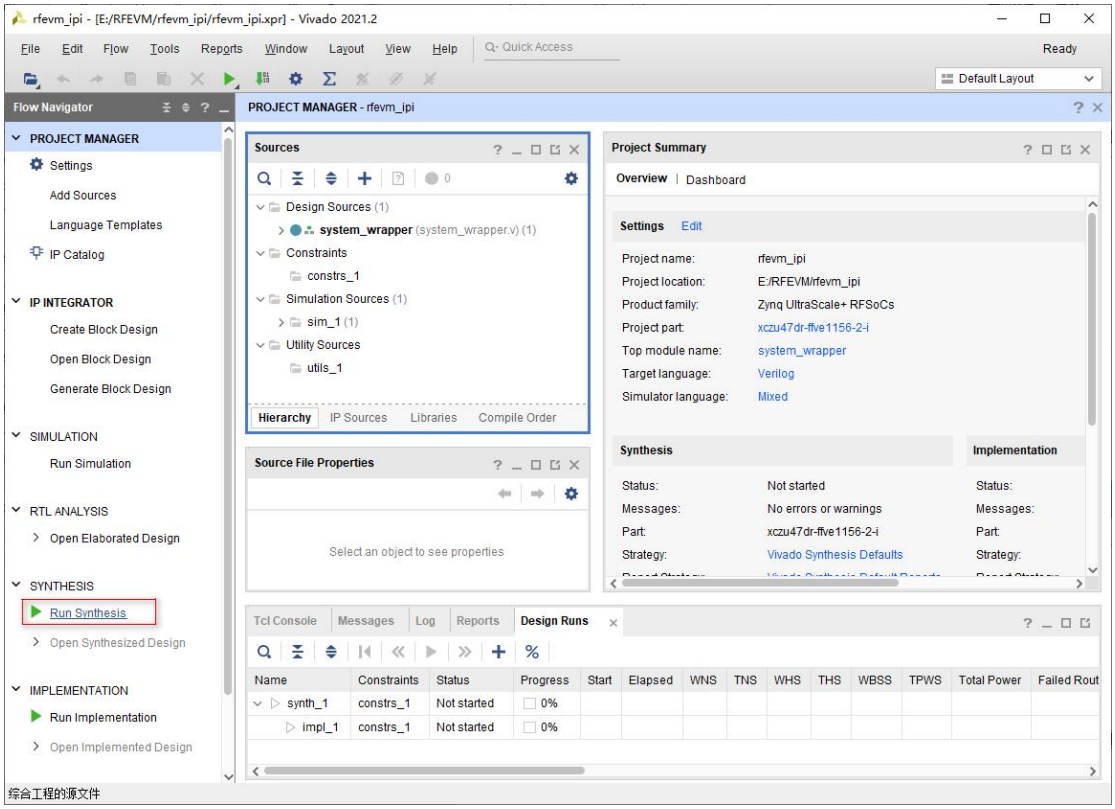
15) 在 system 上右键选择 Create HDL Wrapper



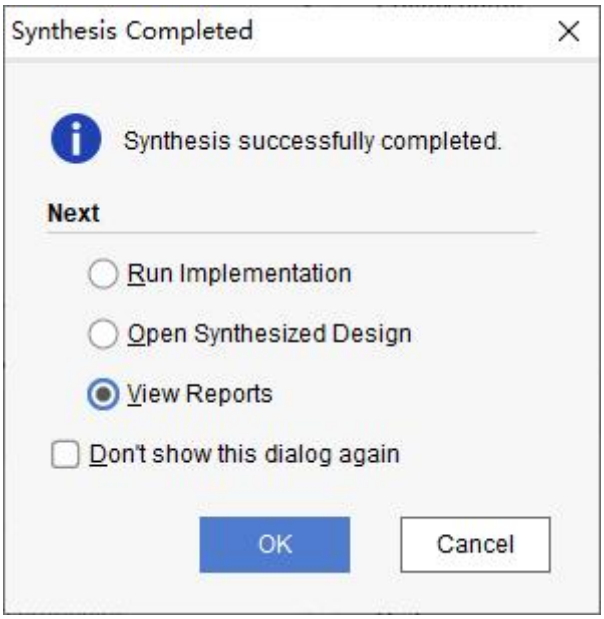
16) 选择 Let Vivado manage wrapper and auto-update



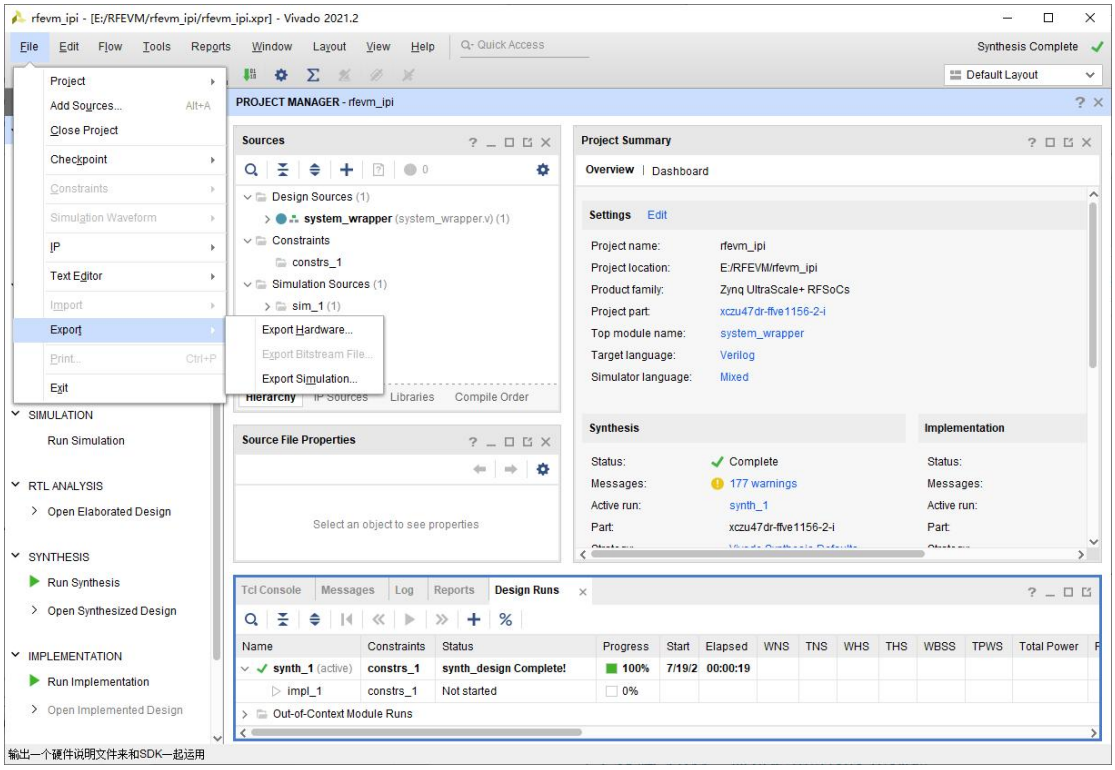
17) 点击 Run Synthesis（*这里我们只使用 PS 的资源，不用产生 bit*）



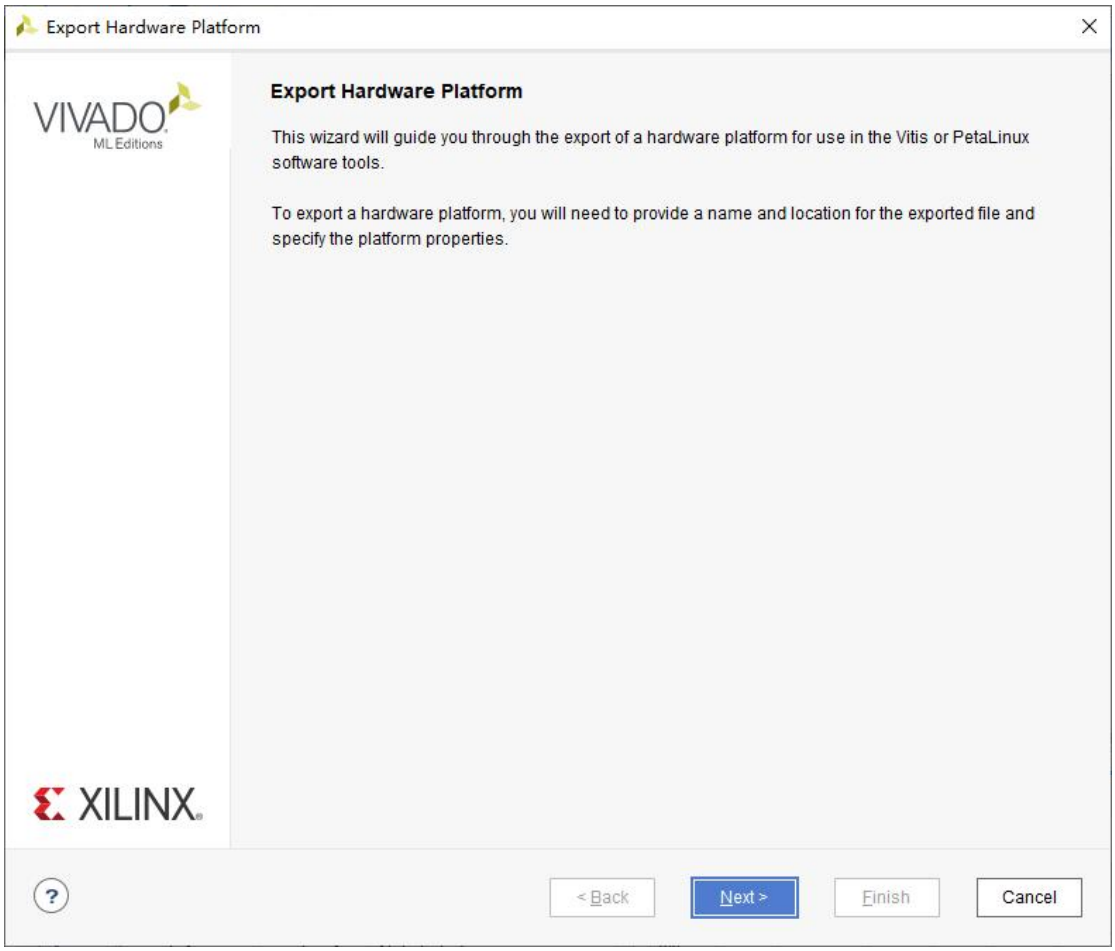
18) 综合完成之后，点击 OK



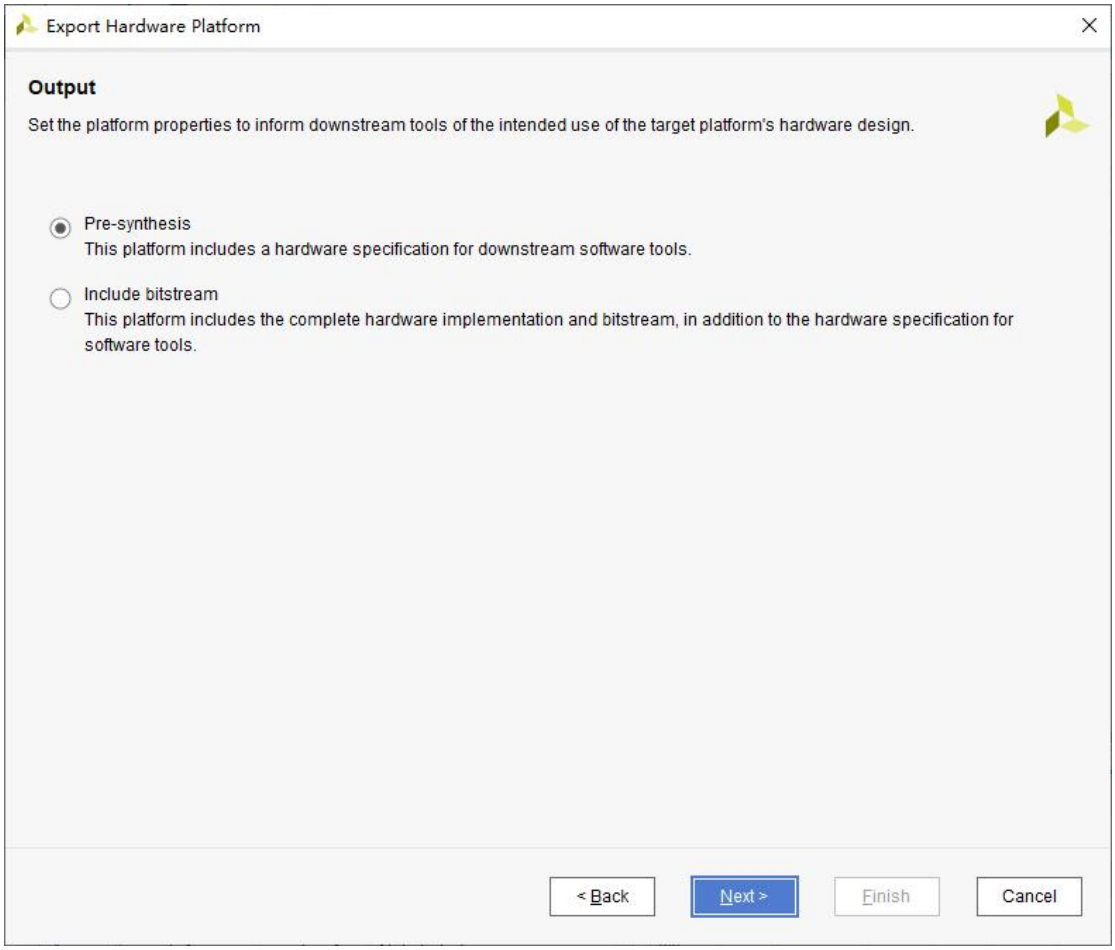
19) 选择 File->Export->Export Hardware


rfevm_ipi - [E:/RFEVM/rfevm_ipi/rfevm_ipi.xpr] - Vivado 2021.2
File Edit Flow Tools Reports Window Layout View Help
Synthesis Complete
Default Layout
PROJECT MANAGER - rfevm_ipi
Project
Add Sources... Alt+A
Close Project
Checkpoint
Constraints
Simulation Waveform
IP
Text Editor
Import
Export
Print... Ctrl+P
Exit
Export Hardware...
Export Bitstream File...
Export Simulation...
SIMULATION
Run Simulation
RTL ANALYSIS
Open Elaborated Design
SYNTHESIS
Run Synthesis
Open Synthesized Design
IMPLEMENTATION
Run Implementation
Open Implemented Design
Sources
Design Sources (1)
system_wrapper (system_wrapper.v) (1)
Constraints
constrs_1
Simulation Sources (1)
sim_1 (1)
Hierarchy IP Sources Libraries Compile Order
Source File Properties
Select an object to see properties
Project Summary
Overview | Dashboard
Settings Edit
Project name: rfevm_ipi
Project location: E:/RFEVM/rfevm_ipi
Product family: Zynq UltraScale+ RFSoCs
Project part: xczu47dr-ffve1156-2-i
Top module name: system_wrapper
Target language: Verilog
Simulator language: Mixed
Synthesis
Status: Complete
Messages: 177 warnings
Active run: synth_1
Part: xczu47dr-ffve1156-2-i
Implementation
Status:
Messages:
Active run:
Part:
Tcl Console Messages Log Reports Design Runs
Name Constraints Status Progress Start Elapsed WNS TNS WHS THS WBSS TPWS Total Power
synth_1 (active) constrs_1 synth_design Complete! 100% 7/19/2 00:00:19
impl_1 constrs_1 Not started 0%
Out-of-Context Module Runs
输出一个硬件说明文件来和SDK一起运用

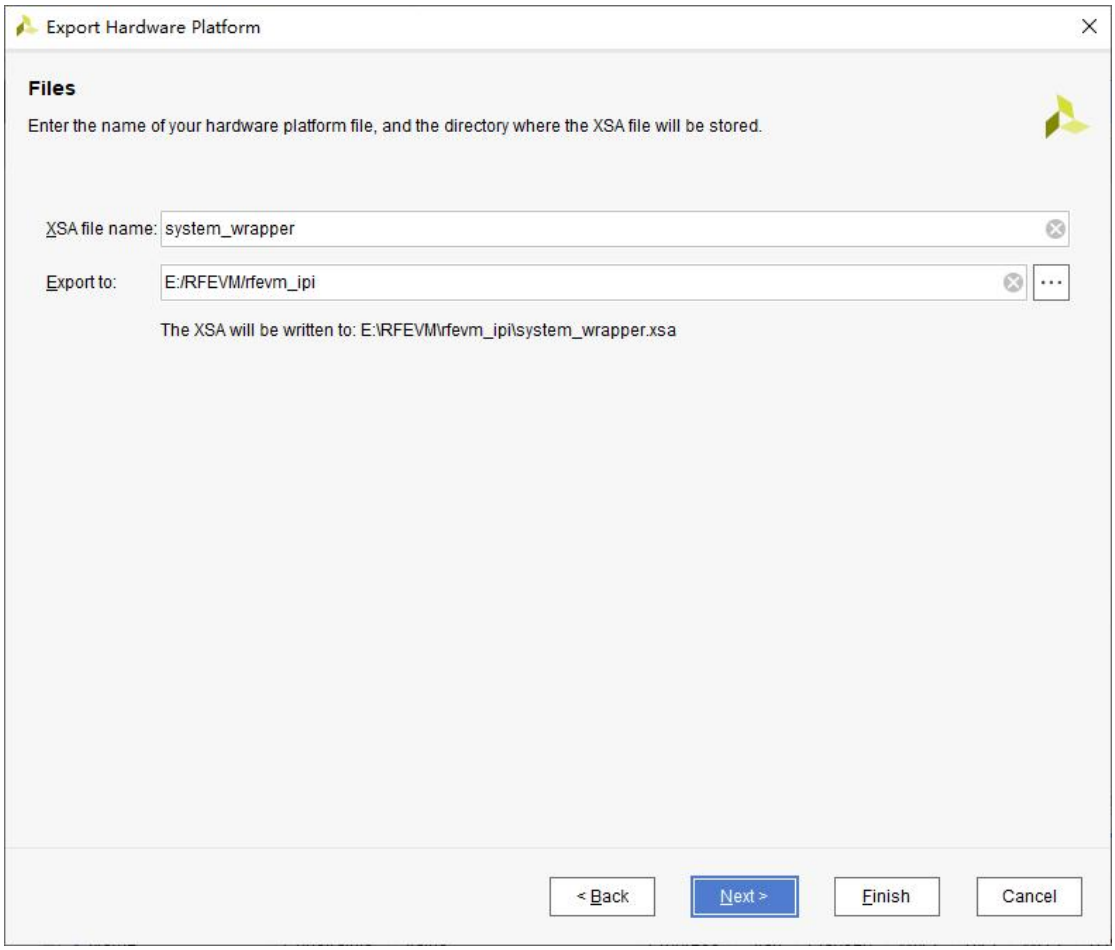
20) 点击 Next


Export Hardware Platform
VIVADO ML Editions
Export Hardware Platform
This wizard will guide you through the export of a hardware platform for use in the Vitis or PetaLinux software tools.
To export a hardware platform, you will need to provide a name and location for the exported file and specify the platform properties.
XILINX
< Back
Next >
Finish
Cancel

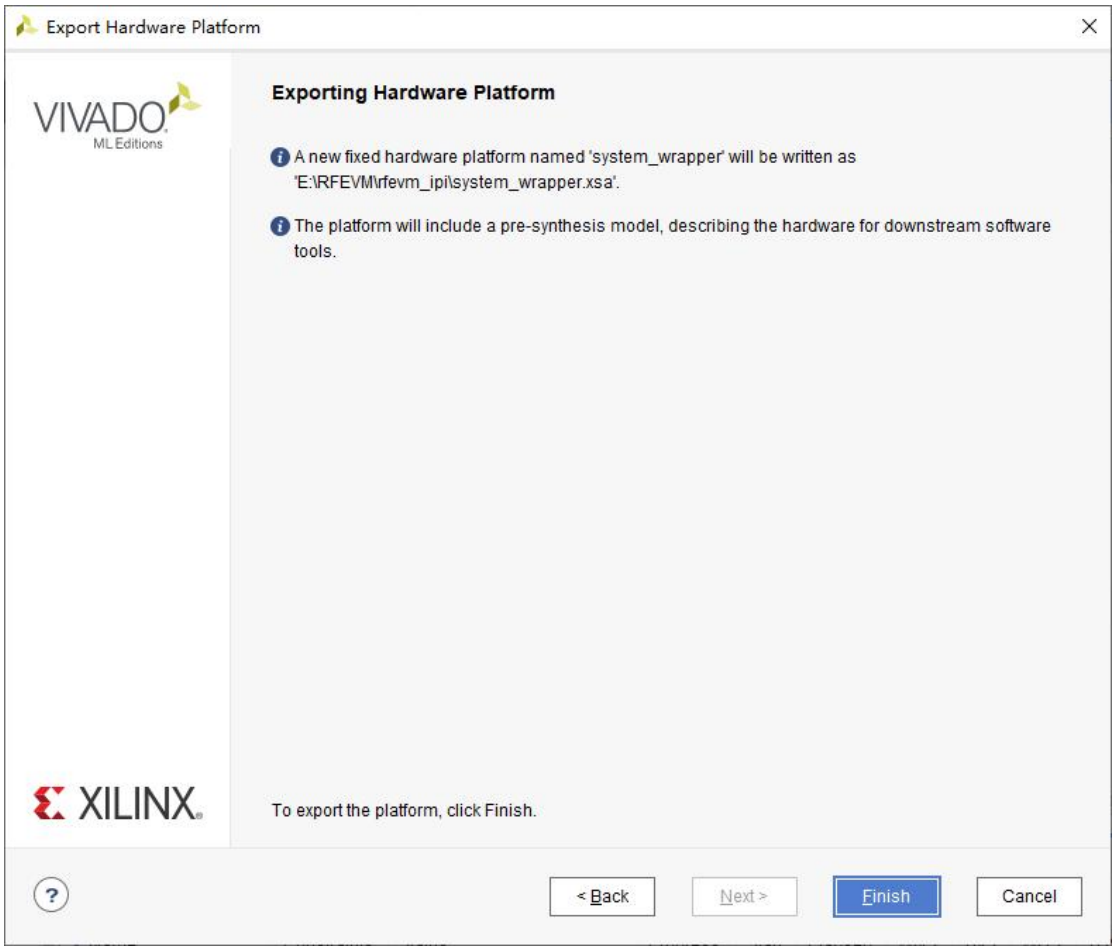
21) 这里我们没有 bit，默认配置，点击 Next



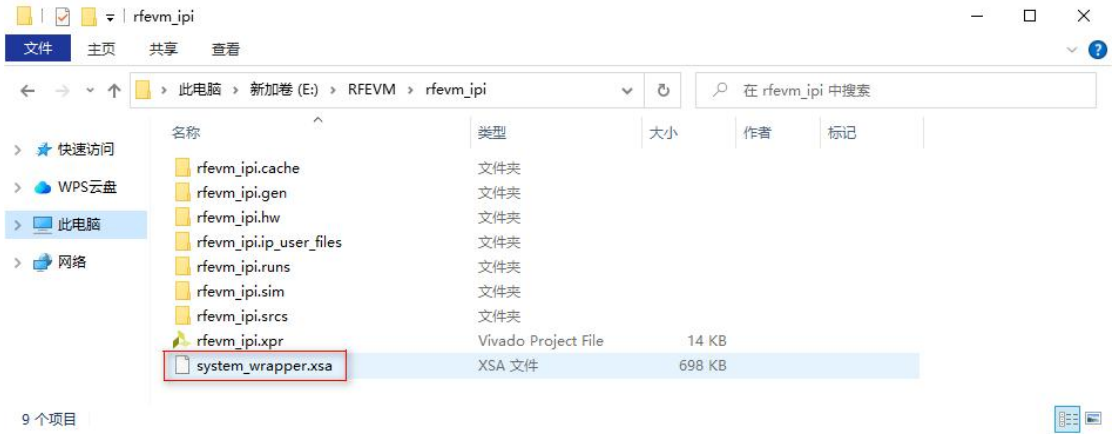
22) 设置名称和路径，点击 Next


Export Hardware Platform
Files
Enter the name of your hardware platform file, and the directory where the XSA file will be stored.
XSA file name: system_wrapper
Export to: E:/RFEVM/rfevm_ipi
The XSA will be written to: E:\RFEVM\rfevm_ipi\system_wrapper.xsa
< Back
Next >
Finish
Cancel

23) 点击 Finish



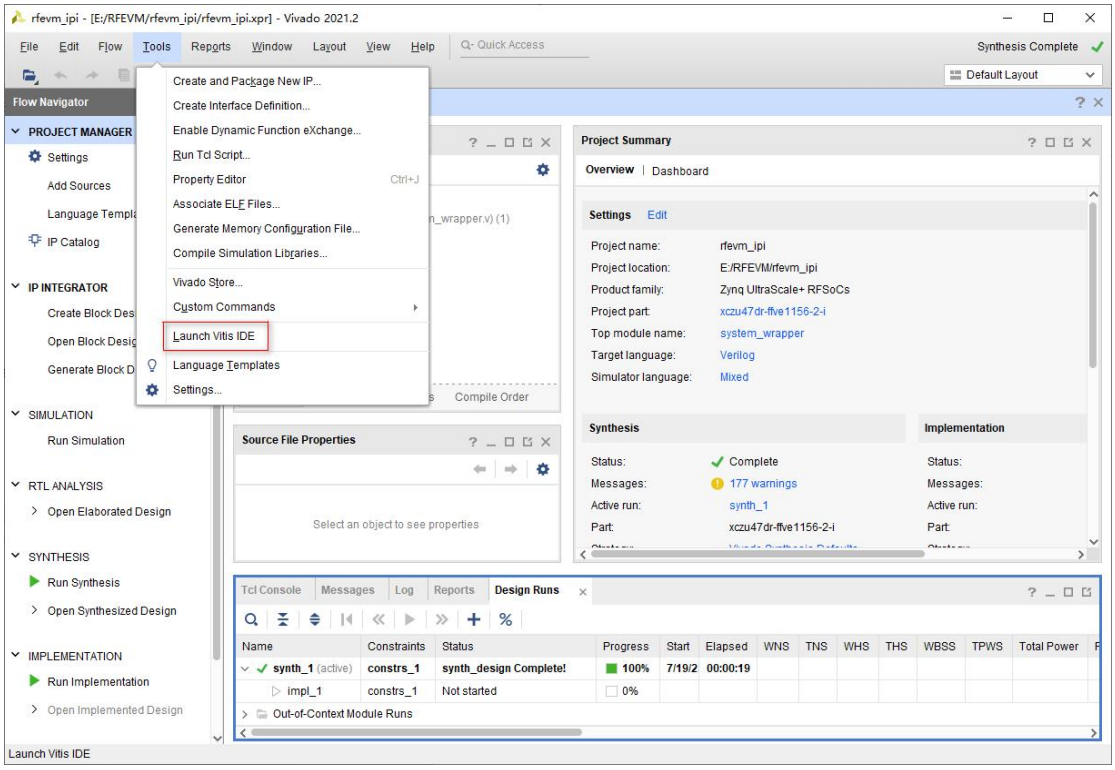
这里我们能看到新生成的 xsa 文件：



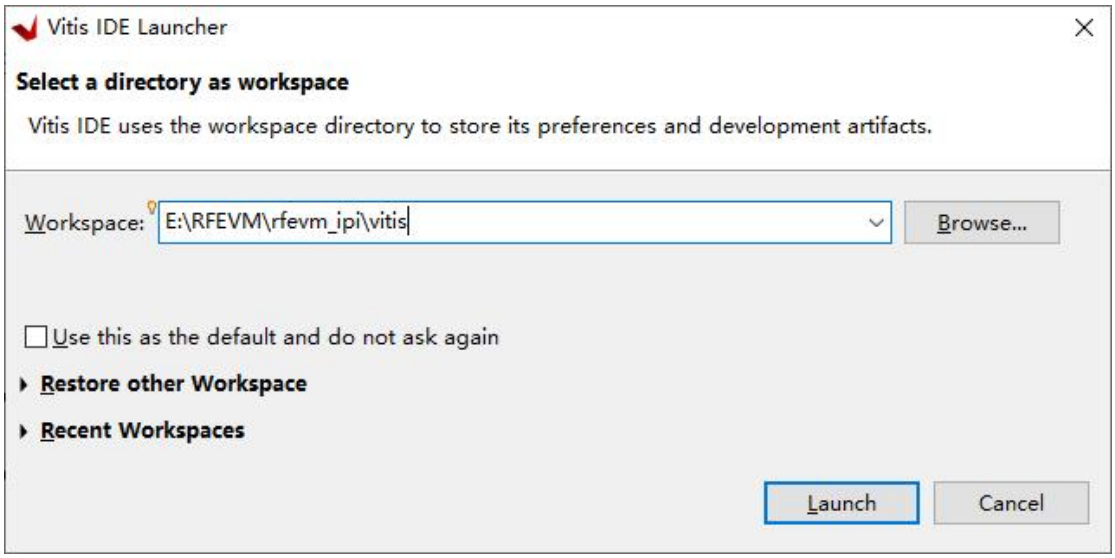
4.1.2 软件工程搭建

上篇我们在 vivado 下搭建好硬件工程，导出 xsa 文件。这里在工程目录下面创建 vitis 文件夹作为 vitis 的工作目录。

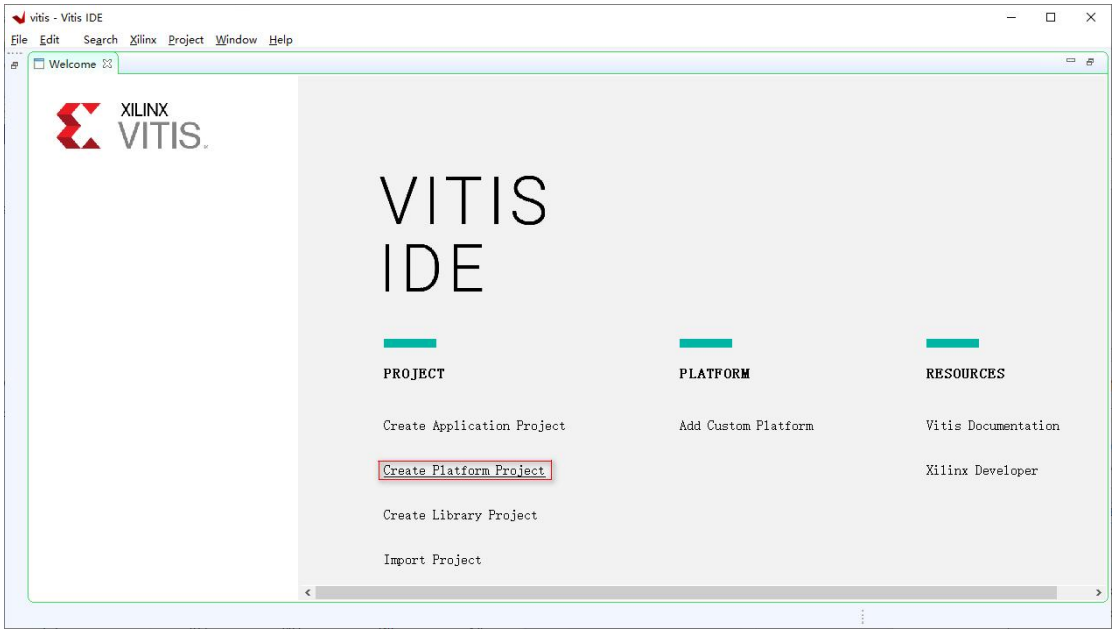
1) 登录 vitis



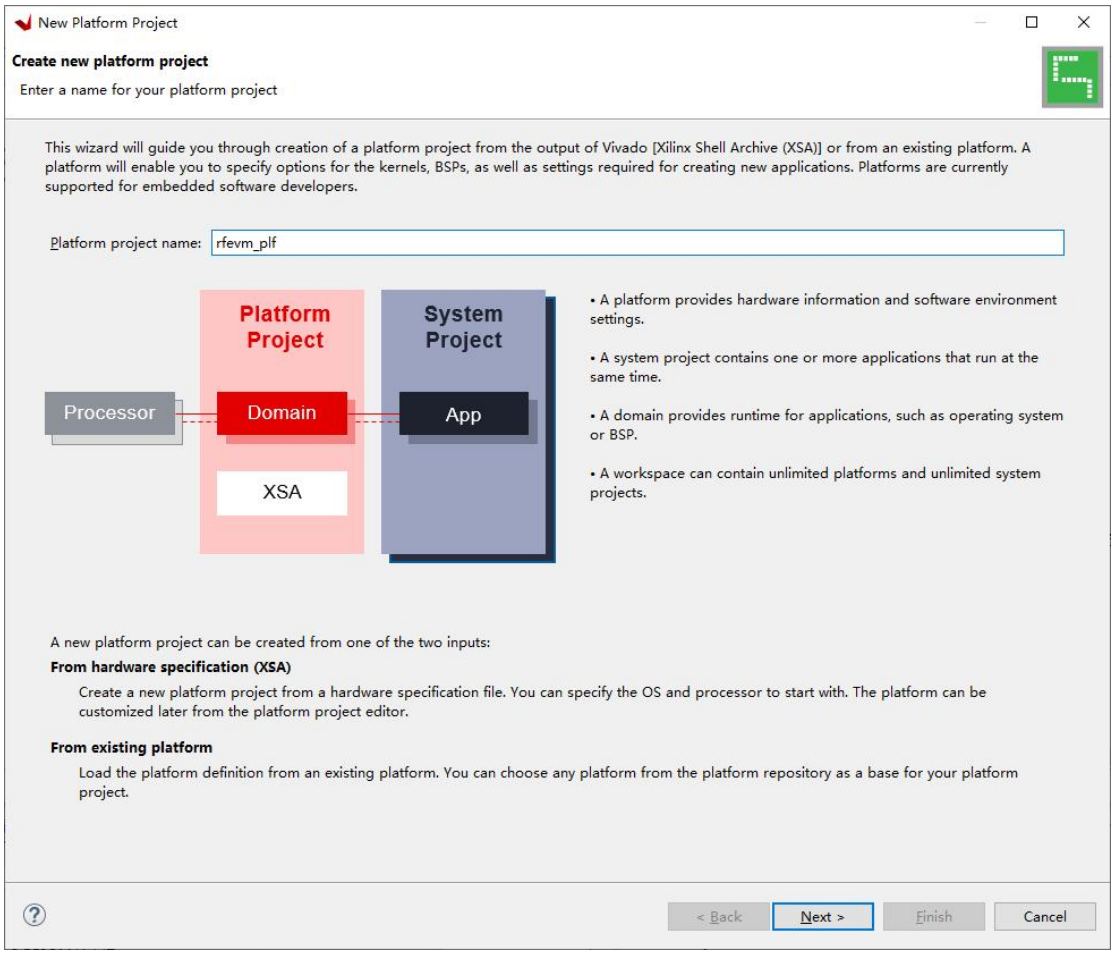
2) 选择工作目录，点击 Launch



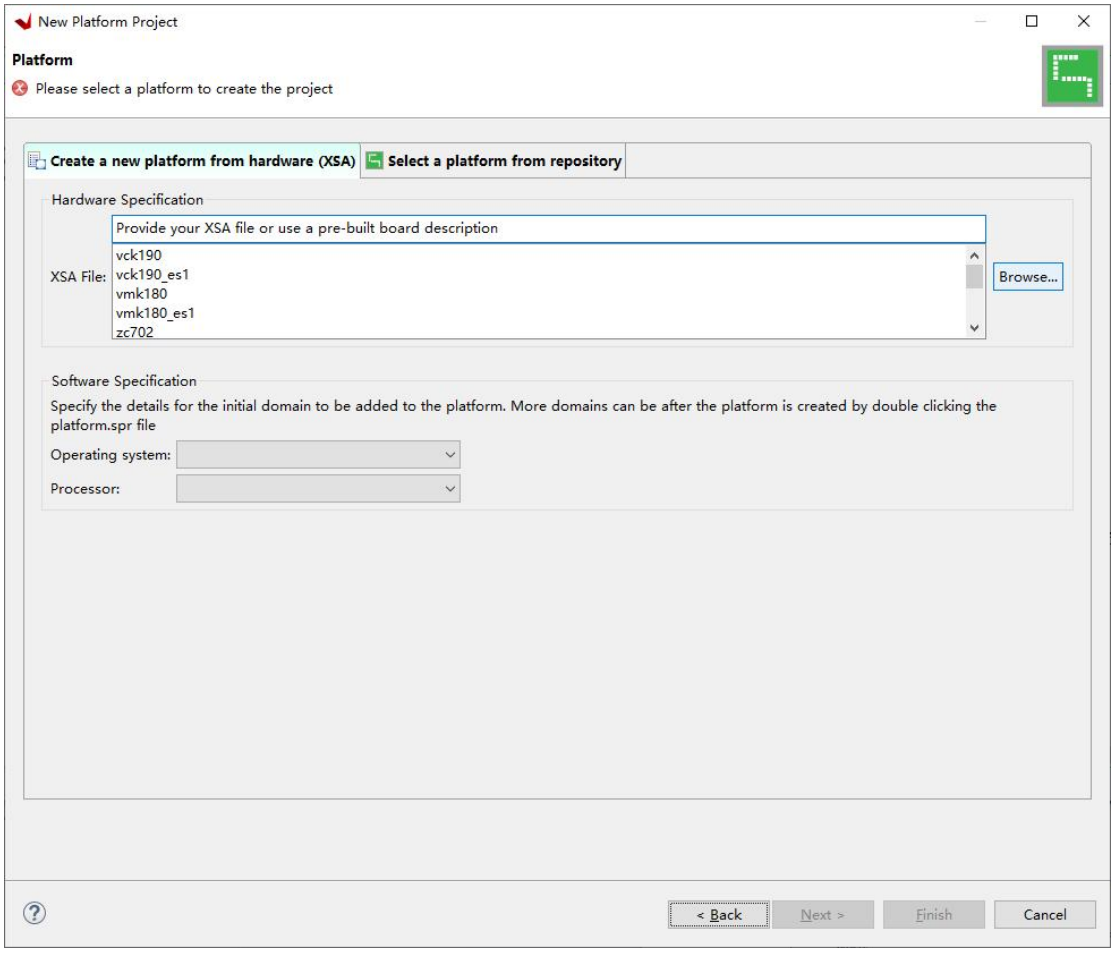
3) 点击 Create Platform Project



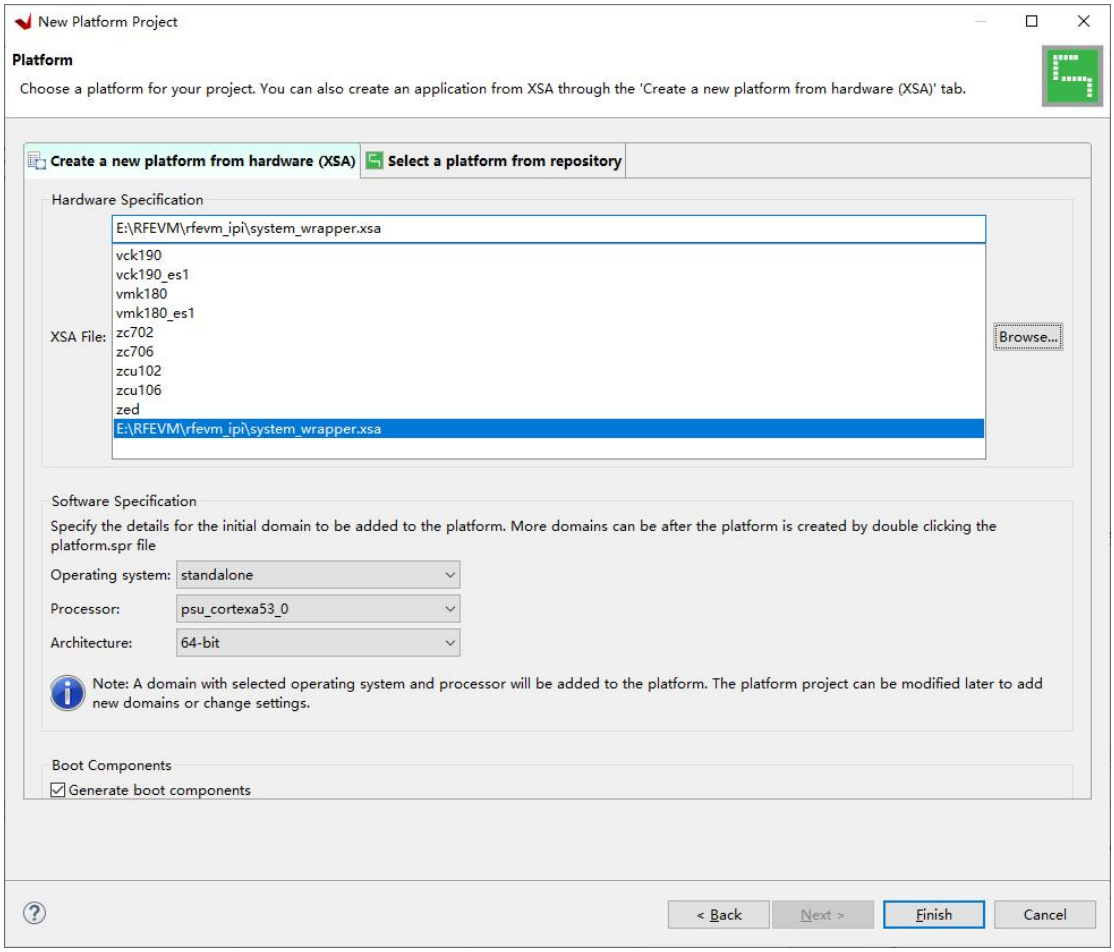
4) 设置工程名称，点击 Next

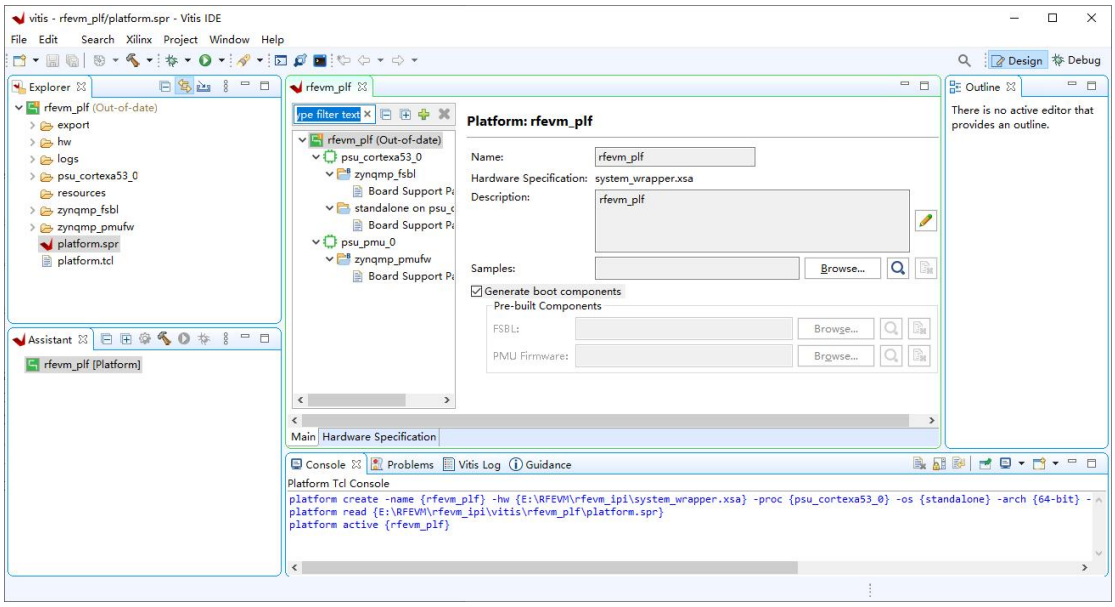


5) 点击 Browse..



6) 选择之前生成的 xsa 文件，点击 Finish





至此我们软件工程搭建完毕，后面在此基础上就可以进行 PS 裸机的实验；

4.2 体验 ARM，裸机输出”Hello World”

“Hello World！”是各种编程语言中最简单，也是最经典的入门实验。这里我们将串口打印“Hello World”作为嵌入式系统的开篇实验，这也是我们步入 PS 部分的始发点。

4.2.1 硬件介绍

RFEVM 底板上预留了一个 JTAG&UART 接口，既可以用它下载调试程序，也可以通过串口和 PC 通信。

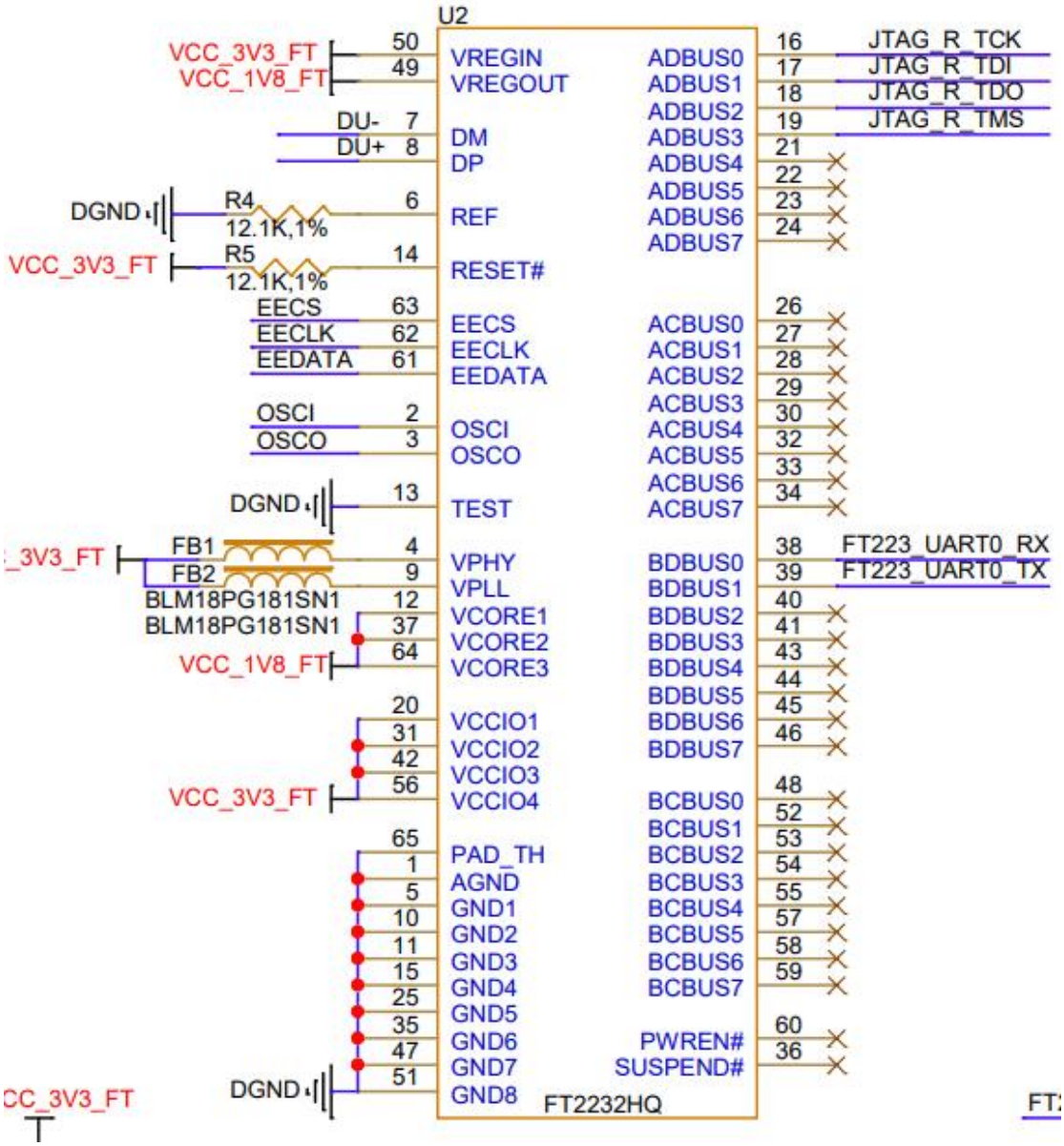
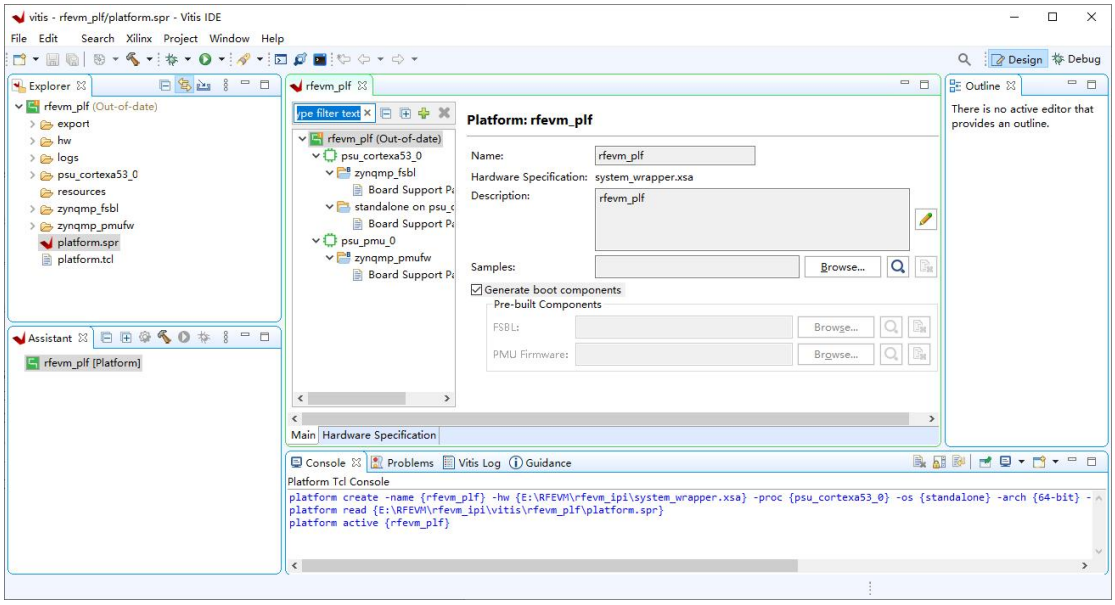


图 10 JTAG%UART 硬件原理图

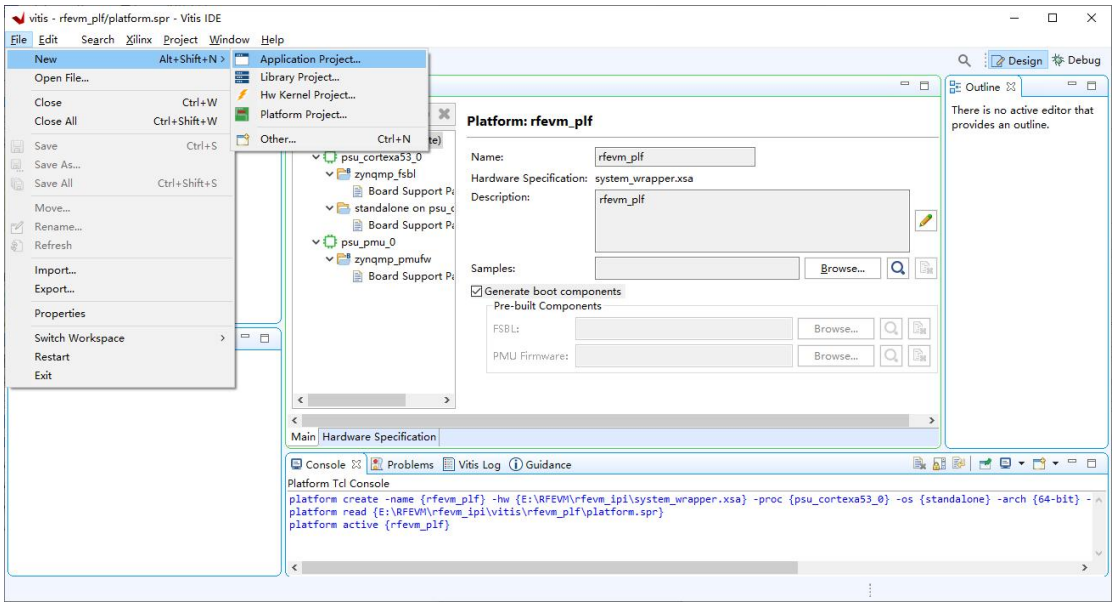
4.2.2 vitis 程序开发

我们在上节的基础上，创建基于串口的应用程序。

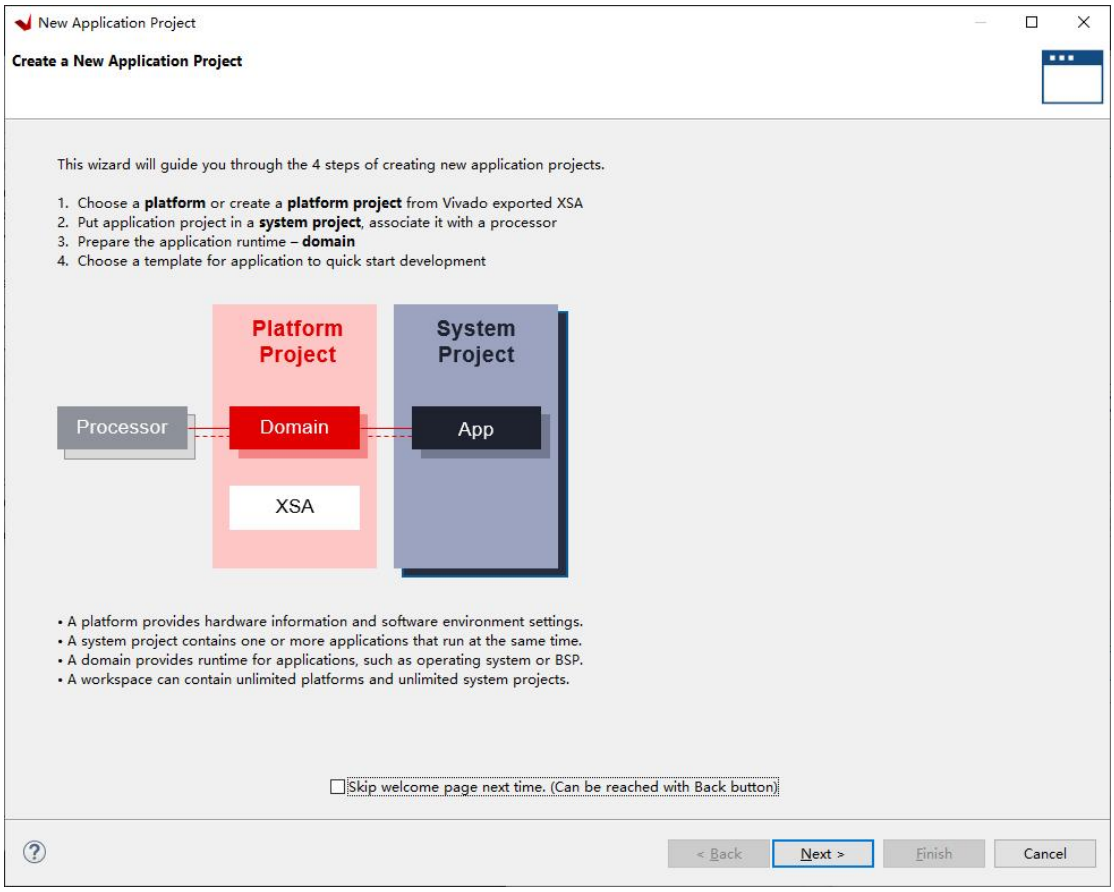
1) 登录 vitis，切换到之前的工作目录



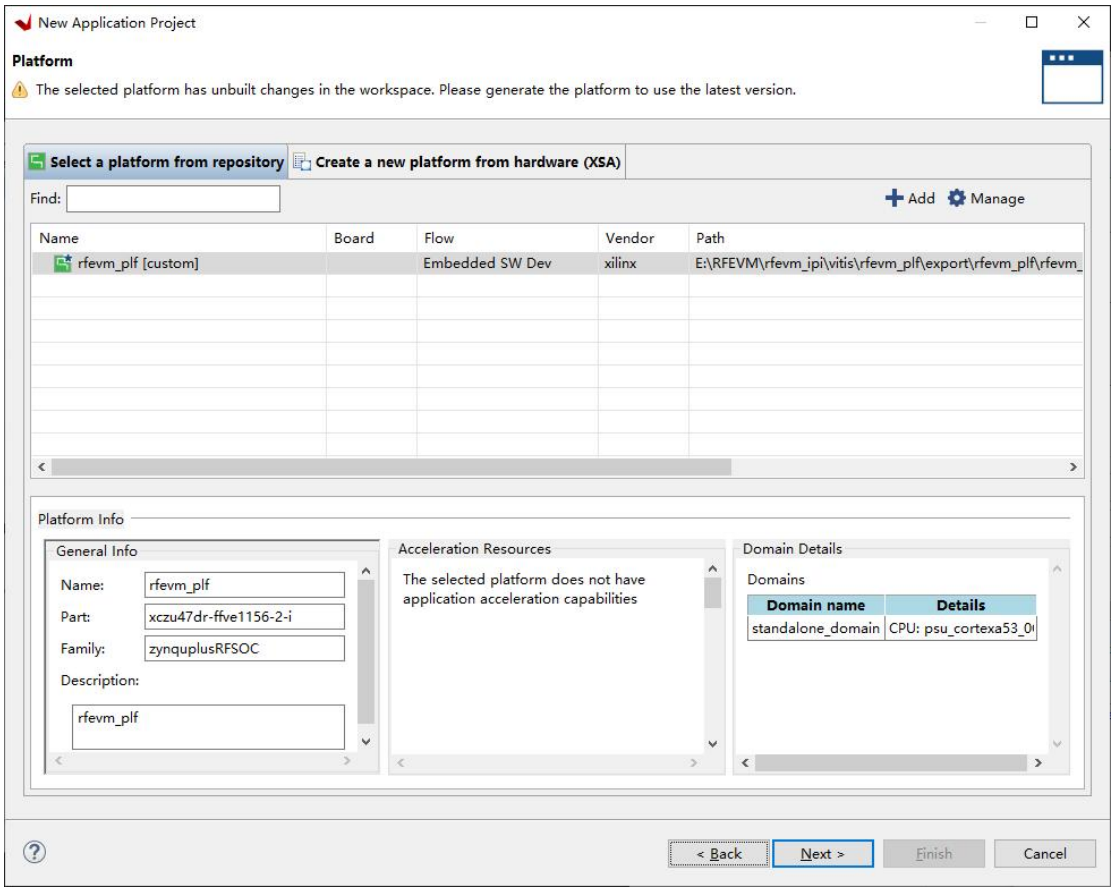
2) 新建应用程序 File->New->Application Project



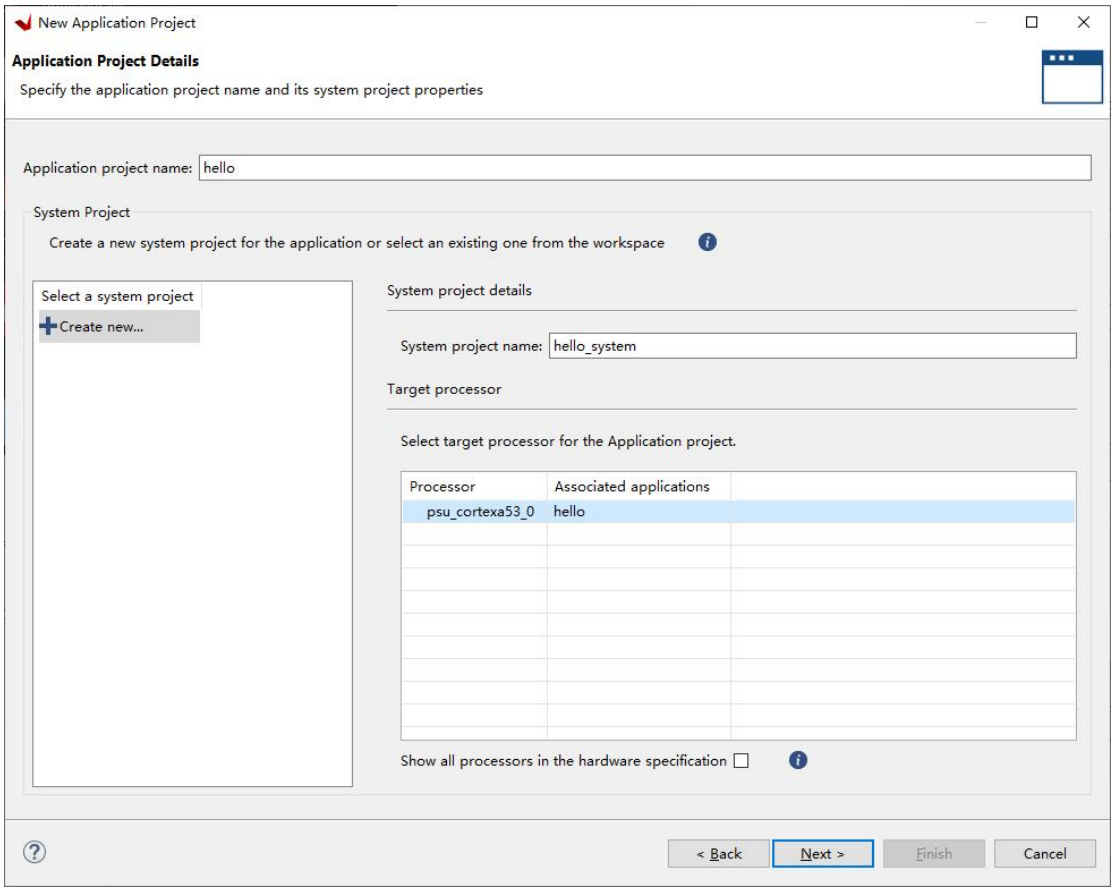
3) 点击 Next



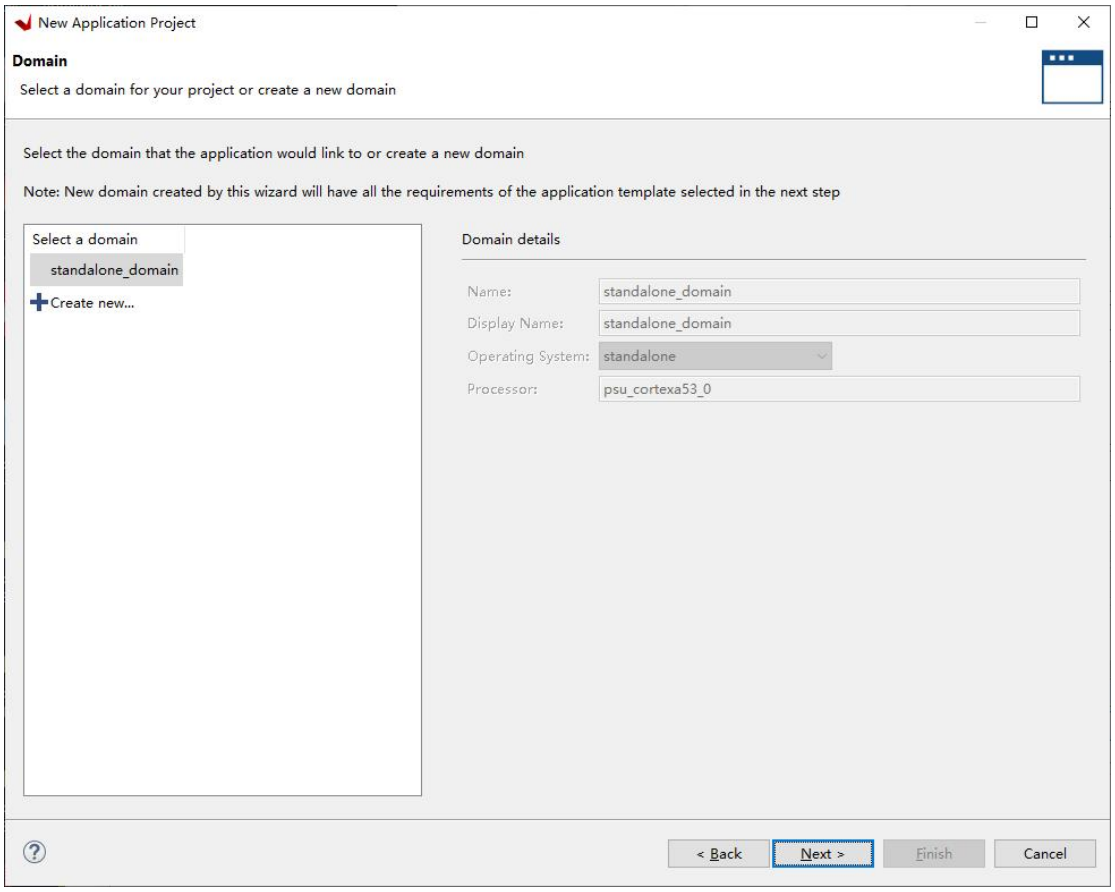
4) 此处默认选择的就是之前生成的 platform，直接点击 Next



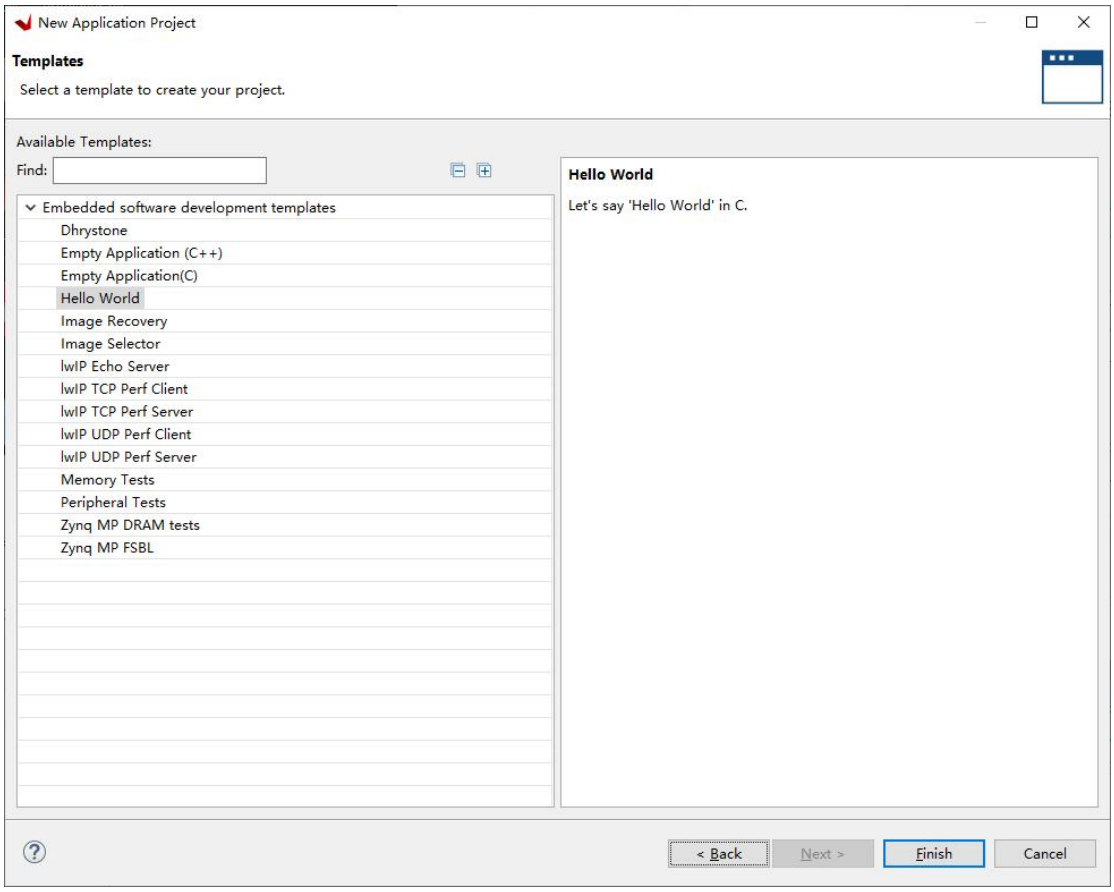
5) 填入 APP 工程名称，其他默认，点击 Next


New Application Project
Application Project Details
Specify the application project name and its system project properties
Application project name: hello
System Project
Create a new system project for the application or select an existing one from the workspace
Select a system project
Create new...
System project details
System project name: hello_system
Target processor
Select target processor for the Application project.
Processor
Associated applications
psu_cortexa53_0
hello
Show all processors in the hardware specification
< Back
Next >
Finish
Cancel

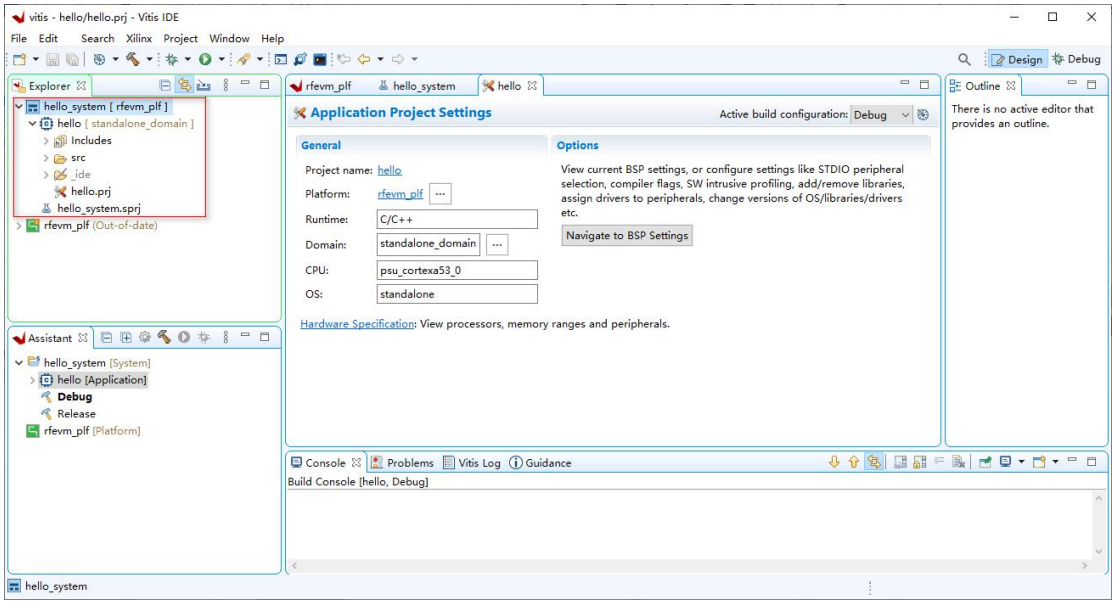
6) 点击 Next



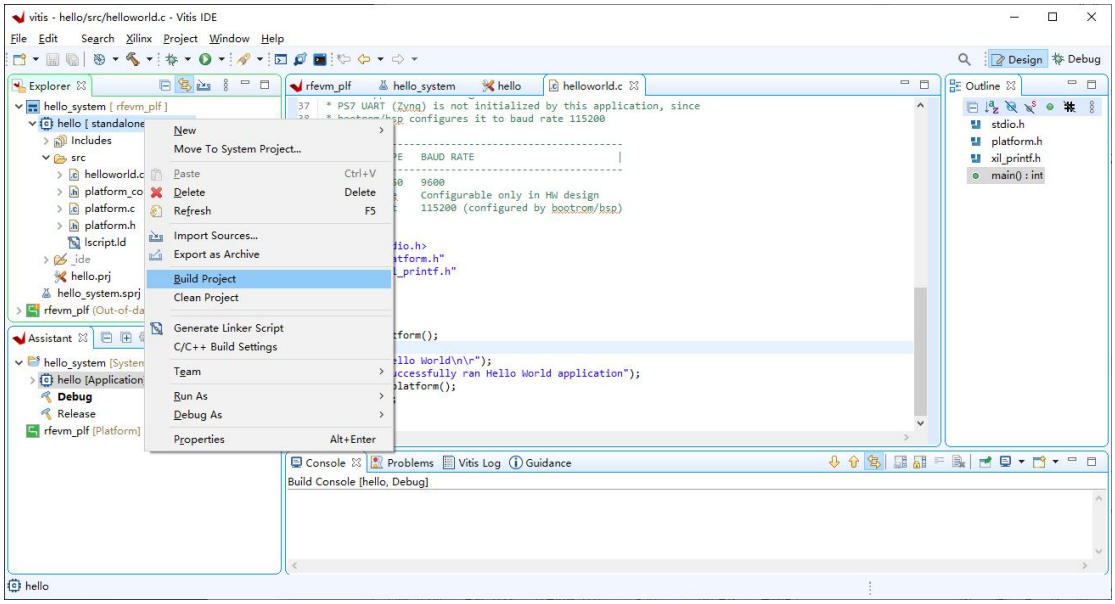
7) 选择 Hello World 模板，点击 Finish



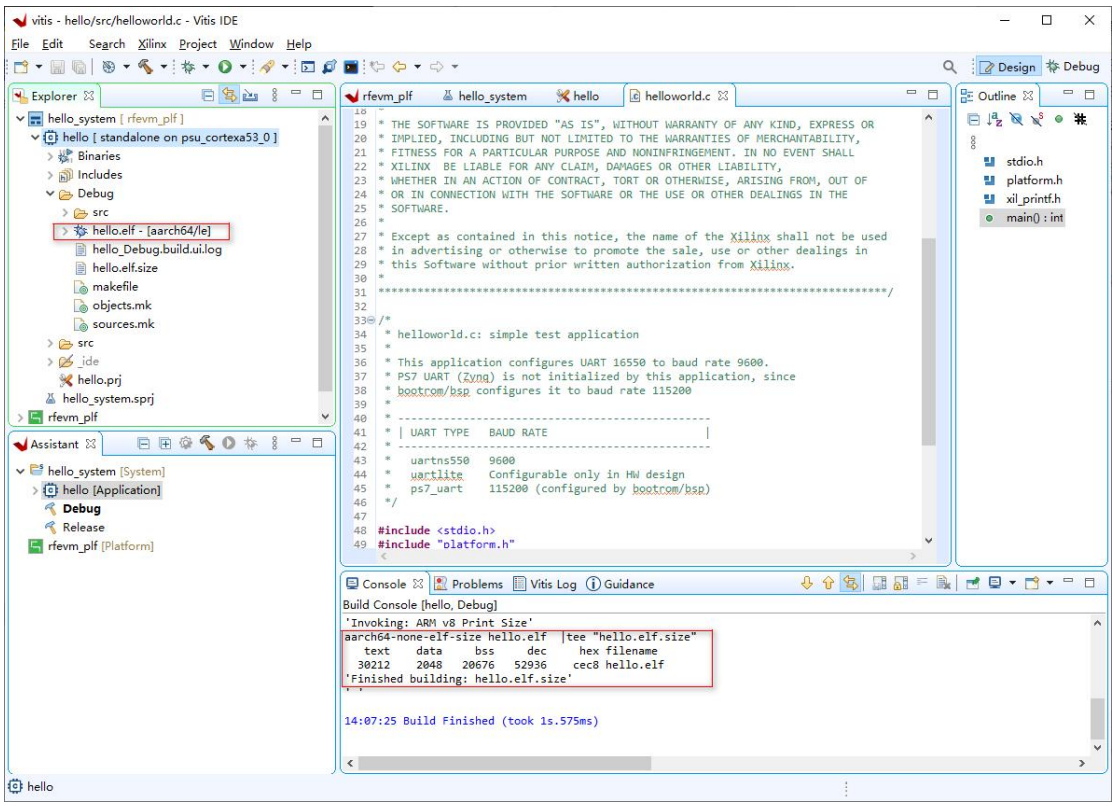
8) 完成之后可以看到生成的 APP 工程



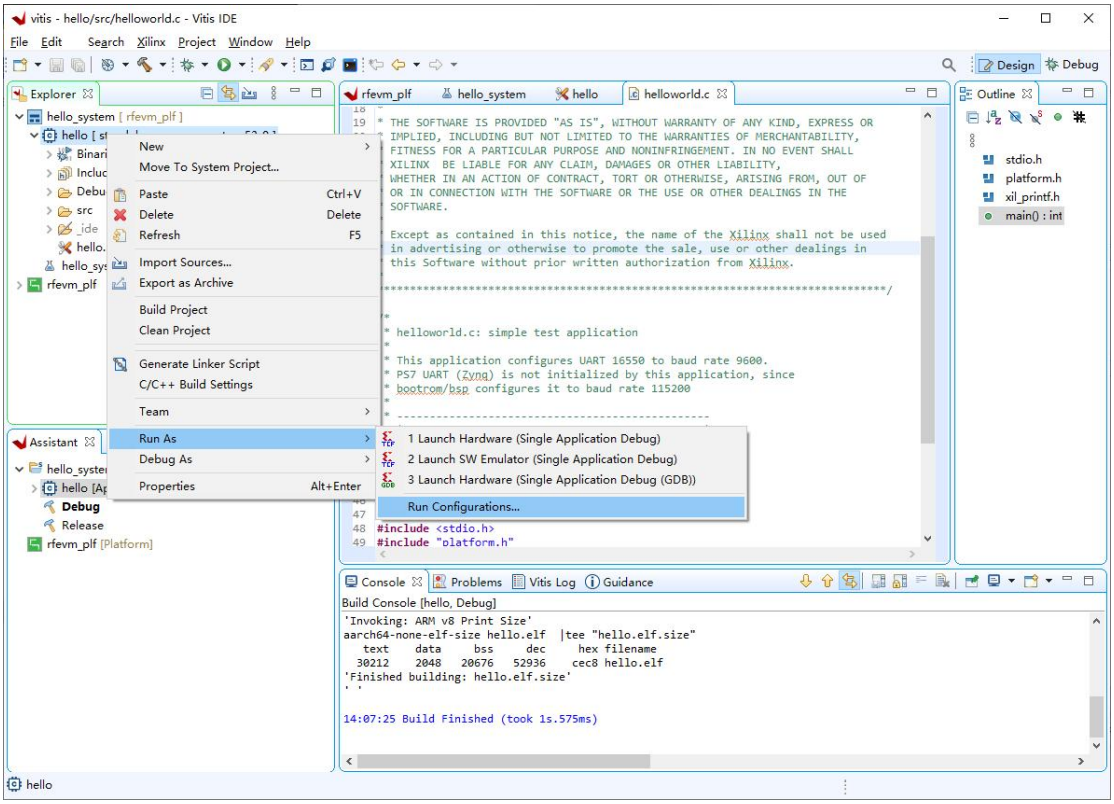
9) 在 APP 工程上右键，选择 Build Project，或者点击菜单栏的锤子按键，进行工程编译；



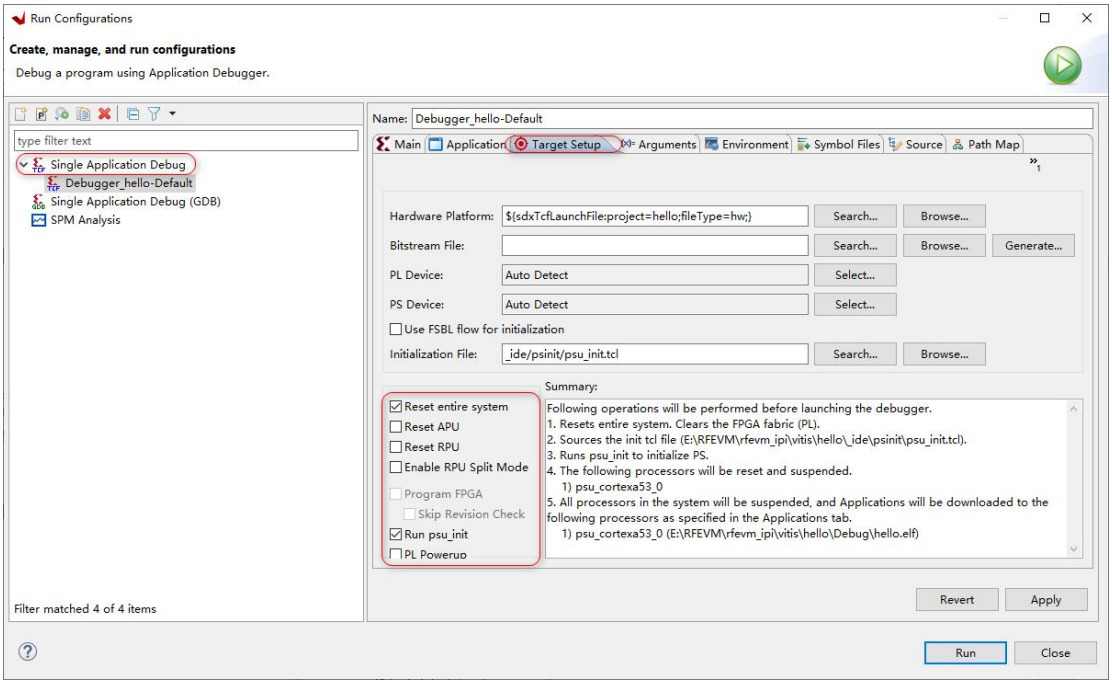
Console 窗口会显示编译信息，编译结束会生成 elf 文件



10) 右键 APP 工程，选择 Run As->Run Configuration

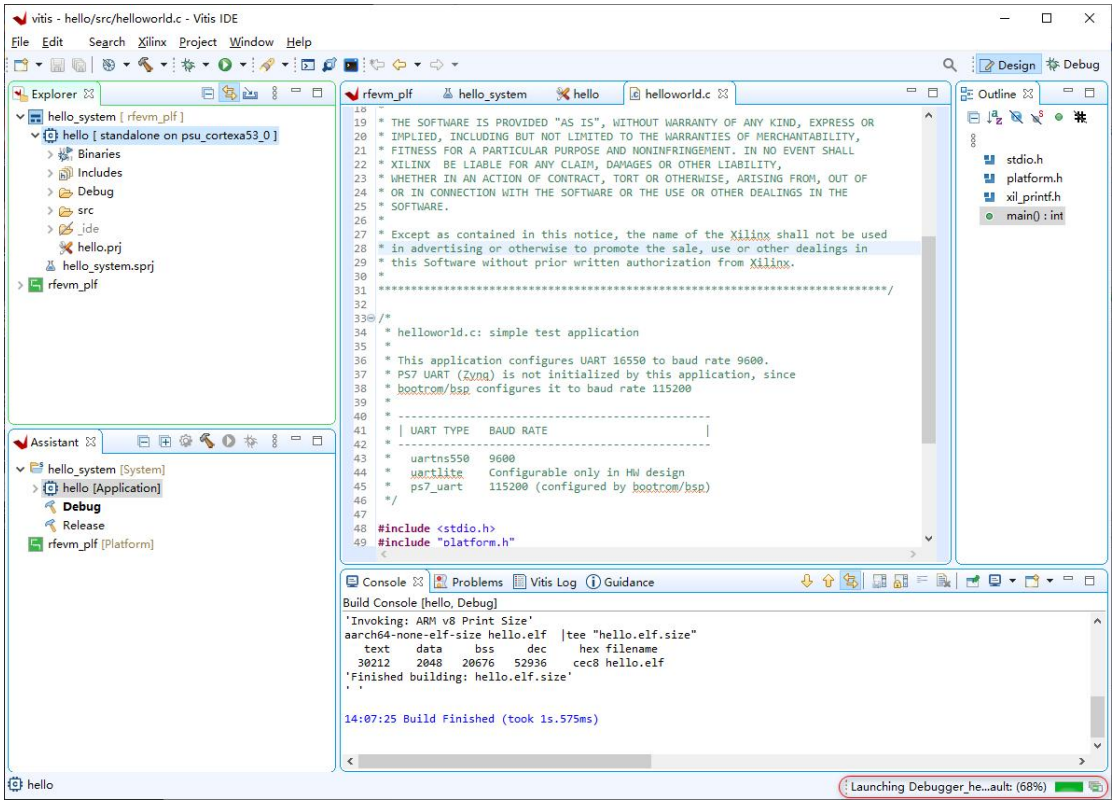


11) 双击 Single Application Debug，生成 Debugger_hello；在 Target Setup 页面进行相关设置。*（这里我们只有 PS 部分，所以不用使能 Program FPGA）*

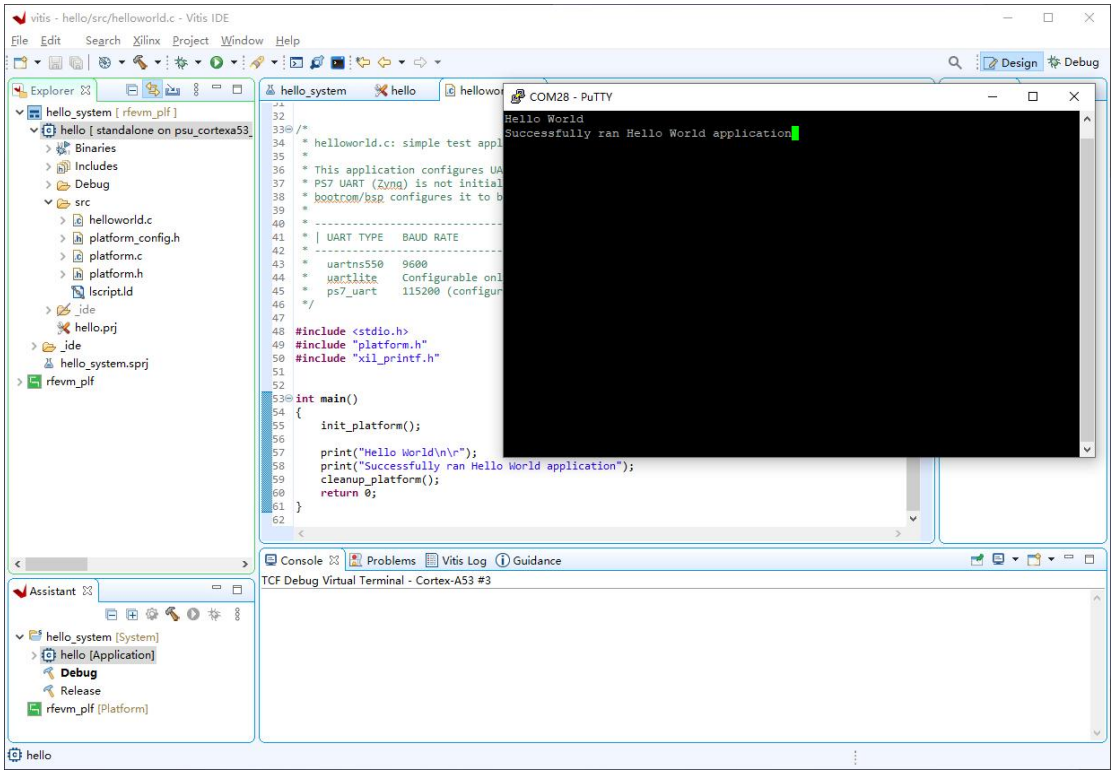


12) PC 侧打开串口调试终端，选择相应的串口号和波特率 115200；

13) 点击 11 步的 Run，右下角会显示下载进度



14) 下载完成后，查看串口输出



4.3 PS 端 RTC 中断实验

RTC（Real Time Clock）实时时钟单元为系统和应用软件提供精确的时间基

准。为了满足高精度的需要，RTC 还包括校准电路以抵消温度和电压波动。RTC 由 VCC-PSAUX 或 VCC-PSBATT 电源供电。当辅助电源可用时，RTC 使用它保持计数器工作。当辅助电源 VCC PSAUX 不可用时，RTC 自动切换到电池电源 VCC PSBATT 电源。

RTC 具有以下特点：

- 既可以使用辅助电源工作也可以使用电池电源工作，当系统掉电时，RTC 会自动切换至电池供电保持持续工作。
- 报警设置和周期性中断（定时中断）。
- 用于高精度计时的复杂校准电路。
- 32 位秒计数器，可计数约 136 年的时间。
- 三个计数器： 32 位秒计数器
  16 位滴答计数器，用于测量基于 32KHz 晶振的秒数。
  用于校准的 4 位小数计数器

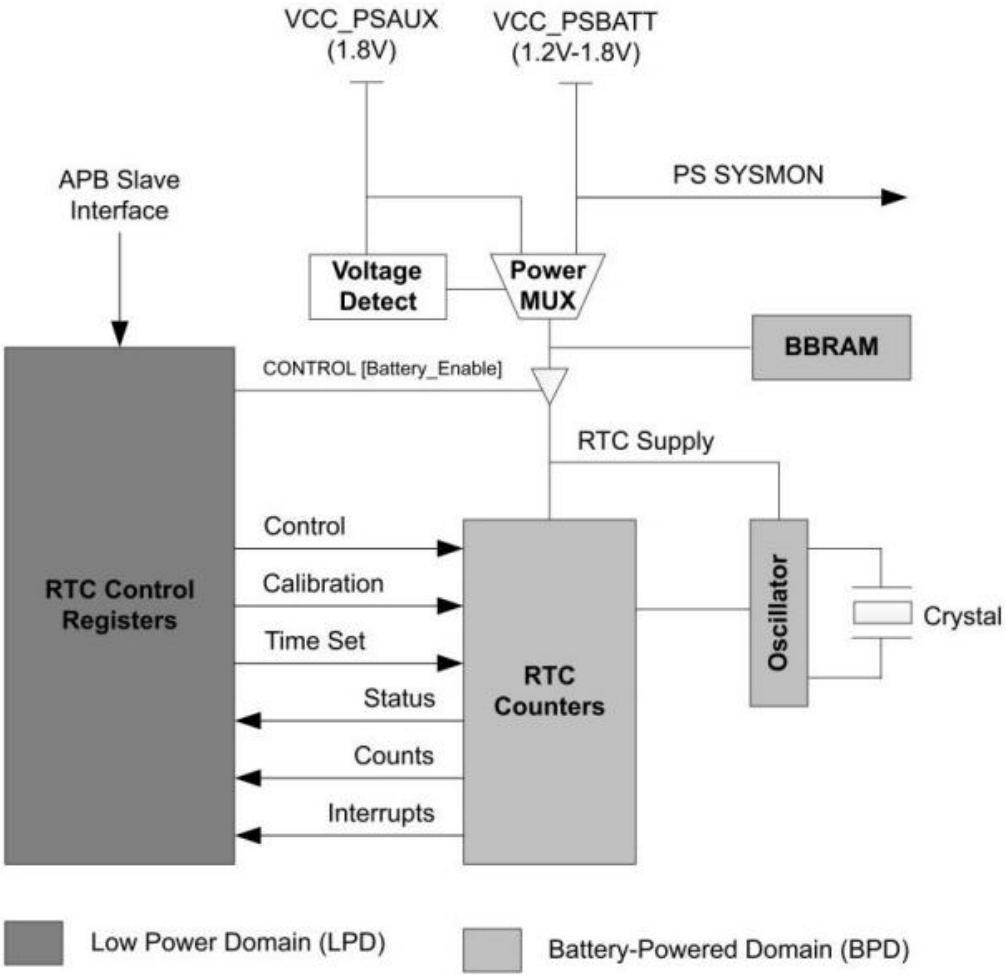


图 11 RTC 控制器结构图

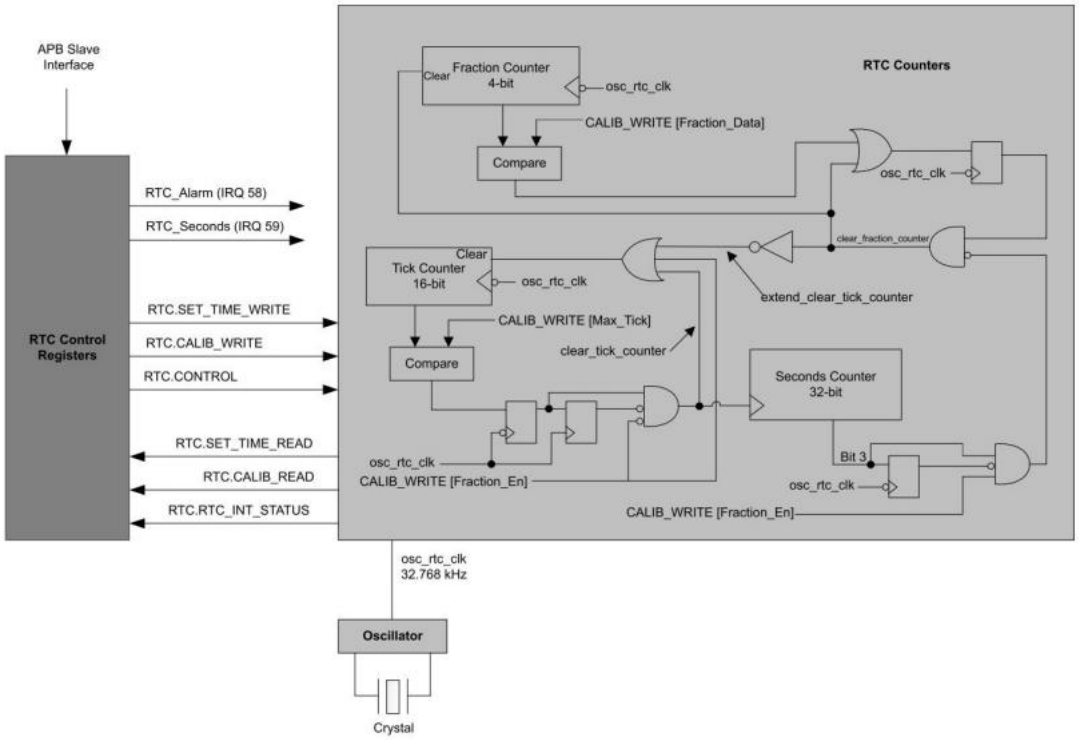
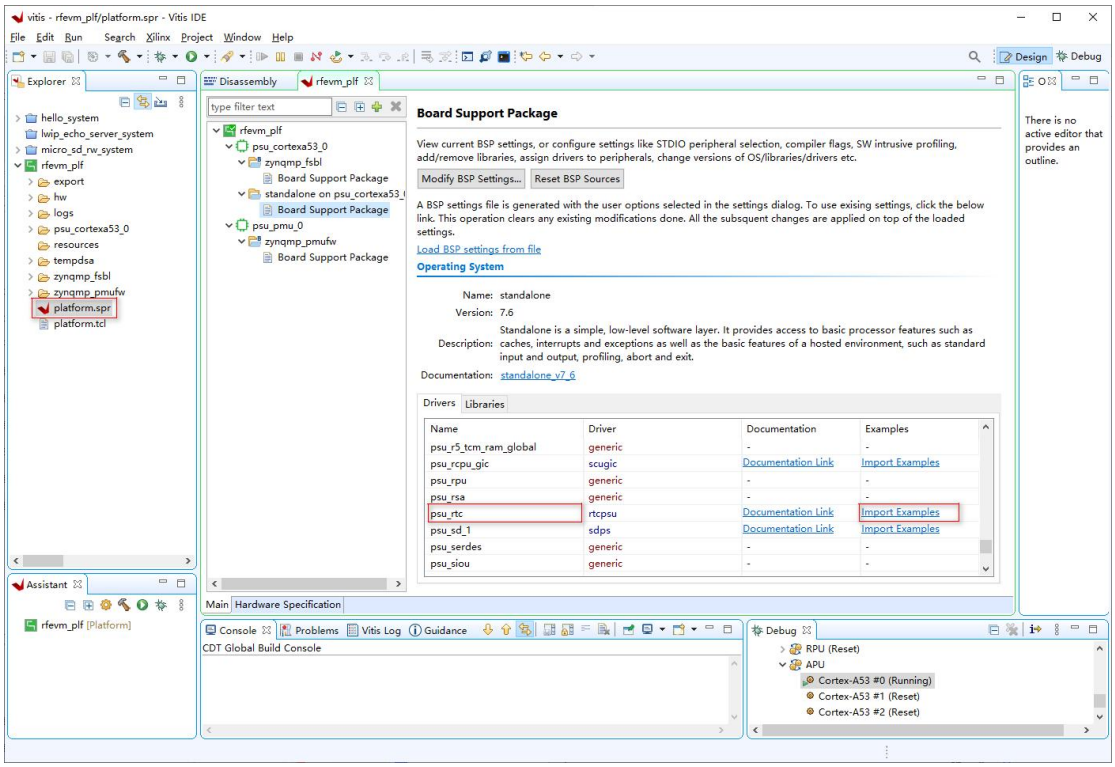


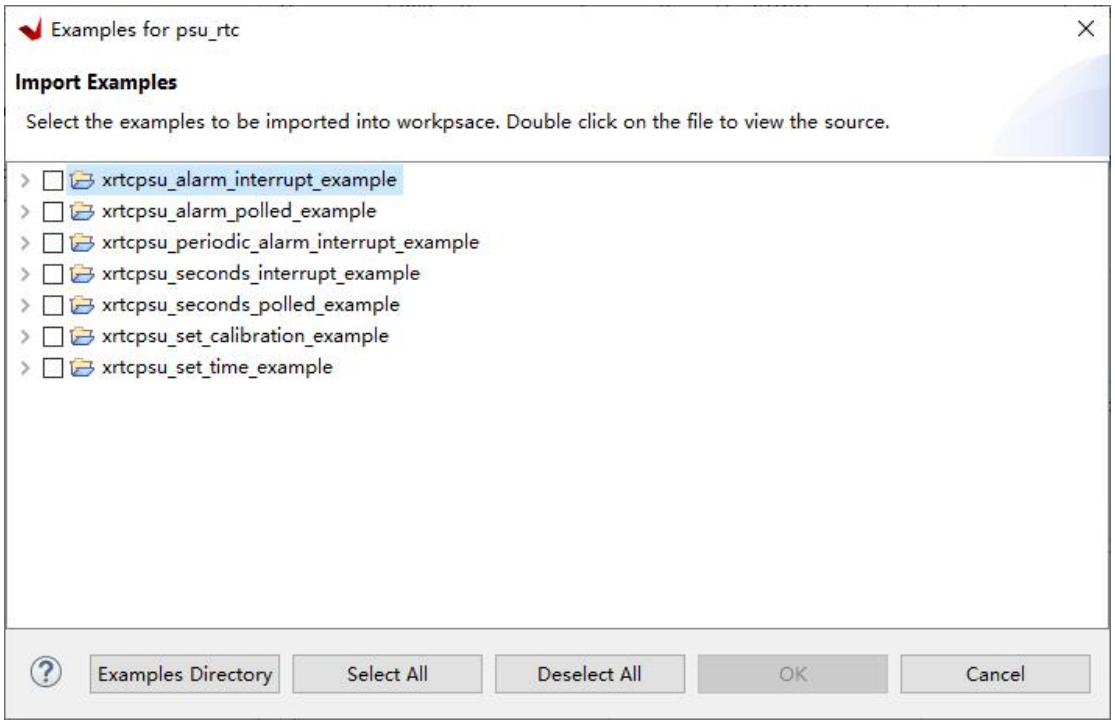
图 12 RTC 功能图

4.3.1 软件设计

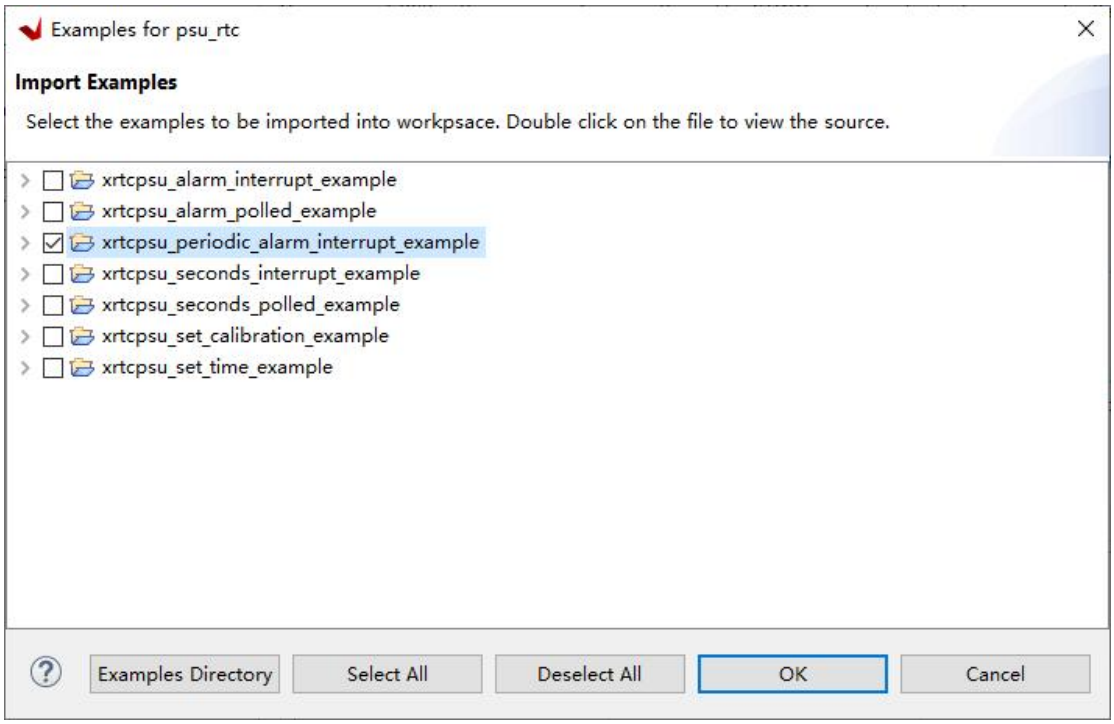
1) 登录 Vitis，切换到之前的工作目录，然后双击 platform.spr，选中 Board Support Package，可以看到 psu_rtc 的文档和导入示例。



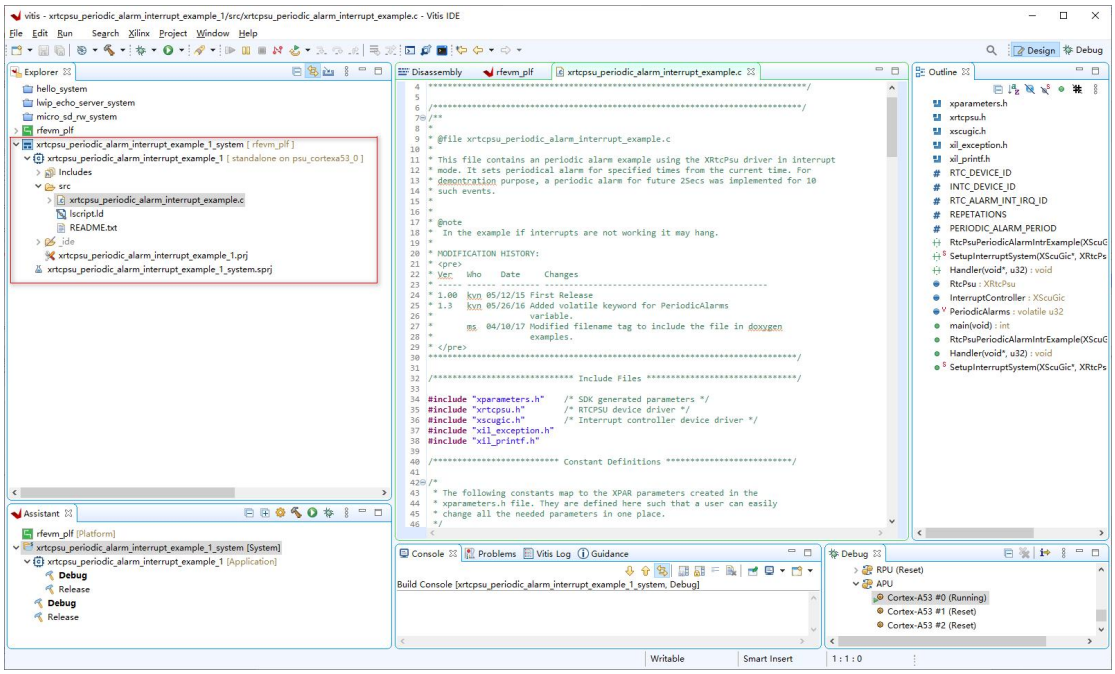
2) 点击 Import Example，会弹出如下图所示的导入示例界面；



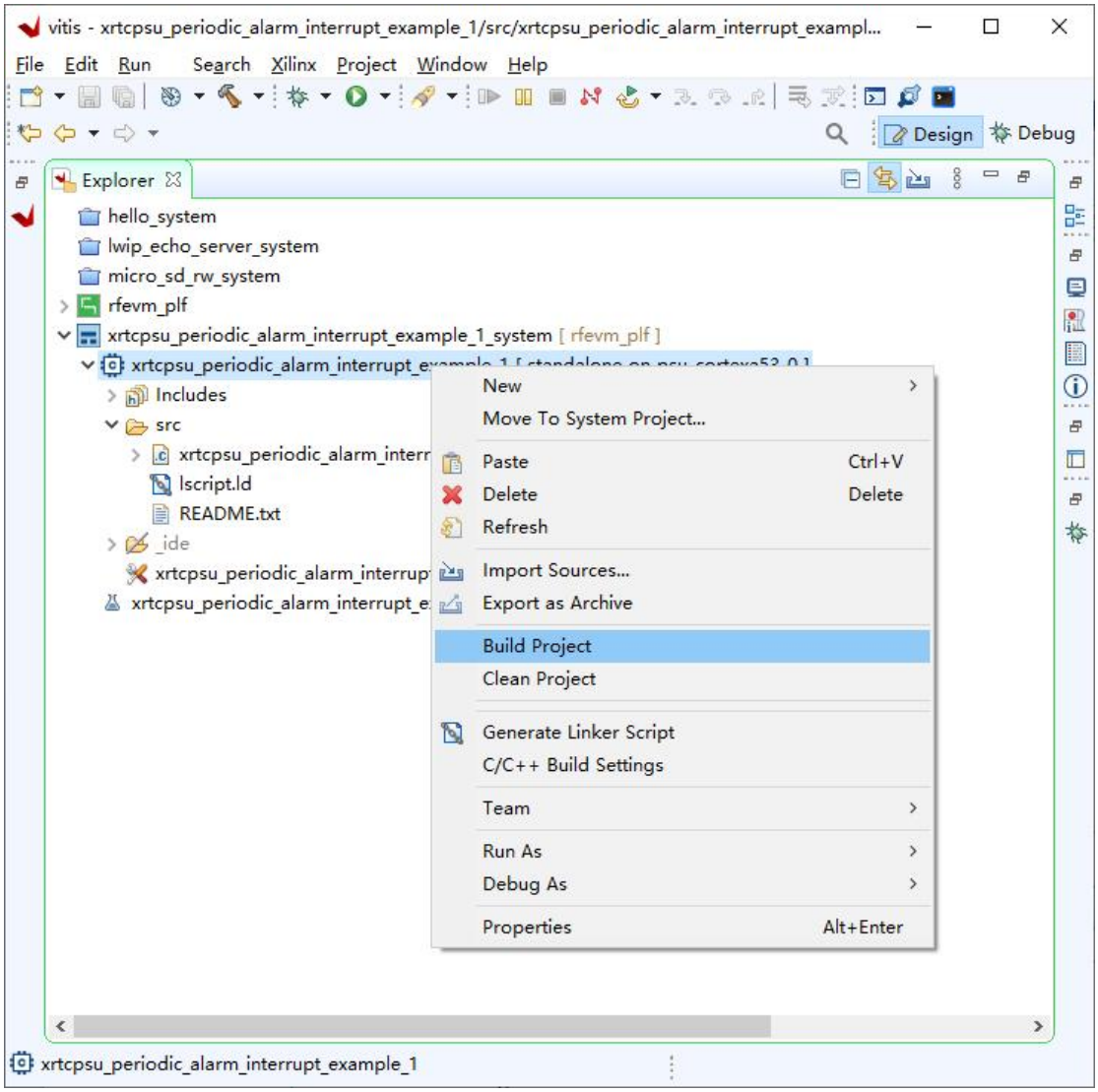
3) 这里我们选中第三个官方示例 RTC 中断周期报警的示例（xrtcpsu_periodic_alarm_interrupt_example），点击 OK



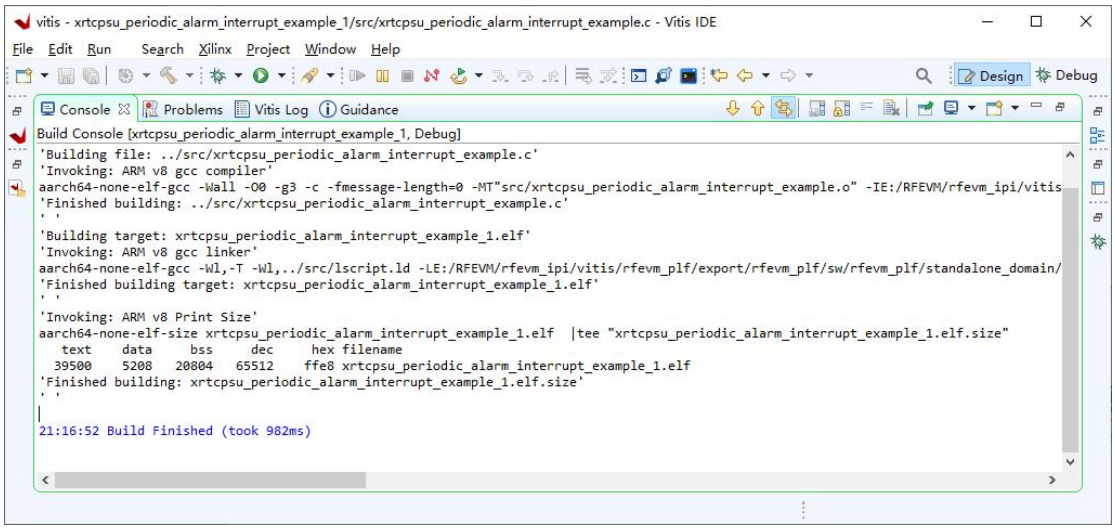
4) 这里我们就导入了 example 工程



5) 选中应用工程，右键后选中 Build Project 对工程进行编译



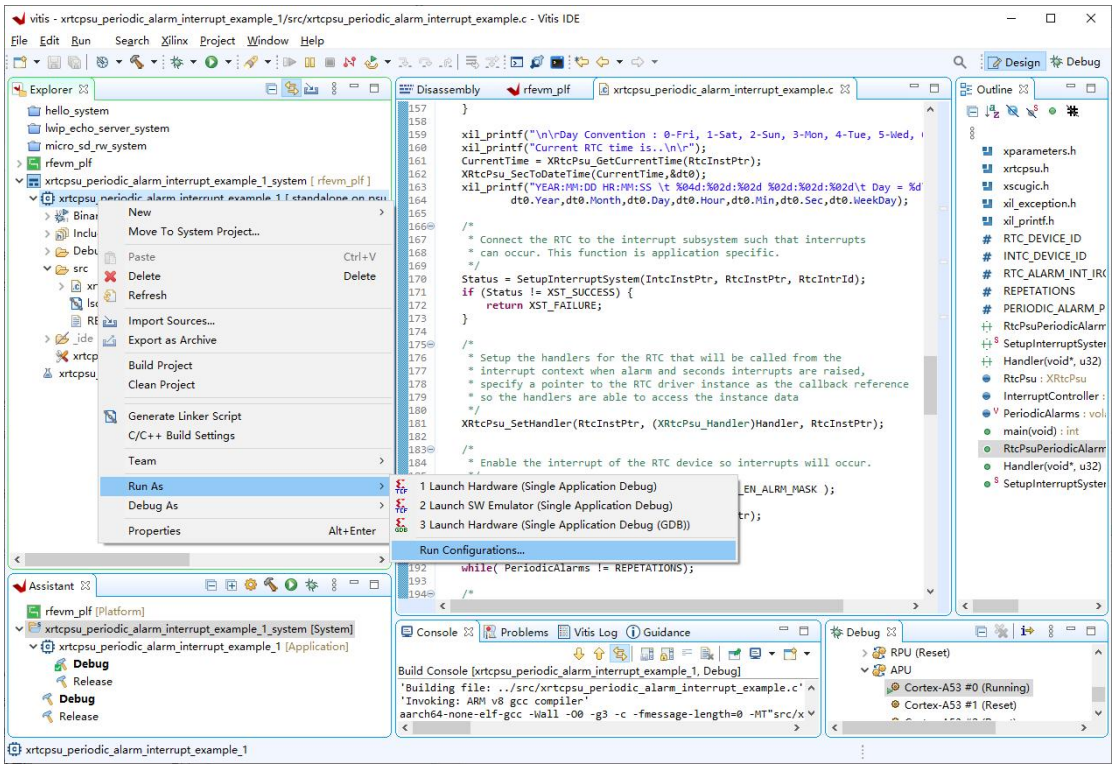
编译进度可以在工具下方的控制台面板（Console）中进行查看，如下图十所：



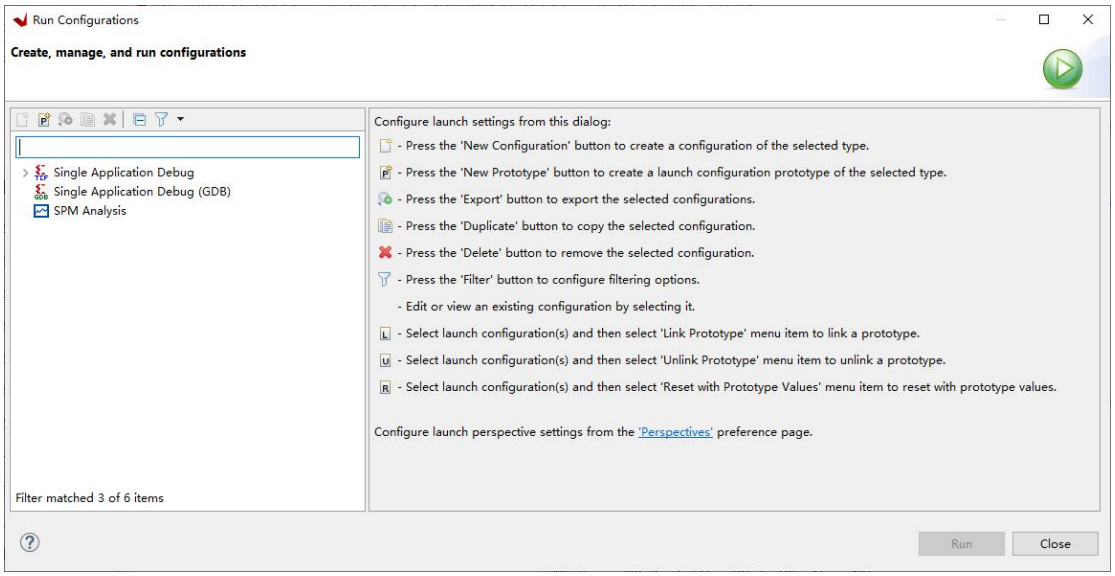
4.3.2 下载验证

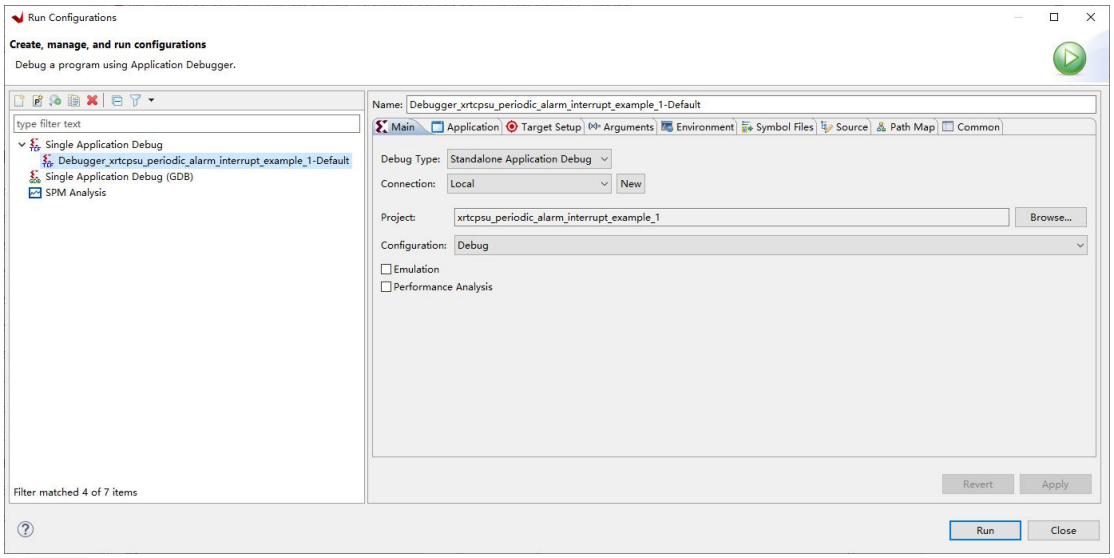
首先我们将 USB 转 Type-C 线缆的 Type-C 端与 RFEVM 开发板上的 JTAG 接口连接，USB 转 Type-C 线缆的另外一端与电脑连接。连接开发板电源给开发板上电。

下载程序，右键点击应用工程，选择 Run As->Run Configuration...

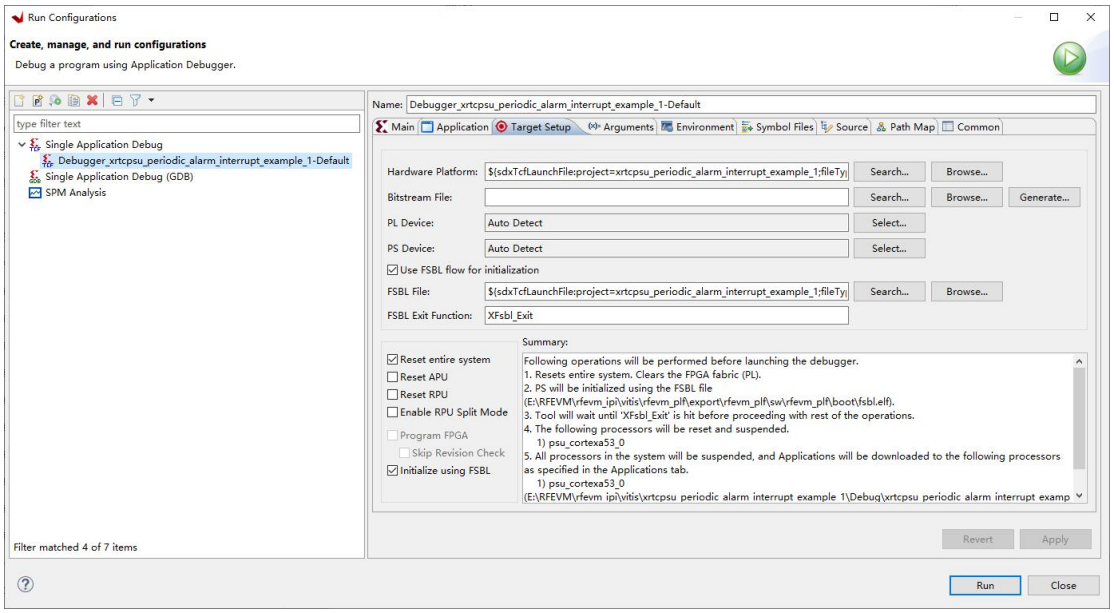


双击 Single Application Debug 一项

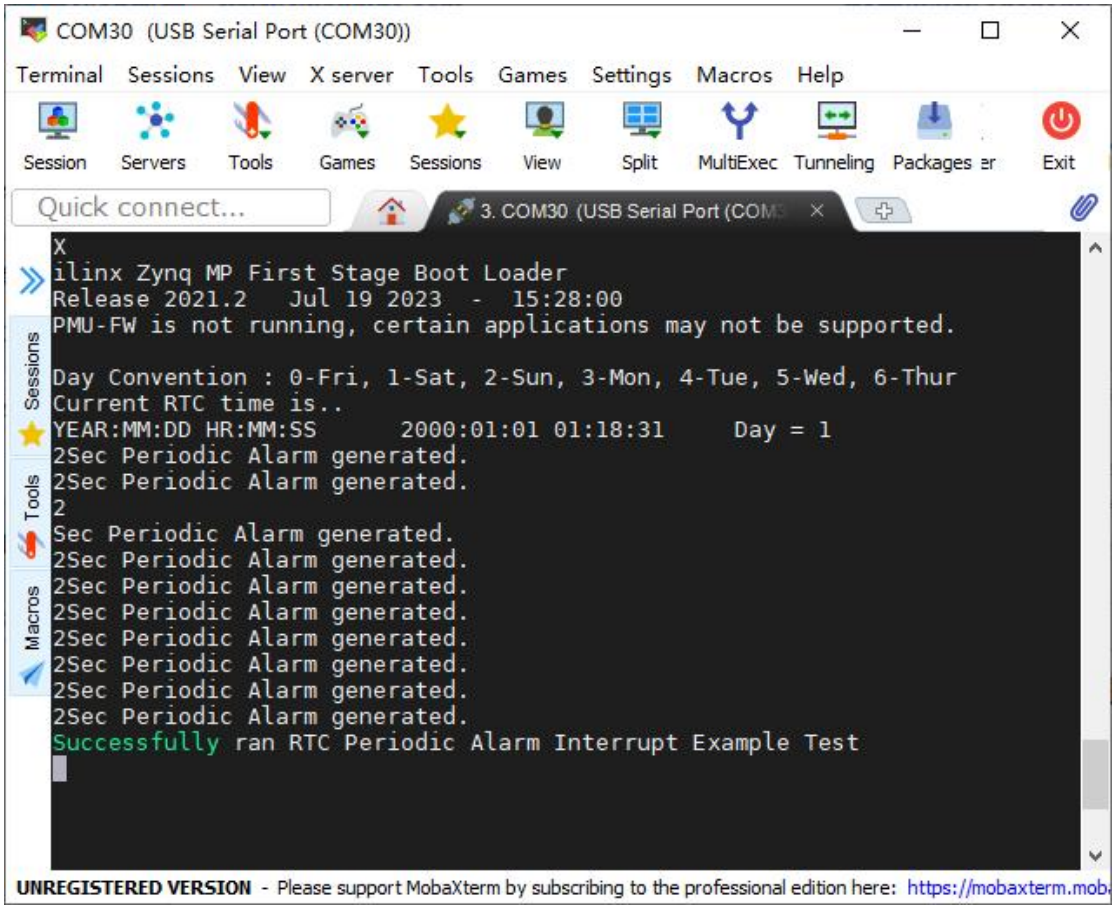




选中 Target Setup 标签页，默认配置，点击 Run



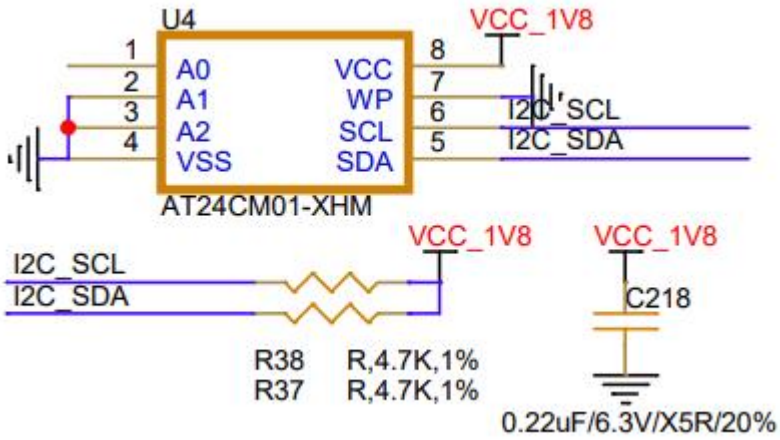
下载完成后，在串口调试终端可以看到串口的要打印信息：



4.4 PS 端 IIC 的使用

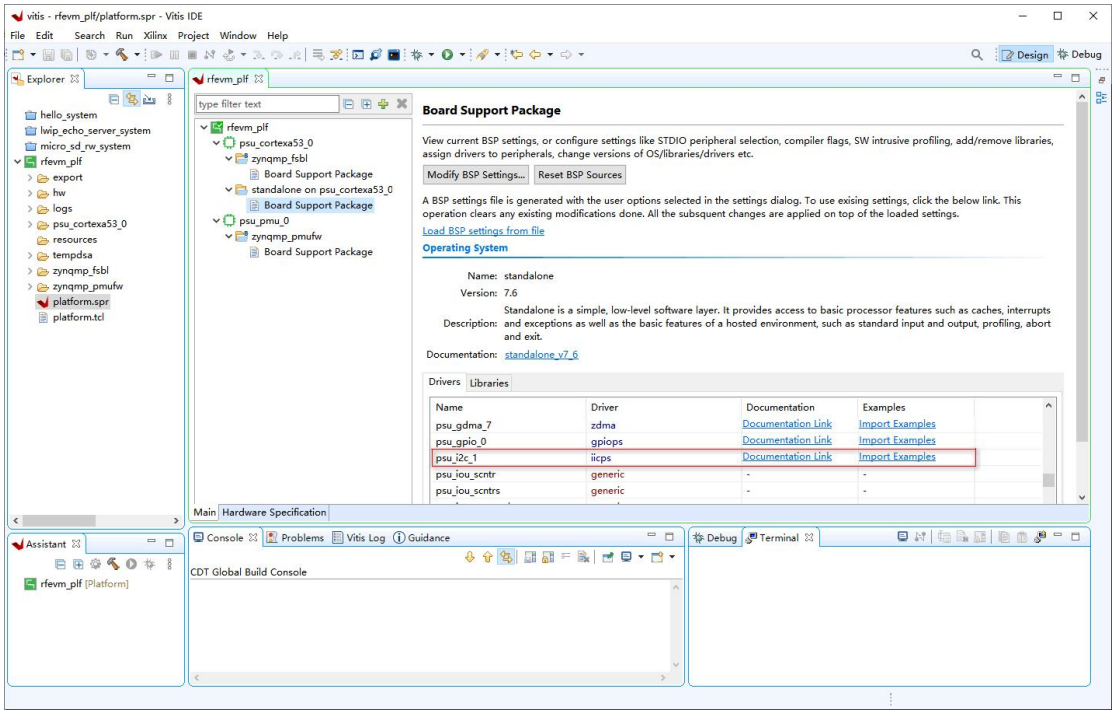
4.4.1 硬件设计

PS 端 IIC 接了一个外设 EEPROM，本节介绍如何利用 IIC 读写外设。

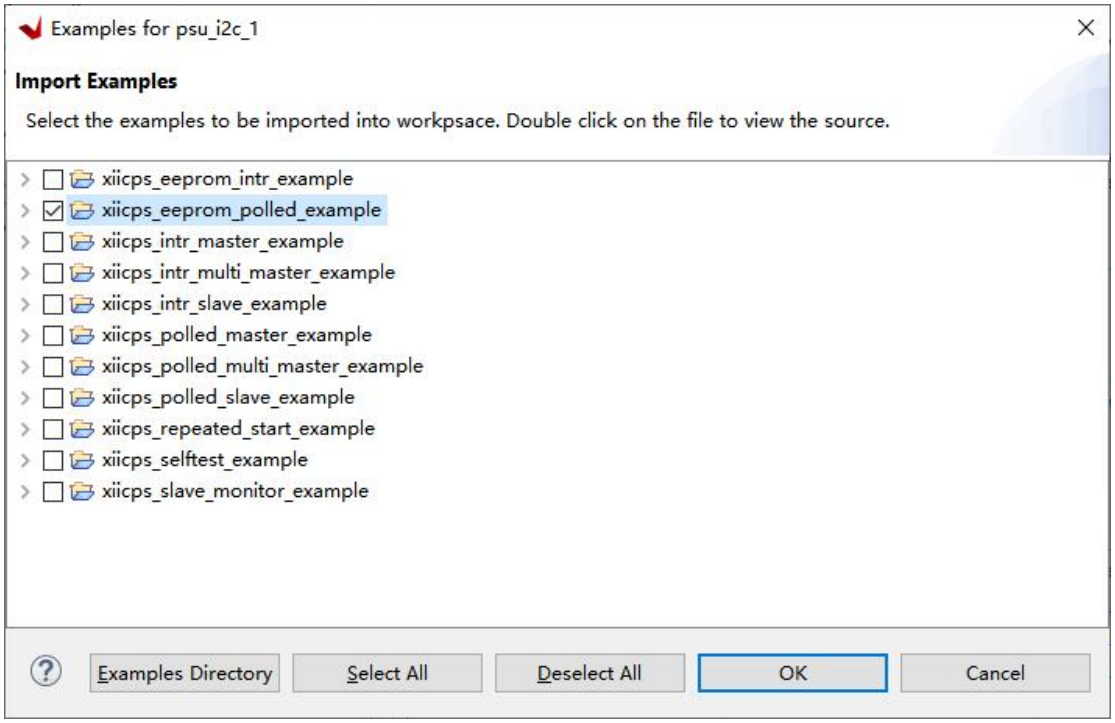


4.4.2 软件设计

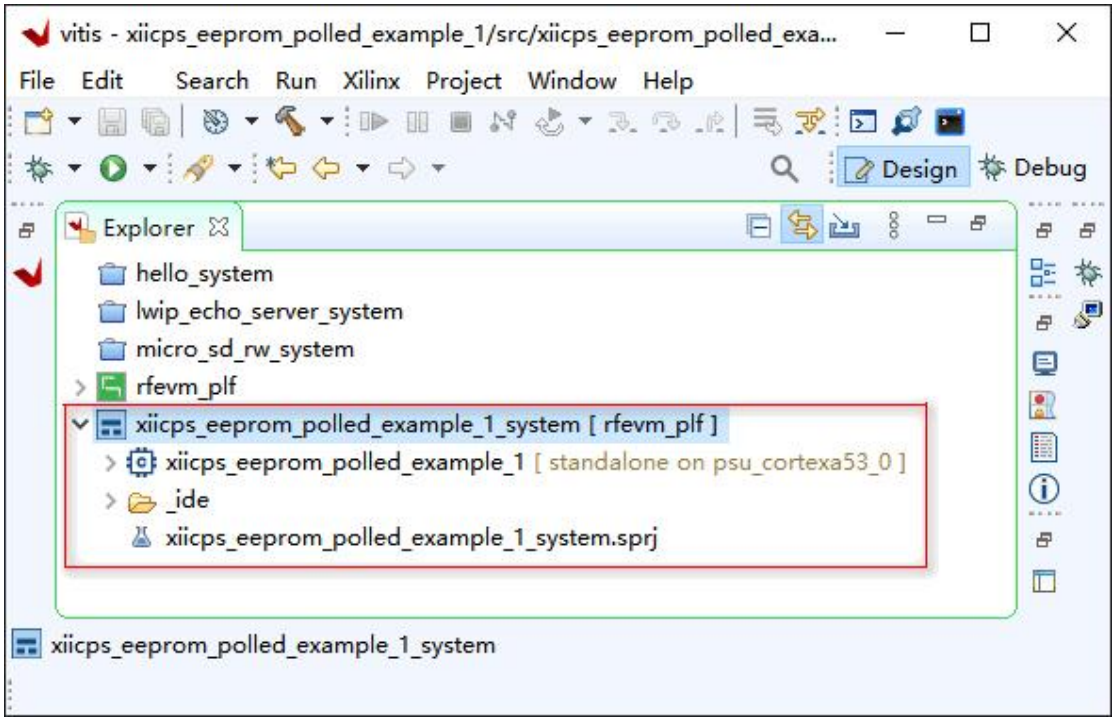
1) 登录 Vitis，切换到之前的工作目录，然后双击 platform.spr，选中 Board Support Package，可以看到 psu_usb_xhci_0 的文档链接和导入示例。



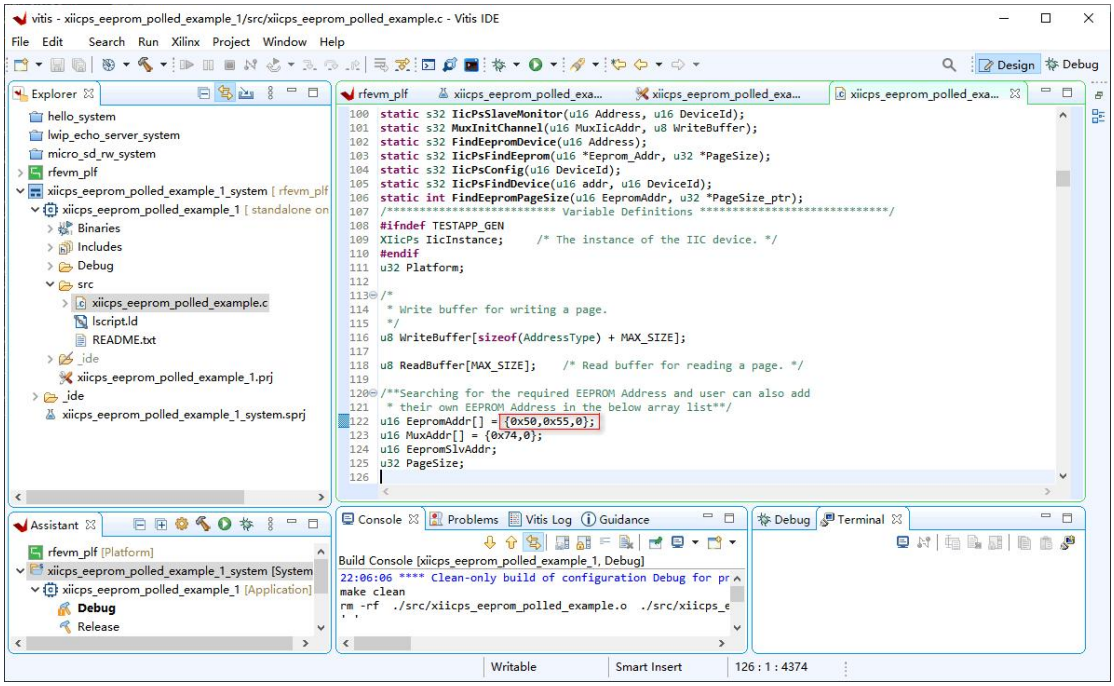
2) 点击 Import Example，然后选中 xiicps_eeprom_polled_example 示例，点击 OK



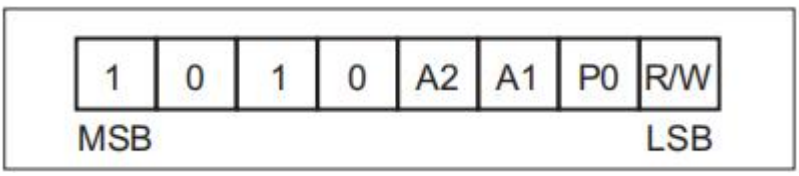
3) 可以看到导入的示例应用工程



4) EEPROM 的设备地址在程序的前面做了定义，这个地址就是系统访问外部 IIC 外设的设备地址，这里 EEPROM 地址修改为 0x50，然后保存。



EEPROM 的设备地址可以在 AT24CM01 的芯片手册上找到。

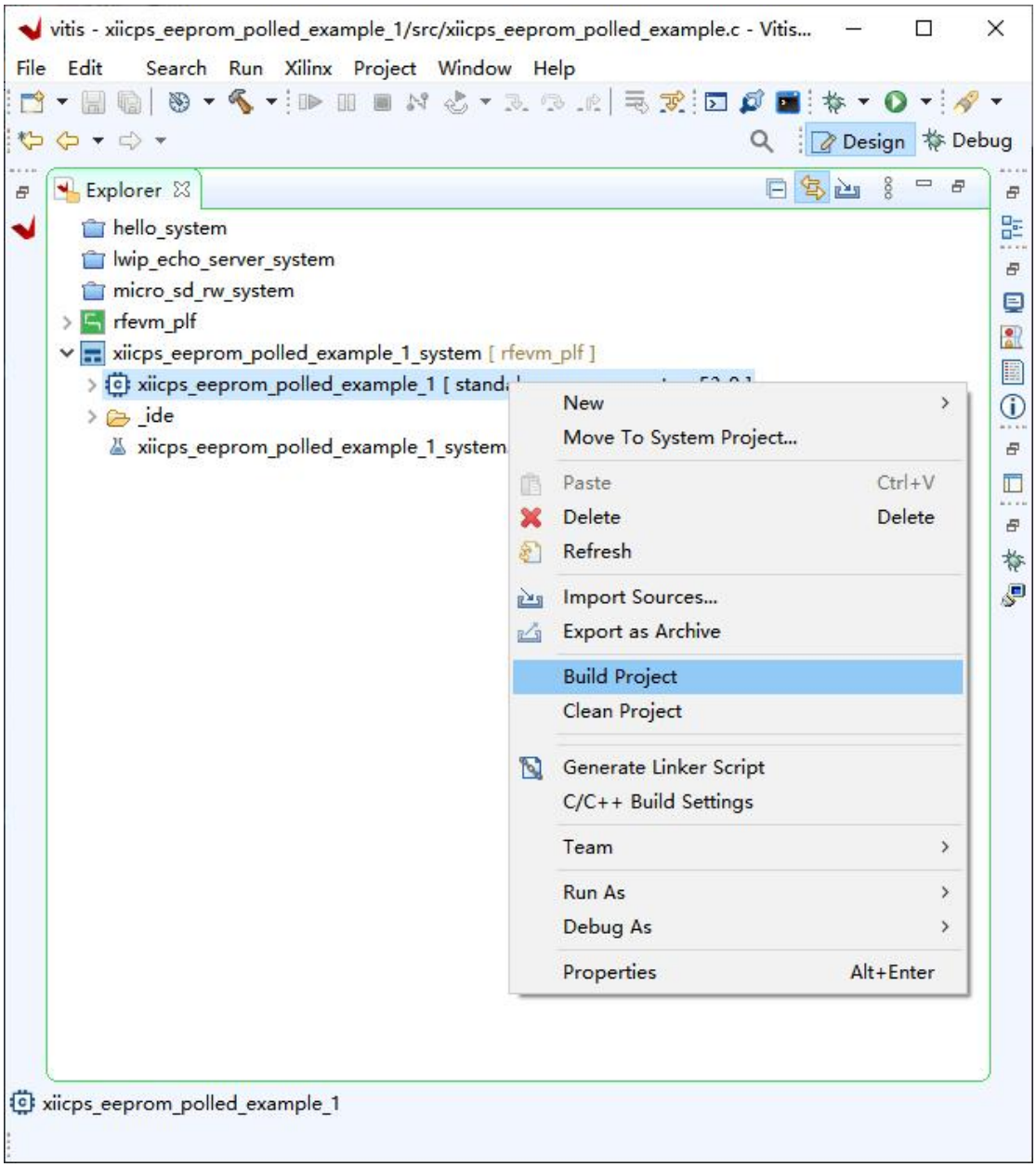


其中 A1、A2 硬件里面直接接地，为 0；

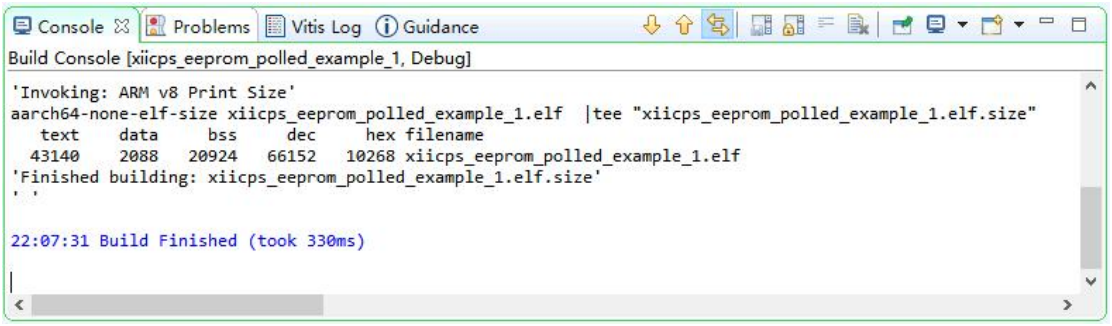
程序流程如下：

ReadBuffer 清 0，WriteBuffer 赋递增数据->写 32 个字节到 EEPROM->读 EEPROM 的 16 个字节到 ReadBuffer->检验是否正确->返回。

5) 选中应用工程，右键点击 Build Project



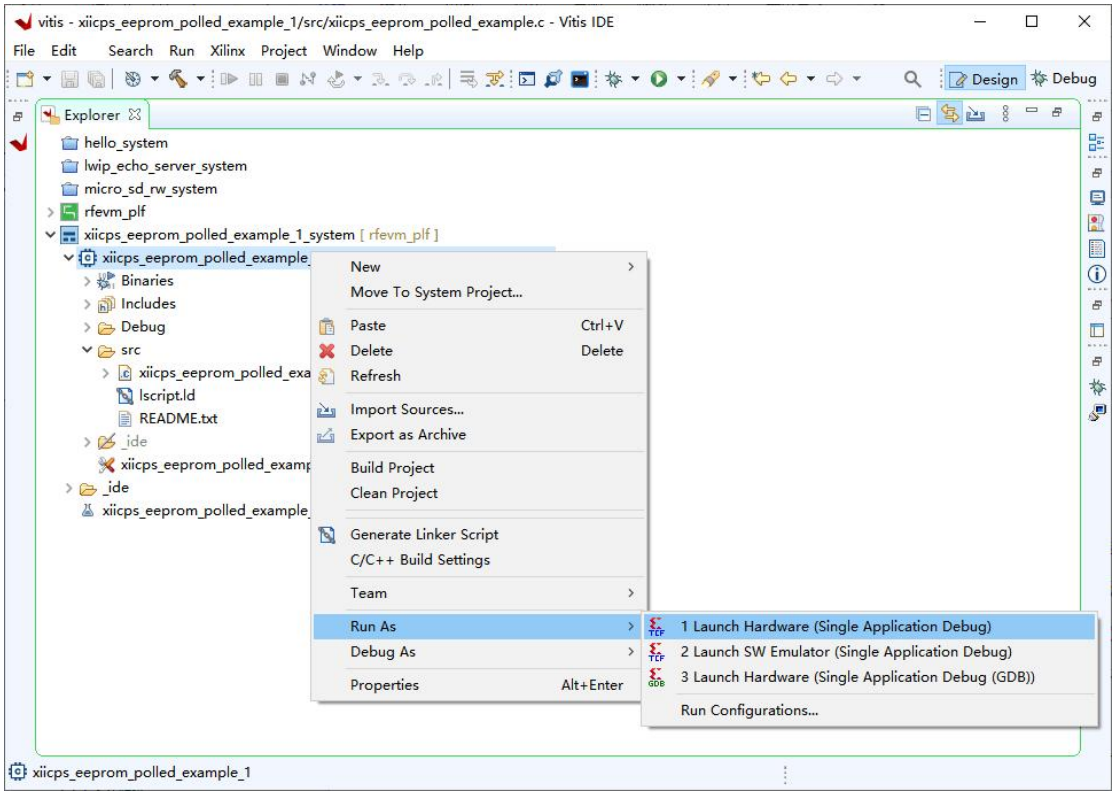
编译信息可以在 Console 窗口中看到



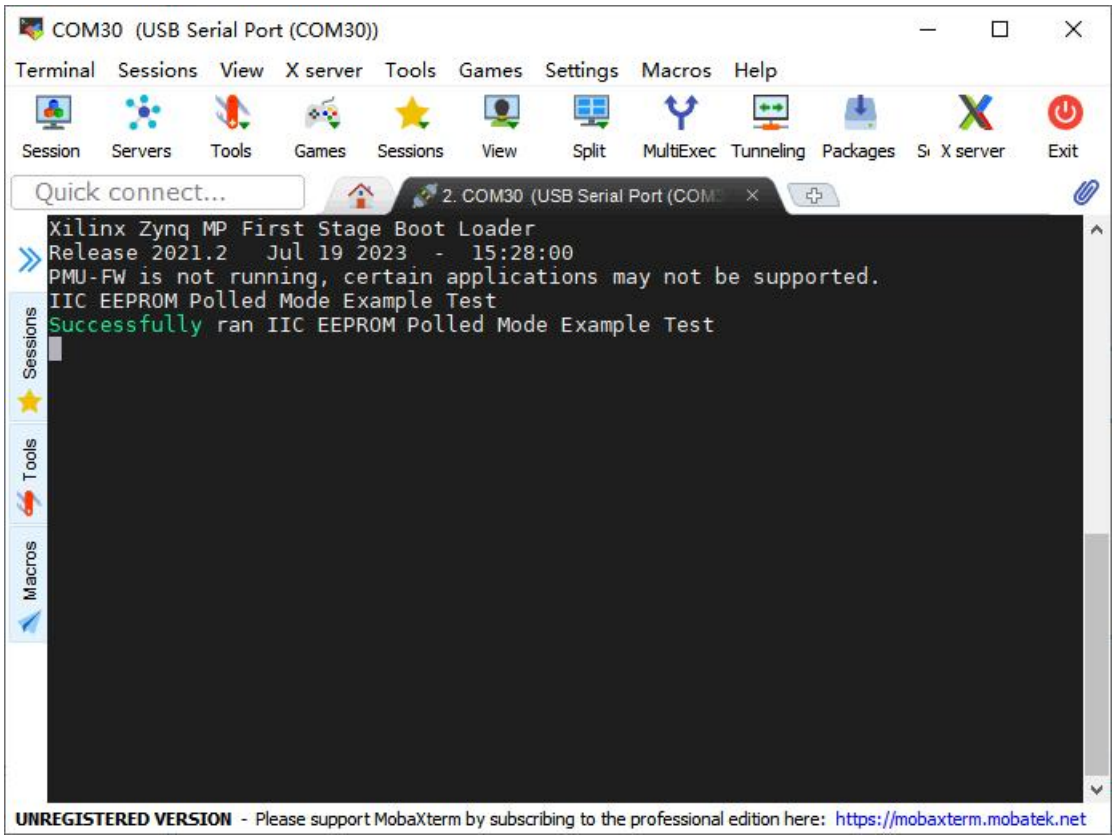
4.4.3 下载验证

首先我们将下载线（USB 转 Type-C 线缆）与开发板的 JTAG 接口连接，另外一端与电脑连接。最后打开开发板的电源开关。

右键应用工程，选择 Run As->Launch Hardware(Single Application Debug)



打开串口终端，下载完成后，可以查看串口输出：



4.5 PS 端以太网使用之 lwip

LWIP 是瑞典计算机科学院（SICS）的 Adam Dunkels 等开发的一个小型开源的 TCP/IP 协议栈，是 TCP/IP 的一种实现方式。LWIP 是轻量级 IP 协议，有无操作系统的支持都可以运行。LWIP 实现的重点是在保持 TCP 协议主要功能的基础上减少对 RAM 的占用，它只需要十几 KB 的 RAM 和 40K 左右的 ROM 就可以运行，这使 LWIP 协议栈适合在低端的嵌入式系统中使用。

这里我们使用 Vitis 软件自带的 Lwip Echo Server 模板了解 LWIP 的使用。

4.5.1 硬件设计

RFEVM 开发板上有一个 PS 端的 RJ45 以太网接口，用于连接以太网线，其原理图如图所示：

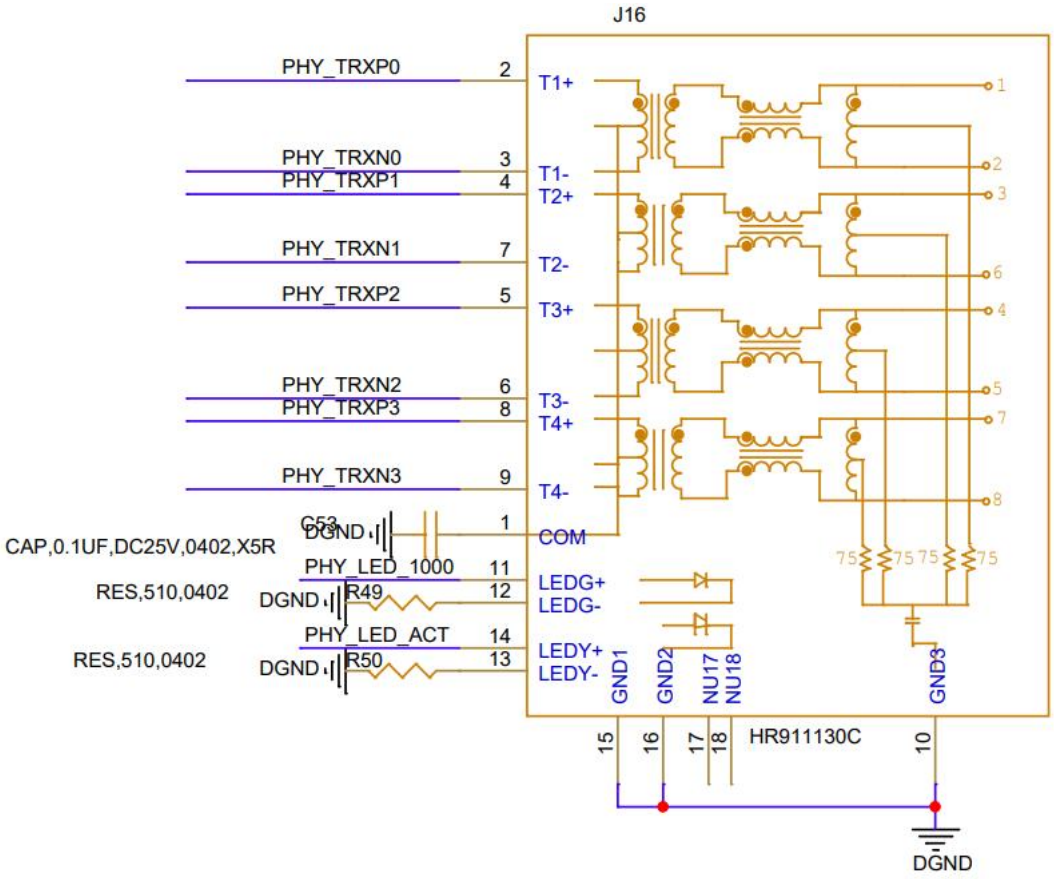


图 13 RJ45 接口原理图

以太网的数据传输离不开以太网 PHY（物理层）芯片的支持。RFEVM 开发板上使用的 PHY 芯片为创锐讯（Atheros）公司的第四代 AR8035，其部分原理图如图所示：

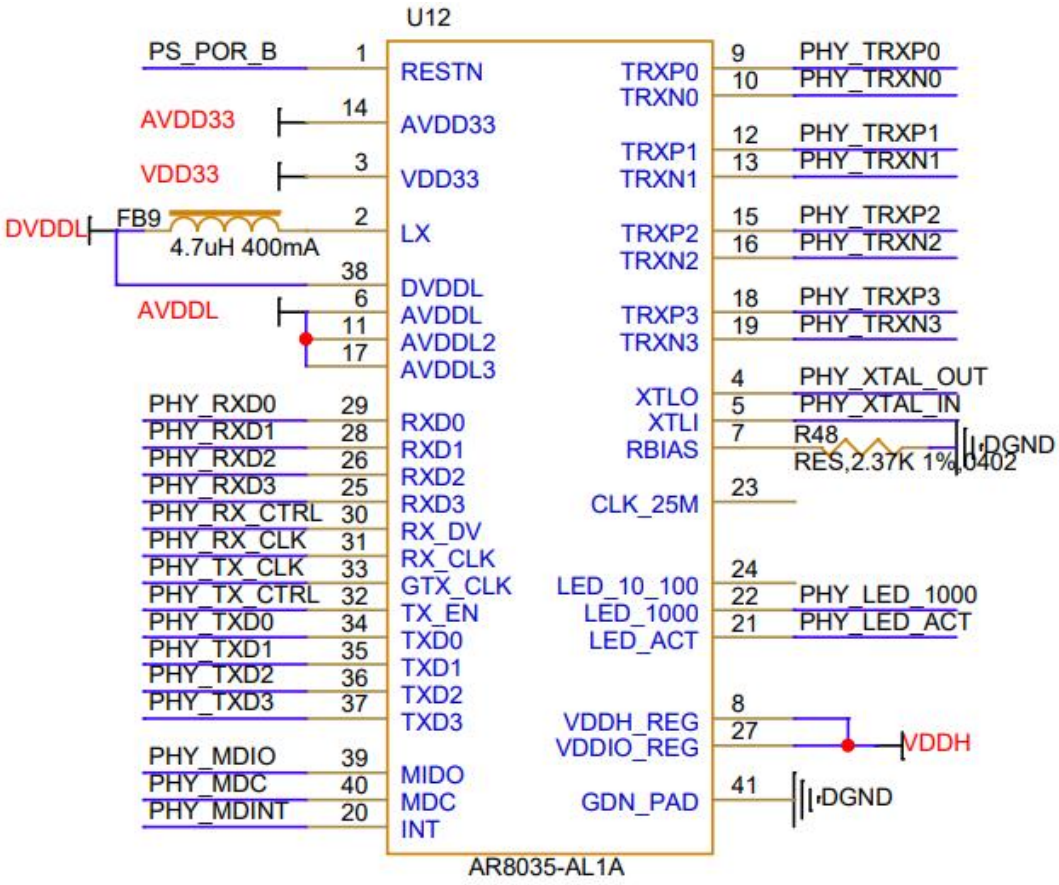


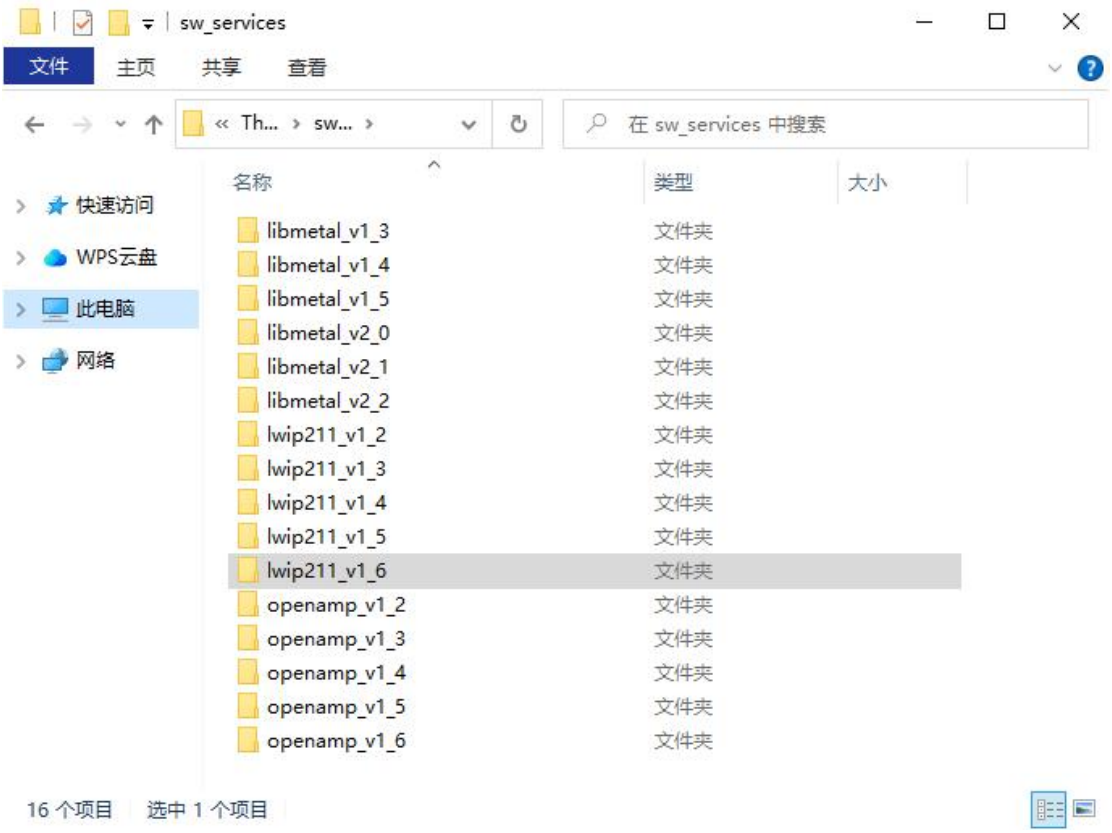
图 14 AR8035 原理图

4.5.2 软件设计

由于自带的 LWIP 库只能识别部分 phy 芯片，如果开发板所用的 phy 芯片不在默认支持范围内，需要修改库文件。也可以直接使用修改过的库替换原有的库。

找到库文件目录：

C:\Xilinx\Vitis\2021.2\data\embeddedsw\ThirdParty\sw_services



找到要修改的文件目录”lwip211_v1_6\src\contrib\ports\xilinx\netif”中文件”xemacpsif_physpeed.c”，添加如下函数。

```
static u32_t get_Atheros_phy_speed(XEmacPs *xemacpsp, u32_t phy_addr)
{
u16_t temp;
u16_t control;
u16_t status;
u16_t status_speed;
u32_t timeout_counter = 0;
u32_t temp_speed;

xil_printf("Start Atheros PHY autonegotiation \r\n");

XEmacPs_PhyRead(xemacpsp, phy_addr, 0xe, &control);
control |= 0x0018;
XEmacPs_PhyWrite(xemacpsp, phy_addr, 0xe, control);

XEmacPs_PhyWrite(xemacpsp,phy_addr, 0x1d, 0x05);
XEmacPs_PhyRead(xemacpsp, phy_addr, 0x1e, &control);
control |= 0x0100;
XEmacPs_PhyWrite(xemacpsp, phy_addr, 0x1e, control);

XEmacPs_PhyWrite(xemacpsp, phy_addr, 0x1d, 0x0);
```

```
XEmacPs_PhyWrite(xemacpsp, phy_addr, 0x1e, 0x8000);

XEmacPs_PhyWrite(xemacpsp,phy_addr, IEEE_PAGE_ADDRESS_REGISTER, 2);
XEmacPs_PhyRead(xemacpsp, phy_addr, IEEE_CONTROL_REG_MAC, &control);
control |= IEEE_RGMII_TXRX_CLOCK_DELAYED_MASK;
XEmacPs_PhyWrite(xemacpsp, phy_addr, IEEE_CONTROL_REG_MAC, control);

XEmacPs_PhyWrite(xemacpsp, phy_addr, IEEE_PAGE_ADDRESS_REGISTER, 0);

XEmacPs_PhyRead(xemacpsp, phy_addr, IEEE_AUTONEGO_ADVERTISE_REG, &control);
control |= IEEE_ASYMMETRIC_PAUSE_MASK;
control |= IEEE_PAUSE_MASK;
control |= ADVERTISE_100;
control |= ADVERTISE_10;
XEmacPs_PhyWrite(xemacpsp, phy_addr, IEEE_AUTONEGO_ADVERTISE_REG, control);

XEmacPs_PhyRead(xemacpsp, phy_addr, IEEE_1000_ADVERTISE_REG_OFFSET,
        &control);
control |= ADVERTISE_1000;
XEmacPs_PhyWrite(xemacpsp, phy_addr, IEEE_1000_ADVERTISE_REG_OFFSET,
        control);

XEmacPs_PhyWrite(xemacpsp, phy_addr, IEEE_PAGE_ADDRESS_REGISTER, 0);
XEmacPs_PhyRead(xemacpsp, phy_addr, IEEE_COPPER_SPECIFIC_CONTROL_REG,
        &control);
control |= (7 << 12); /* max number of gigabit attempts */
control |= (1 << 11); /* enable downshift */
XEmacPs_PhyWrite(xemacpsp, phy_addr, IEEE_COPPER_SPECIFIC_CONTROL_REG,
        control);
XEmacPs_PhyRead(xemacpsp, phy_addr, IEEE_CONTROL_REG_OFFSET, &control);
control |= IEEE_CTRL_AUTONEGOTIATE_ENABLE;
control |= IEEE_STAT_AUTONEGOTIATE_RESTART;
XEmacPs_PhyWrite(xemacpsp, phy_addr, IEEE_CONTROL_REG_OFFSET, control);

XEmacPs_PhyRead(xemacpsp, phy_addr, IEEE_CONTROL_REG_OFFSET, &control);
control |= IEEE_CTRL_RESET_MASK;
XEmacPs_PhyWrite(xemacpsp, phy_addr, IEEE_CONTROL_REG_OFFSET, control);

while (1) {
    XEmacPs_PhyRead(xemacpsp, phy_addr, IEEE_CONTROL_REG_OFFSET, &control);
    if (control & IEEE_CTRL_RESET_MASK)
        continue;
    else
        break;
```

```
    }

    XEmacPs_PhyRead(xemacpsp, phy_addr, IEEE_STATUS_REG_OFFSET, &status);

    xil_printf("Waiting for PHY to complete autonegotiation.\r\n");

    while ( !(status & IEEE_STAT_AUTONEGOTIATE_COMPLETE) ) {
        sleep(1);
        XEmacPs_PhyRead(xemacpsp, phy_addr,
                    IEEE_COPPER_SPECIFIC_STATUS_REG_2,    &temp);
        timeout_counter++;

        if (timeout_counter == 30) {
            xil_printf("Auto negotiation error \r\n");
            return XST_FAILURE;
        }
        XEmacPs_PhyRead(xemacpsp, phy_addr, IEEE_STATUS_REG_OFFSET, &status);
    }
    xil_printf("autonegotiation complete \r\n");

    XEmacPs_PhyRead(xemacpsp, phy_addr,IEEE_SPECIFIC_STATUS_REG,
                &status_speed);
    if (status_speed & 0x400) {
        temp_speed = status_speed & IEEE_SPEED_MASK;

        if (temp_speed == IEEE_SPEED_1000)
            return 1000;
        else if(temp_speed == IEEE_SPEED_100)
            return 100;
        else
            return 10;
    }


    return XST_SUCCESS;
    }
```

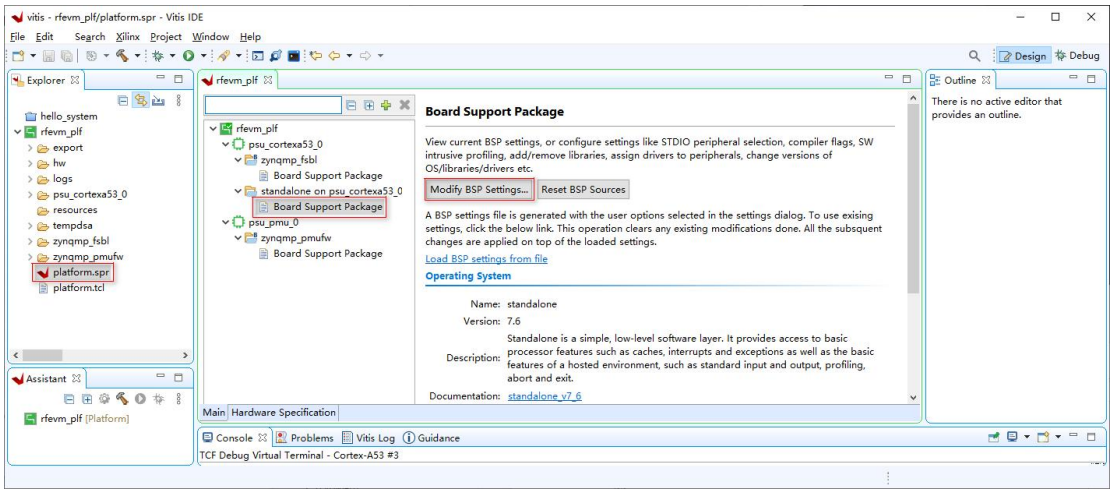
并修改函数

```
    static u32_t get_IEEE_phy_speed(XEmacPs *xemacpsp, u32_t phy_addr)
    {
    u16_t phy_identity;
    u32_t RetStatus;

    XEmacPs_PhyRead(xemacpsp, phy_addr, PHY_IDENTIFIER_1_REG,
                            &phy_identity);
```

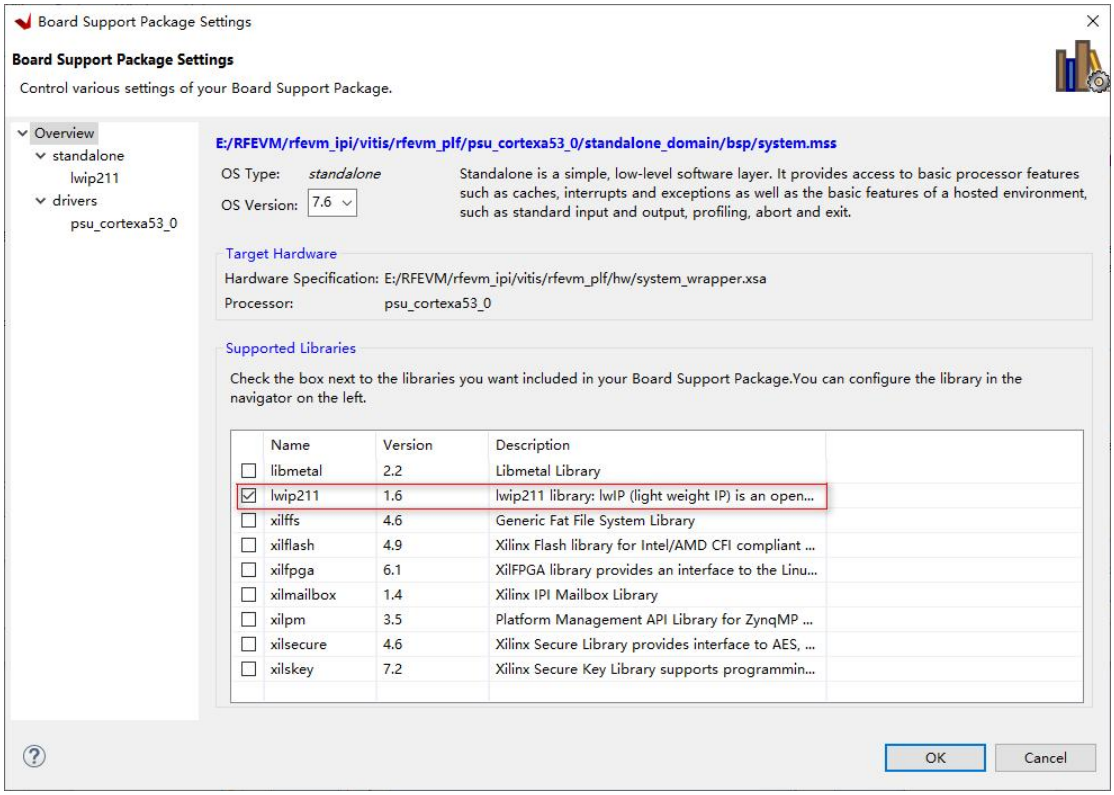
```
if (phy_identity == PHY_TI_IDENTIFIER) {
        RetStatus = get_TI_phy_speed(xemacpsp, phy_addr);
} else if (phy_identity == PHY_REALTEK_IDENTIFIER) {
        RetStatus = get_Realtek_phy_speed(xemacpsp, phy_addr);
} else if (phy_identity == PHY_MARVELL_IDENTIFIER) {
        RetStatus = get_Marvell_phy_speed(xemacpsp, phy_addr);
} else{
        RetStatus = get_Atheros_phy_speed(xemacpsp, phy_addr);
}

return RetStatus;
}
```

BSP 库文件里面添加 lwip 库

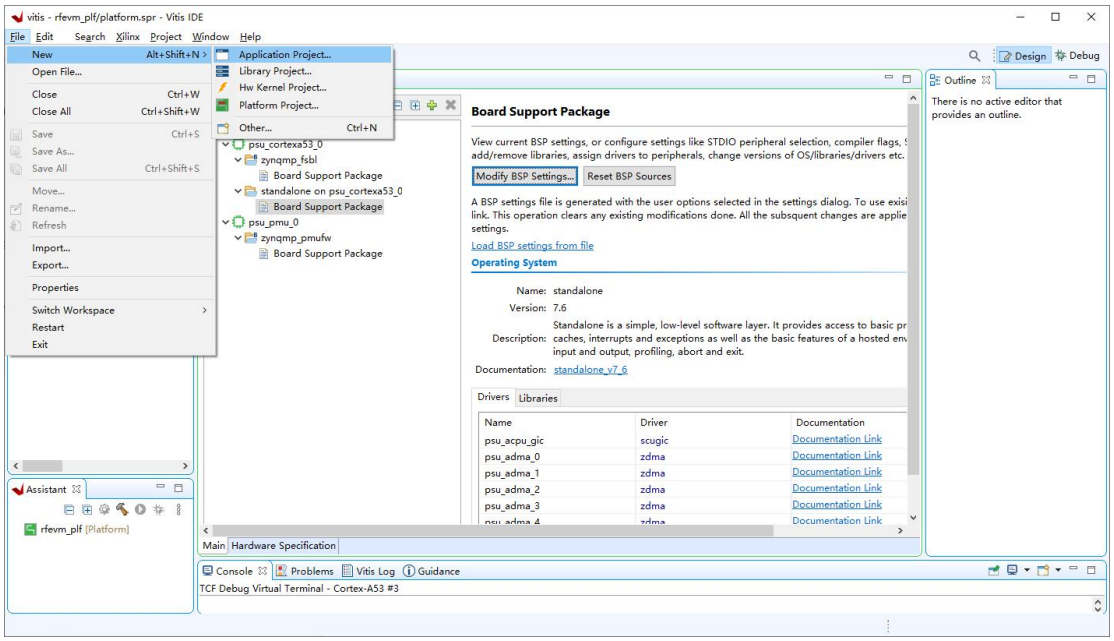
双击 platform.spr，选中 Board Support Package，点击 Modify BSP Settings...



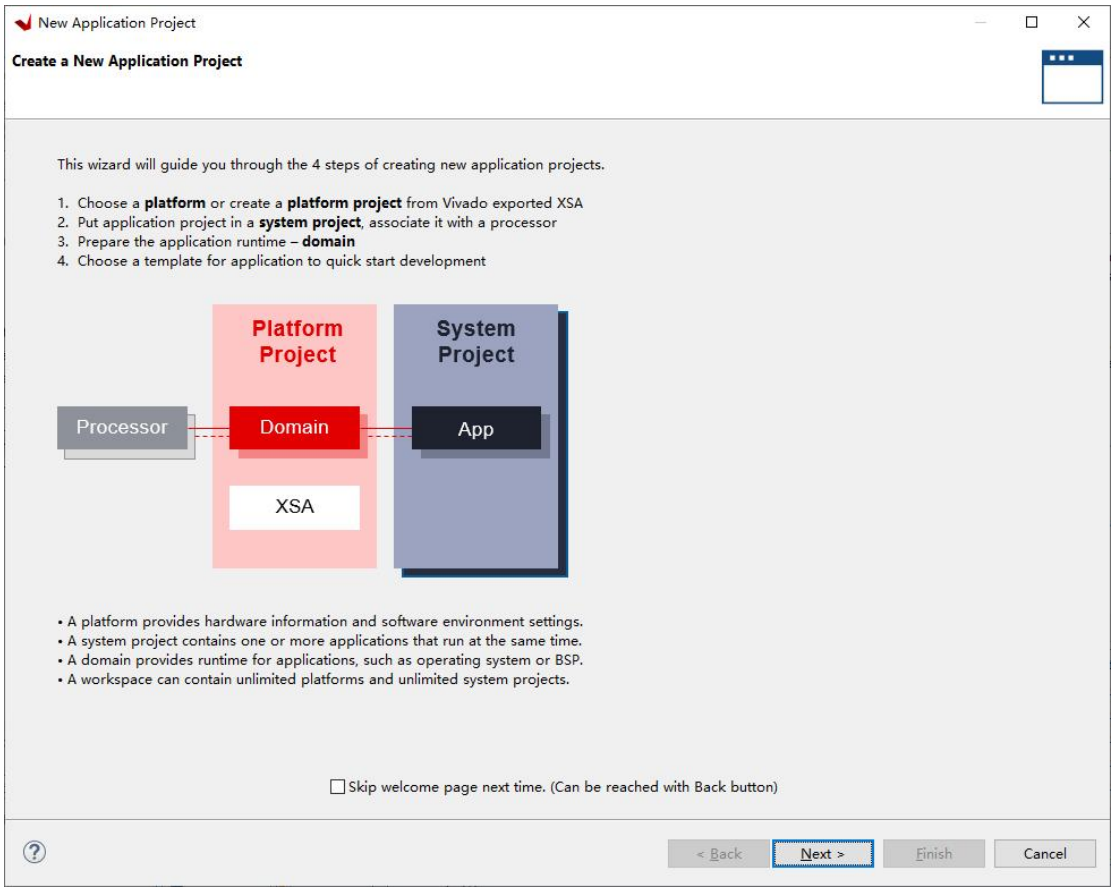
使能 lwip 库，点击 OK



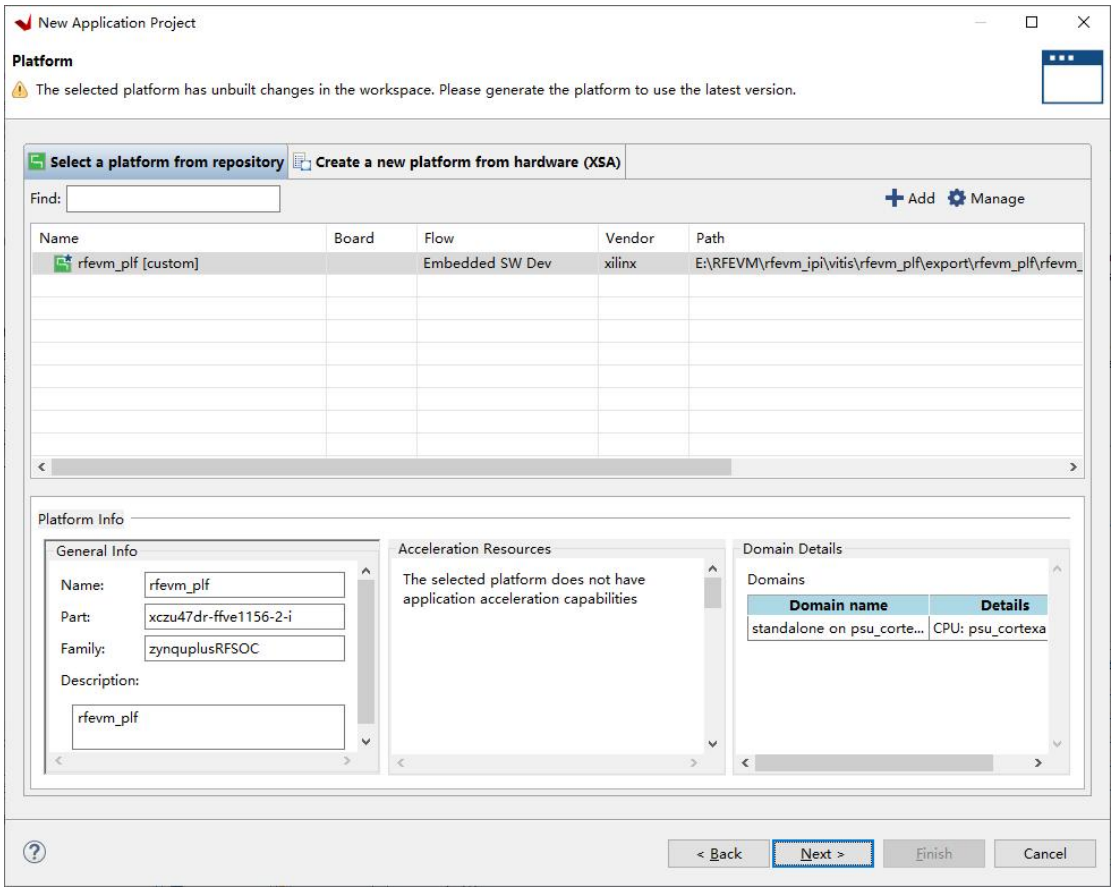
新建应用程序 File->New->Application Project



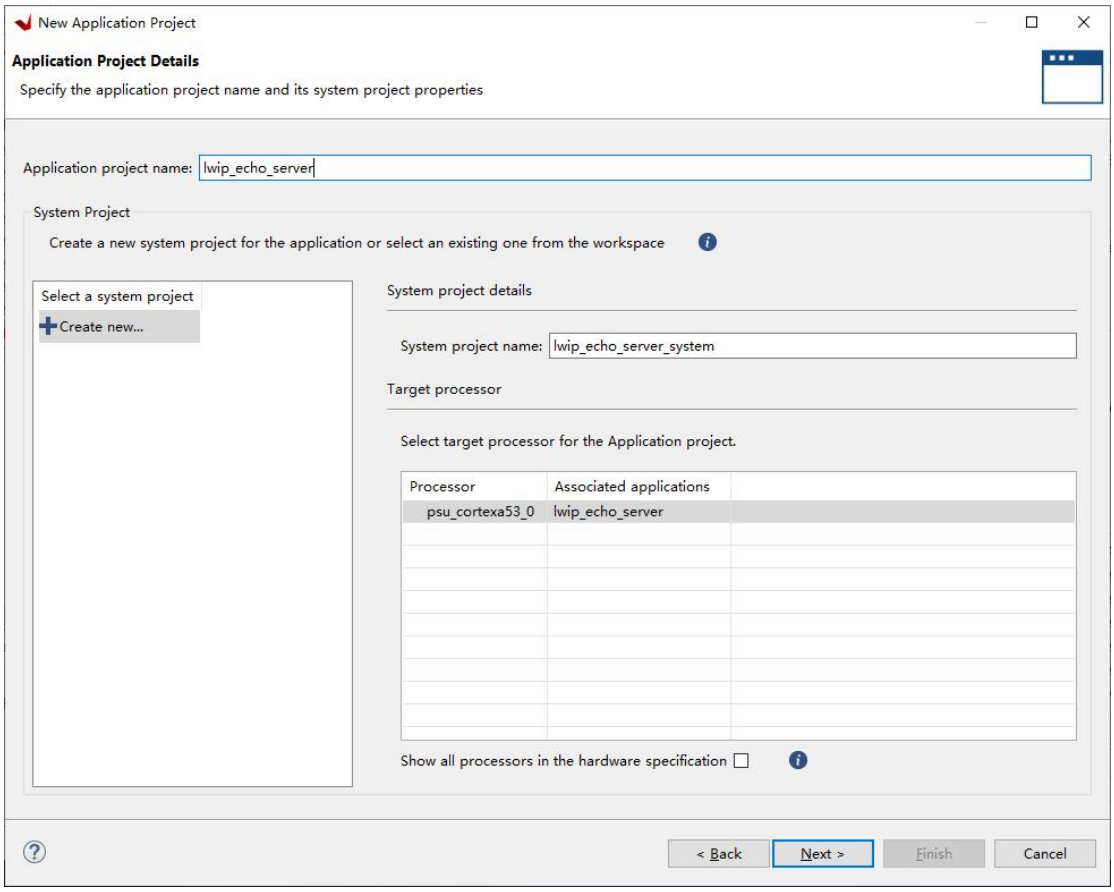
点击 Next


New Application Project
Create a New Application Project
This wizard will guide you through the 4 steps of creating new application projects.
1. Choose a platform or create a platform project from Vivado exported XSA
2. Put application project in a system project, associate it with a processor
3. Prepare the application runtime – domain
4. Choose a template for application to quick start development
Platform Project
System Project
Processor
Domain
App
XSA
• A platform provides hardware information and software environment settings.
• A system project contains one or more applications that run at the same time.
• A domain provides runtime for applications, such as operating system or BSP.
• A workspace can contain unlimited platforms and unlimited system projects.
Skip welcome page next time. (Can be reached with Back button)
< Back
Next >
Finish
Cancel

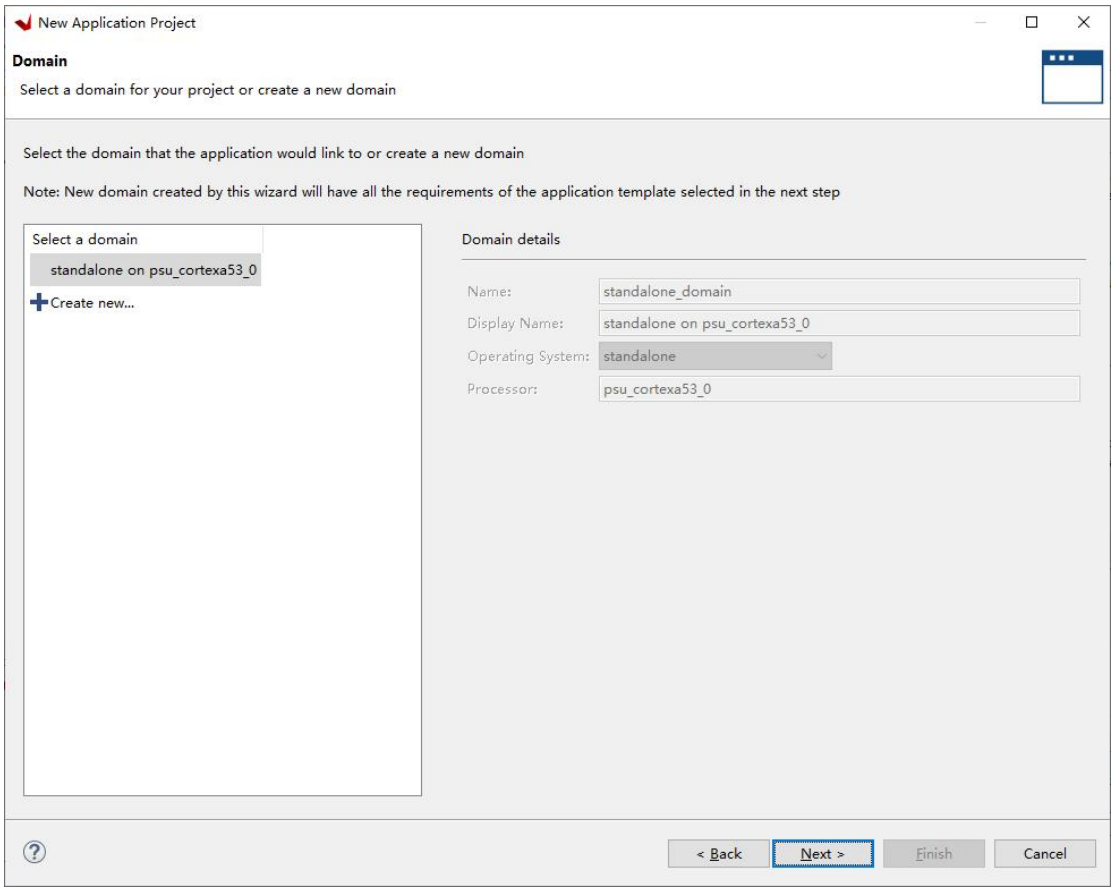
点击 Next



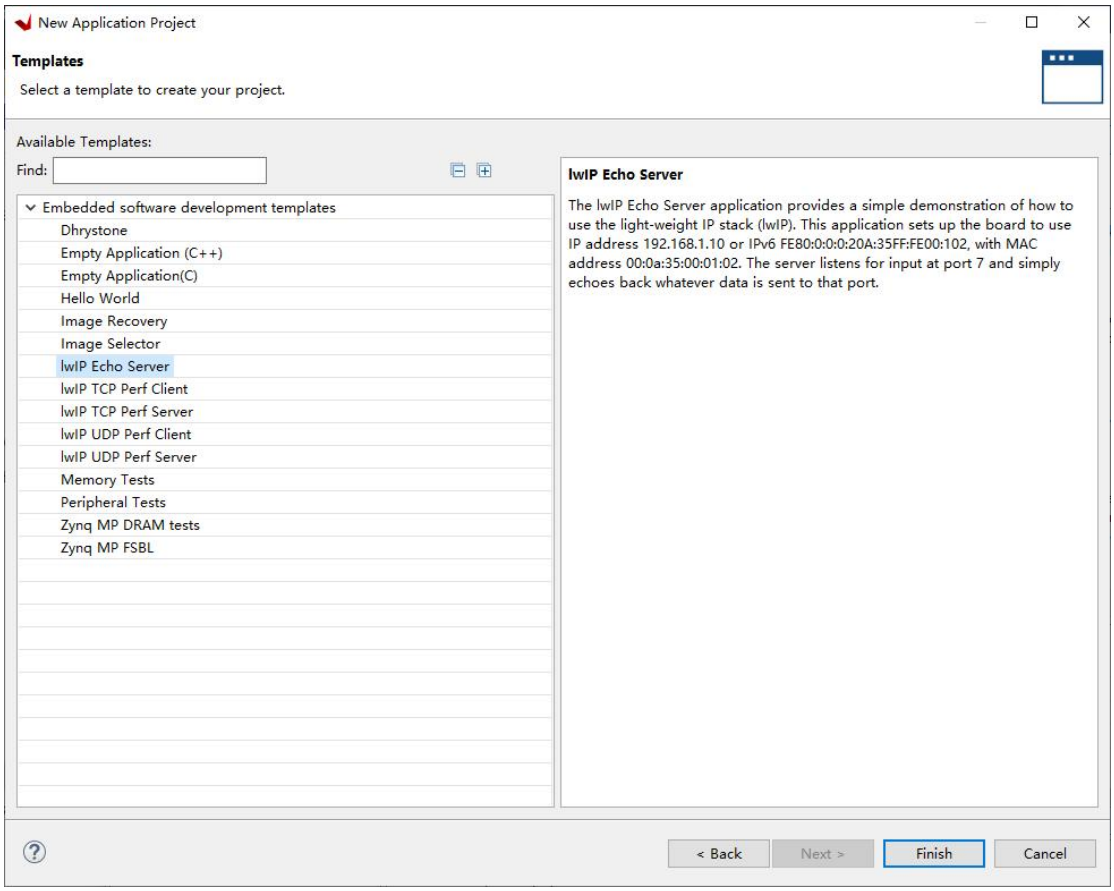
设置 APP 名称，点击 Next


New Application Project
Application Project Details
Specify the application project name and its system project properties
Application project name: lwip_echo_server
System Project
Create a new system project for the application or select an existing one from the workspace
Select a system project
Create new...
System project details
System project name: lwip_echo_server_system
Target processor
Select target processor for the Application project.
Processor
Associated applications
psu_cortexa53_0
lwip_echo_server
Show all processors in the hardware specification
< Back
Next >
Finish
Cancel

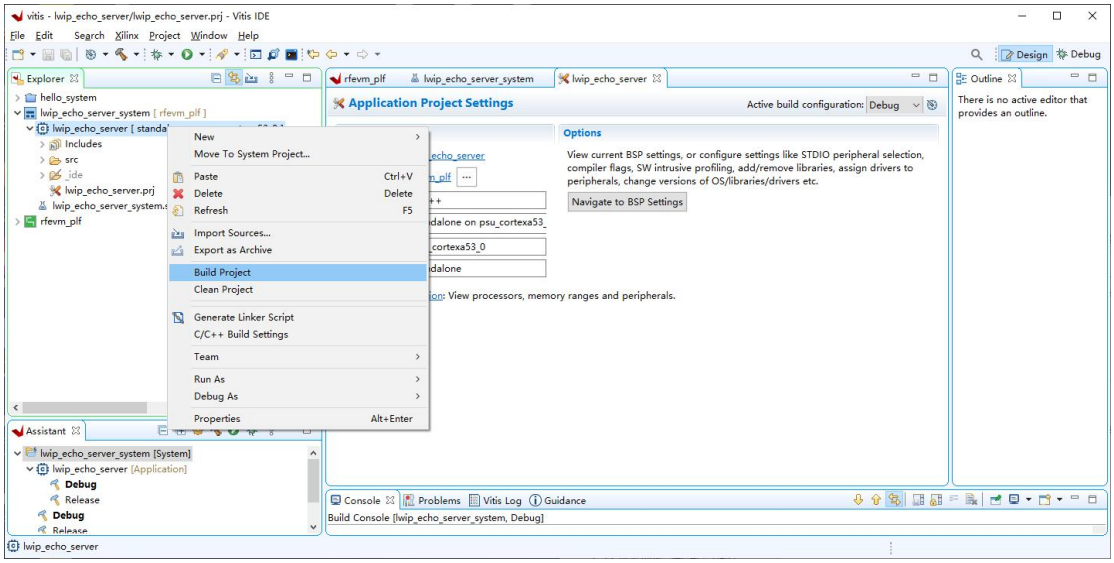
点击 Next

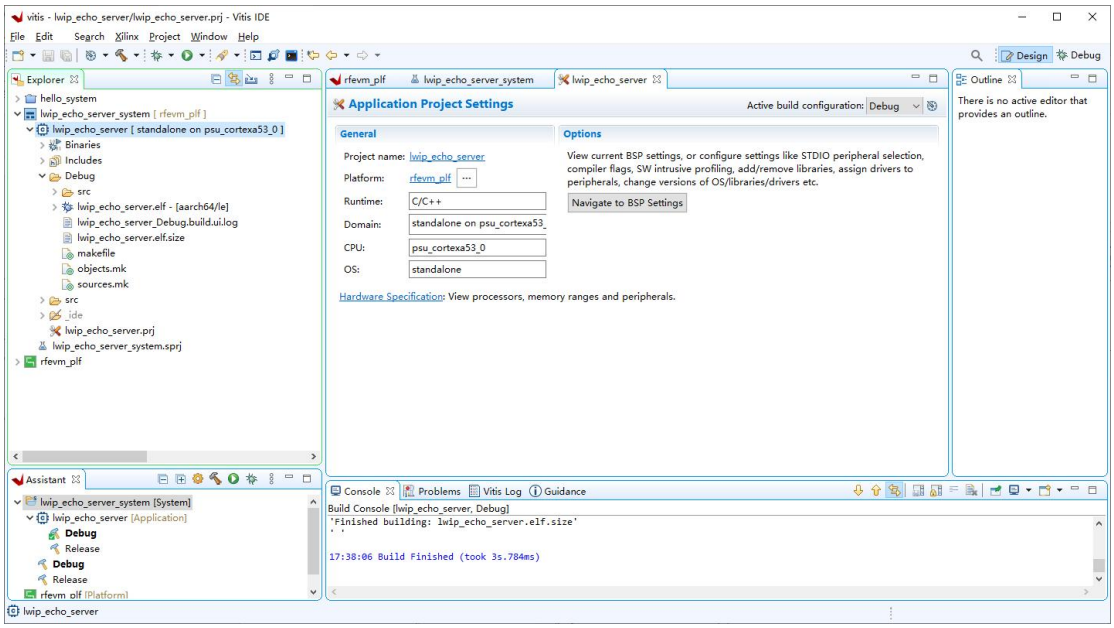


选择 Lwip Echo Server 模板，点击 Finish



右键 lwip_echo_server 工程，点击 Build Project





4.5.3 下载验证

1) 这里我们用千兆网线连接 PC 和 RFEVM 开发板，然后给板卡上电

2) PC 和 RFEVM 的 IP 地址要在同一个网段，才可以相互通信。

RFEVM 以太网的的默认配置：

mac 地址：00: 0a: 35: 00: 01: 02

IP 地址：192.168.1.10

子网掩码：255.255.255.0

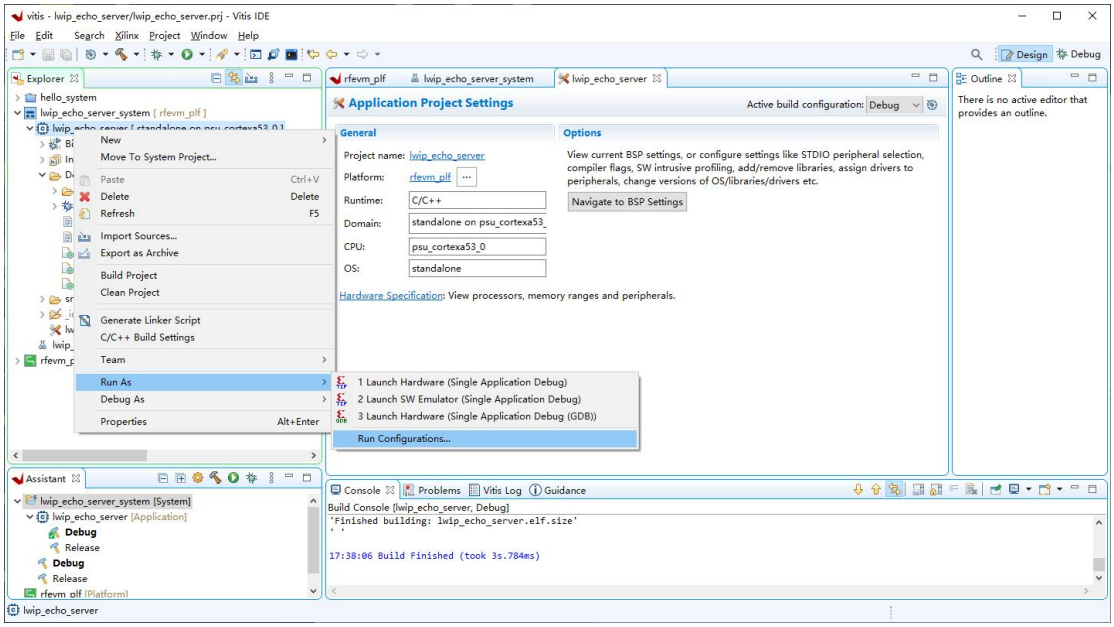
网关：192.168.1.1

Disassembly | rfevm_plf | main.c

```
199     ip_addr_t ipaddr, netmask, gw;
200
201 #endif
202     /* the mac address of the board. this should be unique per board */
203     unsigned char mac_ethernet_address[] =
204     { 0x00, 0x0a, 0x35, 0x00, 0x01, 0x02 };
205
206     echo_netif = &server_netif;
207 #if defined (__arm__) && !defined (ARMR5)
208 #if XPAR_GIGE_PCS_PMA_SGMII_CORE_PRESENT == 1 || XPAR_GIGE_PCS_PMA_1000BASEX_CORE_PRESENT == 1
209     ProgramSi5324();
210     ProgramSfpPhy();
211 #endif
212 #endif
213
214 /* Define this board specific macro in order perform PHY reset on ZCU102 */
215 #ifdef XPS_BOARD_ZCU102
216     if(IicPhyReset()) {
217         xil_printf("Error performing PHY reset \n\r");
218         return -1;
219     }
220 #endif
221
222     init_platform();
223
224 #if LWIP_IPV6==0
225 #if LWIP_DHCP==1
226     ipaddr.addr = 0;
227     gw.addr = 0;
228     netmask.addr = 0;
229 #else
230     /* initialize IP addresses to be used */
231     IP4_ADDR(&ipaddr,  192, 168,   1, 10);
232     IP4_ADDR(&netmask, 255, 255, 255,  0);
233     IP4_ADDR(&gw,      192, 168,   1,  1);
234 #endif
235 #endif
236     print_app_header();
237
238     lwip_init();
239
240 #if (LWIP_IPV6 == 0)
241     /* Add network interface to the netif_list, and set it as default */
242     if (!xemac_add(echo_netif, &ipaddr, &netmask,
```

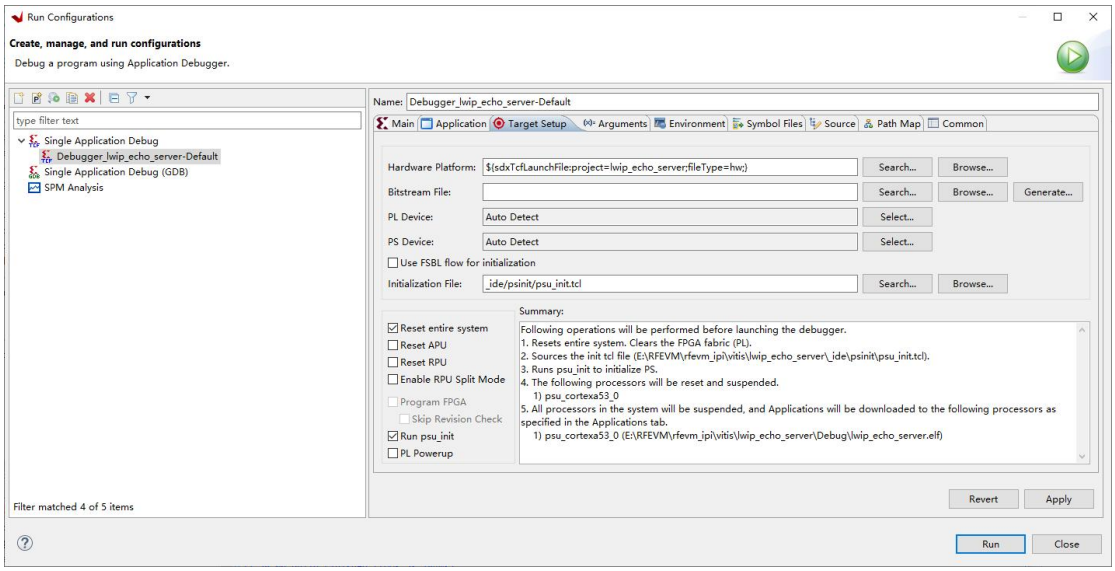
这里我们配置电脑的 IP 地址和板卡处在同一网段

3) 右键 lwip_echo_server 工程，选择 Run As->Run Configuration

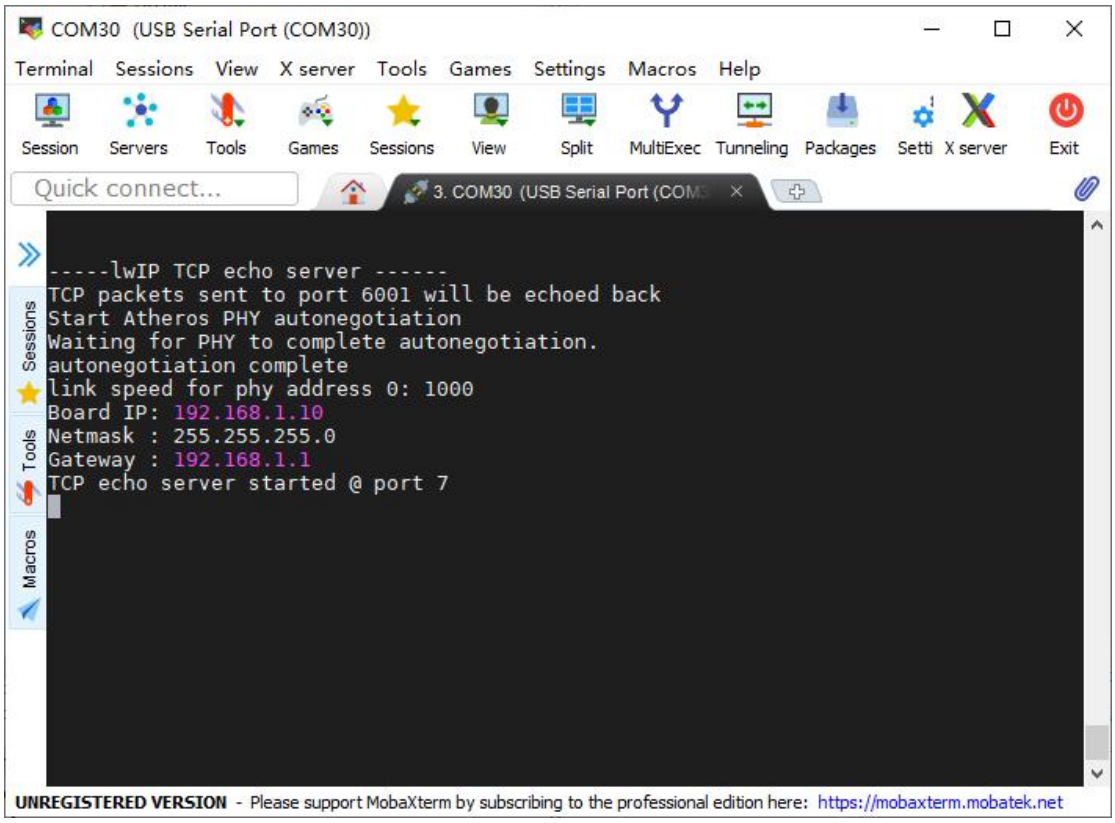


双击 Single Application Debug，生成 Debugger_lwip_echo_server；在 Target Setup 页面进行相关设置。*（这里我们只有 PS 部分，所以不用使能 Program FPGA）*

设置完成之后，点击 Apply->Run

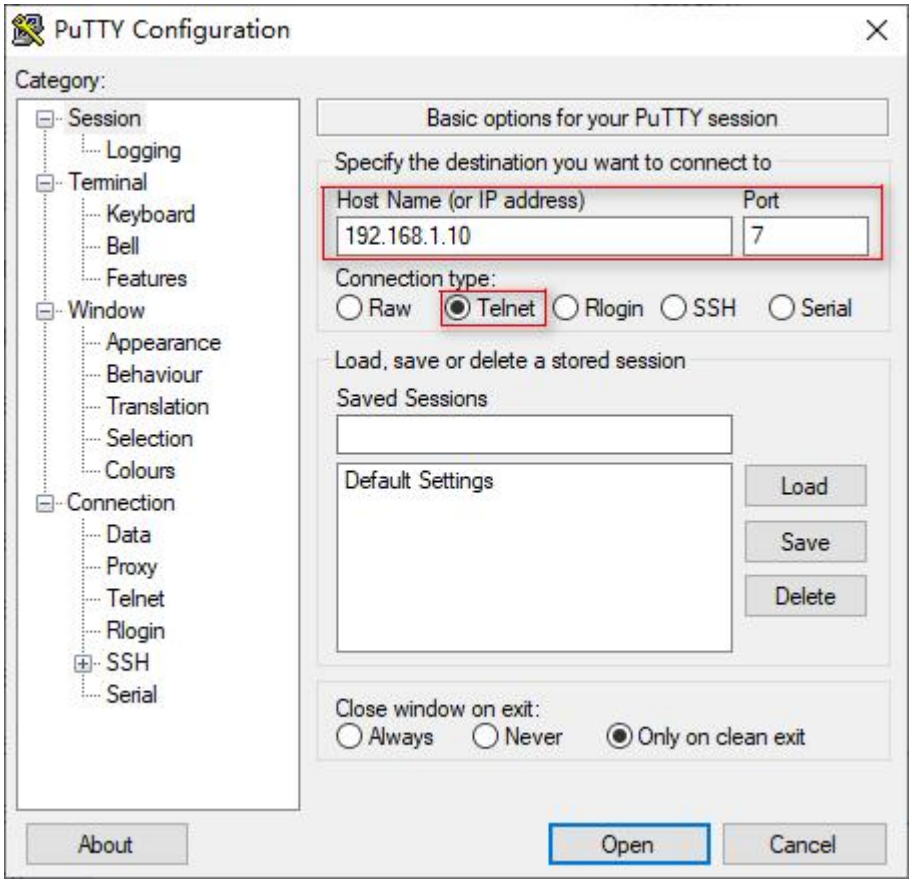


4) 我们打开串口调试终端，下载完程序后，终端会显示程序的打印信息



5) 打开 cmd 窗口，ping 板卡 IP

6) 打开 putty 工具，使用 telnet 连接



7) 当输入一个字符时，开发板回传相同字符

4.6.4 参考文档

https://xilinx-wiki.atlassian.net/wiki/spaces/A/pages/18842366/Standalone+LWIP+lib

rary

4.6 SD 卡读写 TXT 文本实验

RFEVM 底板上 SD 卡接口为小卡的设计，可以连接 Micro SD 卡（也叫 TF 卡），MPSOC 内部集成了两个 SD 卡控制器，并且 Xilinx Vitis 的 standlone 已经移植好了 FATFS（Vitis 软件中叫做 xilffs）文件系统，FATFS 是一个完全开源免费的 FAT 文件系统模块，专门为小型的嵌入式系统而设计。它完全用标准 C 语言编写，所以有良好的硬件平台独立性，可以很方便的移植到各种嵌入式处理器中。因此在 Vitis 中添加 xilffs 库后，就可以在程序中使用 FATFS 中的 API 函数来操作 SD 卡。

使用者无需理会 FATFS 的内部结构和复杂的 FAT 协议，只需要调用 FATFS 模块提供给用户的一系列应用接口函数，如 f_open、f_read、f_write 和 f_close 等，就可以像在 PC 上读/写文件那样简单。

4.6.1 硬件设计

我们的 RFEVM 开发板上面有一个 SD 卡接口，可以连接 Micro SD 卡（TF 卡），原理图如下图所示：

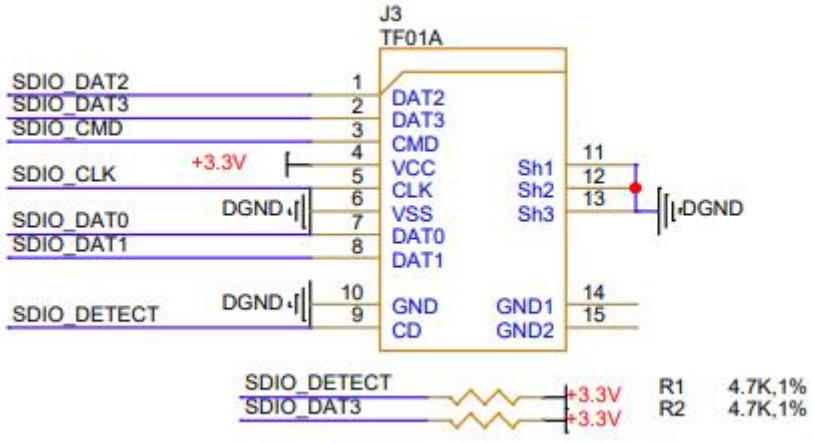


图 15 Micro SD 卡原理图

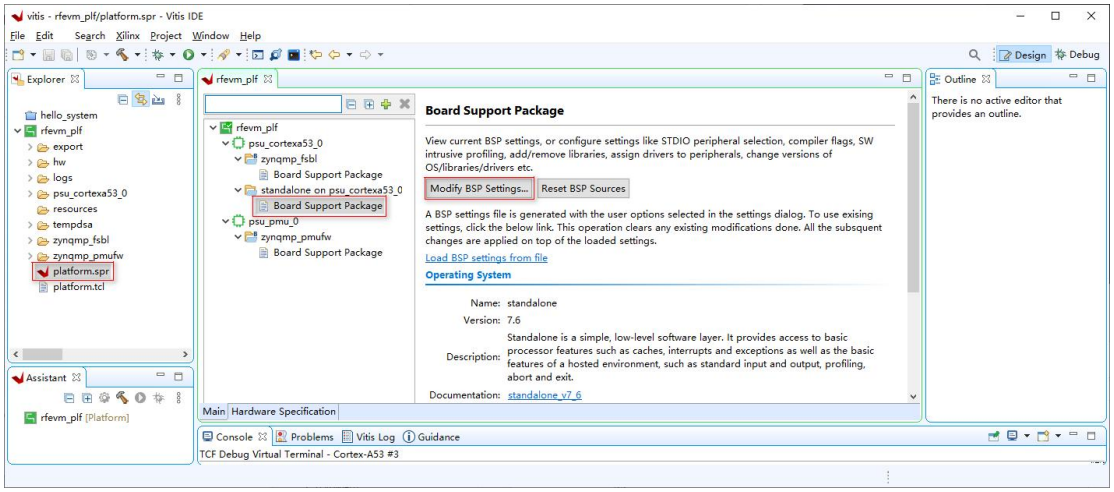
由上图可知，SD 卡共有四根数据线（SDIO_DAT0~SDIO_DAT3），连接至 MPSOC 的引脚。图中 SDIO_DETECT 信号是 TF 卡检测信号，用于标志开发板是否连接了 TF 卡。当开发板没有连接 TF 卡时，此信号为高电平；而当开发板

连接了 TF 卡之后，此信号为低电平，表示开发板已经连接了 TF 卡。

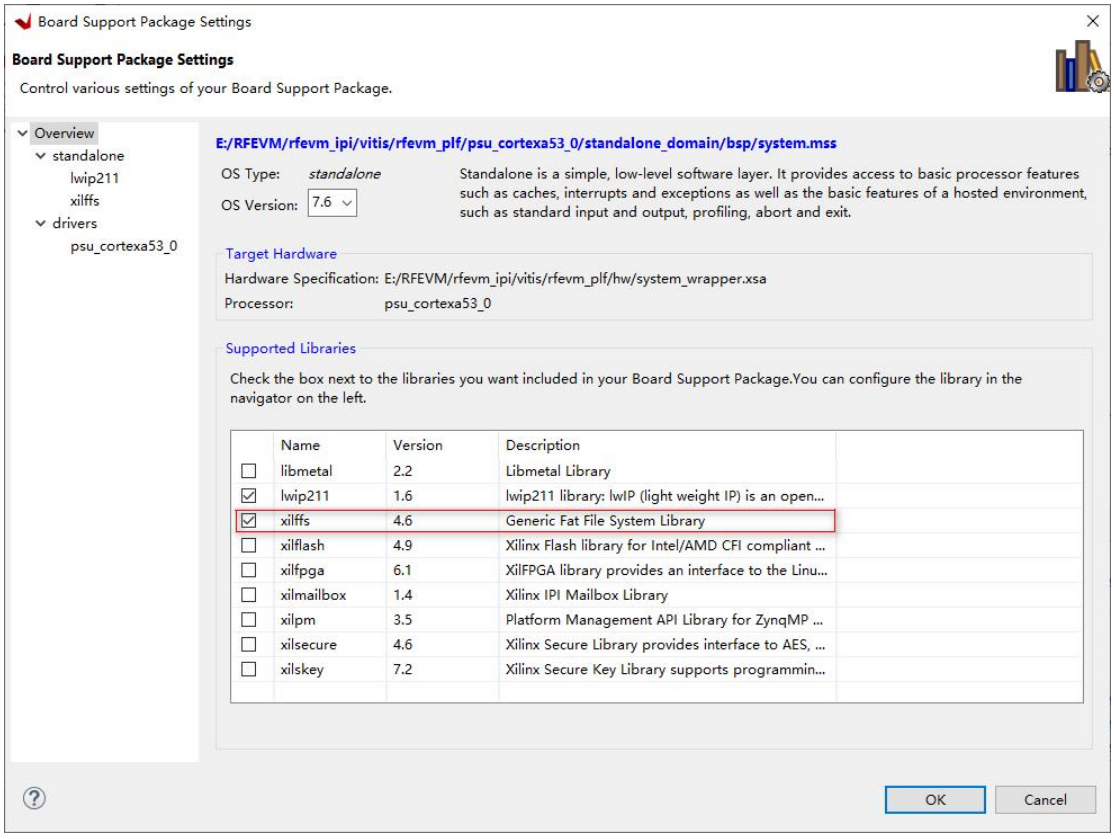
4.6.2 软件设计

BSP 库文件里面添加 xilffs 库

双击 platform.spr，选中 Board Support Package，点击 Modify BSP Settings...

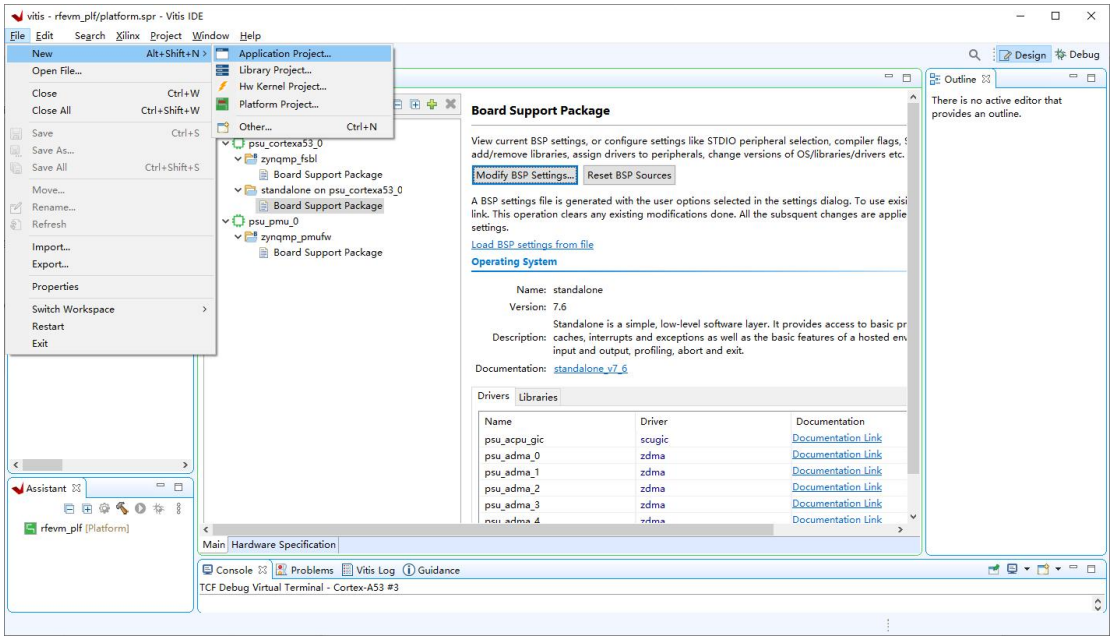


使能“xilffs”库，点击“OK”

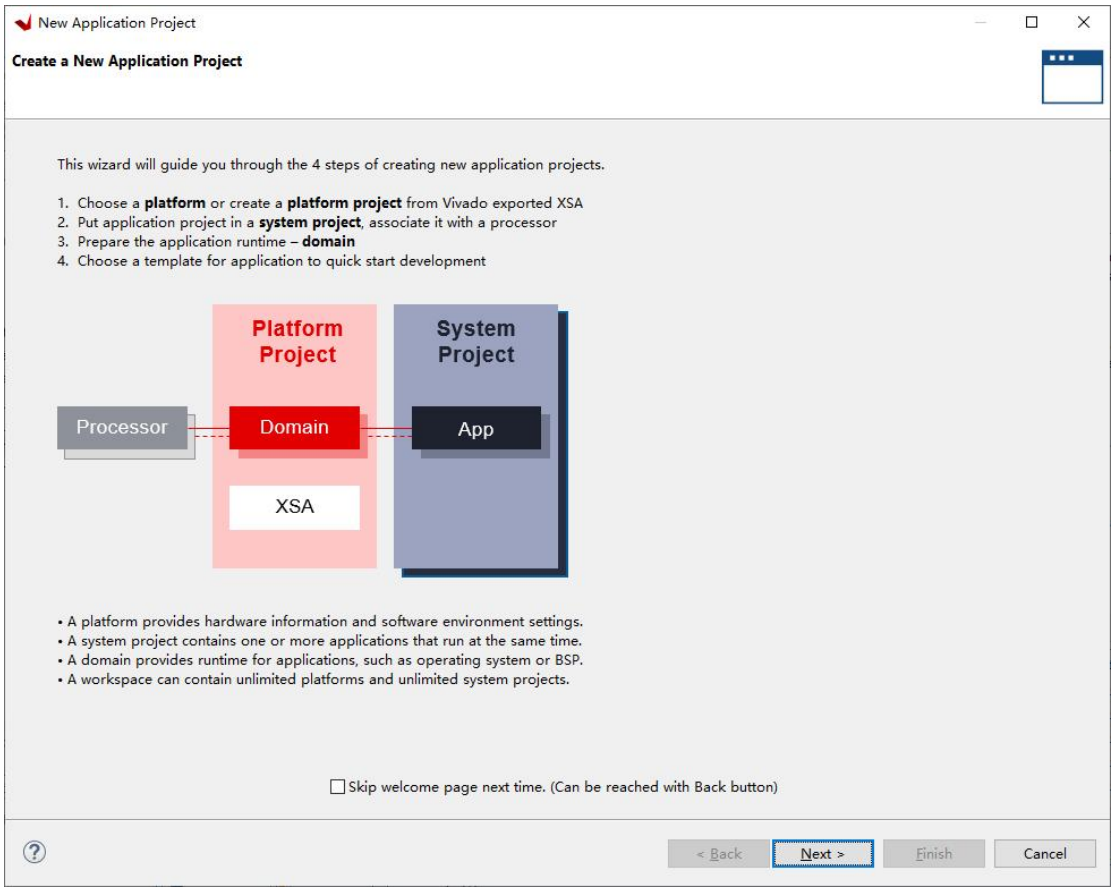


添加完 FATFS 库之后，对 TF 卡的操作会简单很多，我们只需要在程序中调用 FATFS 的库函数即可。

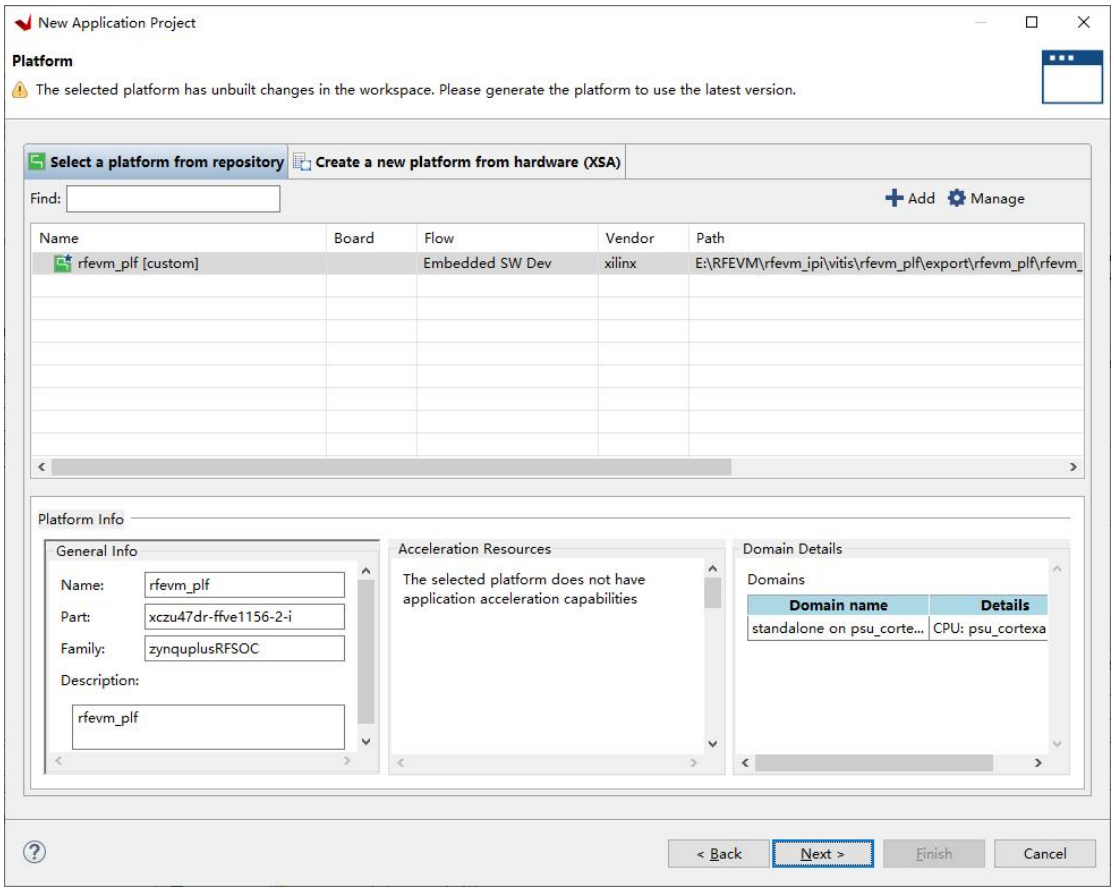
新建应用程序 File->New->Application Project



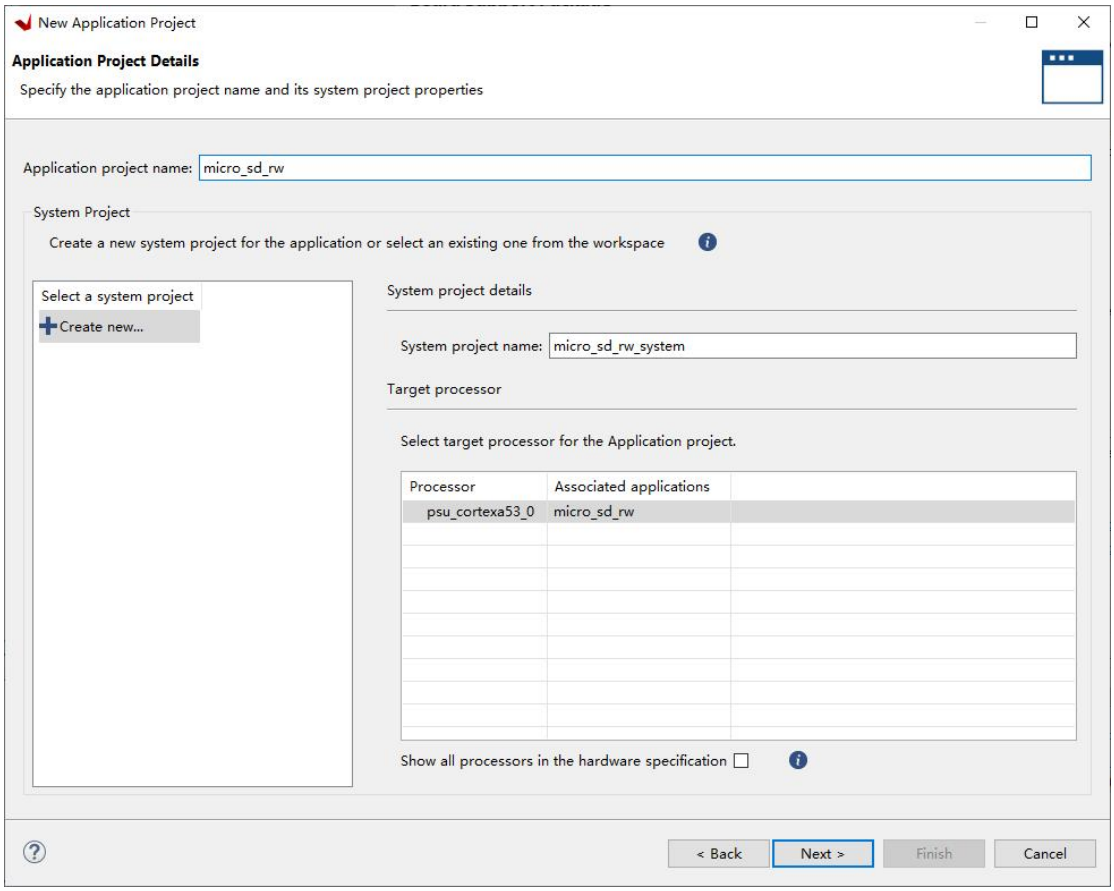
点击 Next



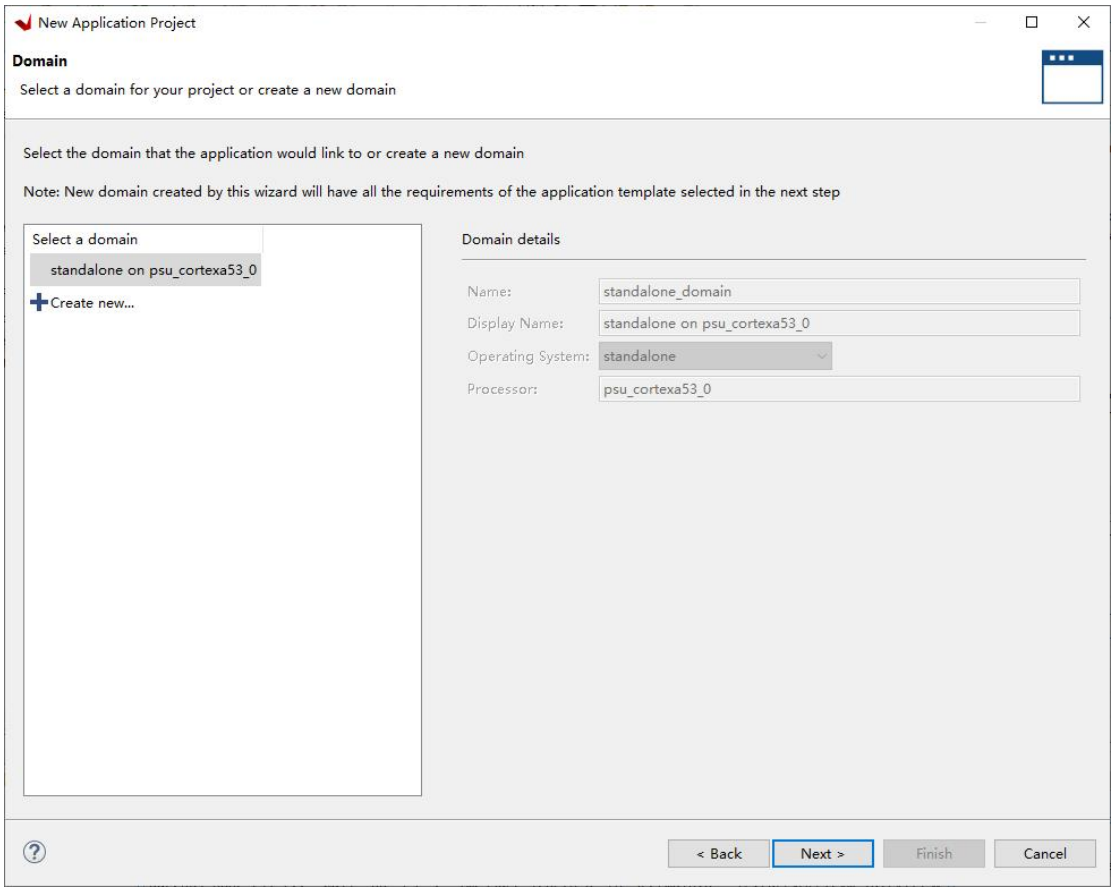
点击 Next



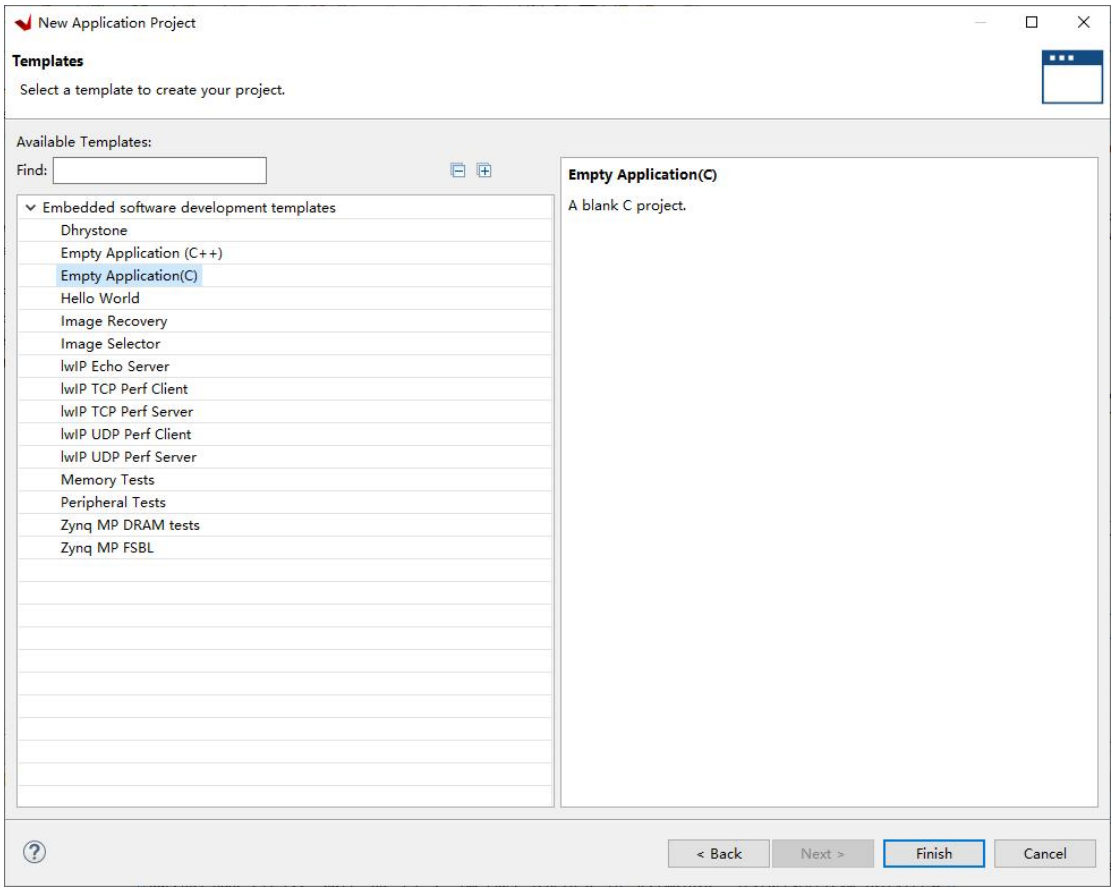
设置应用名称，点击“Next”


New Application Project
Application Project Details
Specify the application project name and its system project properties
Application project name: micro_sd_rw
System Project
Create a new system project for the application or select an existing one from the workspace
Select a system project
Create new...
System project details
System project name: micro_sd_rw_system
Target processor
Select target processor for the Application project.
Processor
Associated applications
psu_cortexa53_0
micro_sd_rw
Show all processors in the hardware specification
< Back
Next >
Finish
Cancel

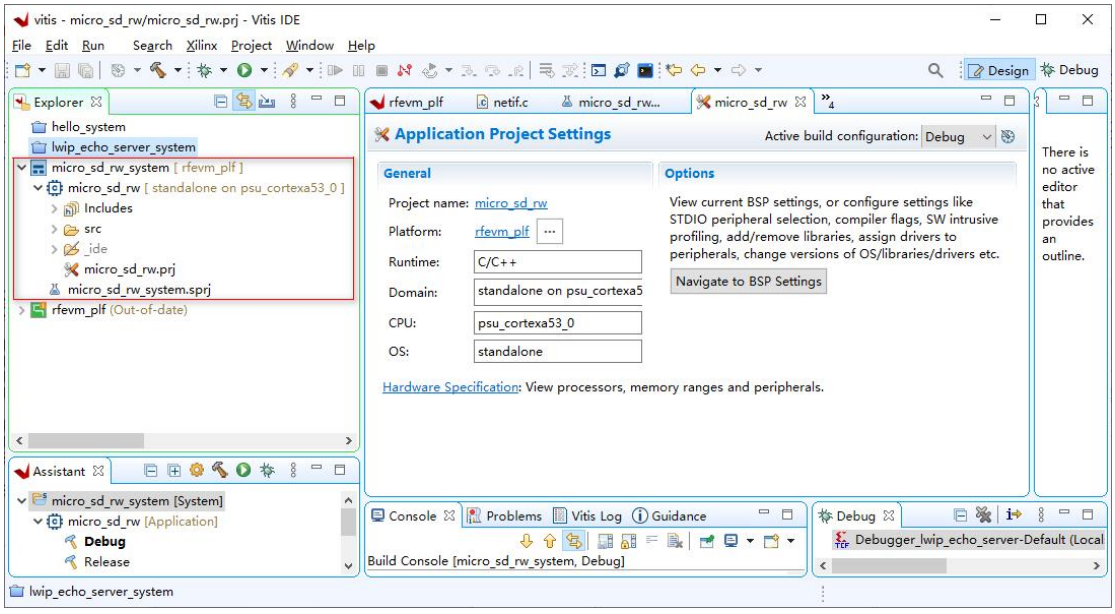
点击“Next”



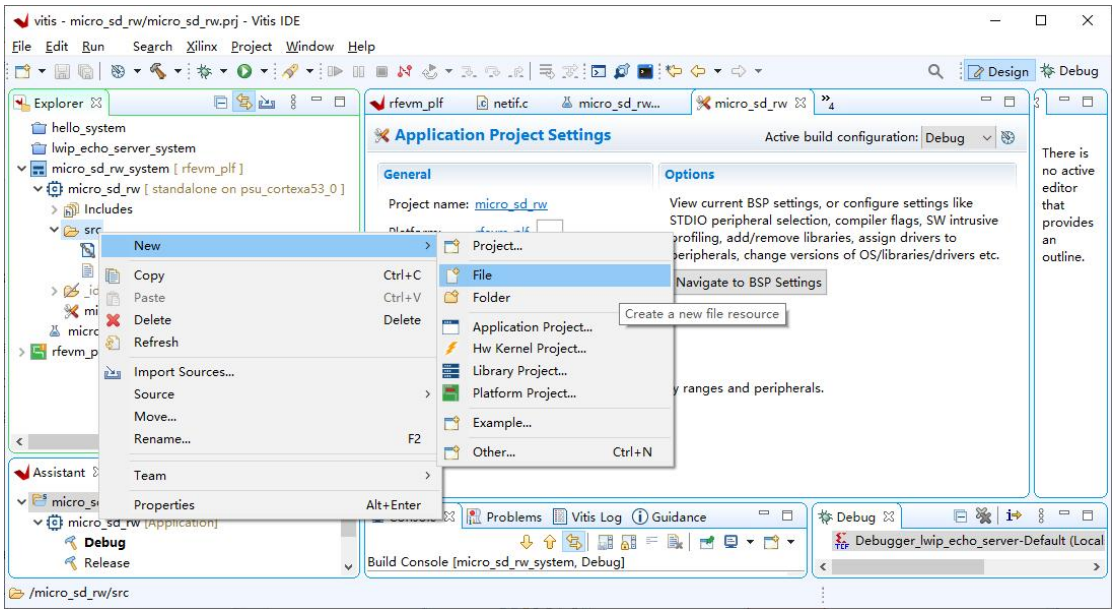
这里我们选择 Empty Application(C)作为模板，点击“Finish”

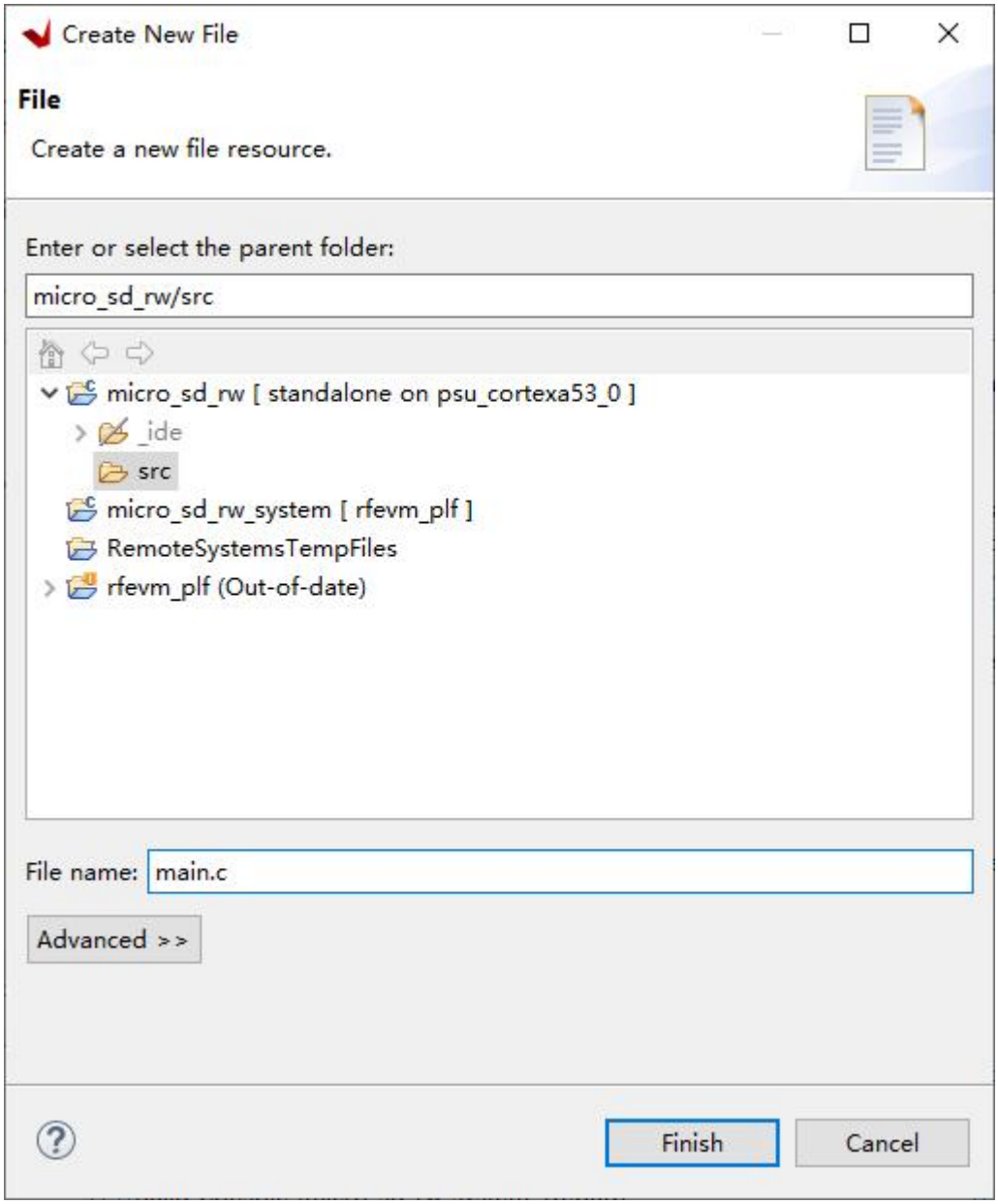


这里我们可以看到新建的应用工程。



接下来为应用工程新建一个源文件“main.c”


vitis - micro_sd_rw/micro_sd_rw.prj - Vitis IDE
File Edit Run Search Xilinx Project Window Help
Design Debug
Explorer
hello_system
lwip_echo_server_system
micro_sd_rw_system [rfevm_plf]
micro_sd_rw [standalone on psu_cortexa53_0]
Includes
src
rfevm_plf
netif.c
micro_sd_rw...
micro_sd_rw
Application Project Settings
Active build configuration: Debug
General
Project name: micro_sd_rw
Options
View current BSP settings, or configure settings like STDIO peripheral selection, compiler flags, SW intrusive profiling, add/remove libraries, assign drivers to peripherals, change versions of OS/libraries/drivers etc.
Navigate to BSP Settings
There is no active editor that provides an outline.
New
Copy Ctrl+C
Paste Ctrl+V
Delete Delete
Refresh
Import Sources...
Source
Move...
Rename... F2
Team
Properties Alt+Enter
Project...
File
Folder
Application Project...
Hw Kernel Project...
Library Project...
Platform Project...
Example...
Other... Ctrl+N
Create a new file resource
Assistant
Debug
Release
Problems
Vitis Log
Guidance
Build Console [micro_sd_rw_system, Debug]
Debug
Debugger_lwip_echo_server-Default (Local
/micro_sd_rw/src



我们在新建的 main.c 文件中输入下列代码：

```
#include "xparameters.h"
#include "xil_printf.h"
#include "ff.h"
#include "xstatus.h"

#define FILE_NAME "WE5G.txt"                    //定义文件名

const char src_str[30] = "www.v3t.com.cn"; //定义文本内容


//main 函数
int main()
{
```

```
    int status,len;
    char dest_str[30] = "";
    static FATFS fatfs;

    FIL fil;            //文件对象
    UINT bw;            //f_write 函数返回已写入的字节数
    UINT br;            //f_read 函数返回已读出的字节数

//注册一个工作区(挂载分区文件系统)
//在使用任何其它文件函数之前，必须使用 f_mount 函数为每个使用卷注册一个工作区
status = f_mount(&fatfs,"0:/", 1);   //挂载 SD 卡
if (status != FR_OK) {
    xil_printf("\r\nUnable to mount FATfs\r\n");
    return -1;
}
    xil_printf("Success to open SD card!\r\n");

    len = strlen(src_str);          //计算字符串长度

    //打开一个文件,如果不存在，则创建一个文件
status=f_open(&fil,"0:/WE5G.txt",FA_OPEN_ALWAYS | FA_WRITE | FA_READ);
if (status != FR_OK) {
    xil_printf("Unable to open file!\r\n");
    return -1;
}
xil_printf("Success to open file.\r\n");
//移动打开的文件对象的文件读/写指针      0:指向文件开头
f_lseek(&fil, 0);
//向文件中写入数据
f_write(&fil,(void*) src_str,len,&bw);
xil_printf("write file return:%d.\r\n",bw);

    //SD 卡读数据
    //sd_read_data(FILE_NAME,(u32)dest_str,len);
f_lseek(&fil, 0);
f_read(&fil,(void*)dest_str,len,&br);
f_close(&fil);
xil_printf("read file return:%d.\r\n",bw);
    //比较写入的字符串和读出的字符串是否相等
    if (strcmp(src_str, dest_str) == 0)
    xil_printf("src_str is equal to dest_str,SD card test success!\r\n");
    else
    xil_printf("src_str is not equal to dest_str,SD card test failed!\r\n");
```

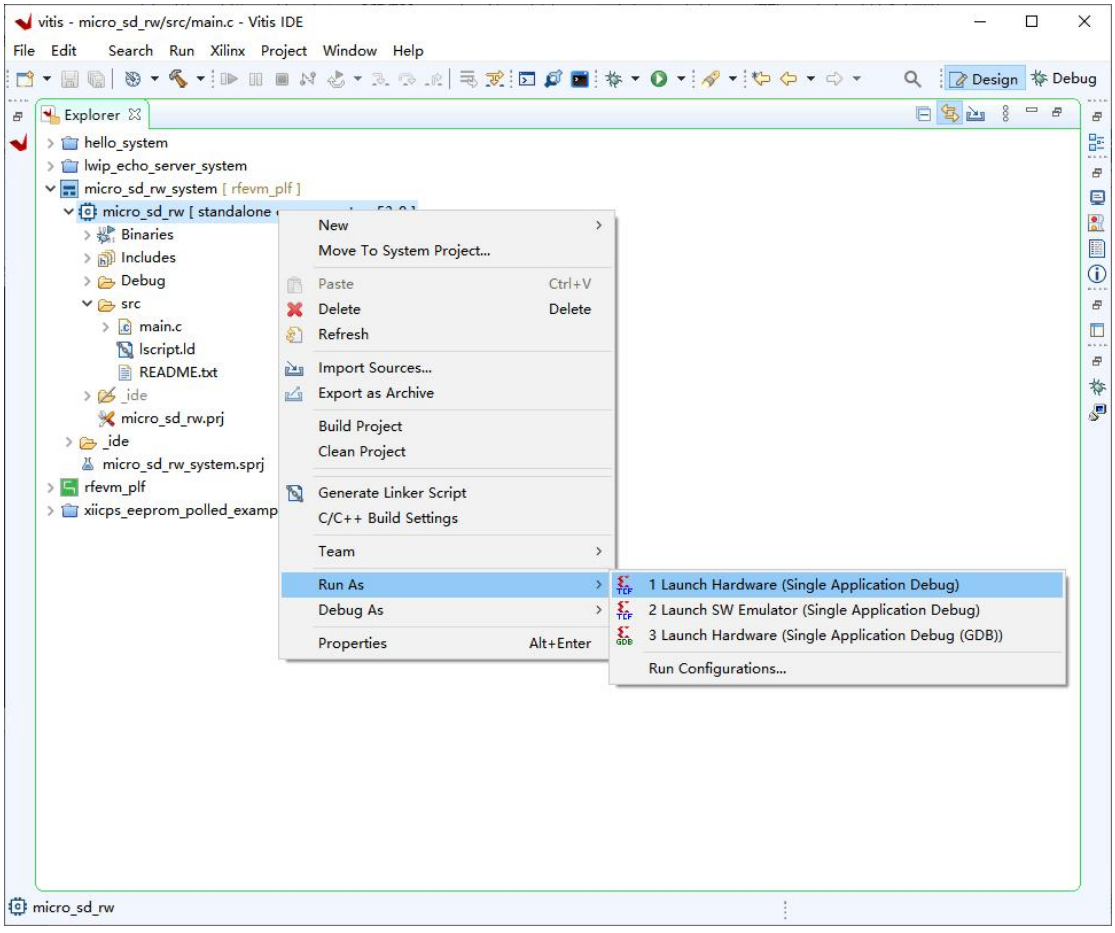
```
        return 0;
}
```

在 main 函数中，实现的功能是向 TF 卡中读写 TXT 文本，并比较写入的数据和读出的数据是否一致。程序首先调用 f_mount()函数挂载 TF 卡，接下来根据待写入字符串的长度，通过 f_write()函数和 f_read()函数对 TF 卡中的 TXT 文本进行写入和读出，然后比较写入和读出的值，并打印比较的结果。

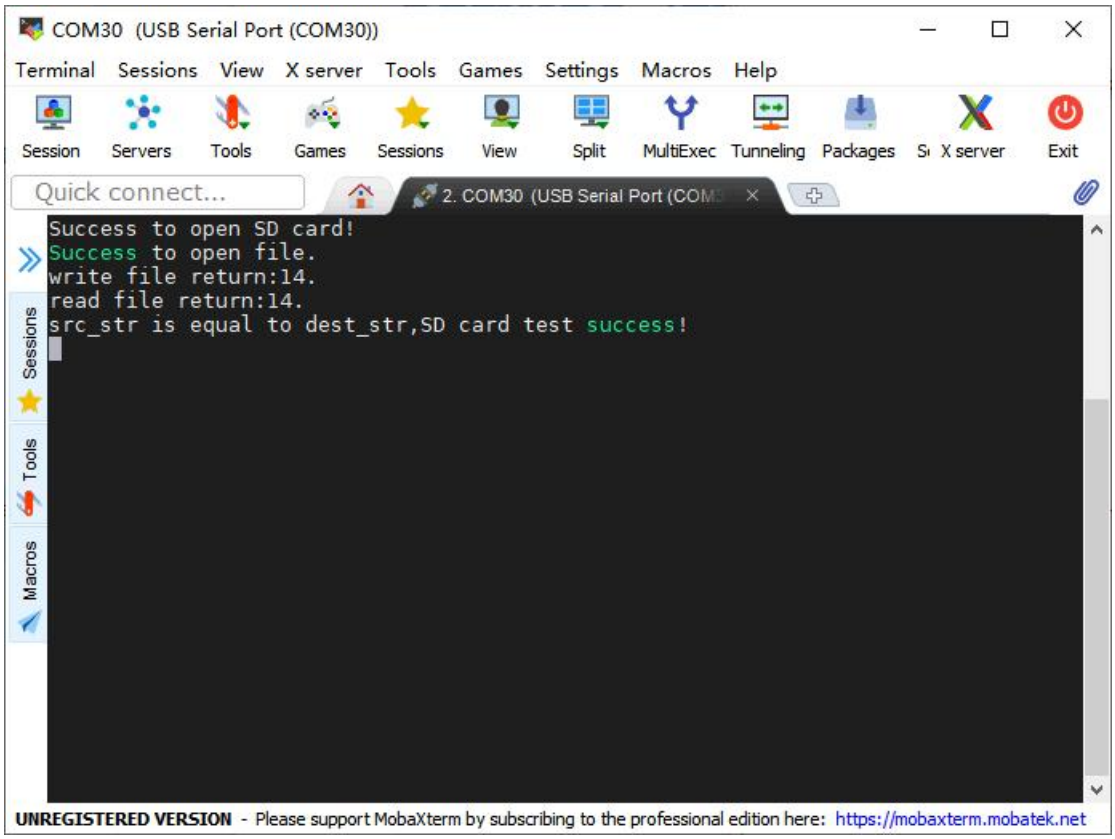
4.6.3 下载验证

首先我们将下载线（USB 转 Type-C 线缆）与开发板的 JTAG 接口连接，另外一端与电脑连接。接下来插入 TF 卡。最后打开开发板的电源开关。

右键应用工程，选择 Run As->Launch Hardware(Single Application Debug)



打开串口终端，下载完成后，可以查看串口输出：



由上图可知，显示写入的字符和读出的字符一致，说明 TF 卡读写测试成功。

接下来从开发板的卡槽中取出 TF 卡，通过连接读卡器来插入电脑的 USB 接口，此时可以看到 TF 卡中的文件内容如下图所示。

打开“WE5G.txt”文本，可以看到文本的内容，如下图所示：

4.6.4 参考文档

http://elm-chan.org/fsw/ff/00index_e.html

4.7 基于 PS 的 USB slave 功能测试

ZYNQ 的 USB 接口可以同时当作 USB HOST 和 USB DEVICE(SLAVE)，USB 功能大多数情况下都是在 linux 下使用的，这里我们在裸机下去测试 USB 功能。

我们使用官方提供的演示 demo，去把 usb 设备模拟成 U 盘的功能进行测试。

4.7.1 硬件设计

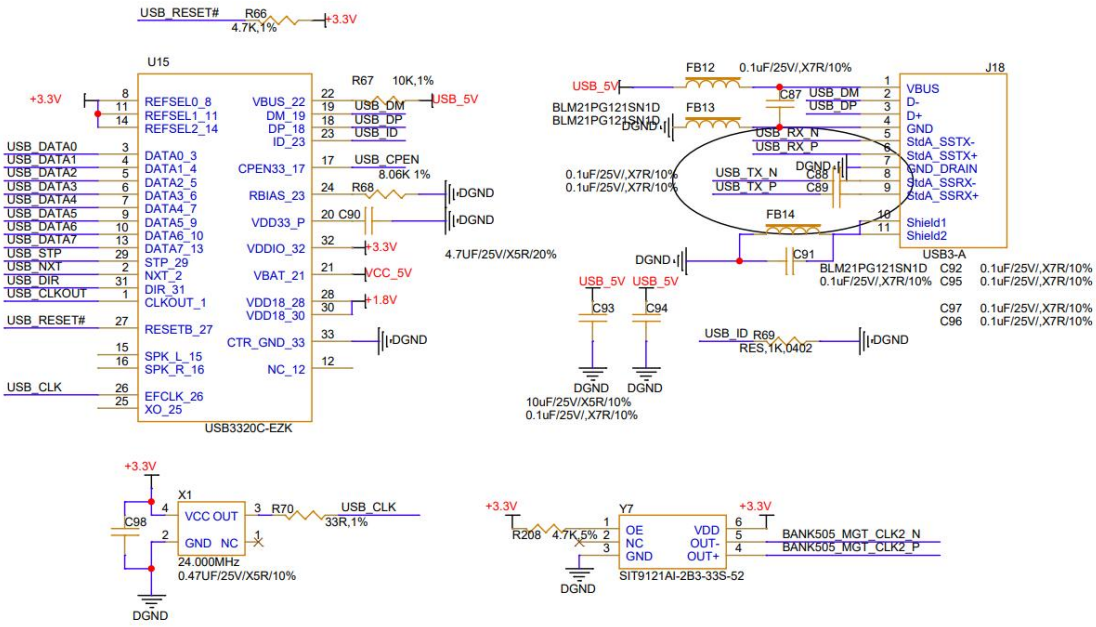
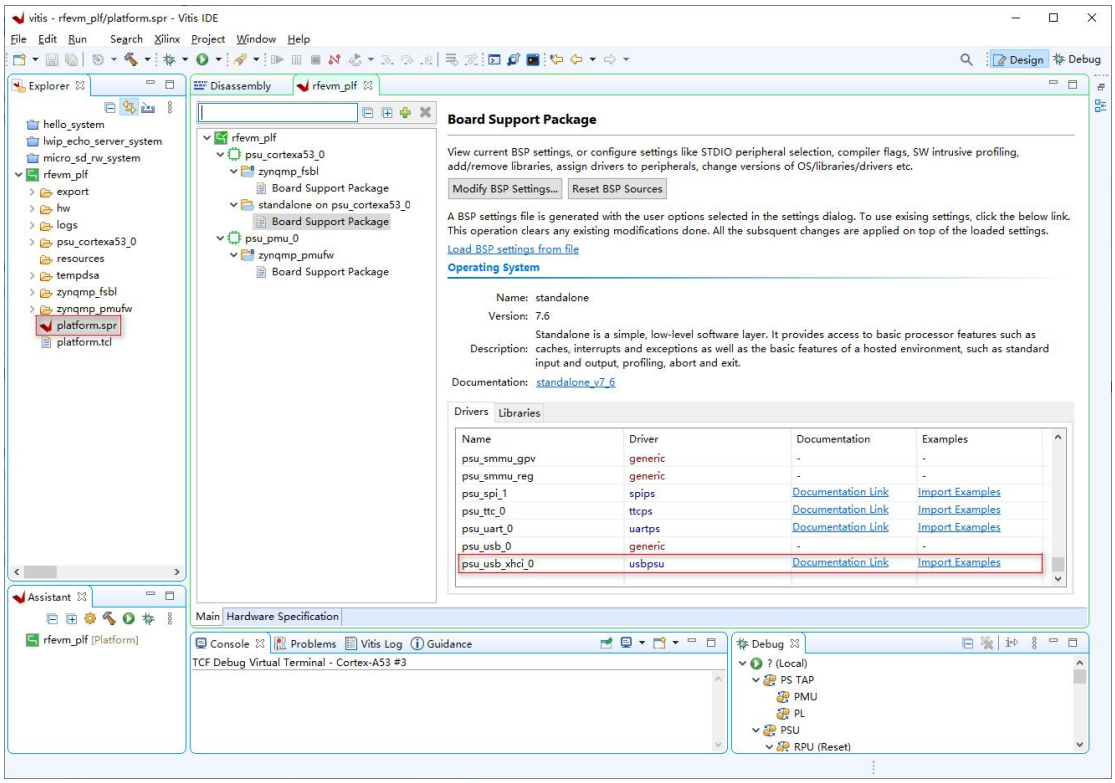


图 16 USB 接口原理图

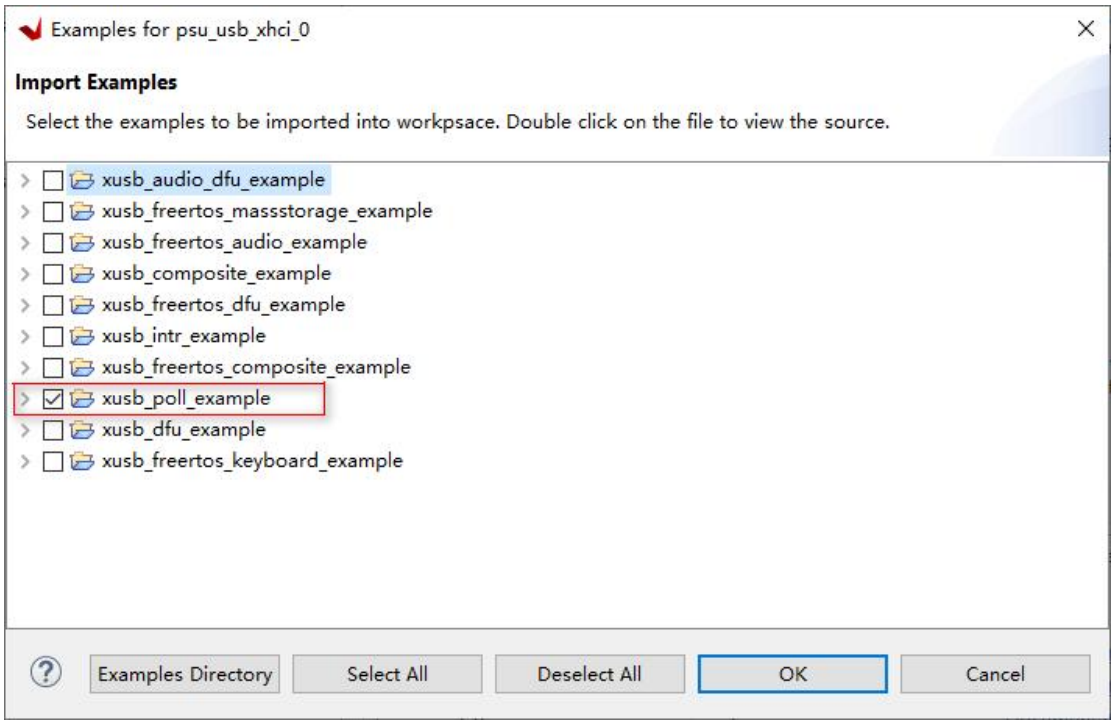
4.7.2 软件设计

1) 登录 Vitis，切换到之前的工作目录，然后双击 platform.spr，选中 Board Support

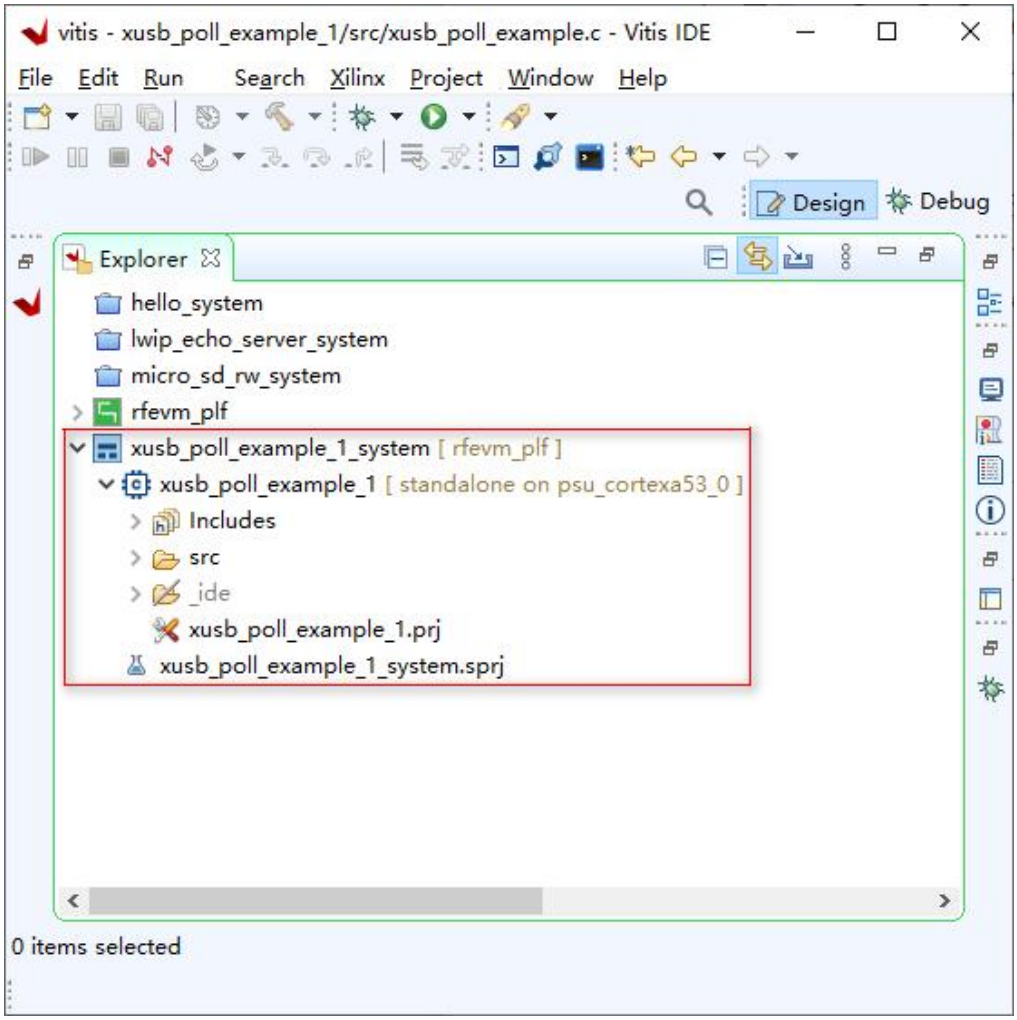
Package，可以看到 psu_usb_xhci_0 的文档链接和导入示例。



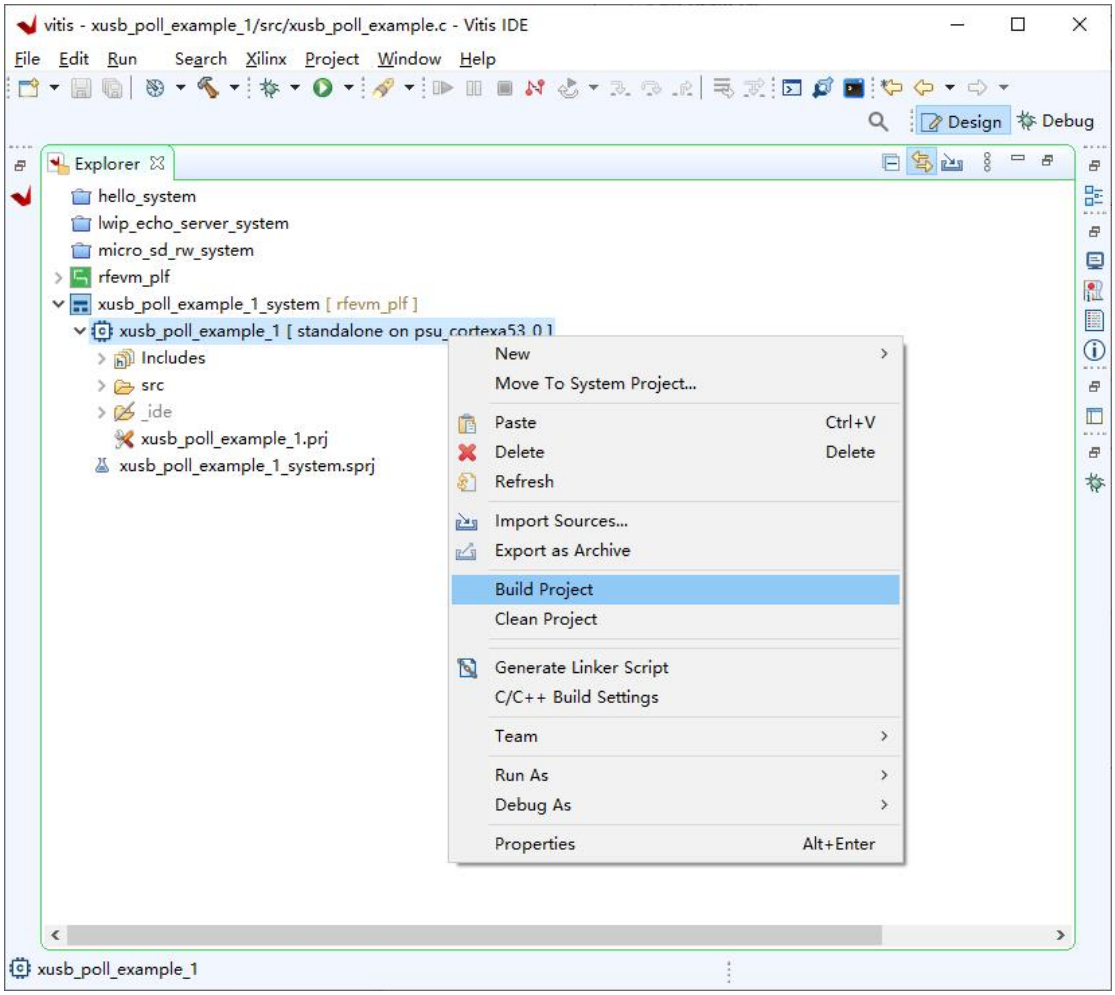
2) 点击 Import Example，然后选中 xusb_poll_example 示例，点击 OK



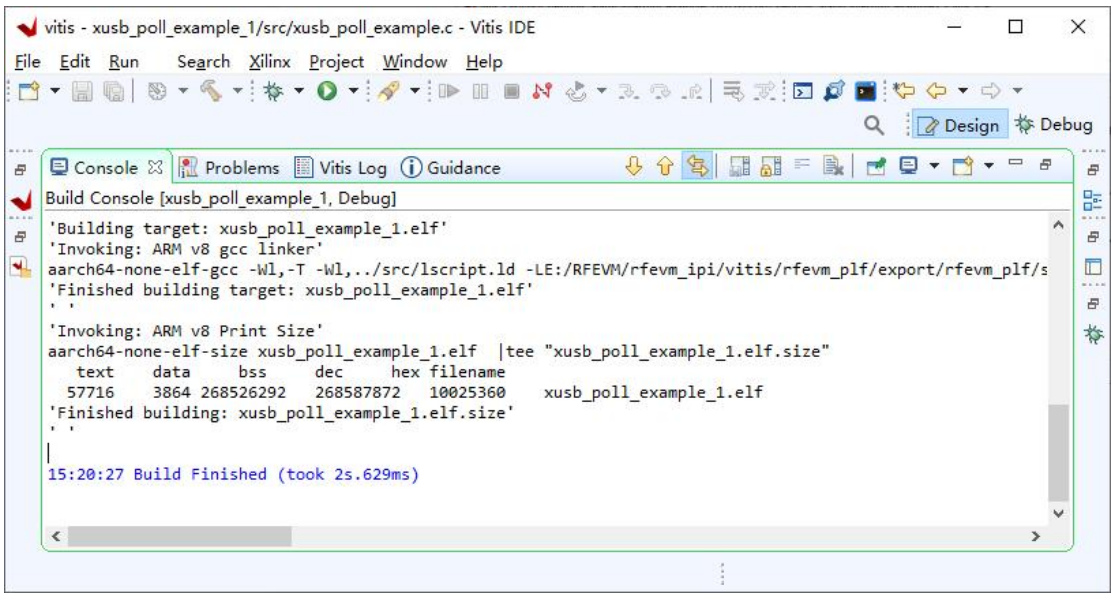
3) 可以看到导入的示例应用工程



4) 选中应用工程，右键点击 Build Project



编译信息可以在 Console 窗口中看到

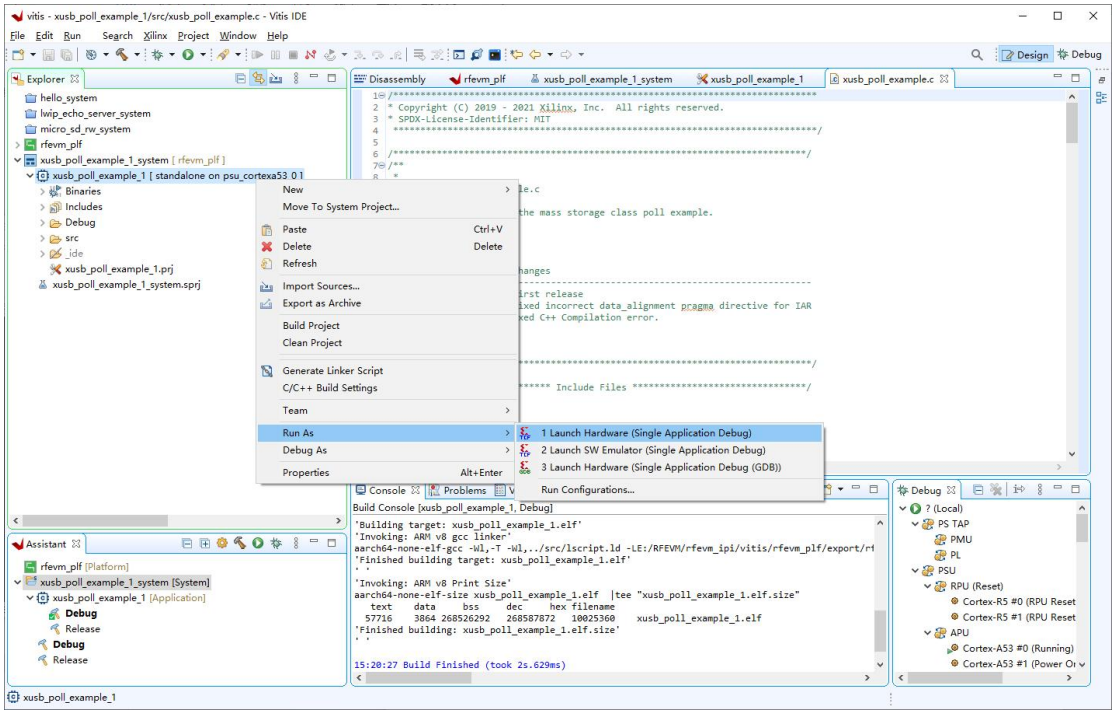


4.7.3 下载验证

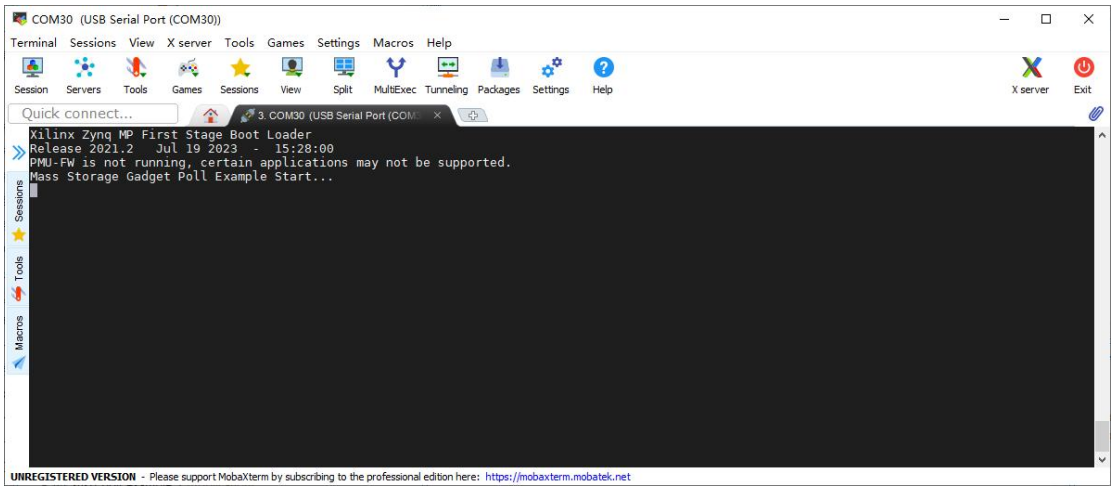
这里我们需要使用 USB 公对公的数据线一端连到 RFEVM 的 USB 接口上，

另外一端连到 PC 端。然后给开发板上电。

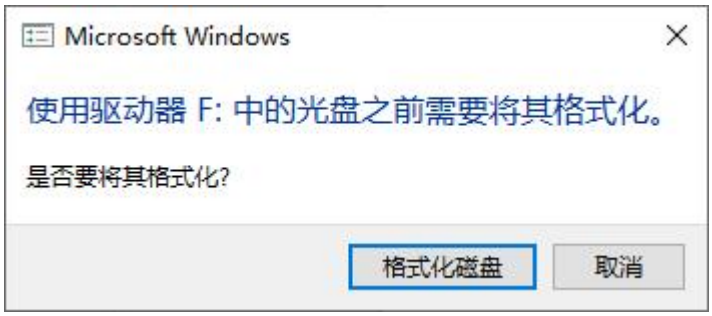
右键应用工程，选择 Run As->Launch Hardware(Single Application Debug)



打开串口终端，下载完成后，可以查看串口输出：

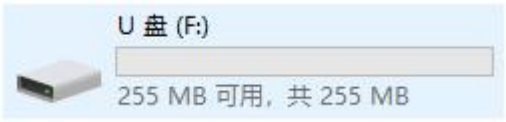


PC 侧会弹出如下窗口，点击“格式化磁盘”即可



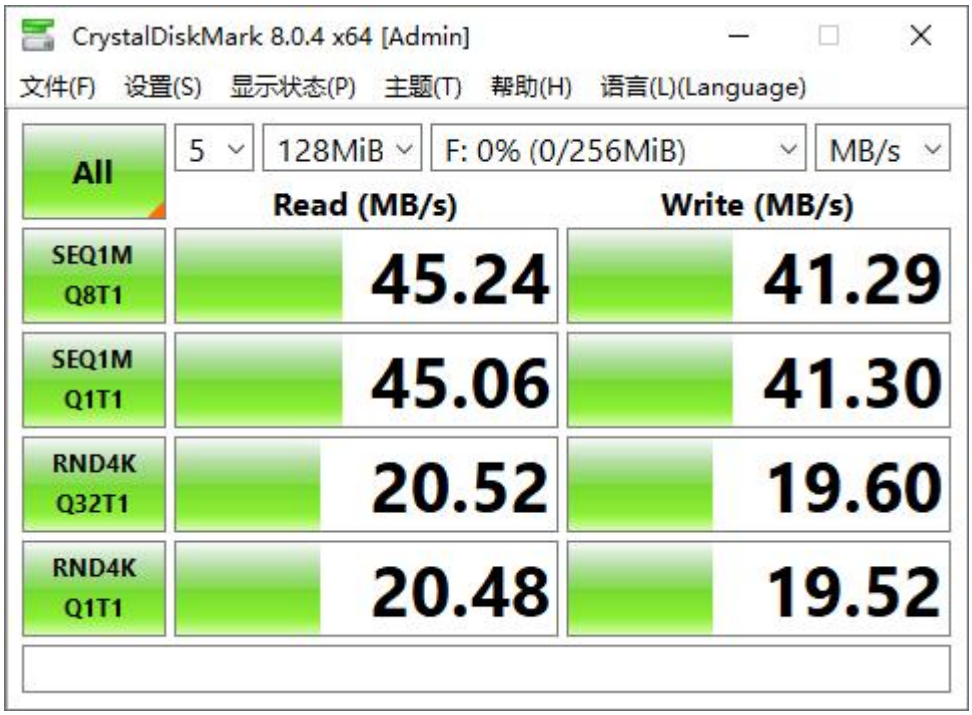
在我的电脑里面，可以看到由 DDR RAM 虚拟出来的 U 盘，可以像正常 U 盘一

样复制粘贴文件，注意这个 U 盘断电数据会丢失请不要放重要文件。



容量大小可以在程序里面去配置，请不要超过板子上的 DDR 容量大小。

我们可以使用 Xilinx WiKi 推荐的 CrystalDiskMark 工具对 U 盘进行性能测速。

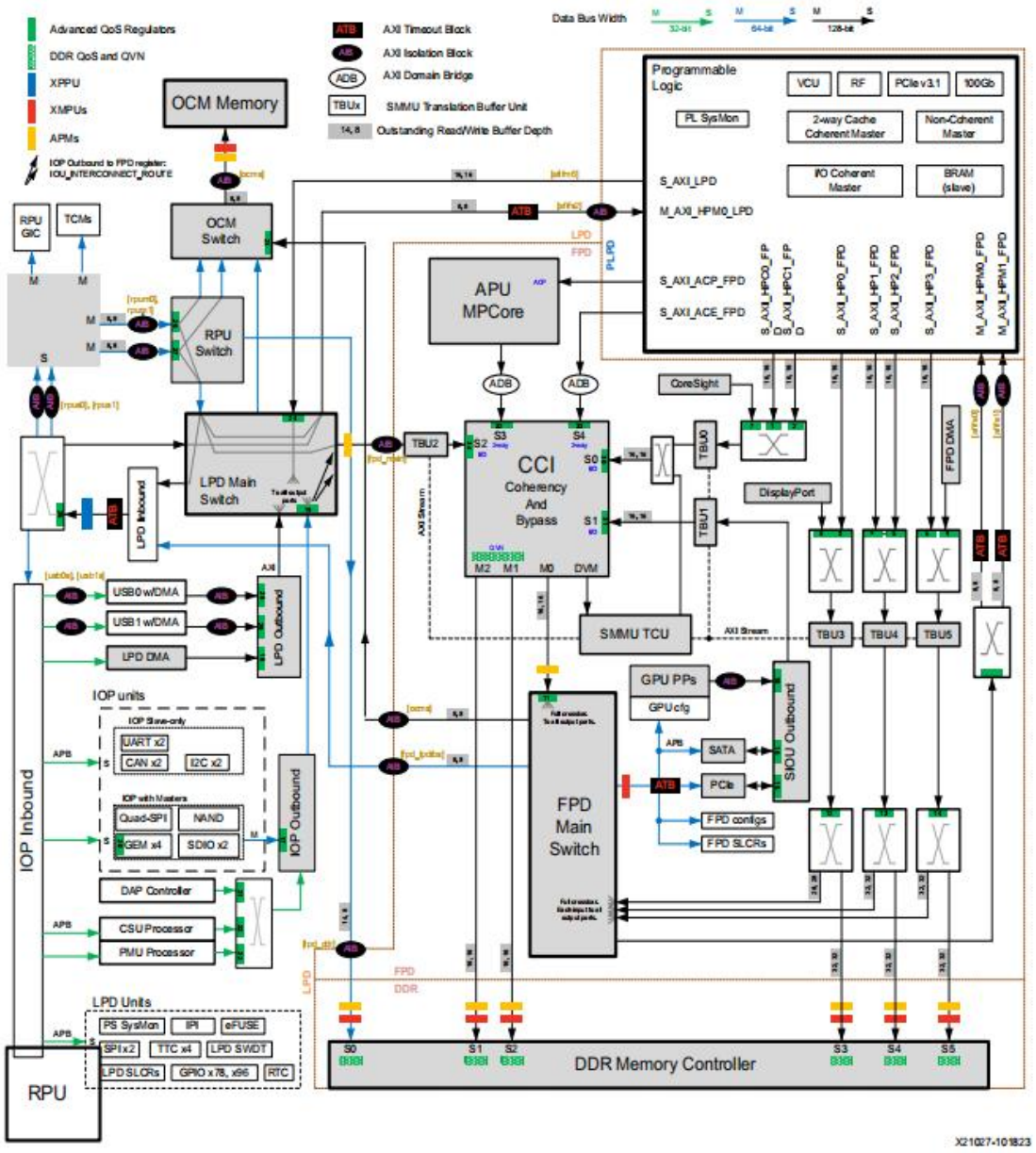


4.7.4 参考文档

https://xilinx-wiki.atlassian.net/wiki/spaces/A/pages/18842468/ZynqMp+USB+Stadalone+Driver

5 PS 与 PL 互联

本章主要介绍 PS 与 PL 之间的互联，在开始之前，首先简要介绍 PS/PL 之间的数据通路，具体请参考《ug1085 Zynq UltraScale+ Device Technical Reference Manual》。



PS/PL 之间的数据通路主要是通用的 AXI Memory 接口，其中 PS 作为主设备的接口有 128-bit 位宽的 HPM0/HPM1，64-bit 位宽的 LPD_PL；PL 作为主设备的接口有 ACP/ACE/HPC0/HPC1/HP0/HP1/HP2/HP3。所有 AXI Memory 接口

的位宽最高都可以支持到 128-bit。

Interface Name	Abbreviation	FIFO Interface	Master
S_AXI_HP{0:3}_FPD	HP{0:3}	AFI_{2:5}	PL
S_AXI_LPD	PL_LPD	AFI_6	PL
S_AXI_ACE_FPD	ACE	None	PL
S_AXI_ACP_FPD	ACP	None	PL
S_AXI_HPC{0,1}_FPD	HPC{0,1}	AFI_{0:1}	PL
M_AXI_HPM{0,1}_FPD	HPM{0,1}	None	PS
M_AXI_HPM0_LPD	LPD_PL	None	PS

摘自《ug1085》

5.1 PS 端 EMIO 的使用

前面介绍了 PL 端 LED 呼吸灯的实验，这里我们使用 PS 端去控制 PL 侧 LED 灯的亮灭。

PL 侧的扩展 IO 也可以用这种方法去控制。

ZYNQ 的 GPIO 分为 MIO（multiplexed I/O）和 EMIO（extended multiplexed I/O）两种。

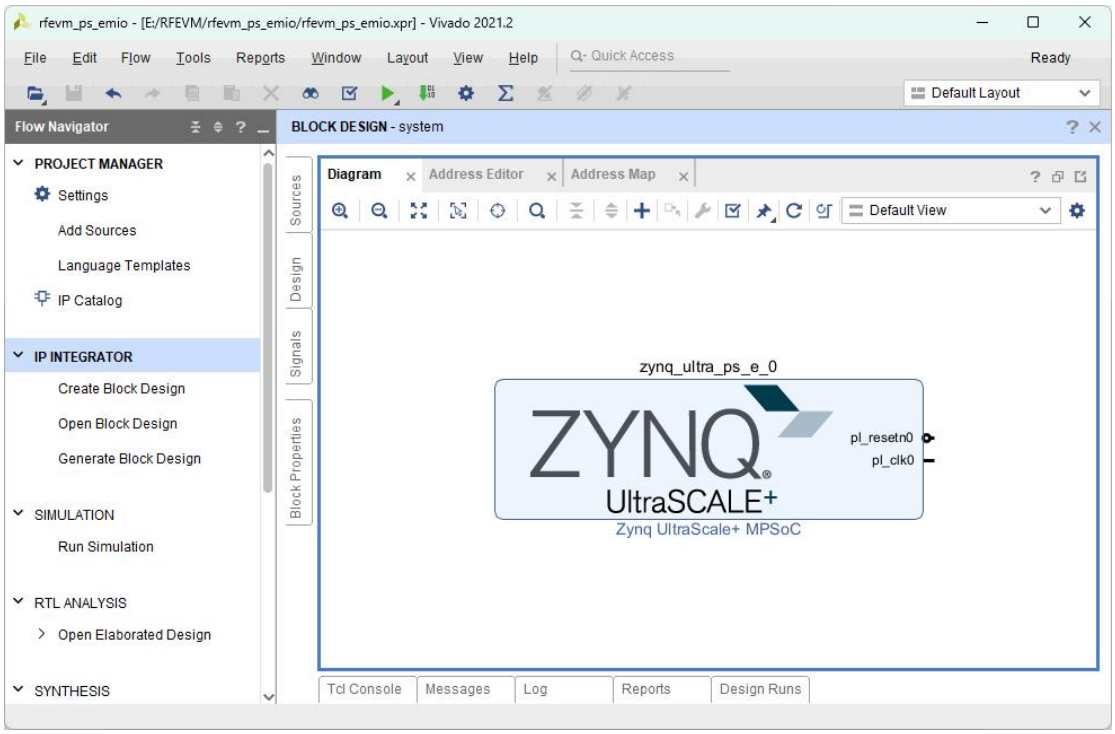
MIO 为 PS 端多功能 IO 接口，IO 与 PS 端连接，每个 IO 可根据需求配置为 GPIO、SPI、UART、TIMER、Ethernet、USB 等多种不同功能。由于 MIO 对 PL 端不可见，因此 PL 端无法使用 MIO 引脚。

EMIO 为 MIO 的扩展 IO，IO 与 PL 端连接。由于 EMIO 对 PS 端、PL 端均可见，因此两者均可使用 EMIO 引脚。一般情况下，如 PS 端 MIO 资源无法满足需求时，PS 端可通过 EMIO 方式进行接口扩展。EMIO 的使用与 MIO 相似，区别在于 EMIO 需使用 Vivado 约束文件对管脚进行分配。

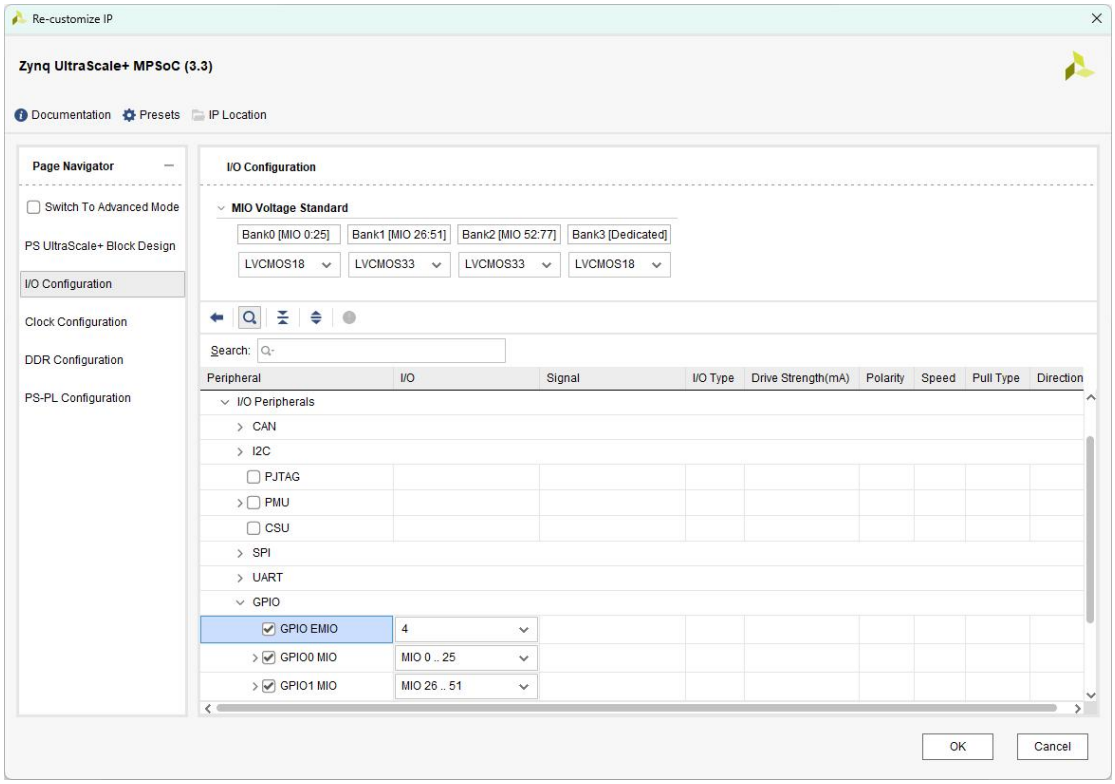
MPSoC 为 PL 提供了 96 个 GPIO，通过 EMIO 管脚连接到 PL。这里我们使用 EMIO 去连接 PL 端的 LED 灯，在 Vitis 里面去控制它的状态。

5.1.1 硬件工程搭建

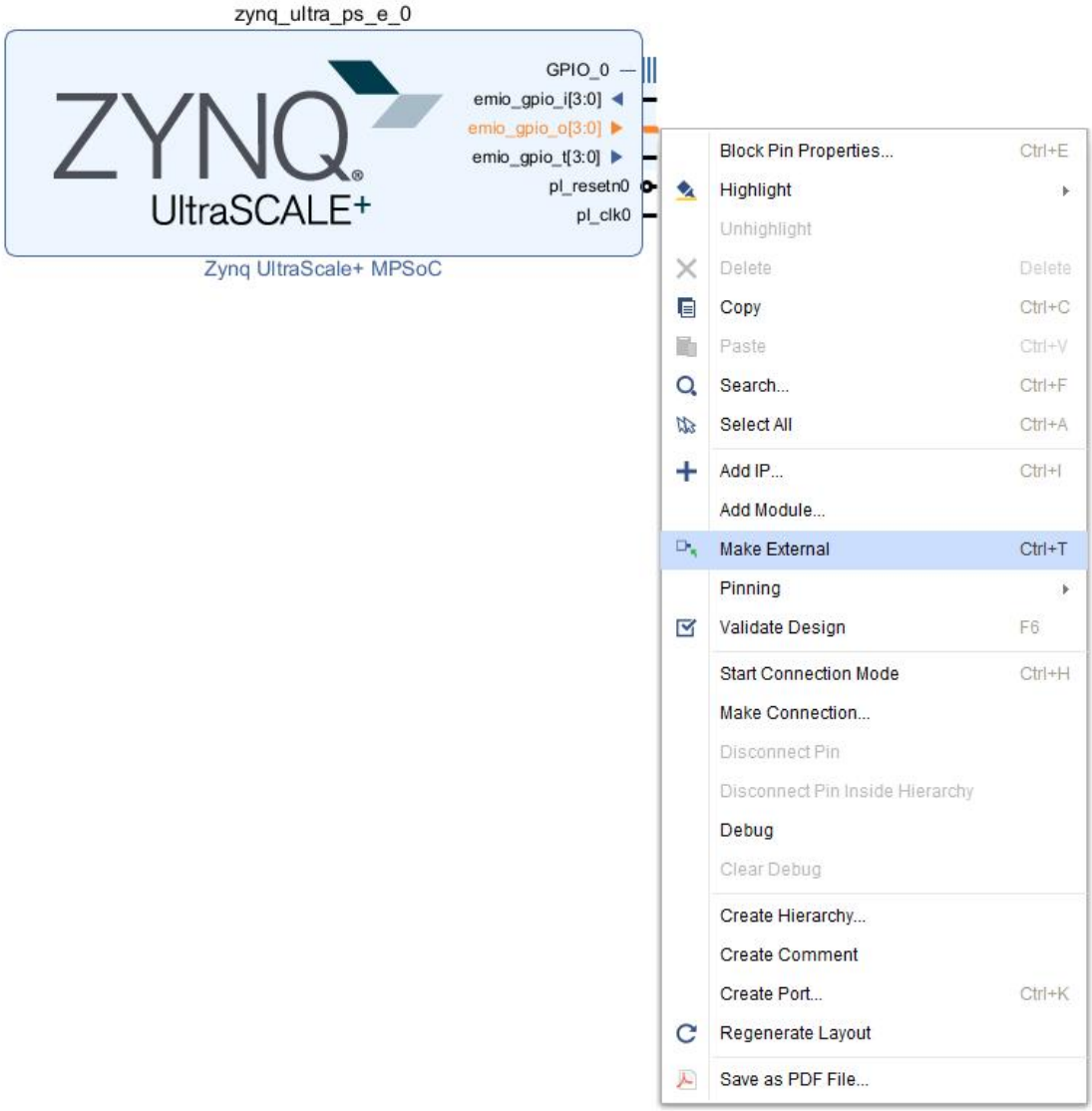
1) 搭建硬件工程，按照上章 IPI 的工程搭建步骤，添加 MPSOC IP 核

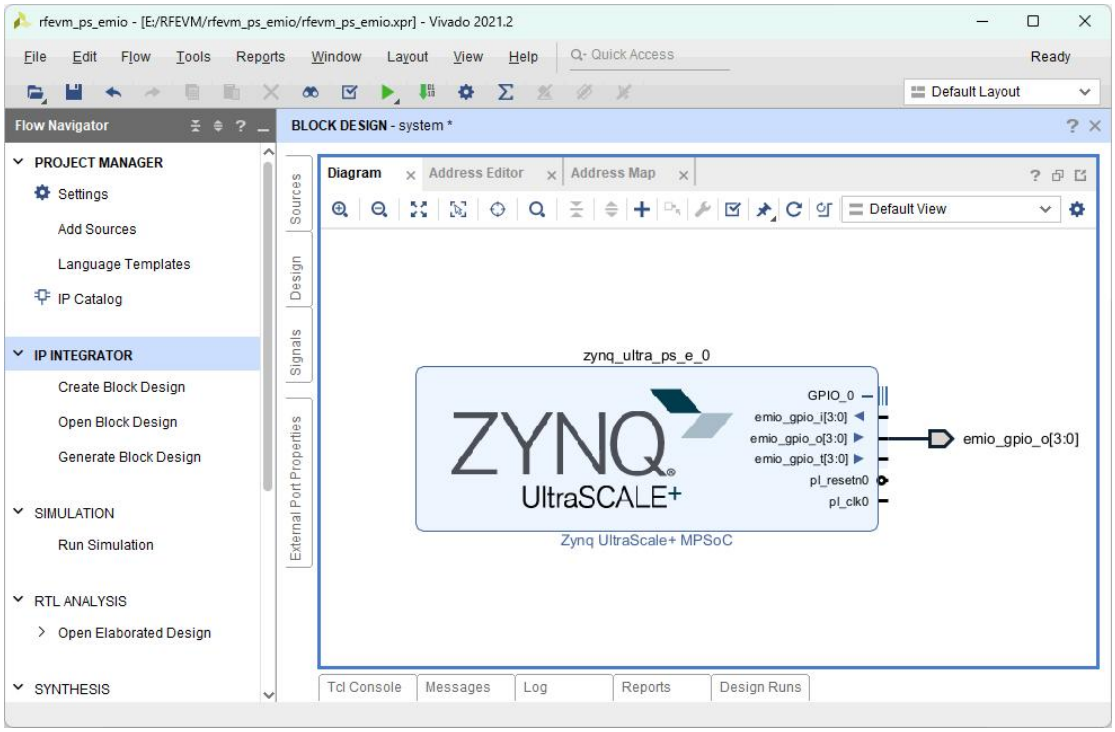


2) 双击 IP，使能 emio 接口，IO 数量选择 4（对应底板上 4 个 led 灯）；

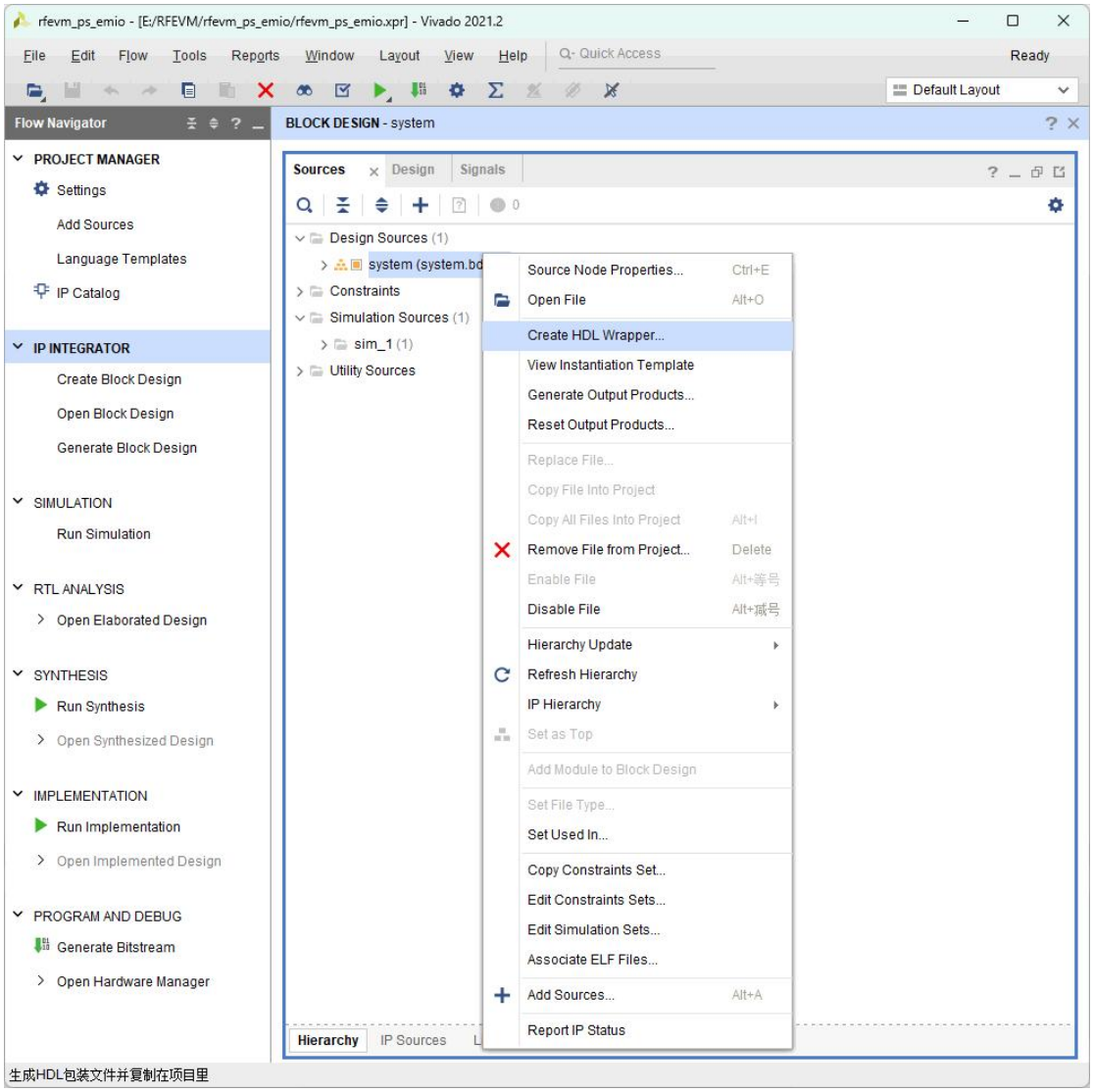


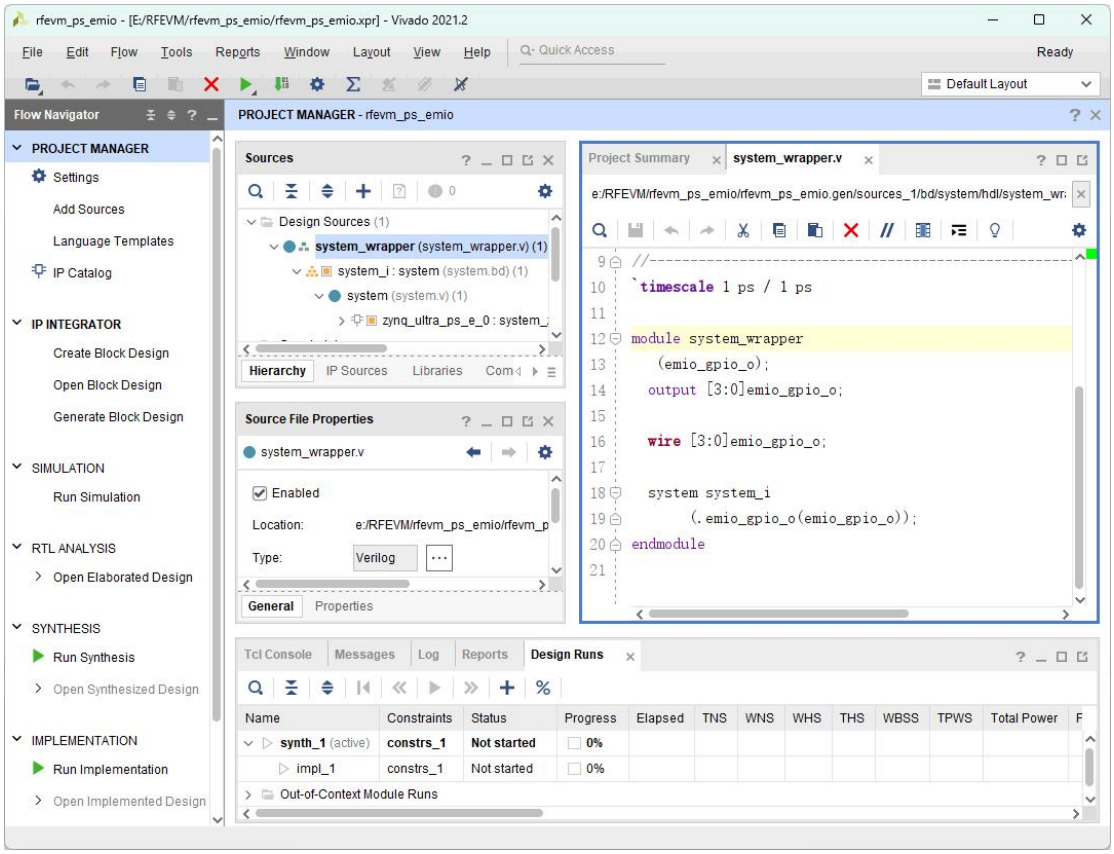
3) 这里我们把 emio 的输出 emio_gpio_o 引出来 Make External。


zynq_ultra_ps_e_0
GPIO_0
emio_gpio_i[3:0]
emio_gpio_o[3:0]
emio_gpio_t[3:0]
pl_resetn0
pl_clk0
ZYNQ
UltraSCALE+
Zynq UltraScale+ MPSoC
Block Pin Properties... Ctrl+E
Highlight
Unhighlight
Delete Delete
Copy Ctrl+C
Paste Ctrl+V
Search... Ctrl+F
Select All Ctrl+A
Add IP... Ctrl+I
Add Module...
Make External Ctrl+T
Pinning
Validate Design F6
Start Connection Mode Ctrl+H
Make Connection...
Disconnect Pin
Disconnect Pin Inside Hierarchy
Debug
Clear Debug
Create Hierarchy...
Create Comment
Create Port... Ctrl+K
Regenerate Layout
Save as PDF File...

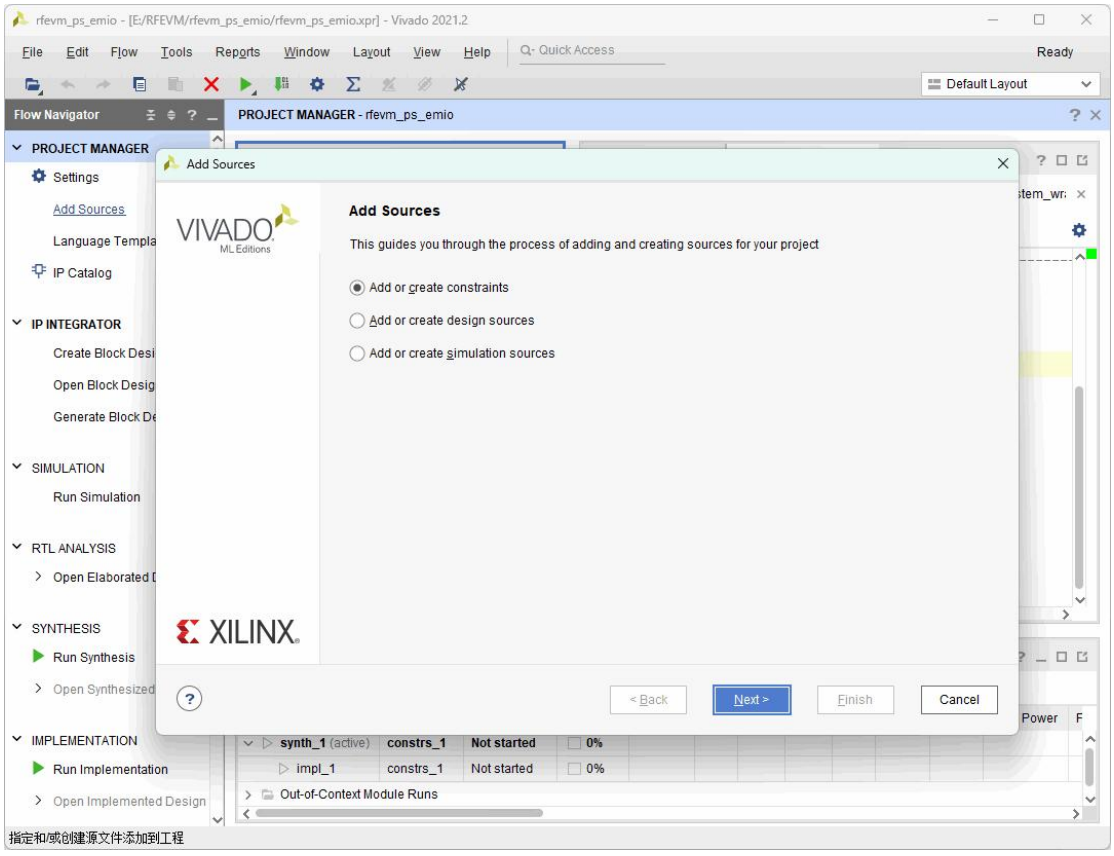

rfevm_ps_emio - [E:/RFEVM/rfevm_ps_emio/rfevm_ps_emio.xpr] - Vivado 2021.2
File Edit Flow Tools Reports Window Layout View Help
Quick Access
Ready
Default Layout
Flow Navigator
BLOCK DESIGN - system *
PROJECT MANAGER
Settings
Add Sources
Language Templates
IP Catalog
IP INTEGRATOR
Create Block Design
Open Block Design
Generate Block Design
SIMULATION
Run Simulation
RTL ANALYSIS
Open Elaborated Design
SYNTHESIS
Diagram
Address Editor
Address Map
Default View
zynq_ultra_ps_e_0
GPIO_0
emio_gpio_i[3:0]
emio_gpio_o[3:0]
emio_gpio_t[3:0]
pl_resetn0
pl_clk0
ZYNQ
UltraSCALE+
Zynq UltraScale+ MPSoC
emio_gpio_o[3:0]
Sources
Design
Signals
External Port Properties
Tcl Console
Messages
Log
Reports
Design Runs

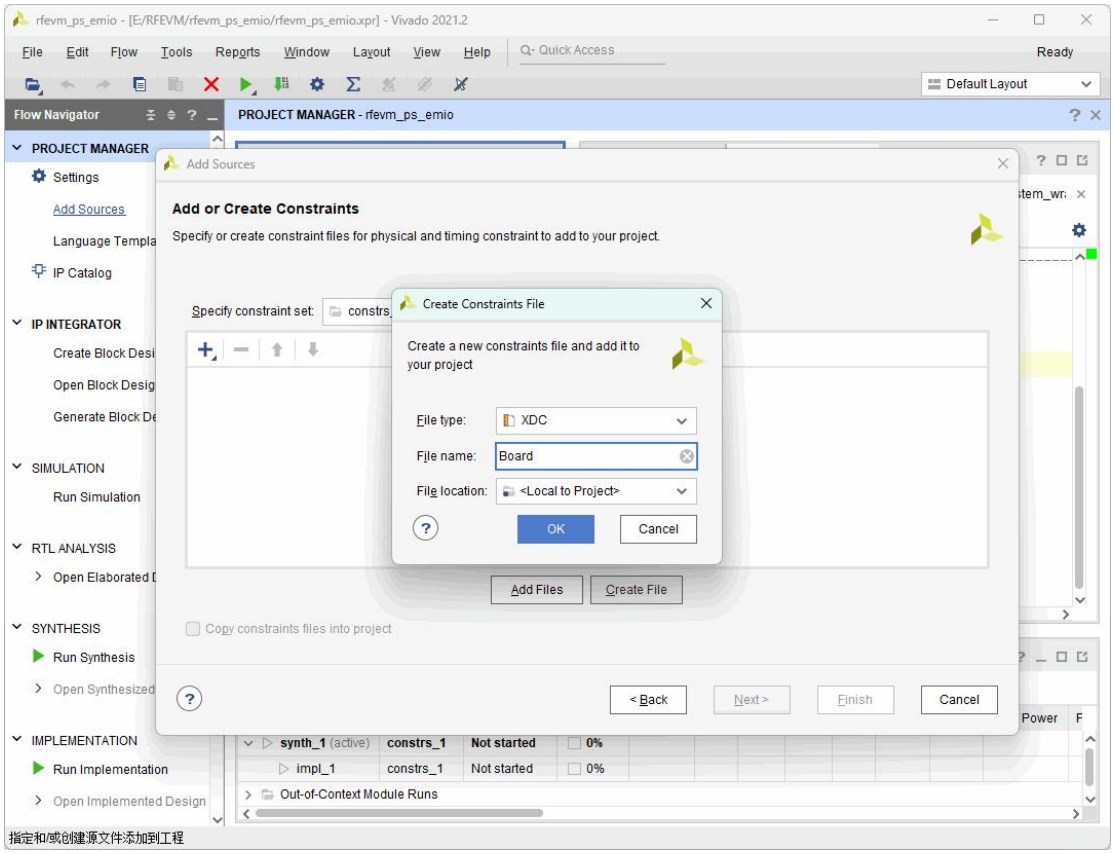
4) 创建 HDL Wrapper...

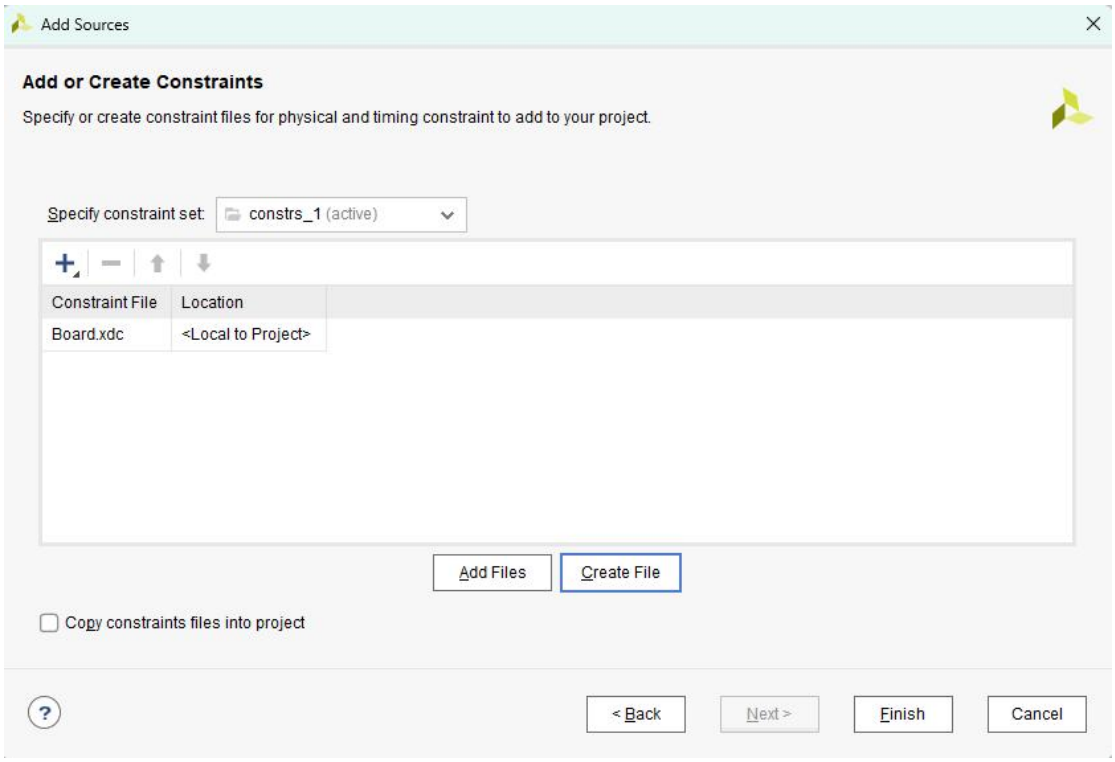



rfevm_ps_emio - [E:/RFEVM/rfevm_ps_emio/rfevm_ps_emio.xpr] - Vivado 2021.2
File Edit Flow Tools Reports Window Layout View Help
Quick Access
Ready
Default Layout
Flow Navigator
PROJECT MANAGER - rfevm_ps_emio
PROJECT MANAGER
Settings
Add Sources
Language Templates
IP Catalog
IP INTEGRATOR
Create Block Design
Open Block Design
Generate Block Design
SIMULATION
Run Simulation
RTL ANALYSIS
Open Elaborated Design
SYNTHESIS
Run Synthesis
Open Synthesized Design
IMPLEMENTATION
Run Implementation
Open Implemented Design
Sources
Design Sources (1)
system_wrapper (system_wrapper.v) (1)
system_i : system (system.bd) (1)
system (system.v) (1)
zynq_ultra_ps_e_0 : system_
Hierarchy IP Sources Libraries
Source File Properties
system_wrapper.v
Enabled
Location: e:/RFEVM/rfevm_ps_emio/rfevm_p
Type: Verilog
General Properties
Project Summary
system_wrapper.v
e:/RFEVM/rfevm_ps_emio/rfevm_ps_emio.gen/sources_1/bd/system/hdl/system_wr
//----------
`timescale 1 ps / 1 ps
module system_wrapper
(emio_gpio_o);
output [3:0]emio_gpio_o;
wire [3:0]emio_gpio_o;
system system_i
(.emio_gpio_o(emio_gpio_o));
endmodule
Tcl Console Messages Log Reports Design Runs
Name Constraints Status Progress Elapsed TNS WNS WHS THS WBSS TPWS Total Power
synth_1 (active) constrs_1 Not started 0%
impl_1 constrs_1 Not started 0%
Out-of-Context Module Runs

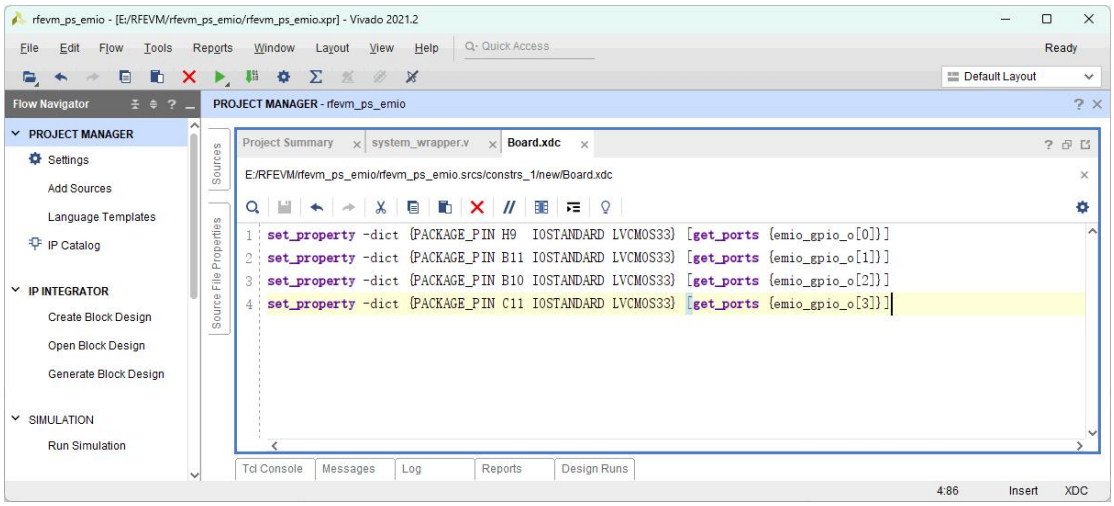
5) 创建约束文件


rfevm_ps_emio - [E:/RFEVM/rfevm_ps_emio/rfevm_ps_emio.xpr] - Vivado 2021.2
File Edit Flow Tools Reports Window Layout View Help
Quick Access
Ready
Default Layout
Flow Navigator
PROJECT MANAGER - rfevm_ps_emio
PROJECT MANAGER
Settings
Add Sources
Language Templa
IP Catalog
IP INTEGRATOR
Create Block Desi
Open Block Desig
Generate Block De
SIMULATION
Run Simulation
RTL ANALYSIS
Open Elaborated
SYNTHESIS
Run Synthesis
Open Synthesized
IMPLEMENTATION
Run Implementation
Open Implemented Design
Add Sources
VIVADO ML Editions
Add Sources
This guides you through the process of adding and creating sources for your project
Add or create constraints
Add or create design sources
Add or create simulation sources
XILINX
< Back
Next >
Finish
Cancel
synth_1 (active) constrs_1 Not started 0%
impl_1 constrs_1 Not started 0%
Out-of-Context Module Runs
指定和/或创建源文件添加到工程



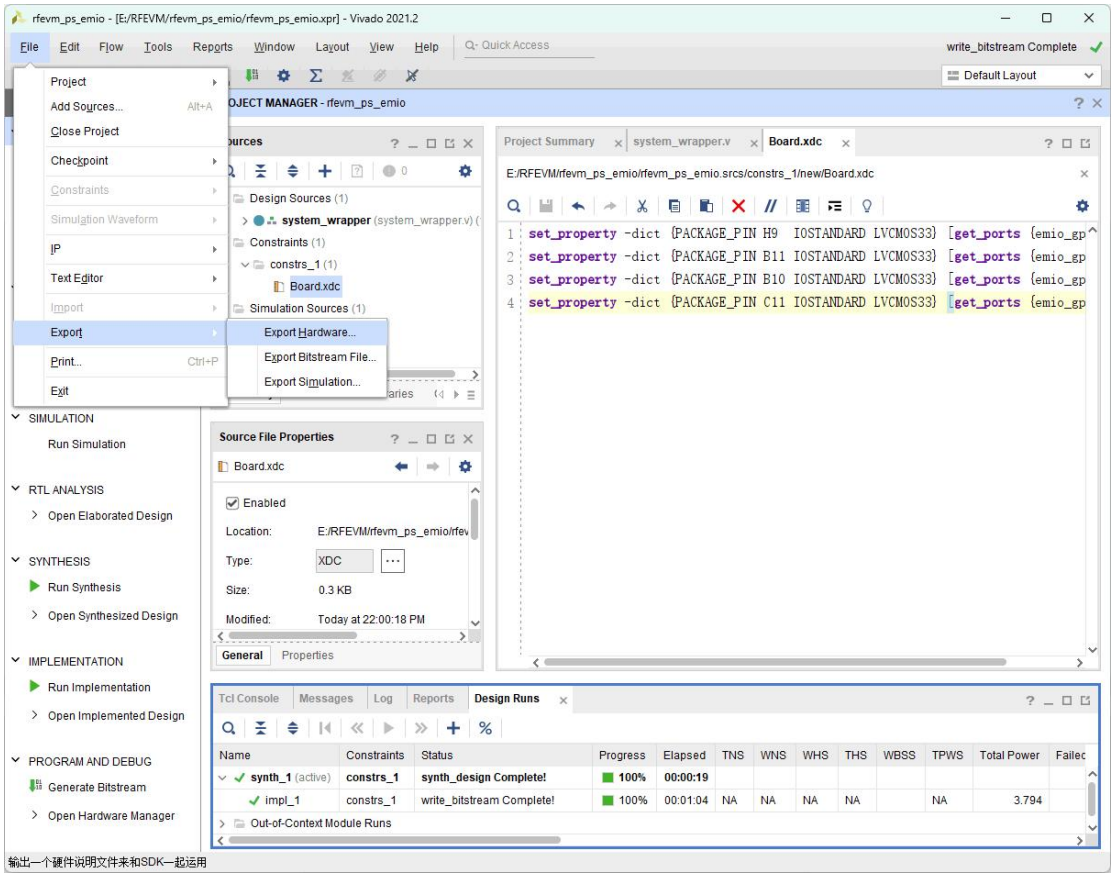


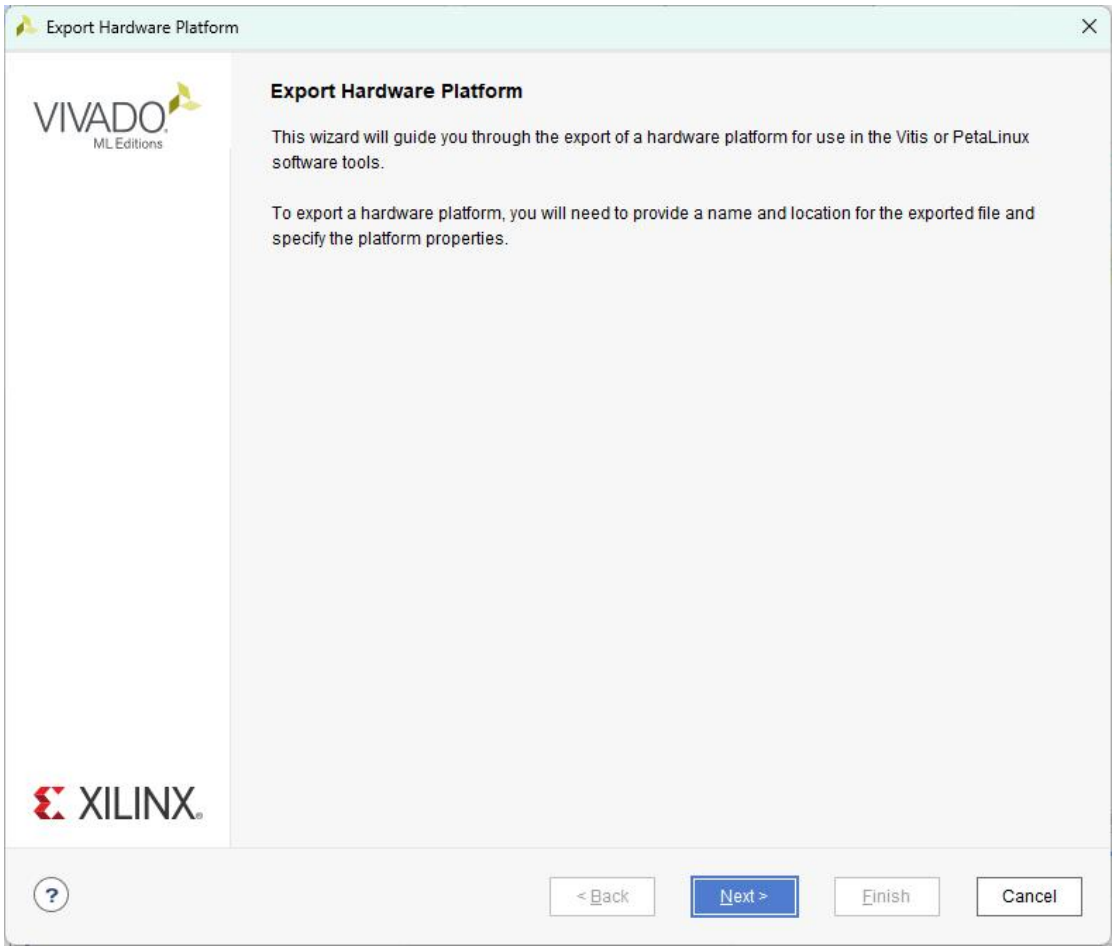
6) 在 xdc 里面添加 led 灯的管脚约束



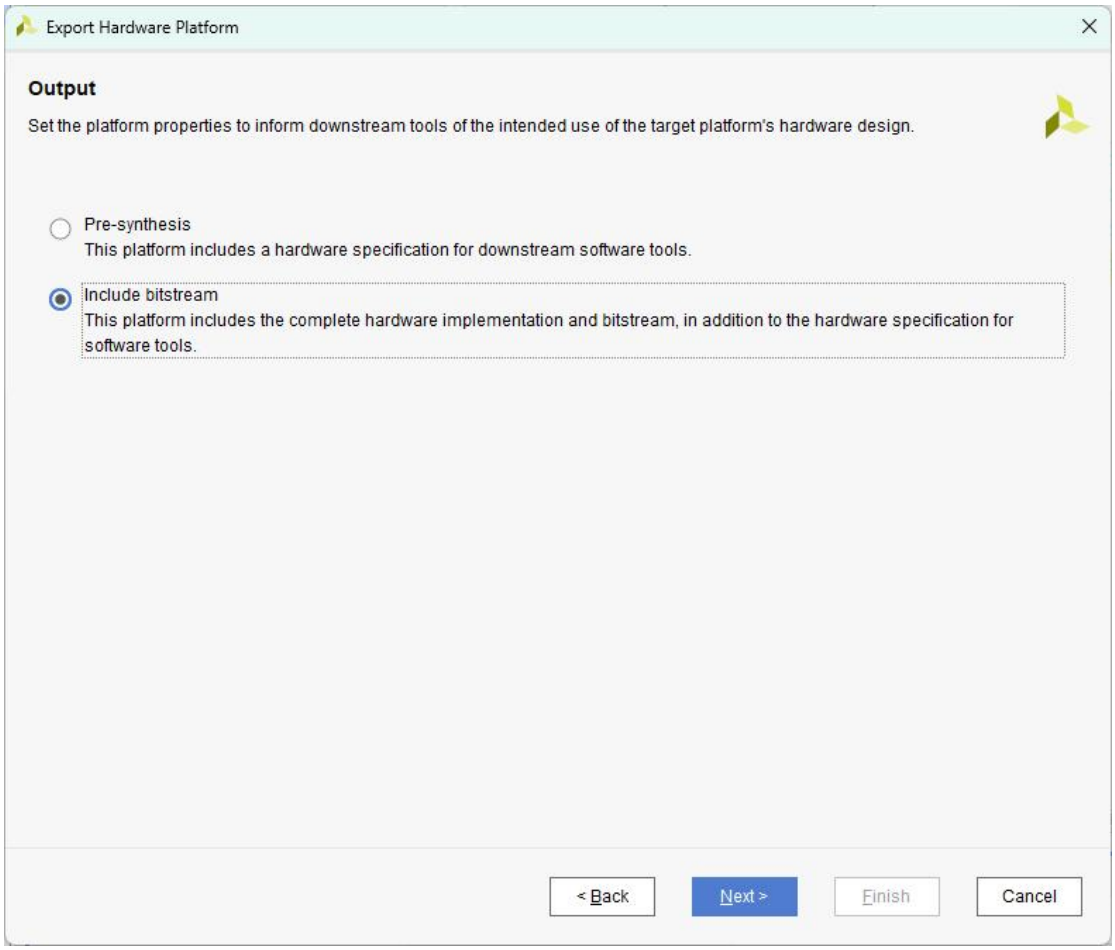
7) 编译生成 bit 文件

8) 编译完成后，导出硬件工程，给后面 vitis 软件开发使用；

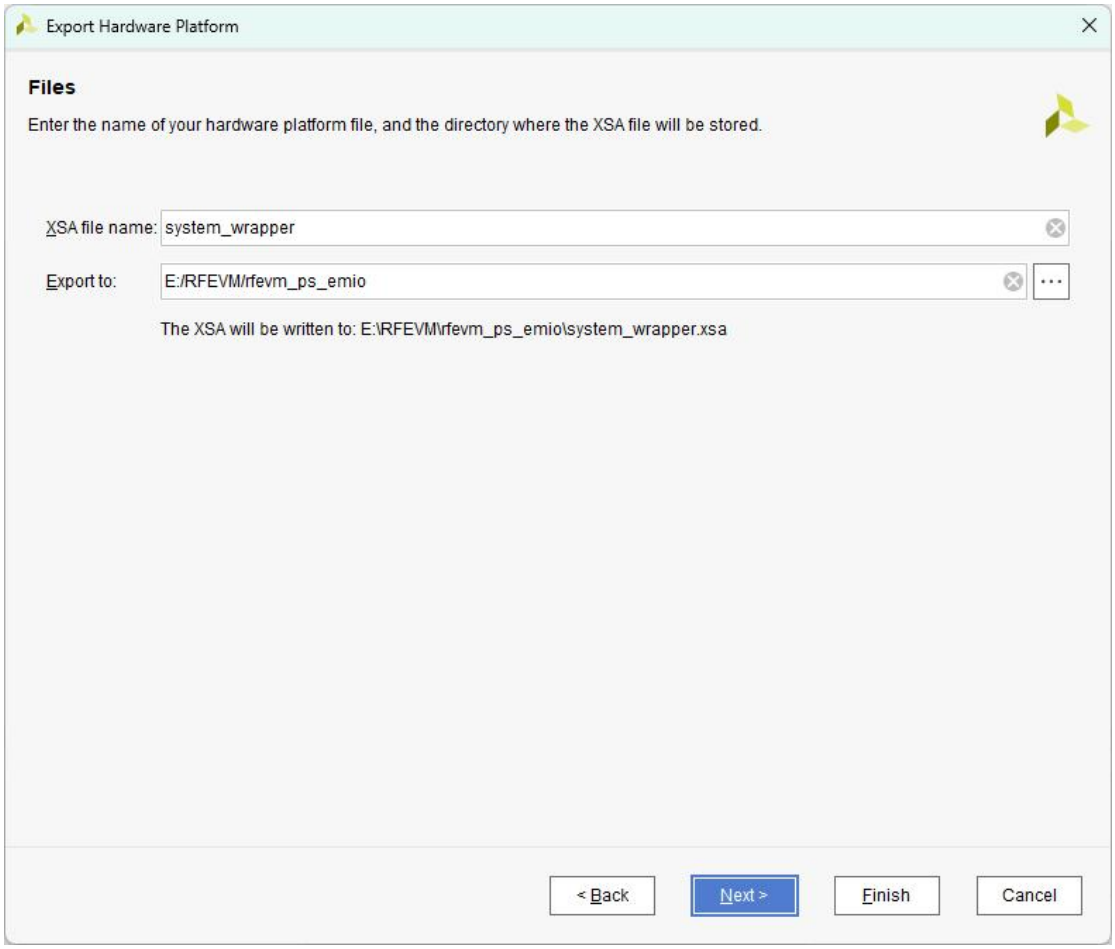


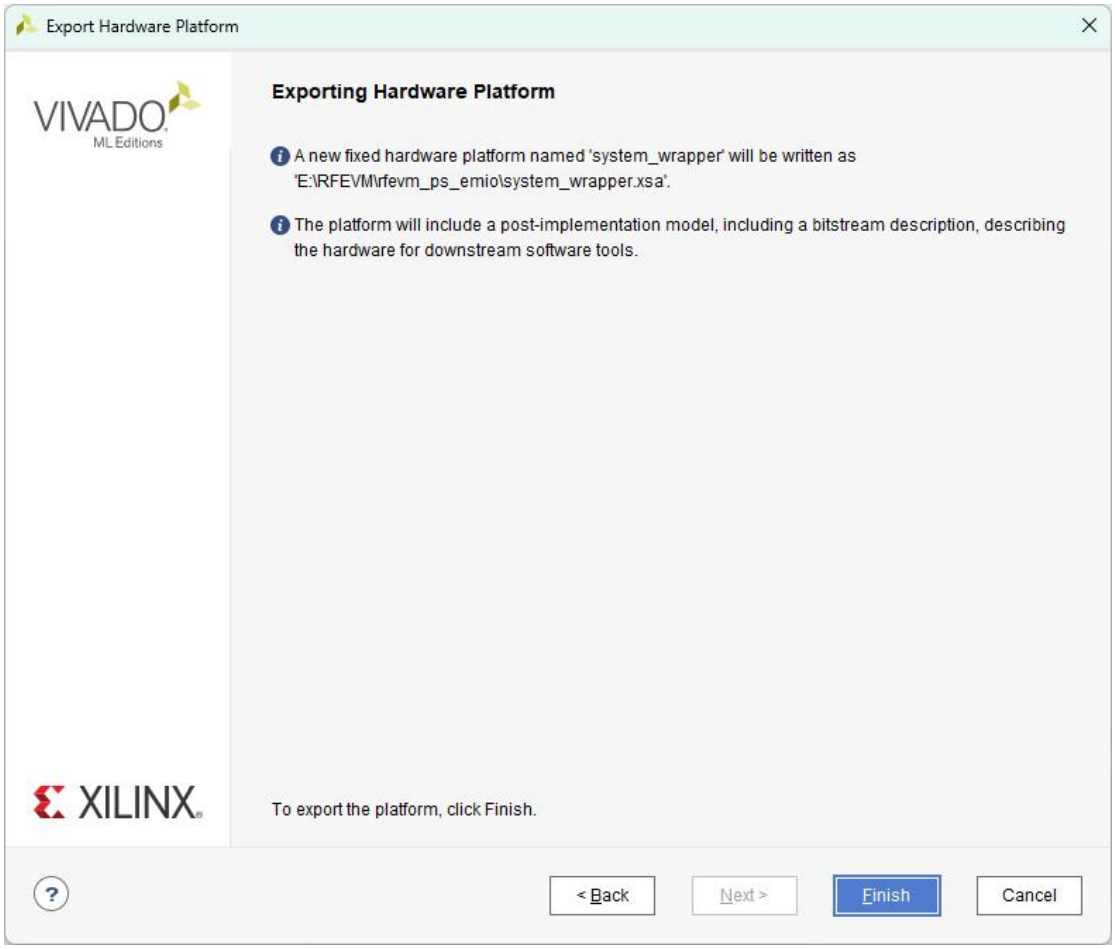


这里我们选择包含 bitstream，点击【Next】

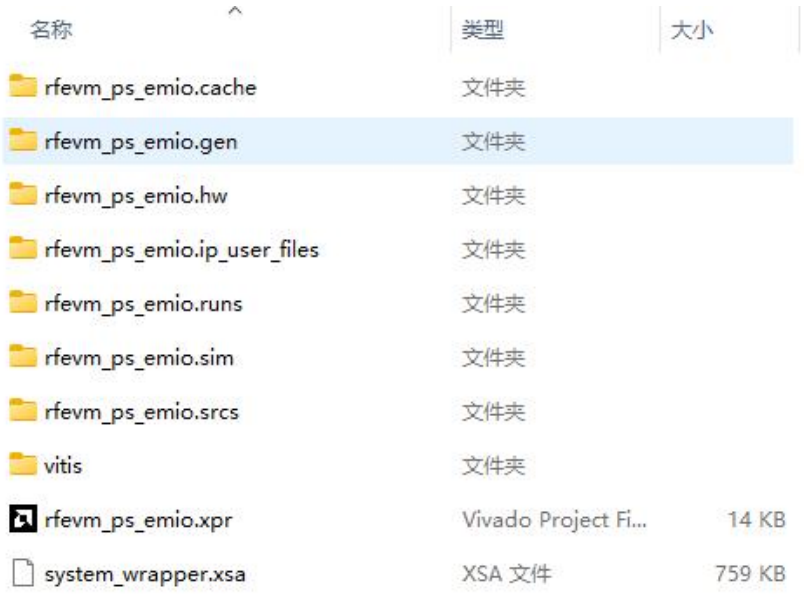


设置 xsa 的名称和路径


Export Hardware Platform
Files
Enter the name of your hardware platform file, and the directory where the XSA file will be stored.
XSA file name:
system_wrapper
Export to:
E:/RFEVM/rfevm_ps_emio
The XSA will be written to: E:\RFEVM\rfevm_ps_emio\system_wrapper.xsa
< Back
Next >
Finish
Cancel



点击【Finish】后，可以在工程路径下面看到新生成的 xsa 文件。

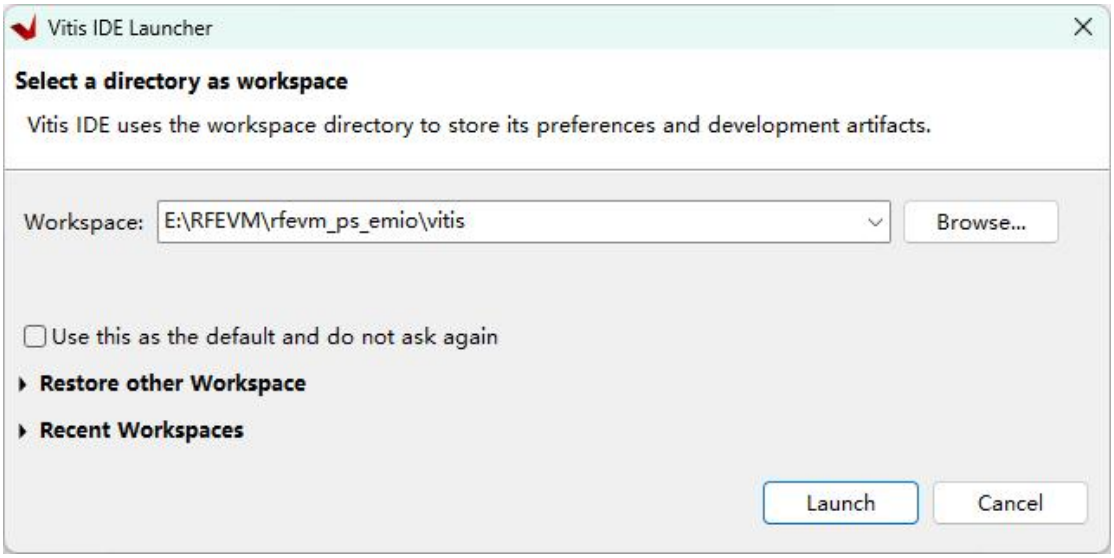


5.1.2 软件工程搭建

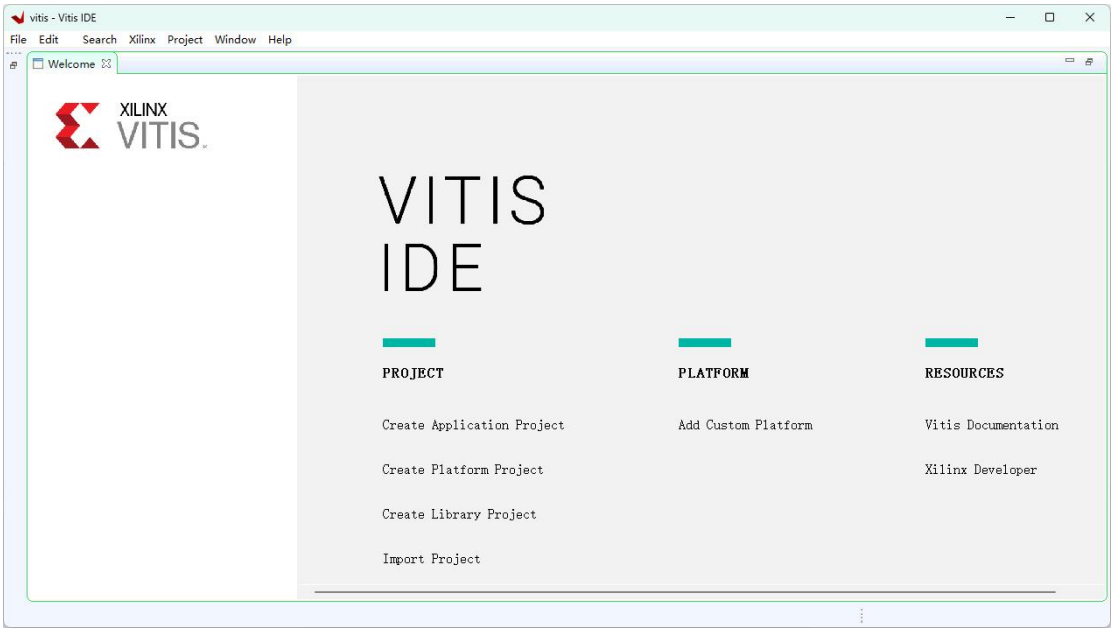
我们在 vivado 下搭建好硬件工程，导出 xsa 文件后，在工程目录下面创建

vitis 文件夹作为 vitis 的工作目录。

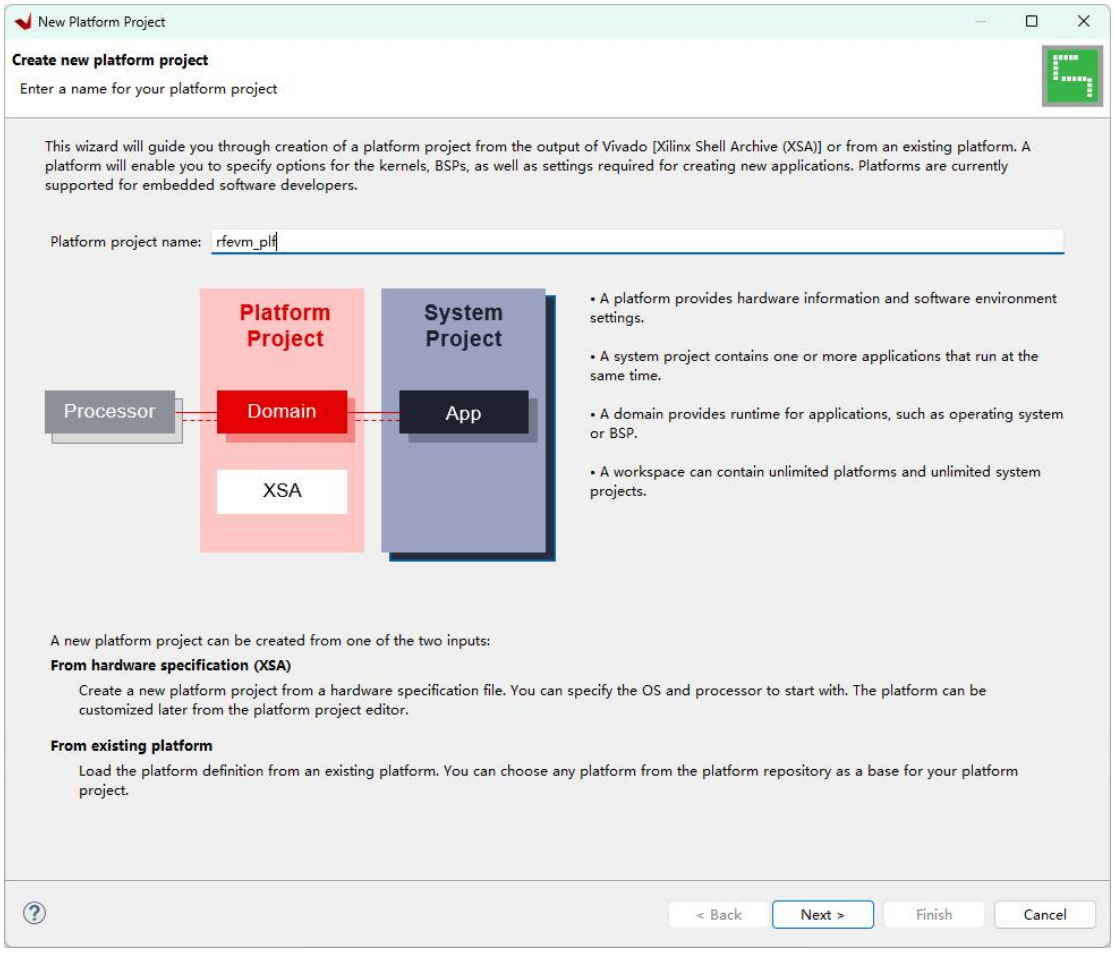
1) 登录 vitis，选择本工程路径下面的 vitis 文件夹作为工作目录



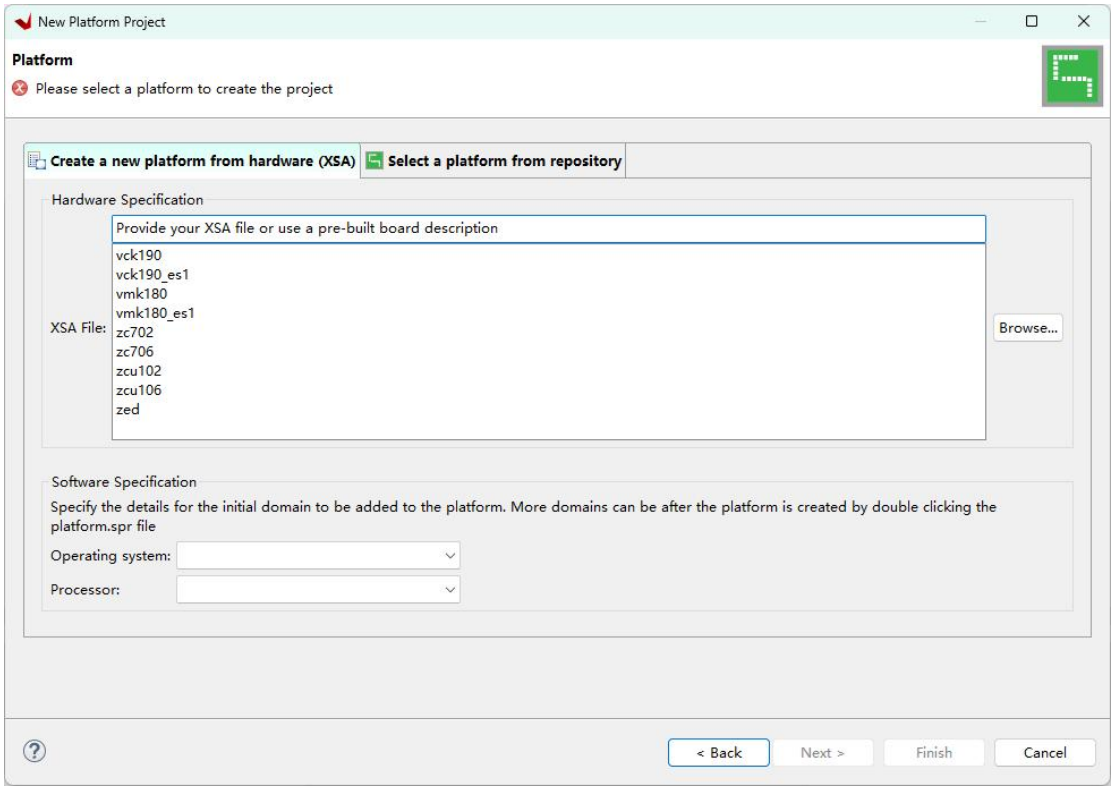
2) 选择【Create Platform Project】



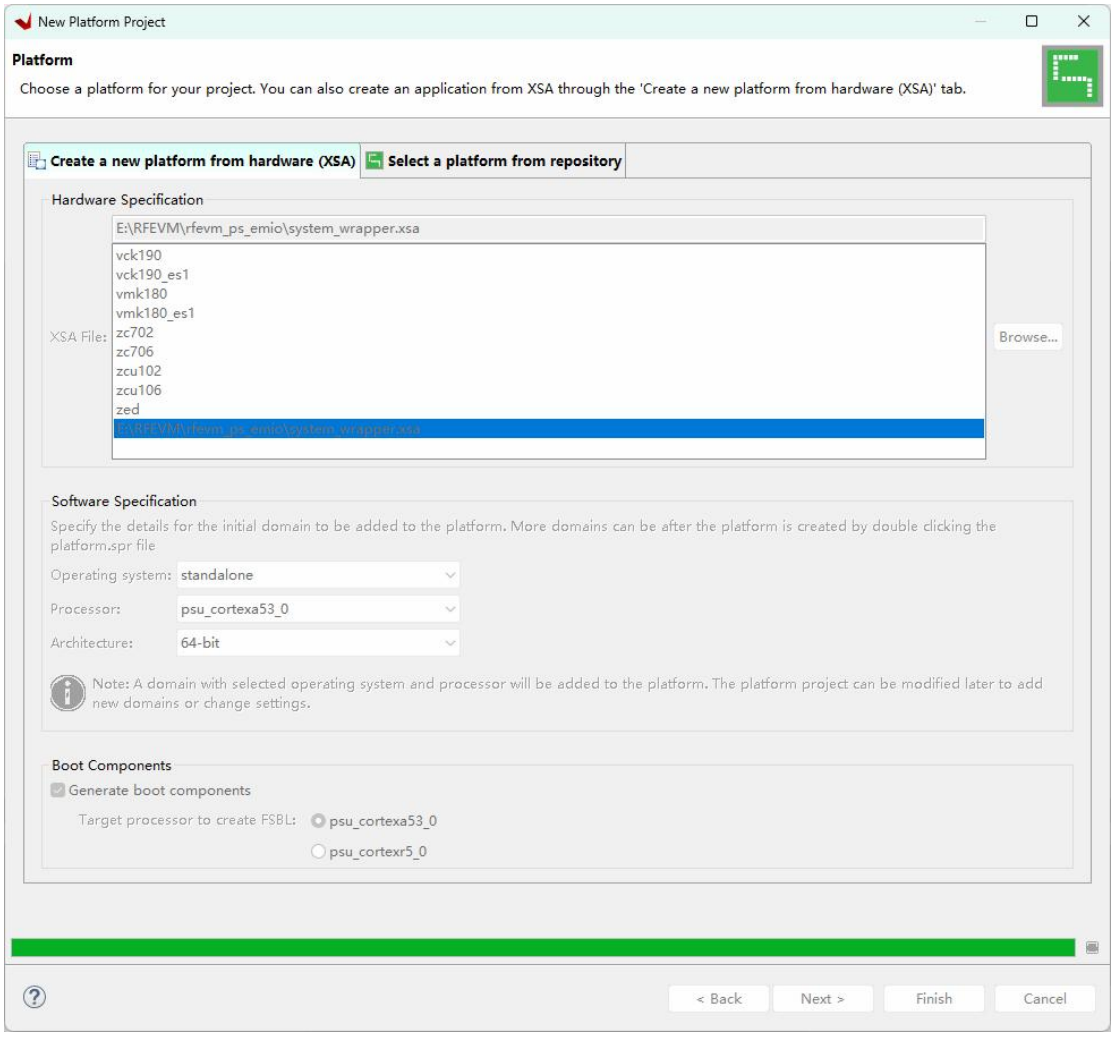
设置平台名称，点击【Next】

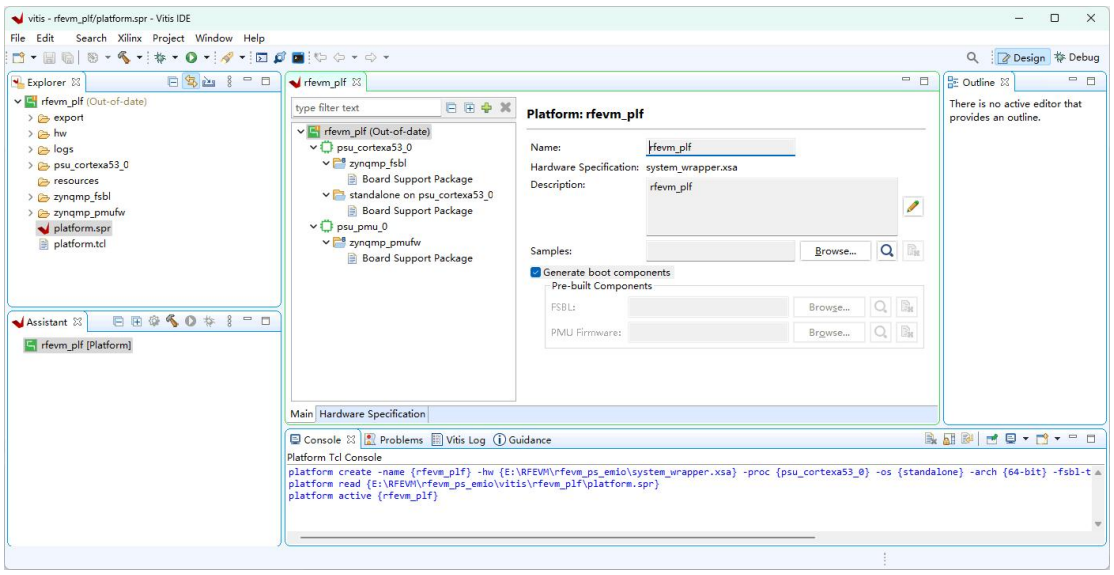


这里点击 Browse，选择生成的 xsa 文件

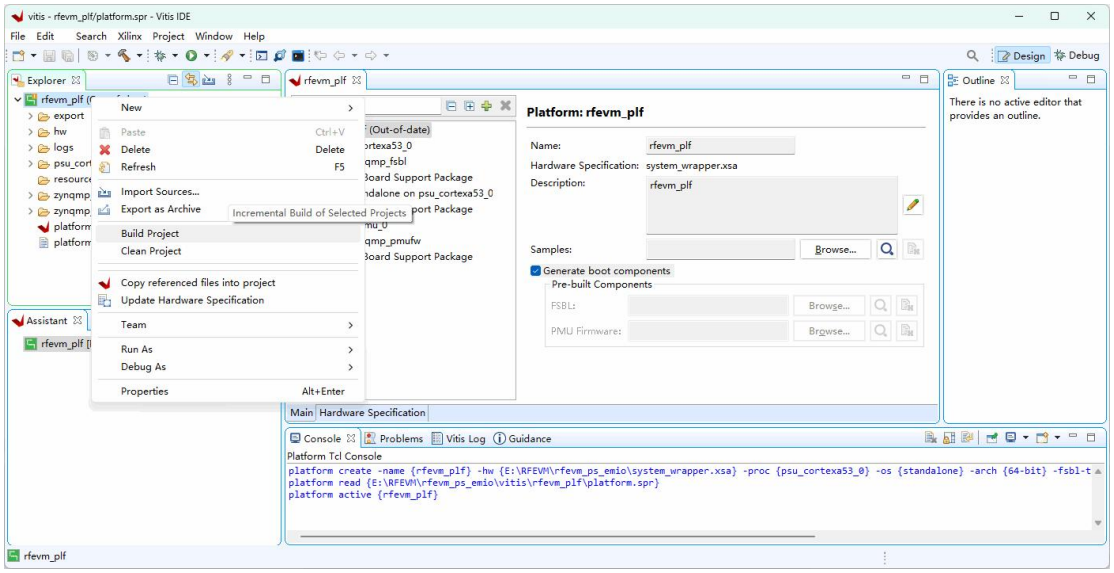


点击【Finish】，生成硬件平台

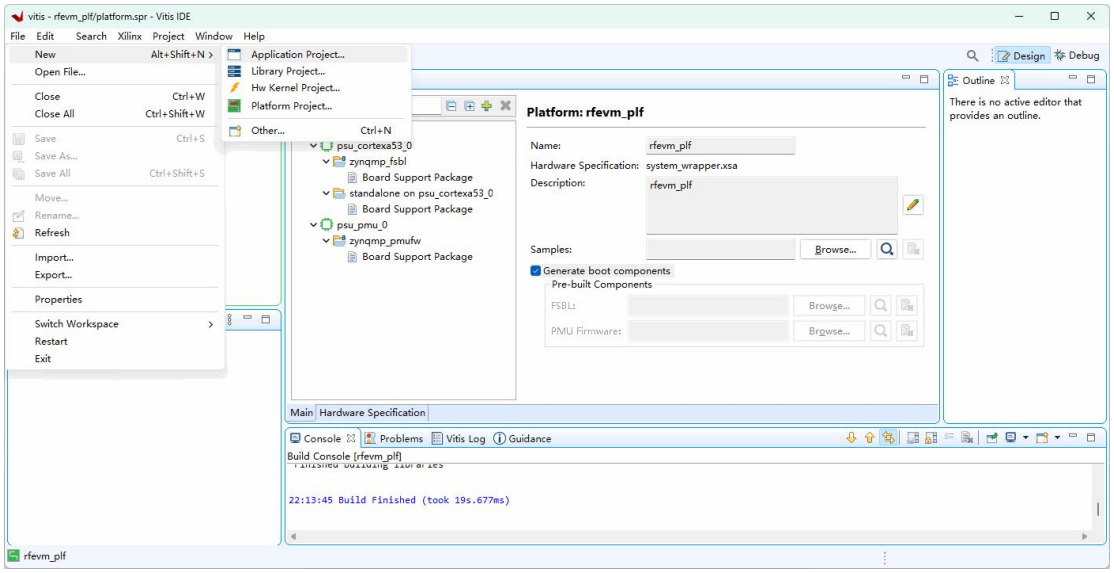




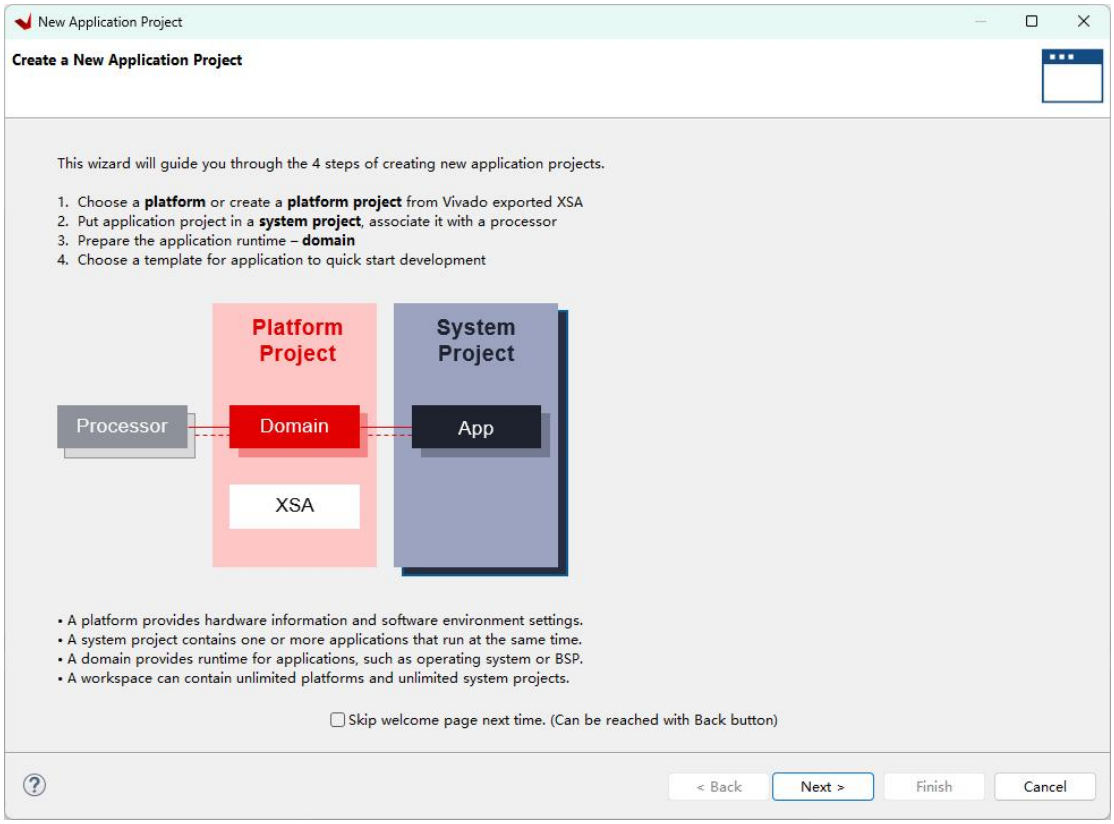
3) 编译 plafform



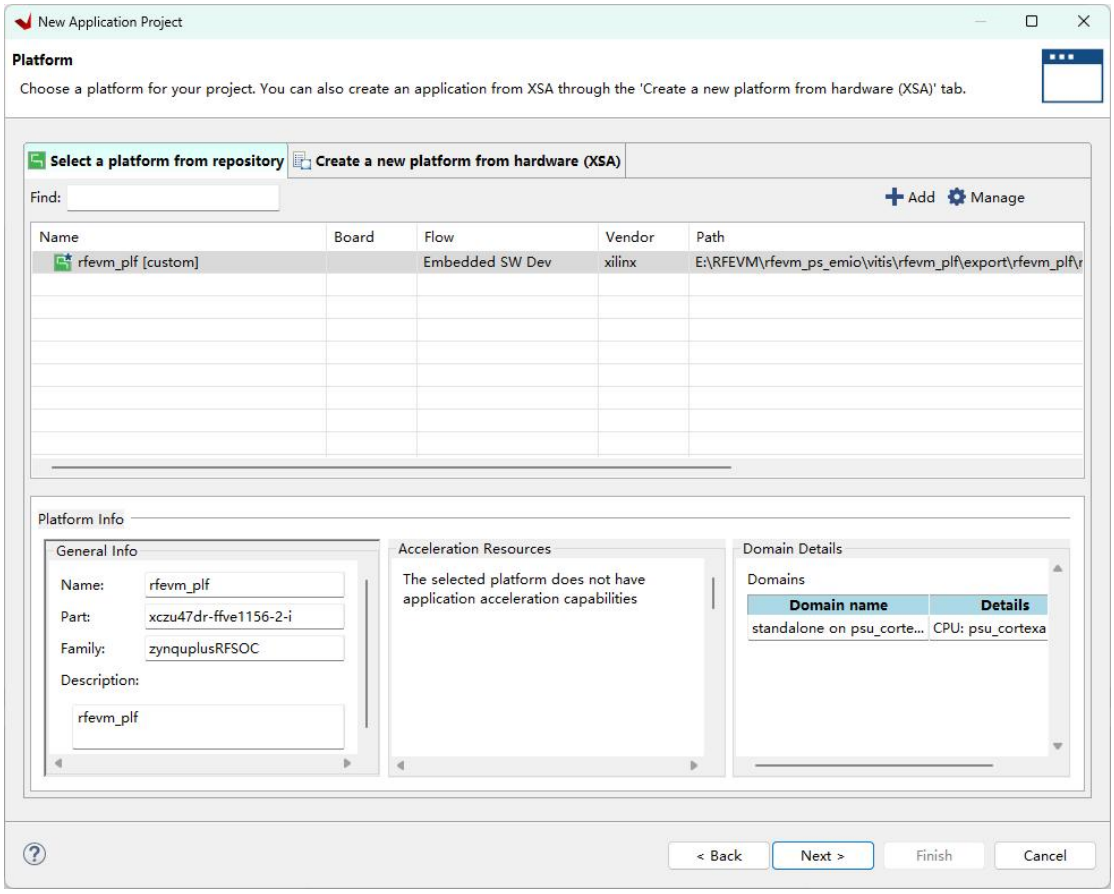
4) 编译完成之后，新建应用工程



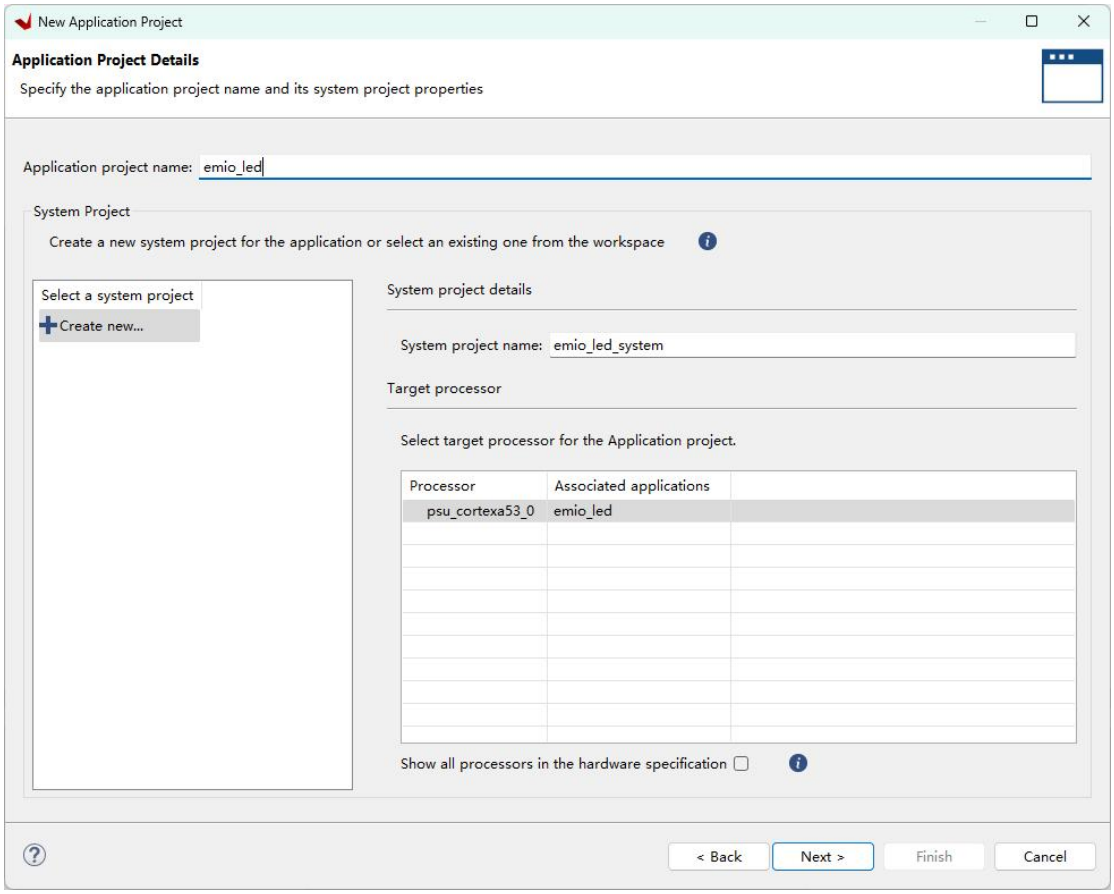
点击【Next】


New Application Project
Create a New Application Project
This wizard will guide you through the 4 steps of creating new application projects.
1. Choose a platform or create a platform project from Vivado exported XSA
2. Put application project in a system project, associate it with a processor
3. Prepare the application runtime – domain
4. Choose a template for application to quick start development
Platform Project
System Project
Processor
Domain
App
XSA
• A platform provides hardware information and software environment settings.
• A system project contains one or more applications that run at the same time.
• A domain provides runtime for applications, such as operating system or BSP.
• A workspace can contain unlimited platforms and unlimited system projects.
Skip welcome page next time. (Can be reached with Back button)
< Back
Next >
Finish
Cancel

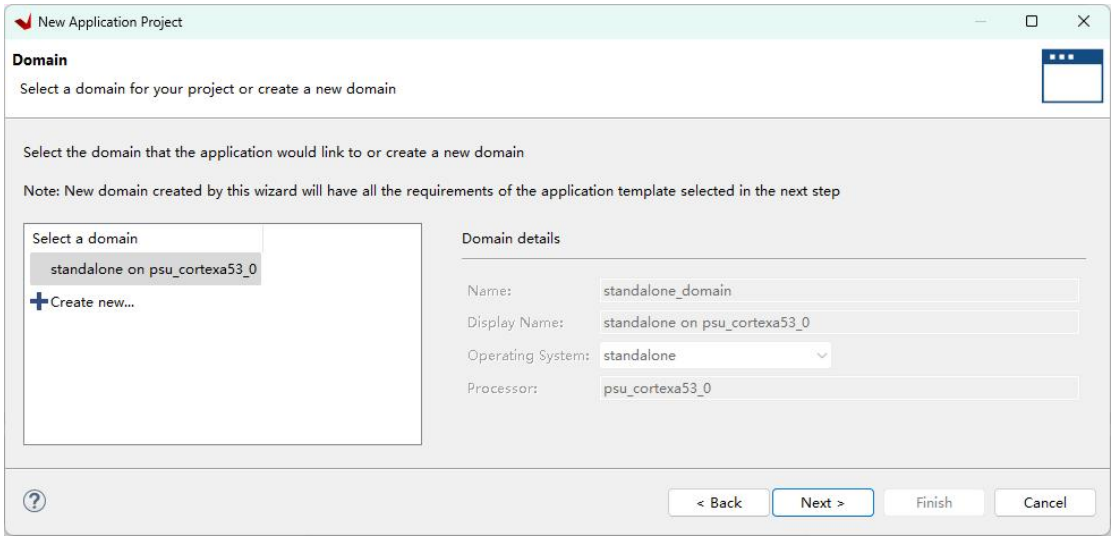
点击【Next】


New Application Project
Platform
Choose a platform for your project. You can also create an application from XSA through the 'Create a new platform from hardware (XSA)' tab.
Select a platform from repository
Create a new platform from hardware (XSA)
Find:
Add
Manage
Name
Board
Flow
Vendor
Path
rfevm_plf [custom]
Embedded SW Dev
xilinx
E:\RFEVM\rfevm_ps_emio\vitis\rfevm_plf\export\rfevm_plf\r
Platform Info
General Info
Name: rfevm_plf
Part: xczu47dr-ffve1156-2-i
Family: zynquplusRFSOC
Description:
rfevm_plf
Acceleration Resources
The selected platform does not have application acceleration capabilities
Domain Details
Domains
Domain name
Details
standalone on psu_corte...
CPU: psu_cortexa
< Back
Next >
Finish
Cancel

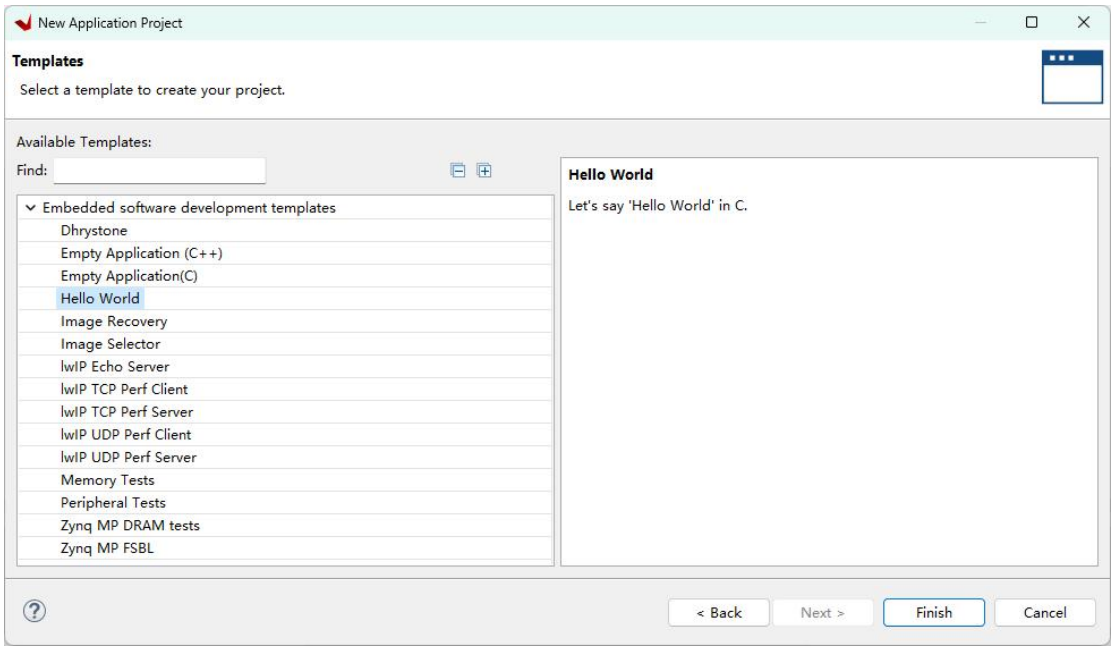
设置应用工程名称，并点击【Next】



点击【Next】



选择“Hello World”模板，点击【Finish】



5) 这里，我们修改 hello world 源文件，添加 GPIO 控制代码

rfevm_plf | emio_led_system | emio_led | helloworld.c

```c
#include "xil_printf.h"
#include "xgpiops.h"
#include "sleep.h"

#define GPIO_DEVICE_ID      XPAR_XGPIOPS_0_DEVICE_ID

XGpioPs Gpio;   /* The driver instance for GPIO Device. */

int main()
{
    init_platform();

    int Status;
    XGpioPs_Config *ConfigPtr;

    print("Hello World\n\r");
    /* Initialize the GPIO driver. */
    ConfigPtr = XGpioPs_LookupConfig(GPIO_DEVICE_ID);

    Status = XGpioPs_CfgInitialize(&Gpio, ConfigPtr,
            ConfigPtr->BaseAddr);
    if (Status != XST_SUCCESS) {
        return XST_FAILURE;
    }

    /*
     * Set the direction for the pin to be output and
     * Enable the Output enable for the LED Pin.
     */
    XGpioPs_SetDirectionPin(&Gpio, 78+0, 1);
    XGpioPs_SetDirectionPin(&Gpio, 78+1, 1);
    XGpioPs_SetDirectionPin(&Gpio, 78+2, 1);
    XGpioPs_SetDirectionPin(&Gpio, 78+3, 1);
    XGpioPs_SetOutputEnablePin(&Gpio, 78+0, 1);
    XGpioPs_SetOutputEnablePin(&Gpio, 78+1, 1);
    XGpioPs_SetOutputEnablePin(&Gpio, 78+2, 1);
    XGpioPs_SetOutputEnablePin(&Gpio, 78+3, 1);
    while(1){
        /* Set the GPIO output to be low. */
        XGpioPs_WritePin(&Gpio, 78+0, 0x0);
        XGpioPs_WritePin(&Gpio, 78+1, 0x0);
        XGpioPs_WritePin(&Gpio, 78+2, 0x0);
        XGpioPs_WritePin(&Gpio, 78+3, 0x0);
        sleep(1) ;
        /* Set the GPIO output to be high. */
        XGpioPs_WritePin(&Gpio, 78+0, 0x1);
        XGpioPs_WritePin(&Gpio, 78+1, 0x1);
        XGpioPs_WritePin(&Gpio, 78+2, 0x1);
        XGpioPs_WritePin(&Gpio, 78+3, 0x1);
        sleep(1) ;
    }

    cleanup_platform();
    return 0;
}
```

注意：mpsoc 有 78 个 MIO，EMIO 的序号，从 78 开始；

5.1.3 下载验证

这里我们在 vitis 里面下载 bit 文件和 elf 文件，下载步骤请参考之前介绍，可以看到的底板上的四了 led 灯一直闪烁。

5.2 PL 端 AXI GPIO 的使用

AXI GPIO 是一种特定的 GPIO（通用输入/输出）控制器，它使用 AXI（高级扩展接口）总线协议进行通信。

AXI GPIO 用于控制和读取 GPIO 引脚的状态。广泛应用于嵌入式系统中，用于与处理器或其它外设进行通信，以控制和读取 GPIO 引脚的状态。它可以用

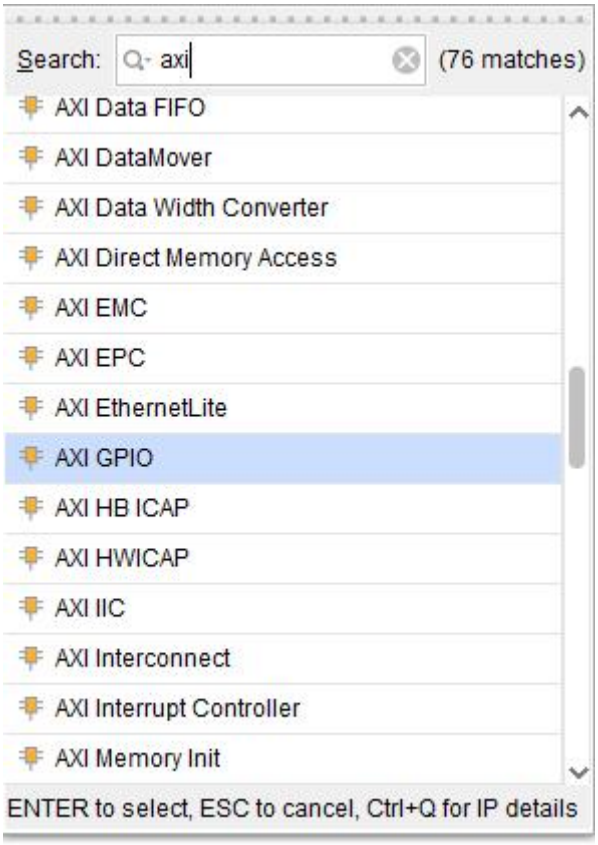
于连接各种外设，如 LED、按钮、传感器等，实现与外部环境的交互。

这里我们使用 AXI GPIO 去控制底板上 led 灯的明灭变化。

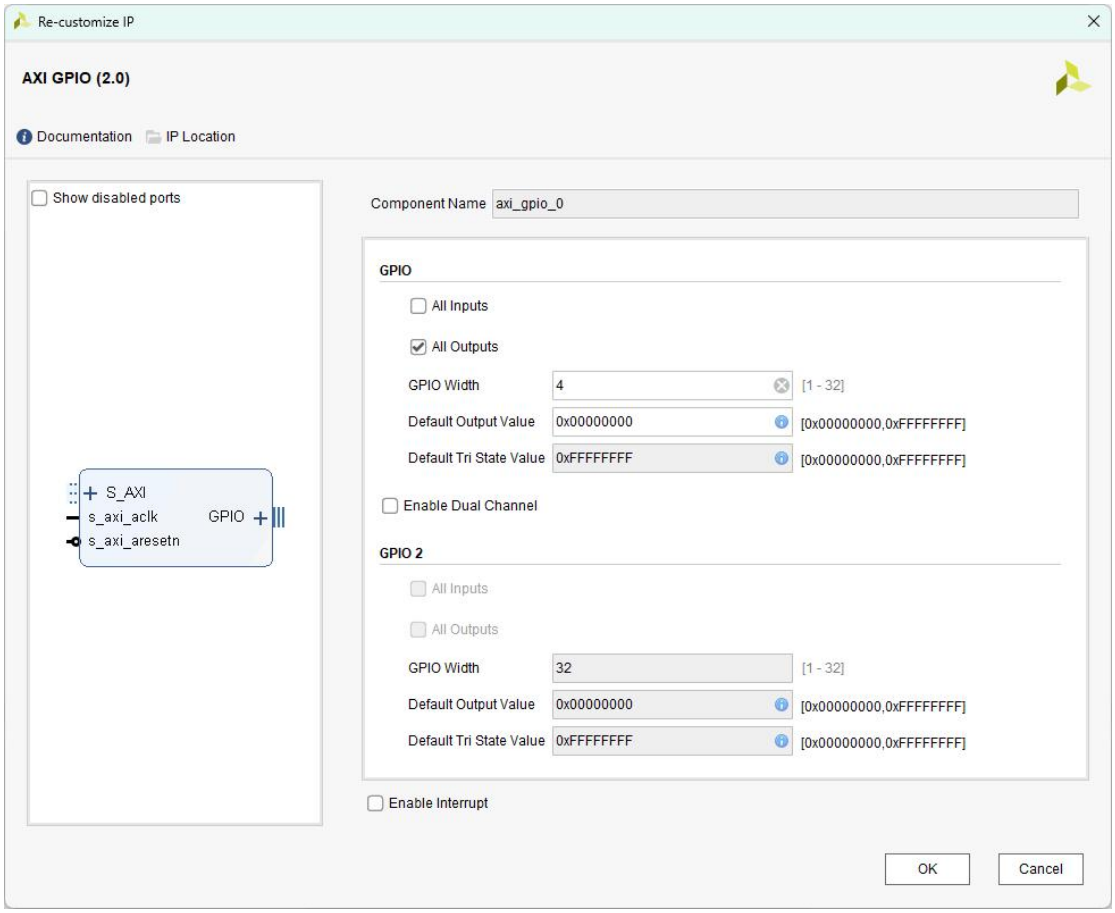
5.2.1 硬件工程搭建

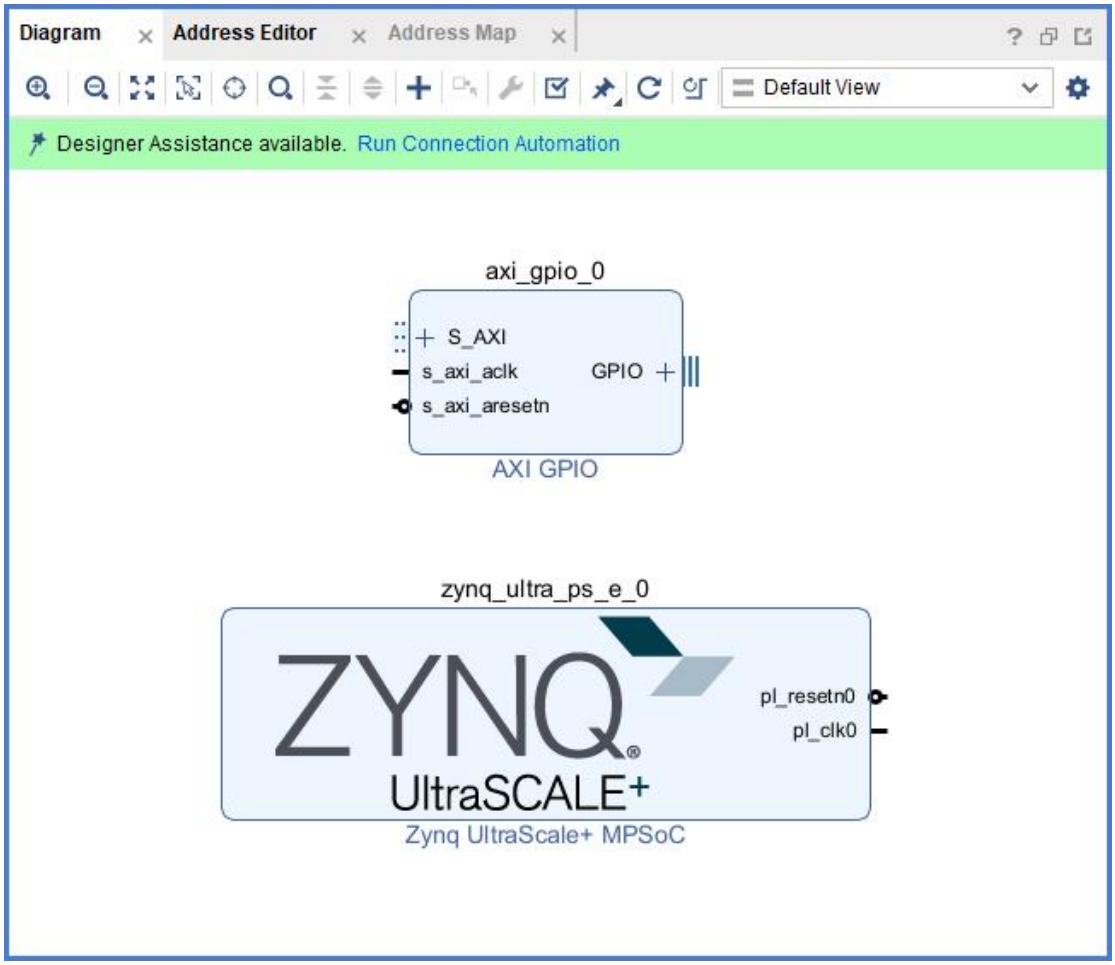
1) 按照上章 IPI 的工程搭建步骤，搭建本实验的硬件工程

2) 在 IPI 界面上，右键添加 AXI GPIO 的 IP 核

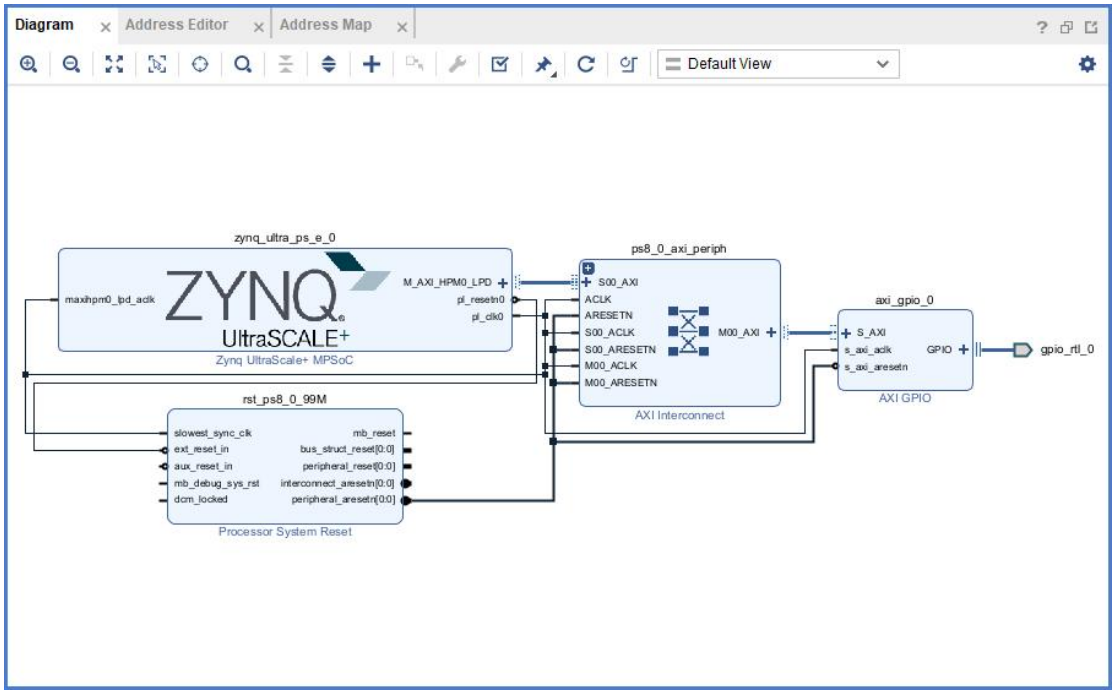


3) 这里我们只使能 GPIO 1，我们只控制 LED 的输出，所以使能全部输出，位宽 4bit，然后点击【OK】


Re-customize IP
AXI GPIO (2.0)
Documentation
IP Location
Show disabled ports
S_AXI
s_axi_aclk
s_axi_aresetn
GPIO
Component Name
axi_gpio_0
GPIO
All Inputs
All Outputs
GPIO Width
4
[1 - 32]
Default Output Value
0x00000000
[0x00000000,0xFFFFFFFF]
Default Tri State Value
0xFFFFFFFF
[0x00000000,0xFFFFFFFF]
Enable Dual Channel
GPIO 2
All Inputs
All Outputs
GPIO Width
32
[1 - 32]
Default Output Value
0x00000000
[0x00000000,0xFFFFFFFF]
Default Tri State Value
0xFFFFFFFF
[0x00000000,0xFFFFFFFF]
Enable Interrupt
OK
Cancel



4) 我们在 MPSoC 中，使能 LPD 接口，然后点击自动连接【Run Connection Automation】



5) 创建 HDL wrapper，然后生成 bit。

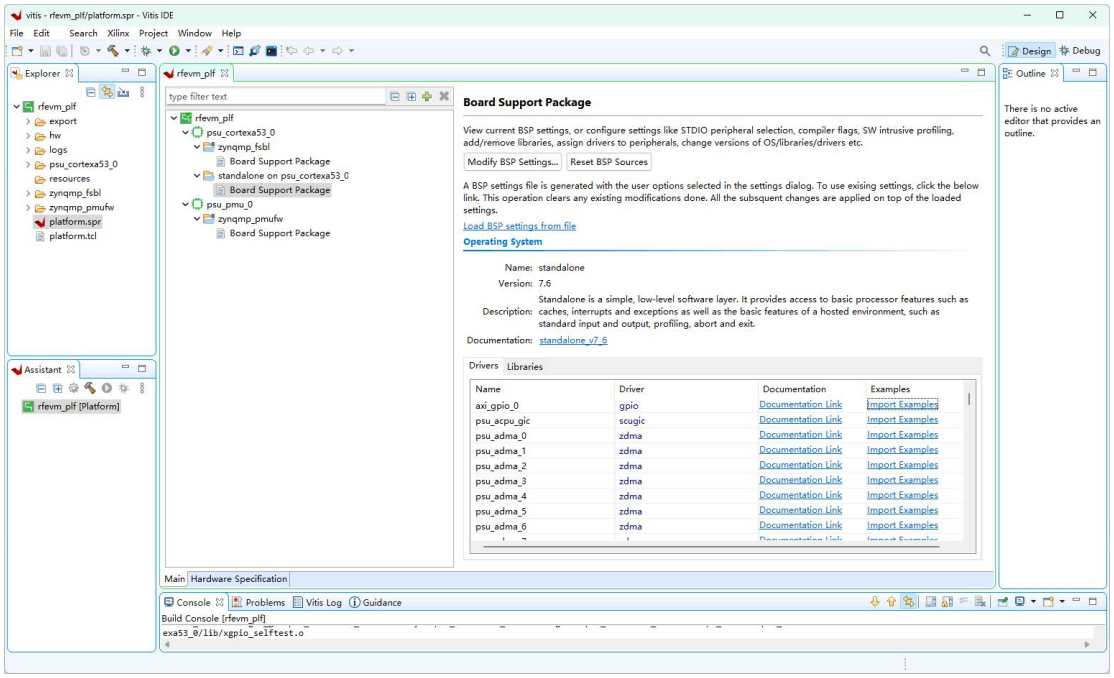
6) 导出硬件工程 xsa 文件（包含 bit 流）。

5.2.2 软件工程搭建

我们在 vivado 下搭建好硬件工程，导出 xsa 文件后，在工程目录下面创建 vitis 文件夹作为 vitis 的工作目录。

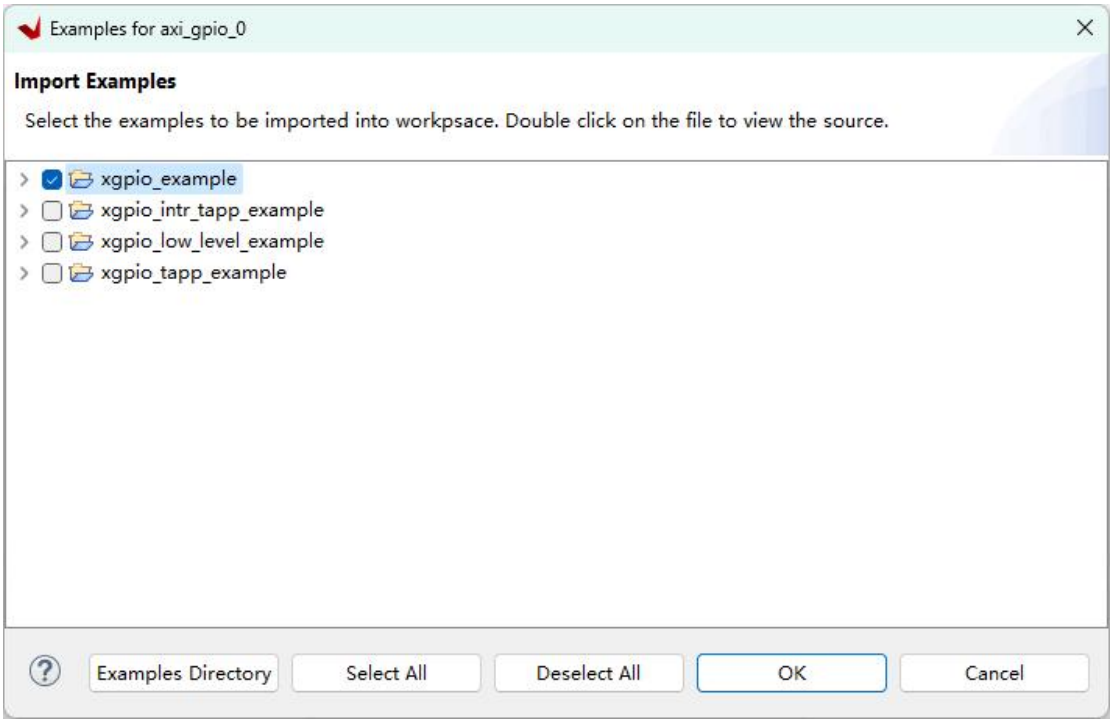
1) 登录 vitis，选择本工程路径下面的 vitis 文件夹作为工作目录

2) 创建硬件平台，具体步骤可参考上节；

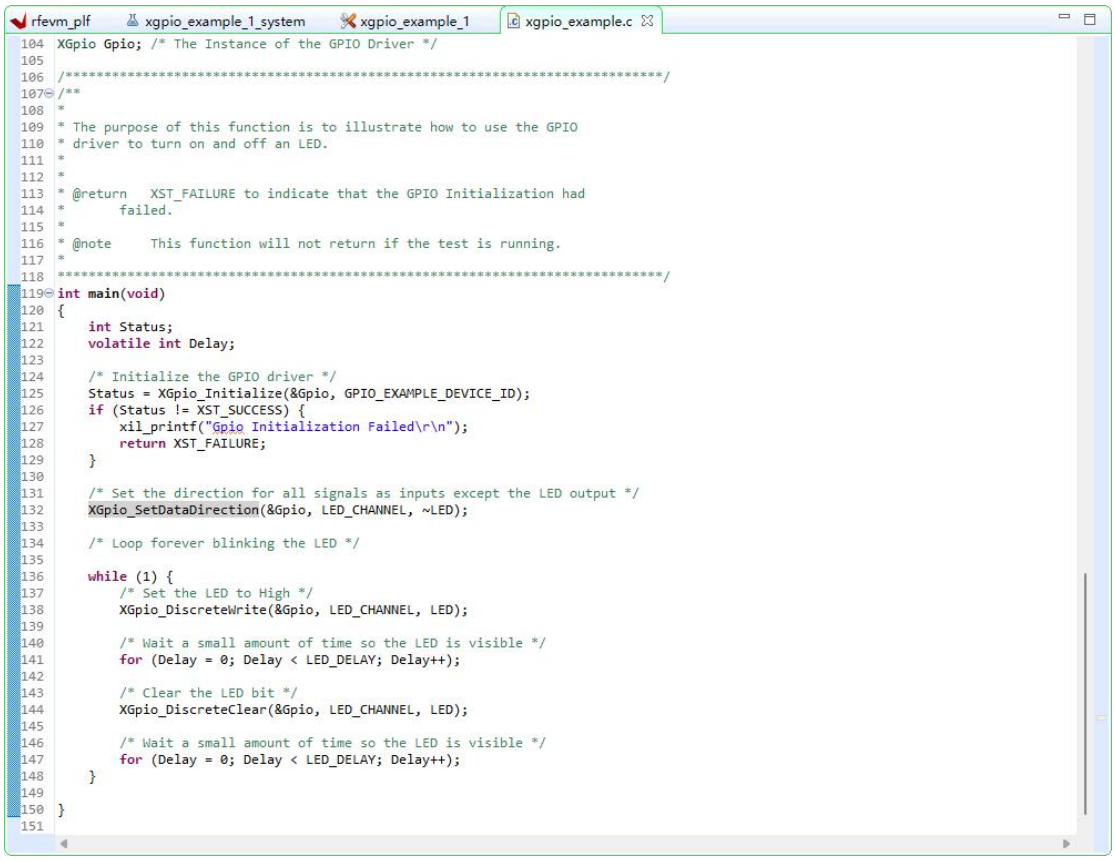


3) 在右下角，axi_gpio_0 里面，选 Import Example

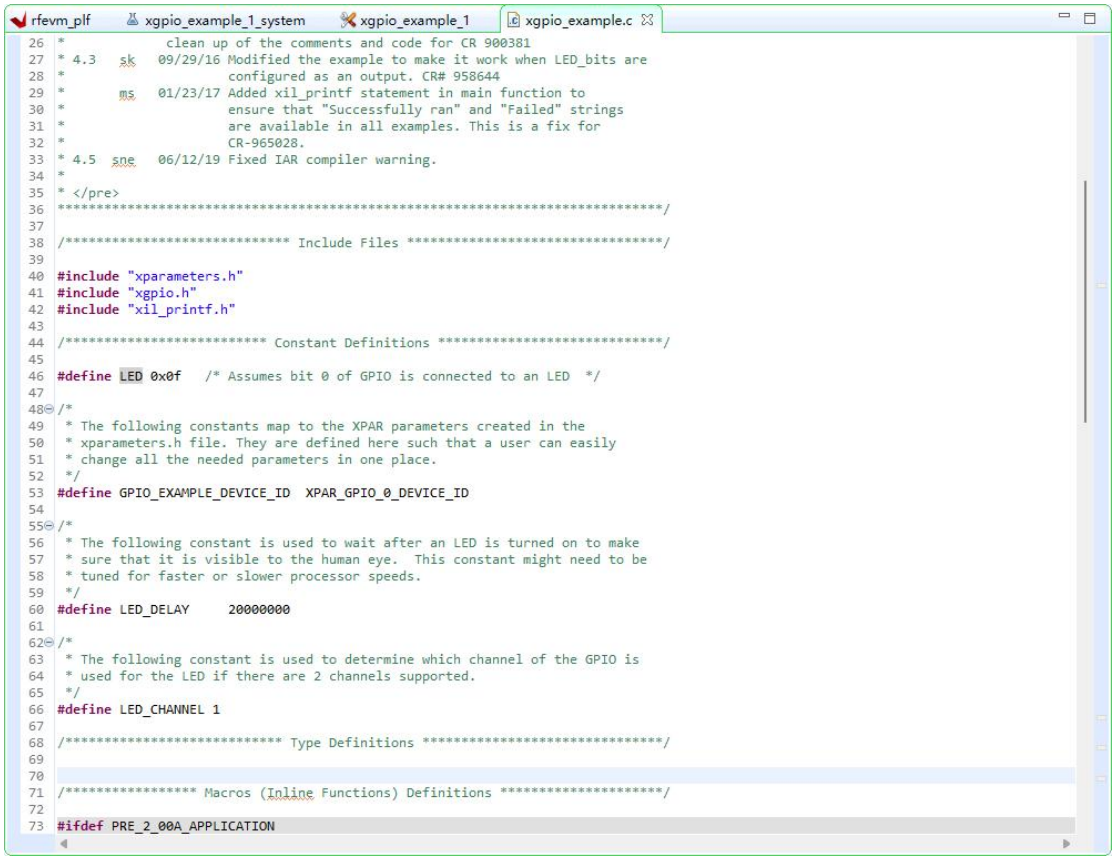
4) 选择“xgpio_example”



5) 打开源文件



这里我们修改宏定义 LED 的值为 0xF，对应 4 个 led 灯。



6) 编译应用程序，生成 elf 文件

5.2.3 下载验证

这里我们在 vitis 里面下载 bit 文件和 elf 文件，下载步骤请参考之前介绍，下载成功后可以看到的底板上的四了 led 灯一直闪烁。

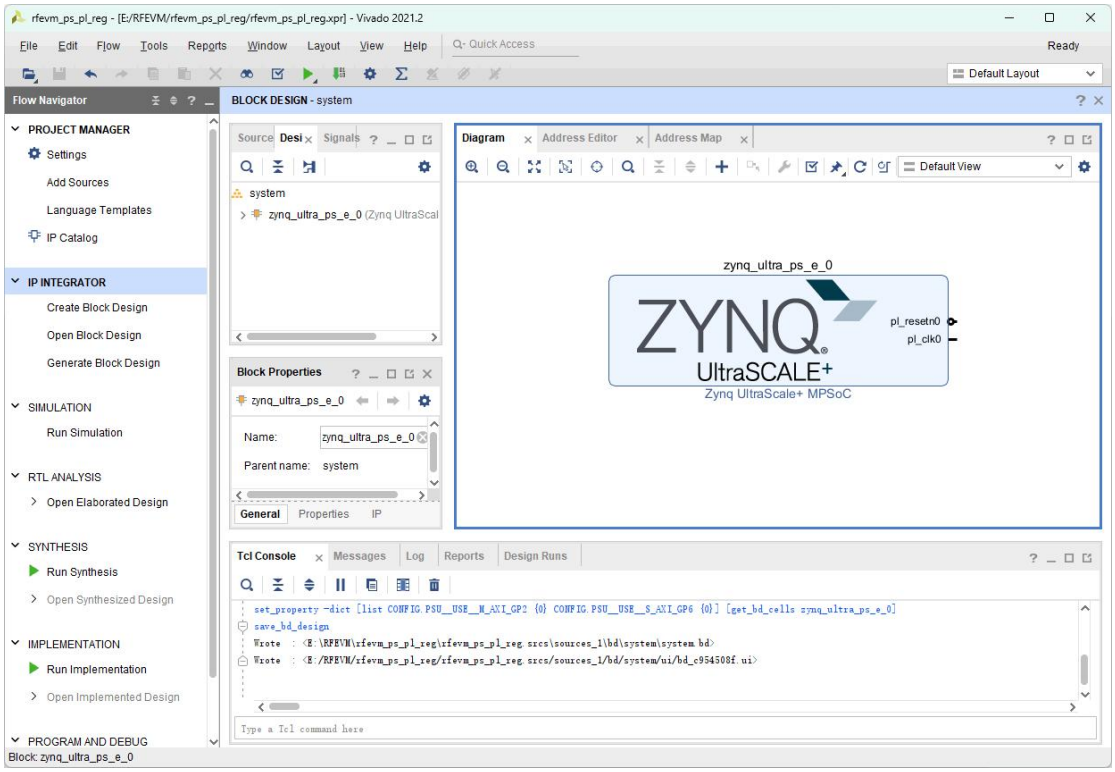
5.3 PS 端读写 PL 端的寄存器

很多时候，PS/PL 之间只需要简单的数据通路。PS 只需要下发有限的参数给 PL，PL 只需要向 PS 反馈有限的状态数据。这种情况下，PL 内部只需要实现 AXI-Lite 接口就可以，PS 通过 AXI 接口去访问寄存器，即向 PL 提供参数，也可以读回 PL 的状态。

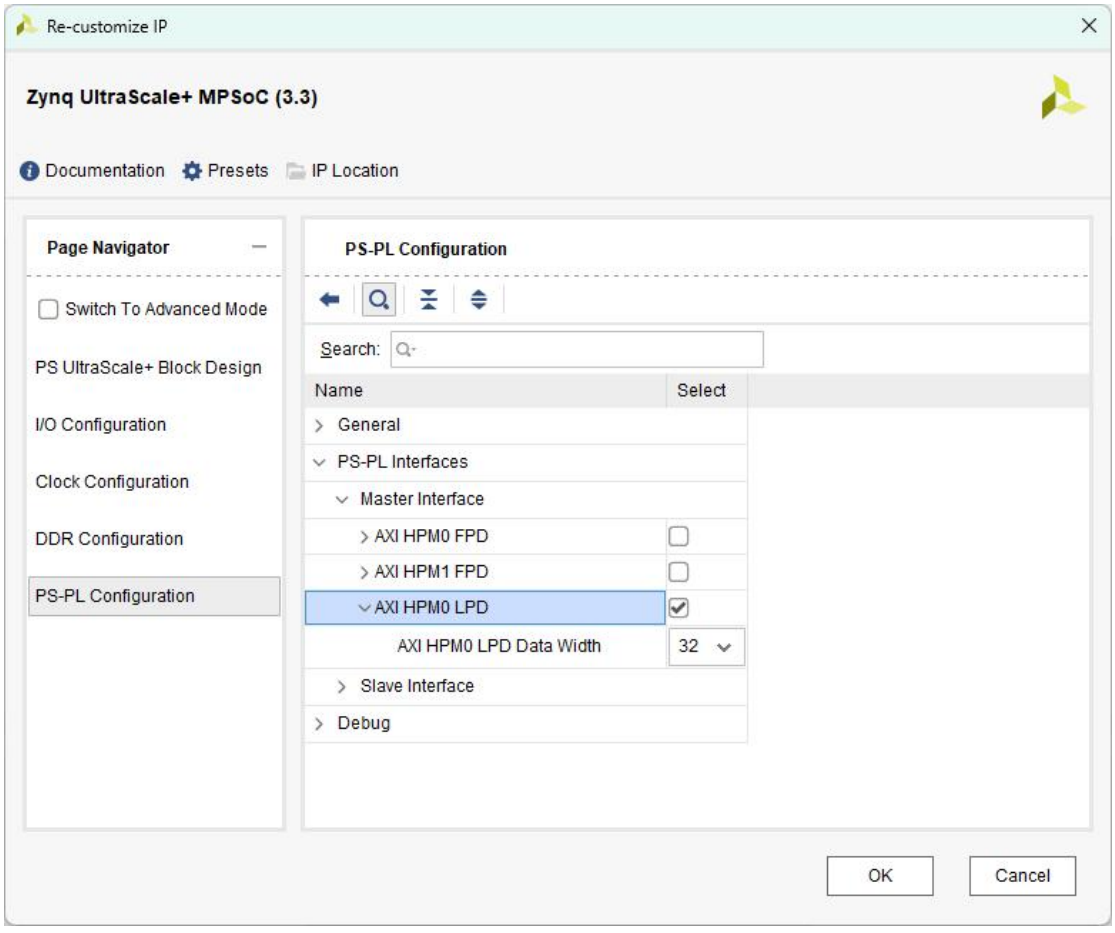
5.3.1 硬件工程搭建

这里我们依然在上篇 IPI 的工程基础上，去搭建硬件工程。

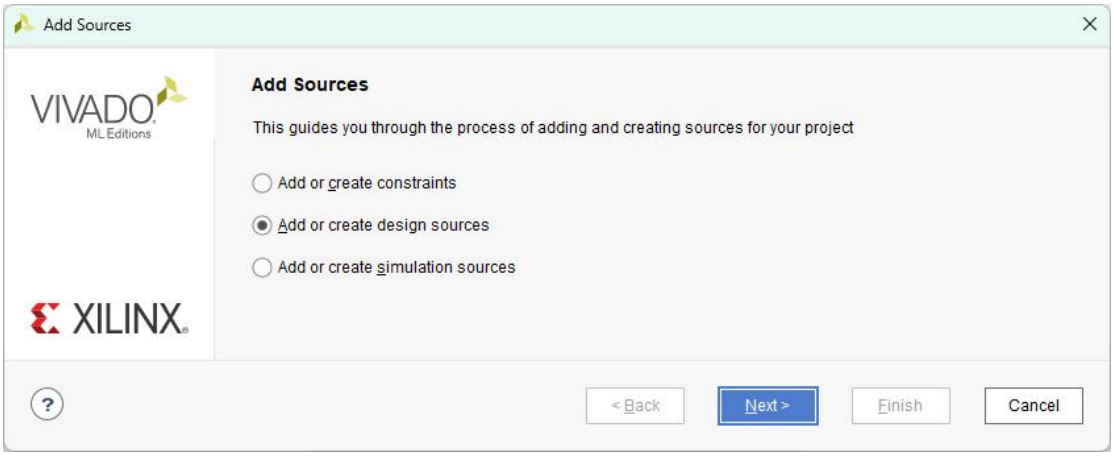
1) 我们按照之前 IPI 的工程去搭建本实验的硬件工程；

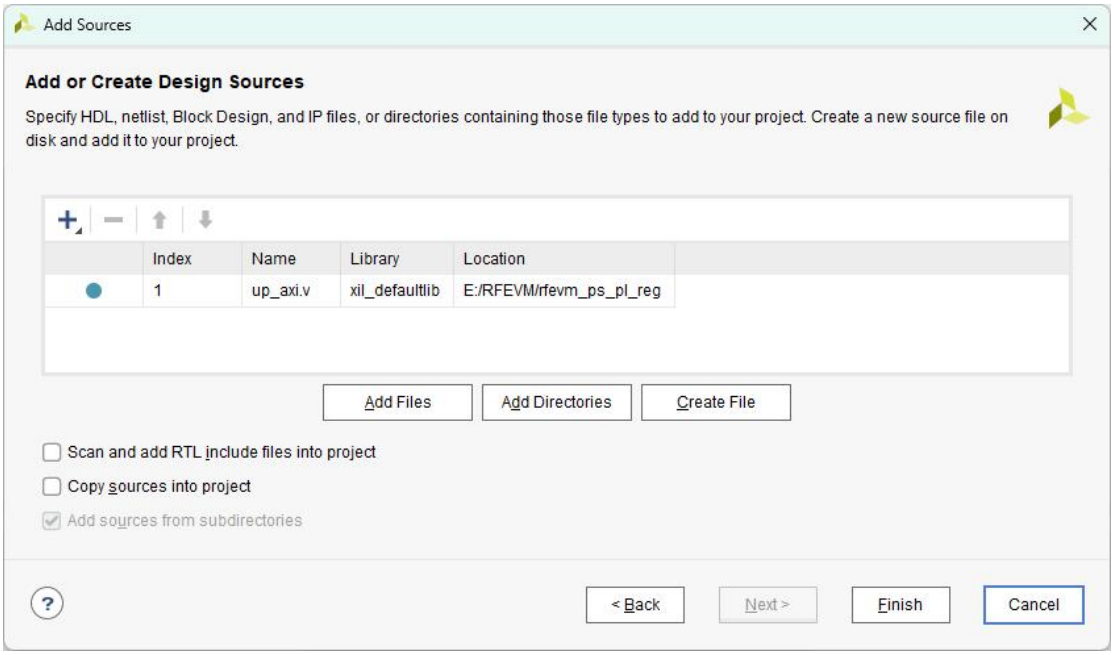


2) 双击 IP，使能 AXI HPM0 LPD 接口；

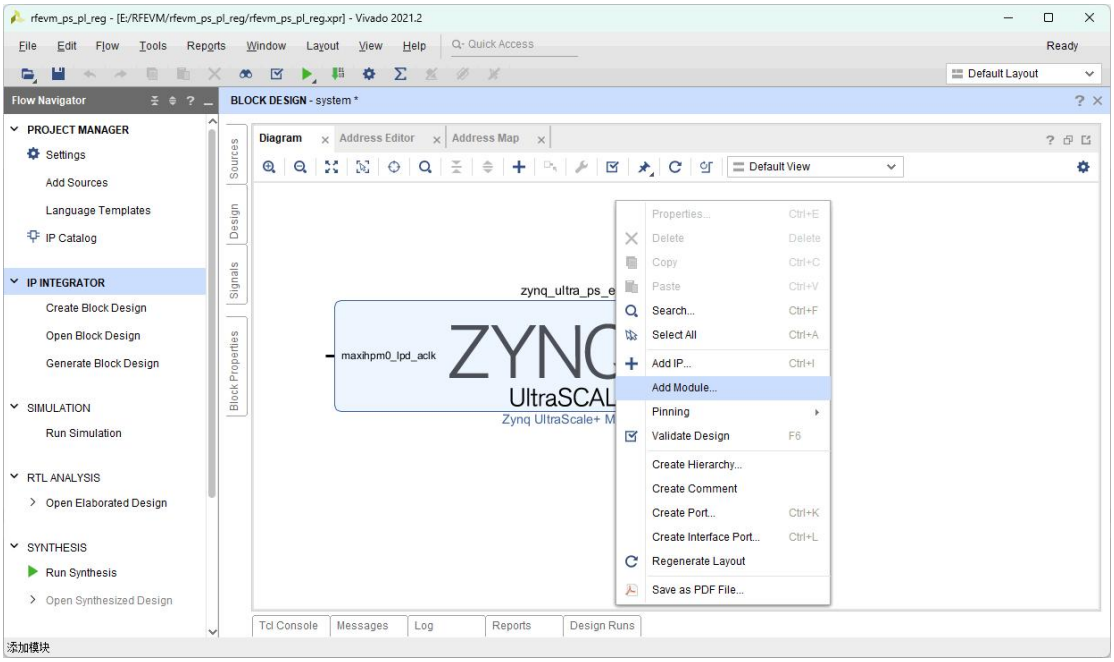


3) 我们在工程里面添加 up_axi.v 源文件（*up_axi 模块实现了 AXI-Lite 协议，这是 adi 的开源代码，我们只需要关注和使用用户侧信号即可*）

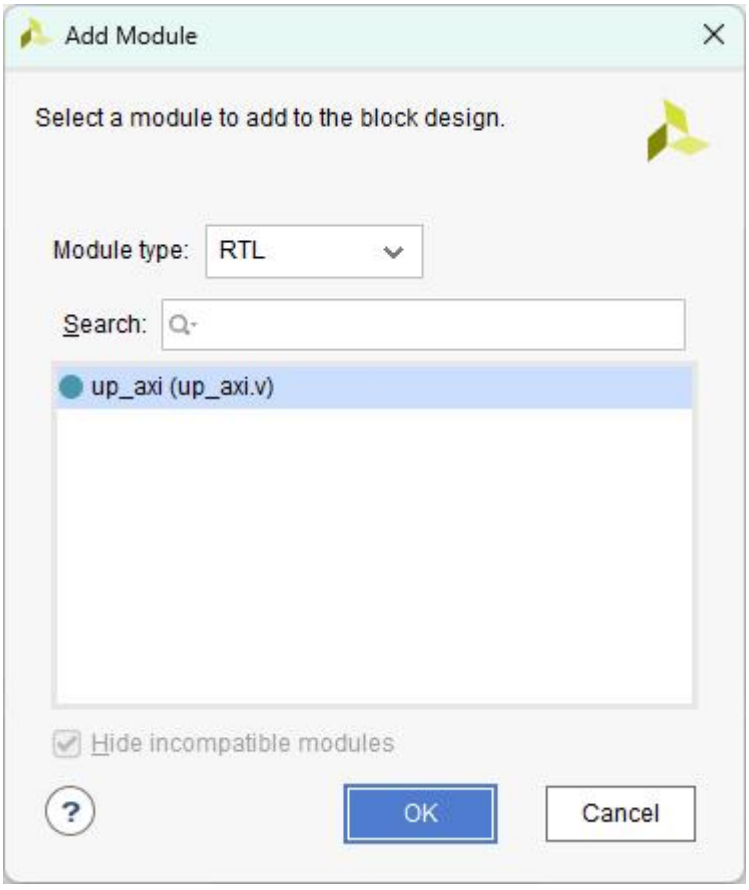




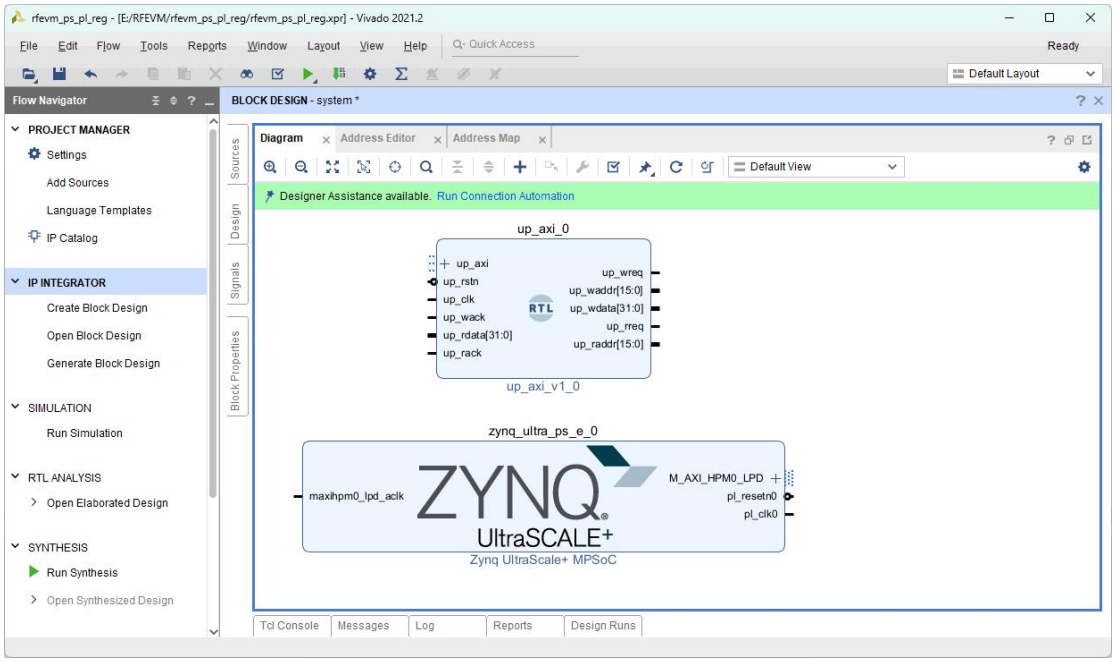
4) 添加完成之后，我们在 IPI 界面里面右键，添加模块



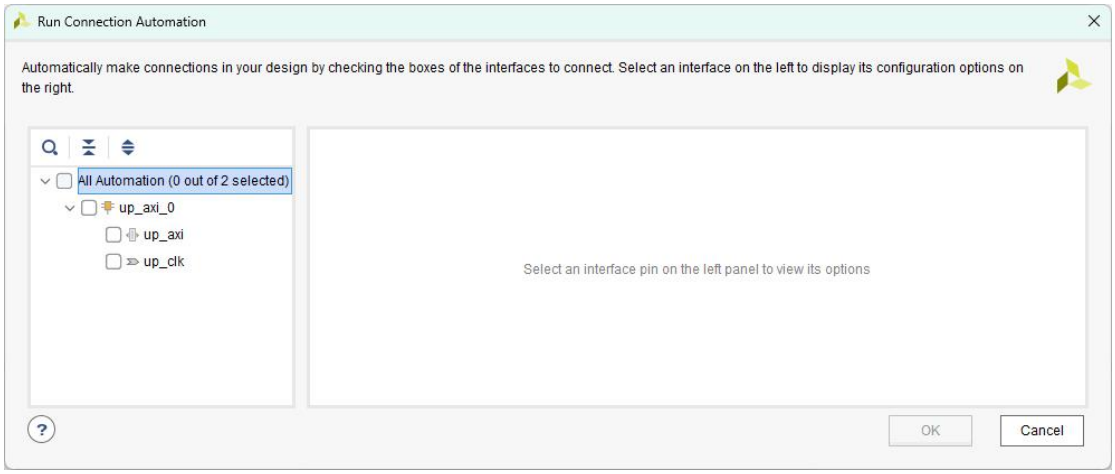
选择我们刚添加的 up_axi.v 模块



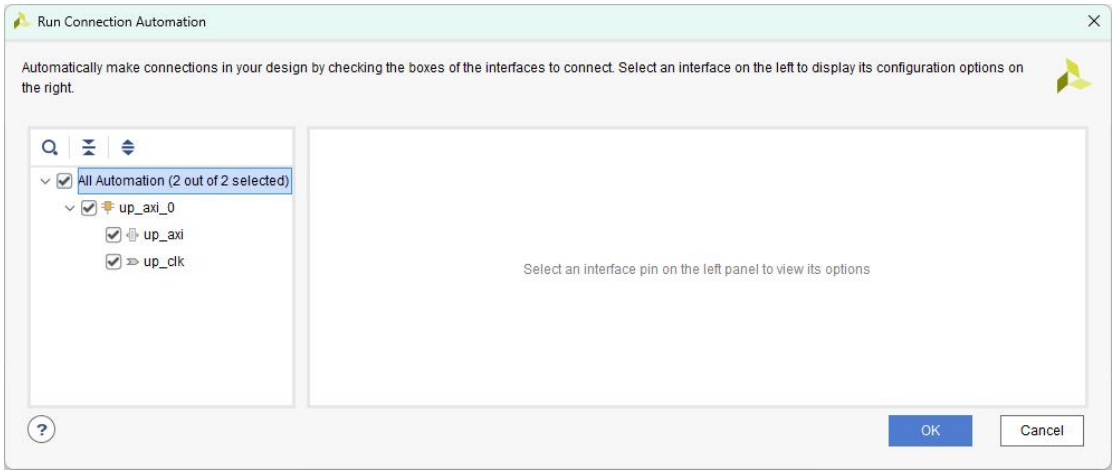
点击【OK】



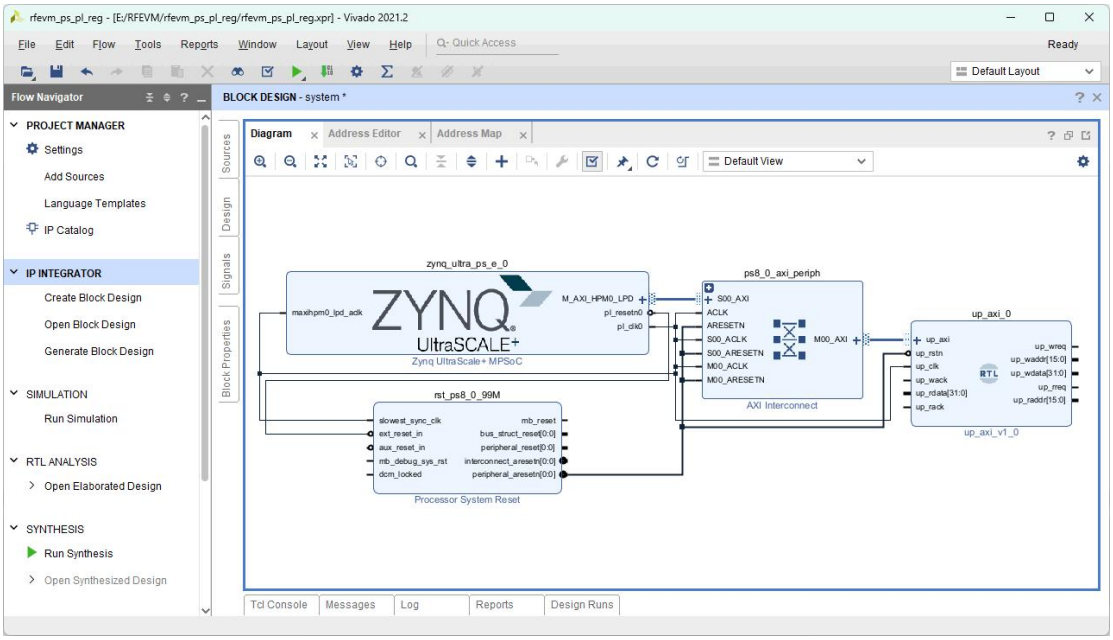
5) 我们选择自动连接【Run Connection Automation】



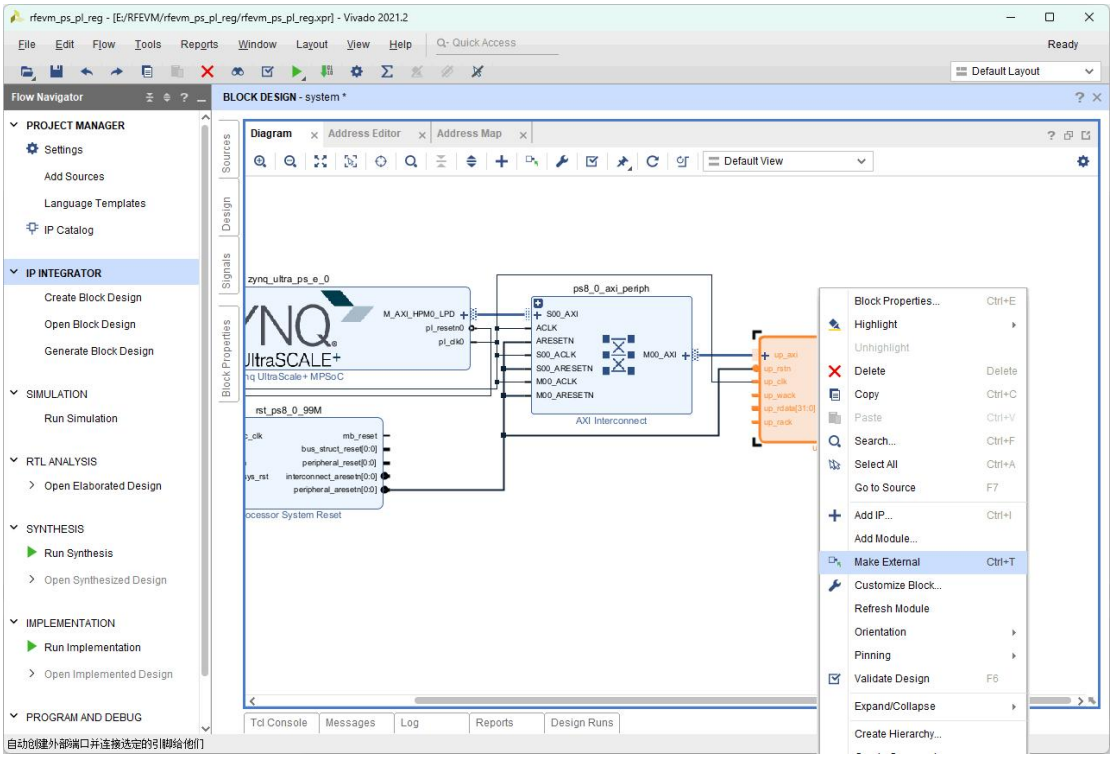
使能 up_axi，点击【OK】

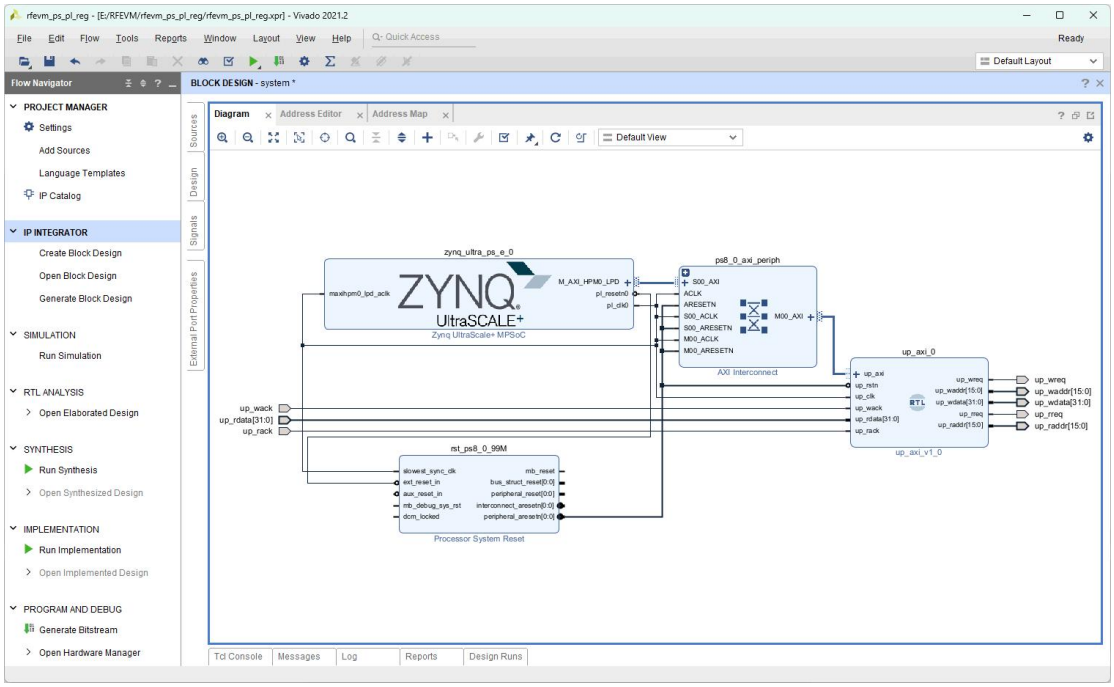


6) 我们可以看到生成的模块连接，这里会自动添加复位和互联模块；

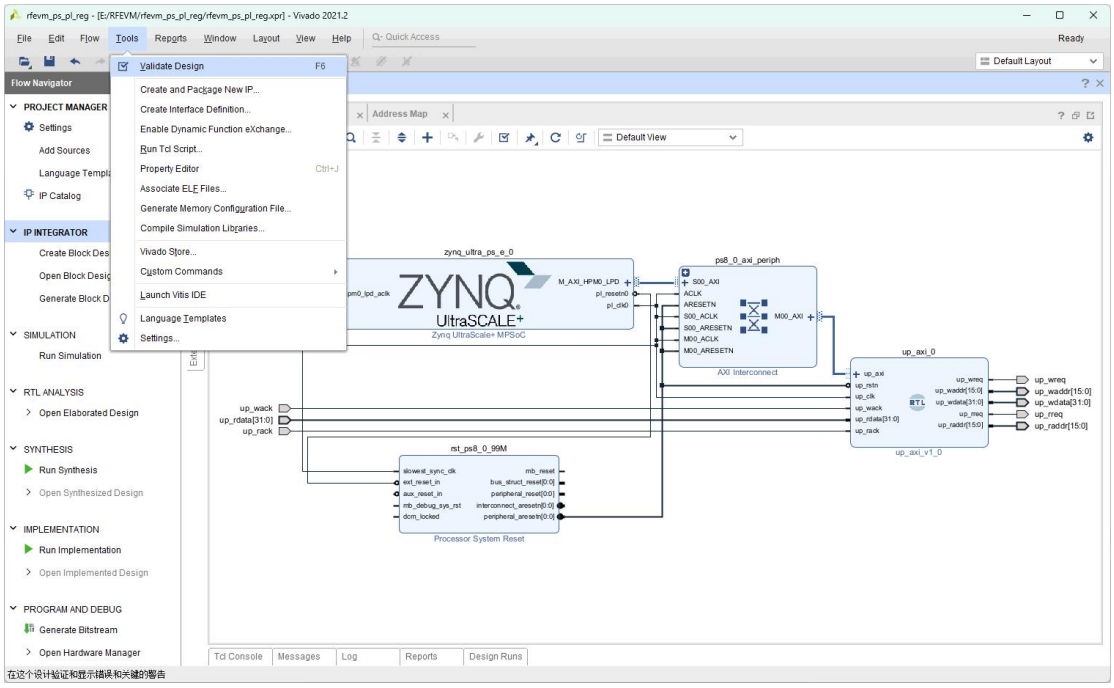


7) 我们引出 up_axi 的用户接口信号，在模块上右键选择【Make External】

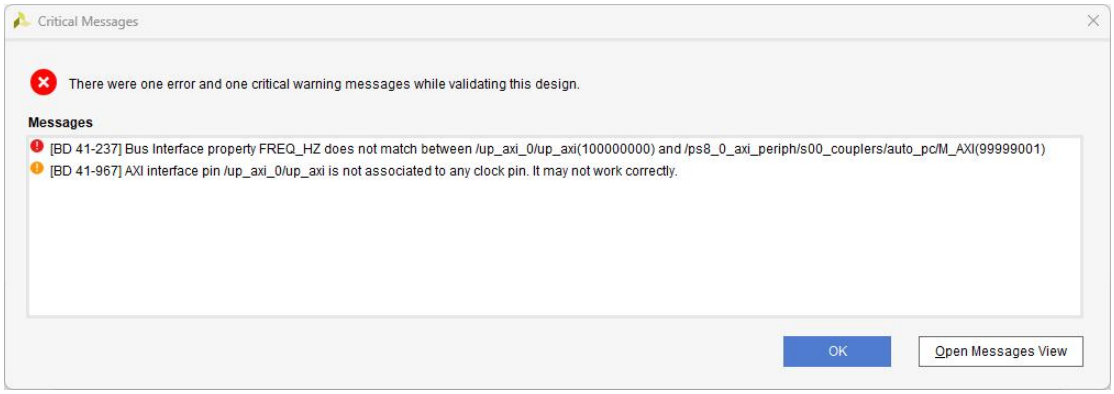




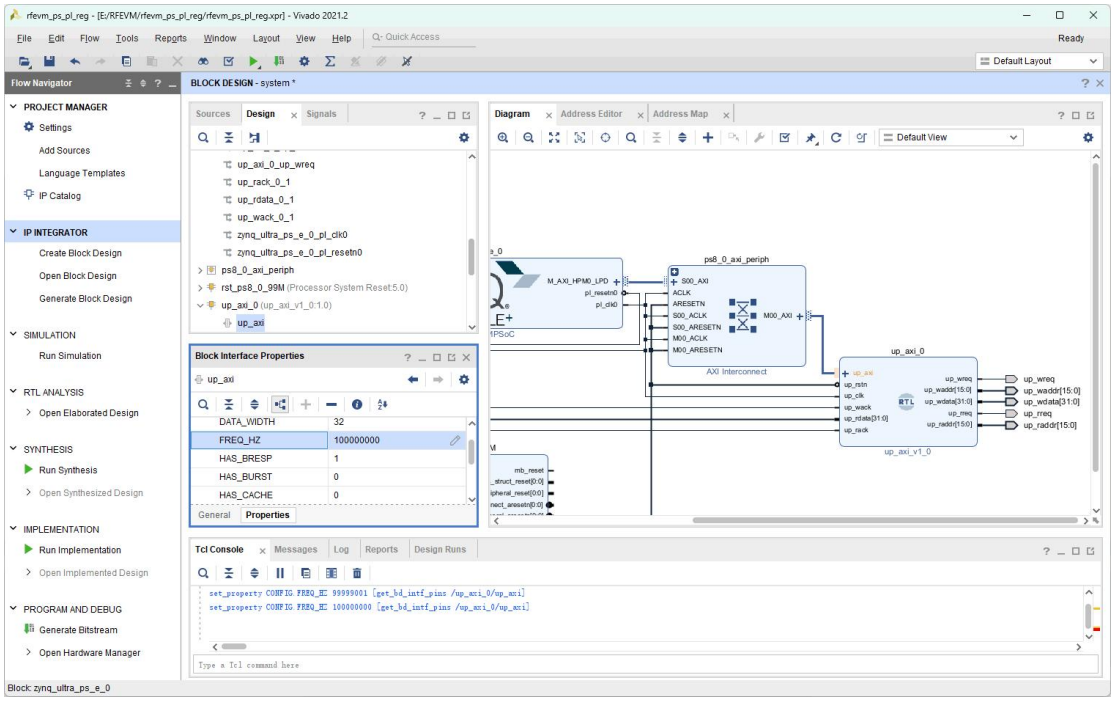
8) Validate 设计

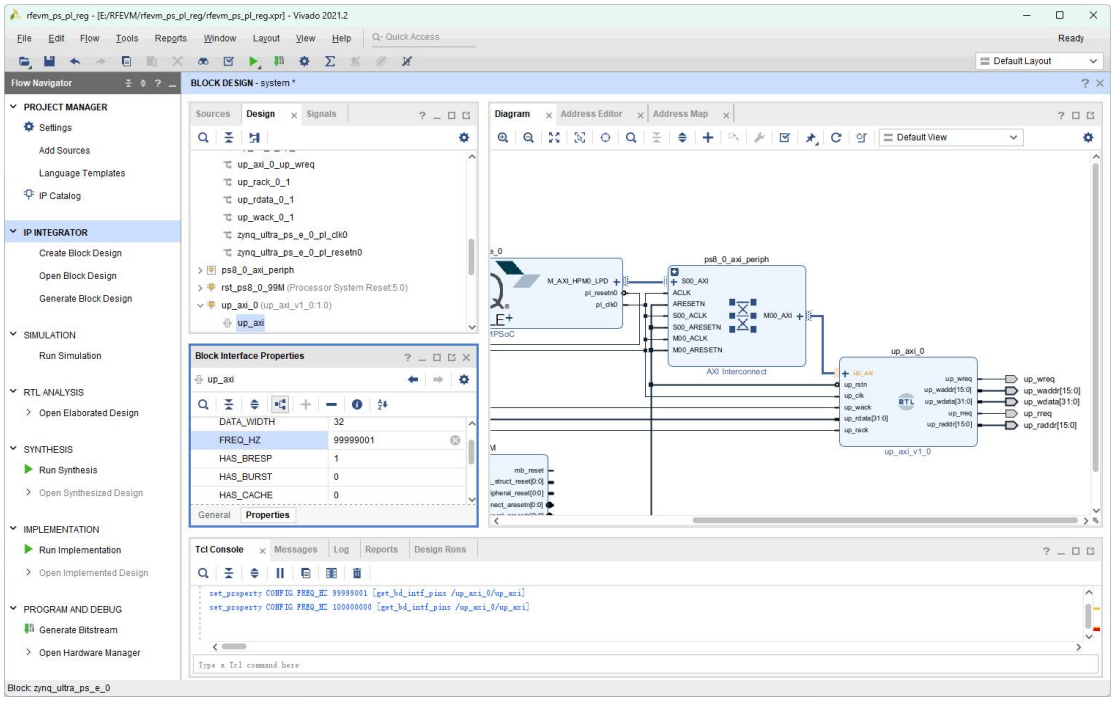


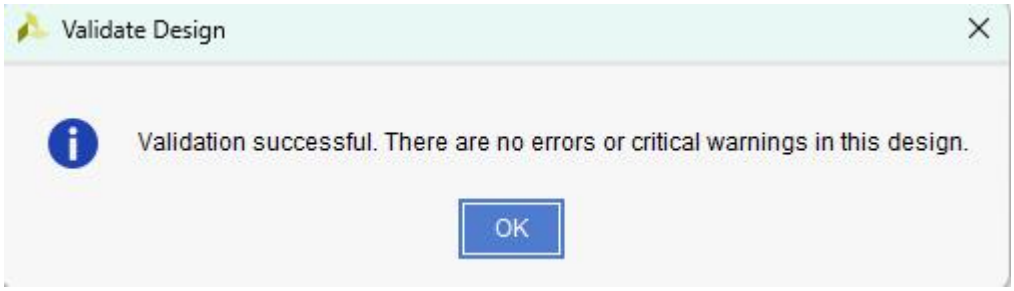
这里提示以下错误，提示时钟频率不一致



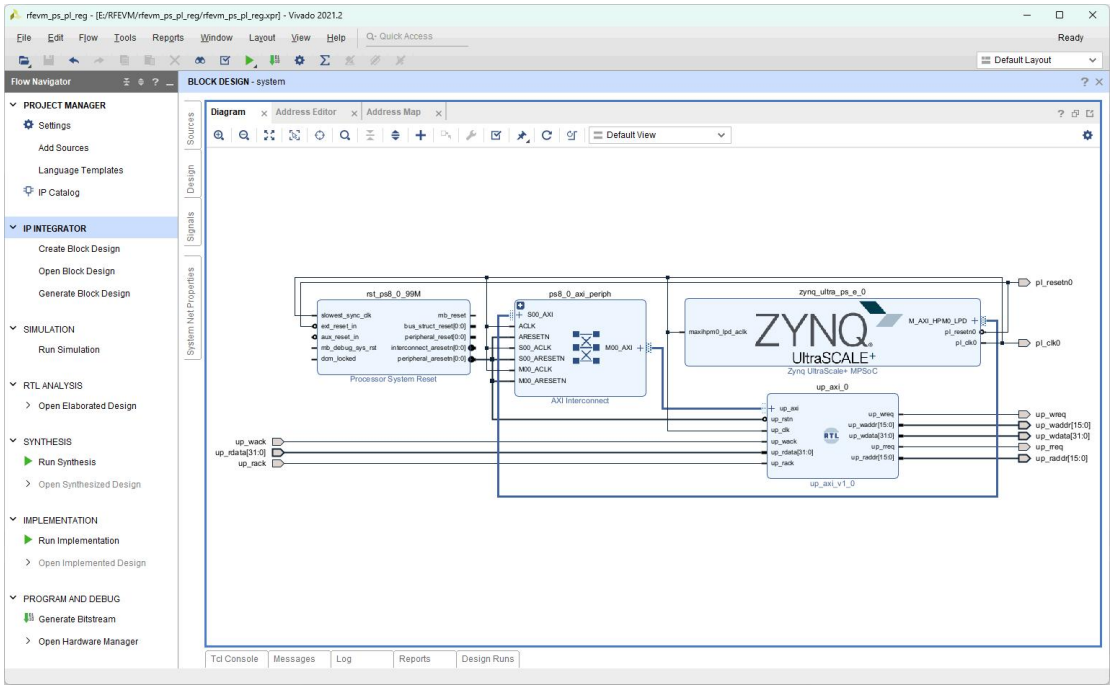
我们选中 up_axi，修改 up_axi 总线的频率，重新 validate 即可；







这里我们也引出时钟和复位信号；



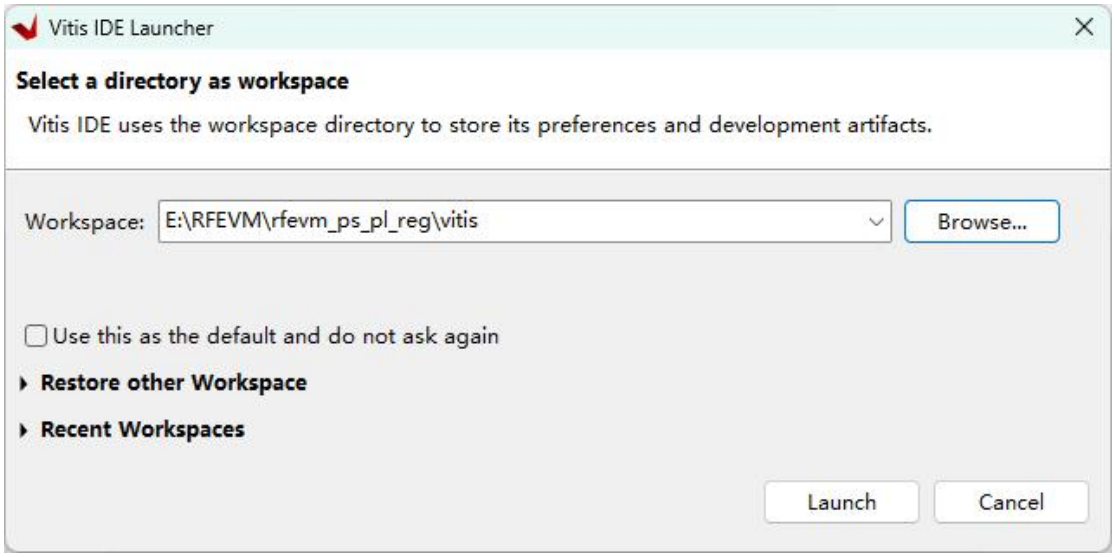
9) 我们新建顶层文件，例化 bd 文件，和寄存器读写模块；

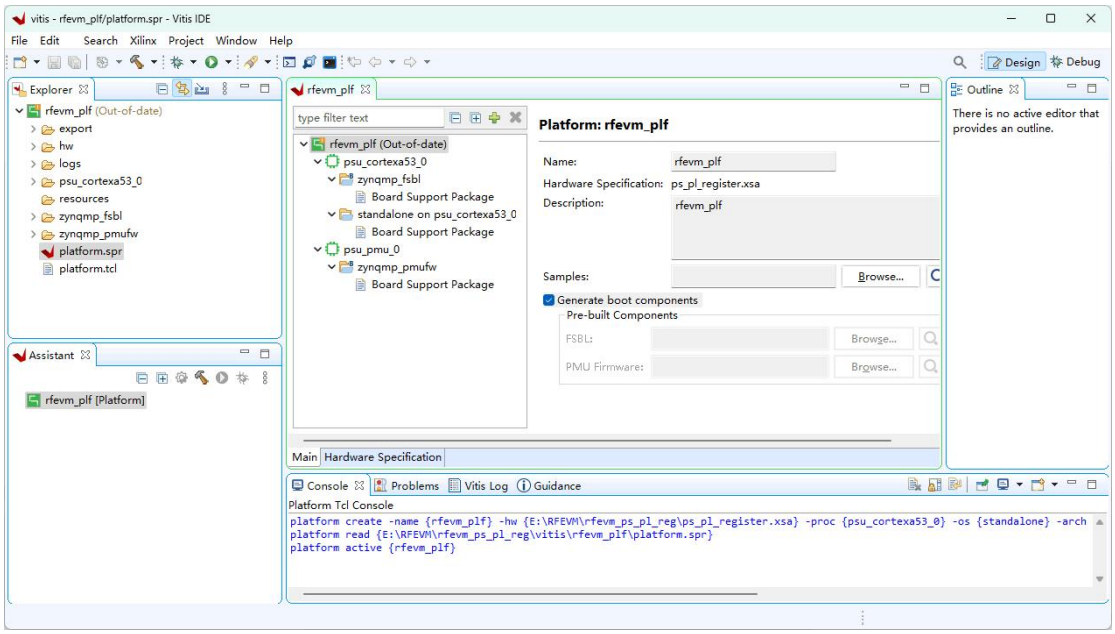
10) 生成 bit，导出硬件工程 xsa 文件；

5.3.2 软件工程搭建

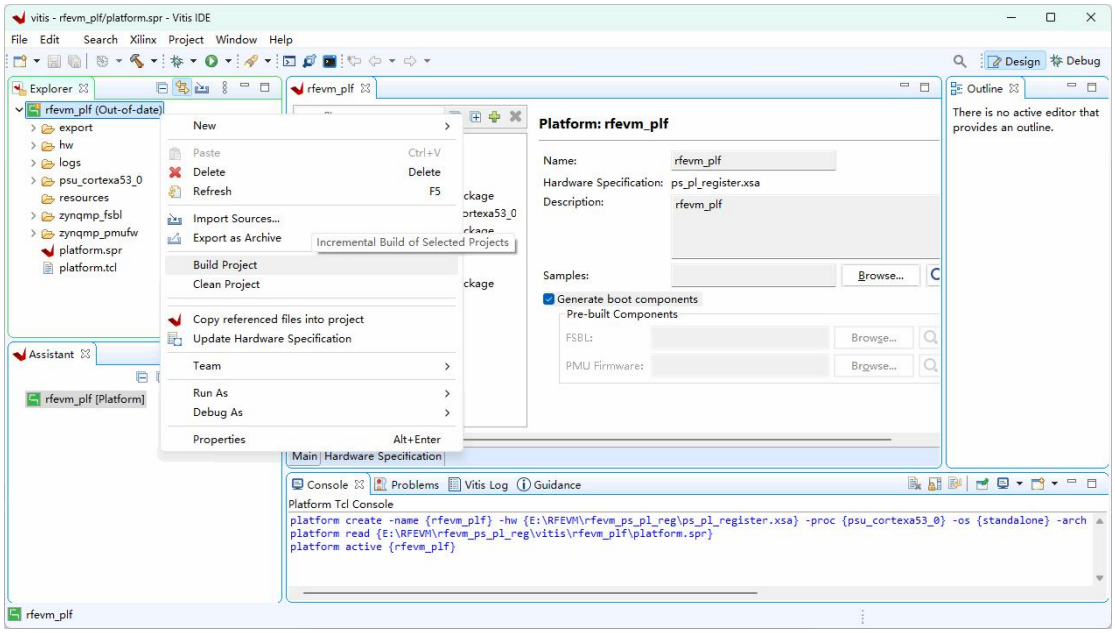
我们在 vivado 下搭建好硬件工程，导出 xsa 文件后，在工程目录下面创建 vitis 文件夹作为 vitis 的工作目录。

1) 登录 vitis，选择本工程路径下面的 vitis 文件夹作为工作目录，点击【Launch】

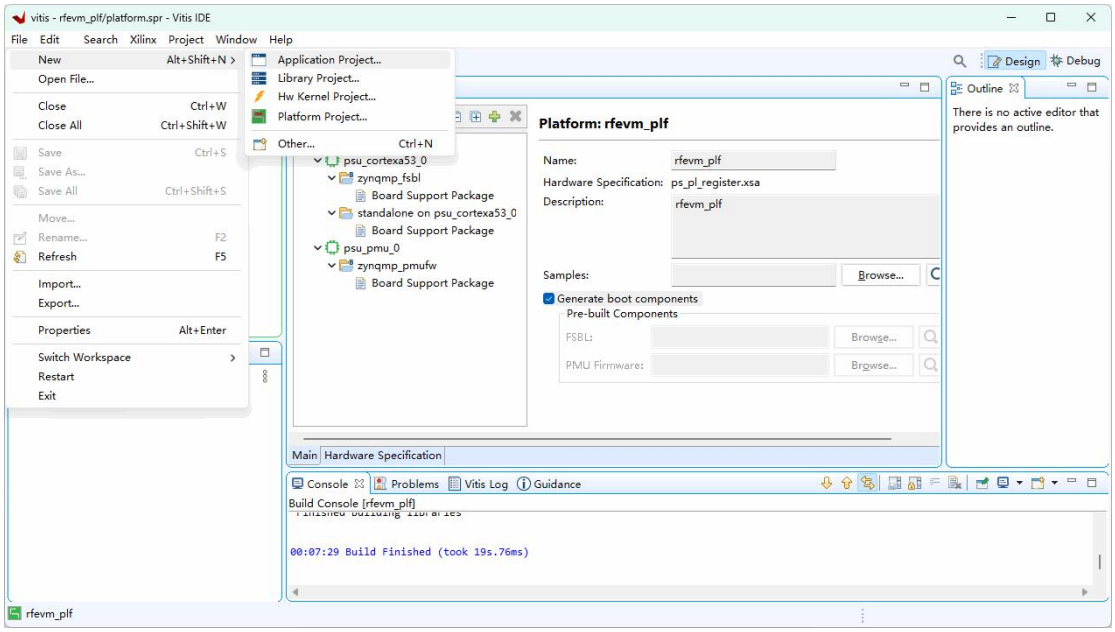




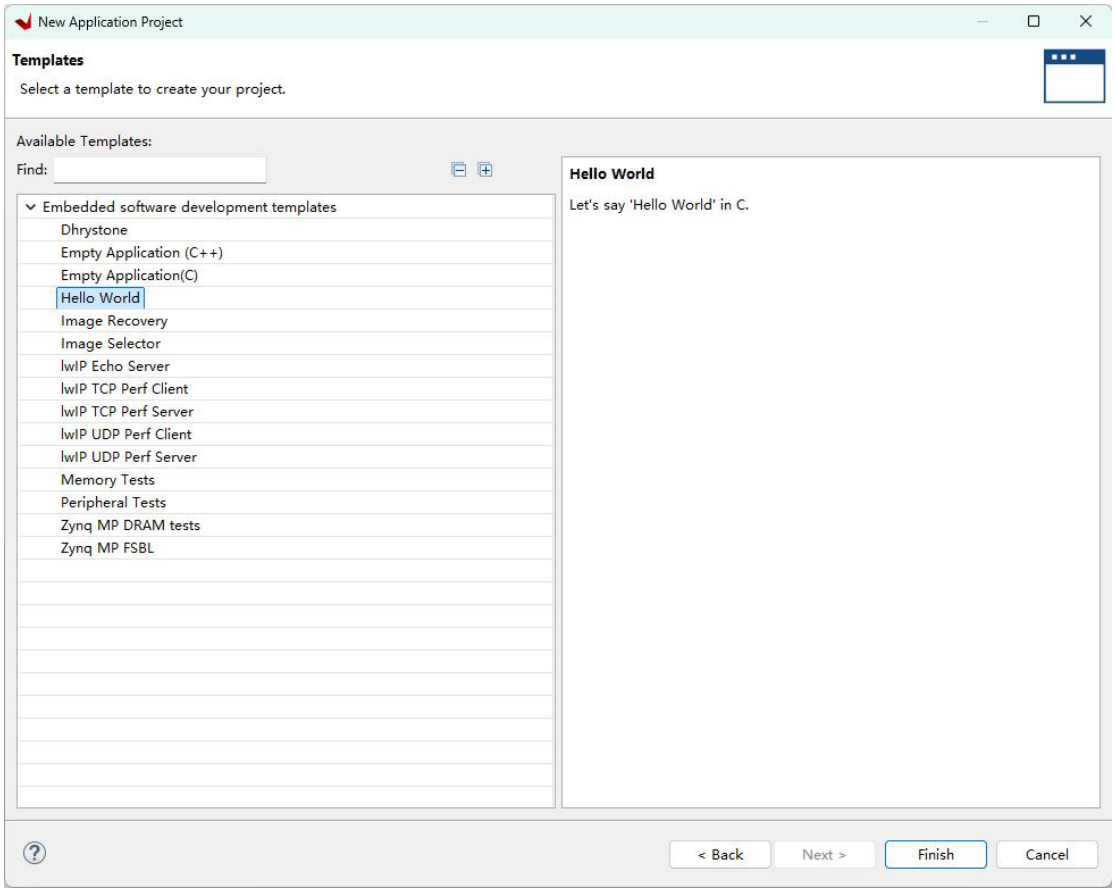
2) 编译硬件平台



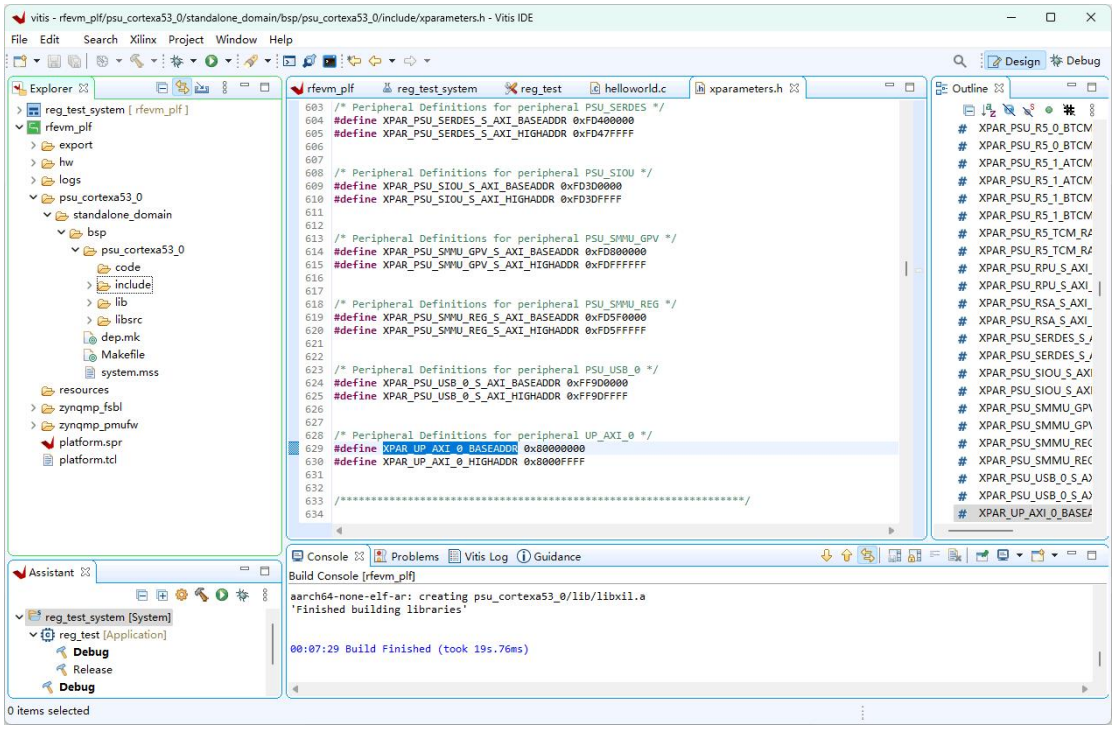
3) 新建应用程序



中间步骤省略，最后选择“Hello World”模板，点击【Finish】



这里我们在 xparameters.h 中查找 up_axi 模块的基地址；

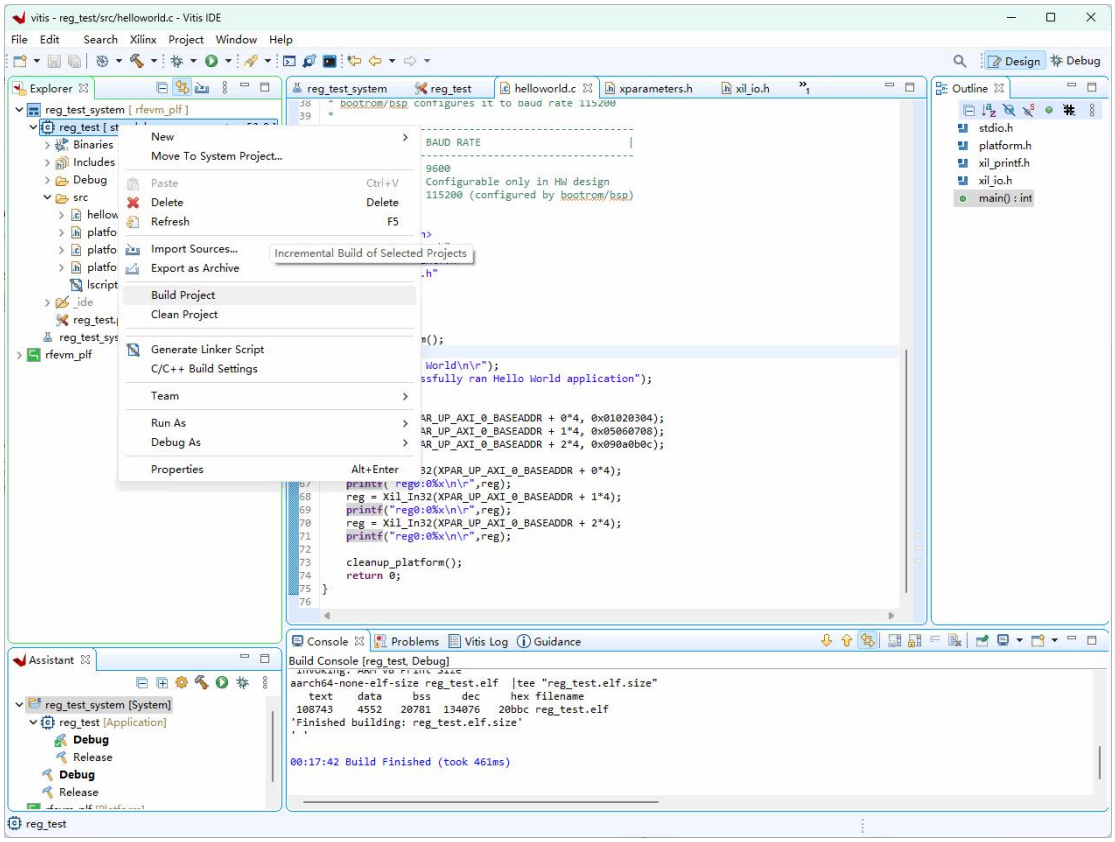


4) 修改 helloworld.c 文件，添加寄存器读写代码，我们在硬件工程里面使能了 3 个寄存器，这里我们对这三个寄存器进行读写测试；

reg_test_system | reg_test | helloworld.c | xparameters.h | xil_io.h

```
 * bootrom/bsp configures it to baud rate 115200
 *
 * ------------------------------------------------
 * | UART TYPE   BAUD RATE                        |
 * ------------------------------------------------
 *   uartns550   9600
 *   uartlite    Configurable only in HW design
 *   ps7_uart    115200 (configured by bootrom/bsp)
 */

#include <stdio.h>
#include "platform.h"
#include "xil_printf.h"
#include "xil_io.h"


int main()
{
    init_platform();

    print("Hello World\n\r");
    print("Successfully ran Hello World application");
    u32 reg;

    Xil_Out32(XPAR_UP_AXI_0_BASEADDR + 0*4, 0x01020304);
    Xil_Out32(XPAR_UP_AXI_0_BASEADDR + 1*4, 0x05060708);
    Xil_Out32(XPAR_UP_AXI_0_BASEADDR + 2*4, 0x090a0b0c);

    reg = Xil_In32(XPAR_UP_AXI_0_BASEADDR + 0*4);
    printf("reg0:0%x\n\r",reg);
    reg = Xil_In32(XPAR_UP_AXI_0_BASEADDR + 1*4);
    printf("reg0:0%x\n\r",reg);
    reg = Xil_In32(XPAR_UP_AXI_0_BASEADDR + 2*4);
    printf("reg0:0%x\n\r",reg);

    cleanup_platform();
    return 0;
}

```

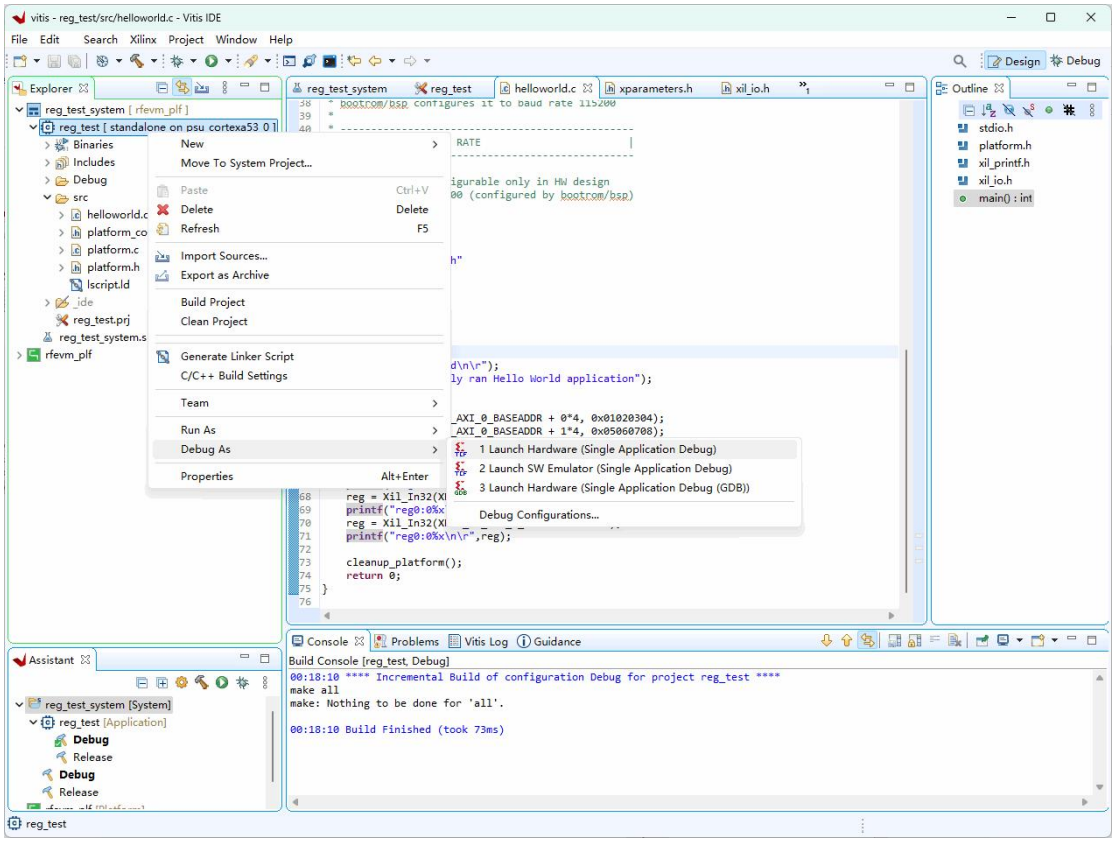
5) 编译应用程序



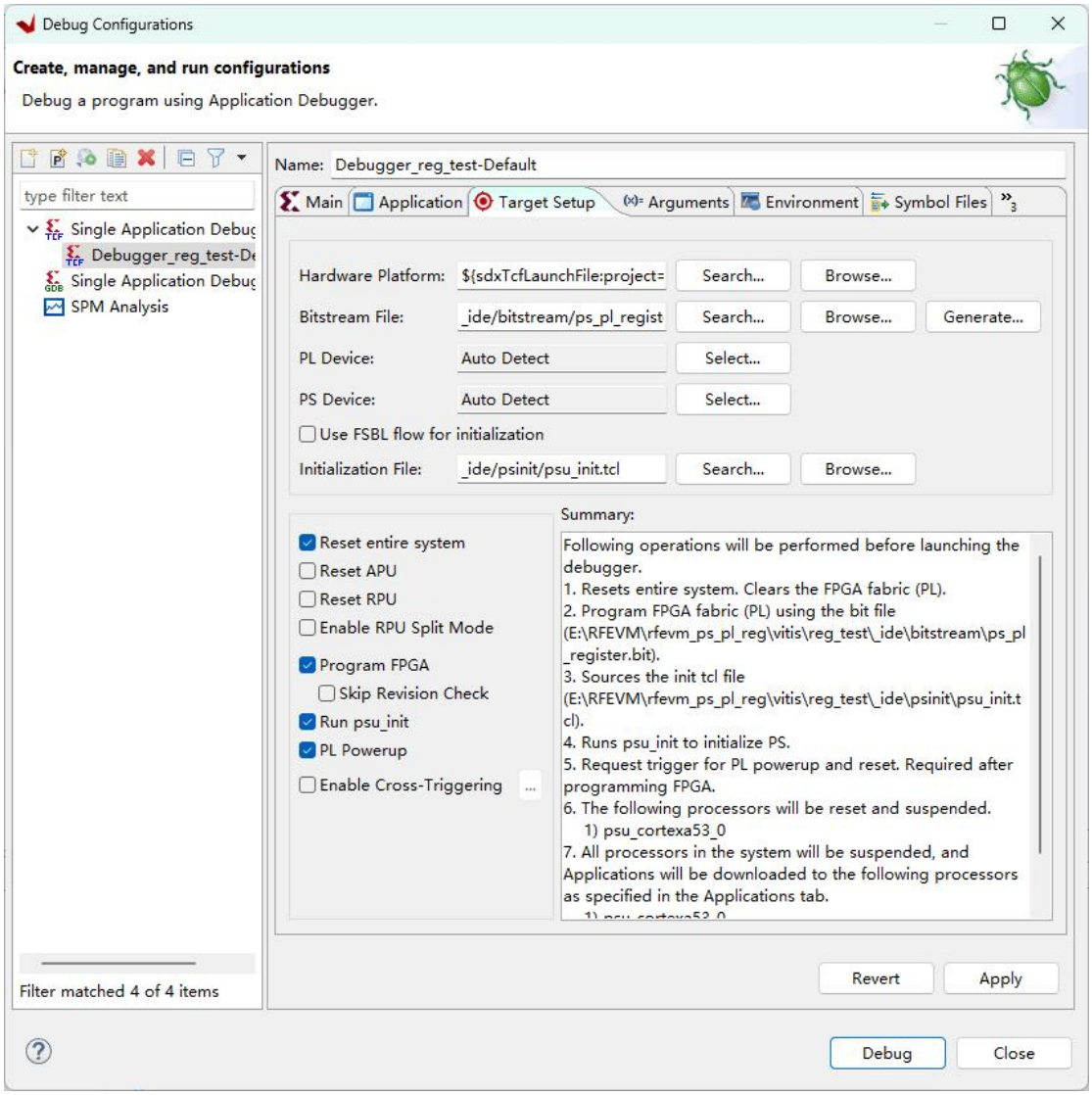
5.3.3 下载验证

我们在 vitis 里面 debug，应用程序。

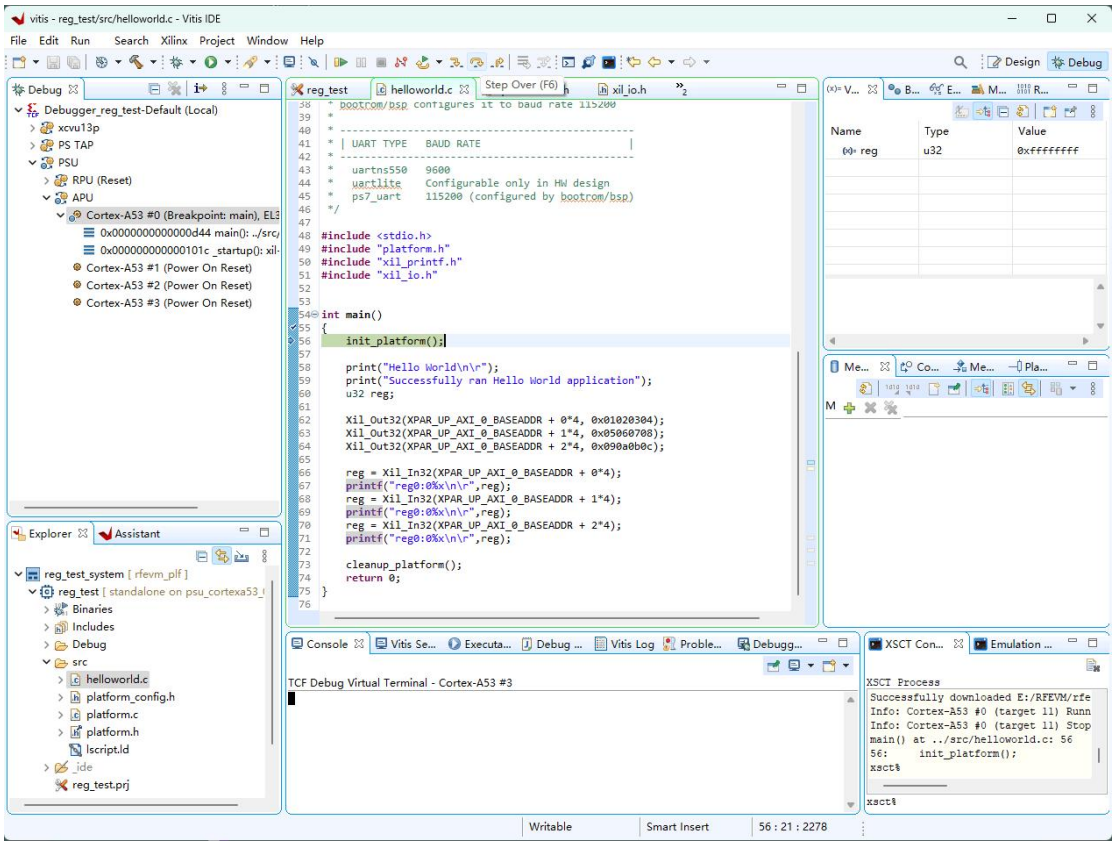
1) 在应用程序上面右键，选择【Debug As】->【Debug configurations...】



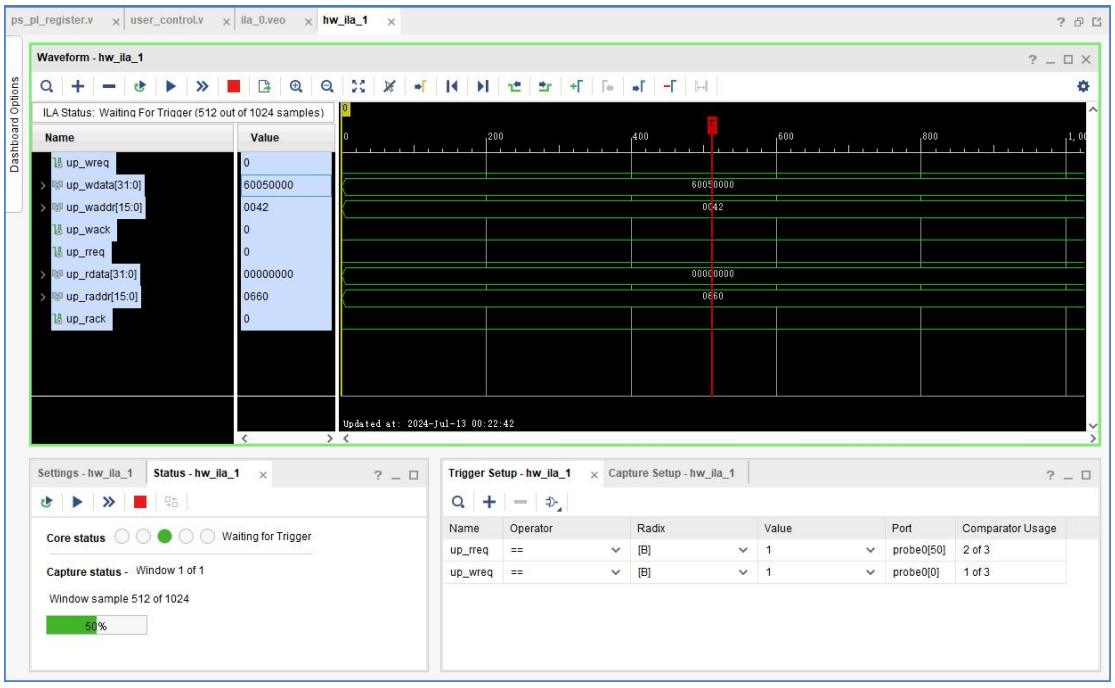
2) 这里我们取消掉“Use FSBL flow for initialization”，点击【Apply】，然后点击【Debug】，下载程序；



3) 这里我们 Step Over 单步运行程序

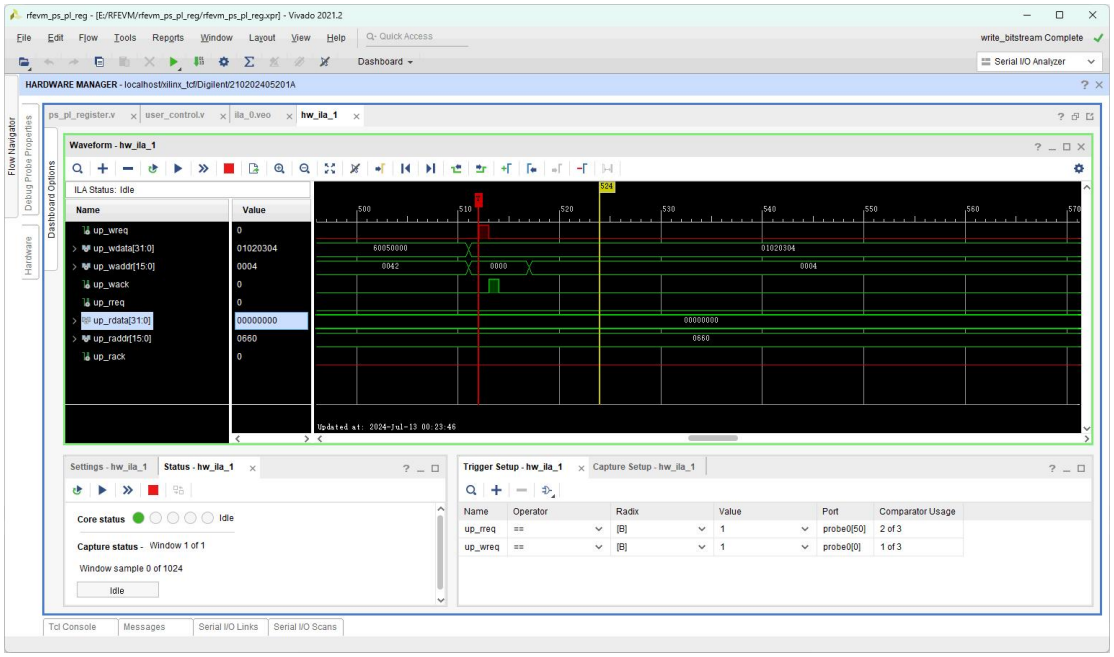


同时打开 chipscope，观测寄存器的读写信号，我们设置读写请求为触发信号（两个条件满足一个就触发）

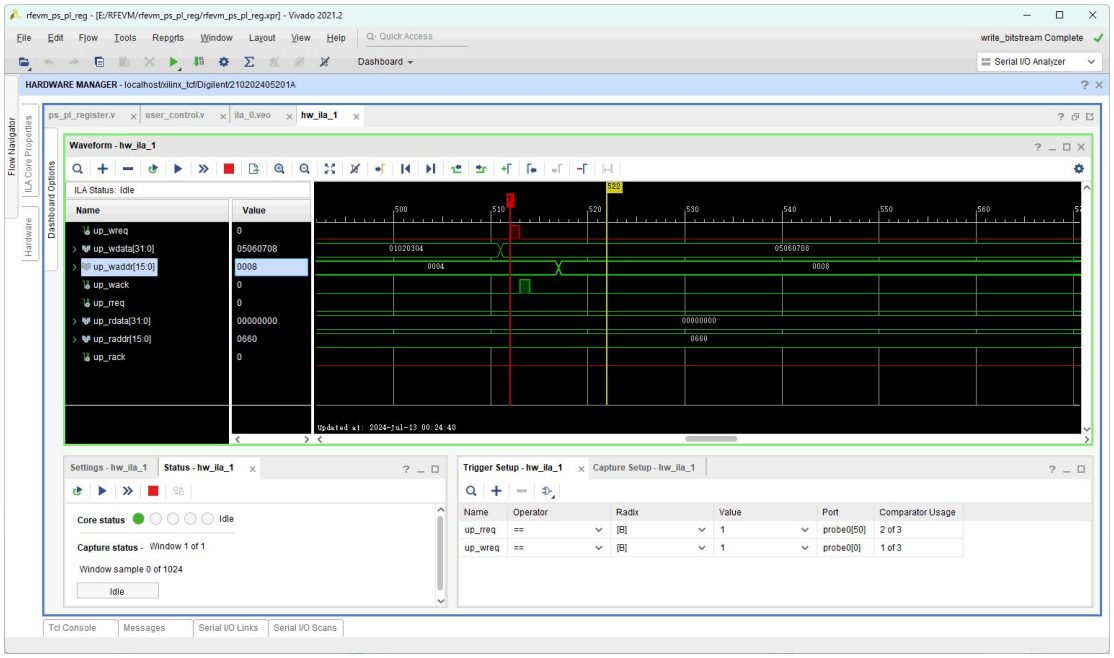


4) 我们在 vitis 里面执行代码，可以看到 ila 里面信号变化

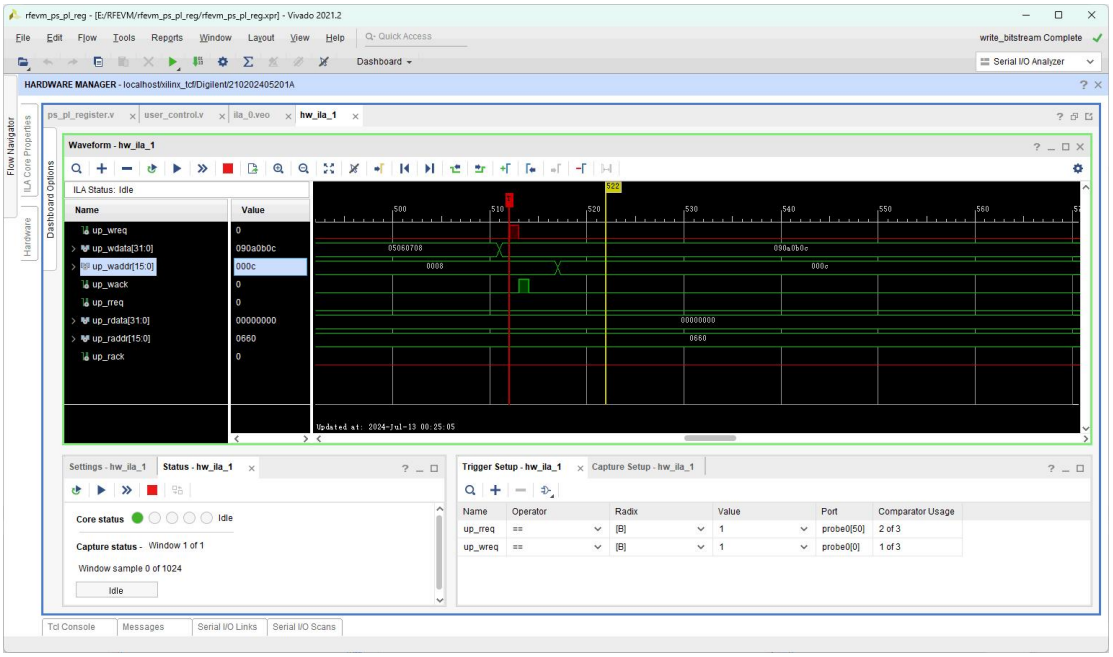
往寄存器偏移地址 0 里面写 0x01020304



往寄存器偏移地址 4 里面写 0x05060708



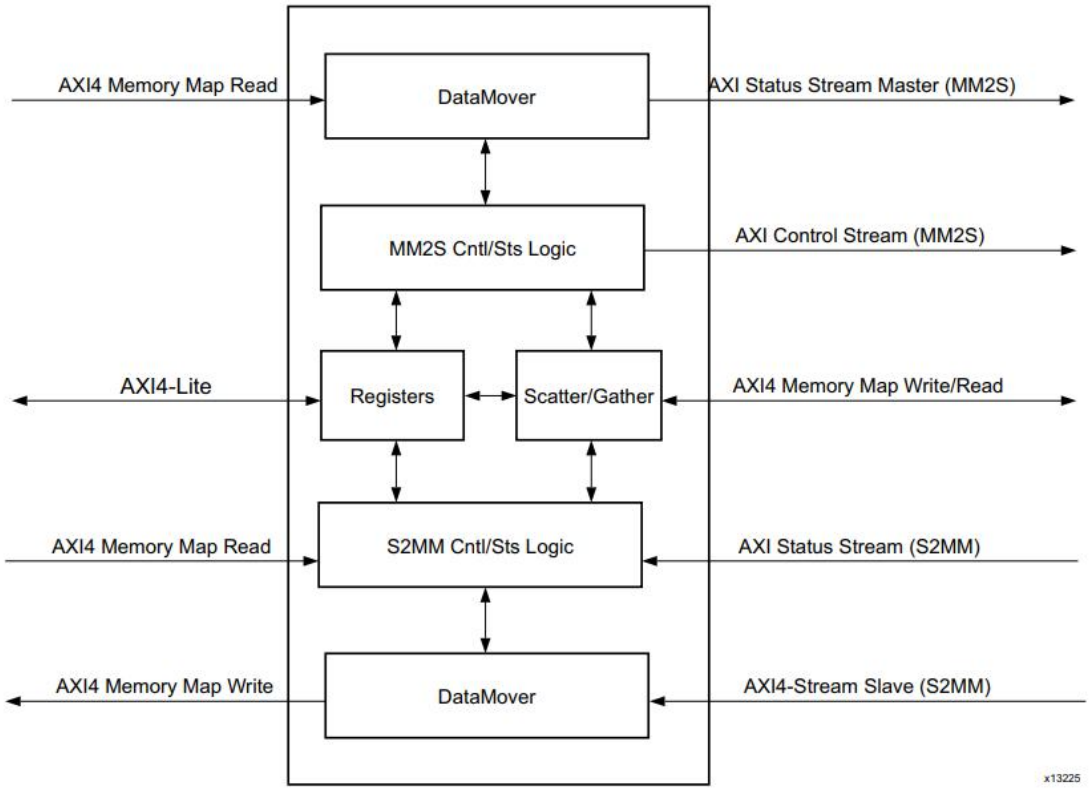
往寄存器偏移地址 8 里面写 0x090a0b0c



5.4 AXI-DMA 环回测试

AXI DMA 提供系统内存和基于总线 AXI4-Stream 目标外设之间的高带宽直接内存访问。

在系统中，AXI DMA 与处理器等其它设备通过 AXI Interconnect 互联。系统处理器通过 AXI4-Lite 接口访问 DMA 内部寄存器，向其写入配置信息，然后根据配置信息，高效的进行数据传输；DMA 也可以启动 S/G 传输模式，通过独立的 AXI S/G 通道从 DDR 中获取描述符，然后协调外设与内存间的数据传输。



AXI DMA 用到了三种总线：

- AXI-Lite 用于对寄存器进行配置
- AXI4 Memory Map 用于与内存交互，又分为 AXI4 Memory Map Read 和 AXI4 Memory Map Write 两个接口，一个是读，一个是写。
- AXI4 Stream 接口用于对外设的读写，其中 AXI4 Stream Master（MM2S）用于对外设写，AXI4 Stream Salve（S2MM）用于对外设读。

AXI DMA 提供了 3 中模式，分别是 Direct Register 模式、Scatter/Gather 模式和 Cyclic DMA 模式，这里我们的介绍下常用的 Direct Register 模式和 Scatter/Gather 模式。Direct Register DMA 模式也就是 Simple DMA。Direct Register 模式提供了一种配置，用于在 MM2S 和 S2MM 通道上执行简单的 DMA 传输，这需要更少的 FPGA 资源。Simple DMA 允许应用程序在 DMA 和 Device 之间定义单个事务。它有两个通道：一个从 DMA 到 Device，另一个从 Device 到 DMA。应用程序必须设置缓冲区地址和长度字段以启动相应通道中的传输。Scatter/Gather DMA 模式允许在单个 DMA 事务中将数据传输到多个存储区域或从多个存储区域传输数据。它相当于将多个 Simple DMA 请求链接在

一起。SGDMA 允许应用程序在内存中定义事务列表，硬件将在应用程序没有进一步干预的情况下处理这些事务。在此期间，应用程序可以继续添加更多工作以保持硬件工作。用户可以通过轮询或中断来检查事务是否完成。SGDMA 处理整个数据包（被定义为表示消息的一系列数据字节）并允许将数据包分解为一个或多个事务。

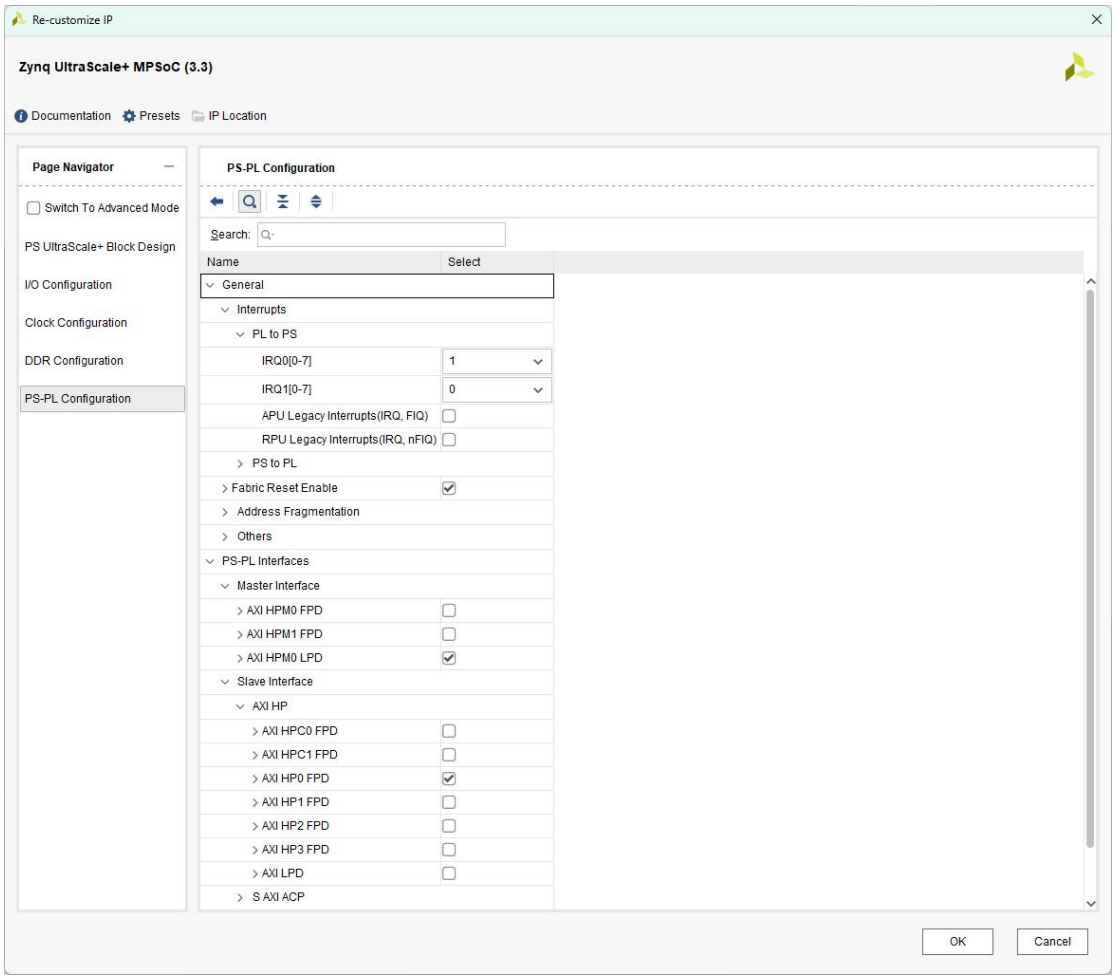
本章中使用直接寄存器 Direct Register 模式，采用 Vitis 给的 DMA 例子，稍作修改，在 DMA 工作结束后，发出结束中断，通知 CPU，使 CPU 读取内存数据。

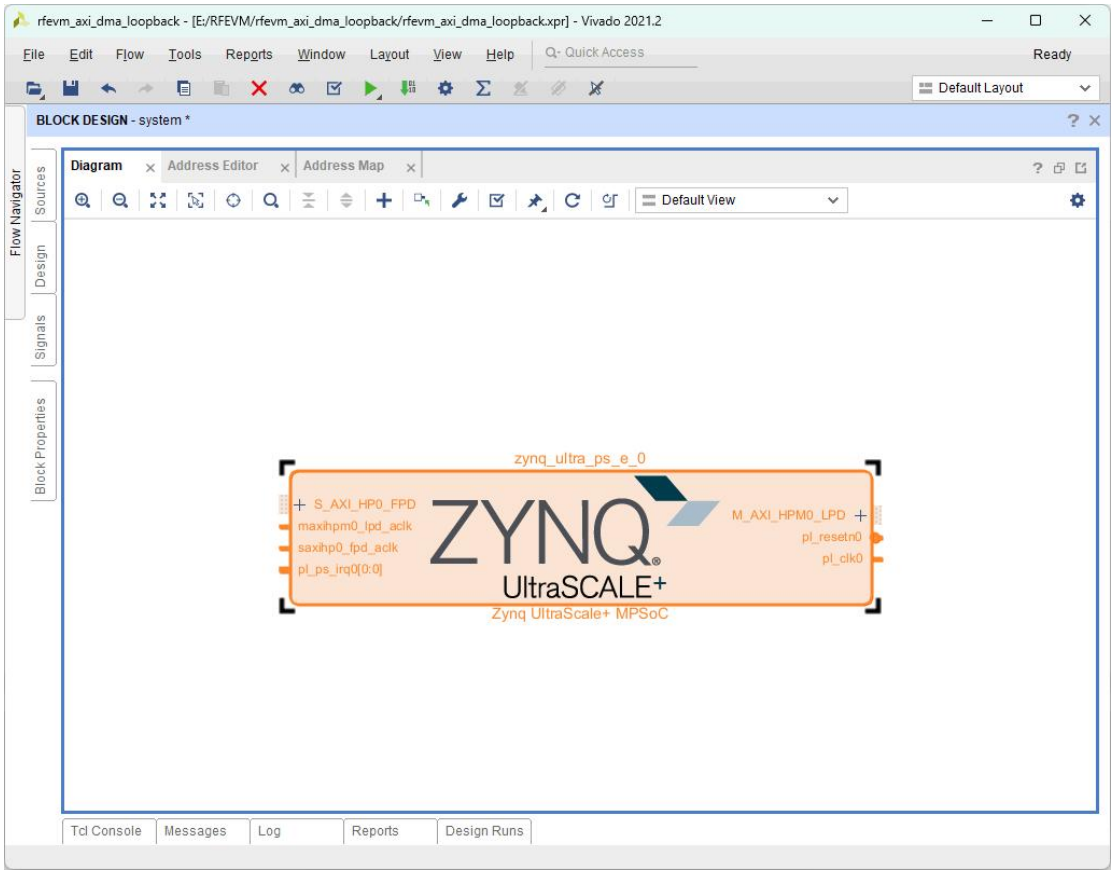
5.4.1 硬件工程搭建

这里我们依然在上篇 IPI 的工程基础上，去搭建硬件工程。

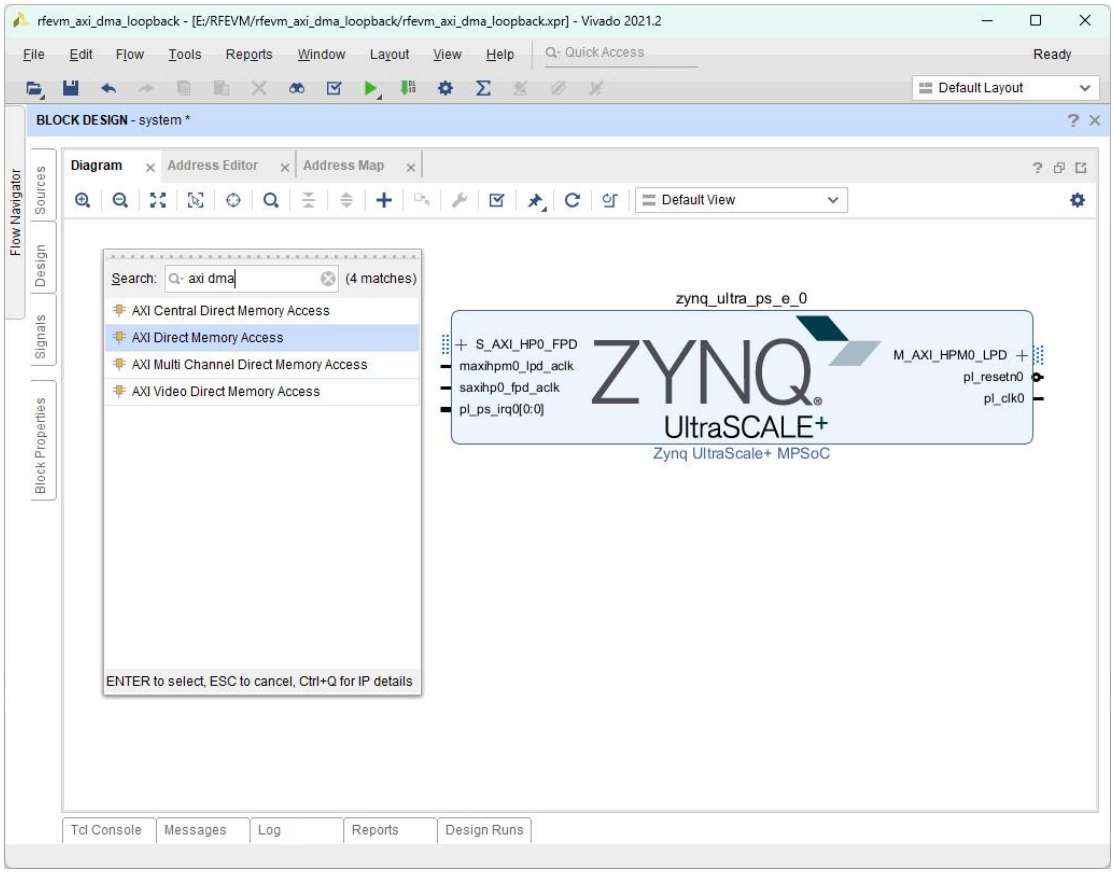
1) 我们按照之前 IPI 的工程去搭建本实验的硬件工程；

2) 双击 mpsoc，使能 HPM0、HP0、PL to PS 中断，点击【OK】

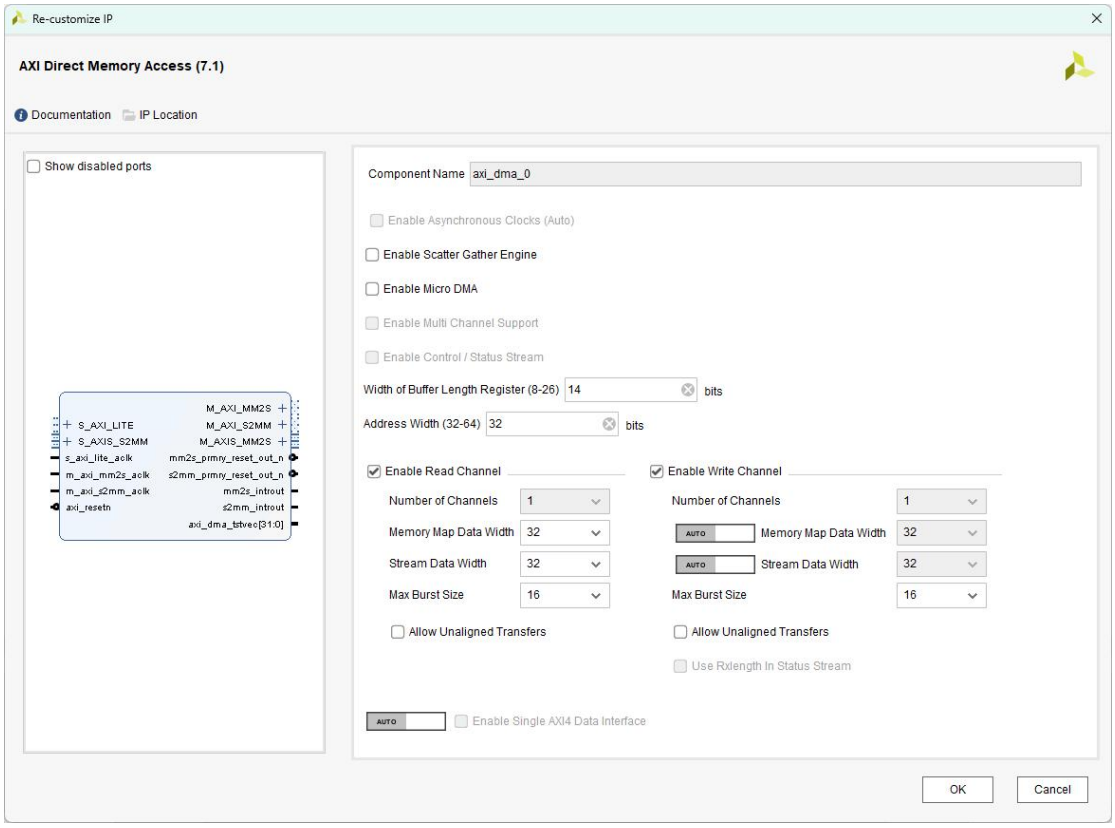




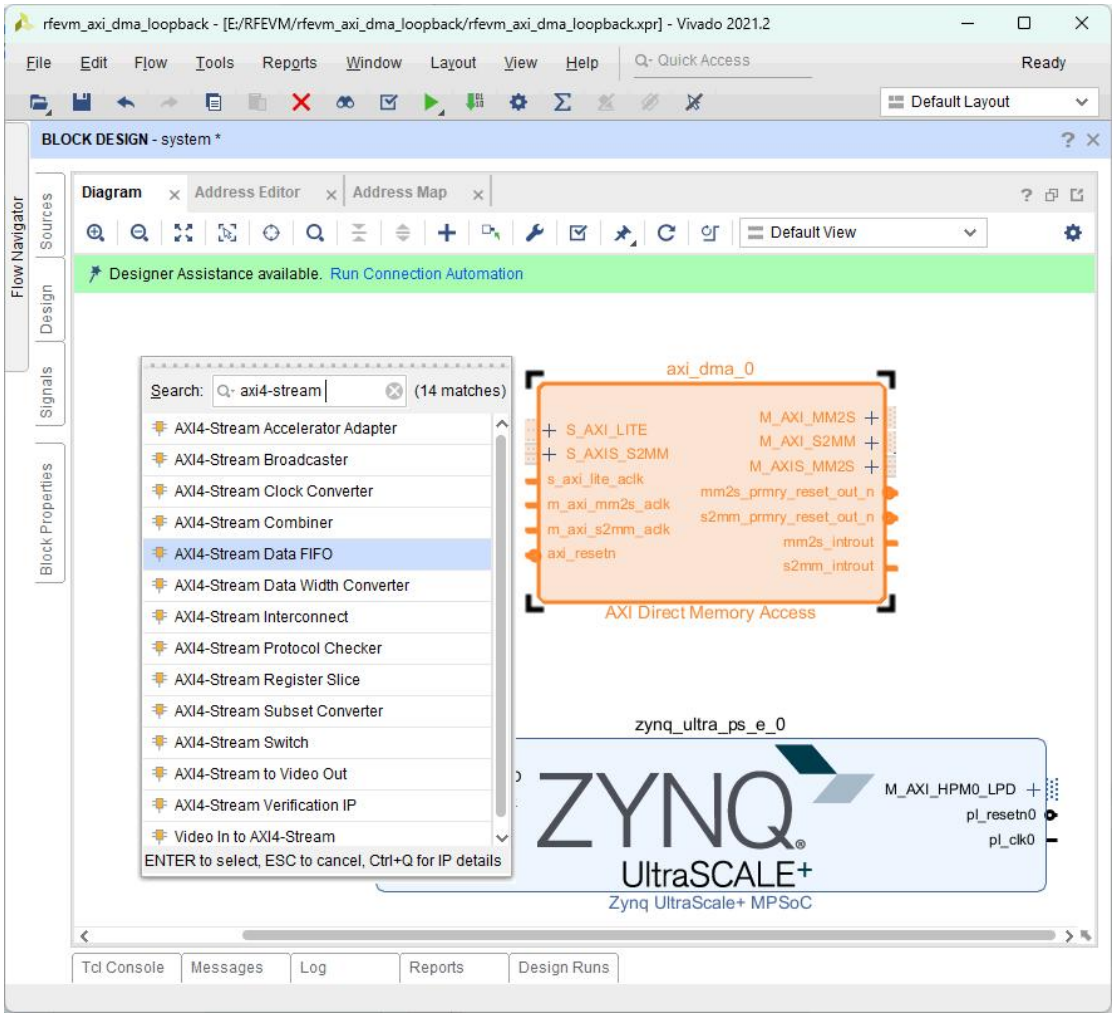
3) 点击“+”，添加 AXI DMA 模块；



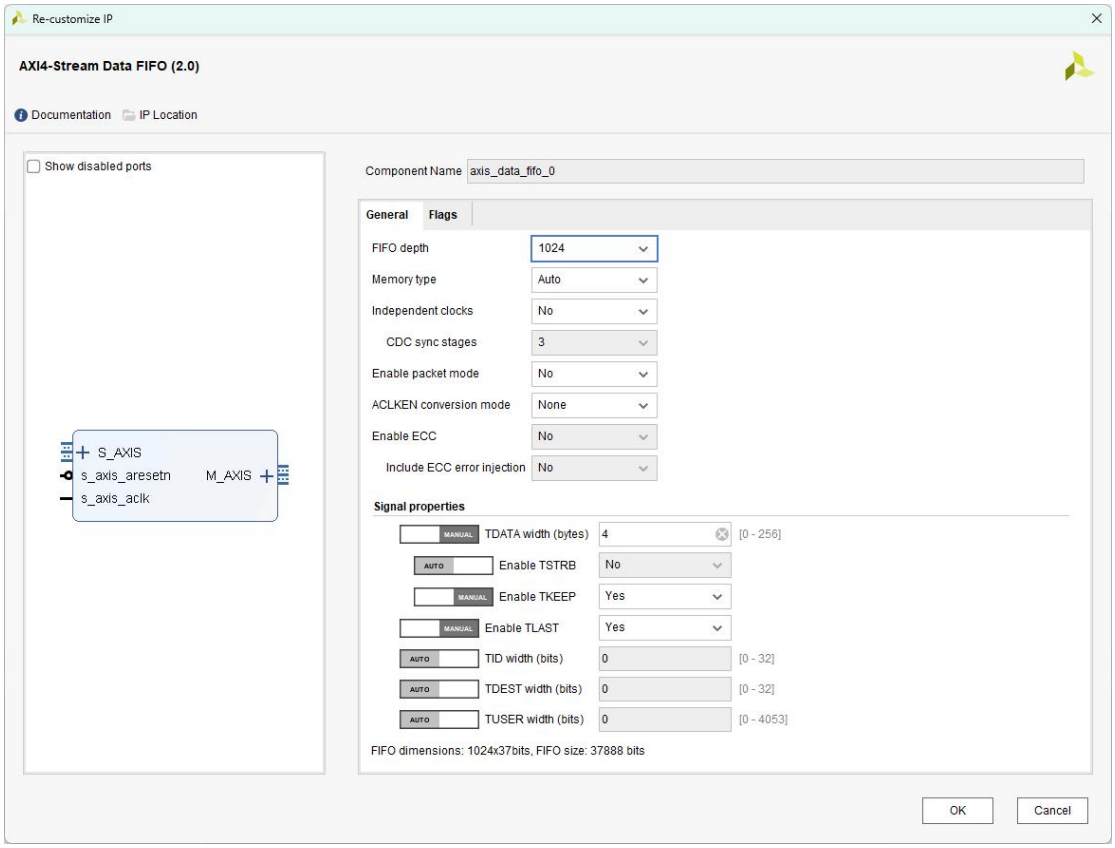
4) 设置 axi dma IP 参数

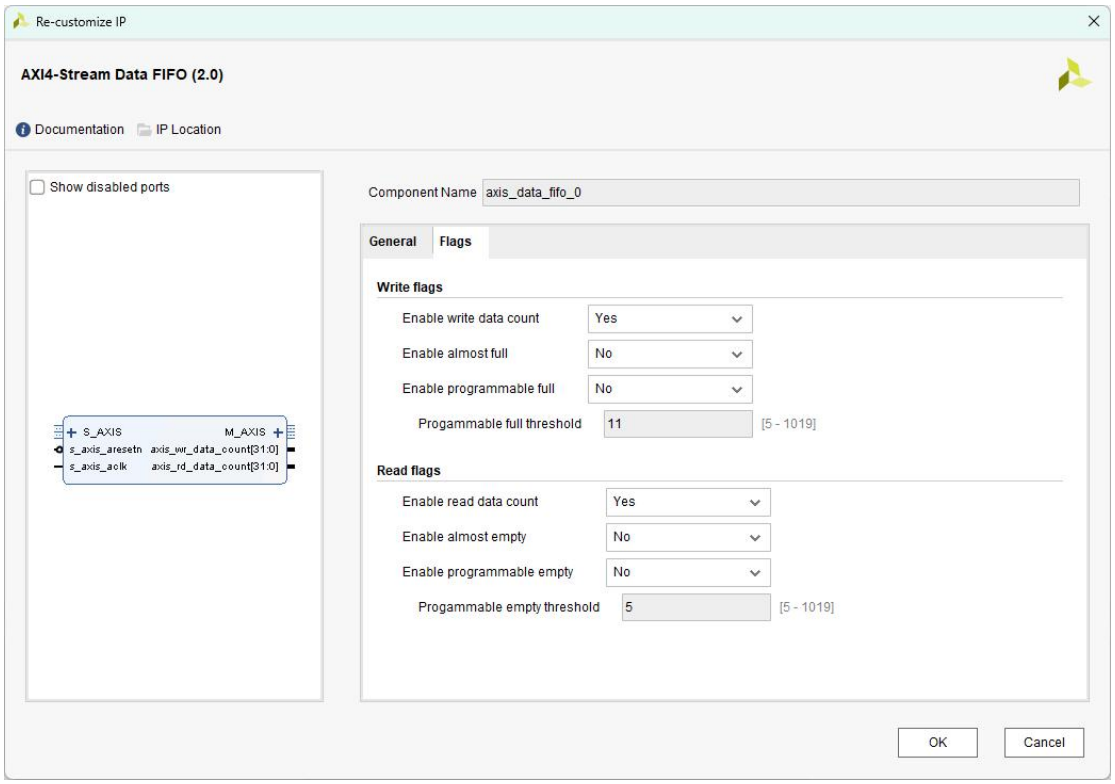


5) 添加 AXI Stream Data FIFO 模块；

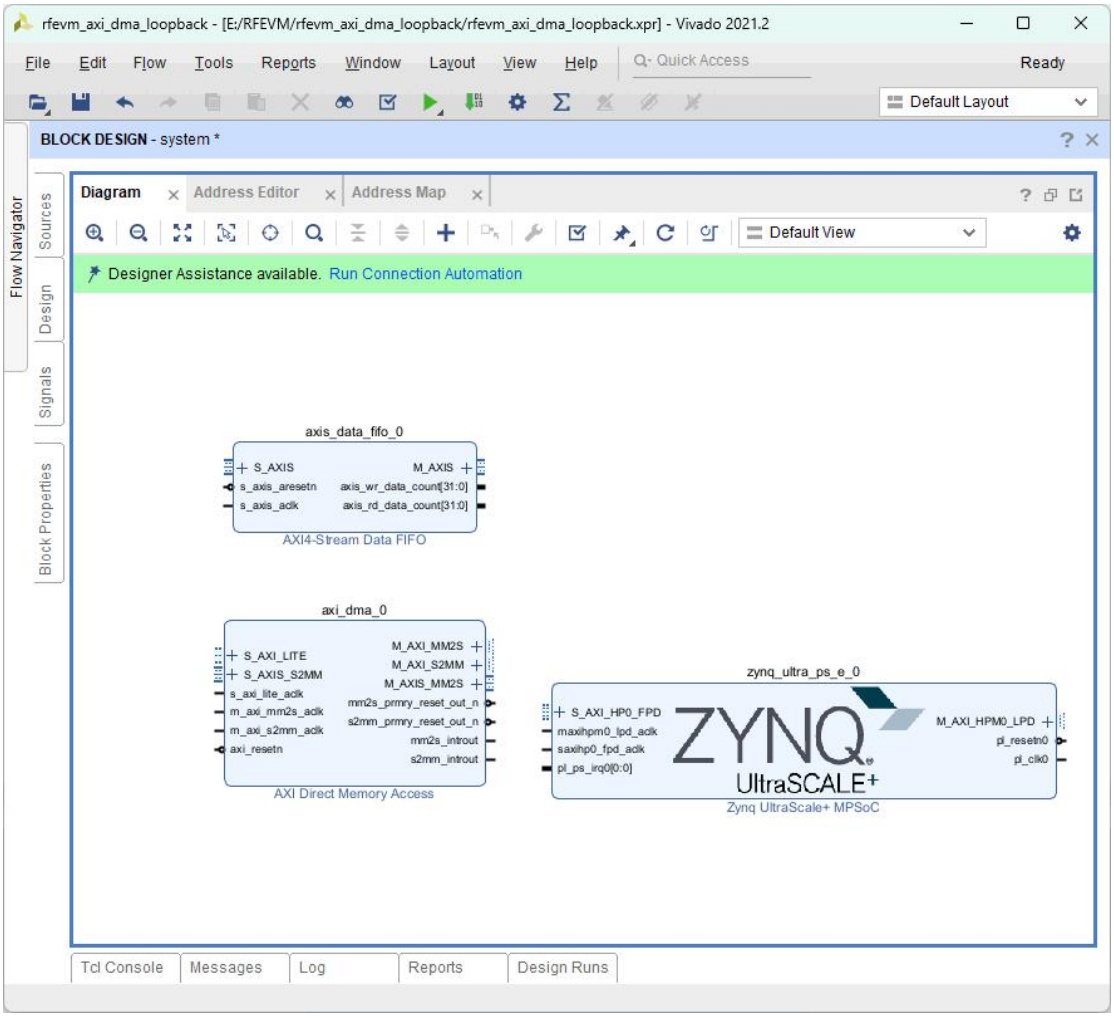

rfevm_axi_dma_loopback - [E:/RFEVM/rfevm_axi_dma_loopback/rfevm_axi_dma_loopback.xpr] - Vivado 2021.2
File Edit Flow Tools Reports Window Layout View Help
Quick Access
Ready
Default Layout
BLOCK DESIGN - system *
Flow Navigator
Sources
Design
Signals
Block Properties
Diagram
Address Editor
Address Map
Default View
Designer Assistance available. Run Connection Automation
Search: axi4-stream (14 matches)
AXI4-Stream Accelerator Adapter
AXI4-Stream Broadcaster
AXI4-Stream Clock Converter
AXI4-Stream Combiner
AXI4-Stream Data FIFO
AXI4-Stream Data Width Converter
AXI4-Stream Interconnect
AXI4-Stream Protocol Checker
AXI4-Stream Register Slice
AXI4-Stream Subset Converter
AXI4-Stream Switch
AXI4-Stream to Video Out
AXI4-Stream Verification IP
Video In to AXI4-Stream
ENTER to select, ESC to cancel, Ctrl+Q for IP details
axi_dma_0
S_AXI_LITE
S_AXIS_S2MM
s_axi_lite_aclk
m_axi_mm2s_aclk
m_axi_s2mm_aclk
axi_resetn
M_AXI_MM2S
M_AXI_S2MM
M_AXIS_MM2S
mm2s_prmry_reset_out_n
s2mm_prmry_reset_out_n
mm2s_introut
s2mm_introut
AXI Direct Memory Access
zynq_ultra_ps_e_0
ZYNQ
UltraSCALE+
M_AXI_HPM0_LPD
pl_resetn0
pl_clk0
Zynq UltraScale+ MPSoC
Tcl Console
Messages
Log
Reports
Design Runs

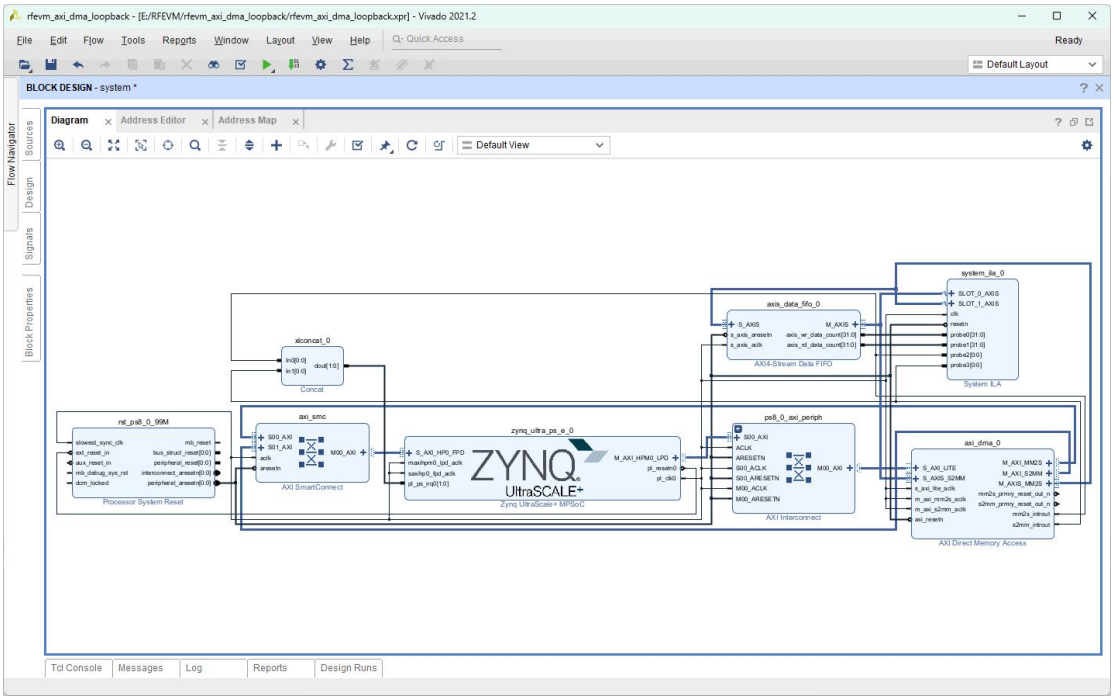
6) 参数设置如下所示





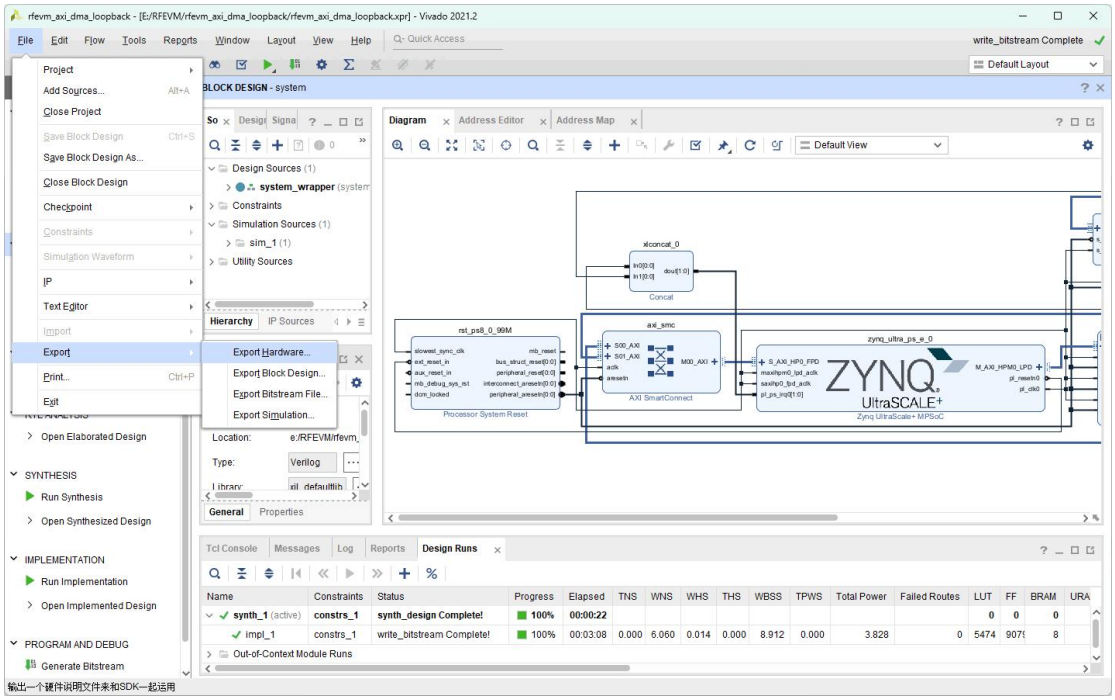
7) 添加 ila 模块，并点击自动连接



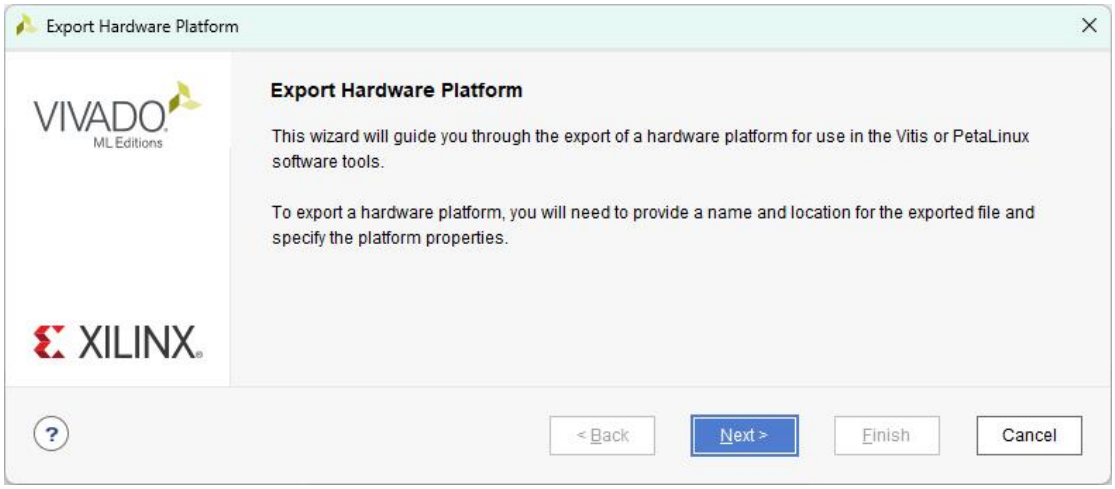


8) 创建 HDL Wrapper，并生成 bitstream；

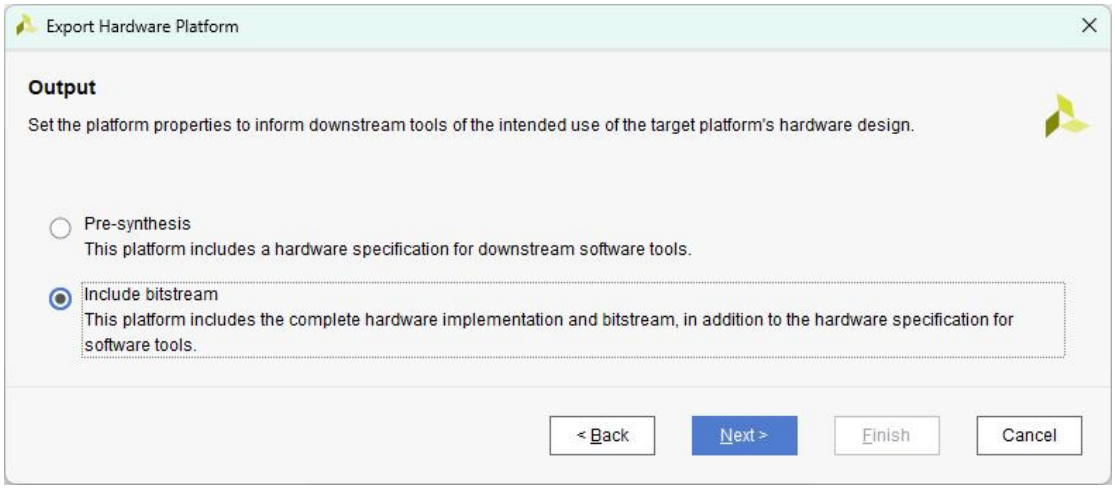
9) 导出硬件 xsa 文件



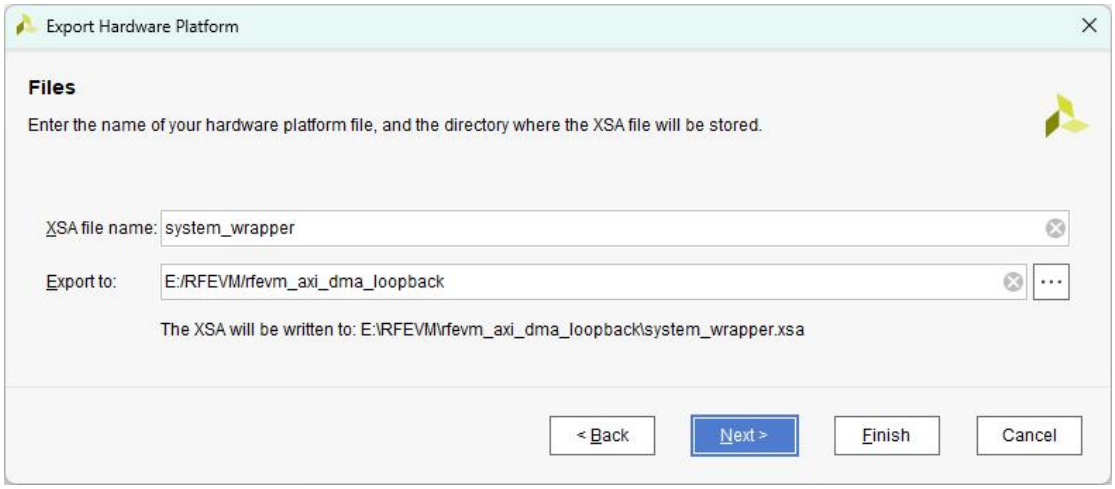
点击【Next】



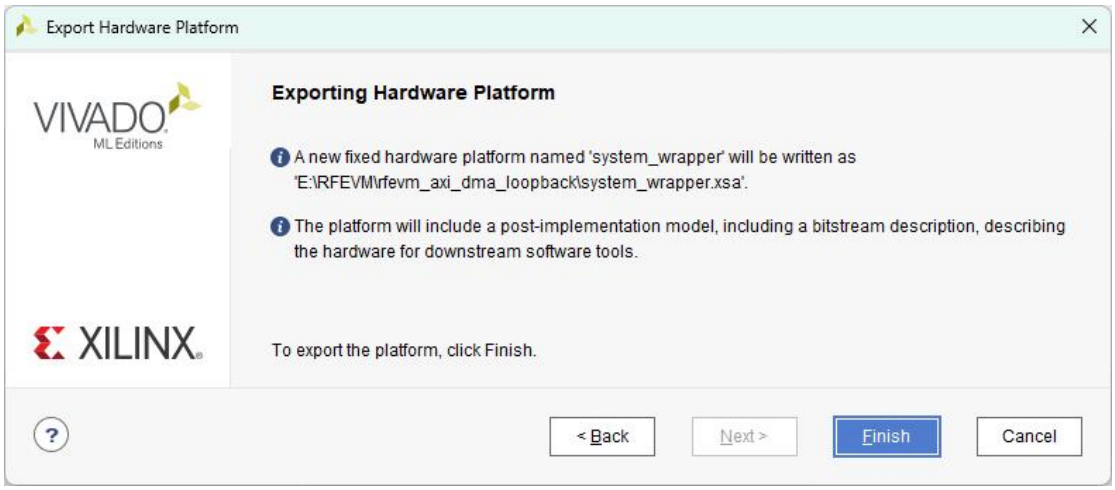
选择包含 bit，点击【Next】



设置名称和导出路径，点击【Next】



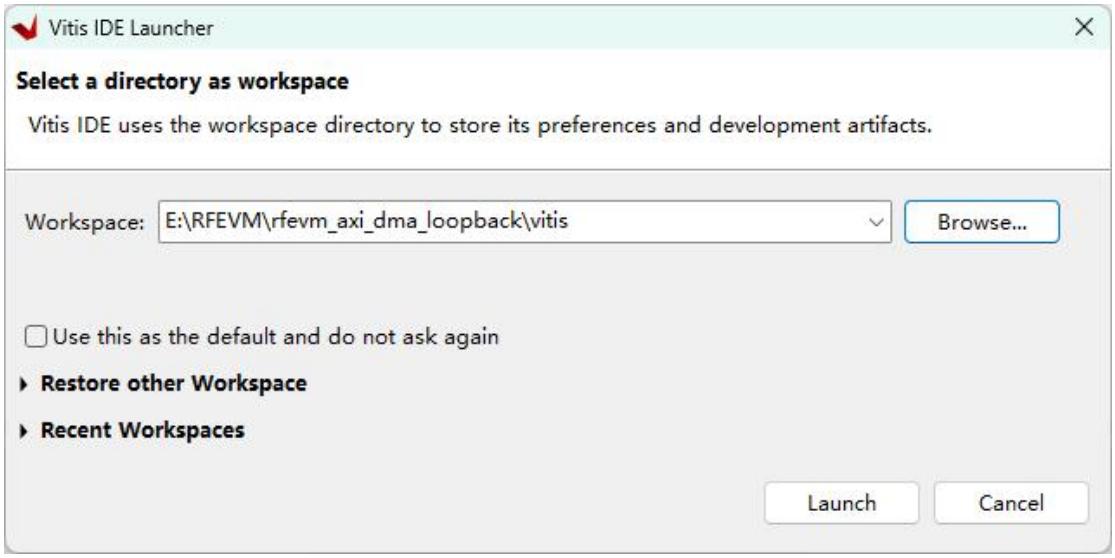
点击【Finish】



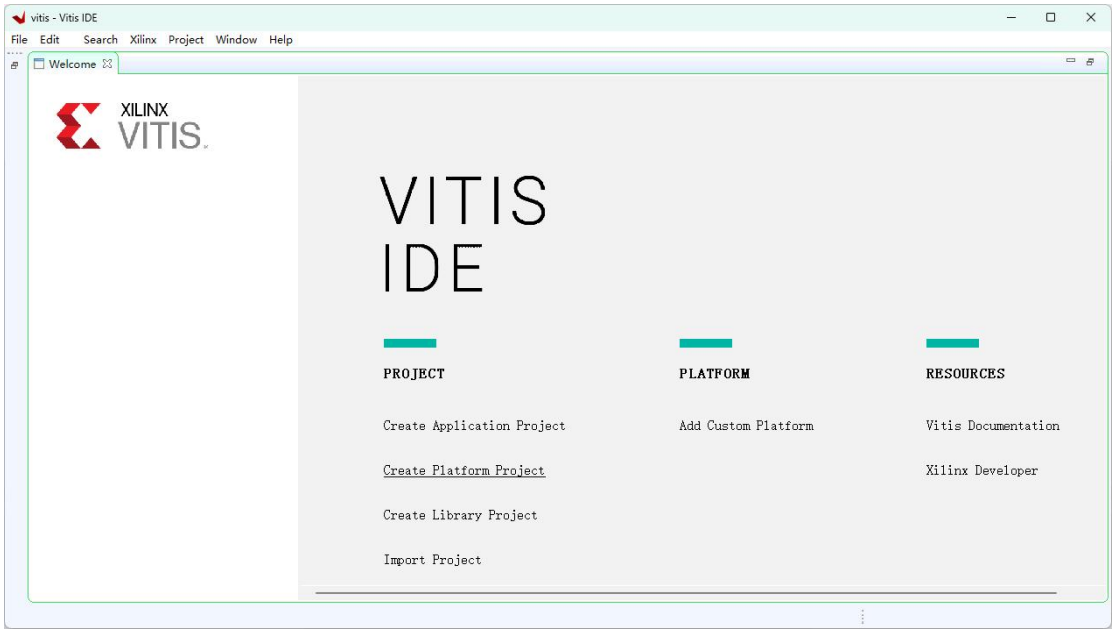
5.4.2 软件工程搭建

我们在 vivado 下搭建好硬件工程，导出 xsa 文件后，在工程目录下面创建 vitis 文件夹作为 vitis 的工作目录。

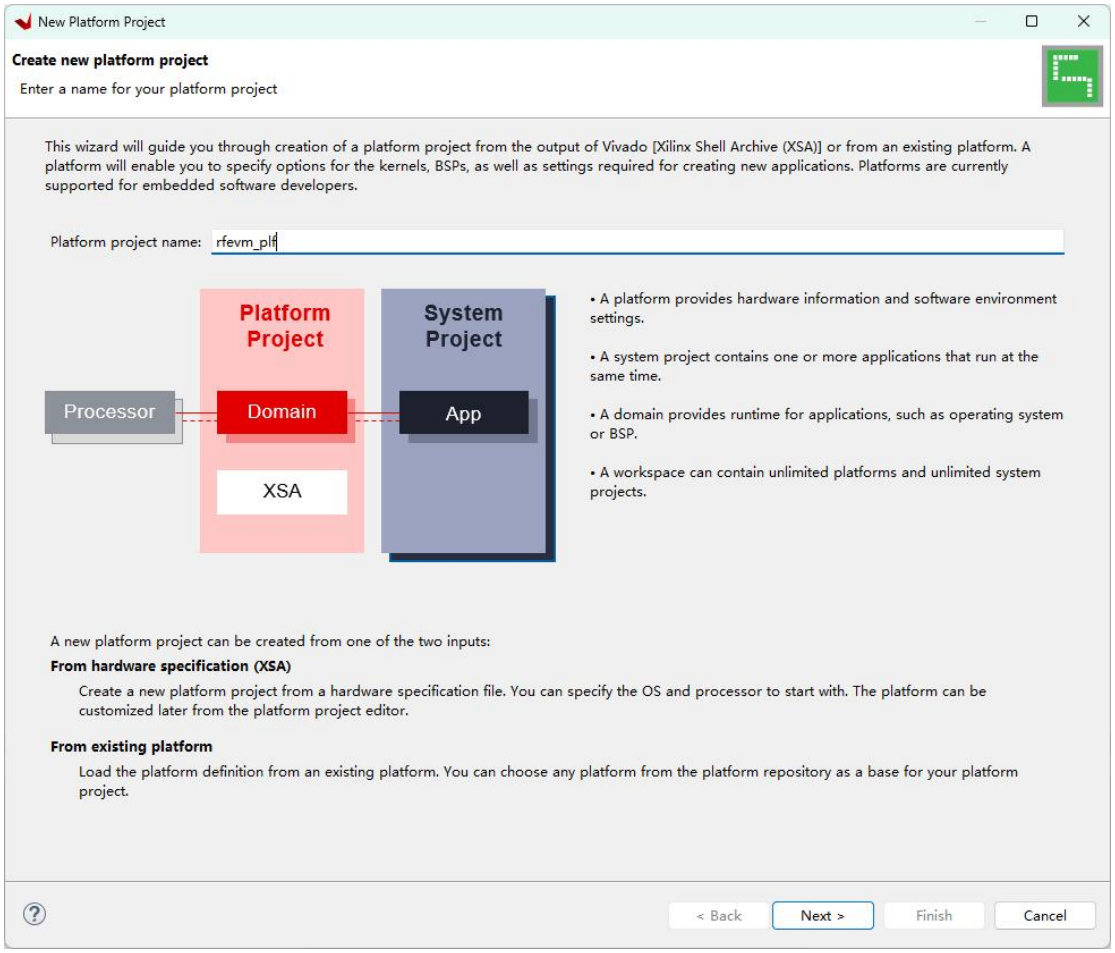
1) 登录 vitis，选择本工程路径下面的 vitis 文件夹作为工作目录，点击【Launch】



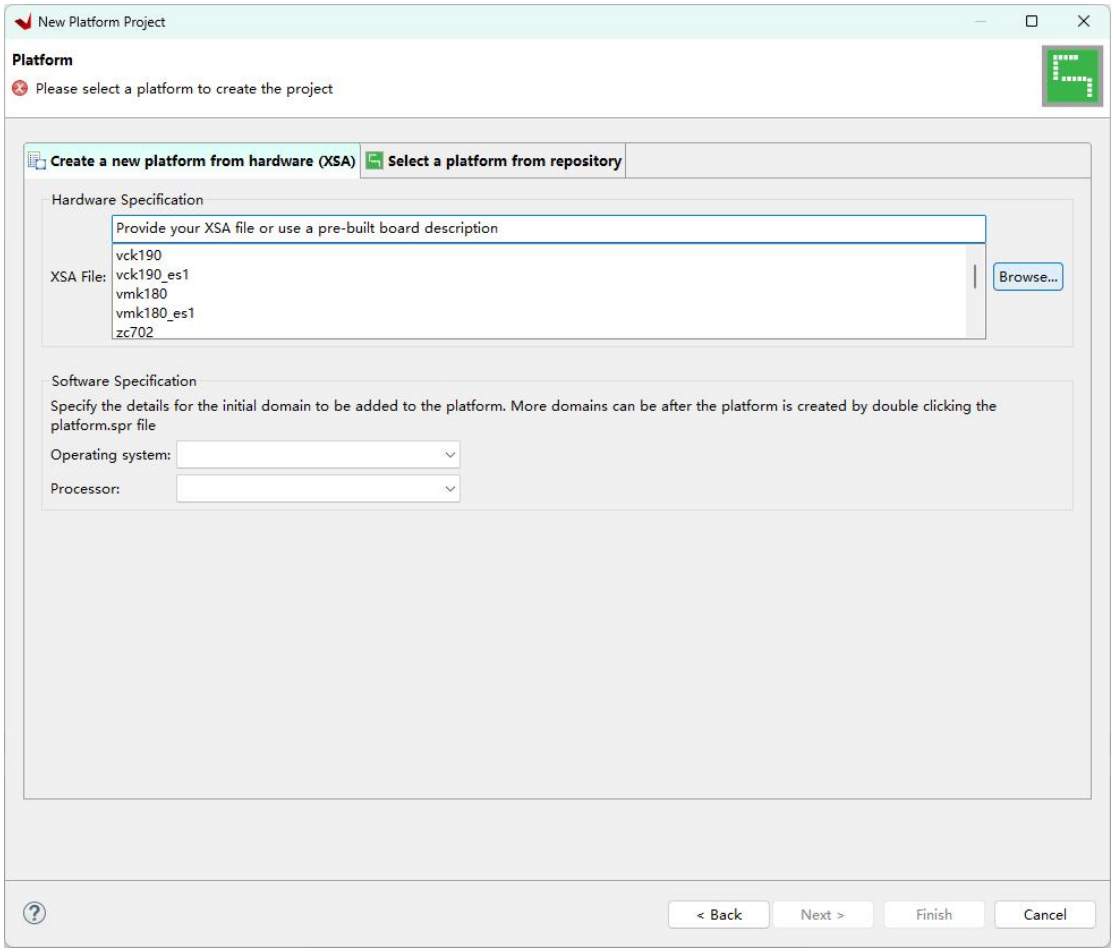
2) 选择创建硬件平台



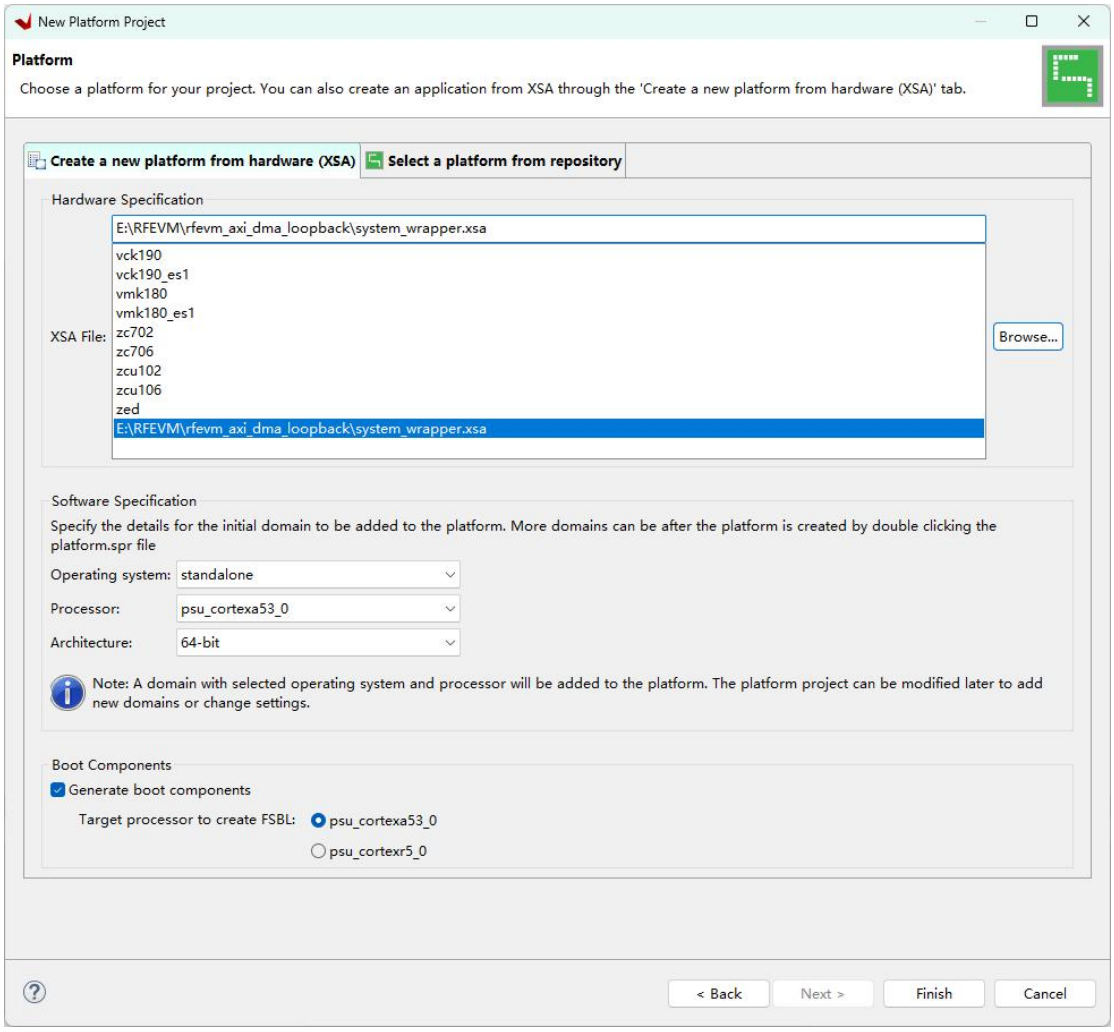
设置名称，点击【Next】



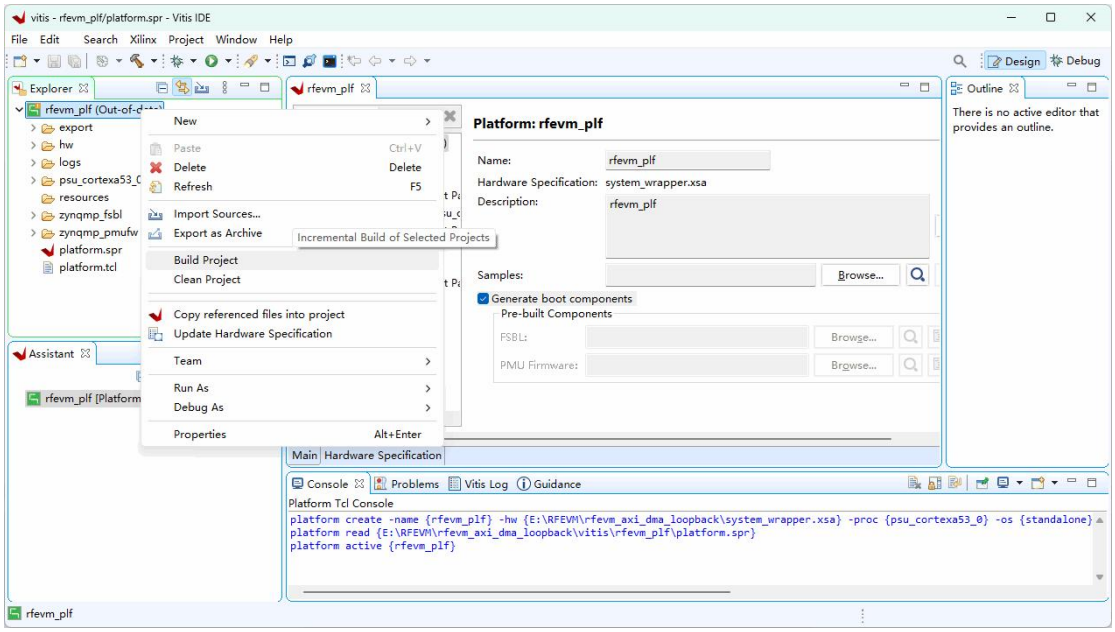
点击【Browse】，选择生成的 xsa 文件；


New Platform Project
Platform
Please select a platform to create the project
Create a new platform from hardware (XSA)
Select a platform from repository
Hardware Specification
Provide your XSA file or use a pre-built board description
XSA File:
vck190
vck190_es1
vmk180
vmk180_es1
zc702
Browse...
Software Specification
Specify the details for the initial domain to be added to the platform. More domains can be after the platform is created by double clicking the platform.spr file
Operating system:
Processor:
< Back
Next >
Finish
Cancel

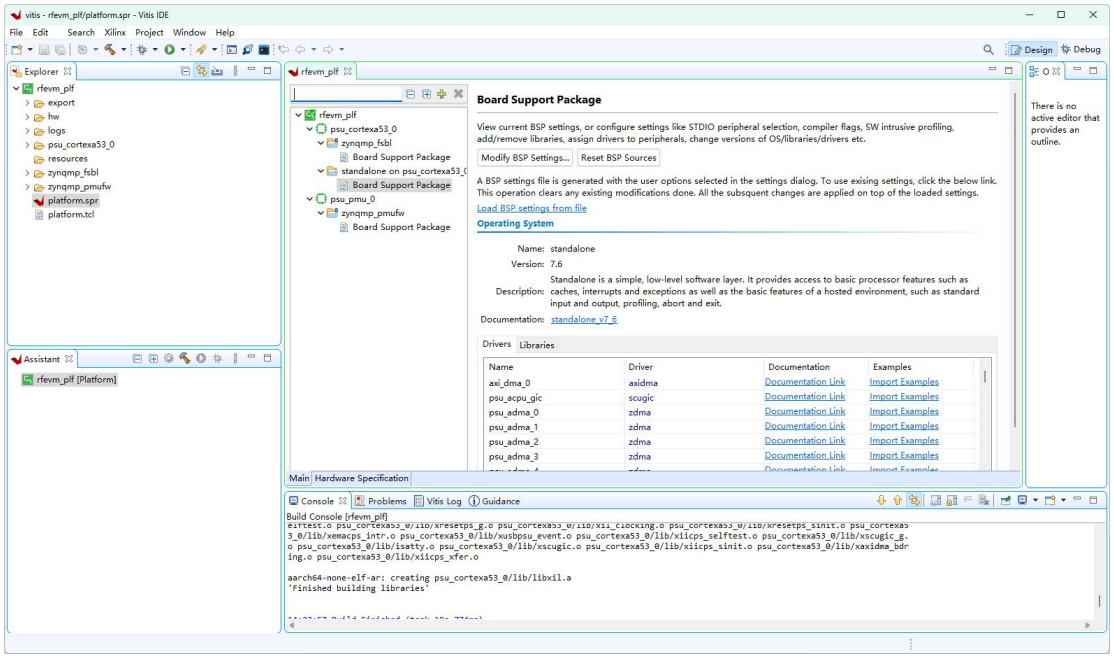
点击【Finish】



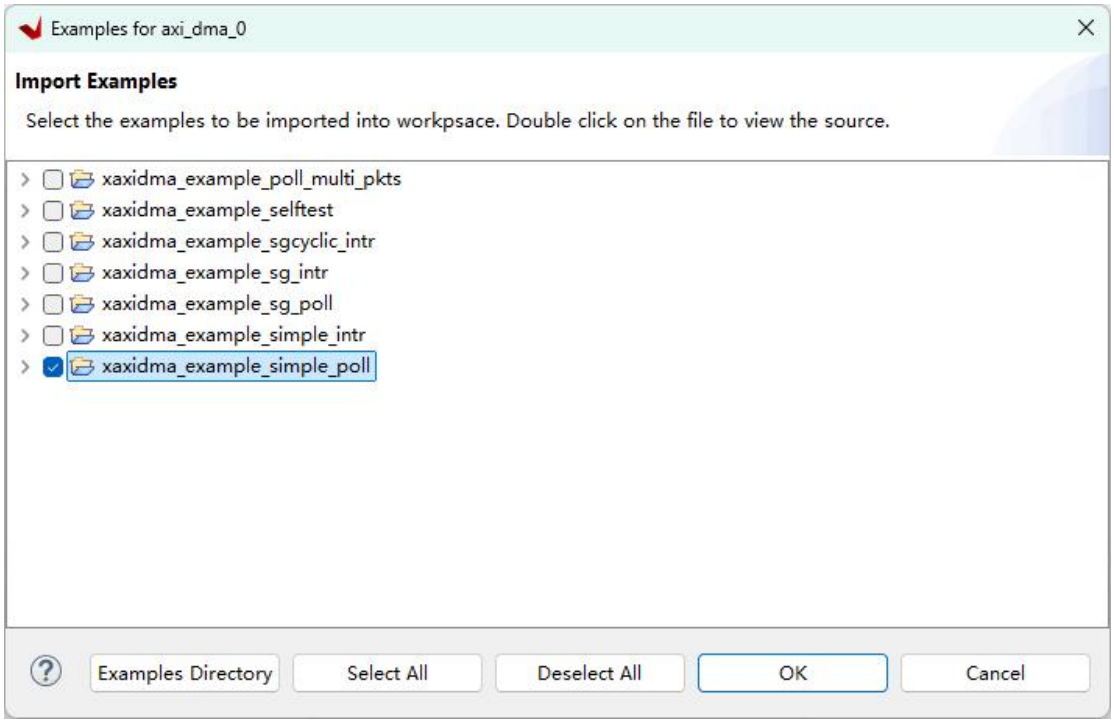
3) 编译硬件平台



4) 点击【Import Example】导入 axi dma 的参考示例



5) 选择 simple_poll 示例



6) 对示例源文件进行修改：设置 MAX_PKT_LEN,就是长度，单位为字节，TEST_START_VALUE 为起始的数据值，NUMBER_OF_TRANSFERS 为测试次数；

rfevm_plf | xaxidma_example_simple_poll_1 | xaxidma_example_simple_poll.c

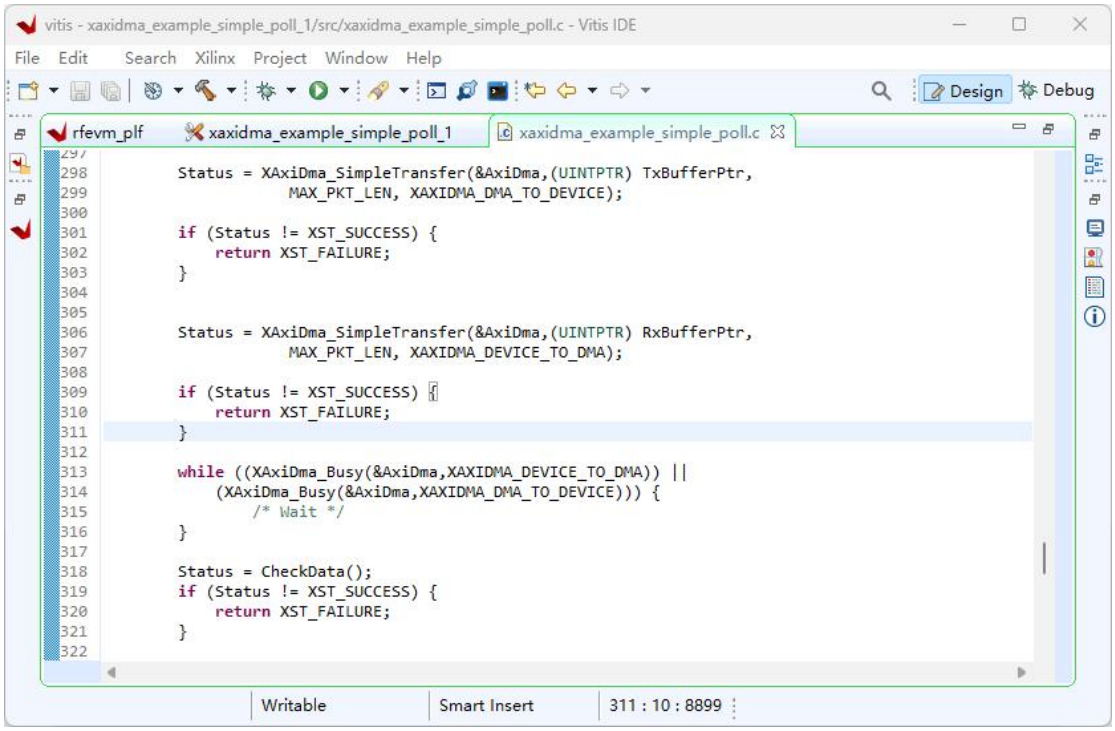
```
84
85  #ifndef DDR_BASE_ADDR
86  #warning CHECK FOR THE VALID DDR ADDRESS IN XPARAMETERS.H, \
87           DEFAULT SET TO 0x01000000
88  #define MEM_BASE_ADDR       0x01000000
89  #else
90  #define MEM_BASE_ADDR       (DDR_BASE_ADDR + 0x1000000)
91  #endif
92
93  #define TX_BUFFER_BASE      (MEM_BASE_ADDR + 0x00100000)
94  #define RX_BUFFER_BASE      (MEM_BASE_ADDR + 0x00300000)
95  #define RX_BUFFER_HIGH      (MEM_BASE_ADDR + 0x004FFFFF)
96
97  #define FIFO_DATABYTE   0x4
98  #define TEST_COUNT      80
99  #define MAX_PKT_LEN     TEST_COUNT*FIFO_DATABYTE//0x20
100
101 #define TEST_START_VALUE    0xC
102
103 #define NUMBER_OF_TRANSFERS 2
104
105
106 /*************************** Type Definitions *****************************/
```

打开 S2MM 的 IOC 中断，关闭 MM2S 的所有中断。在 S2MM 接收完数据后会发出中断。

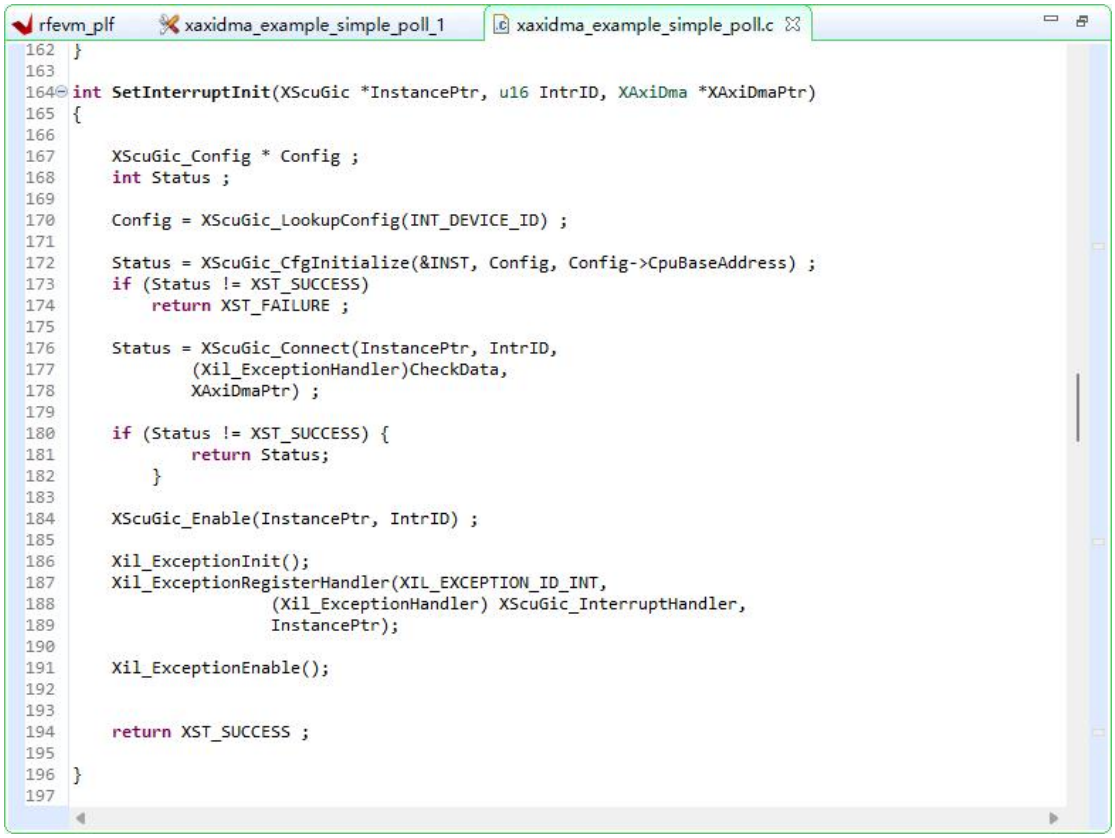
rfevm_plf | xaxidma_example_simple_poll_1 | xaxidma_example_simple_poll.c

```
232
233     if(XAxiDma_HasSg(&AxiDma)){
234         xil_printf("Device configured as SG mode \r\n");
235         return XST_FAILURE;
236     }
237
238     /* Disable interrupts, we use polling mode
239      */
240     XAxiDma_IntrEnable(&AxiDma, XAXIDMA_IRQ_ALL_MASK,
241                        XAXIDMA_DEVICE_TO_DMA);
242     XAxiDma_IntrDisable(&AxiDma, XAXIDMA_IRQ_ALL_MASK,
243                         XAXIDMA_DMA_TO_DEVICE);
244
245     Value = TEST_START_VALUE;
246
247     for(Index = 0; Index < MAX_PKT_LEN; Index ++) {
248             TxBufferPtr[Index] = Value;
249
250             Value = (Value + 1) & 0xFF;
251     }
252     /* Flush the buffers before the DMA transfer, in case the Data Cache
253      * is enabled
254      */
```

打开 MM2S 通路和 S2MM 通路

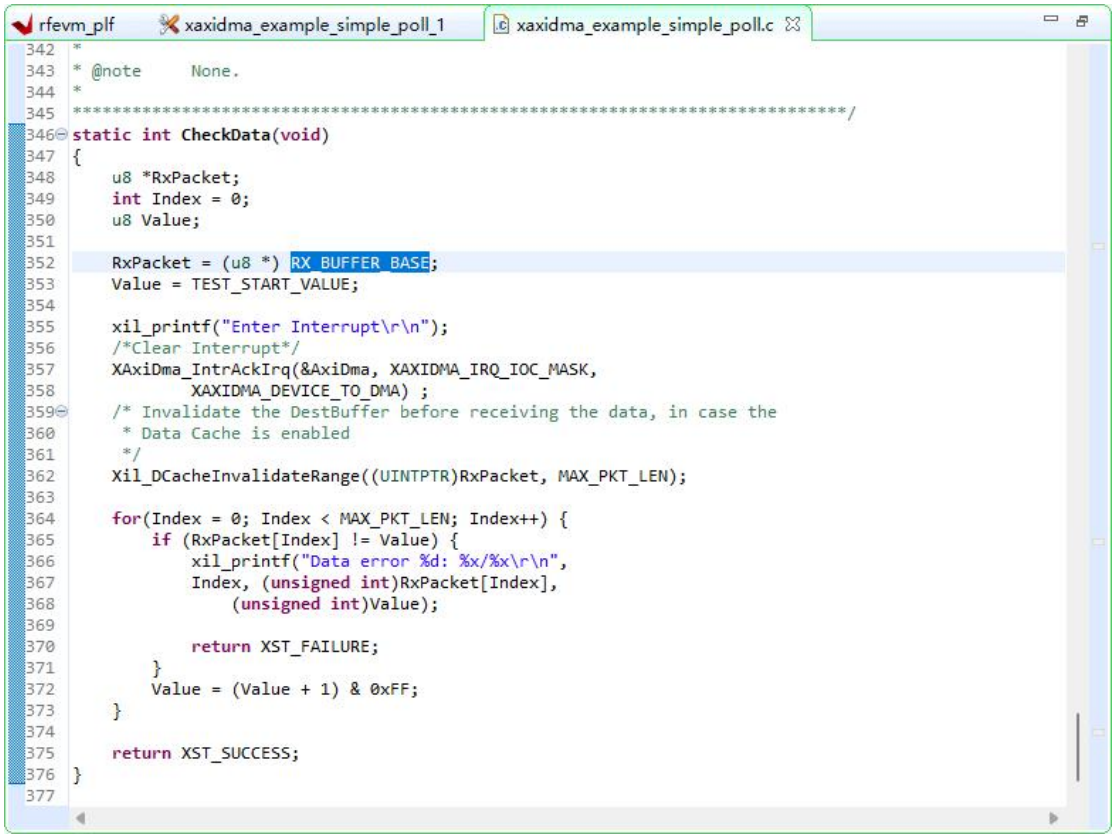


设置中断



在中断服务程序中，首先会清除中断，由于 DDR 中的数据已经更新，但 Cache 中的数据并没有更新，CPU 如果想从 DDR 中读取数据，需要调用 Xil_DCacheInvalidateRang 函数，将 Chche 数据作废，这样 CPU 就能从 DDR 中

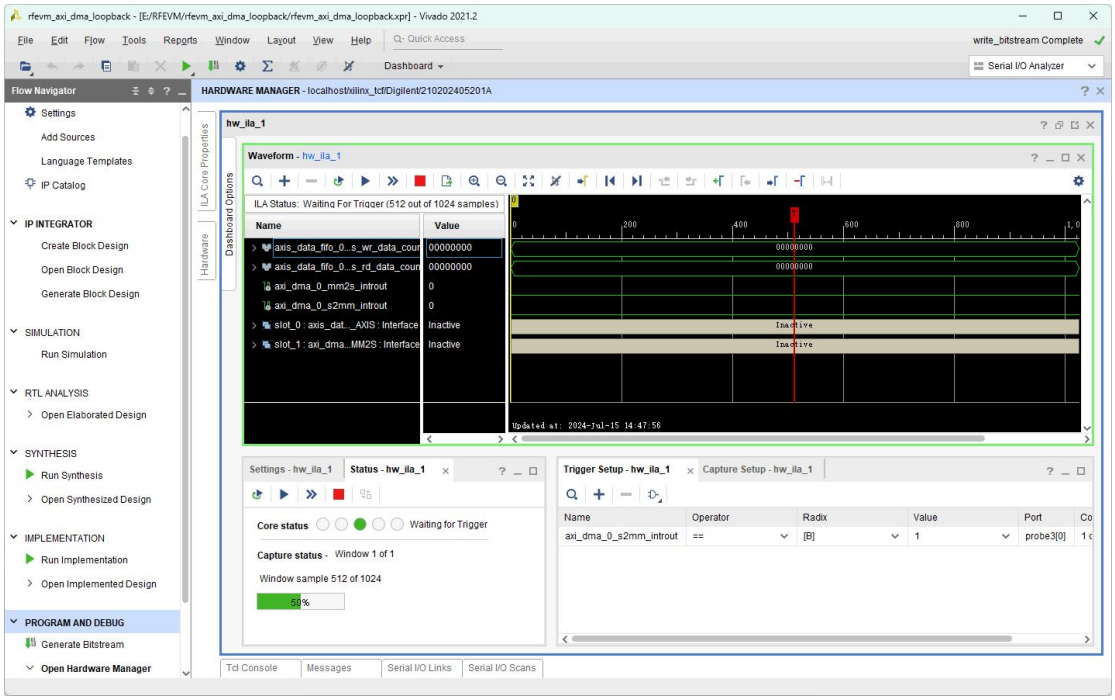
读取正确的数据。同样也要给出内存地址和长度。之后 CPU 从 DDR3 中读取数据进行对比，检验数据的正确性。



5.4.3 下载验证

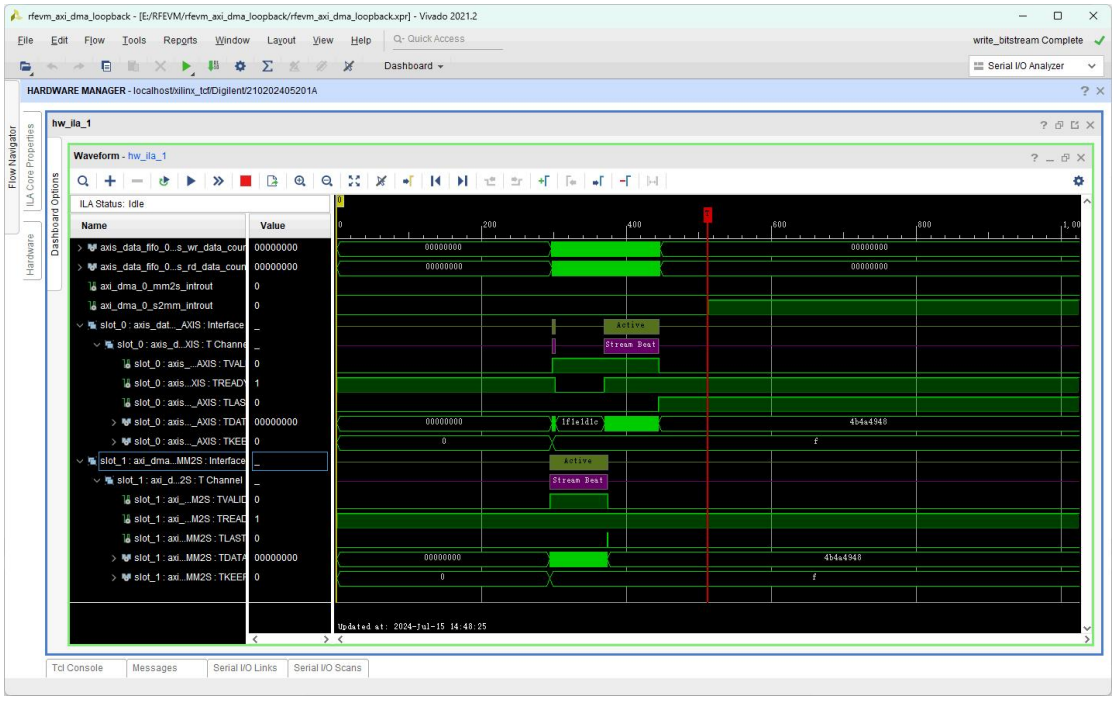
1) 我们在 vitis 里面 debug，应用程序。

2) 等待 bit 下载完成之后，在 vivado 中打开 ila，设置触发条件



3) 在 vitis 的 Debug 界面，不用设置断点，点击 Resume

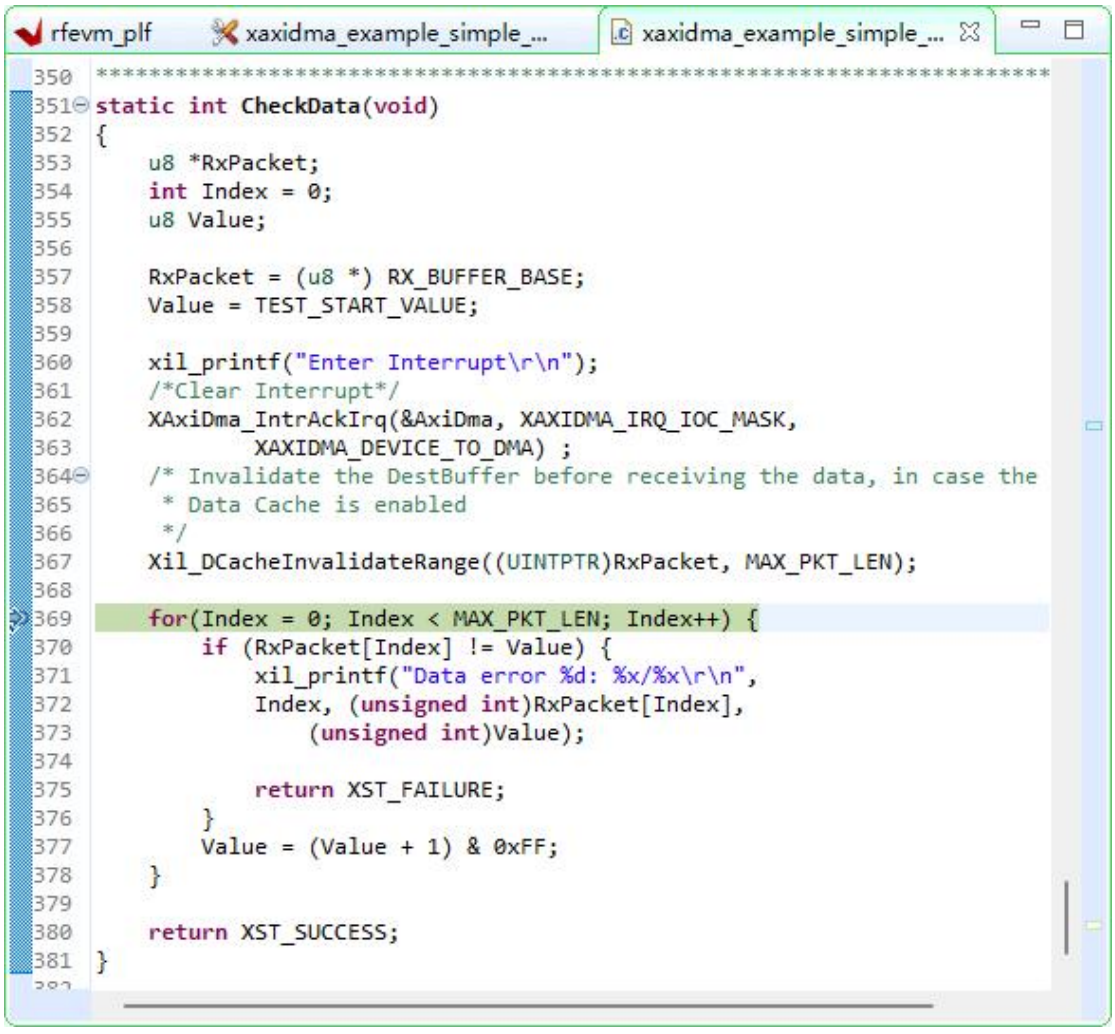
4) 此时可以看到 ila 已经触发，可以观察采集到的数据。

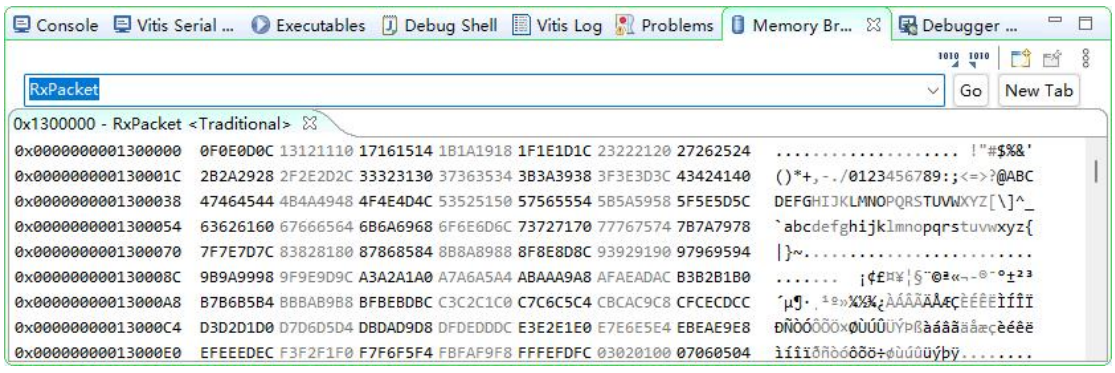


在串口调试工具中，可以看到打印信息

```
--- Entering main() ---
Enter Interrupt
Enter Interrupt
Enter Interrupt
Enter Interrupt
Successfully ran XAxiDma_SimplePoll Example
--- Exiting main() ---
```

5) 我们也可以在 vitis 调试中，设置断点，观察 memory 信息；





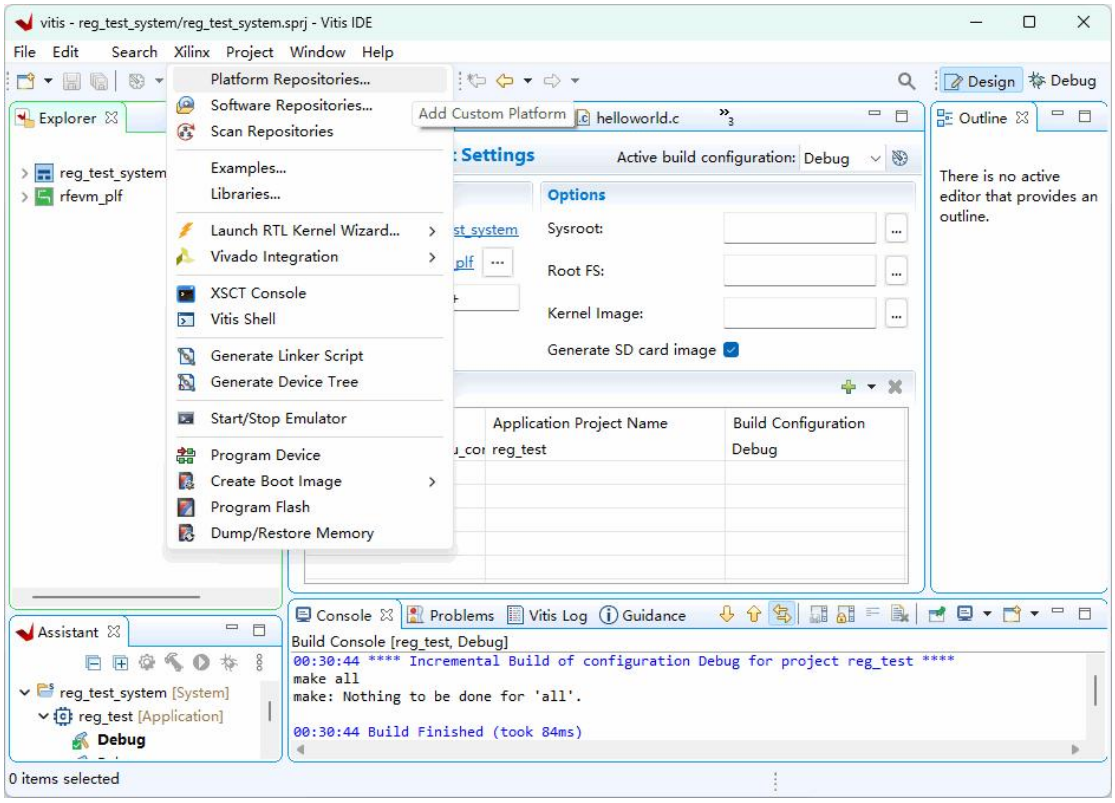
6 FAQs

常见问题解答

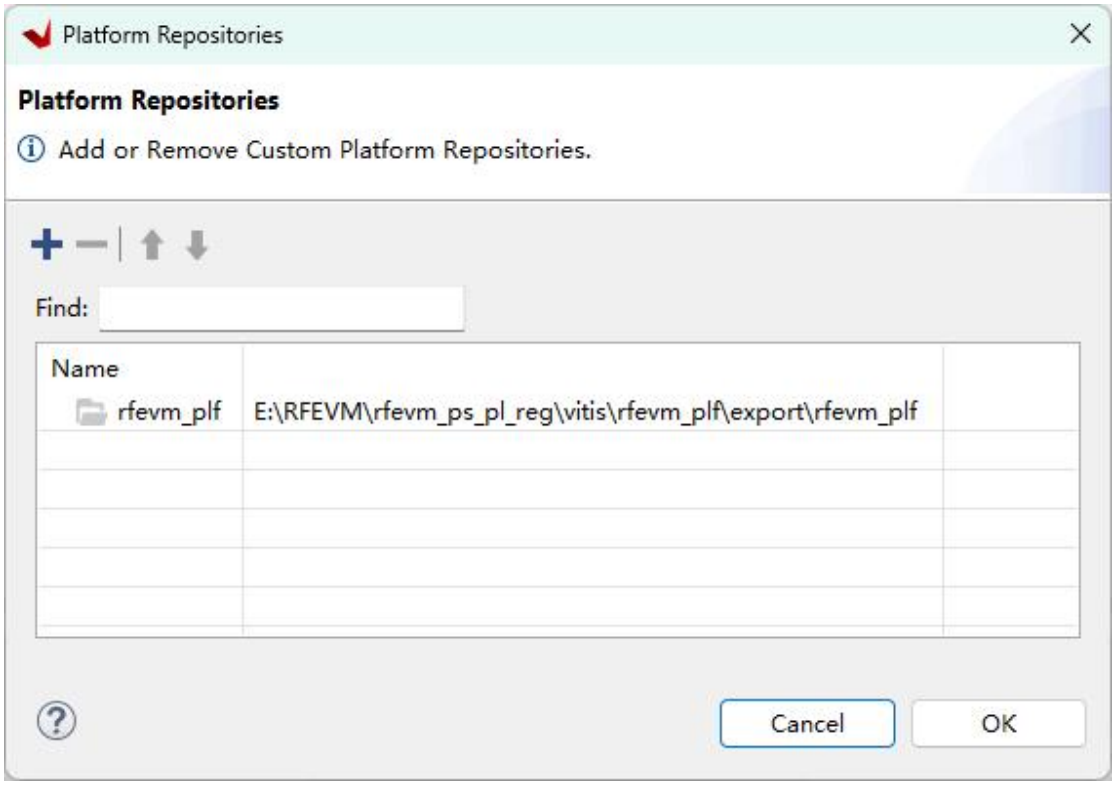
1. 提供的例程，登录 vitis，编译/下载失败？

   例程路径更换之后，Vitis 软件里面 platform 路径需要根据当前实际路径重新指定：

   Xilinx->Platform Repositories...



   这里添加当前的路径就可以



或者删掉 platform，重新建一个 platform，名称和之前设置一样也可以。